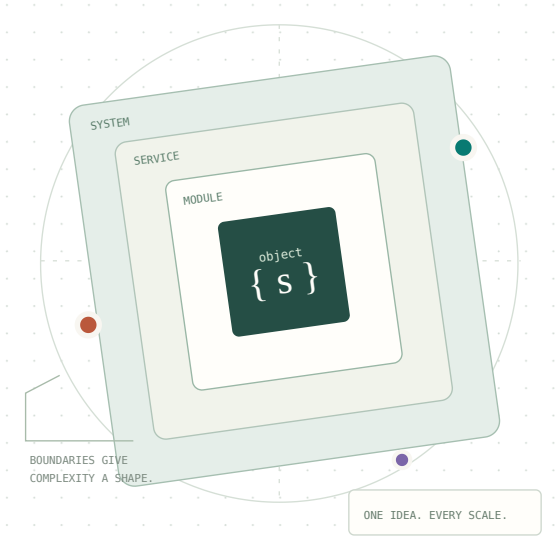
THE SYSTEM DESIGN ATLAS

From objects to *planet-scale systems.*

A system is more than its boxes and arrows. Learn the promises, mechanisms, and trade-offs that connect a single function to a distributed world.

Build the simple version. Find its limit. Understand the next design.



A CONCEPTUAL COVER STUDY · AUTHOR-CREATED

22	99	575	12
UNITS MAPPED	CHAPTERS TO READ	ORIGINAL FIGURES	COMPLETE CASE STUDIES

- **The complete first edition.** All 22 units, 99 chapters, and twelve complete case studies are ready, with worked mechanisms, original figures, exercises, and separate explained solutions. Inspect the coverage → · Download the complete PDF ↓

THE TERRITORY

Six parts. One connected story.

See the concept map ↗

PART 01

System design foundations

16 chapters · 4 units · worked mechanisms, failures and alternatives

Ready to read ↗

PART 02

Low-level design & software architecture

19 chapters · 4 units · worked mechanisms, failures and alternatives

Ready to read ↗

PART 03

Data, storage & scalability

20 chapters · 5 units · worked mechanisms, failures and alternatives

Ready to read ↗

PART 04

Distributed systems & reliability

16 chapters · 4 units · worked mechanisms, failures and alternatives

Ready to read ↗

PART 05

Security & production engineering

16 chapters · 4 units · worked mechanisms, failures and alternatives

Ready to read ↗

PART 06

Complete system design case studies

12 chapters · 1 units · twelve complete design reviews

Ready to read ↗

A PLACE TO BEGIN / UNIT 01

Before you scale a system, learn to *see one*.

Build a to-do application step by step. Save a task, organise its code, choose a design, and follow a request through the system.

Open the foundations unit ↗

1.1 From a program to a system ↗

1.2 Organising code into modules ↗

1.3 From requirements to design choices ↗

1.4 Drawing the system at different levels ↗

01 / THE PROBLEM

Begin where it breaks.

A concept earns its place by solving an observable problem.

02 / THE MECHANISM

Make execution visible.

Inspect the state, ownership, messages, and ordering.

03 / THE JUDGMENT

Keep the trade-off.

Understand what improves, what it costs, and when to choose differently.

PART 1 / UNIT 01

Introduction to System Design

How do we turn a small program into an application people can rely on? We will build a to-do application, one decision at a time.

Begin with a task kept in memory. Save it between runs, organise the code into modules, and work out what the application must do. Then draw the design and trace a request through it.

The running example

Design a basic to-do application. Begin with a single program and progressively identify modules, persistent storage, a web API, and deployment boundaries.

Unit 1 assessment → Explained solutions →

Concept coverage

From a program to a system

Prerequisites: Basic programming 4 original figures

Reading the figures: boxes name parts or states; arrow labels explain how they relate. Dashed arrows show a code dependency or a change in the level of detail. Each caption explains the example.

Imagine that you have written a small to-do program. You enter “Read the design notes,” and the program adds it to your list. You close the program. When you open it the next morning, the list is empty.

The program could add a task, but it could not keep one. To make it useful, we must decide where tasks are stored and what should happen when the program stops. These are our first system design questions.

In this chapter: We will identify the parts of this application, follow a change in its data, and decide when it is safe to display “Saved.”

1.1.1 · What makes a program part of a system?

A **software system** consists of parts that work together to perform a task. In our example, the program accepts a title, checks it, stores a task, and displays the list. Once we add permanent storage, the program and the stored data must work together.

It helps to state what we are designing. Our first application is for one person using one machine. The person can add tasks, list them, and mark them complete. A task reported as saved must still be present after the program restarts. For now, we assume that the machine’s disk remains intact.

We can draw a line around the application and its storage. This is the **system boundary**: it tells us which parts we are considering together. The person uses the system from outside this line.

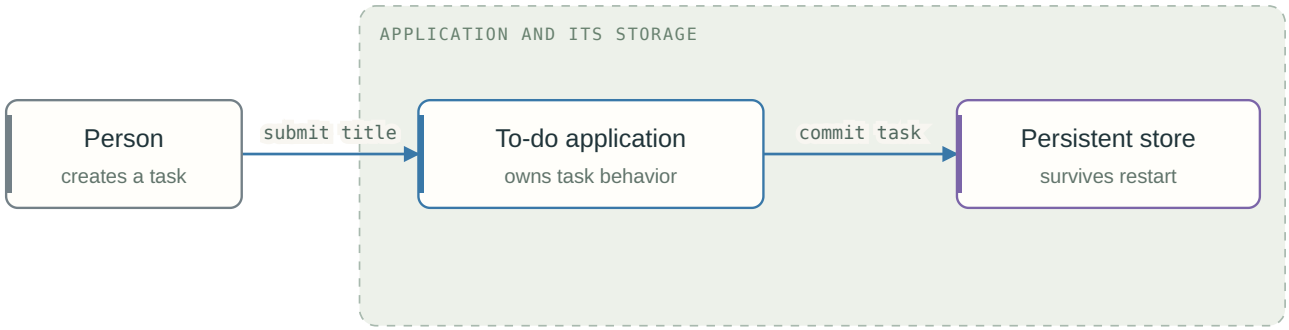


FIG. 1.1.1 **The application and storage work together**

The application needs storage to keep a task after restart. We include both inside the boundary when discussing that requirement.

Why this view: Include the parts needed to explain the behaviour you are studying. Author-created illustration.

The boundary depends on the question. To investigate a slow function, we may study only the running program. To investigate a missing task, we also need to examine storage. When drawing a boundary, state which question it helps answer.

System design is the work of deciding how these parts should be organised and how they should interact to meet the requirements. For our application, this includes choosing where to store tasks, which code checks their titles, and when to tell the user that a task has been saved.

> START WITH THE USER'S EXPECTATION
>
> If the screen says "Saved," what should the user be able to rely on?

1.1.2 · Follow one task through the program

Let us first keep tasks in memory. Memory holds the data used by a running program. In this example, the program creates a new, empty dictionary each time it starts.

The following Python code assumes that calls run one at a time, that the title is text, and that the caller supplies a positive integer identifier. It checks the title length and prevents two tasks from using the same identifier.

```
tasks = {}

def create(task_id, title):
    title = title.strip()
    if not 1 <= len(title) <= 200:
        raise ValueError("title must contain 1–200 characters")
    if task_id in tasks:
        raise ValueError("task id already exists")

    tasks[task_id] = {"title": title, "done": False}
    return {"id": task_id, **tasks[task_id]}

print(create(7, "  Read  "))
# {'id': 7, 'title': 'Read', 'done': False}
```

The call `create(7, " Read ")` first removes spaces from the ends of the title. It checks that the resulting title is between 1 and 200 characters long and that identifier 7 is unused. It then adds one task and returns its details.

Before the call, `tasks` is empty. After the call, it contains task 7. This is a **state change**. The state of a system is the information it currently holds that affects what it will do next. If we now ask for the task list, task 7 will appear because the state has changed.

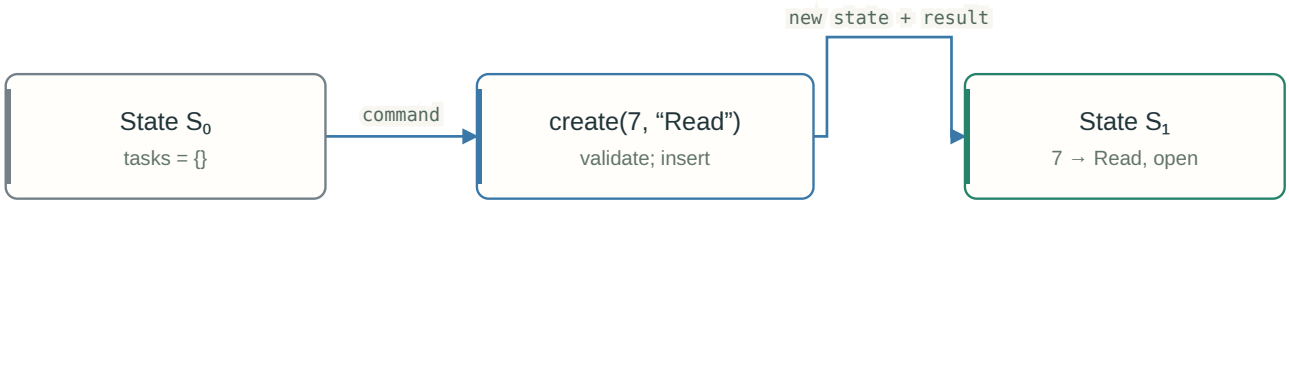


FIG. 1.1.2 **Creating a task changes the stored state**

Before creation, the collection is empty. After create(7, “Read”), it contains task 7, marked incomplete. The function returns the task details.

Why this view: Read the state before and after the operation to see exactly what changed. Author-created illustration.

We can use this one operation to distinguish four related ideas. The title “Read,” the number 7, and the value `False` are **data**. The collection of tasks currently stored is **state**. Trimming and checking the title are **computation**. Passing the title into the function and returning the result are **communication**. Later, communication may also involve files or a network.

Some rules must hold whenever the task collection is valid: each task must have a unique identifier, and each title must contain 1–200 characters after trimming. A rule of this kind is called an **invariant**. If validation fails, the function raises an error before changing the collection.

To complete task 7, we would check that it exists and set `tasks[7]["done"] = True`. Setting that flag to `True` again leaves the task completed. Creating another task is different: it adds another entry. We will return to this distinction when we discuss repeated requests.

<details>
<summary>▼ A closer look · Describing a state change with notation</summary>

We can write an operation as `T(S, c) → (S′, result)`. Here, `S` is the state before the operation, `c` is the command, and `S′` is the state afterward.

For our example, `S` is an empty task collection, `c` is “create task 7 with title Read,” and `S′` contains that task. If the title is empty, the result is an error and `S′ = S`. This notation describes the intended change. By itself, it says nothing about crashes or two callers running at the same time.
</details>

1.1.3 · Decide what “saved” means

The dictionary works while the program is running. When the program restarts, it creates a new dictionary and loses the old list. To meet our restart requirement, we need **persistent storage**: storage that keeps data after the program stops.

For this application, we will use SQLite, a database that can store its data in a local file. The application groups a write into a **transaction**. Completing the transaction successfully is called a **commit**. We will report success only after the database has committed the task.

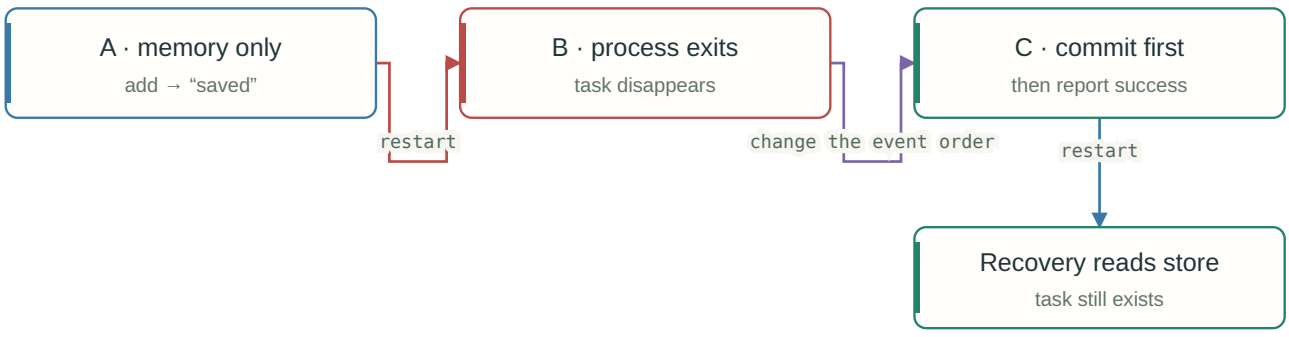


FIG. 1.1.3 **Save before reporting success**

Memory-only tasks disappear after restart. In the revised version, the application commits the task before reporting success. The task remains after a process restart if committed storage is intact.

Why this view: Compare the two versions by restarting the process. Author-created illustration.

The order matters. Suppose the application displays “Saved” and then tries to write the task. If it crashes between these events, the user has been told that a task was saved even though it is missing after restart. Waiting for commit avoids this particular failure.

This is a promise about restarting the program while its committed storage remains intact. If the only disk is destroyed, a local database cannot recover the data from that disk. The SQLite atomic-commit documentation explains how its commit mechanism works and which storage assumptions it relies on.

TRY IT · INTERACTIVE MODEL

What survives a restart?

Choose when the application reports success. Use “Next event” to follow the request, then restart the process and check whether the task remains.

Choice: C · commit before reporting success · 0 events completed

The caller has not received success.

APPLICATION MEMORY	SAVED IN STORAGE	WHAT THE CALLER SEES
Empty	Empty	No success received

Choose “Next event” to continue, or restart to see what remains.

This model follows one request. A commit either finishes completely or does not happen. Restart removes memory but preserves committed storage. No real files are written.

Try the policy that reports success before commit. Advance until the user receives “Saved,” then restart the process. Now repeat with commit before success. The different results come from the order of two events.

The model removes memory when the process restarts and keeps committed storage. It does not simulate a damaged disk or a power failure. The runnable Python example lets you create a task, close the real database connection, and read the task through a new connection.

1.1.4 · Give each part a clear contract

A **component** is a part of the system with a defined responsibility. We will soon separate the code that accepts input, the code that applies task rules, and the code that stores tasks. These components can all run inside the same program.

Components need a way to use one another. An **interface** describes the operations they offer, such as `create(title)` or `list_tasks()`. A **contract** explains how those operations behave.

For example, the name `create(title)` does not tell us whether blank titles are accepted or whether the result has already been saved. We must state those rules. The following contract describes the persistent version of our application, which will assign identifiers itself.

Table 1.1.1 · The contract for creating a task

Situation	Required behaviour	How to check it

Situation	Required behaviour	How to check it
The title has leading or trailing spaces.	Remove those spaces before checking and saving it.	Submit “ Read ” and check that the title is “Read.”
The trimmed title is empty or longer than 200 characters.	Reject it without adding a task.	Compare the task list before and after the attempt.
Creation succeeds.	Return the new identifier after commit.	Restart the program and find the task.
The database write fails.	Report the error instead of reporting success.	Make storage fail and inspect the result.
The task is committed, but its reply is lost.	The task remains saved; the caller may be unsure what happened.	Interrupt the reply after commit.

The last row introduces a problem we will examine in Chapter 1.4. Saving a task and delivering its reply are separate events. The application must account for both.

1.1.5 · Look at the same design at two levels

We can now ask two sets of questions. Where does the application store tasks, and how does it communicate with that storage? Within the application, which functions validate a title and perform the write?

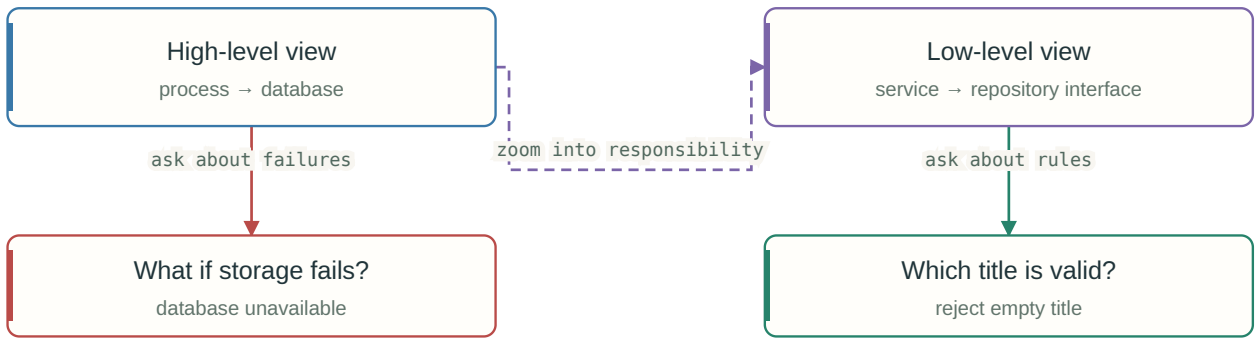


FIG. 1.1.4 **Two levels of detail in one design**

The high-level view shows the application and database. The low-level view shows the service, repository interface, and rules inside the application.

Why this view: Check that both views agree about saving a task before reporting success. Author-created illustration.

High-Level Design (HLD) usually describes the main parts of a system and how they work together. For our application, it shows the user, application, and persistent store. It also states that the application waits for commit before reporting success.

Low-Level Design (LLD) describes how a part is implemented: its functions, classes, interfaces, data structures, and algorithms. Here, it explains how the task service checks a title and calls the storage code.

Software architecture concerns the major choices that shape the system and the reasons for them. Choosing to keep tasks in a local database is one such choice. Different teams use HLD, LLD, and architecture somewhat differently; the useful distinction is the level of detail being discussed.

Both levels must agree. The high-level design may say “save before replying,” but the code must actually wait for the save to finish.

> CHECK YOUR UNDERSTANDING
>
> Putting a dictionary in a class does not make it persistent. Adding a database does not make a reply arrive. A successful restart test does not show what happens when the disk is destroyed. Each result depends on a different part of the design.

Remember: Start with what the application must do. Identify the parts and the data they hold. Then trace the events that must happen before the user receives success.

Before continuing: The application reports “Saved,” then crashes before writing to storage. Which event must move earlier to prevent the task from being lost?

Unit 1 exercises → · Unit 1 explained solutions →

Sources & further reading

These sources explain the ideas and mechanisms used in this chapter. The to-do application is our own teaching example. Links were reviewed on 21 October 2026.

1. Atomic Commit in SQLite ↗

   Official documentation · accessed 21 Oct 2026. Commit and operating-system assumptions.

   `Access reviewed · 21 October 2026`

2. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

Organising code into modules

Prerequisites: 1.1 · From a program to a system 4 original figures

Reading the figures: boxes name parts or states; arrow labels explain how they relate. Dashed arrows show a code dependency or a change in the level of detail. Each caption explains the example.

Our to-do application can now save tasks between runs. We add a web form and a command-line interface. Both contain code that checks the title. Later, we change the title limit from 80 characters to 200, but update only the web form.

The same application now accepts a title through one entry point and rejects it through another. To fix the cause, we need to decide where the rule belongs.

In this chapter: We will divide the application into modules, examine the dependencies between them, and follow a change through the revised design.

1.2.1 · Why does one change require several edits?

A first version of the web handler may do everything in one place: read the request, check the title, write SQL, and build the response. For a small program, this is easy to follow. The difficulty appears when another feature needs some of the same behaviour.

Suppose the command-line interface copies the title check. A CSV exporter also reads the database table directly. We now have two copies of a rule and several places that know how tasks are stored. Changing either the rule or the table can affect several features.



FIG. 1.2.1 **Shared storage details make changes spread**

Several parts read or change the same task data directly. A change to a field or rule may therefore require edits in several places.

Why this view: Follow the arrows to find which parts know about the shared data. Author-created illustration.

Coupling describes the dependencies between parts of a system. In this example, the exporter is coupled to the database layout: it must know the column names. Rename a column, and its query may stop working.

A dependency is often necessary. The task service needs some way to store a task. The design question is how much it needs to know. Calling `repository.insert(title)` requires less knowledge of storage than constructing SQL and opening database files throughout the application.

Cohesion describes how closely the work inside a module belongs together. Checking task titles and completing tasks serve a shared purpose. Password handling, CSV formatting, and database connection setup have different purposes; placing them in a single “helpers” module gives that module little focus.

Coupling asks how one module depends on another. Cohesion asks whether the work within one module belongs together. These ideas help us decide where to put code.

1.2.2 · Separate input, task rules, and storage

Let us give the application three main responsibilities.

- The **input adapter** translates an incoming request into a call the application understands. A web adapter reads an HTTP request; a command-line adapter reads command-line input.
- The **task service** applies task rules. It checks the title and asks storage to create a task.
- The **repository** provides operations for storing and retrieving tasks. Our SQLite implementation contains the database queries and commits each change before returning.

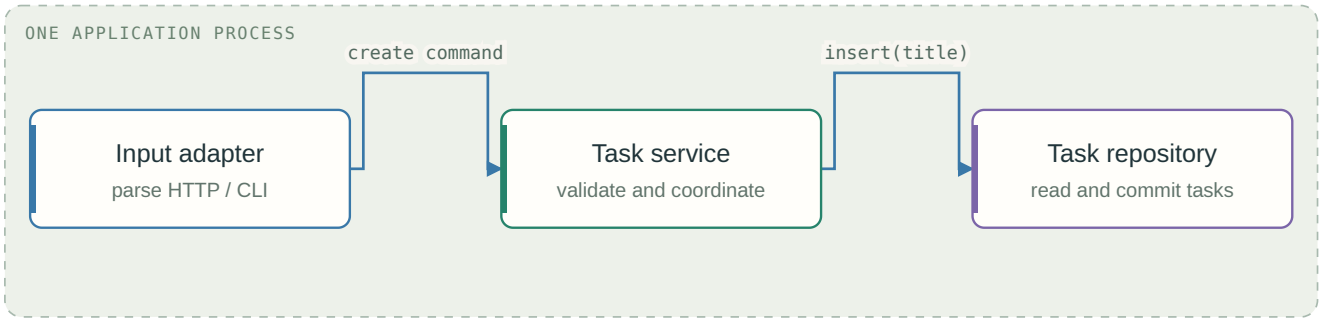


FIG. 1.2.2 **Separate responsibilities inside one process**

The adapter reads input, the service checks task rules, and the repository saves and reads tasks. These parts all run inside one application process.

Why this view: Each kind of work has a clear place in the code. Author-created illustration.

This division is an example of **separation of concerns**: giving different kinds of work their own place. The web adapter handles web details, the service handles task rules, and the repository handles storage details.

Here is the essential structure. This is pseudocode, so it shows the responsibilities without implementing a complete web server.

```
interface TaskRepository:
    insert(title) -> saved Task, or StorageError
    list() -> list of Task records
    set_done(id) -> saved Task, or NotFound, or StorageError

class TaskService:
    constructor(repository):
        self.repository = repository

    create(raw_title):
        title = trim(raw_title)
        require 1 <= length(title) <= 200
        return self.repository.insert(title)

handle_http(request, service):
    title = parse_json(request.body)["title"]
    task = service.create(title)
    return response(201, json(task))
```

Follow a request through this code. The web adapter reads the title from JSON. The service checks it. The repository inserts and commits the task. The saved task returns through the service to the adapter, which builds the response.

The command-line adapter can call the same `service.create(title)`. When the title limit changes, we update the shared rule. Each adapter still checks its own input format: for example, whether a web request contains valid JSON. Task rules and input-format rules have different jobs.

The real Python example implements the service and repository. It also checks that a title is text. A complete HTTP adapter would additionally handle missing fields, invalid JSON, and storage errors.

1.2.3 · Hide details that callers do not need

The service needs to request a saved task. It does not need to know the table name, SQL statement, or file path used to store it. The repository presents a simpler operation, `insert(title)`, while hiding those details. This is **abstraction**.

The contract still matters. In our design, `insert` must return only after commit. The service relies on that behaviour when it returns success. An abstraction should hide implementation details while explaining the behaviour callers depend on.

> ONE OPERATION, SEVERAL HIDDEN STEPS
>
> The service asks “store this task.” The repository handles the query and commit needed to carry out that request.

Encapsulation controls access to a component’s internal data. If every part of the application can change the repository’s records directly, it can bypass the rules we have just defined. Returning a separate task record, rather than a reference that callers can use to edit internal state, helps preserve that control.

Modularity is the organisation of a system into parts with clear responsibilities and interfaces. Separate files can help, but the division is useful only if it limits what each part must know about the others. Ten files that all edit the same internal data can still be difficult to change.

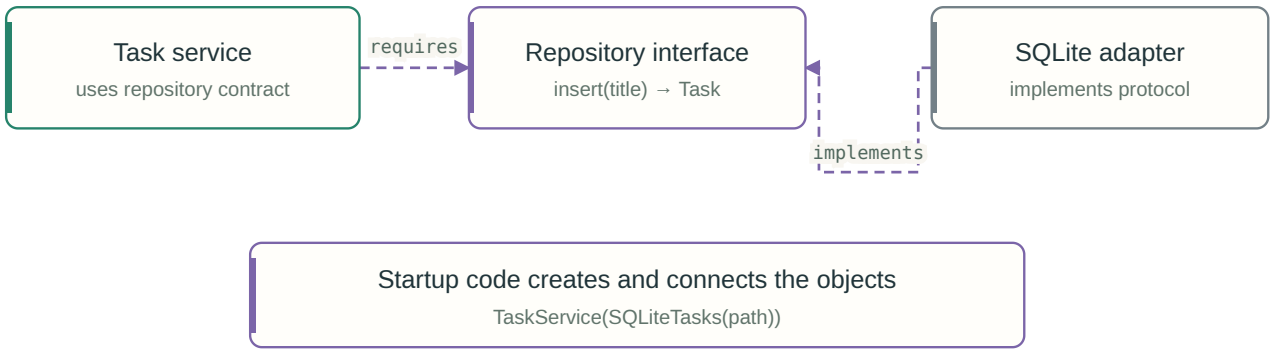


FIG. 1.2.3 **The service uses a repository contract**

The service uses the repository interface. The SQLite adapter implements that interface. Startup code creates the adapter and passes it to the service; during execution, the service calls it.

Why this view: The dashed arrows show code dependencies. These objects can run in the same process. Author-created illustration.

At application startup, we create the SQLite repository and pass it to the service: `TaskService(SQLiteTasks(path))`. Supplying a dependency this way is called **dependency injection**. The startup code that creates and connects these objects is sometimes called the **composition root**. Unit 6 examines these ideas in detail.

Read the dashed arrows in the figure as code dependencies: the service uses the repository contract, and the SQLite adapter implements it. During execution, the service calls that adapter. All of this can happen in one process, without a network call.

1.2.4 · Check the design by changing it

We can judge the division by asking where an edit would be needed. Consider four changes to the application.

Table 1.2.1 · Where should a change be made?

Change	Main place to edit	What else needs checking?
Accept a web form instead of JSON.	Web input adapter.	Does it pass the same title to the service and display errors correctly?

Change	Main place to edit	What else needs checking?
Increase the title limit.	Task service.	Can the database store the longer title? Update any matching database constraint.
Rename a database column.	SQLite repository and the procedure that updates existing data.	Can old records still be read after the change?
Recognise a repeated create request.	Input contract, service, and repository.	Can the application identify the request and recover its earlier result?

The last change affects several parts because it adds new behaviour. A caller must identify a repeated request, and storage must keep enough information to recognise it. Good modules help us see where agreement is needed; they cannot make every change fit inside one file.

TRY IT · INTERACTIVE MODEL

Where would you make the change?

Choose a change and compare the parts that need editing in the two designs.

Scenario: Replace JSON input with a form

Tangled design: Handler · Task logic · SQL inside the handler

Separated design: Input adapter

Update the adapter that reads input. It can pass the same title to the service, which saves it in the same way.

The results follow our example's division of responsibilities. They show where edits are needed, without estimating how long those edits take.

1.2.5 · Understand what a layer hides

We can also arrange the application as a series of **abstraction layers**. Each layer uses the operations offered by the one below it. A user asks to remember a task. The application turns that into a storage operation. The storage implementation uses the operating system to read and write files.

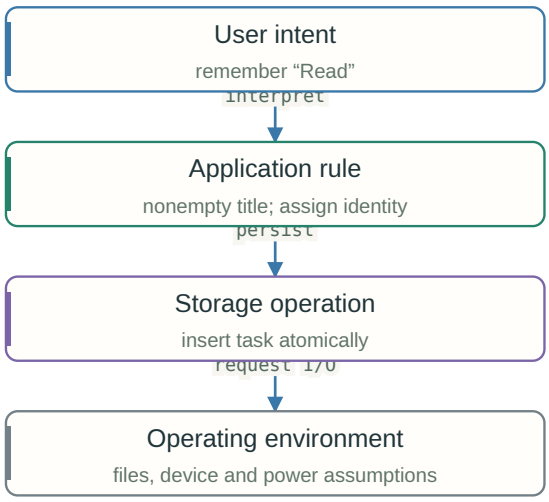


FIG. 1.2.4 **Each layer works at a different level of detail**

The user asks to remember a task. The application applies task rules and requests a storage write. Storage uses files and the underlying device to carry out the write.

Why this view: Follow one request through the layers to connect the user’s action to the work below it. Author-created illustration.

Each step lets us work at a different level of detail. When writing title validation, we do not need to reason about disk blocks. When investigating a damaged database file, those lower-level details may become important.

Some details must be visible in the contract. For example, suppose a repository returns results one at a time and keeps a database connection open until the caller finishes reading. A slow caller can then hold that connection for a long time. The caller needs to know when and how to close the result. Alternatively, the repository could return a bounded page of records and release the connection before returning.

▼ A closer look · Why a test substitute cannot check persistence

We can test title validation using a simple repository that stores tasks in memory. This makes the test quick and easy to control. But the substitute loses its tasks when it is discarded. It cannot demonstrate that SQLite saves a task across a restart.

For that check, use the real repository: create a task, close the connection, open a new connection to the same database, and read the task. The two tests answer different questions. Matching method names does not mean that two implementations have the same behaviour in every situation.

1.2.6 · Use the amount of structure the application needs

In this example, separating the title rule helps because both the web and command-line interfaces use it. Keeping SQL in a repository helps because several features need tasks without needing to know the table layout. These are concrete reasons for the boundaries.

Every added interface also takes work to understand and maintain. A short, temporary script may need only a few functions. As the program grows, introduce a boundary where it protects a rule, hides a storage detail, or allows a real change to be made safely.

The modules still run in one application. If that process crashes, all of them stop. Code organisation improves how we understand, change, and test the application; keeping it running through a machine failure requires further design work.

> COMMON MISTAKES
>
> Separate files do not necessarily have separate responsibilities. An interface does not automatically hide useful details. And fewer dependencies are not always better: the application needs well-defined dependencies to get its work done.

> FURTHER READING
>
> Parnas's 1972 paper examines how the same system can be divided into modules in different ways. It asks which design decisions each module should hide. Our storage example gives a small application of that question. Paper record and abstract ↗

Remember: Give related work a clear home. Let other modules use it through a contract. To check the division, follow a likely change and see which parts need editing.

Before continuing: Both the web form and command-line interface check the title limit. Where should the shared rule move, and which checks should remain in each adapter?

Unit 1 exercises → · Unit 1 explained solutions →

Sources & further reading

These sources explain the ideas and mechanisms used in this chapter. The to-do application is our own teaching example. Links were reviewed on 21 October 2026.

1. On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

   Original research · ACM DOI. Module decomposition criteria; metadata and abstract verified.

   `Access reviewed · 21 October 2026`

2. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   `Access reviewed · 21 October 2026`

From requirements to design choices

Prerequisites: 1.2 · Organising code into modules 4 original figures

Reading the figures: boxes name parts or states; arrow labels explain how they relate. Dashed arrows show a code dependency or a change in the level of detail. Each caption explains the example.

The team asks us to make the to-do application “fast, safe, and easy to maintain.” These are reasonable goals, but they leave important questions unanswered. How fast should a task list appear? Which failures should saved tasks survive? What changes should be easy to make?

Before choosing a design, we need to turn these goals into requirements that people can understand and check.

In this chapter: We will write a small set of requirements, compare storage choices, and record why one choice fits our application.

1.3.1 · State what the application must do

A **requirement** states something the system must do or a condition it must satisfy. “A user can mark a task complete” describes behaviour. “A saved task remains after the application restarts” describes a condition that behaviour must meet.

Requirements about what the system does are called **functional requirements**. Requirements about qualities such as speed, security, and ease of maintenance are commonly called **nonfunctional requirements**. Both influence the design.

Consider a shared version of our application. For this example, assume a team of 20 people, one application machine, and a small budget. During normal use, the application receives at most 5 requests per second. These are teaching assumptions, rather than measurements of a real service.

The team needs to create, list, and complete tasks. Each person’s tasks are private. A successful save must survive an application restart, provided the committed storage remains intact. At least 95% of list requests should finish within 300 milliseconds under an agreed test workload.

The timing requirement gives us something to measure. If a test sends 1,000 list requests, at least 950 must finish within 300 milliseconds. To make results comparable, we must also specify where the test runs, how many tasks it lists, and how much traffic it sends. Unit 2 develops this measurement process.

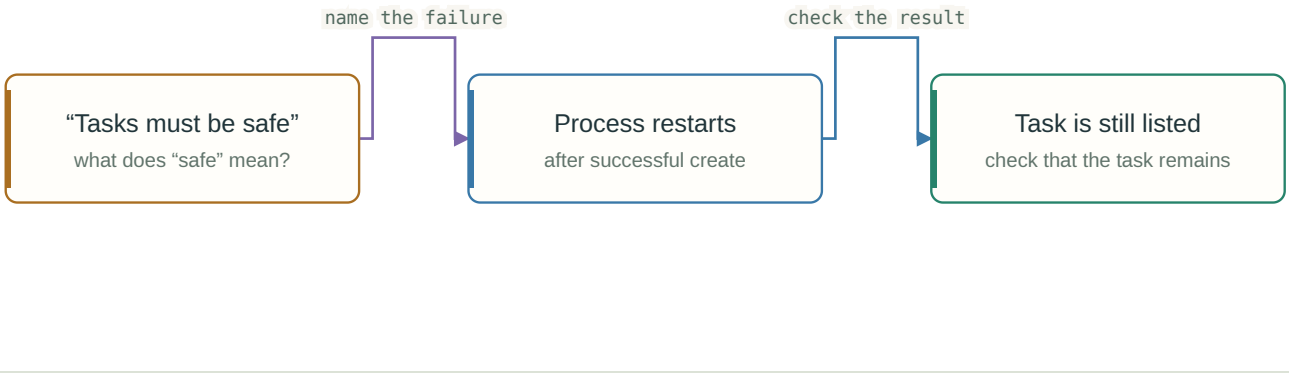


FIG. 1.3.1 **Turn “safe” into a checkable requirement**

Create a task successfully, restart the process, and check that the task remains. This requirement concerns a process restart with committed storage intact.

Why this view: A specific requirement tells us which test to run. Author-created illustration.

Table 1.3.1 · Making a requirement precise

Starting statement	Question to ask	A requirement we can check
“The app is fast.”	Which operation, under how much load?	At least 95% of list requests finish within 300 ms under the agreed workload.
“Tasks are safe.”	Which failure should they survive?	A task reported as saved can be read after the process restarts, with committed storage intact.
“Tasks are private.”	Who is allowed to do what?	Users cannot read or change another person’s tasks through any task endpoint.
“The code is easy to change.”	Which change should be easy?	Add a command-line interface that reuses the task service and repository.

Qualities such as performance, security, reliability, and maintainability are also called **quality attributes** or **architectural characteristics**. Their names help organise a review. A concrete scenario tells us what each one means for this application.

Some requirements fit more than one category. Checking whether a user may complete a task is application behaviour and part of its security. The classification is useful for organising work; the important step is to implement and check the requirement.

1.3.2 · Separate constraints, assumptions, and risks

Suppose the team says that the first version must run on an existing machine. That limits the choices available to us. Such a limit is a **constraint**. A fixed budget or a delivery deadline can also be a constraint.

We have assumed that the disk remains intact when the process restarts. An **assumption** is something we treat as true while making a decision. Writing it down makes clear where the reasoning applies.

Now suppose the only disk fails. We may lose the saved tasks. This is a **risk**: a possible event that could prevent the system from meeting its needs. The team must decide whether to accept it or change the design, for example by keeping recoverable copies elsewhere.

These terms answer different questions. A constraint asks what limits our choices. An assumption asks what our reasoning relies on. A risk asks what could go wrong. They often connect: relying on one intact disk exposes us to the risk of losing that disk.

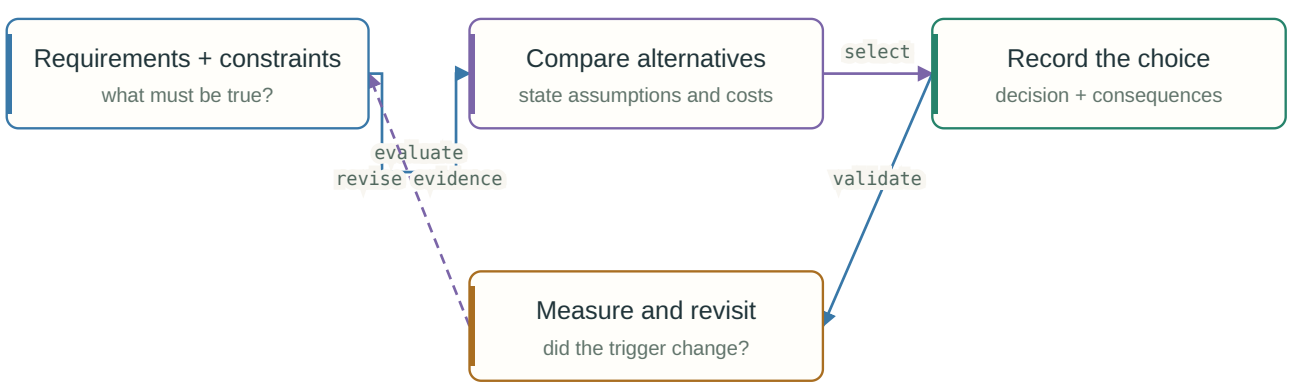


FIG. 1.3.2 **Choose, record, and review a design**

Write down the requirements, compare choices, and record the decision. Later, use measurements and changed requirements to decide whether the design still fits.

Why this view: Keep the reasons for a decision alongside the decision itself. Author-created illustration.

1.3.3 · Compare choices against the requirements

We have three storage candidates. We could keep tasks only in memory, use an embedded database such as SQLite, or run a separate database service. An **embedded database** runs as part of the application rather than as a separate server; here, it stores data in a local file.

First check the mandatory restart requirement. Memory alone fails it. However easy the code is to write, that option cannot meet the agreed need.

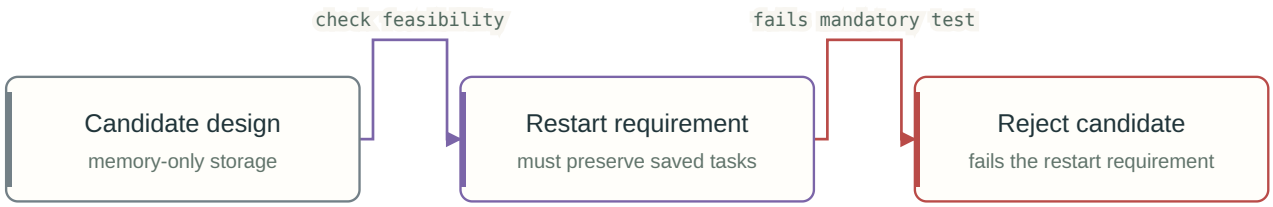


FIG. 1.3.3 **Check requirements before scoring preferences**

Memory-only storage loses tasks after restart, so we reject it for this application. We compare preference scores only for choices that meet the required behaviour.

Why this view: An easy-to-run choice is useful only if it meets the requirements. Author-created illustration.

The other two candidates can meet this requirement with suitable commit behaviour and storage settings. We can then compare their costs and benefits. Embedded storage keeps deployment small. A separate database can be accessed by multiple application machines, but also needs a network connection, access controls, and operating work.

A choice between competing benefits and costs is a **trade-off**. Which cost is worth paying depends on the requirements and on the people who will operate the system.

Table 1.3.2 · Example preference scores: 1 = poor fit, 5 = strong fit

Candidate	Survives process restart?	Simple to run here	Supports multiple app machines	Storage separate from app machine
Memory only	No; reject.	5	1	1

Candidate	Survives process restart?	Simple to run here	Supports multiple app machines	Storage separate from app machine
Embedded store	Yes, under our storage assumptions.	5	2	1
Separate database	Yes, with suitable commit settings.	2	4	4

The scores express judgments about fit for this example. They are not performance measurements. The low score for separating an embedded store from its application machine reflects our local-file setup, rather than a claim about every possible use of SQLite.

We can assign a **weight** to each preference. A higher weight means that preference matters more to the decision. A weighted score multiplies each preference score by its weight, adds the results, and divides by the sum of the weights.

If simplicity has weight 5 and the other two preferences have weight 1 each, the embedded store receives `(5×5 + 1×2 + 1×1) / 7 = 4.00`. The separate database receives `(5×2 + 1×4 + 1×4) / 7 ≈ 2.57`. With these priorities, the embedded store ranks higher.

TRY IT · INTERACTIVE MODEL

Which preferences matter most?

Weights (simplicity, multiple app machines, separate storage): 5, 1, 1.



Embedded store has the higher score with these weights. These are preference scores; they do not measure speed or availability.

Memory only: rejected because tasks disappear after restart.

The scores come from Table 1.3.2. A higher weight gives that preference more influence. Every acceptable choice must still preserve tasks across a process restart.

Try increasing the importance of multiple application machines and separate storage. The ranking may change. This is useful: it reveals how the decision depends on priorities. The memory-only option still fails

the restart requirement, regardless of its score.

1.3.4 · Record the decision and its reasons

A few months later, someone may ask why the team chose a local database. A short **Architecture Decision Record (ADR)** can preserve the answer. It records the situation, the decision, and the consequences. It should also say when the team needs to reconsider.

ADR 001 · ACCEPTED FOR THIS EXAMPLE

Use a local database and commit before reporting success

Situation

A small team uses one application machine. Tasks must survive a process restart. The current requirement does not include keeping the service available after that machine fails.

Decision

Use SQLite on the application machine. Keep input handling, task rules, and storage code in separate modules. Wait for commit before returning success.

Alternatives

Memory alone fails the restart requirement. A separate database is possible, but its extra operating work is not yet justified by the requirements.

Consequences

Deployment is small, and the application has a clear point at which a task is committed. Losing the machine stops the service. Losing its only disk can lose data. We still need backups, restore checks, access controls, and monitoring.

Reconsider when

The application must run on several machines, measured storage contention causes unacceptable delays, or tasks must survive loss of the machine or disk.

Checks to run

Create a task and restart the process. Simulate a storage error. Restore a backup. Measure request times with representative data and traffic.

An ADR need not be long. It needs enough detail for a later reader to understand why the choice made sense. Michael Nygard's article on documenting architecture decisions describes this practice.

If the requirements change, keep the old record and write a new one that replaces it. The earlier decision helps explain which assumption or need has changed.

1.3.5 · Distinguish the qualities we are checking

Some quality terms sound interchangeable. We need to separate them because they require different checks.

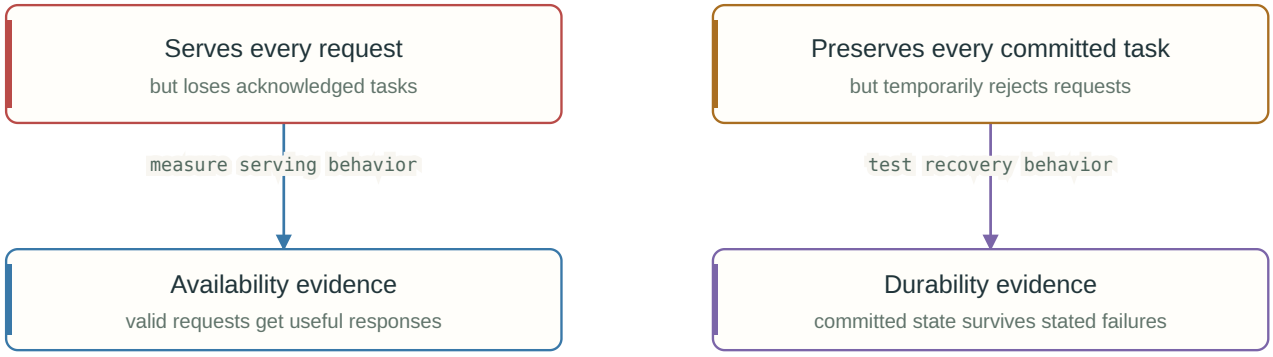


FIG. 1.3.4 **Availability and durability answer different questions**

A service may reply to requests yet lose tasks after restart. Another may keep every committed task but stop accepting requests while it recovers.

Why this view: Check both whether requests succeed and whether saved data survives the stated failure. Author-created illustration.

Availability concerns whether the system can provide the required service when needed. If users cannot list their tasks while the application restarts, it is unavailable for that operation during that interval. When measuring availability, define which requests count and what a useful response is.

Durability concerns whether committed data survives specified failures. Our restart test checks whether a saved task remains after the process stops and starts. It says nothing about losing the only disk.

Reliability usually refers more broadly to providing correct service over time. Teams sometimes use it differently, so a requirement should explain the failures and outcomes it covers.

Consistency concerns which observations of state the system allows. For example, after completing a task, should the user's next list request show it as complete? If the application later has several copies of the data, we must specify when each copy may show the change. Unit 14 examines precise consistency models.

These distinctions explain two possible outcomes. An application may keep replying to requests yet lose tasks it held only in memory. A database may preserve every committed task while refusing requests during recovery. The first has a data-loss problem; the second has a period of unavailability.

▼ A closer look · What makes a design simple enough?

Count the work needed to understand, change, operate, and recover the system. One large function may use few components yet be hard to maintain. A separate managed database may add a network connection while reducing backup work for a team.

The simplest sufficient design meets the agreed requirements with manageable work. Low traffic does not remove the need for access controls or correct saves. At the same time, an imagined future need does not automatically justify extra infrastructure today. Record likely changes and the conditions that would make them necessary.

COMMON MISTAKES

A high preference score cannot compensate for a failed requirement. More users do not, by themselves, tell us which part needs scaling. And a good uptime figure does not show whether saved data survives a failure.

Remember: Write requirements that can be checked. Reject choices that fail them. Compare the remaining choices, then record the reasons and the conditions for reconsidering.

Before continuing: The team still has only 20 users, but it now requires tasks to survive destruction of the only disk. Which part of ADR 001 must be reconsidered?

Unit 1 exercises → · Unit 1 explained solutions →

Sources & further reading

These sources explain the ideas and mechanisms used in this chapter. The to-do application is our own teaching example. Links were reviewed on 21 October 2026.

1. Documenting Architecture Decisions — Michael Nygard, 2011 ↗

   Original practice article. ADR structure and decision history.

   Access reviewed · 21 October 2026

2. AWS Well-Architected: the six pillars ↗

   Official guidance · accessed 21 Oct 2026. Review lenses, not correctness guarantees.

   Access reviewed · 21 October 2026

Drawing the system at different levels

Prerequisites: 1.3 · From requirements to design choices 4 original figures

Reading the figures: boxes name parts or states; arrow labels explain how they relate. Dashed arrows show a code dependency or a change in the level of detail. Each caption explains the example.

A diagram shows three boxes: input adapter, task service, and repository. A reviewer asks whether the application needs three servers. The author intended all three to be modules inside one program.

Both readers saw the same boxes but gave them different meanings. A useful system diagram must make its level of detail clear.

In this chapter: We will draw the to-do application from several viewpoints, trace a request, and use the drawings to find possible failures.

1.4.1 · Choose a view that answers a question

Suppose we want to explain the application to a new team member. We might begin with the person who uses it and the task system as a whole. A developer changing validation needs a closer view of the modules. Someone investigating a machine failure needs to see where the application and data run.

These are different **views** of the same system. Each selects the details needed for a particular question.

The C4 model names four levels for describing system structure: context, containers, components, and code. Moving through them is like zooming in on the application.

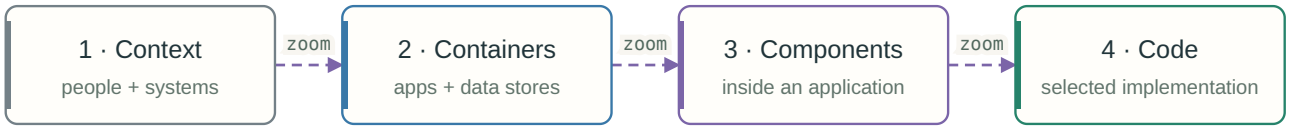


FIG. 1.4.1 **C4: zoom from the system into its code**

Each step shows more internal detail. The dashed arrows mean "zoom in." In C4, a container is an application or data store; it need not be a Docker container.

Why this view: Select the level that helps your reader answer a particular question. Author-created illustration.

Table 1.4.1 · Choosing a diagram

View	Question it answers	What it shows here
System context	Who uses the system, and which other systems interact with it?	A team member uses the task system.
Container	Which applications and data stores make up the system?	Browser client, server application, and task store.
Component	How is an application divided internally?	Input adapter, task service, and storage adapter inside the server application.
Code	How is a selected component implemented?	Task records, service methods, and the repository interface.
Sequence	In what order do events occur in one scenario?	The application validates a request, saves the task, commits, and replies.
Deployment	Where do the application and data run?	One machine runs the application and holds the database file.

A **container** has a particular meaning in C4: an application or data store. It can be a browser application, a server application, or a database. It does not have to be a Docker container. The C4 container definition explains this terminology.

Sequence and deployment views add information about order and location. They support the four structural levels rather than adding more levels of zoom. We can use whichever views help explain the design.

TRY IT · INTERACTIVE MODEL

Zoom in on the same application

Who uses the system?

Team member → uses → Task system

This view shows who uses the system. It leaves the internal parts for a closer view.

Select a level to see which parts it shows. Figure 1.4.1 and Table 1.4.1 explain all four levels without using these controls.

1.4.2 · Show where the program and data live

Our first version was an in-memory script. The second used modules and a local database. Now the team needs to access tasks from several computers, so we add a web interface. A browser sends requests to the server application, which uses the task service and repository.

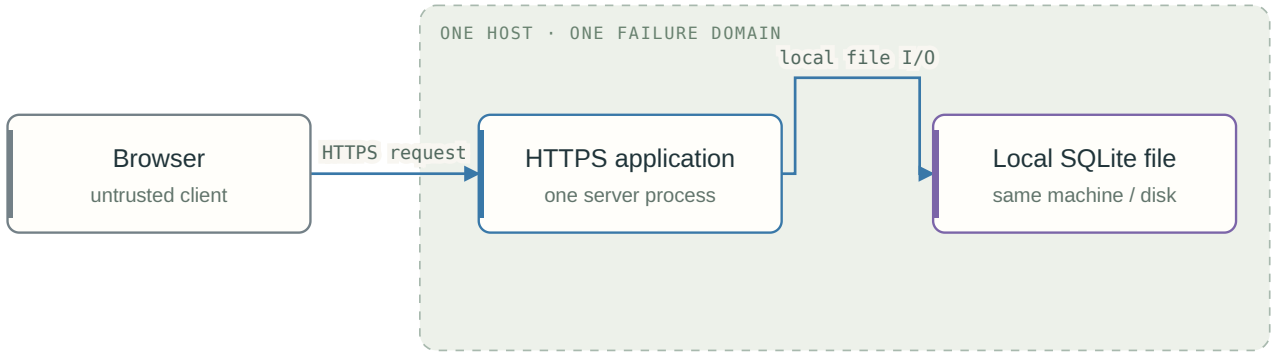


FIG. 1.4.2 **The application and database share one machine**

The browser sends requests to an application on one machine. The database file is on that same machine. If the machine fails, the application and access to the file are both affected.

Why this view: Show physical placement when asking what can fail together. Author-created illustration.

The deployment view shows an important detail: the server application and SQLite file are on the same machine. The browser is elsewhere. If the server machine fails, both the application and access to its database are affected.

Parts that can fail together because of one cause belong to a **failure domain**. Here, the machine is a shared failure domain. Drawing the service and repository as separate boxes does not give them independent machines.

The browser also introduces a **trust boundary**. The server cannot assume that a request is valid merely because it came from the application's usual web page. A user can construct requests directly.

The server must establish who the caller is; this is **authentication**. It must then check whether that caller may access the requested task; this is **authorization**. Every task operation needs the relevant check. Hiding another person's task from the web page does not prevent a direct request for it. Unit 18 develops these checks in detail.

1.4.3 · Trace a request from input to reply

Let us create a task through the web interface. The browser sends `POST /tasks` with a JSON body containing `{"title": "Read"}`. The following sequence describes the intended API; the local Python example is not an HTTP server.

```
1. Identify the caller from a validated session.
2. Read the title and check the input.
3. Begin a database transaction.
4. Insert the task with the caller as its owner.
5. Commit the transaction.
6. Return:
   201 Created
   Location: /tasks/7
   {"id": 7, "title": "Read", "done": false}
```

The `201 Created` response tells the client that creation succeeded. The `Location` header identifies the new resource. HTTP's specification defines this response. Waiting for the database commit is an additional rule of our application.

Now interrupt the sequence. If title validation fails, the application writes no task. If the transaction fails before commit, the application reports an error. If the process crashes after commit but before the reply, task 7 exists even though the browser has not received success.



FIG. 1.4.3 **The task is saved, but the reply is lost**

Task 7 is committed before its reply is lost. The client cannot tell what happened. A fresh create request may add task 8 unless the application recognises it as a retry.

Why this view: Trace the events in order to see why the client can be unsure even when a task exists. Author-created illustration.

What should the browser conclude after waiting too long for a reply? It cannot tell whether the task was saved. The request might never have reached the application, or the task might have been committed and the reply lost.

If the browser sends a fresh create request, the application may add task 8. The user intended one task but now has two. To recognise retries, the client and server need a stable identifier for the original operation. The server must store that identifier with its result so that a repeated attempt can retrieve the earlier result. Chapter 8.2 develops this mechanism, called request deduplication.

This is why event order belongs in the diagram. A drawing with only “request” and “response” can hide the commit that determines whether a task exists.

1.4.4 · Label arrows and copies of data

An arrow can mean a function call, a network request, a code dependency, or a step in a sequence. Label it with the actual relationship. For example, “HTTPS request,” “calls insert,” and “implements interface” describe different things.

Use a legend when colour or line style carries meaning. In this atlas, dashed arrows often show dependencies or zoom relationships, while solid arrows show the labelled action. Read the labels and caption to find the meaning in that particular view.

Also show where copies of data are held. The browser may display a task list it fetched a minute ago. The application may hold a task record in memory. The database contains the committed record. If another user

completes a task, the browser's old list does not update itself. We must decide when it fetches or receives new data.

A diagram that distinguishes these copies makes the freshness question visible. A single label saying "task data" does not tell us which copy a reader is seeing.

▼ A closer look · Four ways to review a diagram

1. **Stop a component.** What stops with it? If several modules share one process, a process crash affects all of them.
2. **Delay a call.** Who is waiting? Decide how long that caller can wait and what happens when the time runs out.
3. **Repeat a request.** Does it add another task? Point to the code and stored information that recognise a retry, if that behaviour is required.
4. **Change a data format.** Which readers and writers must change together? Check how old and new versions will work during the update.

These questions help uncover omissions. Tests are then needed to check the actual behaviour.

1.4.5 · Change the design when there is a reason

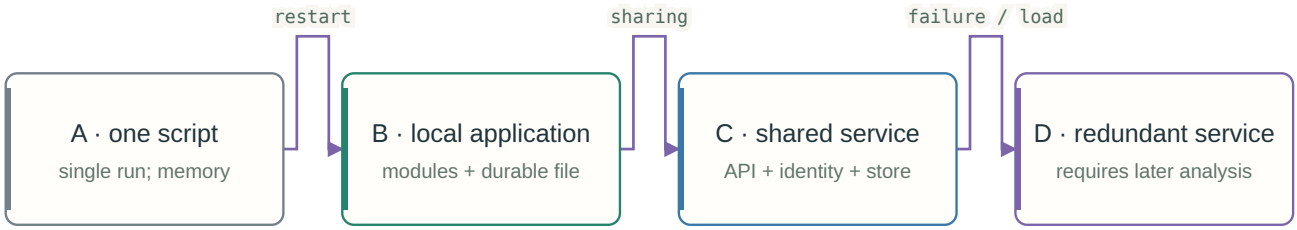


FIG. 1.4.4 **Each design change answers a new need**

Add storage when tasks must survive restart. Add a remote interface when people need shared access. Extra machines require further decisions about data, failures, and load.

Why this view: State the requirement that makes each change necessary. Author-created illustration.

Each change in our example answered a particular need. Persistent storage let tasks survive a process restart. Modules gave shared rules a clear home. A web interface let people use the application remotely. None of these changes made it survive failure of its only machine.

Suppose the application becomes slow. Before adding another server, measure where requests spend their time. A slow database query may need a better query or index. Adding application servers could send even more work to that same query. If computation in the application is the bottleneck and storage has spare capacity, additional workers may help.

Adding a second machine also changes our storage assumptions. If each machine has its own unrelated task database, a user may see different lists depending on which machine handles the request. We need a design for shared or coordinated data; a load balancer alone cannot supply it.

The drawings help us review these consequences together. A structure view assigns responsibilities. A sequence view shows event order. A deployment view shows where execution and data live. They should describe a consistent design.

> COMMON MISTAKES
>
> Three module boxes need not mean three servers. A timeout does not establish that a write failed. And another application machine does not automatically provide another usable copy of the data.

Remember: Choose a diagram for a specific question. Label what the boxes and arrows mean. Check that the code structure, request sequence, and deployment tell the same story.

Before continuing: Which view would help you find a reply sent before commit? Which view would show that the application and its only backup are on the same disk?

Unit 1 exercises → · Unit 1 explained solutions →

Sources & further reading

These sources explain the ideas and mechanisms used in this chapter. The to-do application is our own teaching example. Links were reviewed on 21 October 2026.

1. C4: diagrams ↗

   Original model documentation. Formal diagram levels and supporting views.

   Access reviewed · 21 October 2026

2. C4: container abstraction ↗

   Original model documentation. Meaning of container.

   Access reviewed · 21 October 2026

3. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

PART 1 / UNIT 02

Requirements, Estimation, and Engineering Trade-offs

A team asks us to build a social feed and gives us one number: ten million daily users. It is tempting to start counting servers. But a person who reads a short text post creates different work from someone who uploads a photograph or publishes to a million followers.

The running example

A social platform has 10 million daily active users. Estimate requests per second, traffic at peak, storage growth, image bandwidth, and approximate infrastructure requirements.

Unit 2 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

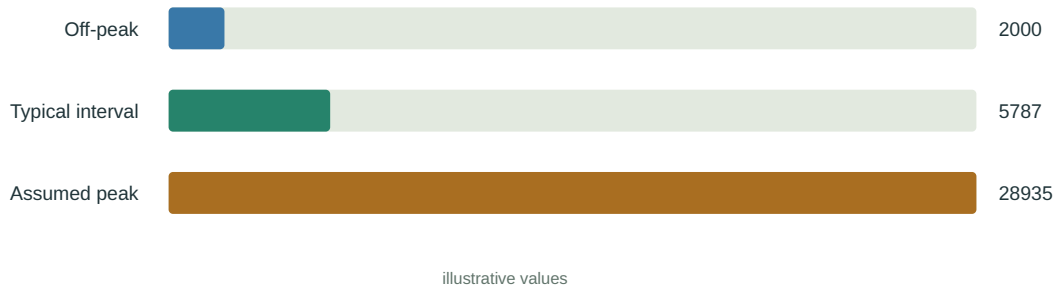


FIG. 2.0.1 **Request-volume timeline**

Request volume across illustrative intervals; these are assumed rates, not a measured daily traffic trace.

Author-created illustration. Numbers are illustrative unless stated otherwise.

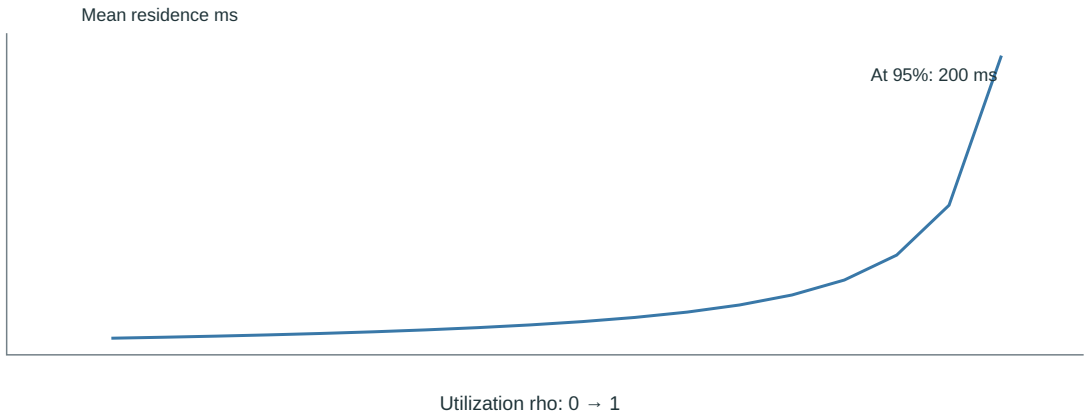


FIG. 2.0.2 **throughput-latency curve**

This single-server M/M/1 model assumes Poisson arrivals and independent exponential service times averaging 10 ms. Mean waiting plus service time rises as utilization approaches one; the curve does not predict p99.

Author-created illustration. Numbers are illustrative unless stated otherwise.

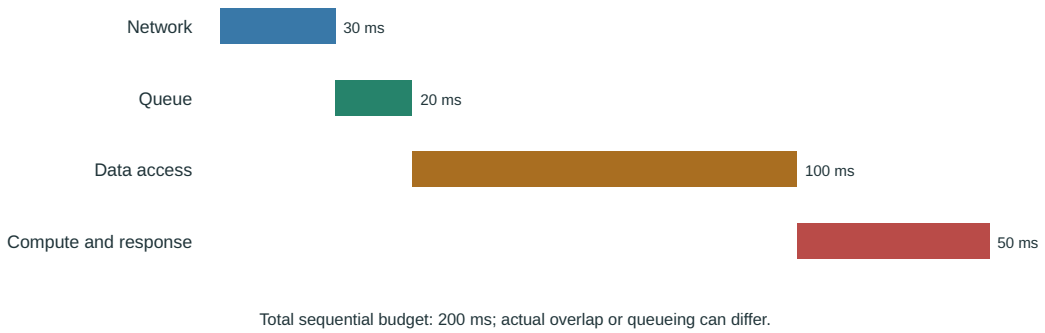


FIG. 2.0.3 **latency waterfall**

A 200 ms sequential budget allocates every stage; real overlap and tails require measurement.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Input	Calculation	Result
Daily API requests	10 million × 50	500 million
Average RPS	500 million / 86400	5787.04
Assumed peak	average × 5	28935.19
Post bytes/day	10 million × 2000	20 GB

FIG. 2.0.4 **capacity planning worksheet**

Explicit workload assumptions create a worksheet, not a benchmark.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Turn user journeys into workloads

Prerequisites: Introduction to System Design 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A team asks us to build a social feed and gives us one number: ten million daily users. It is tempting to start counting servers. But a person who reads a short text post creates different work from someone who uploads a photograph or publishes to a million followers.

Before estimating capacity, we need to find out what a successful visit looks like and what the system must do to make it happen. In this chapter, we will turn user journeys into observable requirements, identify the requests and writes they cause, and decide what we mean when we call a system read-heavy or write-heavy.

2.1.1 · Gathering and prioritizing requirements

Suppose we are asked to build a social feed. Before estimating servers, we need to know what people will do: publish posts, follow accounts, read updates, or report harmful content. Following one of these activities from the user's intention to its result gives us a user journey. Each journey causes different work inside the system.

We might ask the team, “If publishing works but followers cannot see the post, has the feature succeeded?” The answer determines what we must test. A requirement connects a person's intention to an observable result; a feature name such as “posting” leaves too much undecided.

Write the required outcome for each important journey. “Keep an accepted post and show it to followers within five seconds” is specific enough to discuss and test. Then separate essential behaviour from desirable improvements. Investigate requirements whose failure would cause the greatest harm or whose

feasibility is uncertain. Record exclusions too: private messaging should not silently enter an estimate for a public feed.

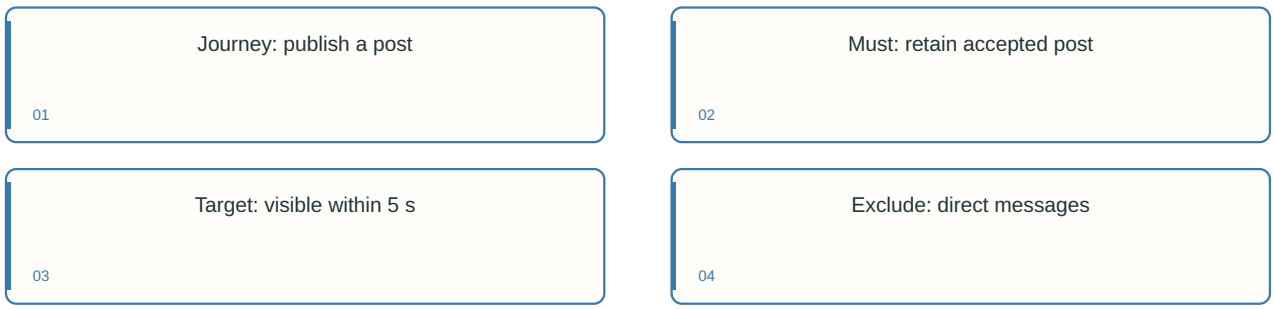


FIG. 2.1.1 **Gathering and prioritizing requirements · the example**

Requirements state necessary outcomes and constraints; prioritisation identifies what matters most and what still needs evidence.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.1.2 · User journeys and workload characteristics

Opening a feed appears to be one action. In an assumed session, however, the browser sends one feed request, fetches twenty images, and records one analytics event. The page first needs a list of posts, then the images named in that list. An estimate that counts only the page opening misses most of these requests. Drawing this short sequence helps us find the work before putting numbers against it.

A workload describes the mixture of operations and how often they arrive. For each operation, estimate its rate, the size of the data it carries, and the other components it needs. A small response can still require expensive computation; an image can require little computation but substantial transfer. Keep these workloads separate when finding bottlenecks: resources whose limits restrict useful work. The first estimates are assumptions. Replace them with measurements once the application is running.

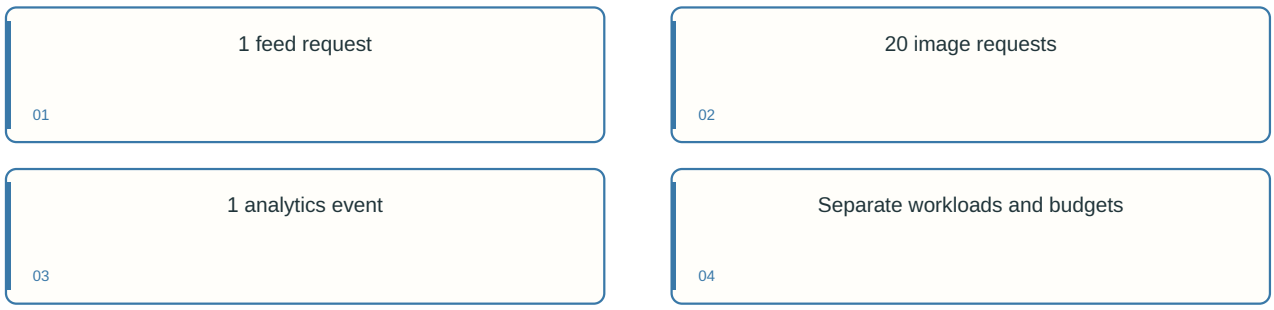


FIG. 2.1.2 **User journeys and workload characteristics · the example**

A workload describes the operations users cause, their frequency, payloads, and dependent work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.1.3 · Read-heavy vs write-heavy systems

Assume users read ten feeds for every post they publish. At the public API, this looks read-heavy. But suppose publishing saves one durable post and then copies a reference into a thousand follower feeds. That one action causes a thousand additional storage writes.

Imagine two authors: one has ten followers and another has a million. The same publish button can cause very different amounts of work. Counting posts alone hides this difference. We need both the frequency of the action and the amount of work that each action produces.

A read/write ratio therefore needs a boundary: are we counting user actions, API requests, or database operations? Creating follower feeds in advance moves work to publication time. Building them when requested moves work to reading time. The choice depends on how often users publish and read, as well as the distribution of follower counts. Additional database copies used to serve reads, called read replicas, can help some reads. If their updates are delayed, however, they may not immediately show a user's new post.

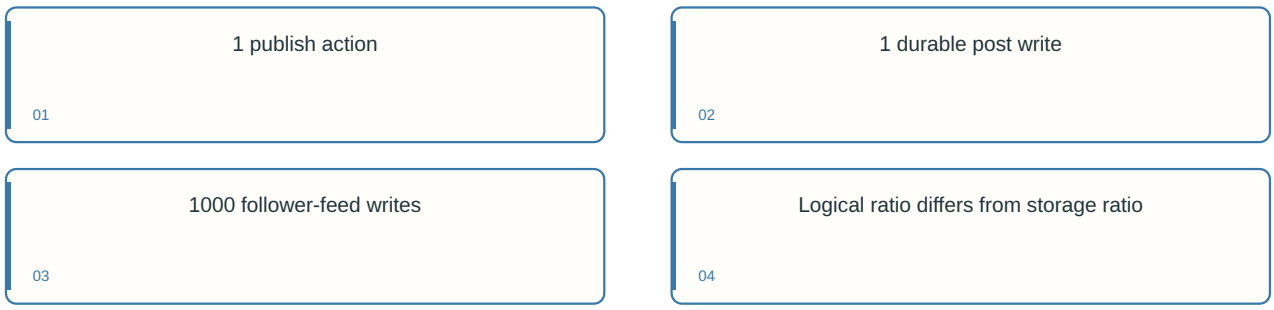


FIG. 2.1.3 **Read-heavy vs write-heavy systems · the example**

A read/write ratio counts work at a specified boundary; one user action can cause many physical operations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

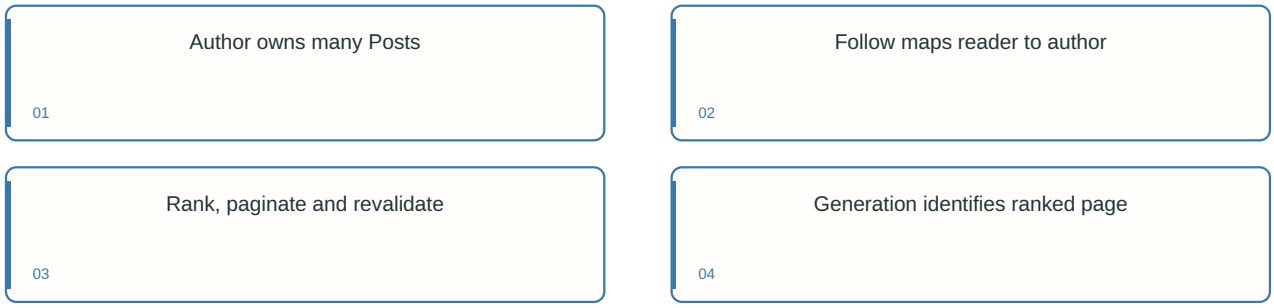


FIG. 2.1.4 **State and ownership in the related design**

The post and access relationships are authoritative; feed candidates are derived references.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How would simultaneous retries change an estimate based on average user sessions?

Retries add attempts without adding new user intentions. A daily session estimate therefore undercounts offered work when failed requests are repeated. If many callers retry at once, those attempts can arrive while the same dependency is still overloaded, creating another concentrated burst.

Model the timing and maximum number of attempts separately from normal sessions. Bound retries and admission, and test useful completions, queue growth, and recovery under that burst. A peak factor based only on ordinary user arrivals may not cover the retry wave.

Remember: Follow a user journey to its required outcome, then count the requests and storage operations at the boundary you are designing.

Try applying it: A new feature lets users upload and view images. Which additional requests, stored bytes, and transfer would you add to the workload, and which assumptions would you ask the team to supply?

Unit 2 exercises → · Unit 2 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Implementing SLOs — The SRE Workbook ↗

   Engineering book. Service indicators, objectives, and error budgets; not a source of the atlas workload numbers.

   `Access reviewed · 21 October 2026`

2. A Proof for the Queuing Formula: L = lambda W — John D. C. Little, 1961 ↗

   Original research · publisher record. Long-run queue accounting; publisher record inspected, full proof access may depend on access rights.

   `Access reviewed · 21 October 2026`

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   `Access reviewed · 21 October 2026`

Measure the experience

Prerequisites: 2.1 · Turn user journeys into workloads 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Our feed is running. A dashboard reports that it completes 180 requests each second, but users say that it keeps getting slower. The completion count alone cannot tell us how many requests are arriving, how long they wait, or how many users give up.

We need measurements that describe what people receive. In this chapter, we will distinguish rate from latency, examine the distribution behind an average, and separate promises about service, stored data, and observations of state. We will then turn one of these promises into a measurable objective and calculate how many failures its error budget allows.

2.2.1 · Latency, throughput, QPS, RPS, TPS

Suppose a server receives 200 requests each second and completes 180. If all the remaining requests stay queued, 200 − 180 = 20 more unfinished requests accumulate every second. After ten seconds, about 20 × 10 = 200 requests remain unfinished, assuming an empty starting queue and no rejection or abandonment. Reporting only the completion rate would hide the growing wait.

Latency is the time one operation takes within a stated boundary. From the caller's perspective, it includes waiting as well as execution. Throughput is the amount of work completed per unit time. A server can keep producing the same number of answers each second while users experience progressively longer delays.

RPS means requests per second, QPS queries per second, and TPS transactions per second. State which work each number counts, since one request may cause several queries. Measure offered work, successful completions, errors, and rejected requests together. Otherwise, an apparently steady throughput can conceal users who are waiting or giving up.

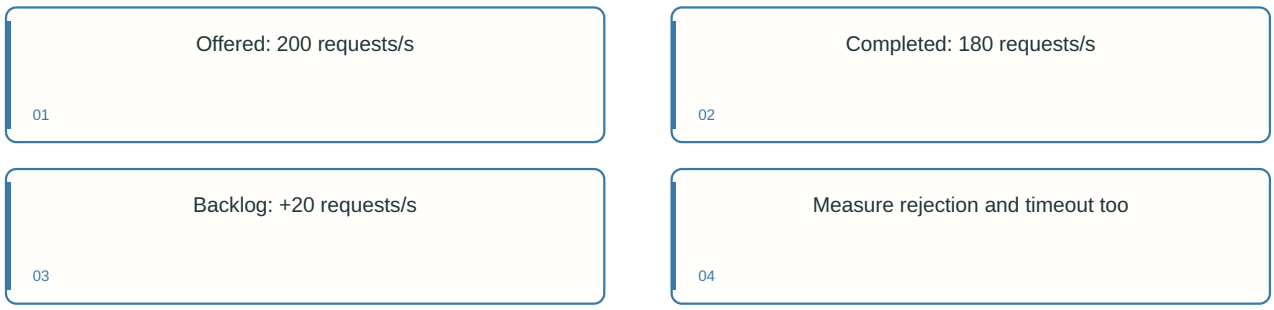


FIG. 2.2.1 **Latency, throughput, QPS, RPS, TPS · the example**

Latency is elapsed time per operation; throughput is completed work per interval. Define which queries, requests, or transactions are counted.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.2.2 · Average vs median vs p95/p99 latency

Consider 100 assumed request times: 99 requests take 10 milliseconds each, and one takes 1,000 milliseconds. The first group contributes 99 × 10 = 990 ms. Adding the slow request gives 1,990 ms, so the mean is 1,990/100 = 19.9 ms. That average tells us little about the user who waited a full second.

To find the median, arrange the observations from fastest to slowest and look at the middle. For these 100 observations, the two middle values are both 10 ms, so the median is 10 ms. A percentile names a position in this ordered set. Under the nearest-rank convention, p95 is the 95th value and p99 is the 99th value. Both are 10 ms here: even p99 can miss the single slowest request.

The slow end of the distribution is called its tail. Inspect the distribution and its extreme observations together; percentile estimates also depend on the convention and sample size. To find a fleet-wide p99, combine compatible observations or histograms, which count observations in latency ranges. Averaging individual servers' p99 values does not produce the fleet p99. Account for timed-out requests rather than discarding them.

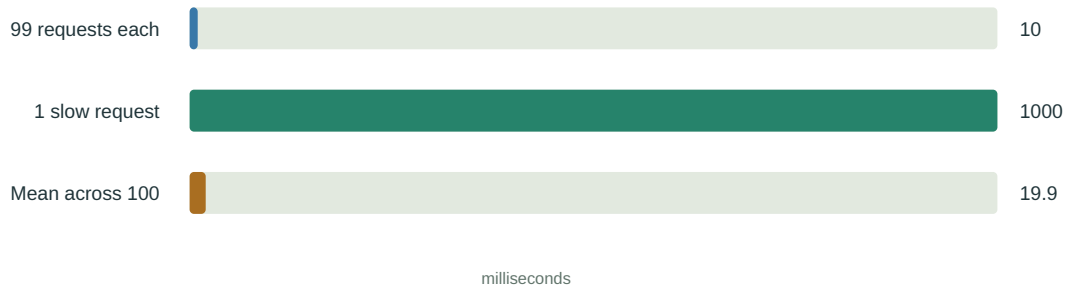


FIG. 2.2.2 **A small mean can hide one slow request**

The mean is (99×10+1000)/100 = 19.9 ms. This is a synthetic distribution, not a percentile convention.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.2.3 · Availability, reliability, durability, consistency

A payment status page can respond quickly while showing yesterday's information. Its ability to respond and the freshness of its answer are separate properties.

Ask four separate questions about the status page. Can it answer now? Is its answer permitted by the reading rules? Will a committed payment survive the failures we promised to tolerate? Does the service keep working correctly over time? Answering one question does not settle the other three.

Availability concerns providing the required service when needed. Durability concerns whether committed data survives specified failures. Reliability usually concerns correct service over time; state how it is being measured. Consistency defines which observations of state are allowed. For example, must the next read show a completed payment? Choose the requirement for each operation. A popularity count may tolerate delayed updates, while selling the last ticket requires stronger coordination. A single label such as “reliable database” leaves these decisions unanswered.

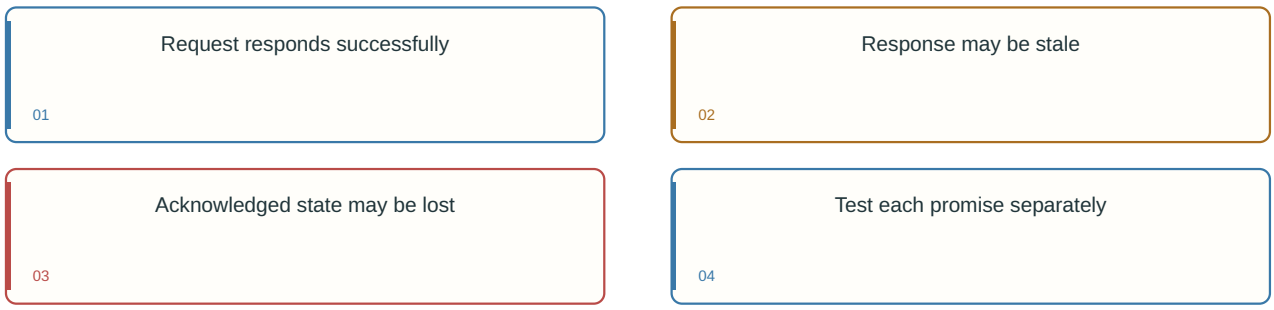


FIG. 2.2.3 **Availability, reliability, durability, consistency · the example**

Availability, durability, reliability, and consistency concern different serving, persistence, correctness, and observation requirements.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.2.4 · SLI, SLO, SLA, and error budgets

Suppose we want to know whether people can use a service successfully. A service-level indicator, or SLI, is a measured property of the service, such as the fraction of eligible requests that succeed. A service-level objective, or SLO, gives that indicator a target and measurement window: for example, 99.9% successful requests over a month. A service-level agreement, or SLA, is an agreement with stated obligations and possibly consequences.

Assume this service handles one million eligible requests in the month. A 99.9% success target leaves 100% − 99.9% = 0.1% for failures. Since 0.1% is 0.001 as a fraction, the allowance is 1,000,000 × 0.001 = 1,000 failed requests. This is its event-based error budget for that population and period. After 600 eligible failures, 1,000 − 600 = 400 remain within the target.

Define eligible requests and successful outcomes before measuring them. Excluding the failures we meant to track would defeat the purpose. A time-based objective uses time as its denominator and gives a different budget. Budget consumption can guide releases and reliability work, but failures concentrated during checkout may require action even while some budget remains.

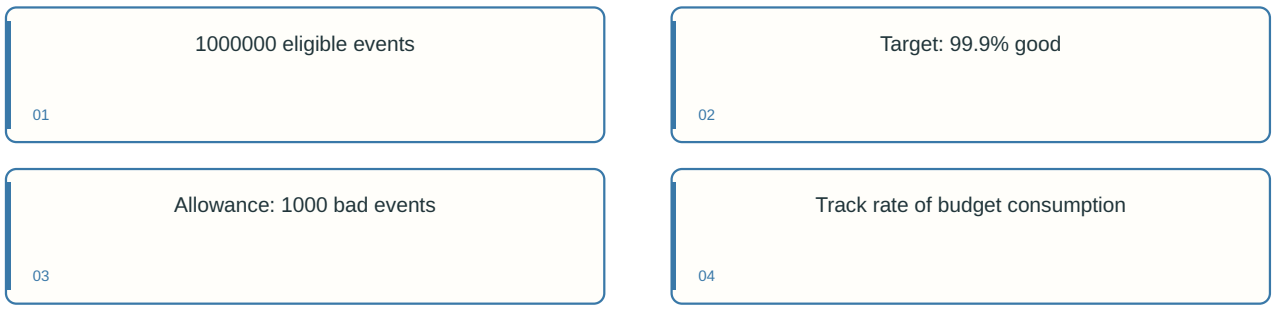


FIG. 2.2.4 **SLI, SLO, SLA, and error budgets · the example**

An SLI measures behaviour, an SLO targets it over a window, and an SLA states agreement obligations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How would simultaneous retries change an estimate based on average user sessions?

Retries add attempts without adding new user intentions. A daily session estimate therefore undercounts offered work when failed requests are repeated. If many callers retry at once, those attempts can arrive while the same dependency is still overloaded, creating another concentrated burst.

Model the timing and maximum number of attempts separately from normal sessions. Bound retries and admission, and test useful completions, queue growth, and recovery under that burst. A peak factor based only on ordinary user arrivals may not cover the retry wave.

Remember: Measure arrivals, useful completions, waits, and failures together. State the population, time window, and precise promise behind each target.

Try applying it: The service reports a 19.9 ms mean, but one of 100 users waited a second. Why might its p99 still be 10 ms, and what evidence should accompany that percentile?

Unit 2 exercises → · Unit 2 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Implementing SLOs — The SRE Workbook ↗

   Engineering book. Service indicators, objectives, and error budgets; not a source of the atlas workload numbers.

   Access reviewed · 21 October 2026

2. A Proof for the Queuing Formula: L = lambda W — John D. C. Little, 1961 ↗

   Original research · publisher record. Long-run queue accounting; publisher record inspected, full proof access may depend on access rights.

   Access reviewed · 21 October 2026

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   Access reviewed · 21 October 2026

Estimate the resources

Prerequisites: 2.2 · Measure the experience 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

We now know which actions the feed must support and how we will judge their results. The next question is how much work those actions produce. Ten million daily users is only a starting point: we still need assumptions about requests, posts, image views, growth, and the times when people arrive together.

In this chapter, we will build a worksheet with those assumptions visible. We will calculate requests per second, daily storage growth, and image transfer separately, then consider peaks and the time needed to add capacity. The resulting estimates will tell us what to test and which resources need separate investigation.

2.3.1 · Capacity and growth planning

Suppose stored data grows by an assumed 10% each month. Starting with one unit of storage, the first month ends with 1 × 1.1 = 1.1 units. The next ends with 1.1 × 1.1 = 1.21 units. Each increase applies to the new, larger total. After twelve months, repeated multiplication gives $1 \times 1.1^{12} \approx 3.14$ units. This is compound growth.

The result is a planning scenario, not evidence that request traffic and CPU usage will also triple. Plan each important resource separately: stored history, frequently accessed data, background jobs, and spare capacity during failures. Include temporary space needed while old and new data layouts coexist during a migration.

Suppose storage is likely to fill first. We need to act early enough to add capacity before that happens, including the time needed to purchase, prepare, and move data onto it. A useful capacity plan therefore

names the limiting resource, the lead time for expansion, and the observation that triggers action. Revisit those assumptions as measurements arrive.

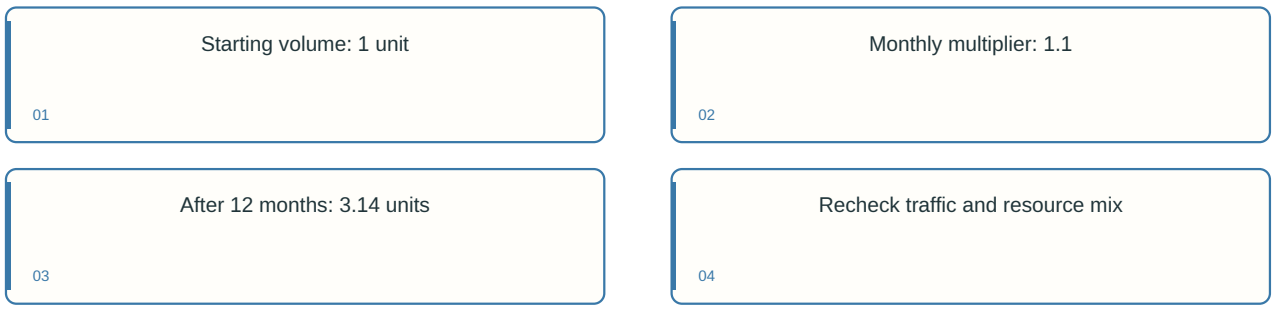


FIG. 2.3.1 **Capacity and growth planning · the example**

Capacity planning connects growth scenarios and resource limits to the time and evidence needed for expansion.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.3.2 · Storage, memory, bandwidth, and CPU estimates

Assume ten million daily users each make fifty API requests. The daily total is 10,000,000 users × 50 requests/user/day = 500,000,000 requests/day. A day contains 24 × 60 × 60 = 86,400 seconds, so the average rate is 500,000,000/86,400 ≈ 5,787 requests per second.

For this daily workload, separately assume that each user publishes one 2 kB post and views ten 200 kB images per day. These daily assumptions differ from the single-session example earlier. Using decimal units, 2 kB is 2,000 bytes, so raw post growth is 10,000,000 × 2,000 = 20,000,000,000 bytes/day, or 20 GB/day. Image delivery is 10,000,000 × 10 × 200,000 = 20,000,000,000,000 bytes/day, or 20 TB/day.

Stored bytes accumulate while they are retained; delivered bytes measure transfer. Spreading 20 TB evenly across the day gives about 231.5 MB/s, or 1.85 gigabits/s because each byte contains eight bits. This average bandwidth excludes protocol overhead and bursts. Add indexes, extra copies, retained history, and transfer overhead to the relevant resource estimates.

Memory depends on the data kept active, including caches and in-flight requests. CPU demand depends on the work each operation performs, which we must measure. Neither follows from the user count alone. These assumed workloads give us inputs for testing, not company benchmarks or a complete server count.

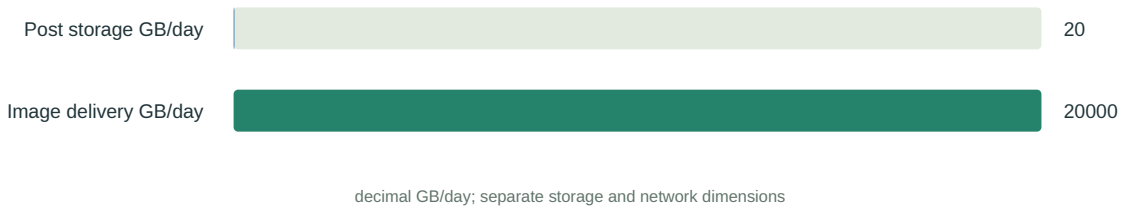


FIG. 2.3.2 **Different workload dimensions have different magnitudes**

Ten million assumed users each write one 2 kB post and view ten 200 kB images. Storage and delivered traffic are separate workloads.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.3.3 · Peak vs average load

A daily average spreads all requests evenly across 24 hours. Real users may arrive together after a notification or event. Suppose, for planning, that peak API traffic is five times the preceding average. The estimate becomes 5 × 5,787.04 ≈ 28,935 requests per second. Five is an assumed multiplier; a measured traffic trace would give us better evidence.

Peak duration matters too. Suppose a spike adds 1,000 requests each second beyond current capacity for 30 seconds. Without rejection or extra completions, it adds 1,000 requests/s × 30 s = 30,000 queued requests. The service must later complete this backlog, so the wait can continue after the incoming rate falls.

A bounded queue can absorb some temporary excess if the resulting wait is acceptable. Existing spare capacity and admission limits must cover the interval before extra instances are ready. Instances that take several minutes to start may arrive after a short spike ends. Image traffic and posting traffic may peak differently, and retries during an outage can add attempts that ordinary user averages miss.

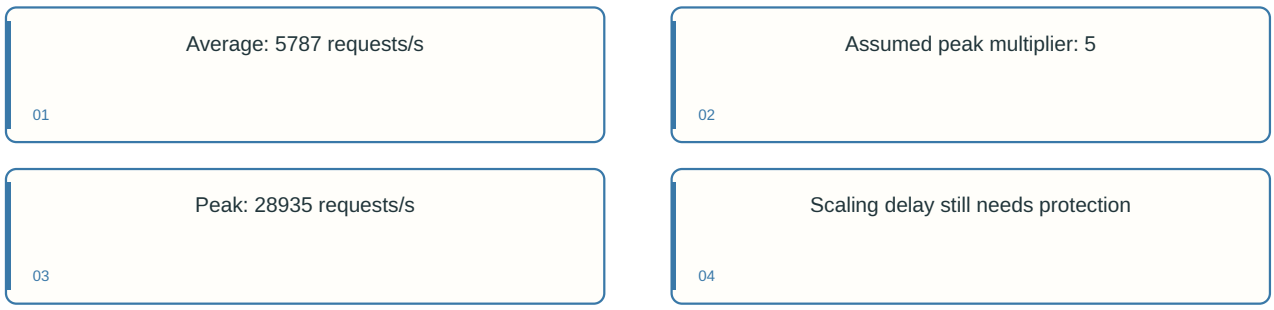


FIG. 2.3.3 **Peak vs average load · the example**

Peak load and its duration can differ substantially from an evenly distributed daily average.

Author-created illustration. Numbers are illustrative unless stated otherwise.

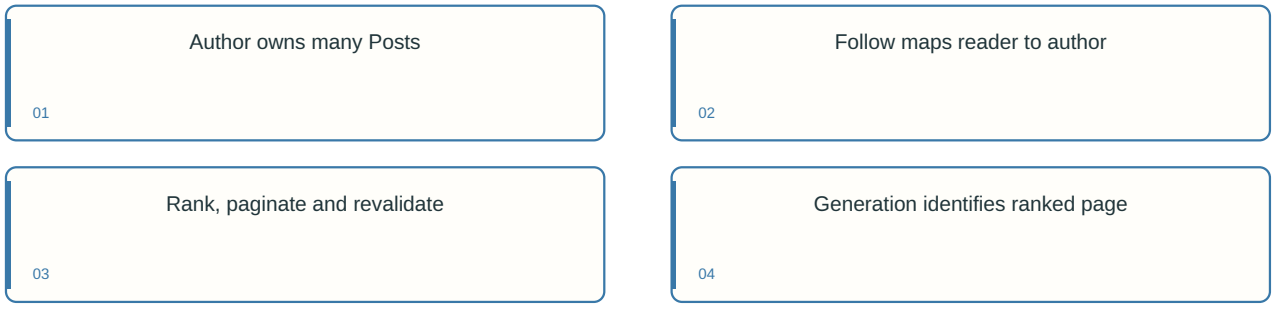


FIG. 2.3.4 **State and ownership in the related design**

The post and access relationships are authoritative; feed candidates are derived references.

Author-created illustration. Numbers are illustrative unless stated otherwise.

INFRASTRUCTURE WORKSHEET · ASSUMED CAPACITY

At 28,935.19 peak requests/s, a hypothetical tested instance capacity of 600 requests/s and a 60% utilization target imply ceil(28,935.19 / 360) = 81 active instances. Reserving capacity for one lost instance gives 82. This does not size the database, image delivery path, or failure domains. Validate each separately; the 600 figure is an assumed input, not a reported benchmark.

▼ Think further · How would simultaneous retries change an estimate based on average user

sessions?

Retries add attempts without adding new user intentions. A daily session estimate therefore undercounts offered work when failed requests are repeated. If many callers retry at once, those attempts can arrive while the same dependency is still overloaded, creating another concentrated burst.

Model the timing and maximum number of attempts separately from normal sessions. Bound retries and admission, and test useful completions, queue growth, and recovery under that burst. A peak factor based only on ordinary user arrivals may not cover the retry wave.

Remember: Keep workload assumptions and units visible. Estimate each resource separately, allow for growth and peaks, and test the capacity needed to serve that work.

Try applying it: If users view twice as many images but publish the same number of posts, which storage and transfer estimates change? What else must you know before changing the API server count?

Unit 2 exercises → · Unit 2 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Implementing SLOs — The SRE Workbook ↗

   Engineering book. Service indicators, objectives, and error budgets; not a source of the atlas workload numbers.

   `Access reviewed · 21 October 2026`

2. A Proof for the Queuing Formula: L = lambda W — John D. C. Little, 1961 ↗

   Original research · publisher record. Long-run queue accounting; publisher record inspected, full proof access may depend on access rights.

   `Access reviewed · 21 October 2026`

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   `Access reviewed · 21 October 2026`

Queues, bottlenecks & defensible decisions

Prerequisites: 2.3 · Estimate the resources 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Suppose the estimated traffic reaches our service, every worker is busy, and the queue keeps growing. Adding workers may help, but some work cannot be divided and some requests must wait for many other calls. We need to understand these limits before choosing a larger deployment.

In this chapter, we will relate average work in progress to time spent in the service, calculate the limit of parallel speedup, and see how dependent calls affect slow requests. We will then compare designs against mandatory requirements and use an explicit decision matrix to discuss the feasible choices.

2.4.1 · Little's Law and queueing intuition

Imagine a stable service completing 500 operations per second, with arrivals balanced by completions over a sufficiently long period. Each operation spends an average of 40 milliseconds inside the boundary we measure, including any waiting there. We want to estimate how many operations are present inside that boundary at once.

Little's Law relates three averages. Let L be the average number of operations in the system, λ the long-run arrival rate, and W the average time an operation spends inside it. Then L = λW. Convert 40 ms to 0.04 seconds to match the rate: 500 operations/s × 0.04 s = 20 operations. About twenty are present on average, either waiting or being served. This does not mean there are exactly twenty at every moment.

If mean residence time rises to 0.4 seconds while stable throughput remains 500 operations/s, the average number present becomes 500 × 0.4 = 200. Longer residence leaves more work in progress. Use consistent boundaries and units: a rate measured at the API cannot simply be combined with time spent only inside the

database. The relation describes stable averages; it neither predicts p99 nor makes a queue with sustained excess arrivals stable.

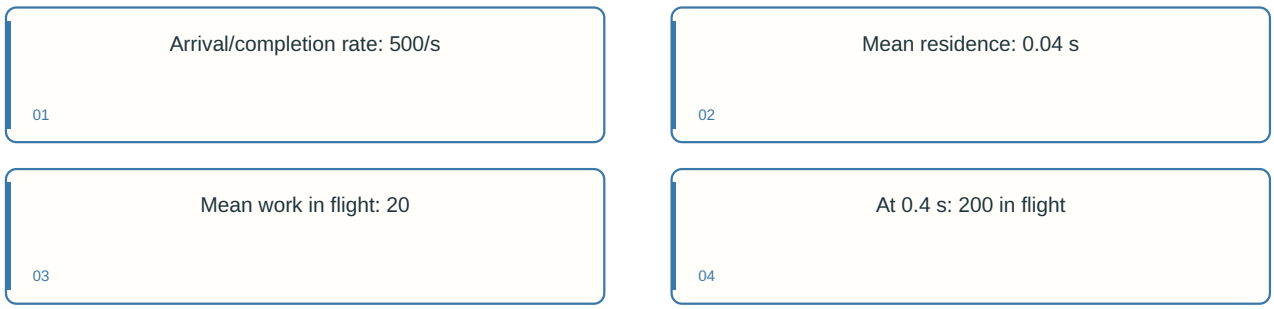


FIG. 2.4.1 **Little's Law and queueing intuition · the example**

Little's Law relates stable long-run averages: work in progress equals rate times residence time within one boundary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.4.2 · Amdahl's Law and bottlenecks

Suppose a job takes 100 seconds on one worker. Twenty seconds must run sequentially, meaning that this part cannot be divided among workers. The other eighty seconds can be shared perfectly. With four workers, the shareable part takes 80/4 = 20 seconds. Total time becomes 20 + 20 = 40 seconds, so the ideal speedup is 100/40 = 2.5 times.

This is an application of Amdahl's Law for a fixed amount of work. Expressed as fractions of the original time, the sequential part is 0.2 and the parallel part is 0.8. The new duration is 0.2 + 0.8/4 = 0.4 of the old duration. Even infinitely many workers cannot remove the sequential twenty seconds, giving an ideal ceiling of 100/20 = five times.

The calculation assumes no extra cost from dividing and coordinating the work. Such costs can reduce the real benefit. Find the stage that limits progress before adding workers. After improving it, measure again: another stage may become the new bottleneck.

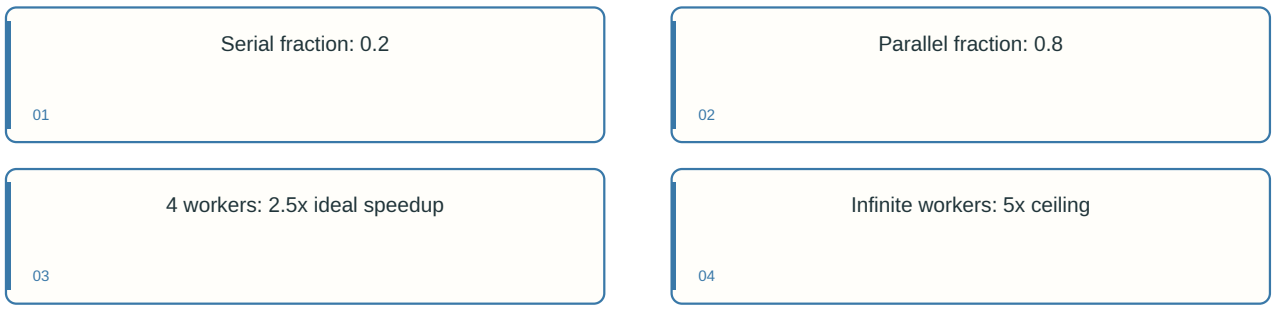


FIG. 2.4.2 **Amdahl's Law and bottlenecks · the example**

Amdahl's Law bounds fixed-work speedup when part of the job remains sequential.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.4.3 · Tail latency and latency budgets

A page request may start fifty calls in parallel and need all their results. Its completion depends on the slowest required call: forty-nine quick answers cannot compensate for a missing last answer. Making several calls from one request is called fan-out.

Assume each call independently has a 1% chance of exceeding a chosen time threshold. Each therefore has a 99%, or 0.99, chance of finishing within it. Independence lets us multiply these probabilities: the chance that all fifty are fast is $0.99^{50} \approx 0.605$. The remaining probability, $1 - 0.605 \approx 0.395$, means about 39.5% of page requests have at least one slow required call in this model. Shared dependencies can correlate delays, so independence must be justified.

We can sometimes change which calls are required: recommendations might arrive later while the main page appears. For the calls that remain required, divide the overall deadline into a latency budget. The illustrated sequential example allows 30 ms for network travel, 20 ms for waiting, 100 ms for data access, and 50 ms for computation and the response: 200 ms altogether. Real overlap and variability need measurement. Sending duplicate calls to reduce waiting spends extra capacity and requires repeated execution to be safe.

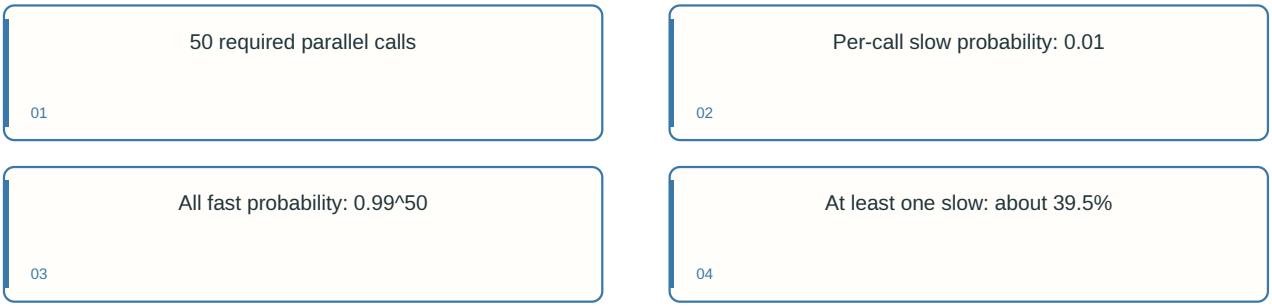


FIG. 2.4.3 **Tail latency and latency budgets · the example**

A request needing many results can be delayed by its slowest dependency; budget the complete path.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.4.4 · Cost, security, operational complexity, and maintainability as constraints

Suppose two designs both handle the expected traffic. One is cheaper but allows a tenant's cached result to reach another tenant. It fails a security requirement and should be rejected before comparing prices.

Write mandatory requirements as pass-or-fail checks first. “A tenant must never receive another tenant's private result” is one such check. We can then compare cost and ease of operation among the candidates that pass. Otherwise, a cheap but unsuitable candidate can look attractive simply because we assigned a small numerical weight to an essential rule.

Other differences may be preferences: one design costs more money but takes less operating work. Include staffing, maintenance, migration, and recovery in the comparison. A team also needs the skills to run what it builds. Compare candidates against the same workload and failure requirements. A price advantage obtained by dropping backups or weakening isolation represents a different service, rather than a like-for-like saving.

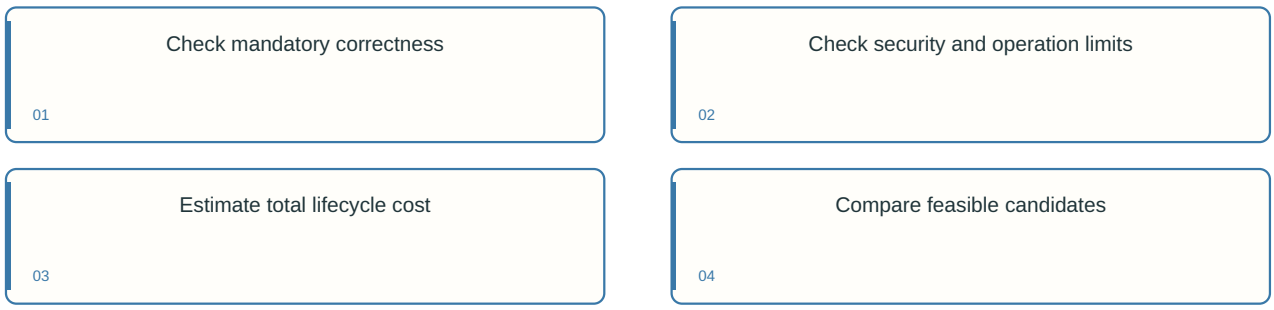


FIG. 2.4.4 **Cost, security, operational complexity, and maintainability as constraints · the example**

Feasible designs must satisfy mandatory security, cost, and operating constraints before preferences are ranked.

Author-created illustration. Numbers are illustrative unless stated otherwise.

2.4.5 · Decision matrices and trade-off analysis

After rejecting designs that fail mandatory requirements, we may still have several reasonable choices. A decision matrix lists the criteria, gives each candidate a score, and weights the criteria according to their importance. It makes the team's judgments visible.

Suppose cost has weight two and ease of operation has weight one. A candidate scoring three on cost and five on operation receives $2 \times 3 + 1 \times 5 = 6 + 5 = 11$. Here a higher score means a more desirable result, so a higher cost score must mean better affordability. Use that same direction in every column and explain how the scores were chosen. The total summarises these judgments; it does not independently establish them.

Try changing uncertain scores and weights. If a small change reverses the result, seek better evidence or prefer a choice that is easier to reverse. Record the choice, reason, and condition for reconsidering it in an architecture decision record, or ADR. Tests are still needed to establish performance and correctness. A high preference score cannot make an unsupported requirement true.

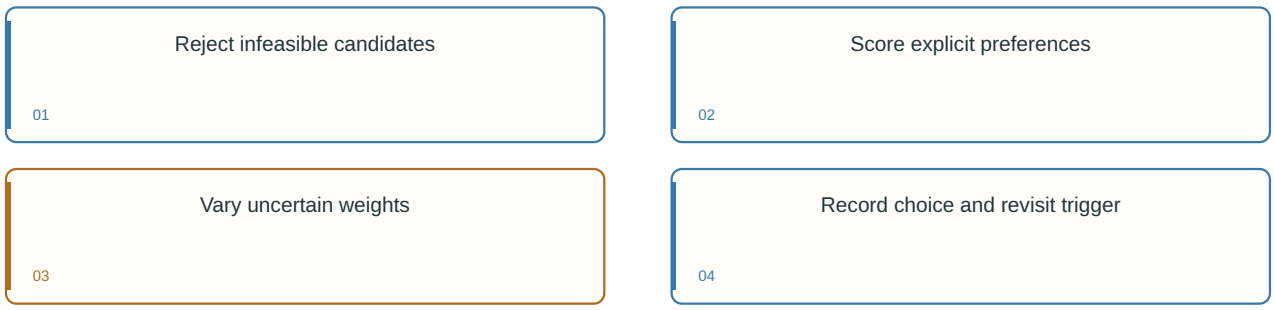


FIG. 2.4.5 **Decision matrices and trade-off analysis · the example**

A decision matrix makes weighted judgments explicit; it does not replace correctness evidence or measurements.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · EXAMPLES/MECHANISMS.PY

```
from mechanisms import capacity, amdahl
print(capacity())
print(amdahl(.8, 4))
```

The capacity() defaults assume ten million users making fifty API requests each day. That is 500 million requests/day. Dividing by 86,400 seconds/day produces about 5,787.04 average requests/s, and the assumed peak factor of five gives 28,935.19 requests/s.

The assumed instance capacity is 600 requests/s, with a 60% utilisation target. Planned capacity is therefore 600 × 0.6 = 360 requests/s per instance. Peak demand divided by this allowance is about 80.38 instances. Rounding upward gives 81 active instances; providing 82 leaves 81 when one instance is lost. The worksheet sizes only this assumed request-serving resource. Database capacity, image delivery, routing, and the placement of instances across possible failures need separate checks.

The second call, amdahl(.8, 4), uses a parallel fraction of 0.8 and four workers. Its remaining sequential fraction is 0.2, so the ideal duration is 0.2 + 0.8/4 = 0.4 of the original. The printed speedup is 1/0.4 = 2.5. This model keeps the amount of work fixed and omits the cost of coordinating workers.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · TRY CHANGING THE INPUTS

Estimate capacity for the social application

Average API rate: **5,787.04 requests/s**; peak: **28,935.19 requests/s**.

Instances needed at the target utilization: **81**; instances if one may be lost: **82**.

Raw posts: 20 GB/day; image delivery: 20 TB/day under the fixed image assumptions.

Start with the stated daily workload, then change one input. Each user views ten 200 kB images and writes one 2 kB post per day. GB and TB use decimal units. The instance capacity is an assumption; database capacity and capacity after a larger outage need their own checks.

▼ Think further · How would simultaneous retries change an estimate based on average user sessions?

Retries add attempts without adding new user intentions. A daily session estimate therefore undercounts offered work when failed requests are repeated. If many callers retry at once, those attempts can arrive while the same dependency is still overloaded, creating another concentrated burst.

Model the timing and maximum number of attempts separately from normal sessions. Bound retries and admission, and test useful completions, queue growth, and recovery under that burst. A peak factor based only on ordinary user arrivals may not cover the retry wave.

Remember: Use queueing and parallelism models within their assumptions. Reject designs that fail mandatory requirements before scoring preferences, and record the evidence behind the choice.

Try applying it: Workers share one database that is already limiting completions. What could happen to the queue and user latency if you add workers without changing the database capacity?

Unit 2 exercises → · Unit 2 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Implementing SLOs — The SRE Workbook ↗

   Engineering book. Service indicators, objectives, and error budgets; not a source of the atlas workload numbers.

   Access reviewed · 21 October 2026

2. A Proof for the Queuing Formula: L = lambda W — John D. C. Little, 1961 ↗

   Original research · publisher record. Long-run queue accounting; publisher record inspected, full proof access may depend on access rights.

   Access reviewed · 21 October 2026

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   Access reviewed · 21 October 2026

PART 1 / UNIT 03

Networking Foundations for System Design

Our to-do application works on one computer. Now let us open it from a browser on another. How will the browser find the application, ask for the task list, and receive the answer?

The running example

Trace what happens when someone enters a website URL, from DNS lookup through server execution to browser rendering.

Unit 3 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

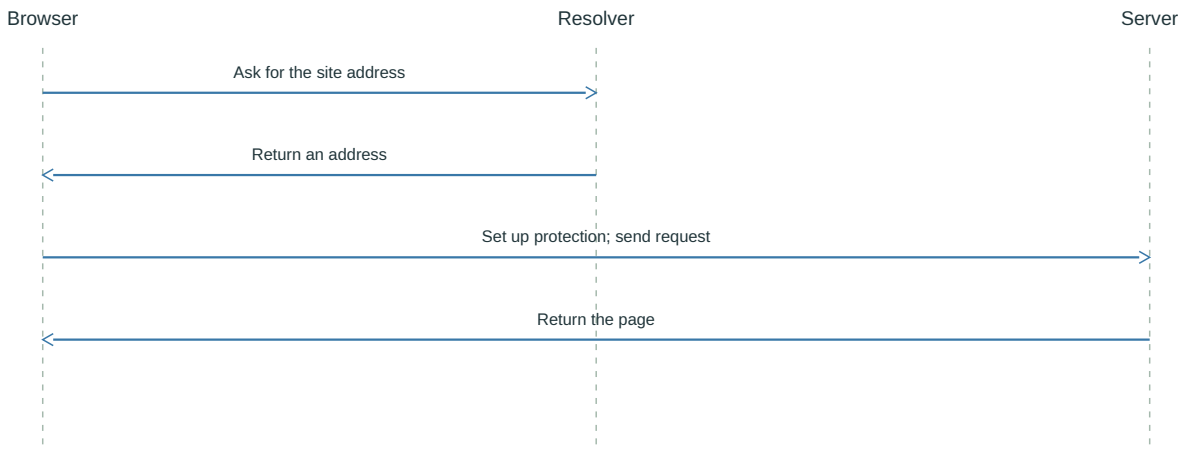


FIG. 3.0.1 **Complete HTTP request timeline**

Follow one request from the browser to the site and back. A saved address or existing connection can avoid some setup.

Author-created illustration. Numbers are illustrative unless stated otherwise.

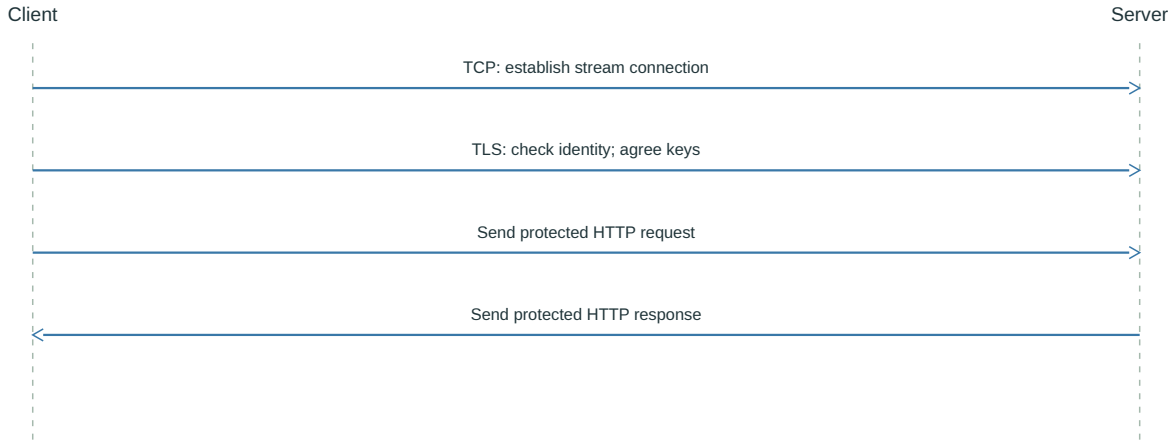
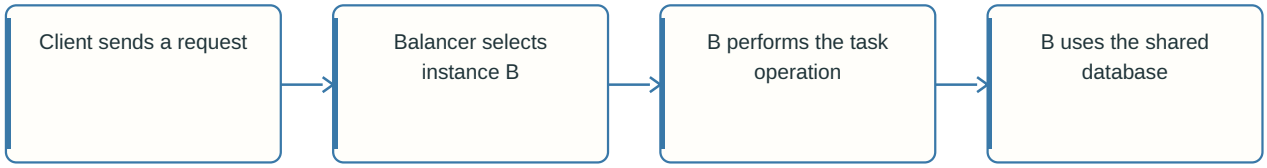


FIG. 3.0.2 **TCP/TLS interaction**

These rows show protocol stages rather than every packet. The cold/warm calculation uses assumed times for these stages.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 3.0.3 **load-balancing topology**

The balancer spreads traffic between application instances. A shared database can still limit all of them.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Method	How updates travel	After a missed update
Polling	Client asks again	Read a newer snapshot
SSE	Server sends events over HTTP	Reconnect and recover events
WebSocket	Both ends can send messages	Reconnect and recover history

FIG. 3.0.4 **WebSocket vs polling communication**

A live connection carries messages. The application must separately decide how a returning client recovers what it missed.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Addresses, sockets & transports

Prerequisites: Introduction to System Design 4 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

Our to-do application works on one computer. Now let us open it from a browser on another. How will the browser find the application, ask for the task list, and receive the answer?

We will follow that request and give each part of the journey a name.

3.1.1 · Follow a request from the browser

The **browser** is the program we use to view the page. It acts as a **client**: it asks another program for something. The **server** receives that request and sends a response.

Use the invented address `https://tasks.example/tasks`. This complete address is a **URL**. The name `tasks.example` identifies the site. The **path**, `/tasks`, identifies what we want from it. The `https` part asks for protected web communication; we will explain that protection in Chapter 3.2.

The browser finds a network address for the site and sets up communication. It then asks for `/tasks`. In a simplified HTTP request, this is `GET /tasks`. **HTTP**, the Hypertext Transfer Protocol, gives web requests and responses their rules. `GET` asks to read a resource, such as this page. A response marked `200 OK` reports success and carries the page data. The browser uses that data to display our task list.

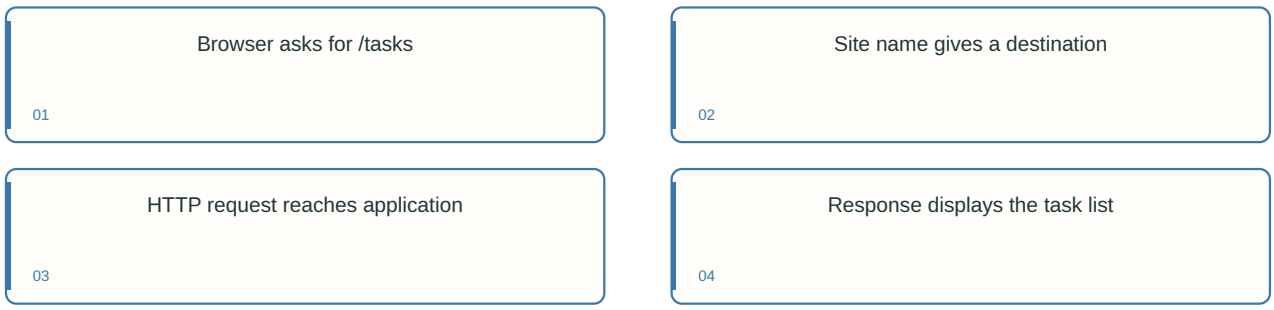


FIG. 3.1.1 **Follow a request from the browser · the example**

A browser sends an HTTP request for a resource, and the server returns a response that the browser can display.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.1.2 · Find the machine and the program

How does the request reach the right program? It needs both a machine's network address and a port number.

An **Internet Protocol (IP) address** identifies a network destination. Suppose our server uses the example address `203.0.113.10`. Data travels in **packets**, small units carrying data and addressing information. A **router** reads a packet's destination address and forwards it along a suitable next link. Repeating this step moves the packet toward the server. Choosing these next links is **routing**.

A **port number** helps the operating system find a listening service on that machine. Our web service listens on port `443`, commonly used for HTTPS. The server's **operating system**, the software managing its programs and network access, uses the port information to find that listener. The program reads and writes data through a **socket**, a communication object provided by the operating system.

The browser also uses a local address and port. The server needs that information to send replies back to this client. Listening only on `127.0.0.1`, the local loopback address, would instead make that listener available only on its own machine. A remote client also needs working routes in both directions.

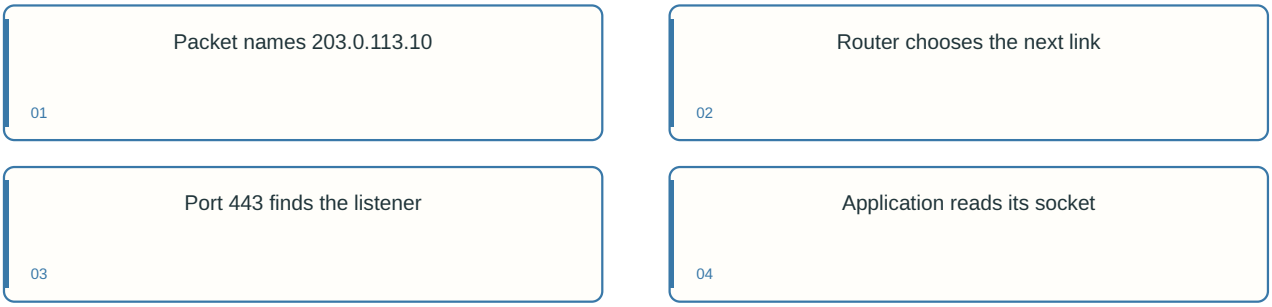


FIG. 3.1.2 **Find the machine and the program · the example**

An IP address leads toward a machine. Its operating system uses ports and sockets to deliver data to the right program.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.1.3 · Choose how data travels

Transport protocols give programs rules for carrying data between them. **TCP** (Transmission Control Protocol), **UDP** (User Datagram Protocol), and **QUIC** provide different transport arrangements.

TCP provides an ordered **byte stream**. Its connections are distinguished by both ends' addresses and ports, allowing many clients to use the same listening service. Suppose the sender writes `HELLO` and then `WORLD`. The receiver may read `HELLOWO`, followed by `RLD`. The bytes remain ordered, but the two writes do not become two separate messages. The application needs a **framing** rule, such as sending each message's length before its contents.

TCP numbers positions in the byte stream. Suppose bytes 1–100 arrive, bytes 101–200 are missing, and bytes 201–300 arrive. The receiver cannot yet deliver the later bytes as the next part of the stream. Acknowledgments report receiving progress, and TCP retransmits missing data. The application can receive the continuous ordered stream once the gap is filled. A connection can still fail before all data arrives.

UDP sends separate **datagrams**, whose boundaries are preserved. It supplies neither TCP's ordered stream nor its retransmission service. The application must arrange any recovery it needs.

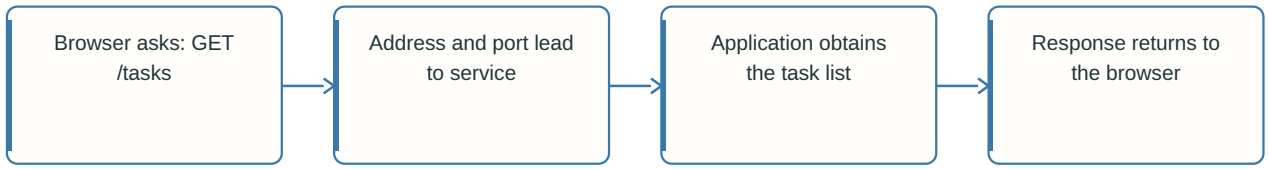
QUIC runs over UDP and supplies its own encryption, loss recovery, and ordered streams. A missing part of one stream need not stop another stream from delivering its data. The streams still share network capacity. QUIC also uses connection identifiers, which can help a connection remain recognisable when an address changes. None of these transports proves that the receiving application has saved or acted on the data.

Transport	What the receiver gets	Application responsibility
TCP	Ordered bytes in one stream	Recognise message boundaries
UDP	Separate datagrams; loss or reordering possible	Arrange any required recovery
QUIC	Ordered bytes in separate streams	Recognise messages within streams

FIG. 3.1.3 **What does each transport deliver?**

TCP and QUIC repair loss under their transport rules. UDP does not supply that service. The application still interprets messages and records their effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 3.1.4 **Follow the task-list request**

Addresses and ports bring data to the program. HTTP tells the program what was requested and gives the response its meaning.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Remember: The name helps find a machine; the address and port lead to a communication endpoint. The transport determines how data is carried, and the application gives that data meaning.

Try applying it: TCP delivers HELLOWORLD in one read. How could the receiver know that it contains two messages?

Unit 3 exercises → · Unit 3 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. RFC 9000: QUIC Transport, 2021 ↗

   Original specification. QUIC streams and transport behavior; application walkthrough is author-created.

   Access reviewed · 21 October 2026

3. RFC 8446: TLS 1.3, 2018 ↗

   Original specification. Secure transport handshake and protocol scope.

   Access reviewed · 21 October 2026

A secure request, one round trip at a time

Prerequisites: 3.1 · Addresses, sockets & transports 5 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

The first page takes longer to open than the next, although the server does the same work. What happened before the first request could be sent?

Let us find the site, protect the connection, and add up the time spent along the way.

3.2.1 · Find an address from a name

We know the name `tasks.example`. How do we obtain its IP address? The **Domain Name System**, or **DNS**, provides records associated with names.

The browser, or the operating system on its behalf, asks a **resolver**, a service that finds DNS answers. If the resolver has no usable saved answer, it follows DNS information to a server responsible for the name. That **authoritative server** supplies the published record. The resolver can then return the address to the client.

A **cache** saves a result for reuse. Suppose the returned record permits caching for 60 seconds. This permitted lifetime is its **time to live**, or **TTL**. A lookup ten seconds later can reuse the saved answer. After its permitted lifetime ends, the resolver normally needs a new answer before reusing that record.

Now change the published address. Existing caches can still hold the earlier answer until it expires. Clients with an open connection may not perform another lookup at all. DNS can also cache negative answers, such as “this name does not exist.” This is why changing a DNS record does not move every client at once.

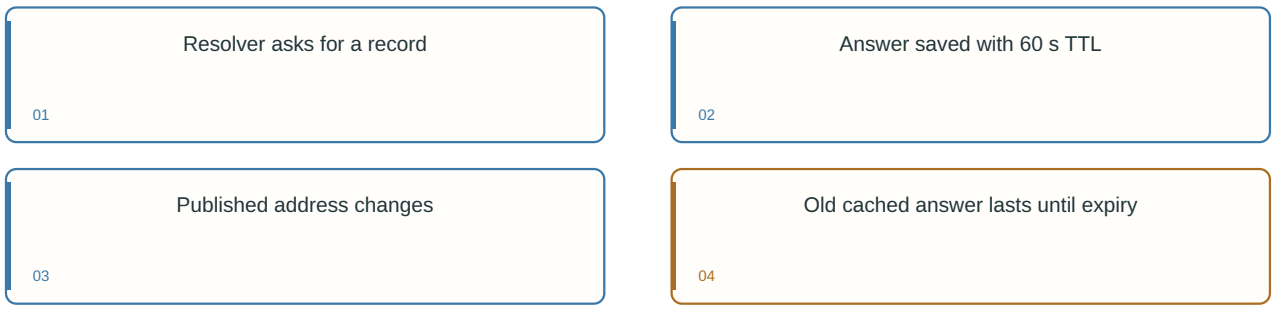


FIG. 3.2.1 **Find an address from a name · the example**

DNS finds records for a name. Cached answers can remain in use until their permitted lifetime ends.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.2.2 · Protect the conversation with TLS

The browser has found a server. It must check that the server really represents `tasks.example` and protect messages as they travel. **HTTPS** uses **Transport Layer Security**, or **TLS**, for this purpose.

A **certificate** is a signed statement linking a name to a public key. The **public key** may be shared; its matching **private key** is kept secret. A **digital signature** made with a private key can be checked with the matching public key. Changes to the signed data make that check fail.

The certificate carries its issuer's signature. The browser checks a chain of these signatures leading to a certificate authority it trusts. It also checks the requested name and the certificate's validity. During this initial exchange, the server proves it holds the required private key and the peers establish traffic keys.

The traffic keys protect later HTTP messages. **Encryption** makes their contents unreadable without the relevant keys. **Integrity protection** allows alteration to be detected. The identity check is **authentication**: establishing who the peer is.

This protection ends where TLS ends. If one server decrypts a request and forwards it, the next connection needs its own protection. The application must also check whether this user may read task 42. That permission check is **authorisation**. A protected connection alone cannot answer it.

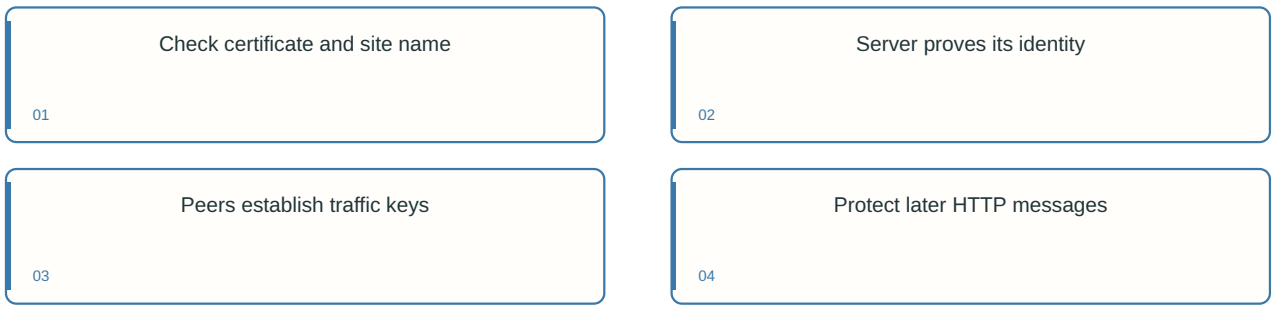


FIG. 3.2.2 **Protect the conversation with TLS · the example**

TLS checks peer identity and protects messages in transit. The application separately checks permission to use a resource.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.2.3 · Carry several HTTP exchanges

Our page needs a document and two images. These are three HTTP exchanges: each has a request and a response. A **header** is a named field carrying information about an HTTP message, such as its content type.

HTTP/1.1 can reuse connections, but an earlier slow response can hold up later responses on the same connection. Browsers commonly use several connections. HTTP/2 divides messages into smaller frames and labels them by **stream**, so data for several exchanges can share one connection. This sharing is called **multiplexing**.

HTTP/2 normally carries those frames over TCP. If a missing TCP byte blocks delivery, frames for several streams can wait behind the same gap. HTTP/3 uses QUIC streams, so loss in one stream need not block delivery in another. All three versions still use HTTP meanings such as `GET` and `200 OK`. They change how messages travel, rather than what those operations mean.

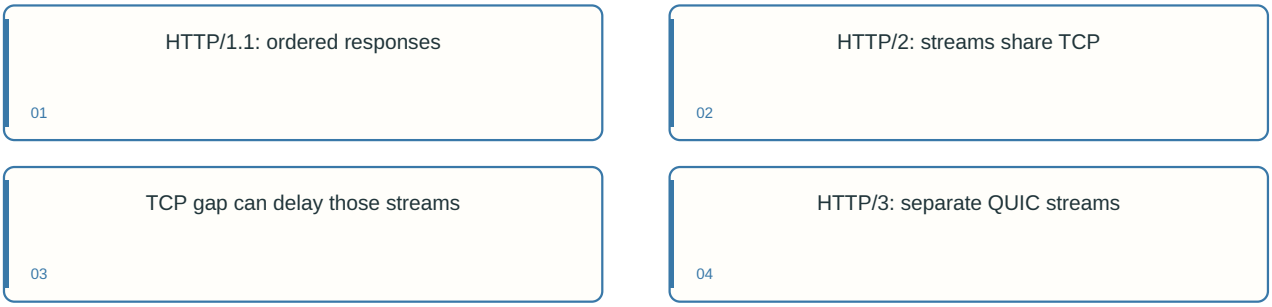


FIG. 3.2.3 **Carry several HTTP exchanges · the example**

HTTP/2 shares a TCP connection between streams; HTTP/3 uses QUIC streams. HTTP request and response meanings remain.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.2.4 · Use a connection again

A **connection** lets both ends keep information about an ongoing transport conversation. A new TCP connection normally begins with three messages, called a **handshake**.

First, the client asks to connect and announces its starting byte-sequence number: SYN. The server acknowledges that message and announces its own starting number: SYN-ACK. The client acknowledges the server: ACK. Both ends now have the starting information for sending and recognising stream data. For HTTPS over TCP, TLS setup adds the protection described earlier.

Once the task page has arrived, the browser can use the same usable connection again. A request needing fresh setup is often called **cold**; one reusing setup is called **warm**. An idle connection can be closed, so reuse lasts only while the path remains usable. Replacing a failed connection does not tell us whether its last request already ran.

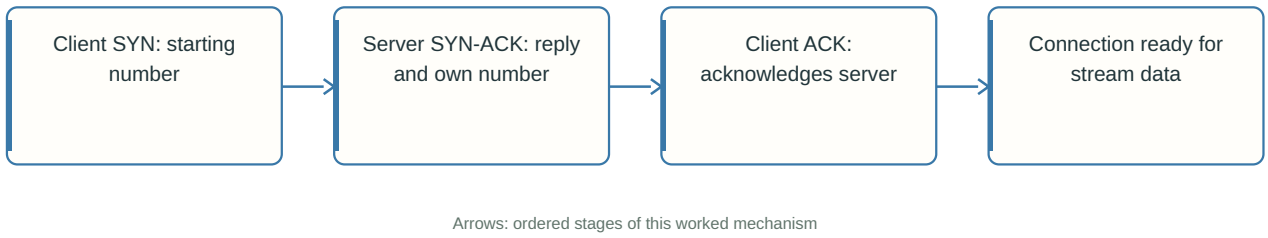


FIG. 3.2.4 **Three messages prepare the TCP connection**

Each end announces its starting sequence number and acknowledges the other. HTTPS over TCP then needs its TLS protection setup.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.2.5 · Add up the waiting time

A **round trip** includes travel to another participant and back. If one exchange must finish before the next starts, their waiting times add.

Assume this simplified cold request needs 20 ms for DNS, 40 ms for TCP setup, 40 ms for TLS setup, and 55 ms for the request, server work, and reply. The elapsed totals are 20 ms, 60 ms, 100 ms, and finally `20 + 40 + 40 + 55 = 155 ms`.

If the next request reuses the lookup and secure connection, the assumed setup work disappears. The same request-and-work stage remains: 55 ms. Its travel is already included, so we do not add another round trip to it.

These are teaching inputs, not measured timings or fixed handshake costs. The calculation shows why a short server calculation can still be part of a much longer wait.

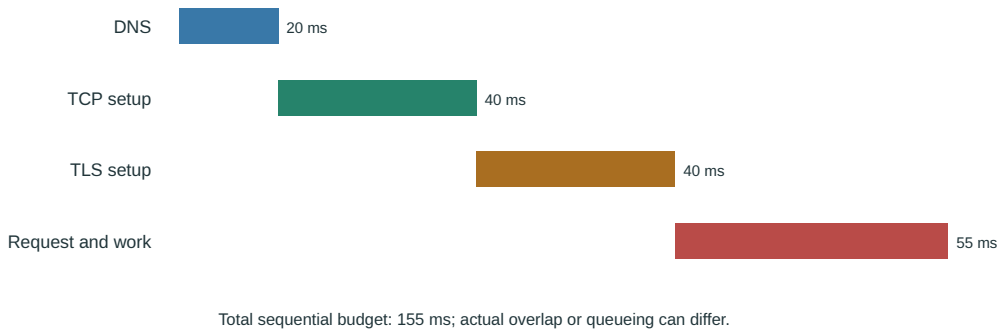


FIG. 3.2.5 **A cold request spends time before server work**

A sequential illustrative cold path totals 155 ms. Reuse, protocol, overlap, and actual distance change the result.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Remember: A request can spend time finding the site and setting up communication before the server handles it. Reusing completed setup can shorten a later request.

Try applying it: Which parts of our 155 ms example are saved when the next request uses the same secure connection?

Unit 3 exercises → · Unit 3 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. RFC 9000: QUIC Transport, 2021 ↗

   Original specification. QUIC streams and transport behavior; application walkthrough is author-created.

   Access reviewed · 21 October 2026

3. RFC 8446: TLS 1.3, 2018 ↗

   Original specification. Secure transport handshake and protocol scope.

   Access reviewed · 21 October 2026

Intermediaries & persistent connections

Prerequisites: 3.2 · A secure request, one round trip at a time 4 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

Our application has become busy, so we run three copies. A request now needs to reach a suitable copy. We also want new tasks to appear without repeatedly reloading the page.

We will follow the components in front of the application, then compare ways to send updates.

3.3.1 · Forward a request through a proxy

A **proxy** receives communication and forwards it on another party's behalf. The names forward and reverse tell us which side it serves.

A **forward proxy** serves clients. A browser sends its request to the proxy, which contacts the destination on the browser's behalf. An organisation might use this arrangement for staff internet access.

A **reverse proxy** fronts a service. Visitors contact it as the site's public server; it forwards requests to an application server behind it. Such a destination is often called a **backend**. The reverse proxy can handle TLS and select a destination by the requested path.

Forwarding information also needs a trust rule. If the application accepts a “user 7” header, a public caller must not be able to invent that identity. A trusted proxy must verify the identity and remove or replace untrusted versions of that header. The application must also control who can reach it directly.

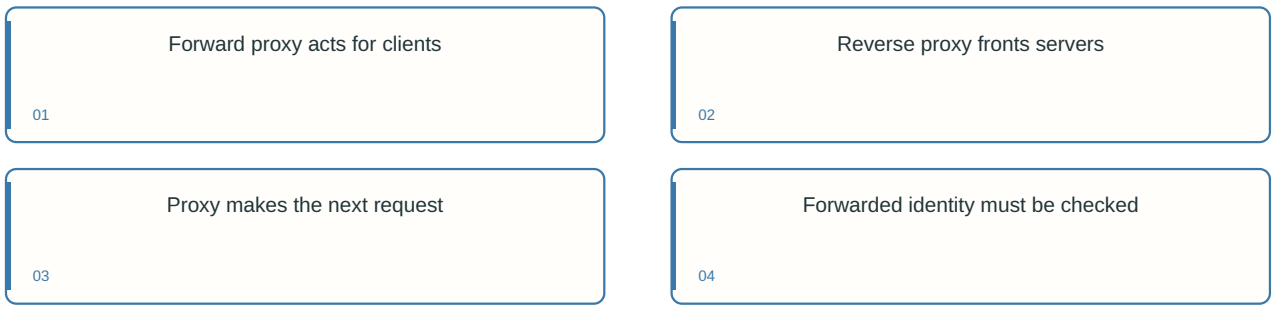


FIG. 3.3.1 **Forward a request through a proxy · the example**

A forward proxy acts for clients. A reverse proxy receives requests in front of application servers.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.3.2 · Share traffic and check common rules

Suppose three copies of our task application can answer requests. A **load balancer** chooses which copy receives traffic. It might send successive requests to A, B, and C so that one copy does not receive all the work.

An **API** is an interface through which programs use application operations. An **API gateway** can perform common checks before requests reach those operations. For example, it may check a sign-in credential or enforce a **quota**, an allowance such as a maximum number of requests.

Follow a request for task 42. The gateway checks who is calling. The balancer selects application B. B then checks whether that user owns task 42 before returning it. These are separate decisions, although one product may perform both gateway and balancing work. All three application copies may still depend on the same database.

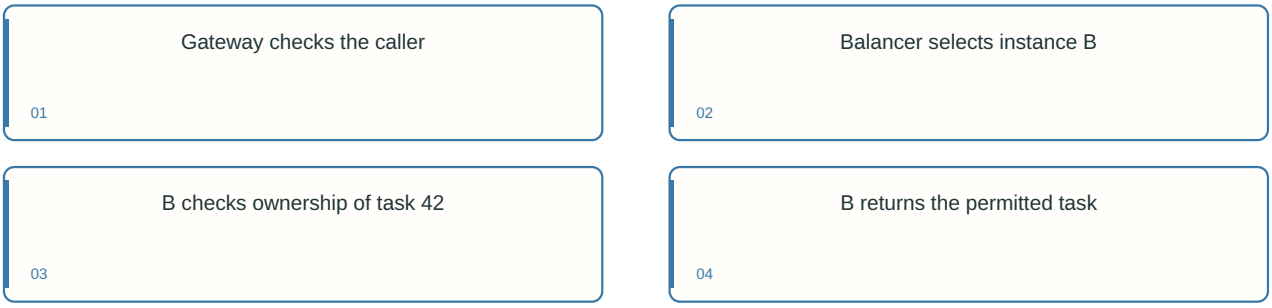


FIG. 3.3.2 **Share traffic and check common rules · the example**

A load balancer chooses an application instance. An API gateway can perform common API checks before application work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.3.3 · Route by port or by path

A balancer can choose a destination using different parts of a request. The familiar labels **Layer 4 (L4)** and **Layer 7 (L7)** refer to the transport and application layers of a network model.

An L4 rule can use IP addresses and transport ports. It can select a server for a TCP connection without reading the HTTP path. Traffic on that connection generally continues to the selected server.

An L7 rule reads application messages. It can send `/tasks` to the task service and `/images` to an image service. To read an encrypted HTTP request, the routing component commonly ends the TLS connection and decrypts the message. The useful distinction is the information used for the choice: transport information or application content.

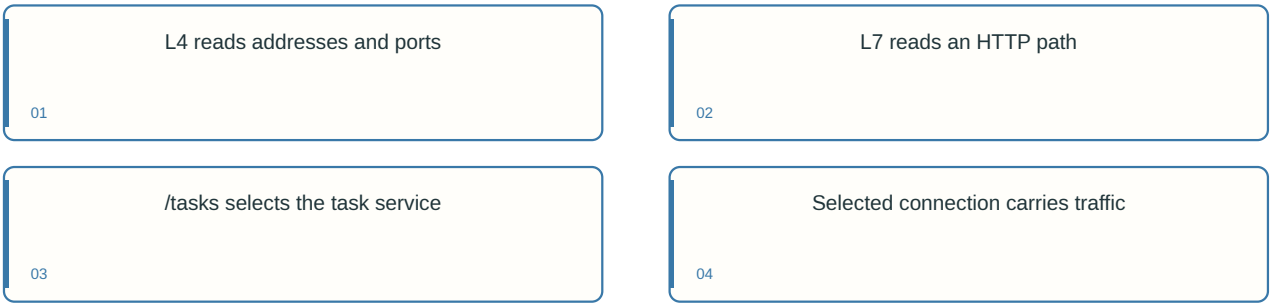


FIG. 3.3.3 **Route by port or by path · the example**

L4 routing uses transport addresses and ports. L7 routing can use HTTP content such as the requested path.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.3.4 · Send updates to the browser

Our task page needs to show new changes. With **polling**, the browser repeatedly asks whether anything has changed. If 1,000 clients ask every five seconds, the average rate is `1,000 / 5 = 200 requests per second`, even when there are no updates.

With **Server-Sent Events (SSE)**, one HTTP response stays open and the server sends a sequence of events through it. With **WebSockets**, either end can send messages over the established channel. Both can avoid repeated empty polls, while keeping connections and buffers in use. A buffer is memory holding data while it waits to be handled or sent.

Suppose a client receives event 20, disconnects, and misses events 21–23. Reopening a connection does not recover them by itself. The application must send retained events after 20, or send a fresh task list. Small **heartbeat** messages can help discover a broken idle path; they do not replace that missed-update recovery.

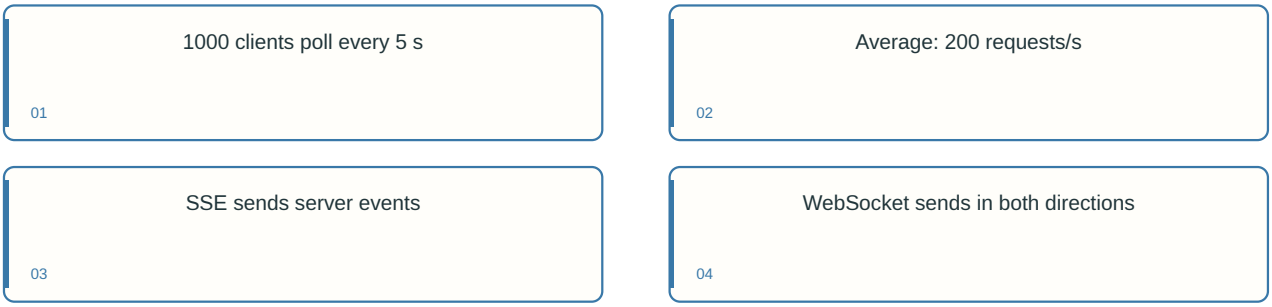


FIG. 3.3.4 **Send updates to the browser · the example**

Polling repeatedly asks for updates. SSE sends server events; WebSockets allows both ends to send. Missed updates still need recovery.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Remember: A proxy forwards traffic, a balancer chooses a server, and a gateway performs common API checks. Updates still need a recovery path when a client disconnects.

Try applying it: A browser last received event 20. Which information would let it recover after missing events 21–23?

Unit 3 exercises → · Unit 3 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. RFC 9000: QUIC Transport, 2021 ↗

   Original specification. QUIC streams and transport behavior; application walkthrough is author-created.

   Access reviewed · 21 October 2026

3. RFC 8446: TLS 1.3, 2018 ↗

   Original specification. Secure transport handshake and protocol scope.

   Access reviewed · 21 October 2026

Discover, route & survive the network

Prerequisites: 3.3 · Intermediaries & persistent connections 5 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

The browser asks to add a task. It waits, but no reply arrives. Is the task missing, or is only the reply missing?

We will follow the request and its return path, then see how addresses, cached copies, time limits, and reusable connections affect the result.

3.4.1 · Follow traffic out and back

A laptop may use private address `10.0.0.7` inside its local network. To reach an internet service, its traffic may pass through **network address translation**, or **NAT**, which changes address information.

For example, a NAT device can replace source `10.0.0.7:51000` with external source `203.0.113.5:62000`. It remembers the mapping. When a reply arrives for the external address and port, it uses that mapping to forward the reply to the laptop. These are illustrative addresses. If the mapping expires, an old connection may no longer have a working return path.

A **firewall** applies rules allowing or blocking traffic. NAT changes addressing; a firewall decides which traffic is permitted. One device can do both.

Ingress means traffic entering a boundary; **egress** means traffic leaving it. Our request is egress from the laptop's network and ingress to the server's network. The reply travels the other way. Checking both directions helps explain why a listening server may still receive no usable connection.

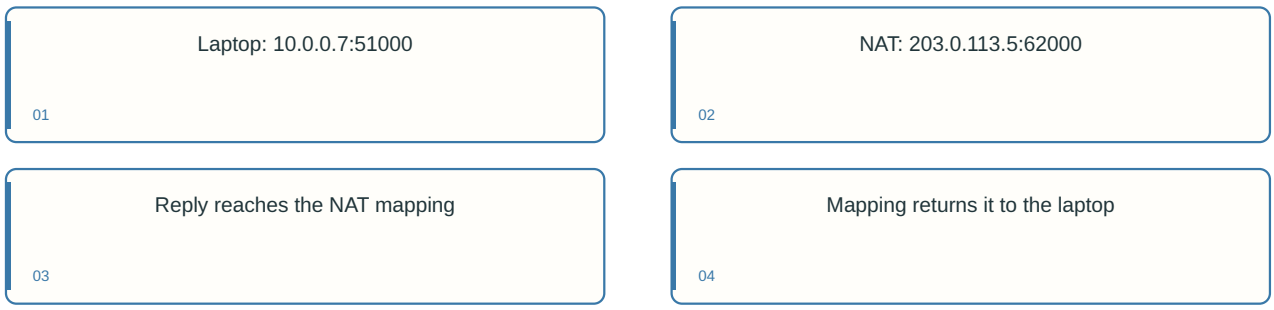


FIG. 3.4.1 **Follow traffic out and back · the example**

NAT translates addresses and remembers return mappings. A firewall allows or blocks traffic. Ingress enters a boundary; egress leaves it.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.4.2 · Find a service whose address can change

Our task application may need a file-conversion service. Its instances can be replaced, so their addresses can change. **Service discovery** finds the current destinations for a service name.

An instance can register its address and port in a **registry**, a list of service instances. The caller obtains the list, selects instance A, and connects. DNS or a deployment platform can also provide this information.

Now let A stop after the caller receives the list. The saved list still names A, so the call can fail. The caller needs to refresh the list and handle an unusable destination. Discovery finds a possible place to call; the operation's own rules decide whether a failed call can safely be retried.

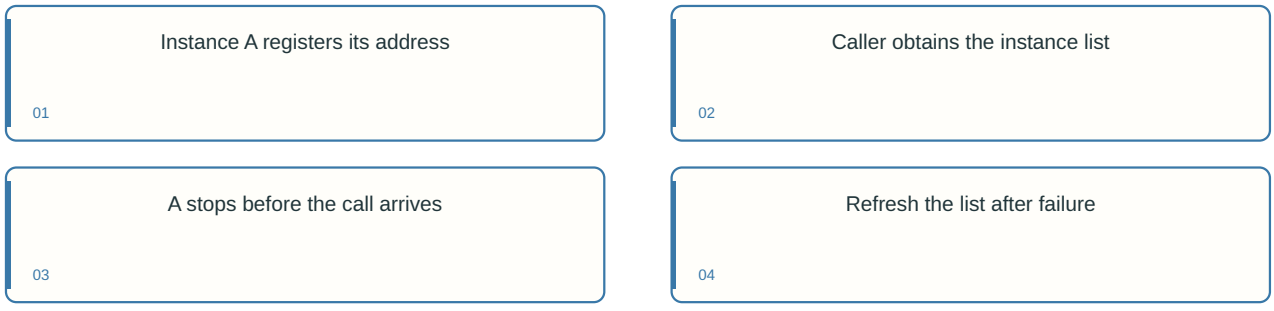


FIG. 3.4.2 **Find a service whose address can change · the example**

Service discovery supplies possible destinations. A destination can stop after the list is obtained, so callers must handle that change.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.4.3 · Reuse a nearby copy

The task page includes a public help image. Its original server is called the **origin**. A **content delivery network (CDN)** keeps permitted copies at serving locations near users. These locations are called **edges**.

The edge looks for the requested image in its cache. Finding a usable copy is a **hit**. Otherwise it is a **miss**: the edge fetches the image from the origin and can save a copy for later requests. A **cache key** identifies which saved copy to look up. Different image sizes need distinguishable keys.

Reuse reduces some travel and origin work. It also needs a freshness rule saying when the copy may be used. A public help image can be shared widely; a private task page cannot be shared just because its URL looks similar. Permission and content differences must be preserved when deciding which requests may share a copy.

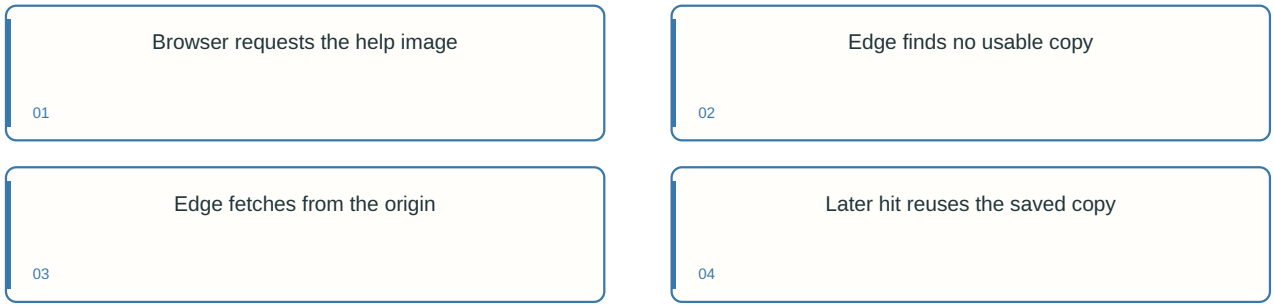


FIG. 3.4.3 **Reuse a nearby copy · the example**

A CDN can reuse a permitted nearby copy. Its cache key, freshness rule, and access rules determine whether reuse is valid.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.4.4 · What does a timeout tell us?

Our client asks to add “Read” and waits one second. No reply arrives. Reaching the waiting limit is a **timeout**. It tells us the reply was not received in time.

Consider two sequences. In the first, the request never reaches the server. In the second, the server saves task 7 but its reply is lost. The client sees the same timeout in both. Repeating the request as a new operation could add “Read” twice.

Give the first request an operation identifier, `op-9`. The server saves the task and records “op-9 created task 7” in the same database transaction. If the client resends `op-9` with the same request, the server

returns that recorded result. The operation identifier recognises the request; the task identifier names the created task. A status lookup can also reveal the result.

Set an overall **deadline**, the latest time the caller will wait, so retries do not restart the full allowance. **Packet loss** means a packet did not reach its destination; a transport may recover it. A **network partition** prevents some participants from communicating even though they may still be running. A timeout alone cannot distinguish these causes from a stopped or slow server.

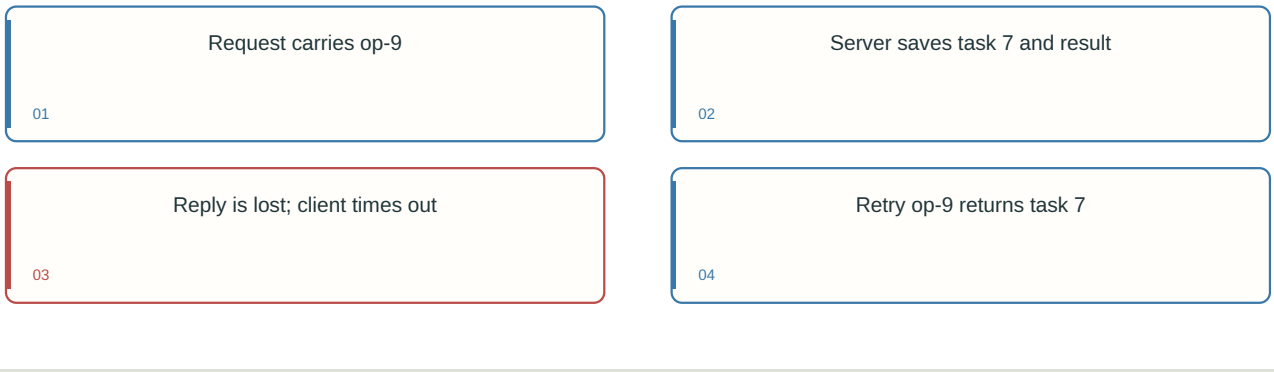


FIG. 3.4.4 **What does a timeout tell us · the example**

A timeout leaves the remote outcome uncertain. A recorded operation identity can let the client find or retry the original operation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

3.4.5 · Borrow and return connections

Opening a database connection has a setup cost. A **connection pool** keeps connections ready for reuse. A request borrows one, performs its database work, and returns it so another request can use it. If every connection is borrowed, the next request waits.

Assume twenty connections, each held for an average of 50 ms, or 0.05 seconds. One continuously busy connection can complete about `1 / 0.05 = 20` borrowing periods per second. Twenty give an ideal rate of `20 × 20 = 400 operations per second`, assuming one operation per period and no other limit.

This calculation uses the assumed hold time. Keeping a connection longer lowers the rate. Failing to return one removes a slot from the pool, even when the database is healthy. Returning borrowed connections on success and error therefore matters as much as creating them.

Keepalive probes are small checks sent on an idle connection to help discover that it has broken. Reusing an HTTP connection, also sometimes called keep-alive, is a different use of the term. Neither tells us whether a particular application operation completed.

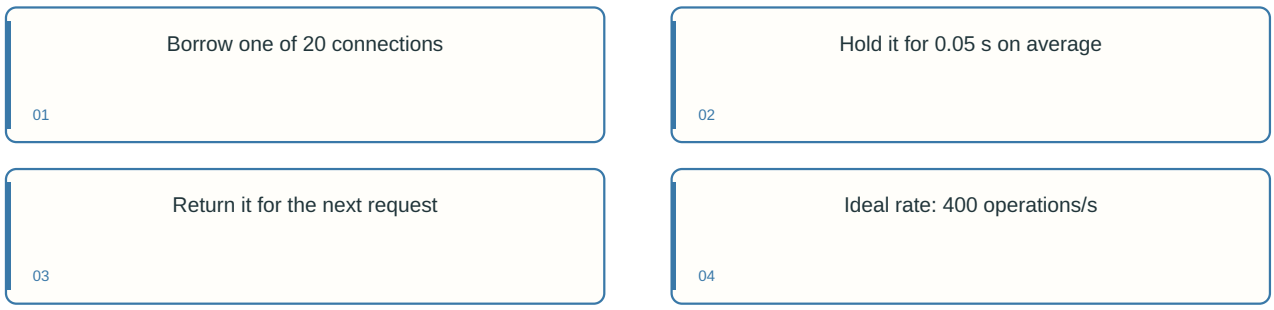


FIG. 3.4.5 **Borrow and return connections · the example**

A connection pool lends reusable connections. Borrowers wait when it is full and must return connections when their use ends.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

The following Python code adds the four times in our example. The dictionary stores a name and duration for each stage. On the warm path, the earlier setup is reused, so only the request-and-work stage remains.

PYTHON · ARITHMETIC MODEL

```
cold = dict(dns=20, tcp=40, tls=40, request_and_work=55)
assert sum(cold.values()) == 155
warm = dict(request_and_work=55)
assert sum(warm.values()) == 55
```

The first assertion checks 155 ms. The second checks 55 ms. Both use assumed durations; the code performs arithmetic rather than measuring a network connection.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · Our application used to call a local function. Now it calls a service in another region. What new events can affect the result?

The request must travel to another machine and the reply must travel back. Setup and travel add time. The server can finish while the reply is lost, leaving the caller uncertain. The interface now

needs a waiting limit and a way to recognise or find the original operation when repeating it could cause another effect.

Remember: A timeout means the reply did not arrive in time. Recover the original operation instead of assuming that the server did nothing.

Try applying it: After "add task 7" times out, what could the client ask that would avoid creating another task?

Unit 3 exercises → · Unit 3 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   `Access reviewed · 21 October 2026`

2. RFC 9000: QUIC Transport, 2021 ↗

   Original specification. QUIC streams and transport behavior; application walkthrough is author-created.

   `Access reviewed · 21 October 2026`

3. RFC 8446: TLS 1.3, 2018 ↗

   Original specification. Secure transport handshake and protocol scope.

   `Access reviewed · 21 October 2026`

PART 1 / UNIT 04

Runtime, Processes, Threads, and Concurrency

Our task application can now receive files. One upload waits for storage while another needs processing. What is running, what is waiting, and where do their bytes stay?

The running example

Build a service that processes 1,000 file uploads. Compare sequential execution, threads, asynchronous I/O, and worker queues. Identify which architecture helps under which workload.

Unit 4 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

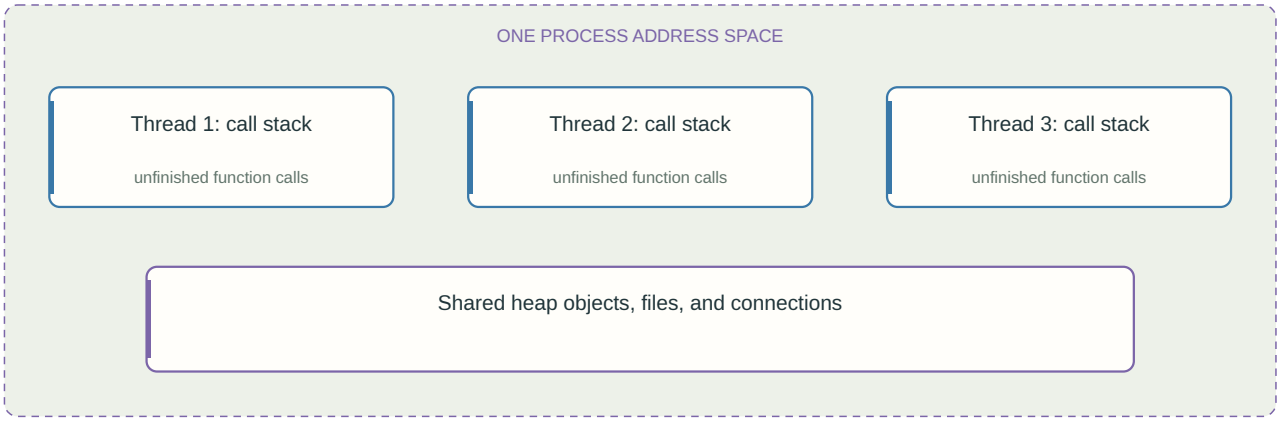


FIG. 4.0.1 **Each thread has a call stack; objects can be shared**

This common layout shows three threads in one process. Each has a call stack, while heap objects and process resources can be shared. Actual storage details depend on the runtime.

Author-created illustration. Numbers are illustrative unless stated otherwise.

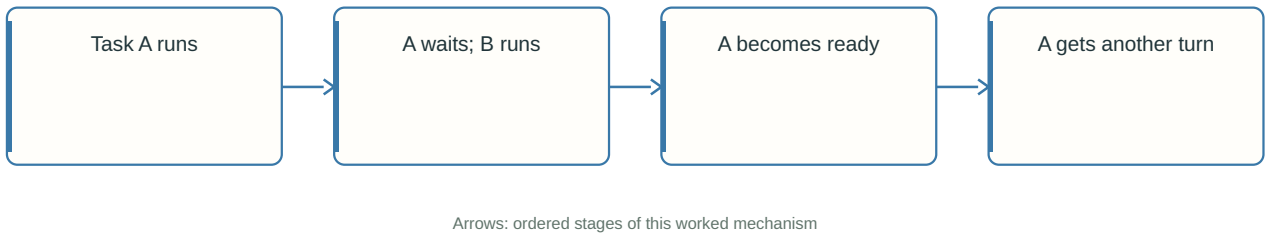
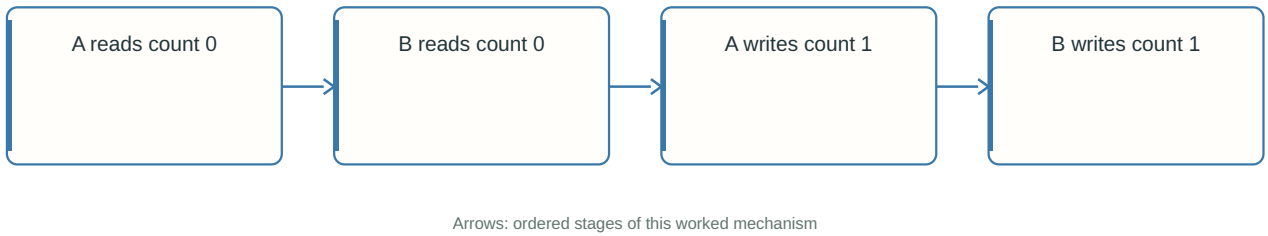


FIG. 4.0.2 **scheduling timeline**

A and B overlap in progress on one core. Simultaneous execution requires parallel resources.

Author-created illustration. Numbers are illustrative unless stated otherwise.

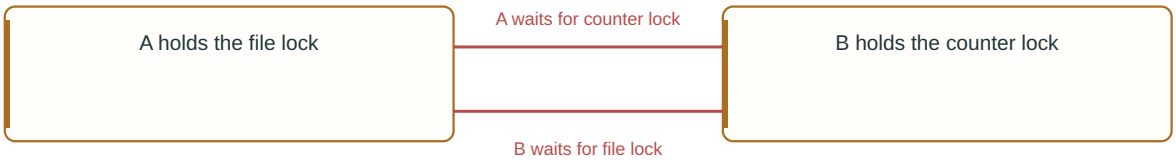


Arrows: ordered stages of this worked mechanism

FIG. 4.0.3 **race-condition interleaving**

Both callers used the same old value. A common lock must cover reading, adding, and writing.

Author-created illustration. Numbers are illustrative unless stated otherwise.

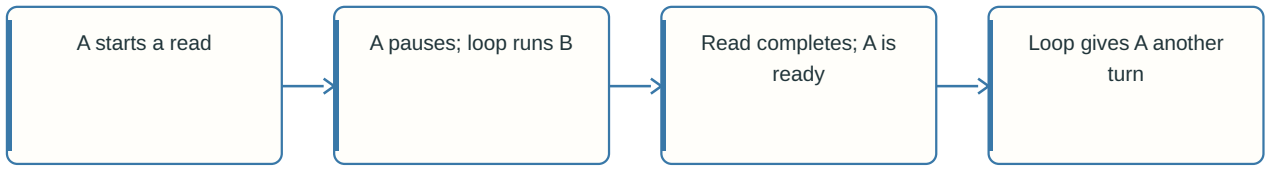


Both workers keep a lock while waiting for the other; neither can finish.

FIG. 4.0.4 **Each worker waits for a lock held by the other**

A holds the file lock and waits for the counter lock. B holds the counter lock and waits for the file lock. Requiring both workers to take the file lock first removes this cycle.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 4.0.5 **event-loop flow**

Completing the read makes A ready. A can continue only when the loop thread runs it.

Author-created illustration. Numbers are illustrative unless stated otherwise.

The runtime owns your resources

Prerequisites: Introduction to System Design 5 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

Our task application can now receive files. One upload waits for storage while another needs processing. What is running, what is waiting, and where do their bytes stay?

Let us follow the program, its threads, and the resources they use.

4.1.1 · Start a program and follow its threads

Our task application now accepts file uploads. The instructions saved on disk form a **program**. When we start it, the operating system creates a **process**, a running instance with memory and resources such as open files. The operating system is the software that manages these resources and access to the processor.

Start the same program twice. We now have two processes. Each has its own running state and its own view of memory, called an **address space**. Ordinary variables in one process are not automatically variables in the other.

Inside a process, a **thread** is a path of execution: a sequence of instructions that can be run. One thread might receive an upload while another works on an earlier file. Threads in the same process can share data. This helps them cooperate, but also lets them interfere with each other's changes.

A source-code module is a way to organise instructions. It need not be a separate process. Several modules can run in one process and stop together if that process crashes.

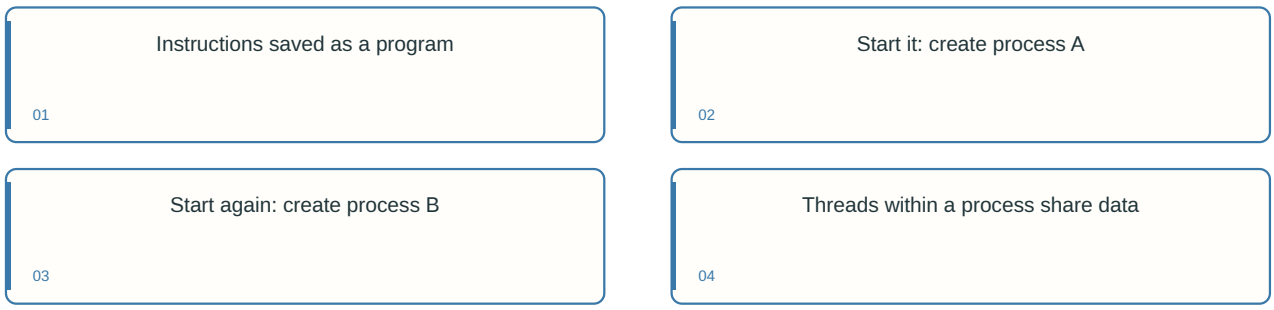


FIG. 4.1.1 **Start a program and follow its threads · the example**

A program is saved instructions. A process runs them. Threads within a process execute work and can share data.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.1.2 · Keep data alive while it is needed

An upload arrives in a **buffer**, an area of memory holding its bytes. A **reference** lets the program refer to that buffer without making another copy of all its contents.

The **call stack** records unfinished function calls and where execution should return. When `handle_upload` calls `read_file`, a new call record is added. When `read_file` returns, its call record is removed. A record can hold local values and references to objects.

Dynamically created objects are commonly kept in an area called the **heap**. Suppose `handle_upload` gives its buffer to background work and then returns. Its call has ended, but the background work still holds a reference. The buffer must remain available until that work stops using it.

Languages manage this storage differently. A **runtime** is the supporting code that carries out language operations. In a runtime with **garbage collection**, memory can be reclaimed when the program can no longer reach an object through the references it keeps. A background task's reference therefore keeps our buffer available, even after the handler returns. That does not promise prompt closure of a file or database connection. Those resources need a clear close or return step when their use ends.

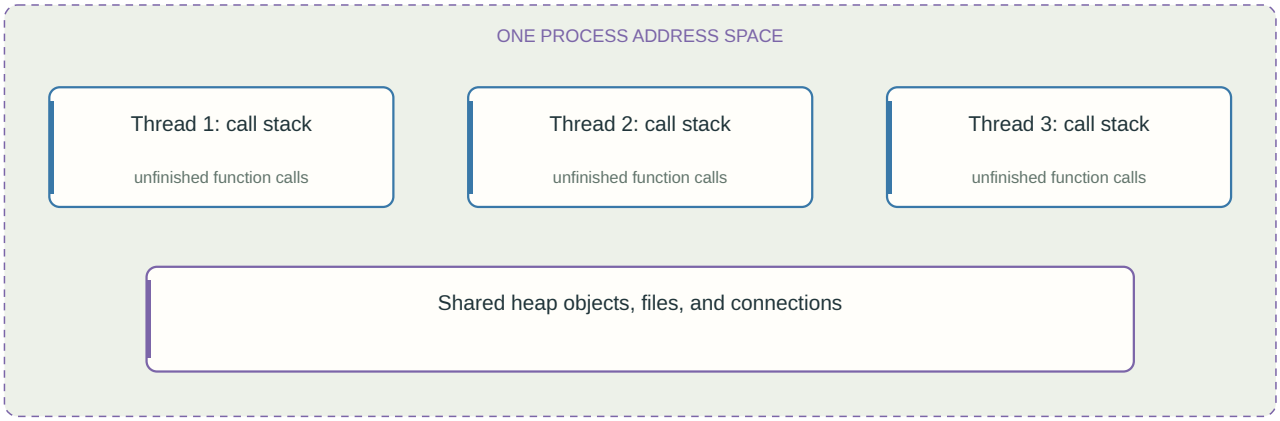


FIG. 4.1.2 **Each thread has a call stack; objects can be shared**

This common layout shows three threads in one process. Each has a call stack, while heap objects and process resources can be shared. Actual storage details depend on the runtime.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.1.3 · Let threads take turns

The **CPU**, or processor, carries out program instructions. A **core** is an execution unit within it. Assume our available core runs one thread at a time. If two threads are ready, they take turns. The operating system's **scheduler** chooses which ready thread runs next.

Suppose A begins reading an upload from storage and must wait. The scheduler can run B during that wait. When the read is ready and A is selected again, A continues from where it paused.

Changing the running thread requires saving enough information to resume it and restoring the next thread's state. This is a **context switch**. Switching itself takes work. If both A and B need continuous calculation, taking turns does not give them a second core. More waiting threads may help use idle time; more ready threads competing for a busy core also add waiting and switching.

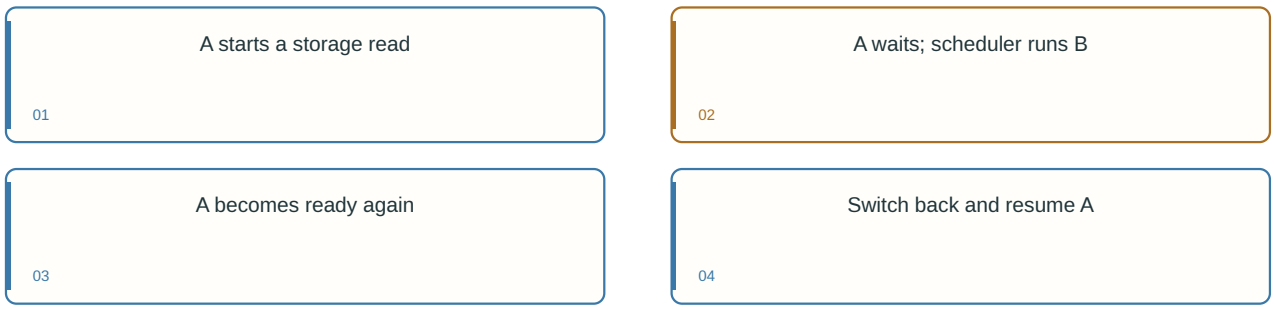


FIG. 4.1.3 **Let threads take turns · the example**

The scheduler chooses a ready thread. A context switch saves one thread's execution state and restores another's.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.1.4 · Find out whether work calculates or waits

An upload can spend time calculating or waiting for data. **Input/output (I/O)** moves data between the program and something such as a network connection or storage. A workload is **I/O-bound** when these operations are its main limit. It is **CPU-bound** when calculation is the main limit.

Assume 1,000 independent uploads each wait 100 ms for storage and need negligible calculation. Handle them one at a time: `1,000 × 100 ms = 100 seconds`. Allow fifty waits at once: there are twenty ideal groups, taking `20 × 100 ms = 2 seconds`.

The second calculation assumes all fifty waits overlap without making storage slower. Actual bandwidth, overhead, and storage capacity can increase the time.

Now let each file need compression instead. A core must execute the compression instructions. Overlapping more waiting operations does not supply more computing power. First ask where the time goes: calculation, I/O, or both. That tells us which resource needs attention.

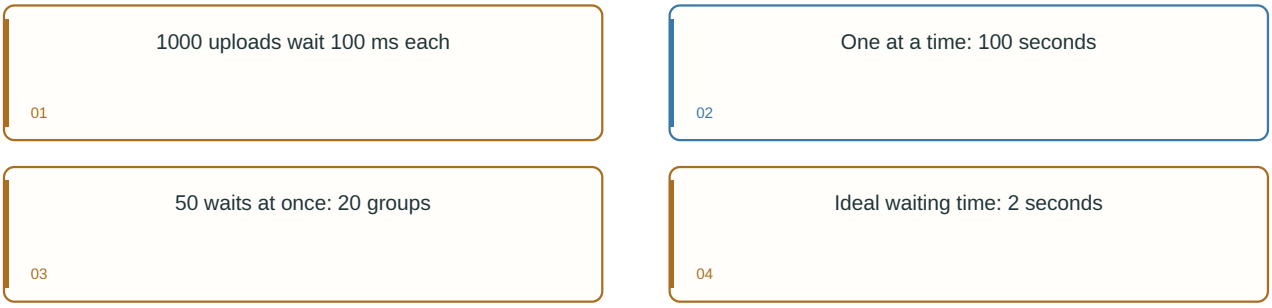


FIG. 4.1.4 **Find out whether work calculates or waits · the example**

I/O-bound work mainly waits for input or output. CPU-bound work mainly needs instructions to be executed.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.1.5 · Start now and receive a result later

Suppose the caller asks our service to convert a file. With a **synchronous** interface, the call returns the result or an error before the caller continues past it.

With an **asynchronous** interface, starting the operation and receiving its result are separate. The caller can receive a way to obtain the later result, do other work, and wait for completion when it needs that result.

This describes how completion is presented to the caller. A separate question is whether a thread is held waiting underneath. An asynchronous operation can still use a library call that blocks its thread. To understand the behaviour, we must follow what the actual call does while it waits.

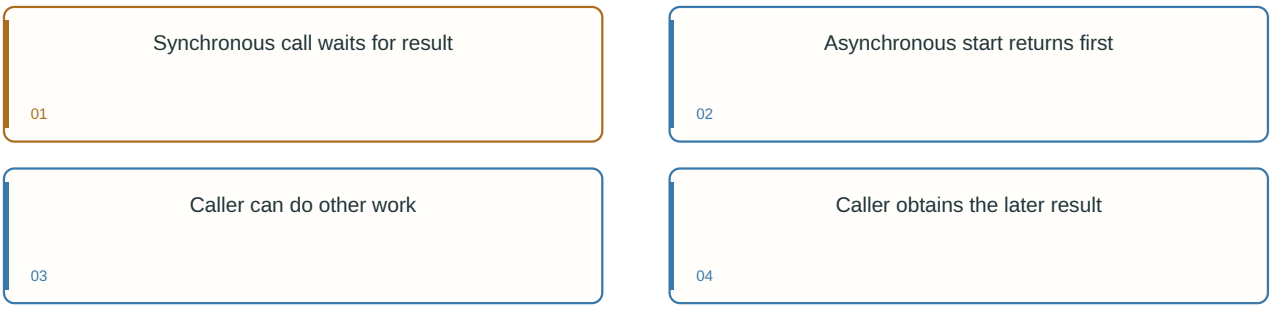


FIG. 4.1.5 **Start now and receive a result later · the example**

A synchronous interface returns completion before the caller proceeds; an asynchronous interface separates starting from receiving the result.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Remember: A process is a running program; its threads execute work and can share data. Follow how long data and other resources remain in use, not just how long a function call lasts.

Try applying it: An upload handler returns while another task still uses its buffer. Why must the buffer remain available?

Unit 4 exercises → · Unit 4 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Operating Systems: Three Easy Pieces ↗

   University textbook. Runtime, concurrency, and persistence.

   `Access reviewed · 21 October 2026`

2. The Go Memory Model ↗

   Language specification · accessed 21 Oct 2026. One concrete memory-model contract; other languages differ.

   `Access reviewed · 21 October 2026`

Tasks, threads & event loops

Prerequisites: 4.1 · The runtime owns your resources 4 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

A file read is unfinished. Must the thread remain idle until the bytes arrive?

We will follow two uploads to see how work can pause, let other work run, and resume. Then we will give waiting jobs a limited number of workers.

4.2.1 · Wait without holding the thread

A thread calls `read` to obtain the next upload bytes. If no bytes are available, a **blocking** read keeps that thread waiting inside the call until data or another ending condition is available. The scheduler can run another thread meanwhile.

A **nonblocking** read can return “not ready” instead. The program arranges for a notification when reading may make progress. The thread can then do other work rather than remain in that read. When notified, the program tries the read again.

The next read may supply only part of the file. Reading a few bytes and finishing the entire upload are different events. Some platforms instead report completion of an operation submitted earlier. In either arrangement, repeatedly checking without waiting for a useful notification can waste CPU time. Nonblocking I/O helps because unfinished waits leave the thread available for other work.

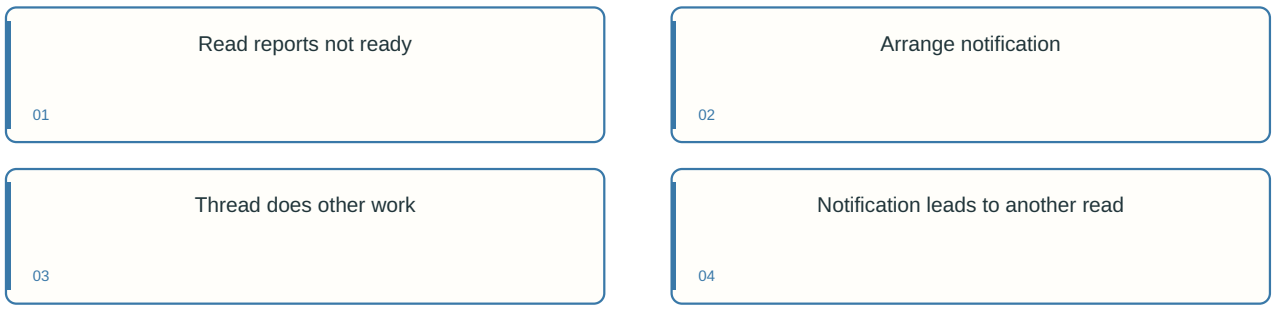


FIG. 4.2.1 **Wait without holding the thread · the example**

A blocking read holds the calling thread waiting. Nonblocking I/O lets that thread do other work while data is unavailable.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.2.2 · Distinguish overlap from simultaneous work

Upload A starts and waits for storage. While A waits, one core runs B. Later it resumes A. Both uploads are in progress during overlapping periods. This is **concurrency**.

Now let two cores execute work on A and B at the same instant. This is **parallelism**. Concurrency is about overlapping progress; parallelism is about simultaneous execution. One core can provide the first without providing the second.

Taking turns can still cause interference. A might read a shared completed-upload count and pause before writing it. B can change the count before A resumes. We will follow that sequence in Chapter 4.3. Shared data needs coordination even when the threads do not execute at the same instant.

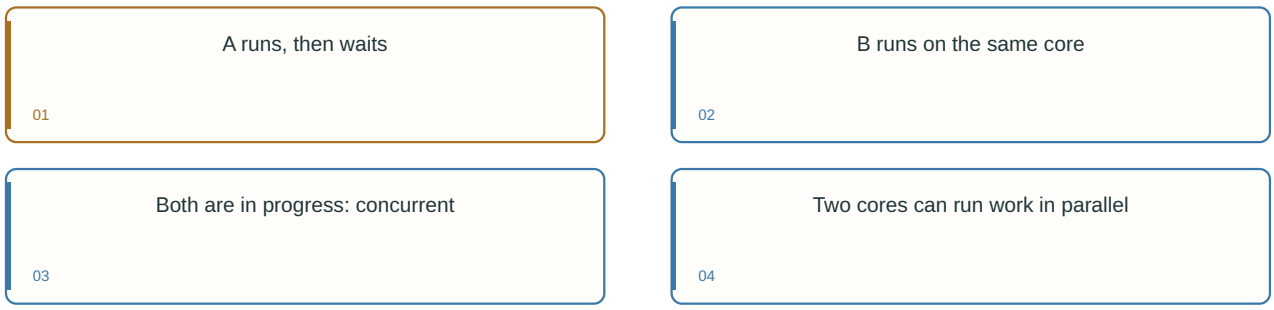


FIG. 4.2.2 **Distinguish overlap from simultaneous work · the example**

Concurrent work overlaps in progress. Parallel work executes simultaneously. Taking turns can still interfere with shared updates.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.2.3 · Follow a task through an event loop

A thread carries out instructions. A **task** represents a piece of work that can pause and resume. A **future** represents a result that may not yet be available; it later holds the result or an error. Languages also use the name **promise** for objects representing later results; their exact interfaces differ.

An **event loop** runs ready pieces of work in turn, often on one thread. A **callback** is a function arranged to run when an event occurs. The loop runs a callback or part of a task, then moves to other ready work.

Follow two uploads. Task A starts a read and reaches `await`, which waits for its result. If the read is unfinished, A pauses. The loop runs B. When A's read completes, its future receives the result and A becomes ready to continue. The code that resumes A is its **continuation**.

Ready does not mean running. If B keeps calculating for two seconds without yielding the loop thread, A waits for its turn even though its bytes have arrived. An `await` on an already available result may not pause at all. What matters is when the running work actually gives the loop a chance to run something else.

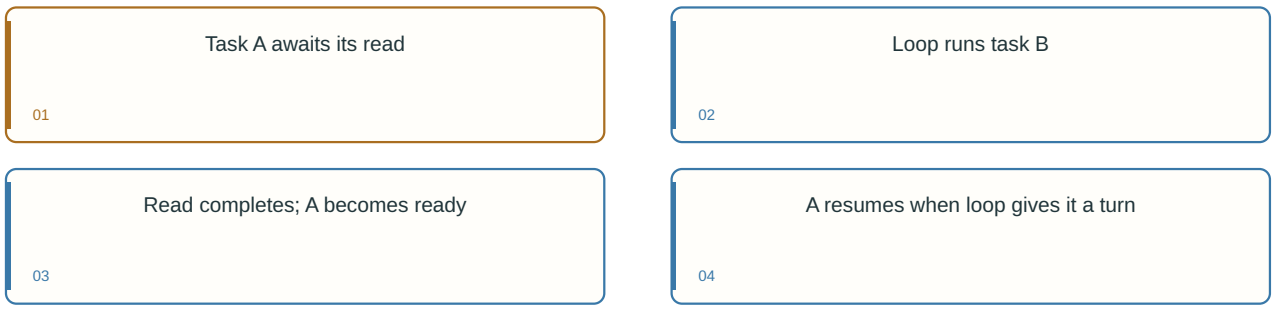


FIG. 4.2.3 **Follow a task through an event loop · the example**

A task can pause on an unfinished future. Completion makes it ready; the event loop must still give it a turn.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.2.4 · Give workers a bounded waiting queue

Let a **job** mean “convert this uploaded file.” A **worker** is a thread or process that takes a job, does the work, and takes another. A **worker pool** is a fixed set of these workers. A thread pool uses threads for that role.

Suppose eight workers are busy. New jobs enter a **queue**, a collection waiting to be taken. Eight workers limit active jobs to eight; they do not limit the number waiting. Give the queue its own limit too.

Keep the file bytes in storage and place small file references in the queue. A waiting job then need not keep the whole upload in memory. In our example, a full queue causes the service to reply “busy” instead of accepting another job.

This queue is held in process memory. A restart removes its entries. If “accepted” must mean that processing will survive a restart, save a job record as well as the file, so a replacement worker can find the unfinished work.

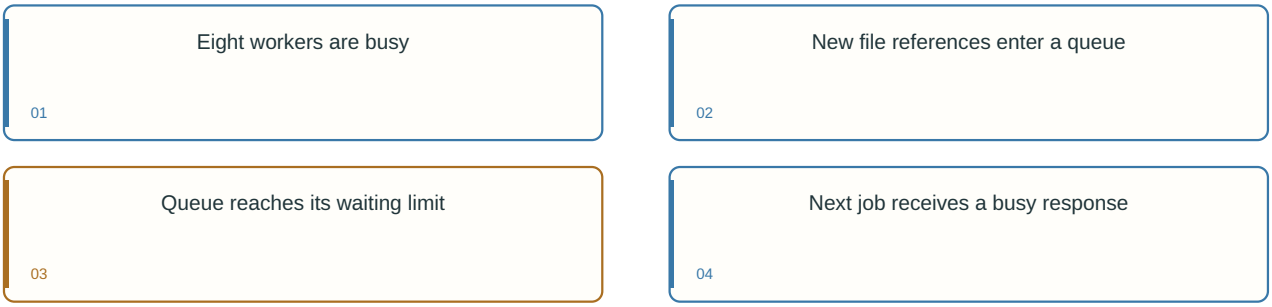


FIG. 4.2.4 **Give workers a bounded waiting queue · the example**

A pool limits active workers. A separate queue limit controls waiting jobs. Restart recovery also needs saved job records.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Remember: Work can overlap while one thread takes turns. A ready result still needs execution time, and a worker limit needs a separate limit on waiting jobs.

Try applying it: A read completes while the event-loop thread is calculating. When can the waiting task continue?

Unit 4 exercises → · Unit 4 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Operating Systems: Three Easy Pieces ↗

   University textbook. Runtime, concurrency, and persistence.

   Access reviewed · 21 October 2026

2. The Go Memory Model ↗

   Language specification · accessed 21 Oct 2026. One concrete memory-model contract; other languages differ.

   Access reviewed · 21 October 2026

Shared memory & coordination

Prerequisites: 4.2 · Tasks, threads & event loops 4 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

Two uploads finish, yet the completed-file count increases by only one. Let us follow the reads and writes to find the missing increment.

We will then use locks and other coordination tools, and see how waiting for those tools can itself prevent progress.

4.3.1 · Share a value or send a message

Two workers finish uploads and need to increase the same completed-file count. With **shared memory**, both can read and change that count directly. They need a common rule for taking turns during an update.

With **message passing**, one owner keeps the count. Worker A sends “one file finished.” The owner takes that message, adds one, and can reply with the new count. It then handles B's message. The workers request changes instead of changing the owner's count themselves.

This arrangement helps only if the owner really controls the state. Passing a reference to a mutable object does not make it private when other callers can still change it. Across a failure, a message may also be repeated or its reply lost. Message passing changes where coordination happens; the chosen delivery and recovery rules still matter.

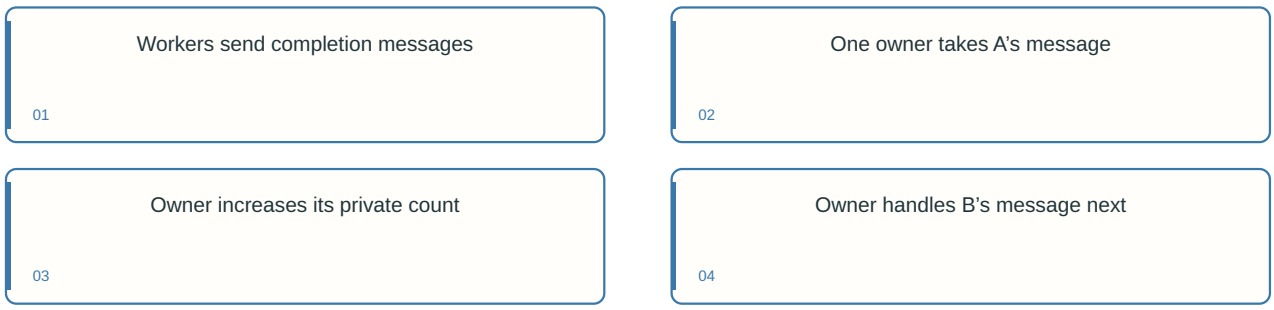


FIG. 4.3.1 **Share a value or send a message · the example**

Shared memory lets callers change the same value. Message passing can give one owner control of that value.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.3.2 · Follow a lost counter update

Start the completed-file count at zero. A reads zero. Before A writes, B also reads zero. A adds one and writes one. B adds one to its earlier value and also writes one. Two completed files have produced a count of one.

This is a **lost update**. The result depends on how the operations take turns, so it is a **race condition**. The error begins with the shared old read, before either final write.

A **critical section** is the part that must be protected from interfering access. Here it includes reading, adding, and writing. With the same suitable lock around all three steps, A changes zero to one; B then reads one and changes it to two. Locking only the assignments leaves both earlier reads unprotected. Every caller changing this protected count must follow the same rule. This is an illustrative interleaving; language memory rules also govern which shared accesses are allowed.

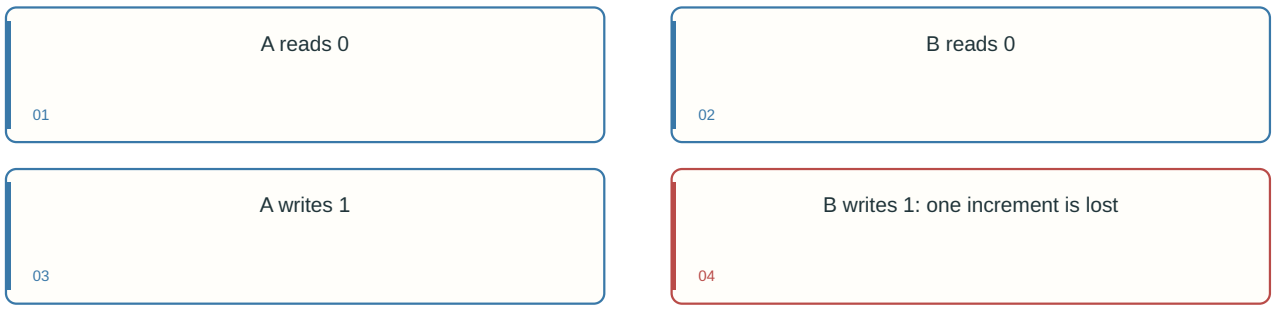


FIG. 4.3.2 **Follow a lost counter update · the example**

Both callers read zero before either writes. Protecting the whole read–modify–write sequence lets two increments produce two.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.3.3 · Choose the right coordination tool

A **mutex** is a lock held by one caller at a time. A acquires the mutex, updates the count, and releases it. B waits to acquire that same mutex before entering the protected section. Separate private locks would not make A wait for B.

A **semaphore** manages a number of permits. With four permits, the first four uploads can enter; a fifth waits. When one upload returns a permit, another may proceed. Four permits limit resource use but do not give exclusive access to a shared counter.

An **atomic operation** is indivisible for its specified action. An atomic increment combines reading, adding, and writing as one operation for that counter.

Separate atomic actions do not automatically form one larger action. If transferring ten changes balances from `100,100` through `90,100` to `90,110`, another observer can encounter the intermediate pair. One repair uses the same lock for both changes and for readers of the pair, so a reader cannot enter halfway through the transfer.

A **condition variable** lets a worker sleep until shared state may have changed. The worker locks the queue, finds it empty, and waits. Waiting releases the mutex. A producer can then acquire it, add a job, notify a waiter, and release it.

The worker reacquires the mutex and checks the queue again before taking a job. It checks in a loop because another worker may have taken the job first, and some interfaces permit wake-ups without a relevant state change.

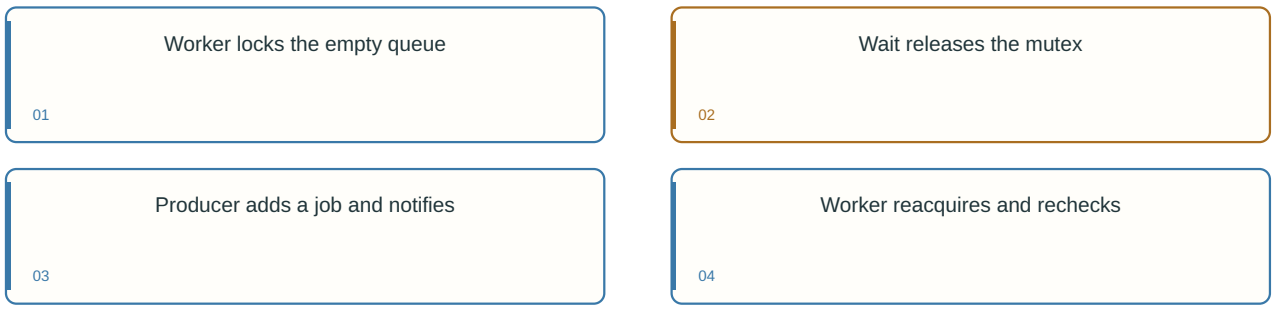


FIG. 4.3.3 **Choose the right coordination tool · the example**

A mutex gives exclusive entry; a semaphore counts permits; an atomic operation protects one action; a condition variable lets a waiter recheck shared state.

Author-created illustration. Numbers are illustrative unless stated otherwise.

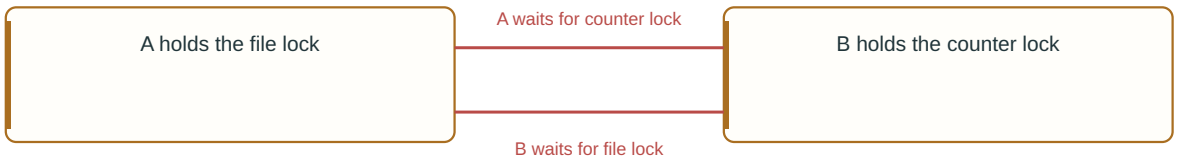
4.3.4 · When work stops making progress

A holds the file lock and waits for the counter lock. B holds the counter lock and waits for the file lock. Each waits for the other to release something, so neither can finish. This is **deadlock**.

Require every caller to acquire the file lock before the counter lock. B can no longer hold the counter lock while waiting for the file lock. This removes the illustrated circular wait. The order must apply to every path that takes both locks.

In **starvation**, one caller keeps missing out while others make progress. For example, A repeatedly obtains a lock before waiting B gets a turn. B can remain stuck even though the system keeps completing A's work.

In **livelock**, callers keep acting but finish nothing. Suppose A and B each take a different lock. Both fail to obtain the second, release the first, and retry in exactly the same pattern. They are active, unlike deadlock, but remain unable to complete. A retry needs to change the failed coordination, rather than repeat it forever.



Both workers keep a lock while waiting for the other; neither can finish.

FIG. 4.3.4 **Each worker waits for a lock held by the other**

A holds the file lock and waits for the counter lock. B holds the counter lock and waits for the file lock. Requiring both workers to take the file lock first removes this cycle.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Remember: Protect the whole update with a common coordination rule. Then examine the waiting sequence too: a correct lock must also be used in an order that permits progress.

Try applying it: Both callers lock only the final counter assignment. Why can the completed-file count still be wrong?

Unit 4 exercises → · Unit 4 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Operating Systems: Three Easy Pieces ↗

   University textbook. Runtime, concurrency, and persistence.

   Access reviewed · 21 October 2026

2. The Go Memory Model ↗

   Language specification · accessed 21 Oct 2026. One concrete memory-model contract; other languages differ.

   Access reviewed · 21 October 2026

Visibility, cancellation & overload

Prerequisites: 4.3 · Shared memory & coordination 4 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

A request ends, but its child work still holds a file and a large buffer. Another worker has data ready to share.

We will see how workers safely share values, limit waiting work, and release resources when an operation ends.

4.4.1 · Make a completed value visible to a reader

A creates an upload record, sets its title, and shares a reference with B. We want B to see the completed title. Merely knowing that A ran earlier does not establish that guarantee.

A language's **memory model** gives rules for which writes another thread is allowed or guaranteed to observe. A **compiler** translates program code into instructions. It and the processor can arrange execution and memory access in ways that an unsynchronised reader cannot safely infer from the source's line order. A log's timestamps do not add the missing synchronisation.

Consider a mutex with the language's required visibility guarantees. A acquires it, sets the title and shared reference, then releases it. B later acquires the same mutex and reads the record. The release-and-acquire relationship makes A's earlier protected writes visible under those rules. The Go memory model specifies such an ordering for mutex unlock and later lock operations.

Happens-before names the ordering relationship used to reason about effects like these. It connects program order with the language's synchronisation rules. The writer's initialisation then happens before the reader's use. This **safe publication** matters even when nobody will change the record afterward. Other languages require their own documented memory-model guarantees.

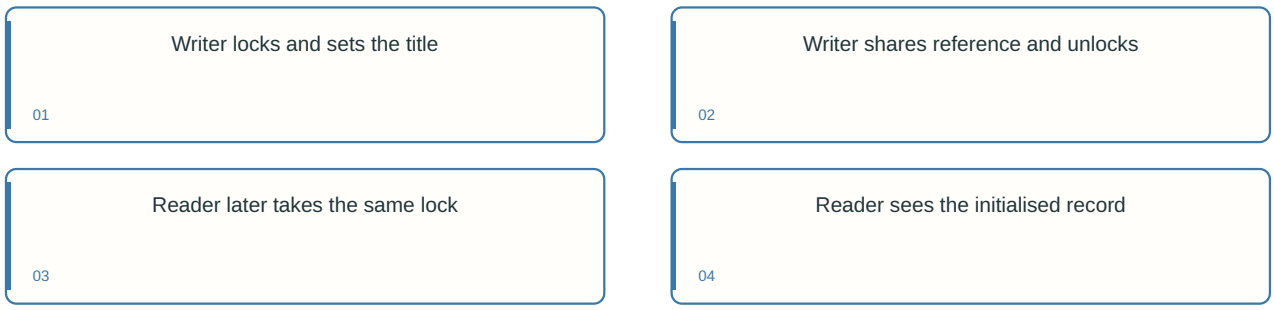


FIG. 4.4.1 **Make a completed value visible to a reader · the example**

Safe publication uses the language's synchronisation rules so the reader can observe the completed value.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.4.2 · Keep shared data safe to use

A task record that never changes after construction is **immutable**. Two workers can read that record without competing to change its fields, once it has been safely shared with them.

Now put immutable records into a mutable dictionary. Adding or removing an entry still changes the dictionary. The records' immutability does not protect those dictionary operations.

An object is **thread-safe** for a stated use when permitted concurrent calls preserve its required behaviour. For our completed-file counter, two allowed increments must leave the correct count. A suitable lock or atomic increment can provide that behaviour. For a collection, its own access rules must protect lookup, insertion, removal, and any combined operation the application needs.

Ask which part changes and who controls it. Keeping state private to one owner, returning copies, or sharing immutable values can reduce competing changes. Returning a writable internal collection lets callers bypass that protection.

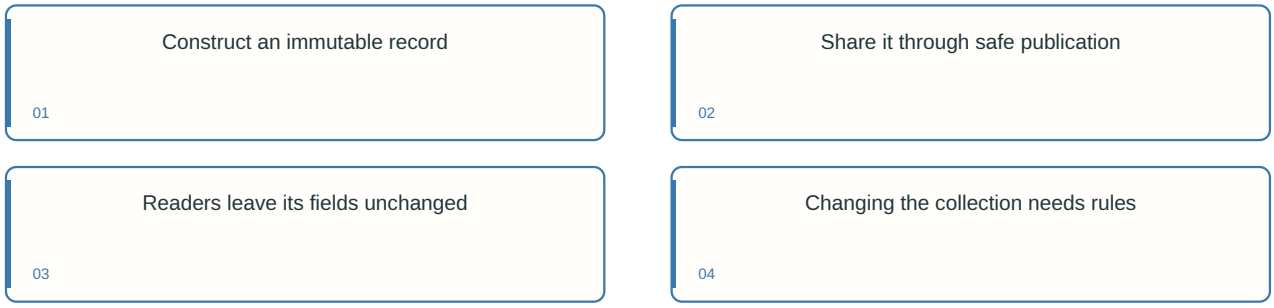


FIG. 4.4.2 **Keep shared data safe to use · the example**

Immutable records stay unchanged. Their mutable collection still needs rules for concurrent access.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.4.3 · Slow arrivals when workers cannot keep up

Assume 150 jobs arrive each second and workers finish 100. With no rejection or abandonment, unfinished work grows by `150 − 100 = 50 jobs per second`. After ten seconds, 500 additional jobs remain. Once workers are busy, this excess waits in the queue.

Those jobs retain memory or other resources. Continued growth can cause **resource exhaustion**: the service runs out of something it needs. A larger queue delays the problem without changing the extra fifty arrivals each second.

Now limit the queue to 100 waiting jobs. When it fills, the service can reply “busy,” and the producer can slow down before trying again. Or a producer can wait for queue space before sending more work. This effect on the producer is **backpressure**: downstream capacity limits upstream work.

A queue limit protects space; the response and the producer's reaction make the feedback useful. Also observe how long jobs wait. Even a queue that fits in memory can contain work that is already too old to be useful.

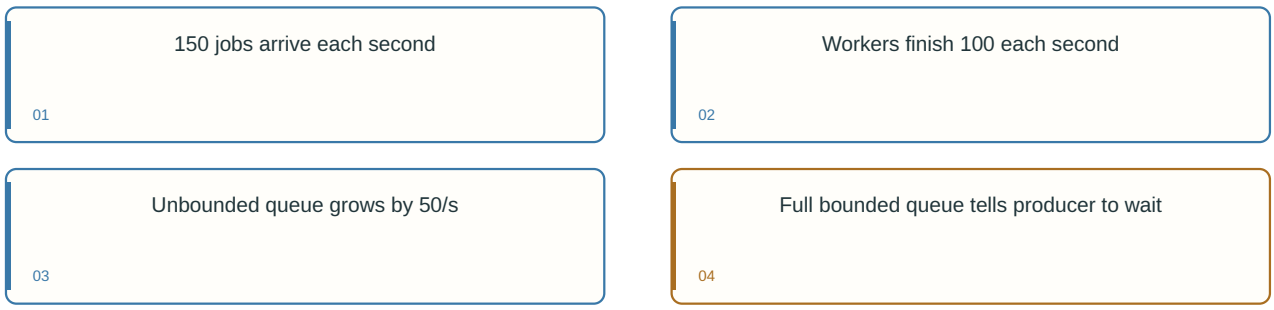


FIG. 4.4.3 **Slow arrivals when workers cannot keep up · the example**

If arrivals exceed completions, waiting work grows. Backpressure makes the producer respond to the receiving side's limited capacity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

4.4.4 · End related tasks and release their resources

One request starts two child tasks. One reads the uploaded file's name and size; the other makes a preview. The request-handling task is their **parent**. With **structured concurrency**, these related tasks belong to a **scope**, a block of work whose parent must account for its children before leaving. The scope keeps track of their completion and failure.

Suppose the request's time limit ends. The parent requests **cancellation**: it asks the children to stop. A child reaches a stopping point, closes its file, releases any borrowed resources, and ends. The parent accounts for its children's termination before leaving their scope. A blocking call that cannot respond promptly can delay this sequence; cancellation is not an instant kill.

Stopping the connection does not reverse work already saved. If the service accepted a job that must continue after the request ends, give it a saved job record and an owner beyond this request. Request-scoped children can then stop and release their resources while a later worker resumes the accepted job.

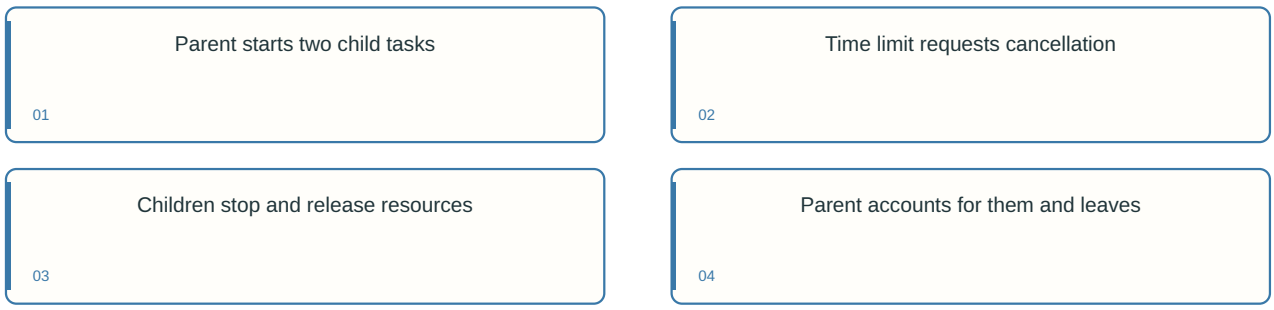


FIG. 4.4.4 **End related tasks and release their resources · the example**

A parent scope accounts for its children. Cancellation lets them stop and release resources; saved continuing jobs need a longer-lived owner.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

Let us connect the worker pool to a definite acceptance rule. This pseudocode stores each upload as a temporary file and tries to add its identifier to a queue. A full queue returns “busy”; it does not accept that job. An accepted entry is queued for this process run.

The queue holds at most 100 waiting identifiers. Fifty copies of worker run. try_put adds an identifier only if there is space; take waits while the queue is empty. convert either finishes or raises ConversionError. The finally block releases that job’s resources on either exit.

These limits cover conversion jobs. The service also needs separate limits on file sizes and simultaneous upload reception, since accept saves the file before checking queue space.

PSEUDOCODE · FIFTY WORKERS AND AT MOST ONE HUNDRED WAITING JOBS

```
waiting = bounded_queue(capacity=100)

def accept(upload):
    file_id = save_temporary_upload(upload)
    if not waiting.try_put(file_id):
        delete_temporary_upload(file_id)
        return "busy"
    return "queued for this run"

def worker():
    while True:
        file_id = waiting.take()
        try:
            convert(file_id)
            record_success(file_id)
        except ConversionError:
            record_failure(file_id)
        finally:
            close_job_resources()
            delete_temporary_upload(file_id)

start_workers(worker, count=50)
```

When fifty workers are busy and one hundred jobs are waiting, the next attempted queue insertion fails. That upload’s temporary file is removed and its caller receives “busy.” When a worker finishes, it takes another waiting identifier.

This example assumes the shown storage and cleanup operations succeed. Its queue is in memory, so restart loses queued entries. To promise restart recovery, store a job record before reporting durable acceptance and let replacement workers find unfinished records.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · TRY CHANGING THE INPUTS

What happens when jobs arrive faster than they finish?

Arrived: **9,000** · completed: **6,000** · waiting: **100** · rejected: **2,900**.

Every arrived job is completed, waiting, or rejected. A larger waiting limit lets more jobs wait, but workers still finish at the same rate.

Start with 150 arrivals and 100 completions each second. The extra jobs must wait or be rejected. Increase the queue limit: does it make workers finish faster, or only let more jobs wait? This model starts empty and treats jobs as a steady flow; it does not simulate individual threads or storage.

Compare waiting with calculation

Assume 1,000 independent files each wait 100 ms for storage and need negligible calculation. One at a time, they take 100 seconds. Fifty waits at once give twenty groups of 100 ms: an ideal two seconds. For this calculation, ignore startup, transfer limits, and other overhead.

The same assumed I/O work, with different ways to run and retain jobs

Arrangement	Ideal elapsed time	What needs a limit?	After process restart
One upload at a time	100 s	One active file; waiting work elsewhere	A saved job record is needed to resume
50 threads doing blocking I/O	2 s	Threads, buffers, connections, and waiting jobs	Memory-only jobs are lost
Nonblocking I/O, 50 files in progress	2 s	Tasks, buffers, connections, and waiting jobs	Memory-only jobs are lost
Saved job queue and 50 workers	At least 2 s, plus queue work	Accepted jobs, waiting jobs, and active workers	Saved records identify work to resume

Now change the work: each file needs 20 ms of CPU time and negligible I/O. The total is 1,000 × 0.02 = 20 CPU-seconds. One executing core needs at least 20 seconds. Eight cores sharing the work perfectly need at least 20/8 = 2.5 seconds. An event loop does not supply those extra cores; the runtime must permit actual parallel execution.

A saved queue preserves job records. It does not remove processing time or prevent every repeated effect during recovery. These calculations compare stated assumptions, rather than measured systems.

▼ Think further · The client disconnects after an upload job is accepted. Should processing stop?

First decide what acceptance promised. If the job must continue, its saved record and a worker outside the request scope own the remaining work. Request-owned children can stop and release their resources. If cancellation of the job is allowed, record that decision so later workers can obey it. Closing the connection alone neither deletes the saved job nor reverses work already completed.

Remember: Share data through the language's synchronisation rules. Bound waiting work and give every file, connection, and child task a clear end to its use.

Try applying it: When a request ends, which child resources should be released, and which saved job may still need to continue?

Unit 4 exercises → · Unit 4 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Operating Systems: Three Easy Pieces ↗

   University textbook. Runtime, concurrency, and persistence.

   `Access reviewed · 21 October 2026`

2. The Go Memory Model ↗

   Language specification · accessed 21 Oct 2026. One concrete memory-model contract; other languages differ.

   `Access reviewed · 21 October 2026`

PART 2 / UNIT 05

Object-Oriented and Modular Software Design

A payment class calculates fees, contacts a provider, and stores the result. When the provider changes a message field, we must search through the same class that holds the fee rules. Let us give these jobs clearer homes.

The running example

Start with a poorly designed payment-processing class. Refactor it into appropriate components, showing which problems each design improvement solves.

Unit 5 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

Work	Before separation	After separation
Check payment rule	One large payment class	Payment rule object
Call provider	Same payment class	Provider adapter
Save result	Same payment class	Repository

FIG. 5.0.1 **Before/after class diagrams**

Each part now owns a specific kind of work. A provider field change can stay inside the adapter while the payment rule remains the same.

Author-created illustration. Numbers are illustrative unless stated otherwise.



FIG. 5.0.2 **dependency graph**

The service uses the application’s gateway contract. The provider adapter implements that contract. The contract does not need to name a provider library.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Question	Inheritance	Composition
How is behaviour built?	Specialise a base class	Connect cooperating objects
Two independent choices	May need many subclasses	Supply each choice separately
What must callers rely on?	Subclass keeps base promises	Each object keeps its contract

FIG. 5.0.3 **inheritance vs composition**

A service can receive a fraud policy and a gateway independently. Inheritance is useful when the specialised class preserves what base-class callers may do.

Author-created illustration. Numbers are illustrative unless stated otherwise.

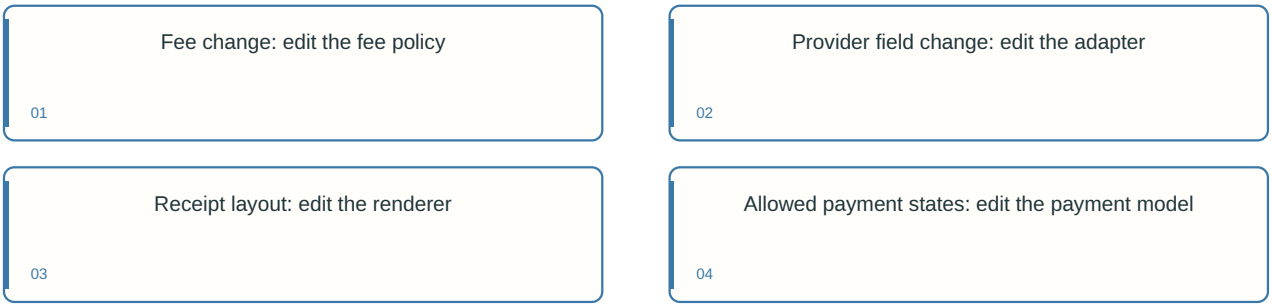


FIG. 5.0.4 **cohesion/coupling illustrations**

These examples show which parts need to change together. Keeping related decisions together helps a change stay in one place.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Model behavior with objects and values

Prerequisites: Introduction to System Design 5 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

A payment class calculates fees, contacts a provider, and stores the result. When the provider changes a message field, we must search through the same class that holds the fee rules. Let us give these jobs clearer homes.

We will start with objects and their operations, then compare ways to combine them. Our test is a simple change: can we add a provider while keeping the amount and result meanings clear?

5.1.1 · Four ways to organise an object

Our payment program checks an amount, asks an outside service to charge the customer, and records the result. This outside service is the **payment provider**. An **object** combines data with operations on that data. A **class** describes how such objects are made. For example, each Payment object has its own identifier and status.

The caller wants to request a charge. It should not need to assemble the provider's message fields. We give it an operation such as `gateway.charge(command)`. A **provider adapter** implements this operation: it translates the command into the provider's format. Offering the useful operation while hiding these details is **abstraction**.

Now suppose any caller can set `payment.status = "paid"`. It could mark a payment paid without a successful charge. Instead, we control changes through an operation that checks the provider's confirmed result. This is **encapsulation**: controlling access to an object's data so its operations can protect the rules. A private field alone is insufficient if another method can change it without checking.

Two gateways can offer the same charge operation. The service calls either in the same way, while each gateway handles a different provider. Using different implementations through the same operation is **polymorphism**. Both must give the caller compatible results.

Inheritance creates a class from an existing class. A Receipt class might provide `amountText()`. A CardReceipt subclass inherits that operation and adds a card-reference display. It can reuse or replace inherited behaviour. Inheritance is one way to organise classes; the first three ideas do not require it.

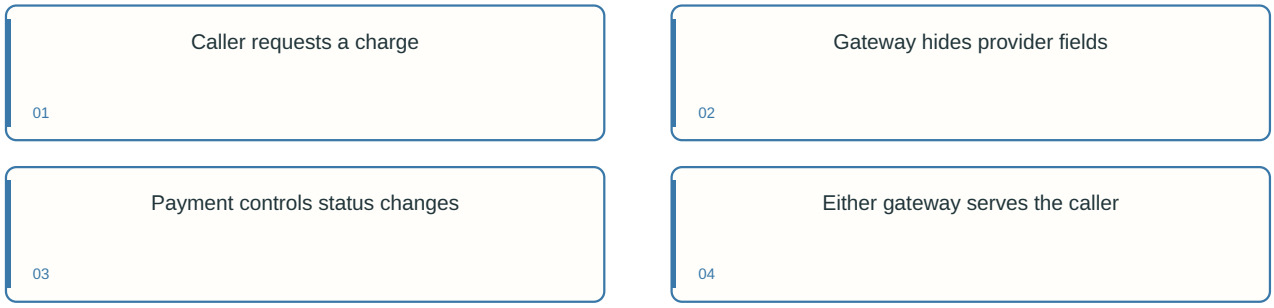


FIG. 5.1.1 **Four ways to organise an object · the example**

Abstraction hides details behind an operation. Encapsulation controls state changes. Polymorphism lets compatible implementations share a calling form. Inheritance derives a class from another class.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.1.2 · Build with parts or derive a class?

Suppose our payment service can use two fraud policies and two payment providers. A fraud policy decides whether to allow a payment attempt. The provider carries out the charge. These are separate choices.

If we encode every combination in subclasses, two policies and two providers can produce four service classes. Add another policy, and we need more combinations. With **composition**, the service holds a policy object and a gateway object. It asks the policy first, then uses the gateway. We can replace either part while keeping the other.

Inheritance is useful when the new class can fulfil the promises of its parent. Suppose Account allows a withdrawal whenever enough money is available. A ReadOnlyAccount that always rejects withdrawal cannot safely replace it for callers relying on that promise. Similar fields do not establish a suitable parent–child relationship.

Composition also needs clear operations between its parts. Use it to separate choices that really vary, rather than creating a new wrapper for every small calculation.

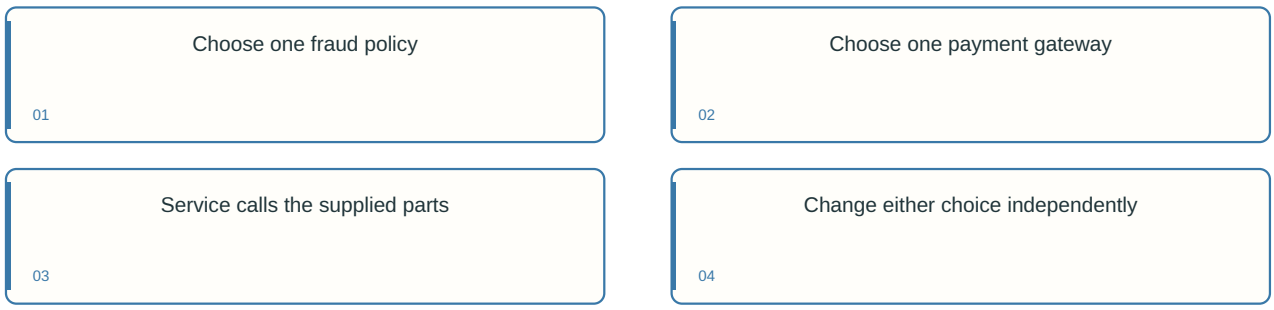


FIG. 5.1.2 **Build with parts or derive a class · the example**

Composition connects objects with separate jobs. Inheritance is suitable only when the subclass preserves the behaviour callers expect from its parent.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.1.3 · State exactly what a charge means

What does `charge(100)` mean? Is it 100 dollars or 100 cents? The operation name leaves an important question unanswered.

An **interface** describes the operations available to a caller. A **contract** states how those operations behave: which inputs are allowed, what the results mean, and what happens on failure. Our command includes `amount_minor=100`, `currency="USD"`, and `operation_id="P7"`. Here, a minor unit is a cent, so the amount is one US dollar. P7 identifies this intended charge.

Our gateway returns three possible outcomes. “Approved” means the provider confirmed that the charge succeeded. “Declined” means it definitely rejected the charge. “Uncertain” means we do not yet know whether it succeeded. A provider timeout can produce the last outcome: it may have charged the customer before its reply was lost.

The contract must also explain repeated commands. If the provider supports recognising P7, a retry with the same details can refer to the original charge. Adding the field ourselves does not create that support. We must use the provider's documented recovery method before deciding whether to try again.

Every replacement gateway must preserve these meanings. Matching the method name and argument types is only part of the agreement.

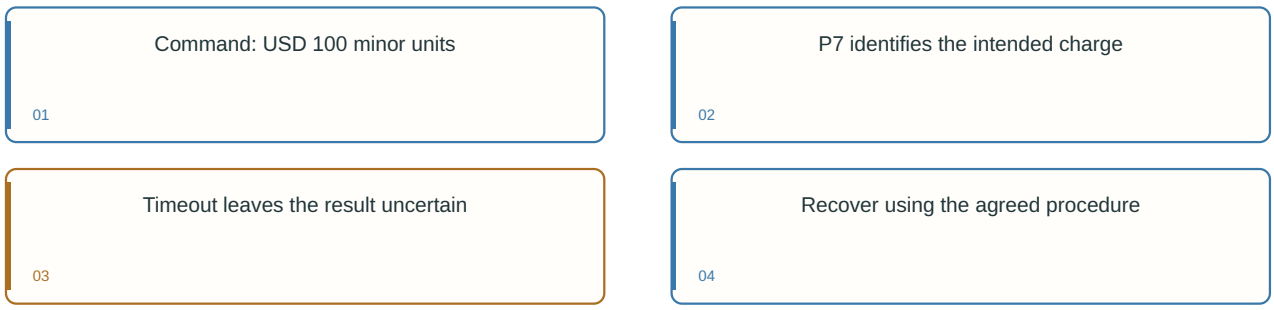


FIG. 5.1.3 **State exactly what a charge means · the example**

An interface names operations. A contract explains valid inputs, result meanings, effects, and failure or retry behaviour.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.1.4 · Keep related work together

A fee change should lead us to fee code. A receipt-layout change should lead us to receipt code. If both require editing one large payment method, their work has become tangled.

Cohesion describes how closely the work inside a module belongs together. A module is an organised part of a program, such as a file or class. A fee module has high cohesion when its operations serve the same fee rules. A class containing fee arithmetic, image resizing, and email formatting has several unrelated jobs.

Coupling describes dependencies between modules. If payment code builds provider-specific messages everywhere, a renamed provider field may require many edits. Put those field names inside the gateway adapter instead. The service still depends on the gateway's behaviour, but it needs less knowledge of the provider.

The aim is a clear place for each change. Dividing one simple operation across many tiny classes can make it harder to follow, even when each class is short.

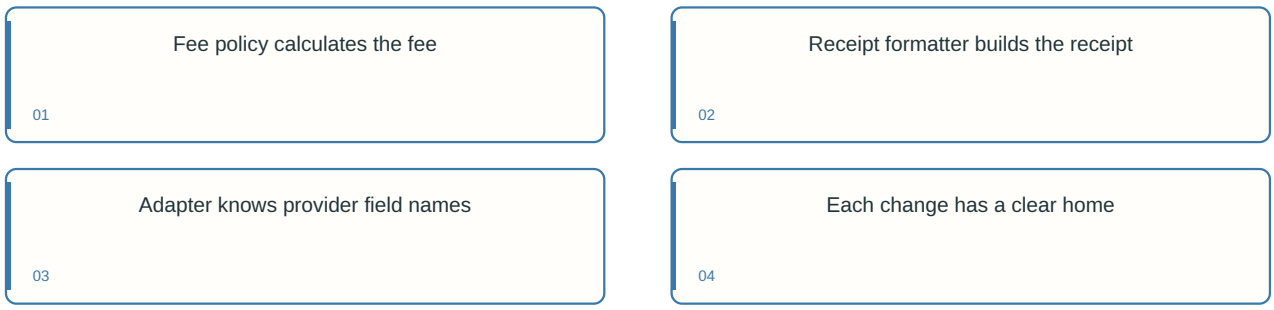


FIG. 5.1.4 **Keep related work together · the example**

Cohesion concerns related work within a module. Coupling concerns what one module must know about another.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.1.5 · Use SOLID to examine a real change

Let us add a second payment provider. **SOLID** names five principles that help us examine the design.

Single responsibility says to keep work that changes for the same reason together. The finance team's fee rule and a provider's message format can change independently. Give them separate owners. One responsibility can still need several methods.

Open/closed asks us to support a chosen kind of extension without rewriting working decisions. If the service already uses a gateway, we can add a provider adapter while keeping its payment logic. This protects that particular variation; a new business requirement may still require changing the service.

Liskov substitution says that a replacement must preserve the behaviour promised to callers. A new gateway must keep the same amount units and outcome meanings. Reporting a timeout as a definite decline breaks that promise, even if the method signatures match.

Interface segregation keeps a caller's interface focused on what it needs. Checkout needs charging; it should not need provider-reporting methods just to make a payment.

Dependency inversion keeps payment rules from depending directly on a provider's software types. Define the gateway operation in terms the application owns, such as its ChargeCommand and ChargeResult. Each provider adapter implements that operation. At runtime the service still calls the adapter, but a provider-specific error code is translated before it reaches the payment rules.

These principles do not require an interface for every class. Add structure where it helps a real change. Otherwise, extra layers can make the original operation harder to understand.

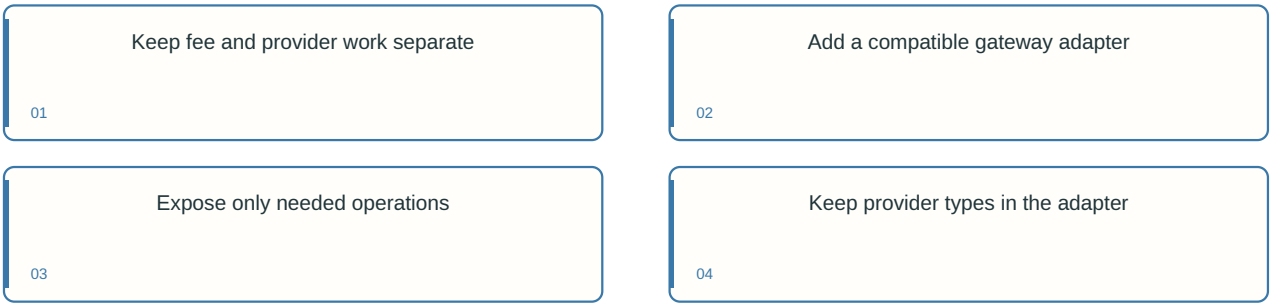


FIG. 5.1.5 **Use SOLID to examine a real change · the example**

SOLID examines responsibilities, extension, valid replacement, focused interfaces, and dependency direction. Apply each principle to an actual change, rather than adding layers by habit.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Remember: Give each rule and changeable detail a clear owner. A replaceable gateway must preserve the behaviour the payment service relies on.

Try applying it: The provider renames a message field. Where should the edit go? Why should the fee calculation stay the same?

Unit 5 exercises → · Unit 5 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

   Original research · ACM DOI. Module decomposition criteria; metadata and abstract verified.

   `Access reviewed · 21 October 2026`

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   `Access reviewed · 21 October 2026`

Principles under pressure

Prerequisites: 5.1 · Model behavior with objects and values 4 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

Our payment service now has several parts. We still need to decide which rules they should share and what each caller should know. Let us follow a fee change, a currency lookup, and an account reservation.

These examples will show when a design principle removes work, and when applying it mechanically adds more work.

5.2.1 · Keep one copy of a shared rule

Suppose, by mistake, both provider adapters contain a copy of the business fee rule. The fee rises from 3 to 4 minor units, but we update only one adapter. Customers now pay different fees depending on the provider. The problem is two maintained copies of one rule.

DRY, “Don't Repeat Yourself,” asks us to avoid duplicated knowledge. Put this shared fee rule in one function and let both adapters use it. Similar-looking code may still represent different rules: a customer-name limit and a receipt-title limit can change independently. Combining them just because both check text length would tie unrelated decisions together.

KISS, commonly expanded as “Keep It Simple, Stupid,” asks for an understandable design. A direct fee function is easier to follow than a configurable rule engine when we need only one calculation.

YAGNI, “You Aren't Gonna Need It,” asks us to defer features with no present need. We can add several fee strategies when the requirements call for them. We still implement the validation and failure handling that today's payment needs.

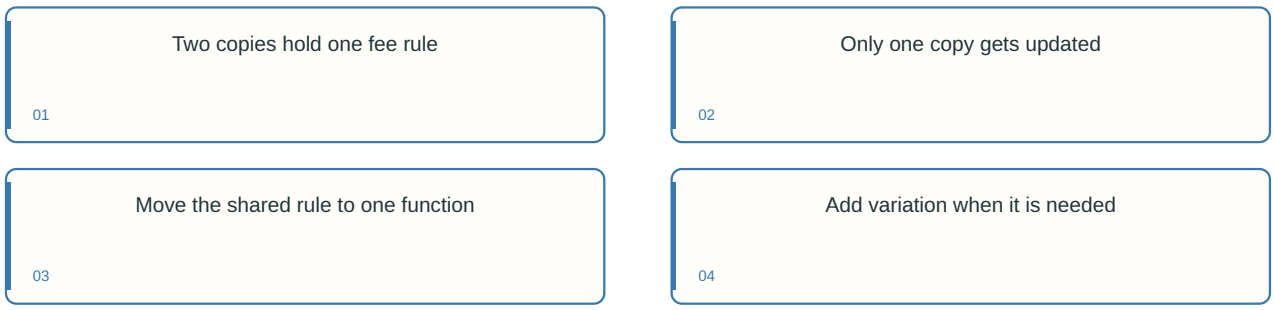


FIG. 5.2.1 **Keep one copy of a shared rule · the example**

DRY avoids duplicate knowledge. KISS favours an understandable solution. YAGNI defers speculative features while keeping present requirements complete.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.2.2 · Avoid knowing the whole object chain

Consider `order.customer.account.settings.currency`. To find one currency, the caller knows how four objects are arranged. If settings move, that caller must change even though it still needs the same answer.

The order can instead offer `paymentCurrency()`. The order's code performs the lookup, and the caller depends on the meaning of that operation. Changing where the currency is stored no longer changes the caller.

The **Law of Demeter** encourages a method to work with objects it knows directly, such as its own parts, its arguments, or objects it creates. It discourages reaching through several collaborators to use their internal parts.

The benefit is less knowledge of internal structure, rather than fewer dots on a line. Some chains deliberately form a public interface, such as a builder that returns itself after each step. Add a forwarding operation when it hides a real representation choice; a chain of empty wrappers can obscure the work instead.

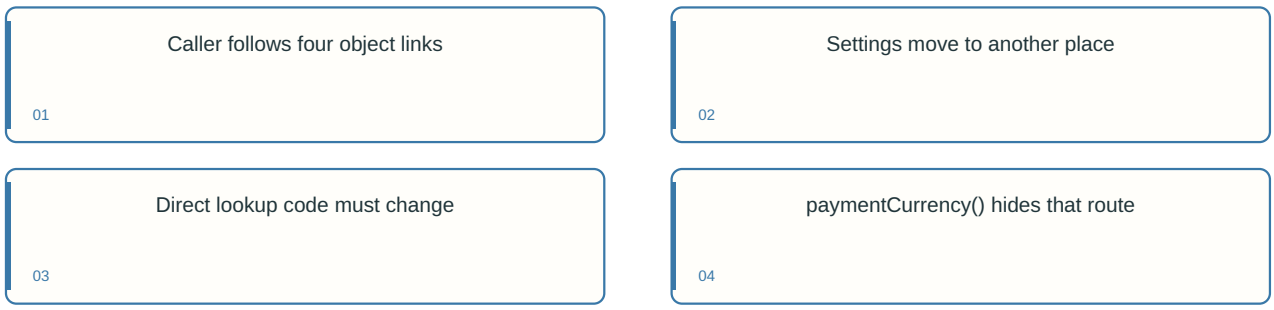


FIG. 5.2.2 **Avoid knowing the whole object chain · the example**

The Law of Demeter limits knowledge of distant objects. A caller should use a useful operation instead of depending on an internal lookup path.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.2.3 · Ask the owner to perform the change

An account holds 1,000 minor units. A caller wants to reserve 600 for a payment. If every caller reads the balance, checks it, and edits the reserved amount, each must implement the same rule correctly.

Instead, ask the account to `reserve(600)`. The account checks the available amount and updates its reservation together. After the first reservation, only 400 is available. Another reservation of 600 must fail. Keeping this rule beside the data is the idea behind **Tell, Don't Ask**: request an action from the object responsible for it.

This does not ban queries. A screen can still ask for the available balance. The important difference is that the screen does not decide and write the reservation itself.

Now let two processes both read 1,000 before either writes. Both separate checks can pass. The database must make the check and update **atomic**, so another writer cannot slip a conflicting change between them. For example, one conditional update can add 600 only if at least 600 remains available. An object method organises the rule; the storage operation enforces it across processes.

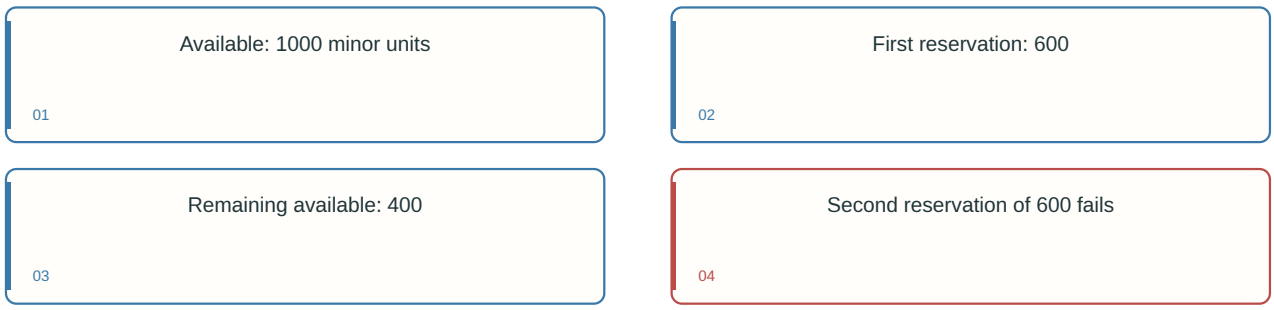


FIG. 5.2.3 **Ask the owner to perform the change · the example**

Tell, Don't Ask keeps a state-changing rule with its owner. Concurrent writers still need an atomic check and update at the shared storage boundary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.2.4 · Make reads and changes easy to distinguish

A screen calls `payment.status()` to display a payment. If that call also charges the customer, refreshing the screen could create another payment. A reader must be able to tell which calls request changes.

A **query** returns information without changing the object's observable state. A **command** requests a change. Calling `status()` should read the status; calling `charge(command)` clearly asks for an effect. This distinction is **command-query separation**.

Its strict form says that commands return no value. In practice, a command often returns a result or an operation identifier. State that convention clearly while keeping the requested effect explicit. A returned value alone does not make an operation a query.

Two status reads can still differ if another operation changes the payment between them. Separating commands and queries does not freeze the system. **CQRS**, Command Query Responsibility Segregation, goes further by giving reads and changes separate models. Ordinary command-query separation needs no separate database or service.

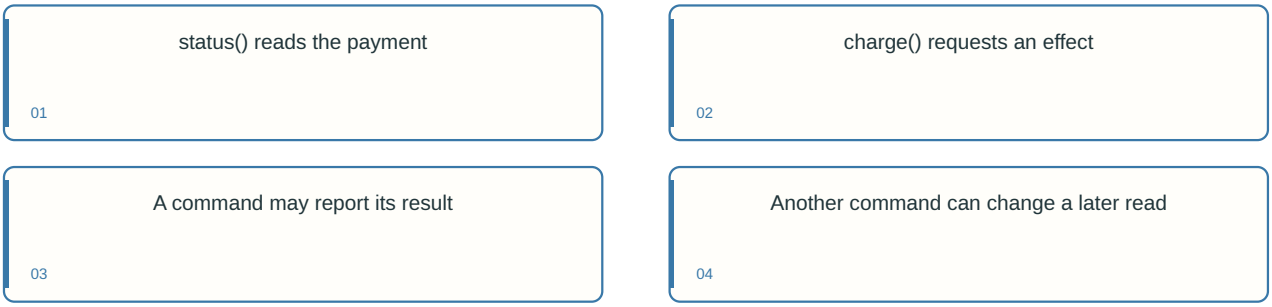


FIG. 5.2.4 **Make reads and changes easy to distinguish · the example**

Queries report information; commands request changes. Clear operation meanings do not require separate read and write infrastructure.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Remember: Share rules with the same meaning, hide unnecessary lookup paths, and keep state-changing checks with their owner. Make reads and changes easy to distinguish.

Try applying it: Two callers each reserve 600 from 1,000 available. Why is reserve(amount) a useful operation, and what must the database do to prevent both from succeeding?

Unit 5 exercises → · Unit 5 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

   Original research · ACM DOI. Module decomposition criteria; metadata and abstract verified.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Side effects, domain models & collaboration

Prerequisites: 5.2 · Principles under pressure 4 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

Payment 42 changes from pending to paid, but its identifier stays 42. Its amount has a different role: another equal USD amount can replace it. We will use this difference to model payments and money.

Then we will follow checkout through inventory and payment calls, including a lost reply, and test its decisions with a replacement gateway.

5.3.1 · Calculate new values, then perform effects

Our payment is 100 minor units and the fee is 3, both in USD. A function can calculate a total of `100 + 3 = 103` and return a new summary. The input amount need not change.

An **immutable** value cannot be changed after it is created. To represent a different amount, we create another value. This lets us keep the original summary beside the revised one without wondering which helper edited it.

A **pure function** returns the same result for the same inputs and changes nothing outside the calculation. Adding the amount and fee can be pure. Charging a provider is different: it creates an external **side effect**. Writing the result to a database is another effect.

Keep the calculation and the effects easy to identify. The new summary does not itself charge anyone. An immutable charge command can still be sent twice, so immutability does not prevent duplicate charges. The code that performs the effect must still recognise the intended operation and recover from an uncertain reply. Very large values may also make copying expensive; measure that cost when choosing a representation.

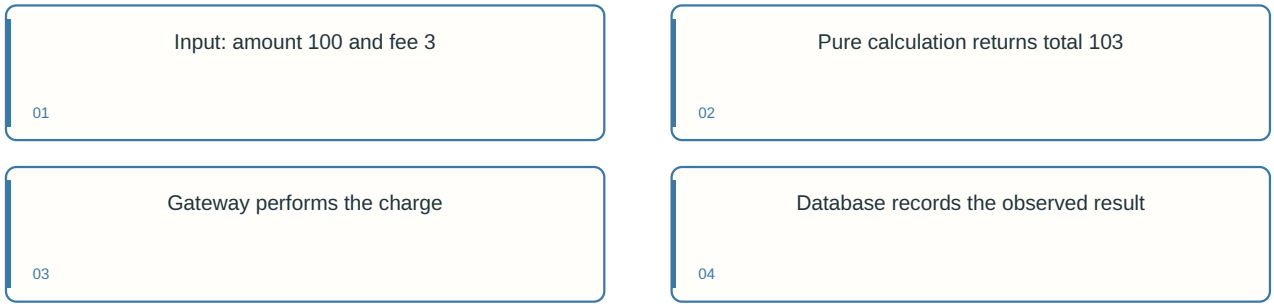


FIG. 5.3.1 **Calculate new values, then perform effects · the example**

Immutable values cannot be edited after creation. Pure functions calculate from inputs without external effects. Provider calls and writes still need explicit execution and recovery.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.3.2 · Distinguish identity from value

Payment 42 begins as pending and later becomes paid. It is still payment 42. We care about its identity and history, so it is an **entity**. Another payment for the same amount is a different entity.

Now consider `Money("USD", 100)`. Two such values mean the same amount and currency. We can replace one with an equal value without referring to a particular object's history. This is a **value object**. Money is often immutable: adding USD 3 produces a new USD 103 value instead of changing USD 100 in place.

These choices are separate. An entity can also have immutable snapshots: the pending and paid snapshots share identifier 42. The identifier connects them as states of the same payment.

A **domain model** expresses the concepts and rules of the problem we are solving. Here, Money describes amounts and Payment describes a payment's progress. A rule that must always hold for a valid object is an **invariant**. For our Payment, a paid status must be backed by a confirmed successful charge. Money addition has a separate input requirement: the currencies must match. Adding USD to EUR needs an explicit conversion rule first.

Some business calculations involve several objects and fit no single entity. A **domain service** can hold such a rule, such as choosing a transfer fee from the sender, recipient, and amount. Provider message formatting remains separate from that business calculation.

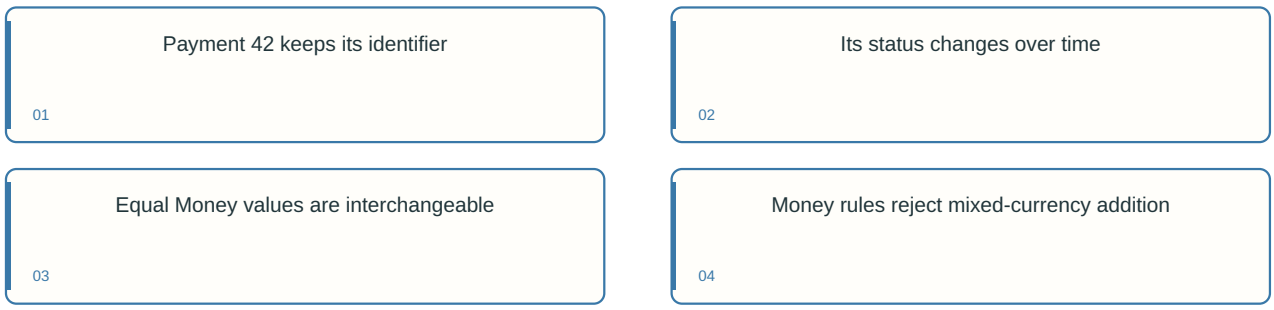


FIG. 5.3.2 **Distinguish identity from value · the example**

An entity is recognised by identity; a value object by its value. A domain model gives business concepts and their rules a clear representation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.3.3 · Follow the conversation between objects

Checkout needs an inventory reservation and a payment. The checkout coordinator asks inventory to reserve the items. Inventory checks stock and returns reservation R8. Checkout records R8 so it can refer to that exact reservation later.

Next, checkout asks the payment service to charge under operation P7. If the charge is confirmed, checkout can mark the order paid. If it is definitely declined, checkout asks inventory to release R8. Each object owns its rules, while the coordinator keeps track of the sequence. This exchange of requests and results is **object collaboration**.

A timeout needs a different next step. P7 may already have charged, so checkout keeps the result pending and seeks the provider's recorded outcome. Returning an error does not undo the reservation or the charge. Releasing a reservation or refunding a charge is another operation whose result must also be tracked.

These collaborators may run inside one program. Moving them into separate services adds network waits and failures; a method call inside a process does not account for those by itself.

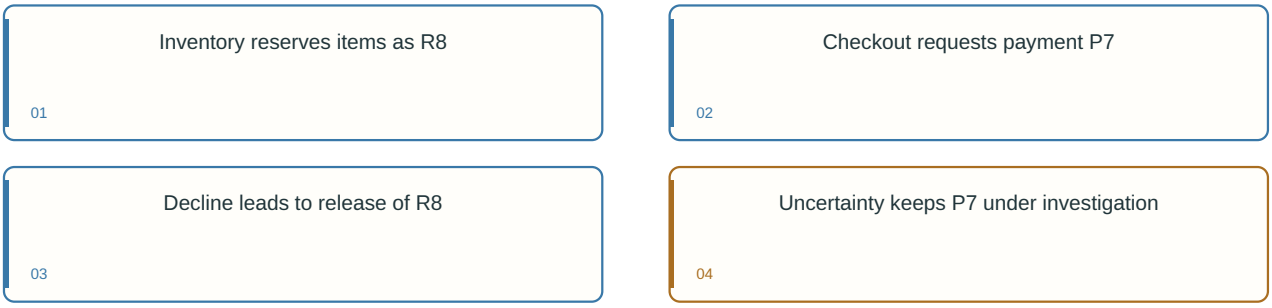


FIG. 5.3.3 **Follow the conversation between objects · the example**

Object collaboration follows requests, results, and ownership. A coordinator tracks progress; an error does not undo earlier effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.3.4 · Test decisions with a replaceable gateway

How can we test a declined payment without charging a real customer? Supply a **test double**, a small replacement gateway whose reply we control. Let it return “declined,” then call the service normally.

Check the result the customer depends on: the payment is recorded as declined, the order is not paid, and shipment has not begun. Repeat with “approved” and “uncertain.” The uncertain case should leave the payment pending, rather than inventing a definite failure.

Passing the gateway into the service, instead of constructing it inside the payment method, is **dependency injection**. It gives the test a place to supply its replacement. The same arrangement lets production code supply a real provider adapter.

A test that checks only private method names may fail after harmless code movement while missing a wrong final status. Check meaningful results and required effects. Tests using the real adapter and storage are still needed for message encoding, database commits, and actual provider behaviour. A fake reply cannot establish that the real provider recognises repeated operation identifiers.

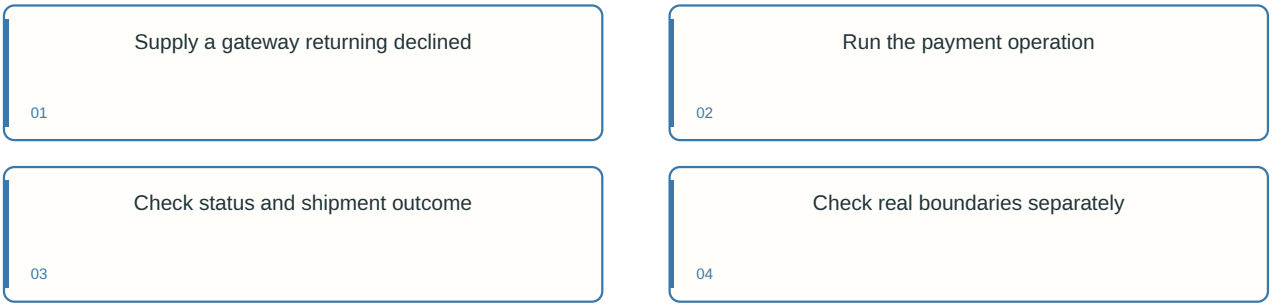


FIG. 5.3.4 **Test decisions with a replaceable gateway · the example**

A test double makes outcomes controllable. Dependency injection supplies the collaborator from outside. Tests must also check the real provider and storage boundaries.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Remember: Distinguish identity from value and calculations from effects. Follow the requests and returned results, and test the state the customer will observe.

Try applying it: A test gateway returns uncertain, and the order stays pending. What has this test shown? What must still be checked with the real provider?

Unit 5 exercises → · Unit 5 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

   Original research · ACM DOI. Module decomposition criteria; metadata and abstract verified.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Design for change, testing & functions

Prerequisites: 5.3 · Side effects, domain models & collaboration 4 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

We want clearer payment code while keeping the same results. Let us extract one piece at a time and check the same examples after each move.

We will also express a payment decision as a function. The representation may change; amount units, valid states, and the meaning of a timeout must remain clear.

5.4.1 · Improve the structure one step at a time

Our large payment method mixes fee calculation, provider messages, and result storage. We want to separate these jobs while keeping the same behaviour for callers. Such a change is **refactoring**.

First record examples of approval, decline, timeout, and storage failure. For each, state the returned result and the resulting payment state. Then extract the fee calculation into a function. Run the same examples and compare the amounts. Next extract provider translation, keeping the outcome meanings unchanged. Check again after that step.

A **code smell** is a clue that some code deserves attention: perhaps one fee rule appears in three places, or adding a provider keeps expanding the same conditional. It is not proof of a bug. The useful repair depends on the difficulty. A duplicated fee rule suggests one shared definition; an awkward name may need only a rename.

If the old code wrongly treats timeout as decline, correcting it changes behaviour. Make that correction explicit and test it as a separate change. Calling it a refactor would hide the new promise from reviewers.



FIG. 5.4.1 **Improve the structure one step at a time · the example**

Refactoring changes structure while preserving observable behaviour. Code smells guide investigation; the repair should address a concrete difficulty.

Author-created illustration. Numbers are illustrative unless stated otherwise.

5.4.2 · Express decisions as functions

We can represent payment decisions with functions as well as objects. Consider `decide(state, result)`. Give it a pending payment and the observed provider result. For “approved,” it returns a paid state. For “declined,” it returns a declined state. For “uncertain,” it leaves the payment pending and requests a later status check.

This decision can be a pure function. It makes no provider call and writes no database row. The same state and result produce the same decision, so a test can check the output directly.

Another part of the program performs the requested work. It records progress, calls the provider when needed, and passes the observed result into the decision function. This arrangement is often called a **functional core with an imperative shell**: pure decisions at the centre, code that performs effects around them.

Replaying the decision is different from repeating a charge. The outer code must keep track of which external operations have already run. A pure function makes the decision easier to inspect; it does not make a remote charge and a local database write happen as one operation. Functions and objects can be combined wherever each makes the rules clearer.

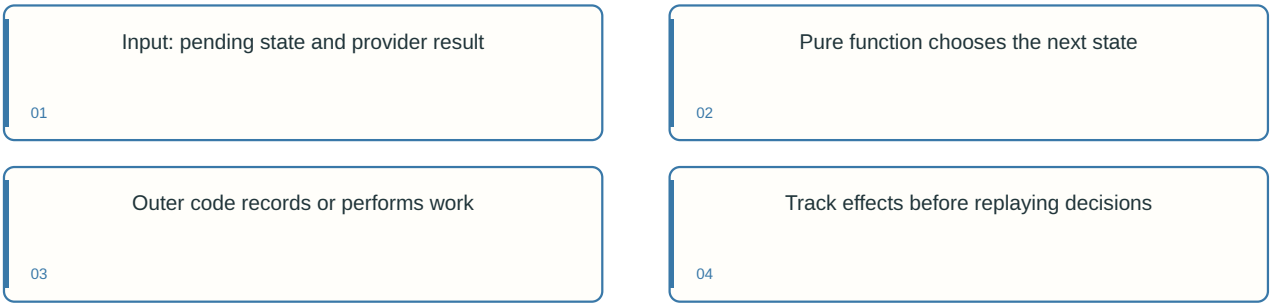


FIG. 5.4.2 **Express decisions as functions · the example**

Pure functions expose decisions as inputs and outputs. Separate code performs and records effects, including recovery from uncertain results.

Author-created illustration. Numbers are illustrative unless stated otherwise.

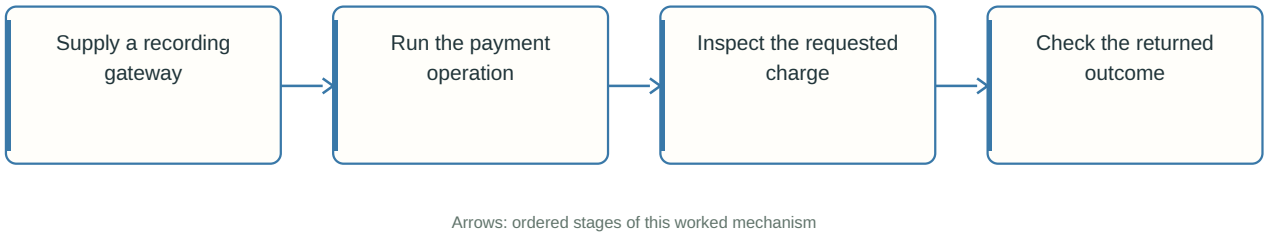


FIG. 5.4.3 **Test the payment rule without making a payment**

The test controls the gateway result and checks the service's decision. A separate test must check communication with the real provider.

Author-created illustration. Numbers are illustrative unless stated otherwise.

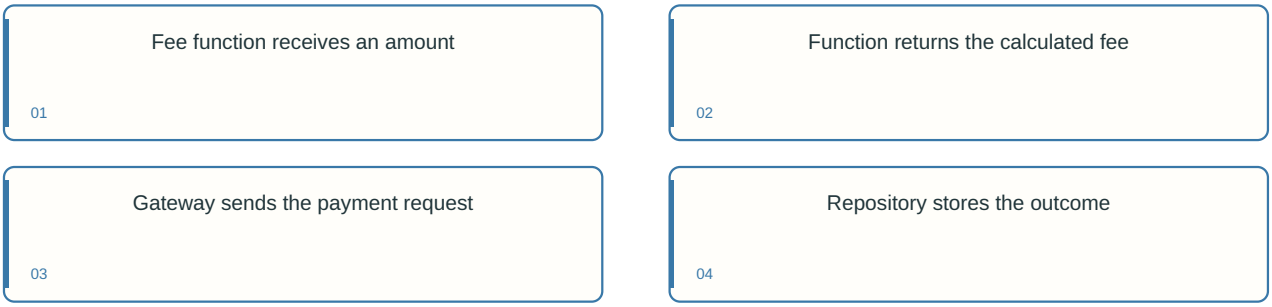


FIG. 5.4.4 **Keep calculation separate from effects**

Calculating a fee need not send a request or change stored data. Separating these steps lets us test the calculation directly.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

The service needs a gateway, but it need not construct one itself. Here we pass in a recording gateway. It saves the supplied command and returns a result chosen by the test.

The command asks to charge one US dollar: 100 USD minor units. Operation P7 names this intended charge. This small example checks the gateway boundary only; it makes no real charge and saves no payment record.

PYTHON · SUPPLY A RECORDING GATEWAY

```
class RecordingGateway:
    def __init__(self, result):
        self.result = result
        self.calls = []

    def charge(self, command):
        self.calls.append(dict(command))
        return self.result


def pay(command, gateway):
    return gateway.charge(command)


command = {"operation_id": "P7",
           "amount_minor": 100, "currency": "USD"}
gateway = RecordingGateway("uncertain")

assert pay(command, gateway) == "uncertain"
assert gateway.calls == [command]
```

The first assertion checks that `pay` returns the gateway's “uncertain” result unchanged. The second checks that the gateway received exactly one command with the supplied identifier, amount, and currency. Both assertions pass.

Returning uncertainty matters: we have no evidence of approval or decline. This example does not implement validation, persistence, provider retry handling, or the order's next state. Those belong in the surrounding service and need their own checks.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · We replace payment objects with pure functions. Which requirements remain?

The amount units, currency rules, permitted states, operation identity, and outcome meanings stay the same. Given the same state and provider result, the function should make the same business decision.

The outer code must still perform and record effects safely. A timeout can still hide a completed charge, and a database write can still fail after that charge. Changing methods into functions does not combine those separate events or make replay safe automatically.

Remember: Refactor in small, checked steps. Pure functions can express decisions; the code that performs effects must still record progress and handle uncertain outcomes.

Try applying it: A function recomputes the decision to charge P7. Why must the code performing the charge first check whether P7 has already run?

Unit 5 exercises → · Unit 5 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

   Original research · ACM DOI. Module decomposition criteria; metadata and abstract verified.

   `Access reviewed · 21 October 2026`

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   `Access reviewed · 21 October 2026`

PART 2 / UNIT 06

Design Patterns, Dependency Injection, and Object Lifecycles

Our notification service sends an order update by email. But even a test must deal with its provider setup. Let us move that setup out of the service and follow what changes. This will help us distinguish receiving a dependency, depending on a contract, and letting a framework make a call.

The running example

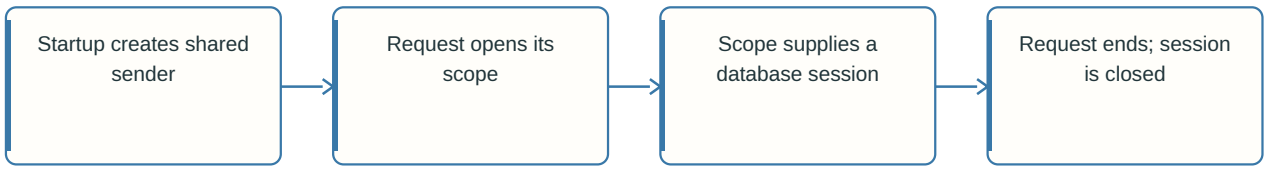
Create a notification platform supporting email, SMS, and push notifications. Evolve it from hardcoded classes to DI, Strategy, Factory, and Decorator patterns.

Unit 6 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.



Arrows: ordered stages of this worked mechanism

FIG. 6.0.1 **Object creation timelines**

This example reuses the sender for the application and the session for one request. The scope owner closes its session; a later job needs a current session of its own.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Lifetime in this example	Request A: two lookups	Request B: two lookups
Scoped	A1 / A1	B1 / B1
Transient	T1 / T2	T3 / T4
Singleton in this container	S / S	S / S

FIG. 6.0.2 **Which requests receive the same object?**

Matching letters name the same object. Here, scoped means one per request scope, transient means a new object per lookup, and singleton means one reused by this container. Framework rules determine the actual boundaries and cleanup.

Author-created illustration. Numbers are illustrative unless stated otherwise.

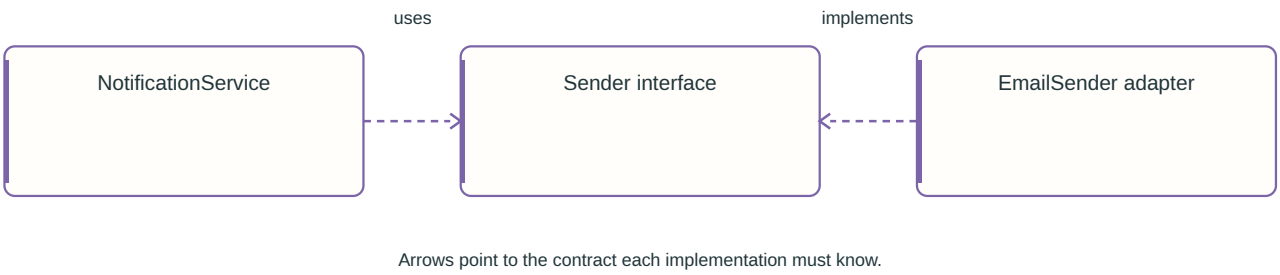


FIG. 6.0.3 **UML class relationships**

The service uses the Sender operation, and EmailSender implements it. These are code relationships between objects; they can all run in the same process.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Question	Before the change	After the change
Service knows	Provider request types	Application's Sender contract
Who builds the sender?	The service itself	Startup code
Test collaborator	Provider library	Recording sender

FIG. 6.0.4 **dependency inversion before/after**

Two decisions changed: the service uses its own contract, and startup supplies an implementation. Supplying the provider library alone would only change construction.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Comparison	First arrangement	Second arrangement
Adapter / Decorator	Translate an interface	Add work around the same interface
Factory / DI	Choose or create an object	Supply a needed object
Strategy / State	Choose an algorithm	Change behaviour with lifecycle state

FIG. 6.0.5 **pattern comparisons**

Similar-looking objects can solve different problems. Follow the operation each one performs before choosing a pattern name.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Dependencies, inversion & composition

Prerequisites: Object-Oriented and Modular Software Design 6 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

Our notification service sends an order update by email. But even a test must deal with its provider setup. Let us move that setup out of the service and follow what changes. This will help us distinguish receiving a dependency, depending on a contract, and letting a framework make a call.

6.1.1 · See what the service needs

Our notification service sends an email when an order is ready. It uses an email sender to do this work. The sender is a **dependency**: another part of the program that the service needs. If the service builds requests using a provider's software library, changing that library may force us to change the service too.

We can first write down what the service needs from any sender. It supplies a recipient and message, and receives an outcome such as “provider accepted” or “rejected.” This agreement is a **contract**. Acceptance means the provider accepted the request; it does not prove that the person read the message.

Call this contract `Sender`. An email adapter translates it into the provider's request and translates the reply back. The service's code now names `Sender`; the adapter's code names both `Sender` and the provider library. This is the direction of their source dependencies. When the program runs, the service still calls the actual email adapter. Which code knows which types, and which object runs next, are separate questions.

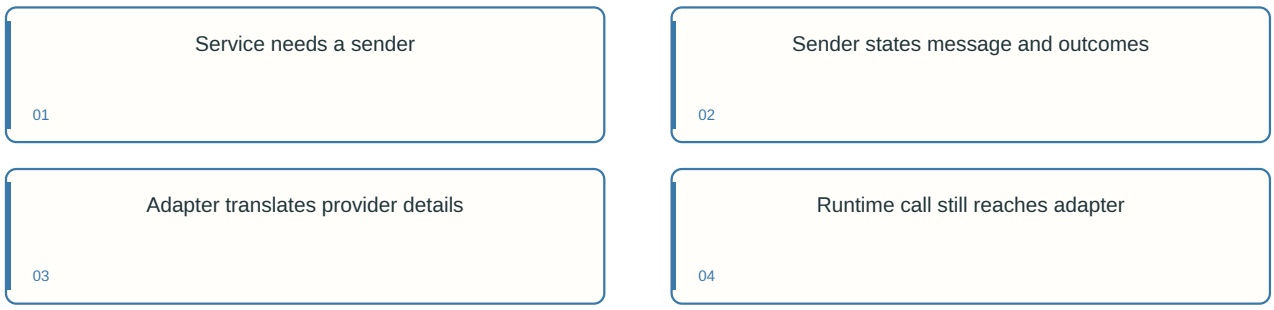


FIG. 6.1.1 **See what the service needs · the example**

A dependency is something code needs; a contract states how that dependency may be used.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.1.2 · Let another part of the program make the call

In a small script, we write `send_receipt()` in the main function and decide when it runs. In a web application, we register a function to handle a request. The framework calls that function when a matching request arrives.

Our function supplies the notification behaviour. The framework controls when to call it. This arrangement is **Inversion of Control**, or **IoC**. Control of part of the execution has moved from our code to the surrounding program. A registered function called later is a **callback**.

This lets many handlers use the framework's request handling without writing their own server loop. It also gives the handler a boundary to follow: when is its work finished, how are errors reported, and who closes resources? Starting work that continues after the handler returns needs a separate owner. A framework can also supply objects to the handler. We will examine that particular technique next.

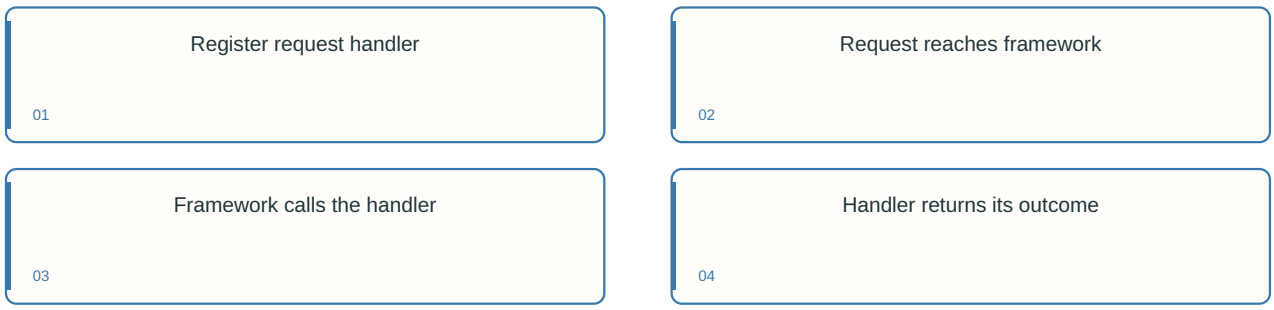


FIG. 6.1.2 **Let another part of the program make the call · the example**

IoC lets a framework or coordinator control when some application code runs.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.1.3 · Pass the sender into the service

Suppose `NotificationService` constructs an `EmailSender` inside itself. The service has chosen its own provider. A test that only wants to inspect a message must now deal with that choice.

Move the construction to startup. Create `sender = EmailSender(config)`, then pass it into `NotificationService(sender)`. Supplying a dependency from outside is **Dependency Injection**, or **DI**. The service keeps the supplied sender and calls its `send` operation when needed.

For a test, pass a `RecordingSender` instead. It records messages in memory without contacting a provider. The test can check the recipient and text using the same service code. Both senders must follow the agreed contract.

Ordinary constructors and function arguments are enough for DI. A container is optional. The useful change is that the service receives its sender instead of choosing and building it during its own work.

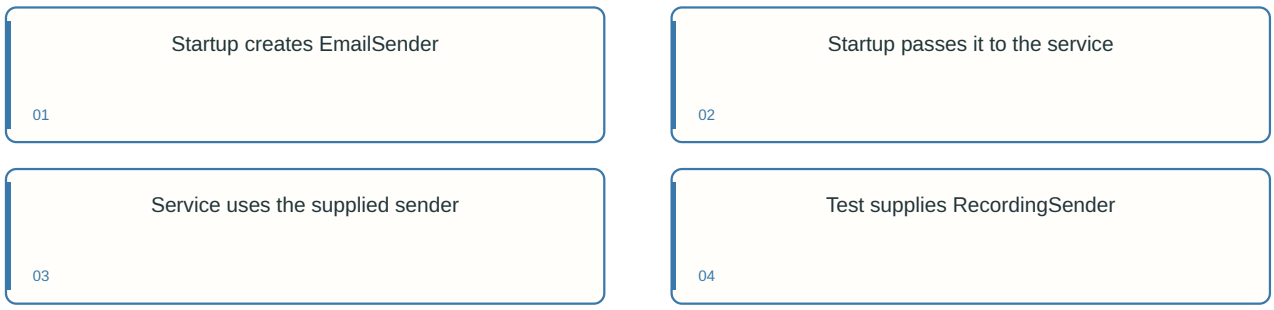


FIG. 6.1.3 **Pass the sender into the service · the example**

DI supplies a dependency from outside the object or operation that uses it.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.1.4 · Supply a dependency when it becomes necessary

Our service always needs a sender. With **constructor injection**, `NotificationService(sender)` receives it as the object is created. We make the constructor reject a missing sender, so a successfully created service is ready to send.

An optional component that records send outcomes can be supplied later through `set_observer(observer)`. This is **setter injection**; assigning a property serves a similar purpose. Until one is supplied, the service can skip observations. If the dependency were mandatory, calls would need to be prevented until it was set. Otherwise, the object could exist without being usable.

With **method injection**, an operation receives what it needs for that call: `render(message, formatter)`. The next call can use another formatter. The method need not keep it.

These forms answer when the dependency arrives. They also affect how long it is retained. A service may use a request's database session during that request, but storing it for later calls would outlive its valid use.

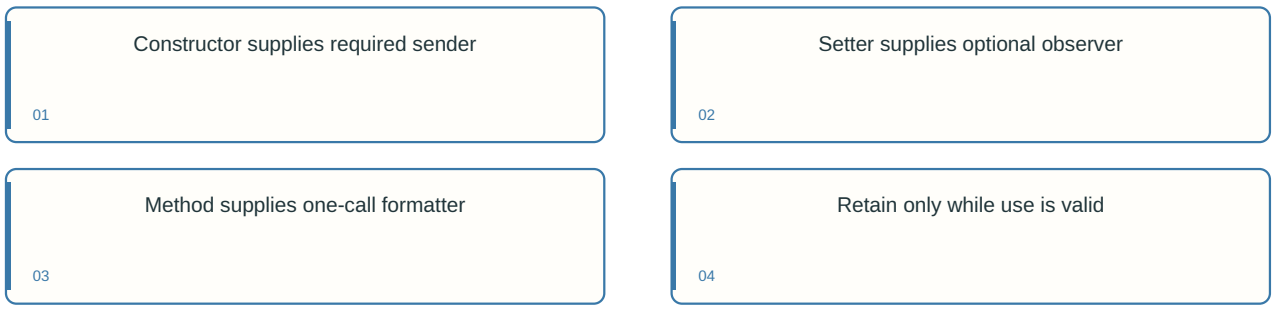


FIG. 6.1.4 **Supply a dependency when it becomes necessary · the example**

Constructor, setter, and method injection supply dependencies at different times.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.1.5 · Separate injection from dependency inversion

Consider `NotificationService(provider_sdk)`. The provider's software library, or **SDK**, has been passed in, so this is DI. But the service may still construct that SDK's special request objects. A provider change can still require a service change.

The **Dependency Inversion Principle**, or **DIP**, concerns this knowledge. Important application rules should depend on suitable contracts rather than on the details that carry them out. Our notification rule needs “send this message,” so it depends on `Sender`. The email adapter implements that contract and handles the SDK's details.

The service and adapter now both refer to the application's contract. The provider can change while the notification rule stays the same, provided the new adapter can satisfy that contract. Passing the adapter into the service combines DI with this dependency direction.

Inspect the values and errors too. Returning a provider-specific response for the service to interpret would carry those details back across the boundary.

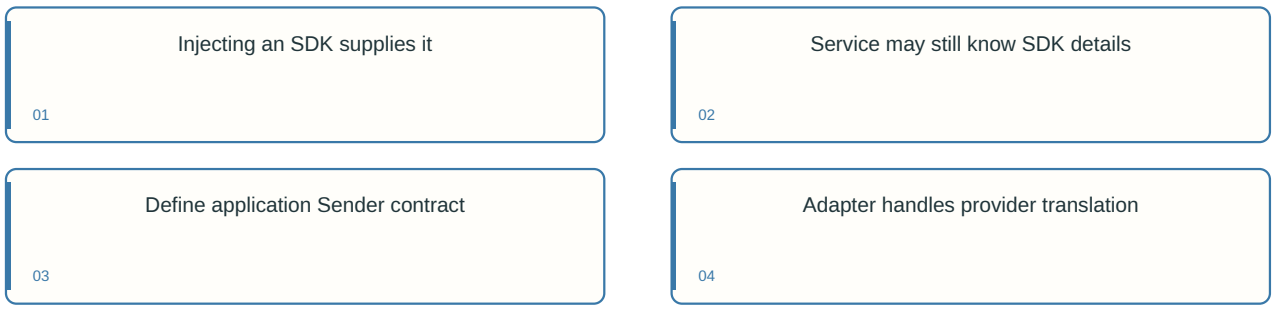


FIG. 6.1.5 **Separate injection from dependency inversion · the example**

DI concerns how a dependency is supplied; DIP concerns which contracts the code depends on.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.1.6 · Assemble the application in one clear place

Before requests arrive, startup code creates an email sender and a notification service. It passes the sender to the service, then gives the service to the request handler. The place where concrete objects are chosen and connected is the **composition root**.

This makes the setup easy to inspect. We can see which provider is used and whether two services share the same sender. The service itself can concentrate on sending notifications.

A **DI container** automates some assembly. We register a rule such as “to obtain Sender, construct EmailSender.” When asked for a service, the container follows its construction rules to obtain that service's dependencies. We also tell it when to reuse objects.

These rules are part of the program. Check that required registrations exist before serving traffic, and define who closes any created resources. For a small application, a few explicit constructor calls may be clearer than container configuration.

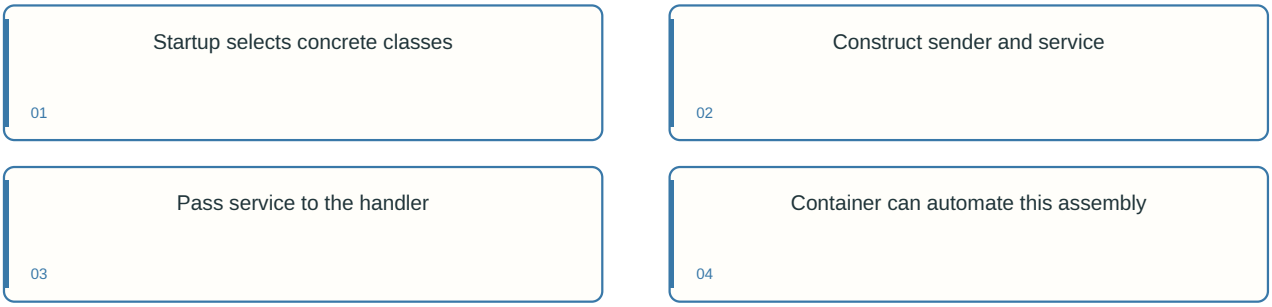


FIG. 6.1.6 **Assemble the application in one clear place · the example**

The composition root connects concrete objects; a DI container can automate their construction and reuse.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Remember: Name the sender's contract, choose an implementation during assembly, and pass it to the service. DI supplies the dependency; DIP concerns which contract the service knows; IoC concerns who controls part of the execution.

Try applying it: A service receives an SMS SDK through its constructor but still interprets provider-specific status codes. What has injection changed, and what would a Sender adapter change?

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Lifetimes, ownership & cleanup

Prerequisites: 6.1 · Dependencies, inversion & composition 6 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

A request gives its open database session to a background job. The response ends, the session closes, and the job has not even started. Let us trace who creates each object, how long it may be used, and who closes it.

6.2.1 · Choose when objects are reused

Two parts of request A need a database session. We want them to share one session, while request B gets another. With a **scoped** lifetime, the container reuses one instance within a **scope**, a boundary such as one request. Here the results are A1, A1 inside request A and B1, B1 inside request B.

With a **transient** lifetime, each request to the container for that dependency creates a new instance. Two such lookups inside request A can return T1 and T2. With a **singleton** lifetime, lookups reuse one instance within the container boundary. Both requests can receive S.

These names describe reuse. They do not by themselves tell us whether a shared object is safe for concurrent calls. A second process can also have its own S. Check the framework's actual boundaries and cleanup rules: “one” must mean one within a defined place, and “scoped” must name a scope.

Lifetime in this example	Request A: two lookups	Request B: two lookups
Scoped	A1 / A1	B1 / B1
Transient	T1 / T2	T3 / T4
Singleton in this container	S / S	S / S

FIG. 6.2.1 **Which requests receive the same object?**

Matching letters name the same object. Here, scoped means one per request scope, transient means a new object per lookup, and singleton means one reused by this container. Framework rules determine the actual boundaries and cleanup.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.2.2 · Give each request or job its own boundary

A request opens a database session and passes it to a background notification job. The response ends and the request closes the session. When the job later tries a query, its reference points to a closed resource.

Give the job its own **scope**: a defined period during which its objects may be reused and must remain valid. The request saves the notification data and queues its identifier. The worker opens a job scope, loads the notification using a current session, does the work, and closes that scope.

A request scope covers one request. A session scope, where supported, may cover several requests in a user's session. A custom scope can cover a job. Frameworks define these boundaries differently, so the start and end must be explicit.

Keeping an object for a longer time also keeps its memory and user data. A scope does not automatically start a database transaction or keep a user's permission current.

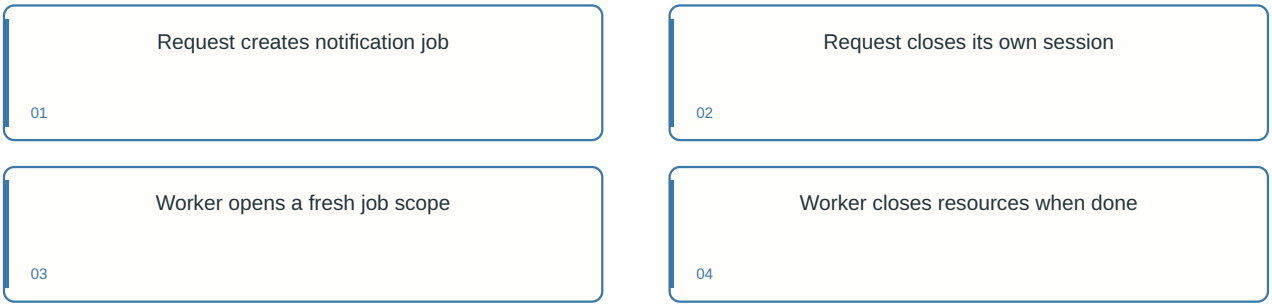


FIG. 6.2.2 **Give each request or job its own boundary · the example**

A scope names a boundary for reuse and cleanup; a later job needs its own valid resources.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.2.3 · Decide who must close the resource

A factory returns a stream. Must the caller close it? The answer depends on **ownership**: responsibility for releasing the resource.

The factory may create a fresh stream and transfer ownership to the caller. The caller then closes it after use. Or it may return an application-owned shared sender. The caller borrows that sender and leaves it open; its owner closes it at shutdown. Creating an object and owning its cleanup are separate decisions.

The return contract should state which case applies. Otherwise, one caller may close a resource another still needs, or everybody may assume somebody else will close it.

Some returned objects still use resources behind them. A result that fetches database rows as we read it needs its connection until reading ends. Either return it with clear close rules, or fetch a bounded set of records before closing the connection and returning those records.

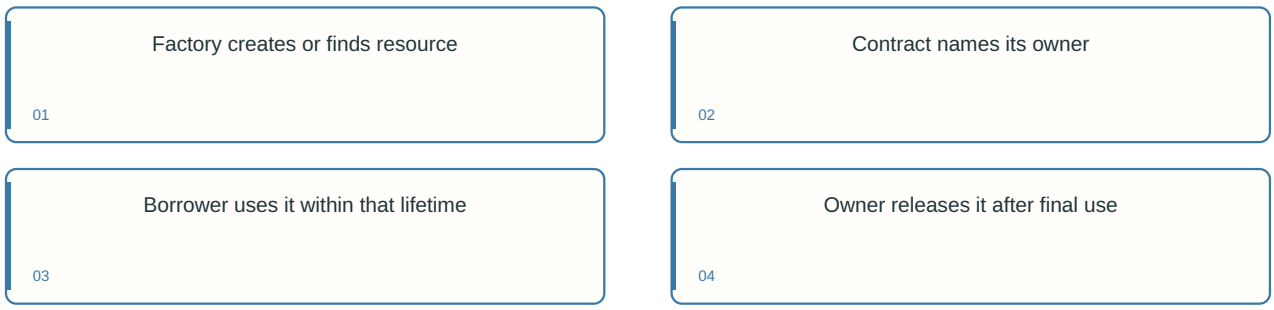


FIG. 6.2.3 **Decide who must close the resource · the example**

Ownership determines who releases a resource; a reference alone does not transfer that responsibility.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.2.4 · Release resources on every exit path

A pool has ten database connections. A request takes one, fails, and never returns it. Only nine remain available. After enough such failures, a healthy request cannot begin because all ten are still held.

Place acquisition and release in one structured resource block, such as Python's `with`, or use a `finally` block. On normal completion or a handled exception, the block releases what it owns. Cancellation also needs a path that runs this cleanup. A forced process stop cannot be assumed to run language cleanup code.

Cleanup may close a stream, return a connection, or release a permit. An unfinished transaction may need rollback before its connection can be reused. Releasing resources preserves capacity for the next operation.

Garbage collection reclaims memory under runtime rules. It gives no timely-release guarantee for a scarce database connection. Follow the resource's cleanup contract even when its object will eventually become unreachable.

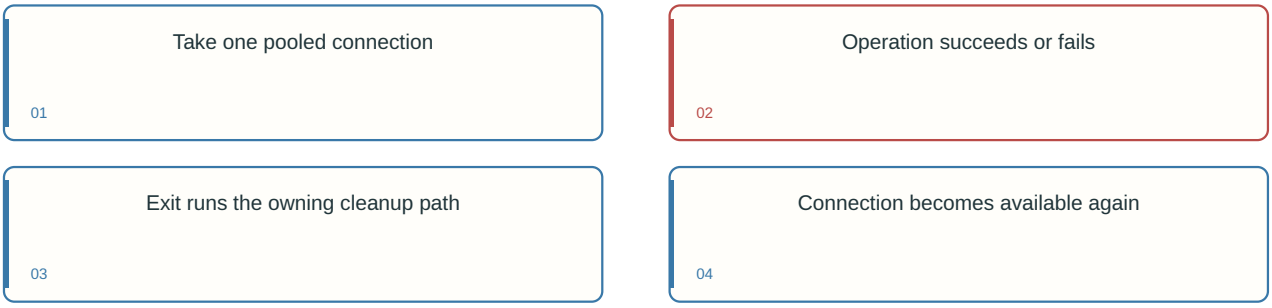


FIG. 6.2.4 **Release resources on every exit path · the example**

Structured cleanup returns owned resources when work ends, including supported error and cancellation paths.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.2.5 · Create an expensive object only when needed

Compiling a report template is expensive, and many runs never produce a report. We can wait until the first report request to create its generator. This is **lazy initialisation**. It avoids unused startup work but makes the first report wait for construction.

Suppose two first requests both see “no generator.” Without coordination, both may construct one. Use the runtime's supported lazy facility or a suitable lock to make the intended check-and-create sequence safe. Other callers must receive a fully initialised object, rather than a reference published halfway through construction.

Construction can fail too. Decide whether later callers retry or receive the saved failure, and release resources left by an unsuccessful attempt. If every ordinary request needs this object, creating it before the application reports readiness can reveal a configuration problem earlier. Laziness changes when the work happens; it does not remove the work or its possible failure.

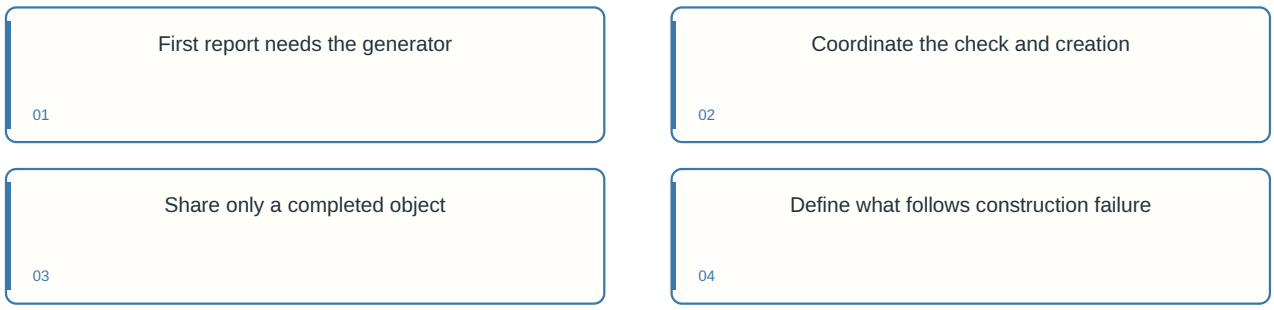


FIG. 6.2.5 **Create an expensive object only when needed · the example**

Lazy initialisation moves construction to first use, together with its delay and possible failure.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.2.6 · Find cycles and dependencies kept too long

To construct `NotificationService`, we need `DeliveryPlanner`. But the planner's constructor needs the notification service. Neither can be completed first. These constructor requirements form a **dependency cycle**.

Find why both links exist. Perhaps each class does work that belongs in a separate coordinator, or both need the same smaller helper. Moving that work can remove a link. A later global lookup may merely postpone the problem.

Now consider a different failure. An application-wide service stores the database session from request A. Request A ends and closes its session. The service remains and tries to use that session during request B. A longer-lived object has kept a dependency beyond its intended scope. This is a **captive dependency**.

Supply a currently valid session for each operation, or inject a narrow factory that opens and closes an operation scope. The operation must finish before that scope closes. Extending every session indefinitely would replace the original problem with uncontrolled sharing.

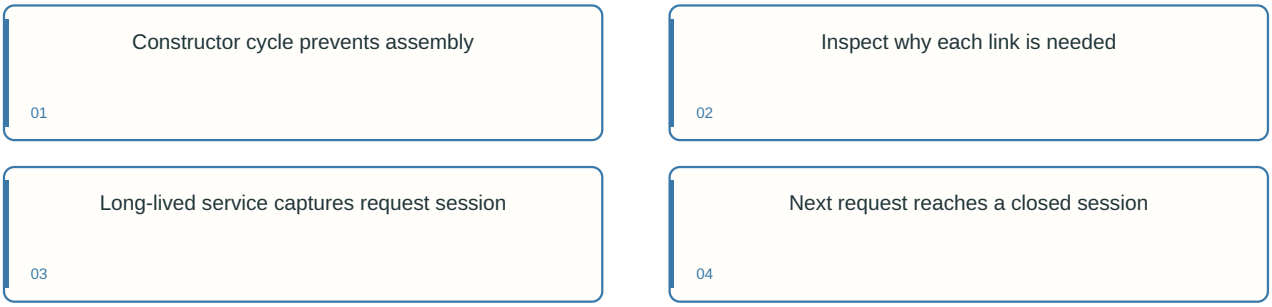


FIG. 6.2.6 **Find cycles and dependencies kept too long · the example**

A cycle blocks construction; a captive dependency can survive construction but be retained beyond its valid lifetime.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Remember: Object reuse needs a stated boundary. Keep each use inside the resource's valid lifetime, and give its owner a cleanup path for normal completion and supported failures.

Try applying it: A notification job starts an hour after the request ends. What should the request pass to it, and when should the worker open and close its database session?

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Injection, locators & factories

Prerequisites: 6.2 · Lifetimes, ownership & cleanup 4 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

A constructor receives a container, but the service keeps asking it for different objects. We must read every method to discover the service's needs. We will compare this lookup with explicit injection, then use a small factory for a choice made when a notification arrives.

6.3.1 · Show dependencies instead of looking them up inside

Compare `NotificationService(sender)` with a service that calls `registry.resolve(Sender)` inside its methods. The first constructor tells us that a sender is needed. For the second service, we must read the methods to discover that need.

An object used to find other services this way is a **service locator**. A missing registration may appear only when the lookup branch runs. Tests may also need shared registry setup before they can call the service.

With DI, assembly code obtains the sender and supplies it explicitly. Tests can supply their own sender directly. Passing the entire container into the constructor still leaves hidden needs if the service uses it to look up arbitrary objects.

A framework integration point may need container access. Keep that lookup at the assembly or scope boundary, then pass the specific collaborators into the application code. This makes the code's needs easier to see and limits what it can obtain.

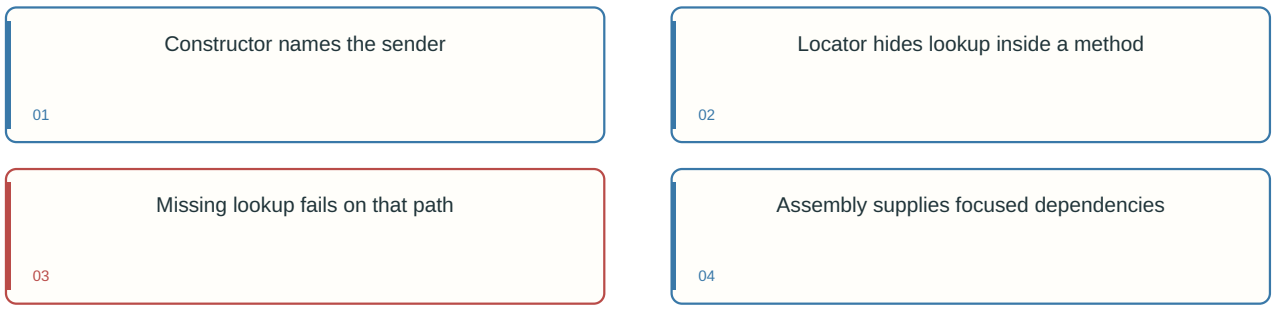


FIG. 6.3.1 **Show dependencies instead of looking them up inside · the example**

A service locator obtains dependencies through lookup; explicit injection makes the receiver's needs visible.

Author-created illustration. Numbers are illustrative unless stated otherwise.

6.3.2 · Inject a factory for a choice made later

Our service once sent only email. Startup could create its sender immediately. Now each notification specifies email or SMS, so the channel is not known until the notification arrives.

Give the service a `SenderFactory`. Its operation `for_channel(channel)` returns a suitable sender. Startup injects the factory; during a call, the service asks it for the chosen channel. **DI** supplies the factory, while the **factory** makes the later selection or construction decision.

A factory does not always create a fresh object. It can return a shared sender already built at startup. Its contract must say whether the returned sender is shared, borrowed from a scope, or newly owned by the caller. This determines whether the service may close or retain it.

Keep the operation narrow. The need to select a sender does not require giving the service permission to obtain every object in the application.

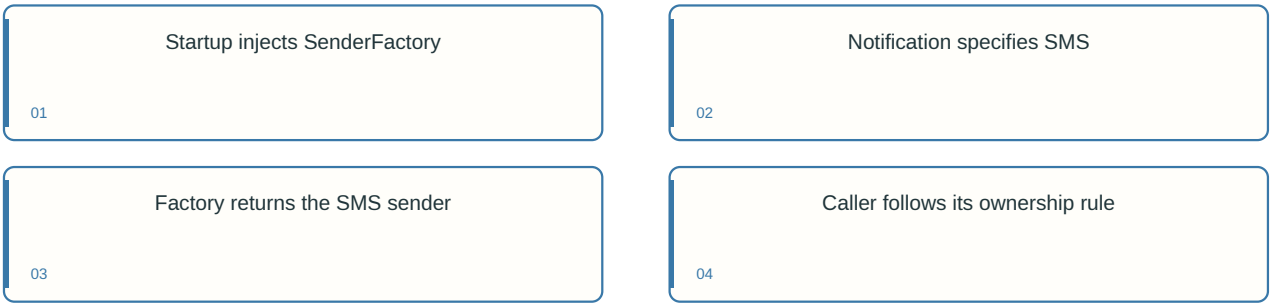
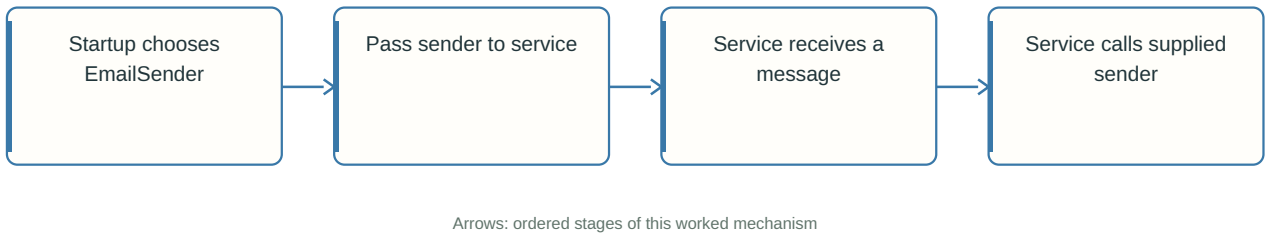


FIG. 6.3.2 **Inject a factory for a choice made later · the example**

DI can supply a factory that selects or constructs a dependency when later input is available.

Author-created illustration. Numbers are illustrative unless stated otherwise.

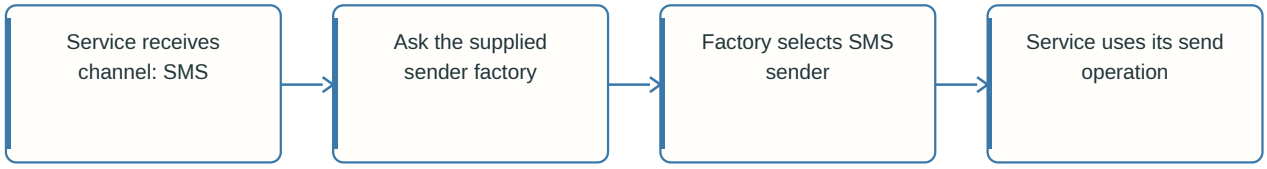


Arrows: ordered stages of this worked mechanism

FIG. 6.3.3 **Supply the sender from outside**

The constructor shows that the service needs a sender. It does not need to search a global registry during the operation.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 6.3.4 **Choose a sender when the message arrives**

Injection supplies the factory at construction. The factory handles the choice that becomes known only when a message arrives.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Remember: Supply known dependencies explicitly. When later input determines a choice, inject a focused factory and state who owns the returned object.

Try applying it: An email sender is shared by the application, while an SMS sender is newly created for each call. What must the factory tell callers about retaining and closing each result?

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Creational patterns

Prerequisites: 6.3 · Injection, locators & factories 6 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

Creating a notification can be as simple as calling a constructor. Different needs arise when we choose a writer, obtain matching rendering parts, collect settings in stages, or copy a layout. Each example below follows one construction problem.

6.4.1 · Choose how an object is created

A notification has a recipient, a message, and optional settings. A constructor can often check these values and return a usable object. Start there.

More complex creation raises different questions. Should a subclass select a writer? Must a renderer and template come from the same family? Are we collecting settings over several steps, copying a configured object, or restricting access to one instance?

Creational patterns name arrangements for these needs. The next five examples follow the construction step by step. The choice depends on what must vary. A pattern is useful when it makes that choice clearer; it should not make a simple constructor harder to follow.

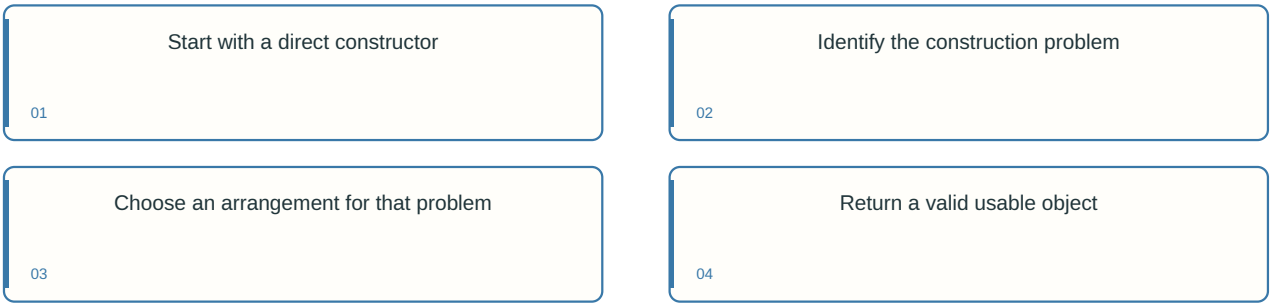


FIG. 6.4.1 **Choose how an object is created · the example**

Creational patterns organise how objects are selected, assembled, copied, or shared.

Author-created illustration. Numbers are illustrative unless stated otherwise.

PATTERN / HOW IT WORKS

Factory Method

Suppose an exporter always writes the same rows, but one version produces CSV and another produces HTML. We can keep the export steps in a base class and let each subclass choose its writer.

The base operation calls `make_writer()`, then uses the returned writer. `CsvExporter` overrides that creation operation to return `CsvWriter`; `HtmlExporter` returns `HtmlWriter`. This overridable creation operation is the **Factory Method**. The export code knows the writer contract without choosing the concrete class.

This fits an existing class hierarchy with a varying creation step. If there is no reason for exporter subclasses, passing a writer into one exporter can be simpler. Both arrangements separate choice from use, but the override is what defines Factory Method here.

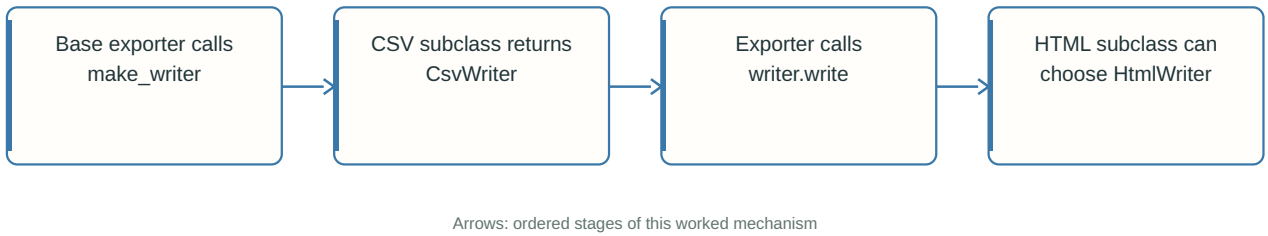


FIG. 6.4.2 **Factory Method · the example**

A subclass overrides a creation operation while the surrounding processing steps stay shared.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
class Exporter:
    export(rows):
        writer = self.make_writer()
        return writer.write(rows)
    make_writer(): raise NotImplemented

class CsvExporter(Exporter):
    make_writer(): return CsvWriter()
```

PATTERN / HOW IT WORKS

Abstract Factory

An HTML email renderer expects HTML templates. A plain-text renderer expects plain-text templates. Choosing them independently can produce a pair that cannot work together.

An **Abstract Factory** supplies a whole family of related objects. Choose `HtmlNotificationFactory`, then obtain both its renderer and its template. The caller uses their common contracts. Switching to the plain-text factory switches both choices together.

The family must actually preserve compatibility: the HTML factory cannot return a plain-text template merely because the method names match. This arrangement helps when several products vary together. If we need only one sender, a factory for that single sender is enough.

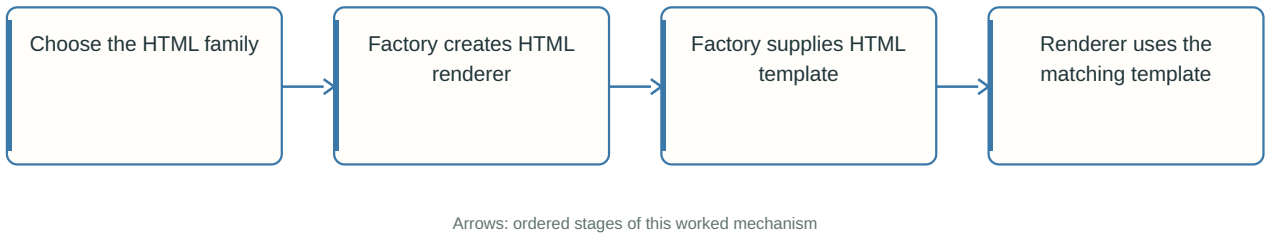


FIG. 6.4.3 **Abstract Factory · the example**

One selected factory supplies a compatible family of related objects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
family = HtmlNotificationFactory()
renderer = family.renderer()
template = family.template()
output = renderer.render(template, data)
```

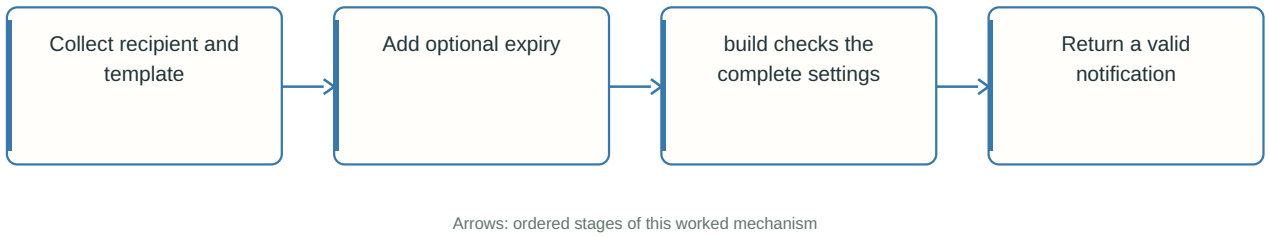
PATTERN / HOW IT WORKS

Builder

A notification needs a recipient and template, while expiry and delivery options may arrive from later steps in a form. We can collect these choices in a **Builder**.

Calls such as `to(recipient)` and `template(template_id)` update the builder's draft settings. `build()` checks that required fields are present and that the options agree. Only then does it return a usable notification. An incomplete builder is allowed; an invalid finished notification is rejected.

The builder should copy any mutable settings the finished object must own. Otherwise, changing the builder later might change an already built notification. When all inputs are already available together, a constructor with named arguments and validation may express the same task more simply.



Arrows: ordered stages of this worked mechanism

FIG. 6.4.4 **Builder · the example**

A builder collects choices in stages and validates them before returning a finished object.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
builder.to(recipient).template(template_id)
notification = builder.expires_at(deadline).build()
# build rejects missing fields and copies owned settings
```

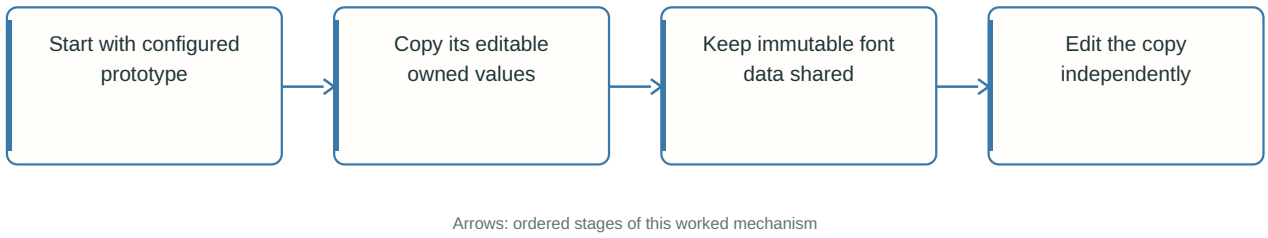
PATTERN / HOW IT WORKS

Prototype

Many notification layouts start from the same configured layout. Rather than repeat its setup, we can create a new layout by copying that example. The configured example is a **Prototype**.

The copy must have its own editable values. If both layouts refer to one mutable list of paragraphs, editing the copy can change the original too. `clone_owned_values()` copies that list and the editable paragraph data it contains. Immutable font data can remain shared because neither layout changes it.

Decide what copying means for each field. A live connection or lock usually cannot be duplicated like paragraph text. Prototype is useful when copying the supported configuration is clearer than rebuilding it; it does not make every object safe to clone.



Arrows: ordered stages of this worked mechanism

FIG. 6.4.5 **Prototype · the example**

Prototype creates an object by copying configured state with explicit rules for copied and shared values.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
layout = prototype.clone_owned_values()
layout.title = "Monthly report"
layout.paragraphs[0].text = "Your summary"
# Original editable values stay unchanged
```

PATTERN / HOW IT WORKS

Singleton

Several parts of one application need the same immutable routing configuration. A **Singleton** restricts access to one instance within a defined boundary.

In a traditional implementation, callers cannot invoke the constructor directly. An accessor constructs the instance on first use and returns that same instance afterward. If callers can arrive concurrently, the construction step needs coordination. Here the lock protects the check and construction together.

The boundary in this example is one loaded module in one process. Another process has its own instance. Reusing one object also does not make its mutable fields safe for concurrent changes.

A DI container can provide similar reuse without a global accessor: create one configuration at startup and inject it wherever needed. That often makes the dependency easier to see.

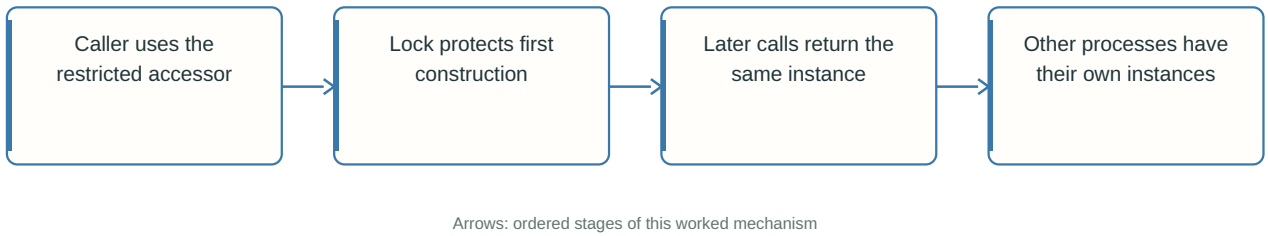


FIG. 6.4.6 **Singleton · the example**

Singleton restricts access to one instance within a stated program boundary; it supplies no cross-process uniqueness.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
# Module-private constructor and state; illustrative pseudocode
instance = None
get_configuration():
    with creation_lock:
        if instance is None:
            instance = Configuration.load()
        return instance
```

Remember: Choose the construction arrangement for the actual need. A finished object must be usable, and copied or shared values need clear ownership.

Try applying it: All notification settings are available at once. A named-argument constructor checks them together. What change to this workflow might make a builder useful?

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Structural patterns

Prerequisites: 6.4 · Creational patterns 8 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

We add timing to a sender, translate an SMS library's interface, and check access to a saved notification. Each change can add a wrapper, but each wrapper does different work. Follow what it receives, changes, and passes onward.

6.5.1 · Arrange objects around an existing operation

Our service has a sender. We may need to translate an old provider's interface, measure each send, or check access before calling it. Each change can place another object around the sender, but the purpose differs.

Structural patterns describe arrangements like these. Some translate between interfaces. Some group objects or add behaviour around an operation. Others simplify access or share repeated data.

For each example below, follow the call: what does the added object receive, what does it do, and what does it pass onward? This reveals the pattern more clearly than the wrapper's shape. A timing wrapper and a provider translator can look similar in a diagram while making different promises to their callers.

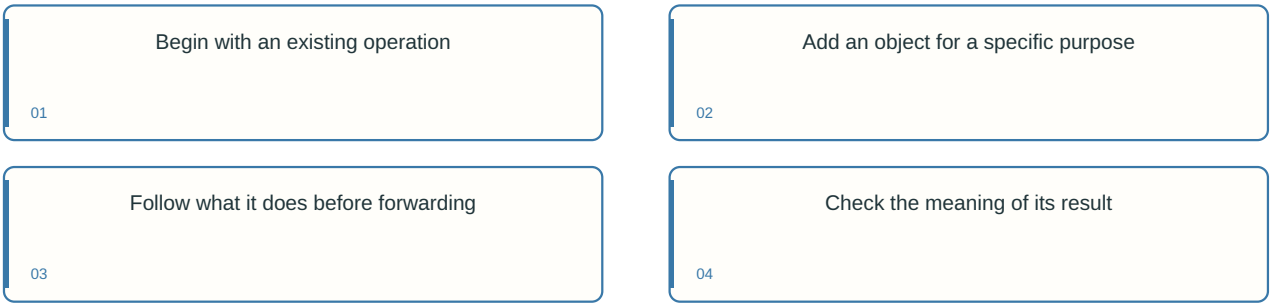


FIG. 6.5.1 **Arrange objects around an existing operation · the example**

Structural patterns arrange objects to translate, extend, group, simplify, share, or control access.

Author-created illustration. Numbers are illustrative unless stated otherwise.

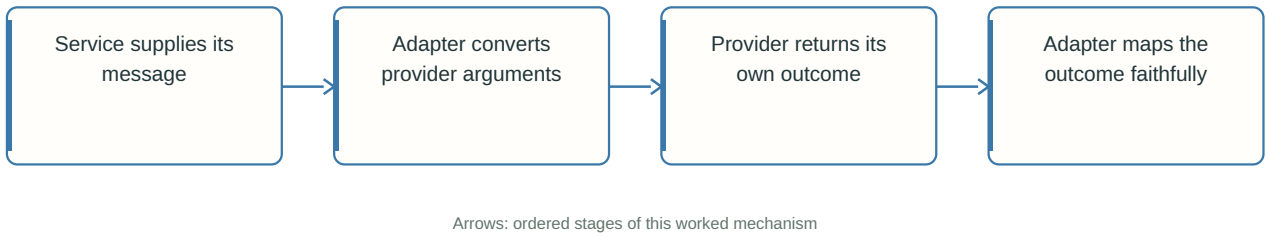
PATTERN / HOW IT WORKS

Adapter

Our service calls `send(message)`, but an older SMS library accepts a phone number, text, and provider-specific options. We want to use that library without making the service learn its interface.

An **Adapter** receives the application's message, converts it into the library's arguments, and calls the library. It then converts the reply into the outcomes expected by `Sender`.

The reply conversion matters as much as the argument conversion. If the provider says “queued,” the adapter must not report “delivered.” When an outcome cannot be represented faithfully, the contract needs to expose that difference. An adapter translates an interface; it cannot create a guarantee the provider does not offer.



Arrows: ordered stages of this worked mechanism

FIG. 6.5.2 **Adapter · the example**

Adapter translates an existing interface and its outcomes into the contract a caller needs.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
send(message):
    raw = sdk.submit(encode(message))
    return map_provider_outcome(raw)
```

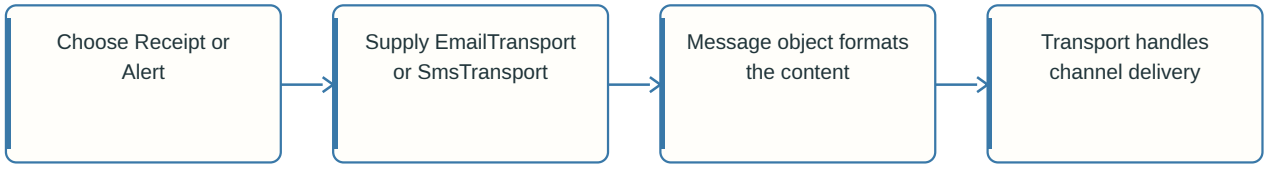
PATTERN / HOW IT WORKS

Bridge

We send both receipts and alerts, and each can travel by email or SMS. A subclass for every combination would give us EmailReceipt, SmsReceipt, EmailAlert, and SmsAlert. A third transport would add still more classes.

With a **Bridge**, the message object holds a transport object. `Receipt` formats receipt data and passes the result to its transport. `Alert` formats alert data in its own way. Either can receive an email or SMS transport through the same sending contract.

The two choices can now vary separately: message behaviour and transport behaviour. Their contract must still fit real constraints. If SMS has a length limit, the design must define splitting or rejection rather than assume any email body can be sent unchanged.



Arrows: ordered stages of this worked mechanism

FIG. 6.5.3 **Bridge · the example**

Bridge keeps two kinds of variation separate by having one object use another through a contract.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
class Receipt:
    constructor(transport): self.transport = transport
    deliver(data):
        text = format_receipt(data)
        return self.transport.send(text)
```

PATTERN / HOW IT WORKS

Composite

A notification can target one person or a group containing people and smaller groups. Callers should not need a separate traversal algorithm for every nesting level.

A **Composite** gives a person and a group the same operation, such as `recipients()`. A person returns their identifier. A group asks its children for their recipients and combines the results. The caller can start with either kind.

Give every person and group a distinct identifier. Our groups may overlap, so the traversal records visited identifiers. A person in two groups then appears once. The same check stops a group cycle from looping forever. This defines membership expansion, not delivery: sending to the resulting people still needs its own per-recipient outcomes and failure handling.

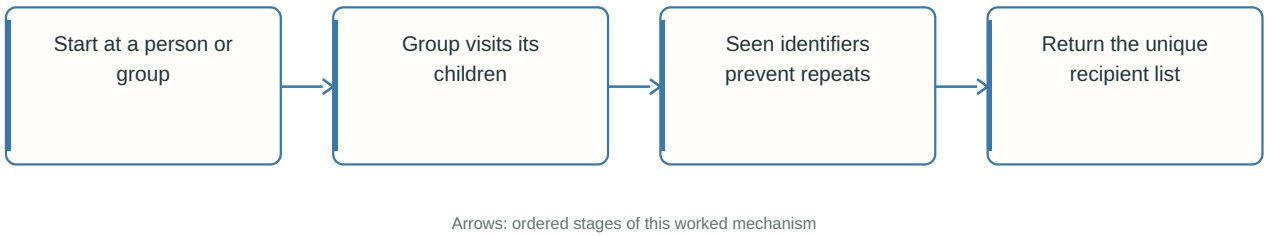


FIG. 6.5.4 **Composite · the example**

Composite lets callers use single objects and groups through the same operation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
Person.recipients(seen):
    if self.id in seen: return []
    seen.add(self.id)
    return [self.id]

Group.recipients(seen):
    if self.id in seen: return []
    seen.add(self.id)
    return flatten(child.recipients(seen)
                   for child in self.children)

people = root.recipients(set())
```

PATTERN / HOW IT WORKS

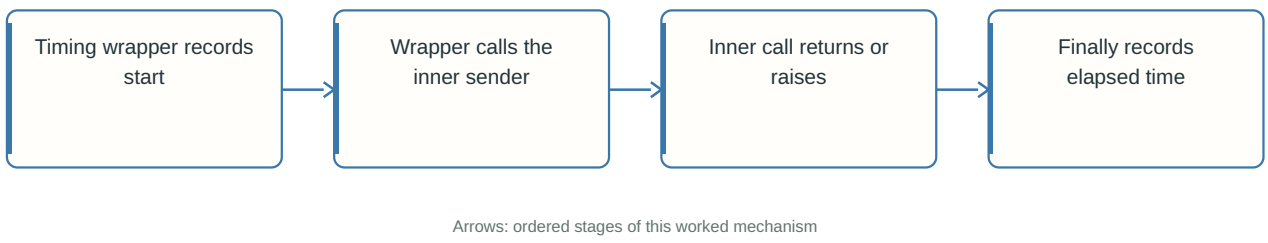
Decorator

We want to time sends without putting timing code into every provider adapter. A `TimingSender` holds another sender and offers the same `send` operation.

It records the start time, calls the inner sender, and records the duration when that call ends. This is a **Decorator**: it adds behaviour around a collaborator while keeping its interface. The `finally` step records duration when the send raises an error too.

Order changes what is measured. `TimingSender(RetrySender(sender))` times the whole call, including retries. `RetrySender(TimingSender(sender))` times each attempt. Before adding retries, define how

repeated attempts avoid unwanted duplicate sends. Wrapping the call does not settle that delivery question.



Arrows: ordered stages of this worked mechanism

FIG. 6.5.5 **Decorator · the example**

Decorator adds behaviour around an operation while preserving its interface; wrapper order can change the meaning.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
send(message):
    start = clock()
    try: return inner.send(message)
    finally: metrics.observe(clock() - start)
```

PATTERN / HOW IT WORKS

Facade

Submitting a notification requires checking its fields, preparing a rendering plan, and saving a delivery request. A caller should not have to coordinate all three helpers for every submission.

A **Facade** offers a focused entry point such as `submit(command)`. It performs those steps in the required order and returns a useful result. The helpers remain responsible for their own work; the facade provides a simpler way to use them together.

Here success means the delivery request was saved. Actual sending happens later, so the facade returns an accepted identifier rather than claiming delivery. A simple entry point must still expose meaningful outcomes, such as invalid input or failure to save the request. Simpler calling code should not require guessing what happened.

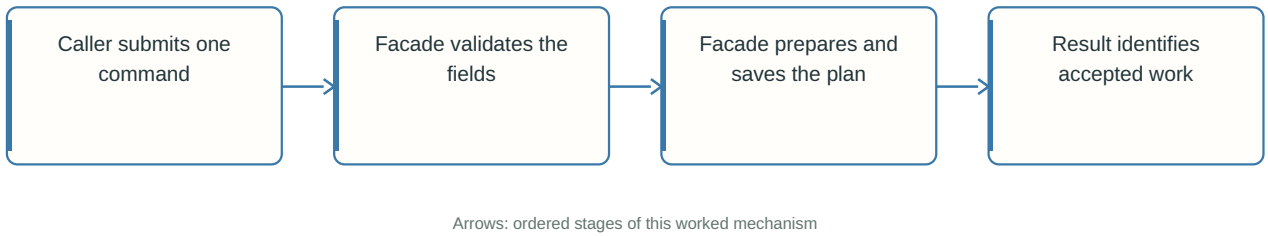


FIG. 6.5.6 **Facade · the example**

Facade provides a focused entry point to several cooperating operations with clear result meanings.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
submit(command):
    validated = validate(command)
    plan = render_plan(validated)
    return intents.persist(plan)
```

PATTERN / HOW IT WORKS

Flyweight

Ten thousand notifications use the same template version. Parsing a separate copy of that template for every message wastes memory.

A **Flyweight** shares the part that is the same across uses. We store one parsed, immutable template for that version. Rendering supplies the recipient's name and other changing values separately. Two messages can use the same template without sharing their personal data.

This separation is essential. Putting a recipient's name into the shared template would let one render change what another sees. Keep shared data unchanged and pass per-message values as separate context. The collection holding shared templates also needs a size or eviction policy if new versions keep arriving; sharing does not make an unlimited collection small.

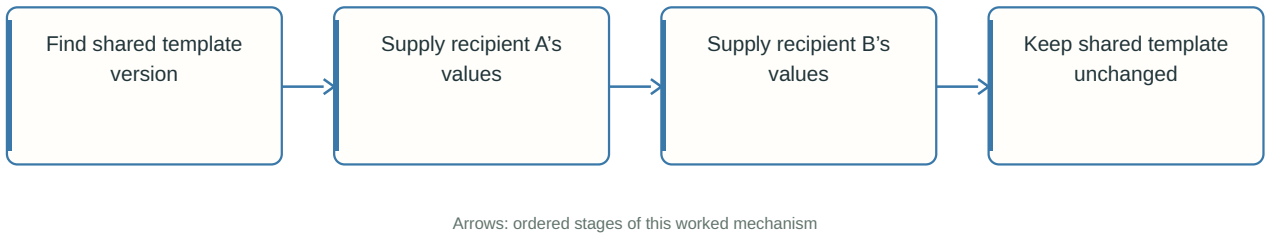


FIG. 6.5.7 **Flyweight · the example**

Flyweight shares suitable unchanged data while each use supplies its own changing context.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
template = parsed_templates.get(version)
output_a = render(template, context_a)
output_b = render(template, context_b)
```

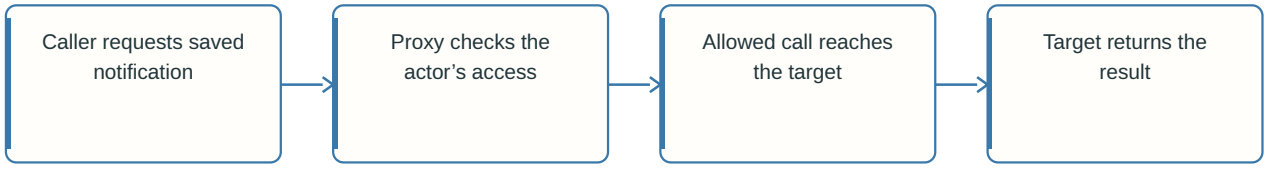
PATTERN / HOW IT WORKS

Proxy

A caller wants to read a saved notification, but it must first have permission. We can place an object with the same read operation in front of the actual storage object.

This **Proxy** checks the actor's access and only then calls the target. The caller uses the familiar read interface. The proxy controls access to the object behind it.

Other proxies can delay creating an expensive target or represent a remote target. Their purpose should be clear to the caller where it affects behaviour. A remote read can time out even though an ordinary local field read would not. Our access proxy must also prevent callers from bypassing its check through another unguarded route.



Arrows: ordered stages of this worked mechanism

FIG. 6.5.8 **Proxy · the example**

Proxy represents a target through its interface while controlling access, creation, or location.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
read(notification_id, actor):
    authorize(actor, notification_id)
    return target.read(notification_id)
```

Remember: An object’s purpose identifies the pattern. Translating an interface, adding timing, and controlling access make different promises even when their diagrams look similar.

Try applying it: Which measures the whole call: TimingSender(RetrySender(sender)) or RetrySender(TimingSender(sender))? What must be defined before either wrapper retries a send?

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Behavioral patterns

Prerequisites: 6.5 · Structural patterns 11 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

Choosing a renderer differs from advancing a notification from Draft to Queued. Telling listeners about an event differs from recording a command for later. These examples give each behaviour a clear place and show how the calls proceed.

6.6.1 · Give each behaviour a clear home

A notification service must choose how to render a message, tell interested listeners that it was accepted, and permit actions that fit its current status. These are different decisions.

Behavioural patterns help organise such decisions and the calls between objects. The examples below show where a chosen algorithm lives, how listeners are called, how an action is represented, and how an object's history changes its permitted actions.

Some patterns are well expressed with functions or small tables. Others benefit from separate objects. Follow the changing rule first, then choose a representation that makes that rule easy to read. Adding a class for every decision is not a requirement.

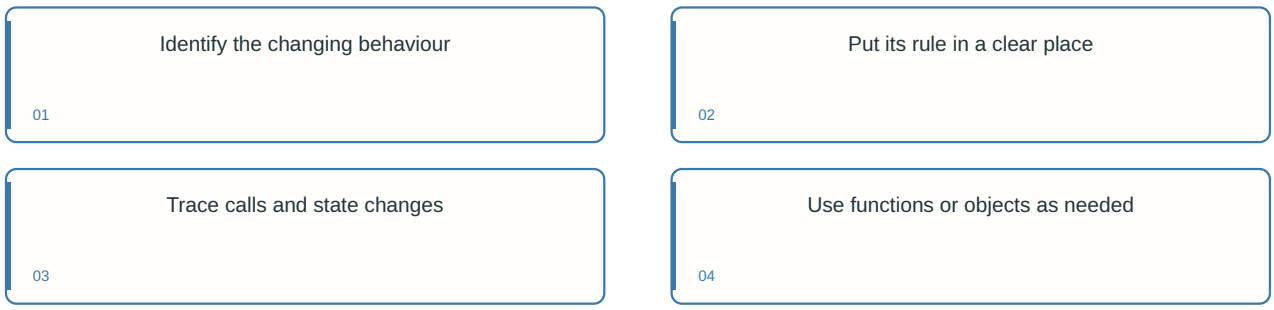


FIG. 6.6.1 **Give each behaviour a clear home · the example**

Behavioural patterns organise decisions, actions, reactions, and coordination between objects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

PATTERN / HOW IT WORKS

Strategy

Our service renders a message as plain text or HTML. The caller chooses a format, but the service should not contain the details of every rendering algorithm.

A **Strategy** puts each algorithm behind the same operation, such as `render(data)`. The service receives the chosen renderer and calls it. Supplying another renderer changes how the content is produced without changing the calling steps.

The shared contract must specify the output. An HTML result needs to be identified and escaped appropriately; it cannot silently be treated as safe plain text. A function can serve as a strategy when one operation is enough. The useful separation is the replaceable algorithm, rather than a required class hierarchy.

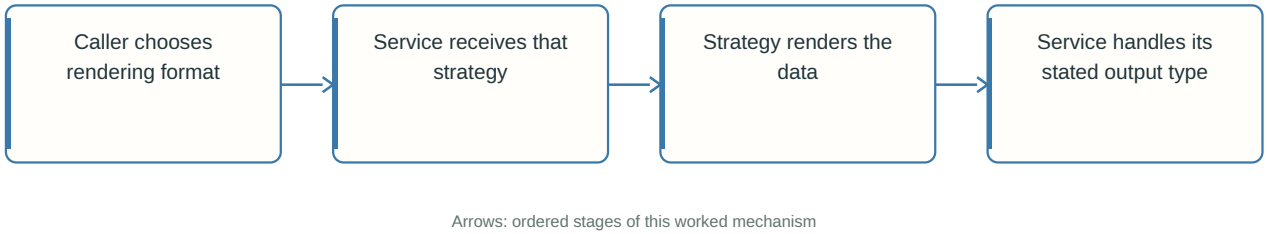


FIG. 6.6.2 **Strategy · the example**

Strategy supplies a replaceable algorithm through a shared operation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
renderer = renderers.for_format(command.format)
content = renderer.render(command.data)
require(content.format == command.format)
```

PATTERN / HOW IT WORKS

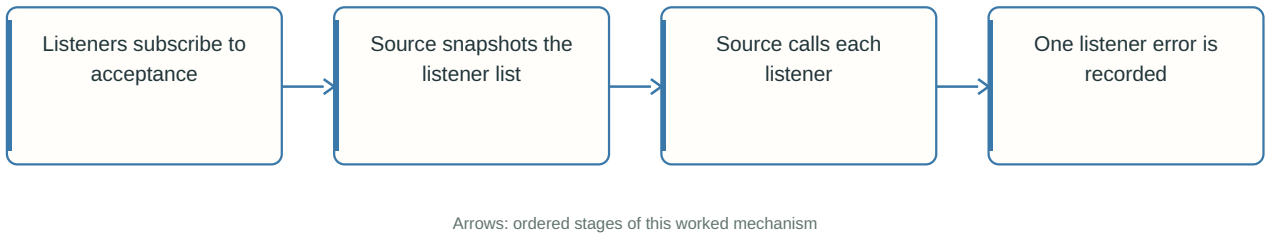
Observer

After accepting a notification, the service updates a counter and refreshes an on-screen list. We do not want the acceptance code to call every interested component by name.

With **Observer**, interested objects subscribe to an event source. When acceptance occurs, the source calls the registered listeners with the event. A subscription records this interest; removing it ends that interest.

In this example, callbacks run synchronously in registration order. The source copies the listener list before calling them, so changes during a callback affect the next event. It records a listener's error and continues to the next listener.

These are chosen rules, not automatic guarantees of Observer. A process crash can lose an in-memory event, and a slow callback delays this call. Delivery that must survive a restart needs saved event data and a recovery mechanism.



Arrows: ordered stages of this worked mechanism

FIG. 6.6.3 **Observer · the example**

Observer calls subscribed listeners when an event occurs under explicit timing and failure rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
for listener in snapshot(subscribers):
    try: listener.on_accepted(event)
    except Exception as error: record_listener_error(error)
```

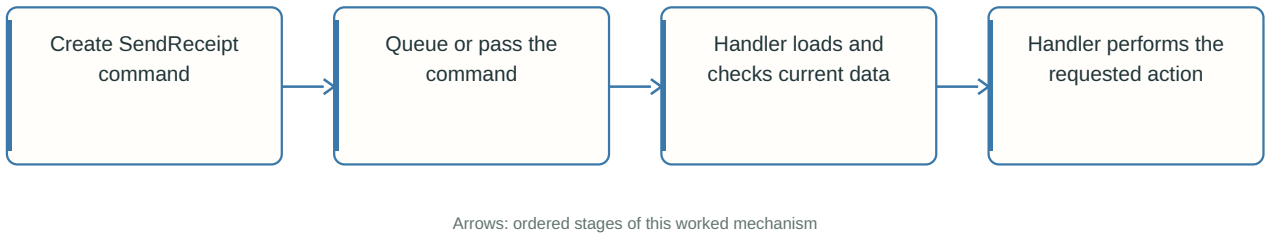
PATTERN / HOW IT WORKS

Command

A worker may need to send a receipt later. Instead of immediately calling a sender, the application can create a value describing the requested action: `SendReceipt(order_id, operation_id)`.

This value is a **Command**. A handler reads it, loads the required order, checks the permitted action, and performs the work. The request is separate from its execution. That makes it possible to queue or record the request.

The command should contain stable data, not an open request connection. If it is saved for later, its format also needs to remain readable by the worker. Replaying the same command can repeat an email unless the handler and provider use a defined duplicate-handling protocol. Recording an intention does not itself make retries safe.



Arrows: ordered stages of this worked mechanism

FIG. 6.6.4 **Command · the example**

Command represents a requested action as data so its execution can be handled separately.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
command = SendReceipt(order_id, operation_id)
handler.handle(command)
# Handler obtains its own valid resources
```

PATTERN / HOW IT WORKS

State

A draft notification can be edited. Once it has been submitted, editing that same delivery request is no longer allowed. After confirmed delivery, sending it again is also a different action.

The **State** pattern puts behaviour for each lifecycle stage in a state object. The notification delegates an action to its current state. `Draft.submit` can save the accepted request and move to `Queued`. `Queued.edit` rejects the edit. The object's history determines which behaviour is active.

State changes need evidence: a delivery confirmation can permit the transition to Delivered, while an unverified caller request cannot. A small transition table may be simpler than separate state classes. Either form must enforce the same allowed changes. Unlike selecting a rendering strategy, selecting a convenient state cannot bypass those lifecycle rules.

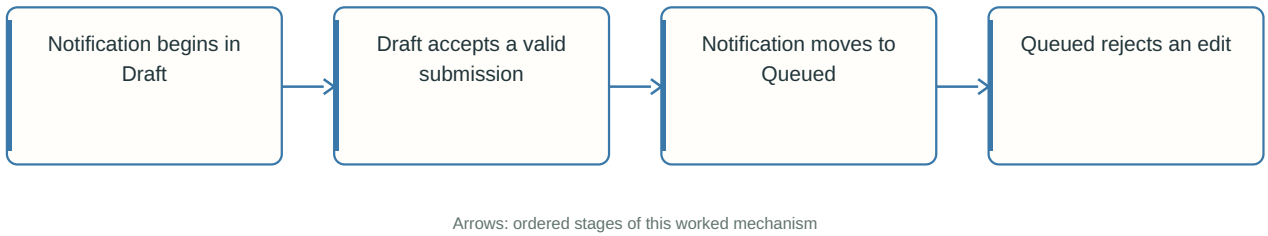


FIG. 6.6.5 **State · the example**

State changes an object's permitted behaviour as its lifecycle advances through allowed transitions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
submit(notification):
    return notification.state.submit(notification)

Draft.submit(notification):
    save_submission_and_state(notification, next_state="Queued")
Queued.submit(notification): raise AlreadySubmitted
```

PATTERN / HOW IT WORKS

Chain of Responsibility

Before a notification request reaches the service, we check its caller, enforce a request limit, and parse its input. Each step can either stop the request or pass it onward.

A **Chain of Responsibility** connects these handlers in order. Authentication first rejects an unknown caller. An allowed caller reaches the request-limit check. A request still allowed then reaches parsing and the application.

Each handler must say when it calls the next one. Forgetting that call can swallow a request; calling it twice can repeat the work. Order matters too: rejecting an over-budget request before expensive parsing avoids work we did not intend to admit. This chain organises calls in one execution. It does not save progress for recovery after a process crash.

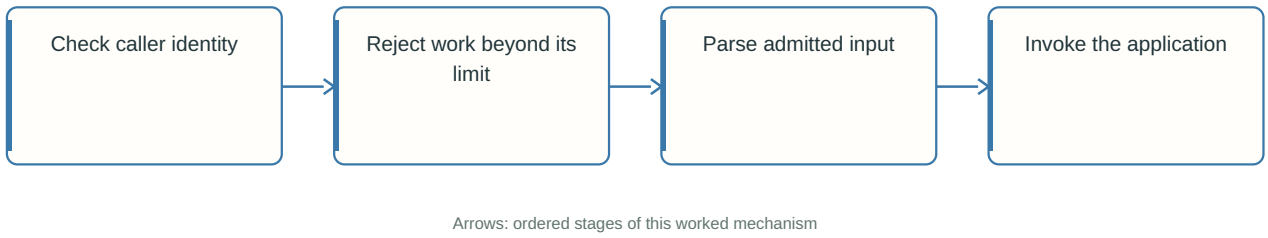


FIG. 6.6.6 **Chain of Responsibility · the example**

Chain of Responsibility passes work through handlers that explicitly stop or continue the request.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
handler = auth(rate_limit(parse(application)))
response = handler(request)
# Each wrapper either returns a rejection or calls next once
```

PATTERN / HOW IT WORKS

Template Method

Every notification preparation must validate input, render content, and save the prepared result. Email and SMS use different rendering steps, but both need that same order.

A **Template Method** defines the shared sequence in a base class. Its `prepare` operation calls `validate`, the overridable `render`, and then `save`. An email subclass supplies HTML rendering; an SMS subclass supplies text rendering. Callers use the shared `prepare` entry point.

The extension rules matter. The subclass should replace the permitted rendering step without bypassing validation. Keep resource cleanup around the whole operation if any step acquires resources. When several steps vary independently, passing separate strategies may be easier than creating a subclass for every combination.

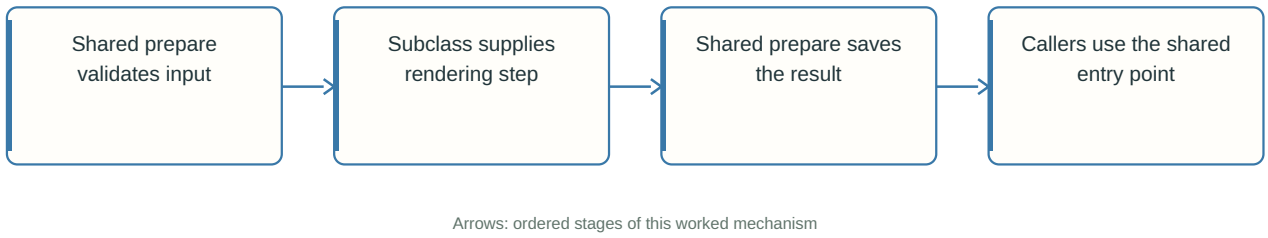


FIG. 6.6.7 **Template Method · the example**

Template Method fixes a sequence of steps while letting subclasses replace selected steps.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
class Preparation:
    prepare(input):
        validate(input)
        content = self.render(input)
        return save(content)
    render(input): raise NotImplemented

class SmsPreparation(Preparation):
    render(input): return render_sms_text(input)
```

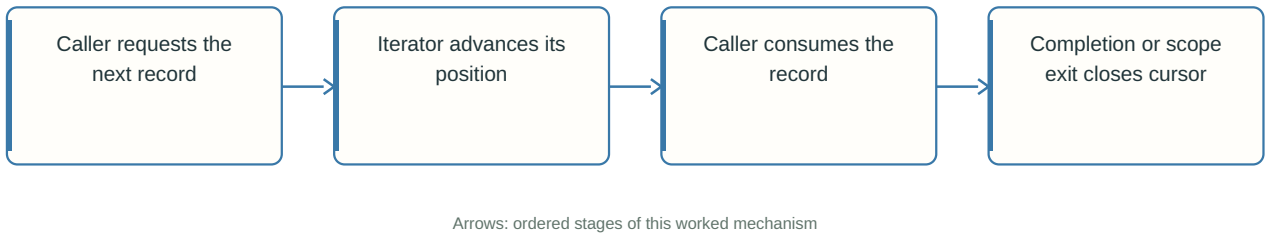
PATTERN / HOW IT WORKS

Iterator

A caller wants to read saved notifications one at a time. It should not have to know whether they are stored in an array, a tree, or rows fetched from a database.

An **Iterator** keeps the traversal position and supplies the next item until there are no more. A loop asks for each item through that operation. The contract should state the ordering and what happens if the underlying collection changes.

A database iterator may fetch rows only as the caller advances. It can therefore keep a connection open throughout the loop. Use it inside an owning resource block so leaving early or raising an error closes the cursor. For a small bounded result, fetching a list first may let the connection close sooner.



Arrows: ordered stages of this worked mechanism

FIG. 6.6.8 **Iterator · the example**

Iterator supplies items in sequence while hiding the traversal details and respecting resource lifetimes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
with open_bounded_cursor(query) as rows:
    for row in rows:
        consume(row)
```

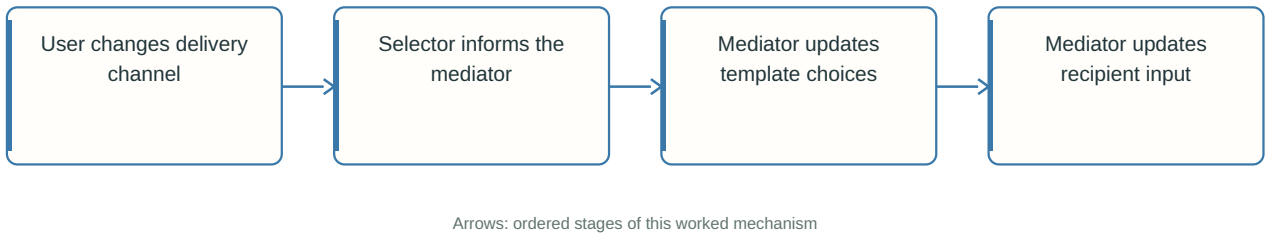
PATTERN / HOW IT WORKS

Mediator

In a notification editor, changing the channel affects the template choices and the recipient field. If every control calls all the others, each must know much of the form.

A **Mediator** coordinates these interactions. The channel selector reports its change to a form coordinator. The coordinator updates the template choices and recipient field. The controls know the coordinator instead of knowing one another's detailed APIs.

The coordinator owns this form's interaction rules. It should not grow into the owner of every application rule. Validation and saving can still live in their own collaborators. Mediator simplifies who communicates with whom; it does not remove the need to understand the order and effects of those updates.



Arrows: ordered stages of this worked mechanism

FIG. 6.6.9 **Mediator · the example**

Mediator places a focused coordination rule between collaborators instead of linking each directly to the others.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
channel_changed(channel):
    templates.show_for(channel)
    recipient_field.use_format(address_format(channel))
```

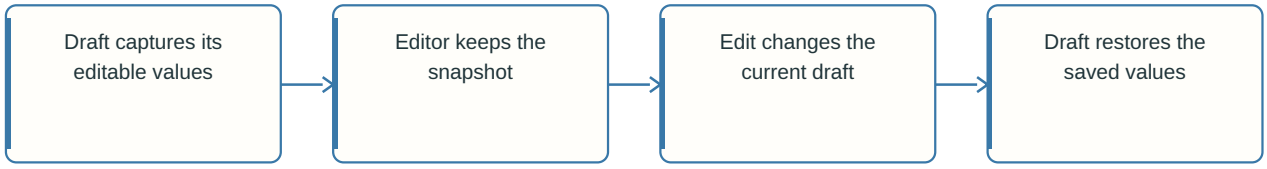
PATTERN / HOW IT WORKS

Memento

A person edits a notification draft and wants to undo the last edit. Before the change, the draft can provide a snapshot of the state it supports restoring.

That snapshot is a **Memento**. The editor keeps it without rewriting the draft's internal fields. If the edit is unwanted, the editor gives the snapshot back to `restore`. The draft decides which values to restore and how.

The saved values must not change along with the draft. A snapshot that only points to the same editable paragraph list would fail this test; the draft must copy its owned editable data. Restoring this local draft cannot unsend a notification already delivered. Undoing values and reversing an external effect are different operations.



Arrows: ordered stages of this worked mechanism

FIG. 6.6.10 **Memento · the example**

Memento captures supported local state so its owner can restore it later.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
before = draft.snapshot_owned_values()
draft.apply(edit)
draft.restore(before)
# Restores local draft data, not external deliveries
```

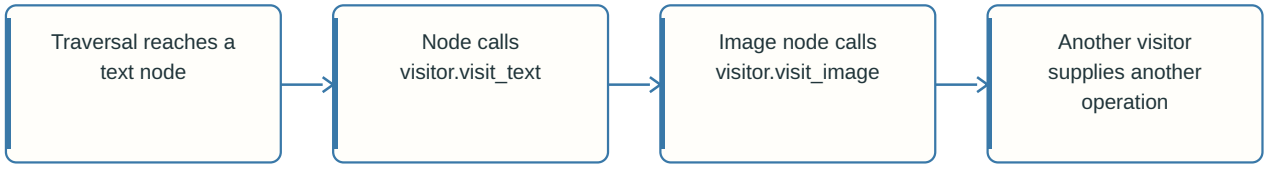
PATTERN / HOW IT WORKS

Visitor

A message document contains text and image nodes. We want to export both kinds to HTML, then add another operation that checks accessibility. Putting every new operation into every node class would spread the work.

A **Visitor** groups the handling for one operation. During traversal, a text node calls `visit_text` and an image node calls `visit_image` on the supplied visitor. An HTML visitor provides export behaviour; an accessibility visitor provides checks.

This works well when the node kinds are fairly stable and new operations are common. Adding a new node kind, such as video, means updating visitors that must support it. Unknown kinds need an explicit result rather than silently disappearing. Pattern matching can offer a simpler alternative in languages that support it.



Arrows: ordered stages of this worked mechanism

FIG. 6.6.11 **Visitor · the example**

Visitor groups an operation across known element kinds; adding a kind can require changes to many visitors.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
TextNode.accept(visitor): return visitor.visit_text(self)
ImageNode.accept(visitor): return visitor.visit_image(self)

class HtmlVisitor:
    visit_text(node): return escape_html(node.text)
    visit_image(node): return render_image(node)
```

Remember: Identify the rule or interaction first. Then use a function, table, or object arrangement that makes its order and outcomes clear.

Try applying it: A caller can choose HTML rendering but cannot simply choose Delivered as a notification's state. Why does the second choice require different rules?

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   Access reviewed · 21 October 2026

Enterprise patterns & boundaries

Prerequisites: 6.6 · Behavioral patterns 5 original figures

Read the labels and caption with each figure. Boxes show parts or states; arrows show how they are connected.

Accepting a notification saves both its message and its delivery job. A progress screen then reads their status. We will separate data access, readiness rules, grouped changes, and read models, while keeping the database's actual commit boundary clear.

6.7.1 · Separate data access, rules, and commit decisions

Accepting a notification may mean saving both its message and its delivery job. The application also needs rules about which notifications are valid, and a screen needs to list their progress.

These needs lead to different boundaries. A **Repository** supplies application-level data operations. A **Specification** expresses a rule as a test. A **Unit of Work** tracks related changes so they can be saved together. **CQRS** separates the models used to change data and to read it when their needs differ.

We will examine each below. They can all be used within one application and one database. What matters is which work each part owns. In particular, saving related database changes together does not make an email provider part of that same transaction.

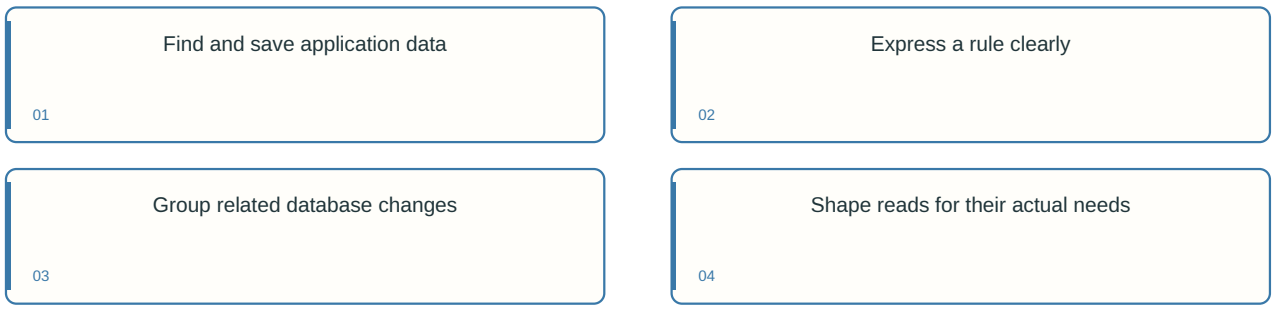


FIG. 6.7.1 **Separate data access, rules, and commit decisions · the example**

Data access, rule evaluation, grouped changes, and read models solve different problems.

Author-created illustration. Numbers are illustrative unless stated otherwise.

PATTERN / HOW IT WORKS

Repository

The service needs to find a saved notification and change its delivery status. It should not have to know the database's table names to do this.

A **Repository** offers data operations in application terms, such as `get(notification_id)` and `save(notification)`. Its implementation performs the storage queries. The service loads the notification, applies the delivery rule, and asks the repository to save the changed value.

The contract must explain what these calls promise. A missing identifier needs a defined result. Concurrent changes need a conflict rule. Loading and saving do not automatically form one transaction merely because they use the same repository. In this example, a Unit of Work supplies that transaction. A test repository can check service decisions, but only real storage checks establish actual database behaviour.

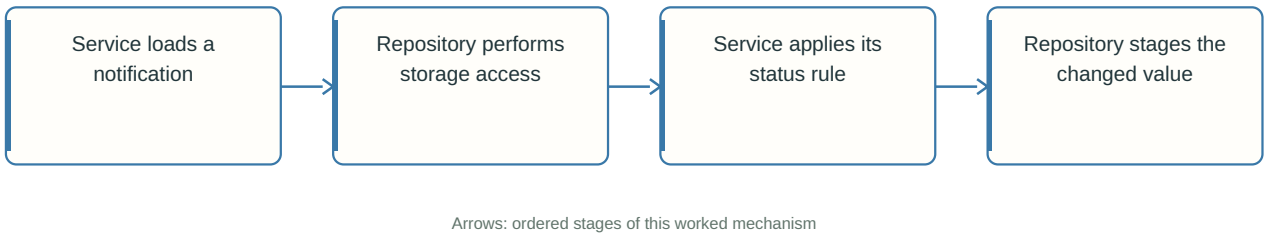


FIG. 6.7.2 **Repository · the example**

Repository supplies application-level operations for finding and saving data under an explicit storage contract.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
with unit_of_work() as uow:
    notification = uow.notifications.get(notification_id)
    notification.record_delivery(verified_receipt)
    uow.notifications.save(notification)
    uow.commit()
```

PATTERN / HOW IT WORKS

Unit of Work

Accepting a notification adds a message record and a delivery-job record. Saving only the message would leave accepted work with no job to send it.

A **Unit of Work** tracks the records added, changed, or removed during an operation and coordinates saving them. In this example, both repositories use one transaction in the same database. `commit()` writes the pending changes and asks the database to commit them together. If the operation exits before a successful commit, the owning block rolls back unfinished work and releases its resources.

This groups the local changes. Calling an email provider inside the block would not put the provider into that database transaction. Database rollback cannot retract an email. The boundary follows the actual participating storage, not the indentation of the code.

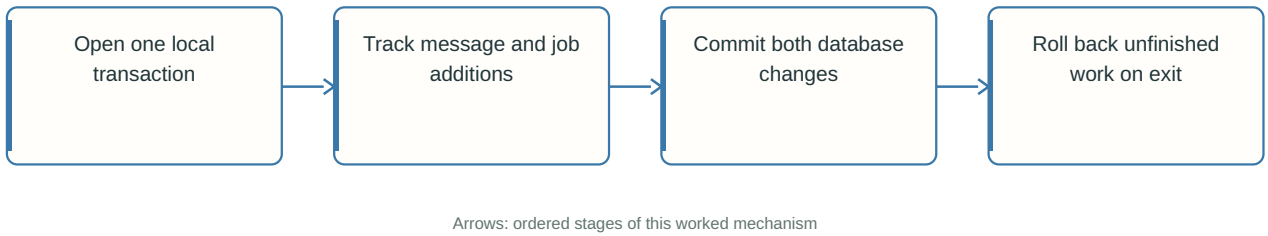


FIG. 6.7.3 **Unit of Work · the example**

Unit of Work tracks related data changes and coordinates writing them within a defined transaction boundary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
with unit_of_work() as uow:
    uow.notifications.add(notification)
    uow.delivery_jobs.add(job)
    uow.commit()
```

PATTERN / HOW IT WORKS

Specification

A scheduled notification is ready only if its scheduled time has arrived and it has not been cancelled. We want this rule to have one clear name.

A **Specification** expresses a condition as a test, such as `ReadyToSend.satisfied_by(notification, now)`. A test returning true or false is a **predicate**. Small specifications can be combined when the application needs reusable combinations of rules.

A true result describes the values just tested. Another caller could cancel the notification before we claim it for sending. The final update must therefore recheck the required current conditions. Here `claim_if_still_ready` checks the saved version and readiness rule atomically, and returns no claim if either changed. A plain function is enough when no rule-composition machinery is needed.

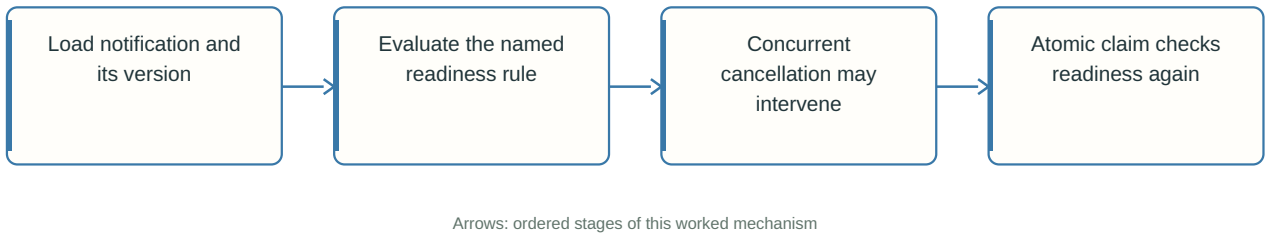


FIG. 6.7.4 **Specification · the example**

Specification names a reusable condition; changing data later still requires checking the current rule.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
if ReadyToSend.satisfied_by(notification, now):
    claim = repository.claim_if_still_ready(
        notification.id, notification.version, now)
    # No claim means the current conditions did not permit it
```

PATTERN / HOW IT WORKS

CQRS

Submitting a notification needs rules that prevent invalid changes. A progress screen needs a convenient list of recipients, times, and statuses. Those jobs may benefit from different data models.

Command Query Responsibility Segregation, or **CQRS**, separates the model used for changes from the model used for reads. A command goes through the change rules. A query reads the view prepared for its needs.

A process called a **projection** can update the reading view from accepted changes. If it runs later, the screen may briefly lag behind a successful command. The application must explain that delay and arrange recovery when projection work fails.

The models can share one process and database. CQRS does not require saving every change as an event or deploying separate services. When one simple model serves both jobs well, maintaining two models may add unnecessary work.

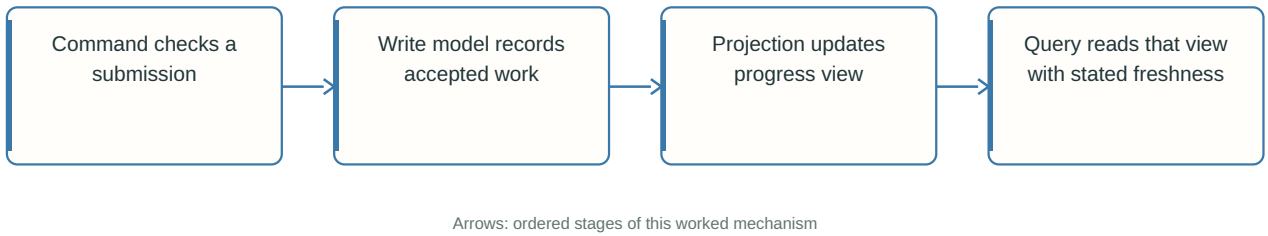


FIG. 6.7.5 **CQRS · the example**

CQRS separates models for changing and reading data when their different needs justify the extra work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

```
accepted = commands.submit_notification(command)
rows = queries.notification_progress(account_id)
# A separately updated progress view may lag behind acceptance
```

Try the mechanism in code

Let us model the three reuse rules in one Python process. `Container` keeps one singleton. Each `Scope` keeps its own scoped resource, while `transient()` returns a new object on every call. These classes are a small teaching model, not a framework container.

The resource has a `closed` flag. Leaving a `with` block closes it, including when the block raises an exception. We keep one reference after scope A ends to test whether the reference also keeps the resource usable.

PYTHON · IDENTITIES AND SCOPE CLEANUP

```
class Resource:
    def __init__(self):
        self.closed = False

    def use(self):
        if self.closed:
            raise RuntimeError("resource is closed")
        return "in use"

    def close(self):
        self.closed = True


class Container:
    def __init__(self):
        self.singleton = object()

    def scope(self):
        return Scope(self)


class Scope:
    def __init__(self, container):
        self.container = container
        self.scoped = Resource()

    def __enter__(self):
        return self

    def __exit__(self, error_type, error, traceback):
        self.scoped.close()

    def transient(self):
        return object()


container = Container()
with container.scope() as a:
    retained = a.scoped
    shared = a.container.singleton
    assert a.scoped is a.scoped
    assert a.transient() is not a.transient()
    assert a.scoped.use() == "in use"
assert retained.closed
```

```
try:
    retained.use()
except RuntimeError as error:
    assert str(error) == "resource is closed"
else:
    raise AssertionError("closed resource was usable")

try:
    with container.scope() as b:
        second = b.scoped
        assert second is not retained
        assert b.scoped is b.scoped
        assert b.container.singleton is shared
        raise ValueError("operation failed")
except ValueError:
    pass
assert second.closed
print("Identity and cleanup checks passed")
```

Inside scope A, two reads of `scoped` return the same resource. Two calls to `transient()` create different objects. Scope B has a different scoped resource, while both scopes refer to the same container singleton.

On leaving scope A, `retained` still points to its resource, but `use()` raises an error because the resource is closed. Scope B also closes its resource when the operation raises `ValueError`. The original operation error reaches the caller because `__exit__` does not suppress it.

This demonstrates object identity and two cleanup paths. The model is single-threaded and does not implement database transactions, asynchronous cancellation, or a real DI framework.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · A saved notification job runs after its creating request and user session have ended. How should it obtain resources and permission to act?

Save the necessary job data and identifiers rather than an open connection or session object. The worker opens its own scope, obtains current resources, loads the data, performs the work, and releases those resources.

The application also needs an explicit permission rule. Some jobs carry out work validly accepted earlier; others must check current permission before acting. Choose that rule for this operation. An old session object cannot silently stand for permission that remains current forever.

Remember: Repository exposes data operations, Specification names a condition, Unit of Work coordinates related writes, and CQRS separates change and read models when needed.

Try applying it: One database transaction saves a notification and its job. The code also calls an email provider. Which effects can a database rollback reverse, and which need their own recovery rules?

Unit 6 exercises → · Unit 6 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   `Access reviewed · 21 October 2026`

2. The Catalog of Design Patterns — Refactoring Guru ↗

   Engineering teaching reference. Pattern taxonomy; miniature examples and judgments are original to this atlas.

   `Access reviewed · 21 October 2026`

PART 2 / UNIT 07

Application Architecture and Domain-Driven Design

A shopping application can separate input, policy, and storage without putting them on different machines. Compare architecture styles by their dependency rules and the changes they help contain.

The running example

Design an online shopping application first as a modular monolith, then examine the conditions under which selected domains should become independent services.

Unit 7 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

View	Boundary rule	Runtime implication
Layered	Separate technical roles	Can remain one process
Hexagonal	Ports express app needs	Adapters call in or out
Clean/Onion	Policy dependencies inward	Runtime effects still reach adapters

FIG. 7.0.1 **Layered vs Hexagonal**

The styles organize source responsibilities, not mandatory physical deployment layers.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Design	Deployment	Main cost
Modular monolith	One release unit	Must enforce internal ownership
Microservices	Independent processes	Network, versions, operations
Distributed monolith	Many coupled processes	Costs without independence

FIG. 7.0.2 **modular monolith vs microservices**

Choose according to actual independent operation and change needs.

Author-created illustration. Numbers are illustrative unless stated otherwise.

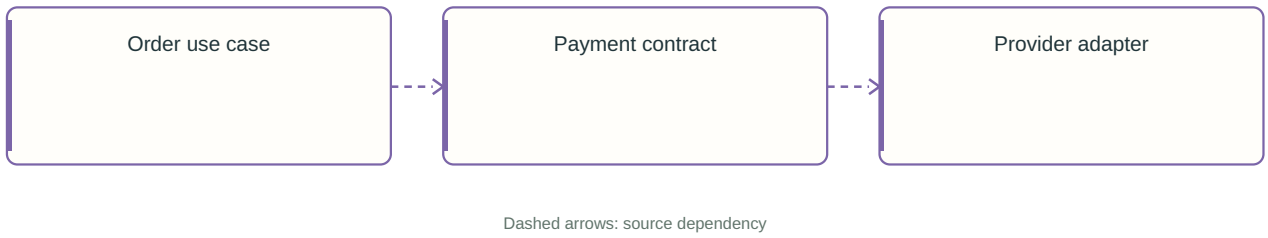


FIG. 7.0.3 **dependency arrows**

Dependency arrows express knowledge; runtime execution reaches the selected adapter.

Author-created illustration. Numbers are illustrative unless stated otherwise.

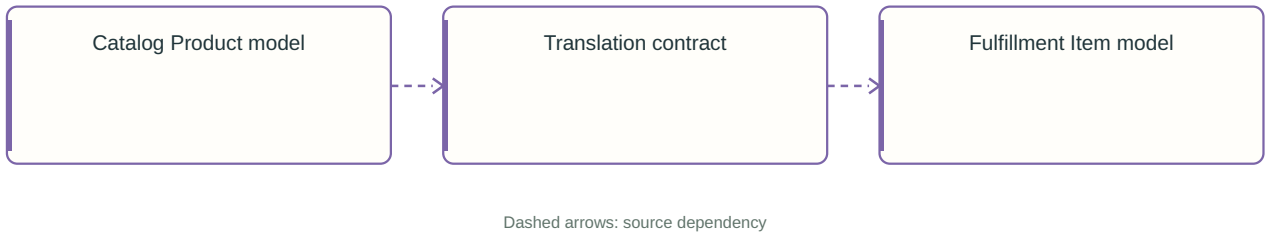


FIG. 7.0.4 **bounded context map**

Bounded contexts retain different valid meanings rather than sharing every field globally.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Layers, monoliths & ports

Prerequisites: Object-Oriented and Modular Software Design, Design Patterns, Dependency Injection, and Object Lifecycles

6 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A shopping application can separate input, policy, and storage without putting them on different machines. Compare architecture styles by their dependency rules and the changes they help contain.

7.1.1 · Layered/N-tier architecture

A shopping request arrives through a controller, invokes an application operation, applies pricing rules, and stores the order. Layered architecture gives these different kinds of work clear places: presentation, application coordination, domain policy, and infrastructure.

The layers can all run in one process. Deploying them as separate physical tiers is another choice. Use the division to protect useful rules: a controller should not bypass order validation simply because it can reach an SQL helper. Follow a change, such as a new pricing rule, to see whether the boundary helps. Classes that only forward every call add work without necessarily hiding any detail.

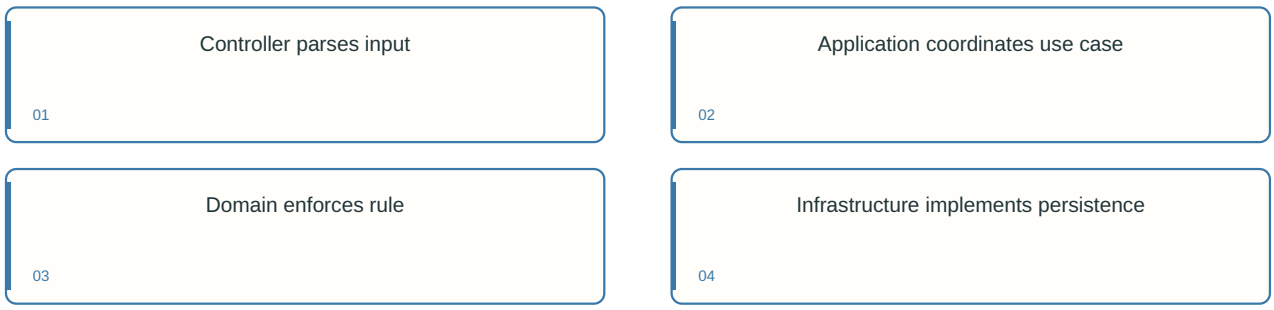


FIG. 7.1.1 **Layered/N-tier architecture · the example**

Layered architecture separates kinds of responsibility; physical tiers are a separate deployment choice.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.1.2 · Modular monoliths

Catalog, orders, and payments can be modules of one deployable application. Each exposes operations and owns its data, while calls between them remain local. This is a modular monolith.

The word “modular” matters. If every module writes every table, changing a payment record can still break the whole program. Use agreed interfaces and enforce dependency rules through review or useful automated checks. This arrangement keeps deployment and local transactions relatively simple while giving business responsibilities separate homes. Later extraction remains possible, but it is an option supported by the boundaries, rather than a benefit obtained automatically by naming folders.

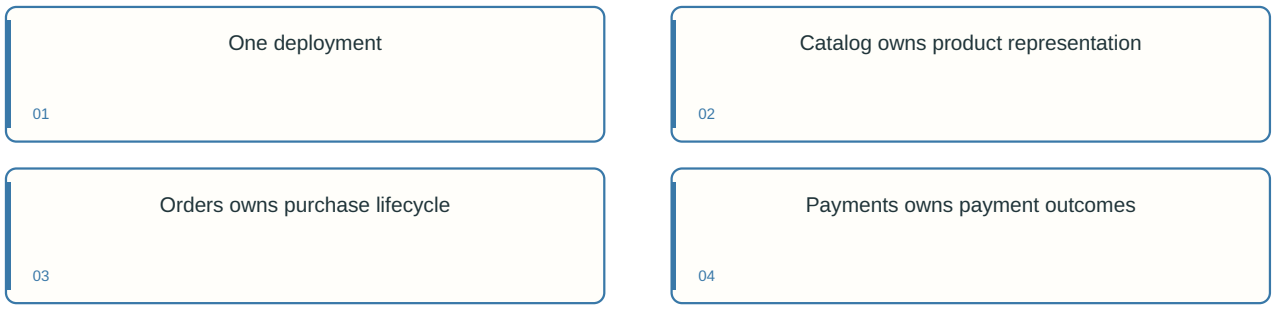


FIG. 7.1.2 **Modular monoliths · the example**

A modular monolith has one release unit with enforced internal contracts and data ownership.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.1.3 · Monolith vs microservices

Suppose inventory needs to scale independently and has a team ready to operate it. Making it a separate service may provide a useful deployment and ownership boundary. It also replaces some local calls with network calls that can time out or partially fail.

Microservices aim for useful independent deployment and operation. A monolith can be well organised, while several services can remain tightly coupled. If five services must release together for each change, ask how much independence was gained. Compare the motivation for extraction with the added work in data coordination, compatibility, monitoring, and recovery. A process boundary should solve a demonstrated problem.

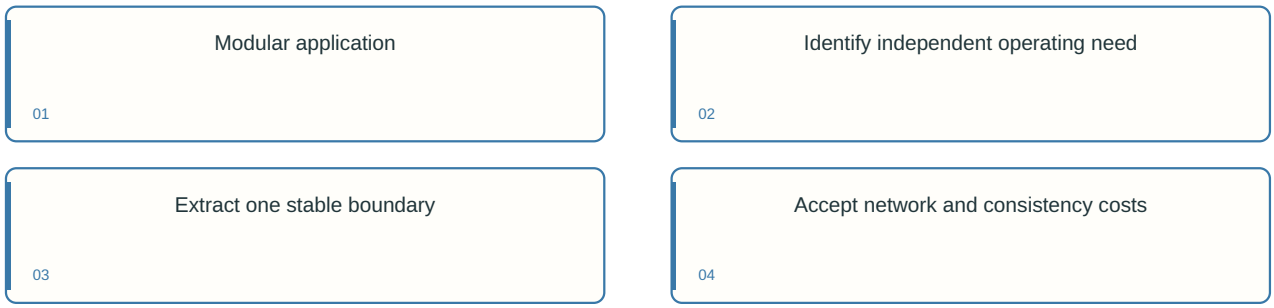


FIG. 7.1.3 **Monolith vs microservices · the example**

Microservices seek useful independent deployment and operation while adding network, version, and coordination work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.1.4 · Clean Architecture

An order rule should be testable without constructing an HTTP response or starting a web framework. Clean Architecture supports this by making framework and storage details depend on interfaces used by the application's policy.

The inward arrows in its usual diagrams describe source dependencies. During execution, a use case can still call an outer storage adapter through an injected interface. Avoid making core rules depend on ORM-specific records or transport response objects. The separation costs mappings and contracts, so choose a proportionate version for a small application. The important question is whether policy can be understood and exercised without unrelated infrastructure details.

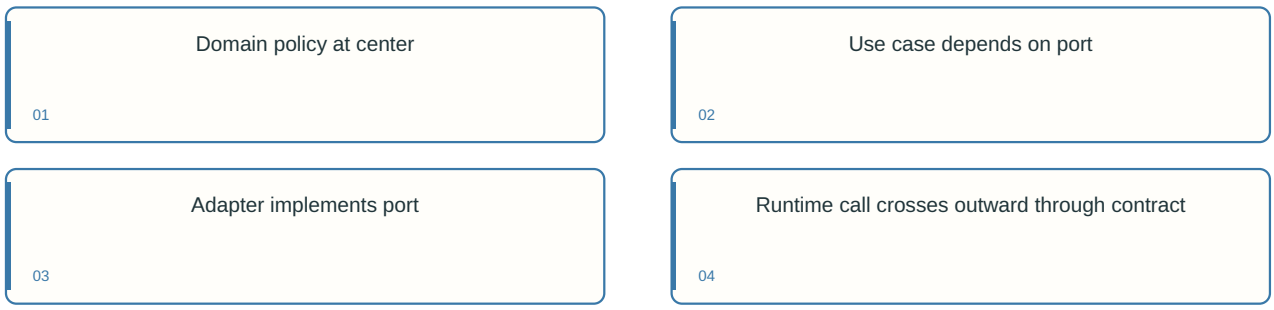


FIG. 7.1.4 **Clean Architecture · the example**

Clean Architecture directs source dependencies toward policy through application-owned abstractions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.1.5 · Hexagonal/Ports and Adapters

An order can be created from a web request or a command-line tool. Both adapters can invoke the same application operation. Likewise, an SQL adapter and a test repository can satisfy the application's storage needs.

Hexagonal architecture calls these application-facing contracts ports and their implementations adapters. An inbound port expresses an operation the application offers; an outbound port expresses work it needs from elsewhere. Design ports around those needs rather than copying an entire vendor API. The drawing does not imply separate machines. Test application decisions with controlled adapters, then separately check real encoding, transactions, and failure behaviour at the external boundaries.

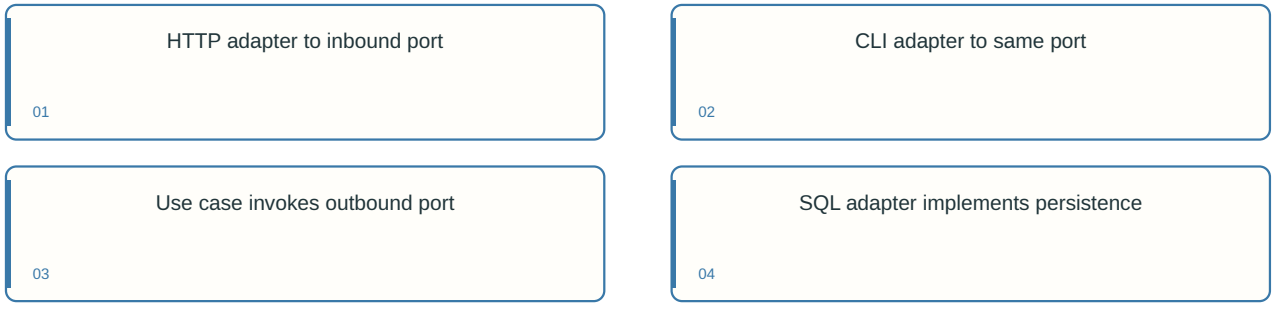


FIG. 7.1.5 **Hexagonal/Ports and Adapters · the example**

Hexagonal architecture uses application ports and adapters to separate core operations from external mechanisms.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.1.6 · Onion Architecture

Onion Architecture arranges domain policy, application services, and infrastructure so that dependencies point toward the core rules. An order rule can then use an application-owned repository contract rather than import database-specific code.

Its vocabulary overlaps with Clean and Hexagonal Architecture, and teams draw the layers differently. Compare the actual dependency and ownership rules instead of treating the shape as a guarantee. Can pricing run without a web server? Does a failed write remain visible to the caller? These questions reveal whether the separation helps. Extra layers are worthwhile when they protect such properties, rather than simply matching a diagram.

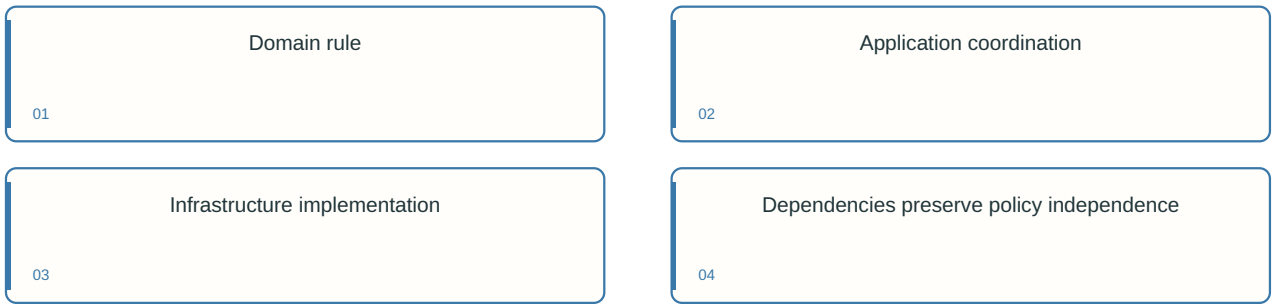


FIG. 7.1.6 **Onion Architecture · the example**

Onion Architecture organises inward policy dependencies; concrete contracts matter more than diagram shape.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What evidence would justify extracting payments into a separate service?

Look for stable capability ownership, different scaling or isolation needs, and an operating team prepared for network and version failures. Compare those benefits with added data coordination and recovery. Preserve existing payment invariants and histories through migration. A separate deployment is useful only when the required independence can actually be exercised.

Remember: Code boundaries and deployment boundaries solve related but different problems.

Try applying it: What benefit would justify turning a well-separated module into a remote service?

Unit 7 exercises → · Unit 7 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Architecture styles — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Style comparisons; provider-specific recommendations are not mandatory in the atlas.

   Access reviewed · 21 October 2026

2. Bounded Context — Martin Fowler, 2014 ↗

   Original engineering explanation. Model boundaries and language.

   Access reviewed · 21 October 2026

3. Microservices — James Lewis and Martin Fowler, 2014 ↗

   Original engineering article. Architecture characteristics and costs; no exact company architecture is asserted here.

   Access reviewed · 21 October 2026

Model a bounded context

Prerequisites: 7.1 · Layers, monoliths & ports 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Catalog and fulfilment use different meanings for a product. Model those meanings with domain experts, then choose where rules need an atomic aggregate and which facts should become events.

7.2.1 · Domain-Driven Design (DDD)

Ask a shop's merchandising and warehouse staff what “available stock” means. Merchandising may mean stock that can be advertised. A reservation worker may mean units that can still be promised to a buyer. Treating both as one unexamined field can produce wrong decisions.

Domain-Driven Design develops a model with the people who understand the business. The model names concepts and makes their rules explicit. Use this effort where business meaning is complex or disputed. A simple lookup table may need only straightforward data handling. DDD's value comes from clearer decisions and shared understanding, rather than assigning a class to every database row.

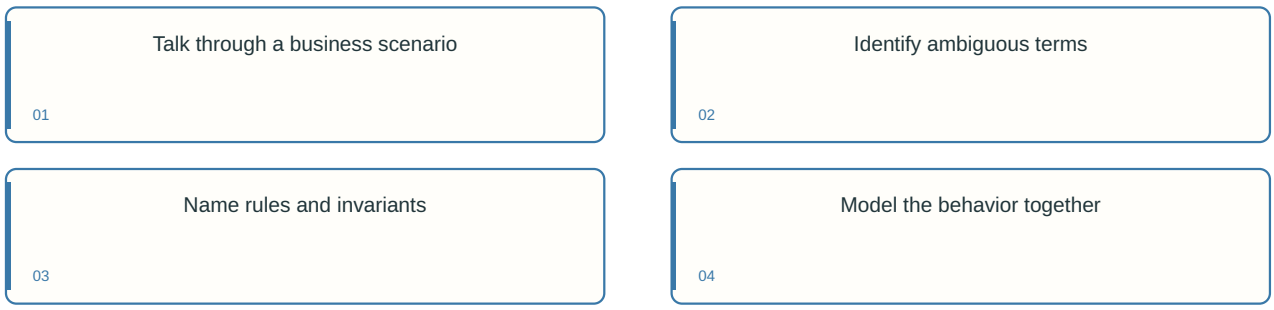


FIG. 7.2.1 **Domain-Driven Design (DDD) · the example**

Domain-Driven Design develops a shared business model with domain experts where rules and meanings need clarification.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.2.2 · Bounded contexts and ubiquitous language

Catalog's Product may describe a title, images, and marketing categories. Fulfilment's Item may describe a physical unit and its shelf location. Both models can be correct for their own work.

A bounded context states where a model and its vocabulary apply. Within it, a ubiquitous language gives team members and code consistent names for the business concepts. Between contexts, translate deliberately and identify who owns each fact. One universal Product schema may obscure real differences. A context is a modelling boundary; deciding whether it becomes a separate service, repository, or team requires further reasons.

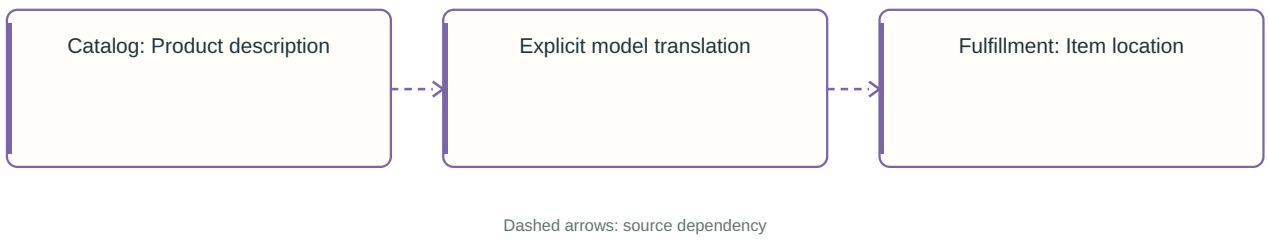


FIG. 7.2.2 **Context boundaries protect different business meanings**

The source-dependency boundary is not automatically a network deployment. Translation prevents one universal schema from erasing meaningful domain differences.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.2.3 · Aggregates, entities, value objects, domain services

An order's line items and total must agree when it is committed. We can give an Order root responsibility for changing those related values together. An aggregate is this kind of consistency boundary, with a root controlling changes to its members.

Entities have identity through change; value objects are distinguished by their values. A domain service can express a rule that does not belong to one entity. Keep an aggregate focused on invariants that need a common atomic change. Putting the entire shop into one aggregate creates contention and loading costs. Rules spanning aggregates need an explicit coordination design; an object reference alone does not create a distributed transaction.

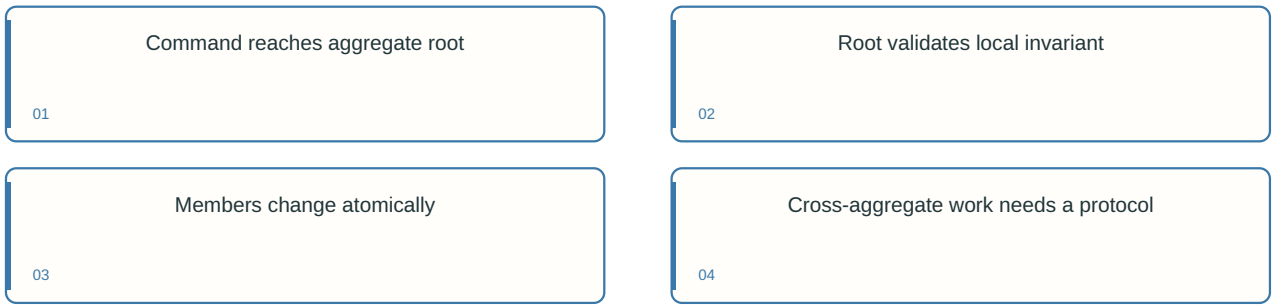


FIG. 7.2.3 **Aggregates, entities, value objects, domain services · the example**

An aggregate groups changes requiring one consistency boundary, with a root controlling its members' transitions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.2.4 · Domain events

After an order is accepted, other parts of the system may need to know. OrderPlaced records a business fact. Give it a stable identity, clear time meaning, and information that its intended consumers can interpret across versions.

Creating an event object in memory is not the same as publishing it reliably. The process may commit the order and die before sending the event. An outbox records order state and publication intent in one local transaction, then relays the event. Consumers still need to handle repeated and delayed delivery. Define a fact's meaning before deciding how the broker will carry it.

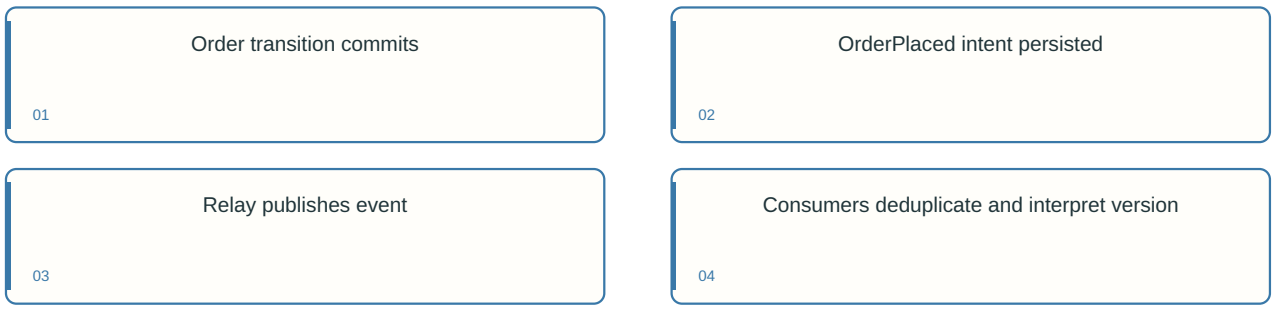


FIG. 7.2.4 **Domain events · the example**

A domain event records a meaningful accepted fact; durable publication needs a separate explicit mechanism.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What evidence would justify extracting payments into a separate service?

Look for stable capability ownership, different scaling or isolation needs, and an operating team prepared for network and version failures. Compare those benefits with added data coordination and recovery. Preserve existing payment invariants and histories through migration. A separate deployment is useful only when the required independence can actually be exercised.

Remember: A model is useful within a context whose vocabulary, ownership, and invariants are understood.

Try applying it: Which purchase-time facts should an order retain when the current catalog changes?

Unit 7 exercises → · Unit 7 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Architecture styles — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Style comparisons; provider-specific recommendations are not mandatory in the atlas.

   Access reviewed · 21 October 2026

2. Bounded Context — Martin Fowler, 2014 ↗

   Original engineering explanation. Model boundaries and language.

   Access reviewed · 21 October 2026

3. Microservices — James Lewis and Martin Fowler, 2014 ↗

   Original engineering article. Architecture characteristics and costs; no exact company architecture is asserted here.

   Access reviewed · 21 October 2026

Keep business policy inside

Prerequisites: 7.2 · Model a bounded context 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An order rule should not be rewritten in controllers, broker configuration, and provider adapters. Assign the business decision, its coordination, and its technical effects to clear owners.

7.3.1 · Dependency rules and module boundaries

Suppose the orders module updates a payment table directly. It now knows the payment representation and can bypass payment rules. A payment interface gives that interaction a controlled place.

Dependency rules specify which modules may know about or call others. Enforce important boundaries with appropriate import checks, database permissions, or architecture tests. A cycle can reveal that coordination has no clear owner. Shared helpers also need scrutiny: one “common model” can accidentally force unrelated contexts to change together. Where meanings genuinely differ, use an explicit translation rather than sharing a type merely because its fields currently look similar.

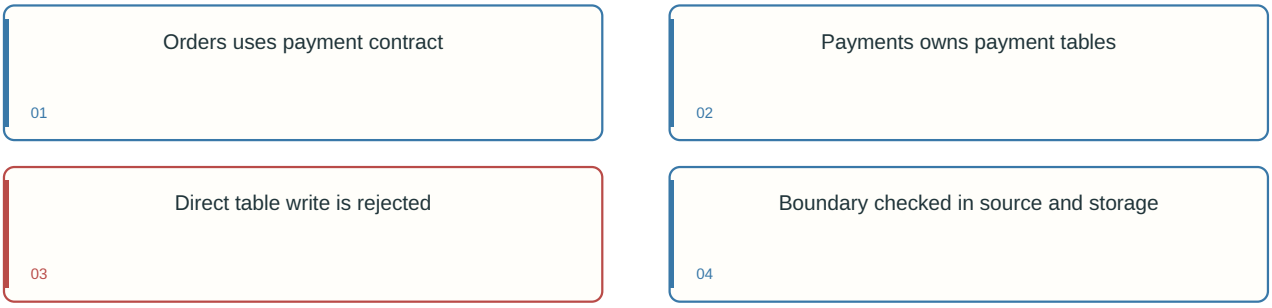


FIG. 7.3.1 **Dependency rules and module boundaries · the example**

Module dependency rules limit knowledge and bypass paths, using review and useful enforcement where appropriate.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.3.2 · Business logic vs orchestration vs infrastructure

“A refunded order must not ship” is a business rule. Asking the payment service for a refund and then updating progress is orchestration. Encoding and sending a provider request is infrastructure work.

These responsibilities may begin in one function, but distinguishing them helps assign tests and handle failures. Test the shipping rule as a decision. Test orchestration across success and partial failure. Test the provider adapter against its actual protocol. Moving a sequence onto a queue does not make its business rules belong to the broker. Keep the owner of each decision visible even when the execution arrangement changes.

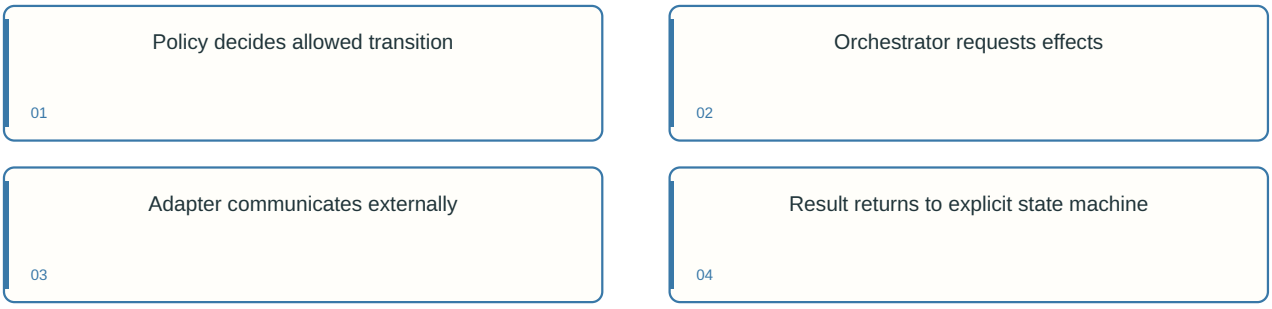


FIG. 7.3.2 **Business logic vs orchestration vs infrastructure · the example**

Business logic decides outcomes, orchestration sequences work, and infrastructure performs technical effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.3.3 · Controllers, services, repositories, adapters

A controller reads a request and builds a response. An application service coordinates an operation. A repository offers data operations, and an adapter connects a contract to a particular protocol or technology.

These are roles, not required class-name endings. A service can be a function; a domain service is not necessarily a network service. Examine the work the code performs. If a controller calculates prices and retries transactions, creating an empty “services” directory has not moved those responsibilities. Choose names and boundaries that help readers follow one real operation from its input to its result.

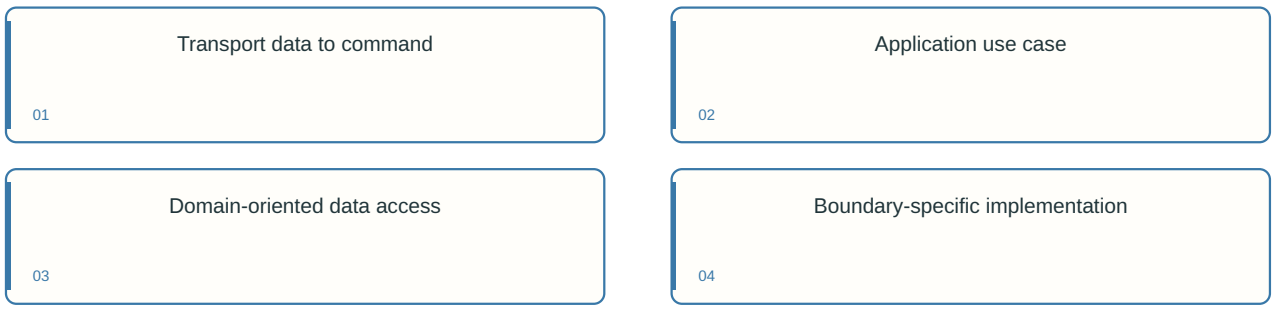


FIG. 7.3.3 **Controllers, services, repositories, adapters · the example**

Controllers, services, repositories, and adapters are responsibilities, not mandatory class-name conventions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.3.4 · Anti-corruption layers

A legacy payment provider may use “settled” for an authorisation that our application still considers pending. Copying that label directly would import the provider's meaning into our own model.

An anti-corruption layer translates the external model into the local one. It maps known states, retains useful provider references, and handles unknown meanings explicitly. This adds mapping and maintenance work, but keeps external quirks from spreading through business code. Never turn an unmapped result silently into success. Where the provider's state is uncertain or different from ours, preserve enough information to investigate and reconcile it.

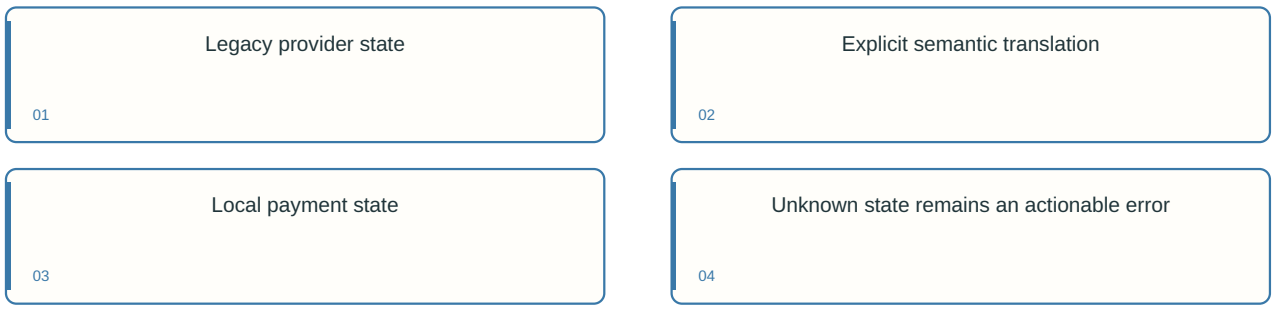


FIG. 7.3.4 **Anti-corruption layers · the example**

An anti-corruption layer translates external meanings so they do not silently redefine the local model.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.3.5 · Event-driven architecture

An accepted order can trigger a receipt without making checkout wait for email delivery. Event-driven architecture allows components to react independently to recorded events. This can remove a synchronous dependency from the user's path.

Decide which steps may actually be delayed. Inventory and payment requirements may require stronger coordination than receipts. Event processing also adds delivery, ordering, replay, and monitoring work. Name the owner of each event and state how it is persisted, recognised on redelivery, and recovered after a failure. A broker transports messages; the application must still arrange for the required business outcome.

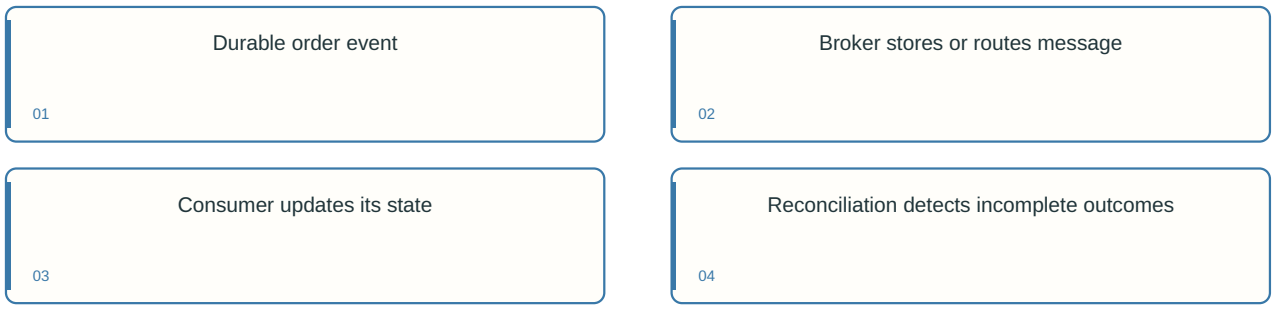


FIG. 7.3.5 **Event-driven architecture · the example**

Event-driven components react independently, with delivery, ordering, replay, and progress obligations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What evidence would justify extracting payments into a separate service?

Look for stable capability ownership, different scaling or isolation needs, and an operating team prepared for network and version failures. Compare those benefits with added data coordination and recovery. Preserve existing payment invariants and histories through migration. A separate deployment is useful only when the required independence can actually be exercised.

Remember: Preserve domain meaning while translating transport, storage, and external models at explicit boundaries.

Try applying it: How would an anti-corruption layer handle a provider status whose name differs from your own model?

Unit 7 exercises → · Unit 7 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Architecture styles — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Style comparisons; provider-specific recommendations are not mandatory in the atlas.

   Access reviewed · 21 October 2026

2. Bounded Context — Martin Fowler, 2014 ↗

   Original engineering explanation. Model boundaries and language.

   Access reviewed · 21 October 2026

3. Microservices — James Lewis and Martin Fowler, 2014 ↗

   Original engineering article. Architecture characteristics and costs; no exact company architecture is asserted here.

   Access reviewed · 21 October 2026

Evolution without a distributed monolith

Prerequisites: 7.3 · Keep business policy inside 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Five services always deploy together and share writable tables. Review whether the process split delivered independence, and use evidence to evolve toward a more manageable design.

7.4.1 · Serverless and function-oriented architectures

A cloud function can process an uploaded file when an event arrives. The platform manages invocation and some execution capacity, but the function still has deadlines, concurrency limits, credentials, and possible retries.

Treat function-local memory as temporary unless the platform guarantees otherwise. Store required progress durably, and recognise repeated invocations before applying repeated effects. Small independently triggered tasks can fit this arrangement well. A long sequence with human review or remote uncertainty needs a durable owner for its state, often a workflow engine. “Serverless” changes operating responsibilities; it leaves application correctness and recovery decisions to be designed.

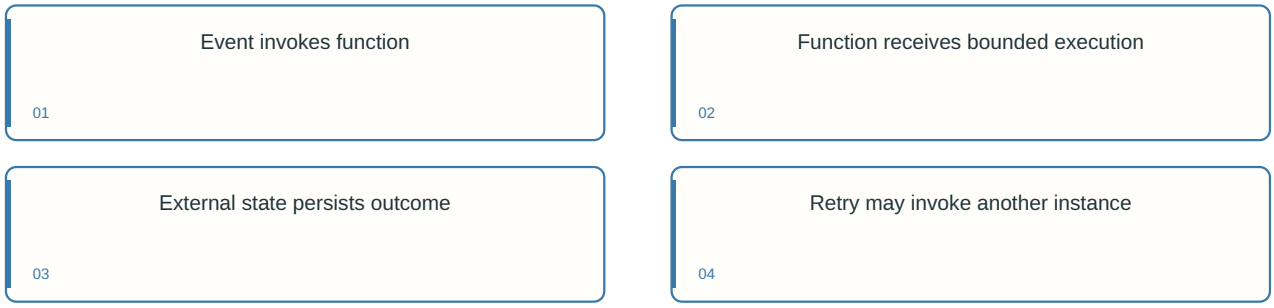


FIG. 7.4.1 **Serverless and function-oriented architectures · the example**

Serverless execution delegates platform work while retaining limits, state, identity, and retry requirements.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.4.2 · Evolutionary architecture

The application must keep tenant data isolated while releases change its internals. A check that attempts a cross-tenant read can protect that property during development. Similar checks can protect module boundaries, migration compatibility, or a latency budget.

Evolutionary architecture uses such feedback to change a system while preserving important qualities. A fitness check should express a useful property, rather than freeze an incidental class layout. Record why a choice exists and when evidence should prompt reconsideration. Evolution may involve removing an unnecessary service or simplifying a contract. It is a process of guided change, not an obligation to replace the architecture repeatedly.

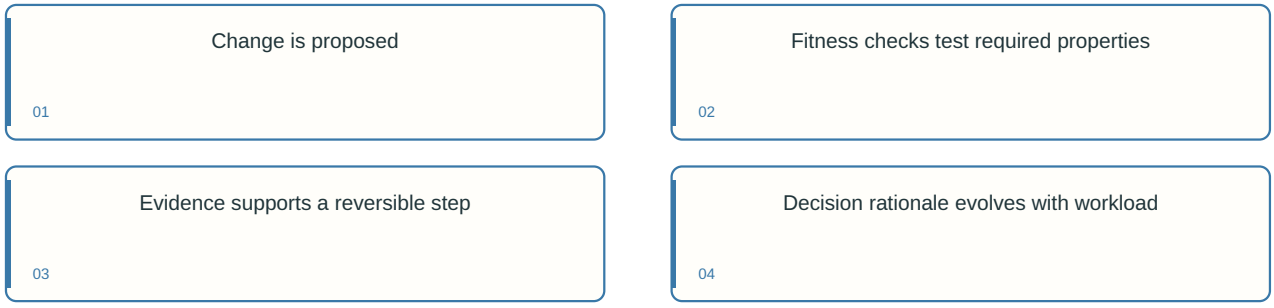


FIG. 7.4.2 **Evolutionary architecture · the example**

Evolutionary architecture changes through evidence while checks protect important required properties.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.4.3 · Technical vs business boundaries

Controllers and database adapters are technical groupings. Orders and fulfilment are business capabilities. The two kinds of division can coexist: each business module can have its own internal input, policy, and storage code.

Making separate network services for “validation” and “SQL” may force every business operation to call both. Begin with ownership of decisions and facts, then choose the technical arrangement inside those boundaries. A bounded context describes a model's scope. It does not automatically dictate a deployment or team structure. Follow actual business changes to see which parts need to evolve together.

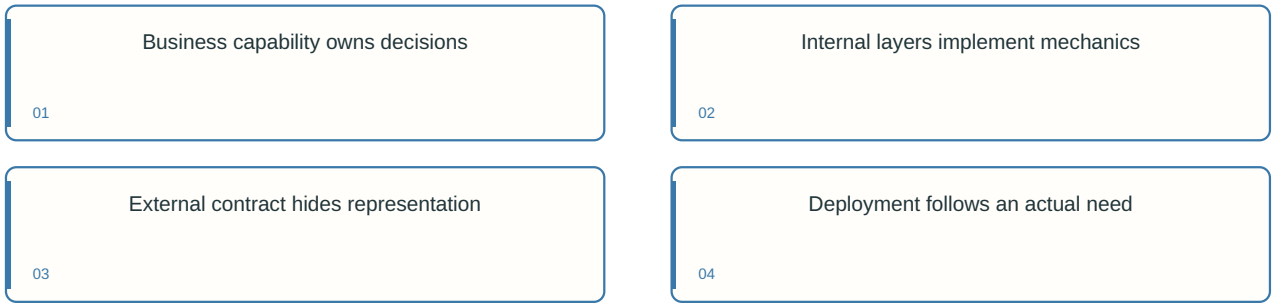


FIG. 7.4.3 **Technical vs business boundaries · the example**

Business boundaries group capabilities; technical layers organise mechanisms within or across them.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.4.4 · Distributed monolith antipattern

Five services share writable tables and must be released together. Most requests synchronously pass through all five. Although deployed separately, they cannot change or fail independently. This is commonly called a distributed monolith.

Network boundaries now add timeouts and debugging work without delivering the expected independence. Clarify data ownership and contracts before promising independent services. Sometimes consolidating unstable boundaries into a modular application is the easier improvement. In other cases, redesigning the interactions can justify the separation. Judge progress by independent behaviour and manageable failures, rather than the number of deployed processes.

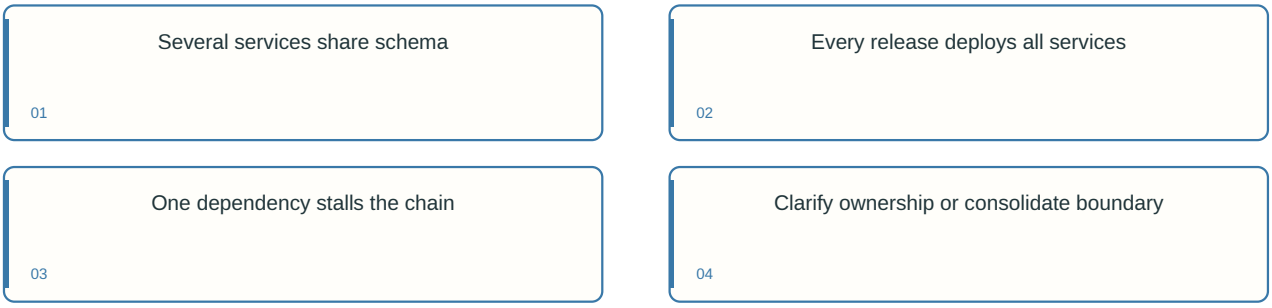


FIG. 7.4.4 **Distributed monolith antipattern · the example**

A distributed monolith has multiple processes without useful independent change or operation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

7.4.5 · When not to use microservices

A small team is still discovering how orders and fulfilment should work. Splitting them into many services immediately would add contract and operating work while the boundaries are changing. A modular monolith can provide a simpler starting point.

Consider extraction when a stable capability has distinct scaling, isolation, or ownership needs and the team can operate it. Include distributed transactions, version compatibility, telemetry, security, and incident response in the cost. Neither low traffic nor a service count settles the choice. Prefer the design that meets the actual requirements and can be changed and recovered by the people responsible for it.

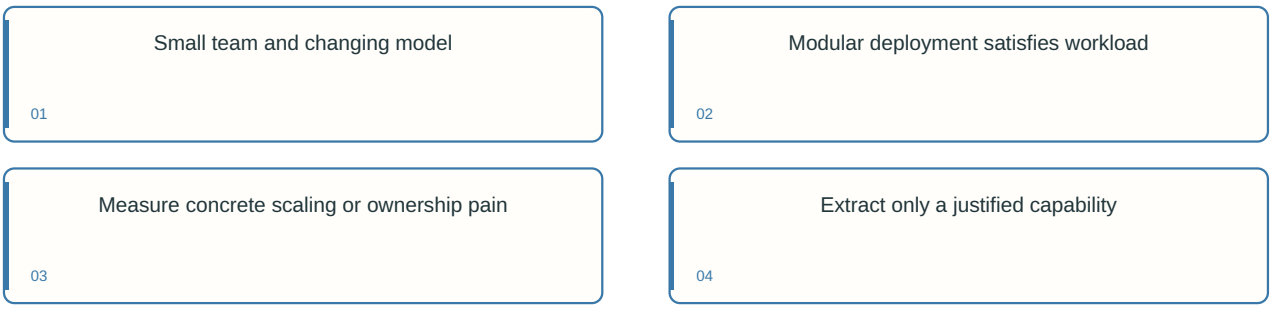


FIG. 7.4.5 **When not to use microservices · the example**

Microservice extraction needs concrete benefits and operating capability sufficient to justify its added coordination cost.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PSEUDOCODE · MODULE CONTRACT

```
place_order(command):
    priced = catalog.quote(command.items)
    reservation = inventory.reserve(command.items)
    payment = payments.authorize(command.operation_id, priced.total)
    return orders.record_outcomes(priced, reservation, payment)
# Modules expose contracts; they do not update each other's private tables.
```

The modules communicate through their operations rather than writing each other's private tables. Their results let the caller record what actually happened.

Calling three remote services in this order does not create one atomic transaction. Some local changes can share a deliberate transaction, while external effects need a workflow and recovery rules.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · What evidence would justify extracting payments into a separate service?

Look for stable capability ownership, different scaling or isolation needs, and an operating team prepared for network and version failures. Compare those benefits with added data coordination and recovery. Preserve existing payment invariants and histories through migration. A separate deployment is useful only when the required independence can actually be exercised.

Remember: Change architecture for demonstrated ownership, load, or failure needs while protecting existing properties.

Try applying it: When might consolidating services make the system easier to change and recover?

Unit 7 exercises → · Unit 7 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Architecture styles — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Style comparisons; provider-specific recommendations are not mandatory in the atlas.

   Access reviewed · 21 October 2026

2. Bounded Context — Martin Fowler, 2014 ↗

   Original engineering explanation. Model boundaries and language.

   Access reviewed · 21 October 2026

3. Microservices — James Lewis and Martin Fowler, 2014 ↗

   Original engineering article. Architecture characteristics and costs; no exact company architecture is asserted here.

   Access reviewed · 21 October 2026

PART 2 / UNIT 08

API Design and Communication Contracts

A client asks to create a payment. It needs a clear resource identity, valid amount representation, possible states, and bounded ways to read results. Build the contract before selecting request syntax.

The running example

Design a payment API with create-payment, payment-status, refunds, idempotency protection, duplicate requests, and failed network responses.

Unit 8 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

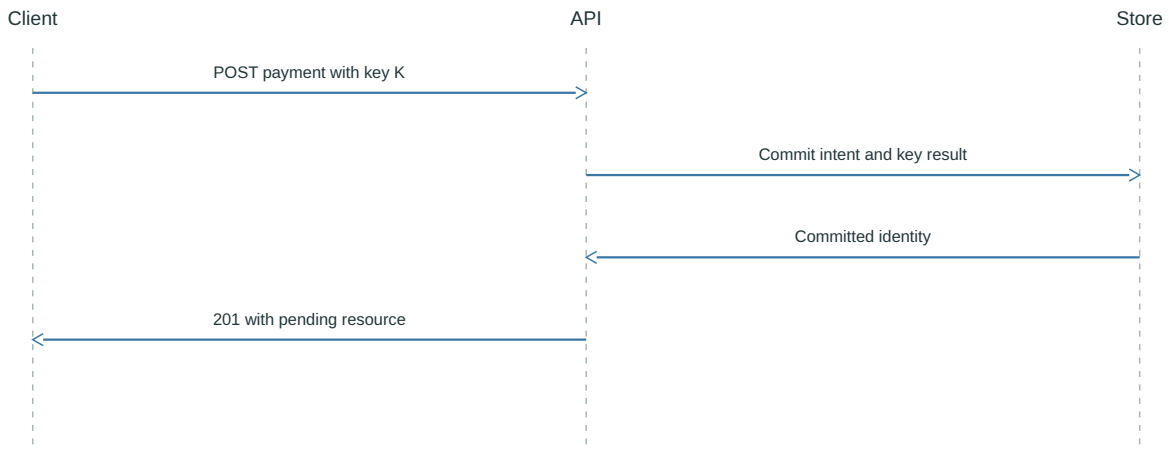


FIG. 8.0.1 **REST request flow**

HTTP creation and financial completion are different outcomes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

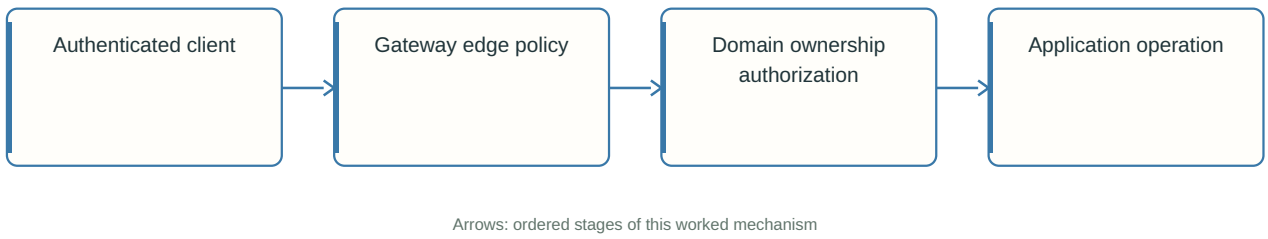


FIG. 8.0.2 **API gateway topology**

An edge gateway cannot decide every resource-specific permission without the appropriate domain context.

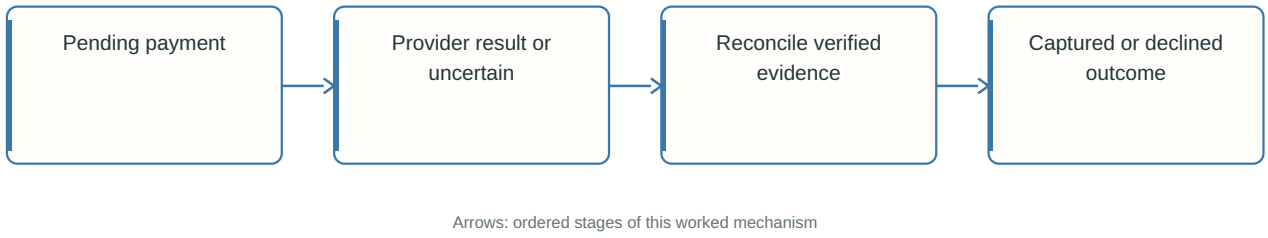
Author-created illustration. Numbers are illustrative unless stated otherwise.

Contract	Caller observes	State owner
Synchronous completion	Waits for final result	Operation still may be uncertain
Asynchronous acceptance	Receives operation ID	Durable workflow
Status query	Reads current progress	Authoritative or stated-freshness view

FIG. 8.0.3 **synchronous vs asynchronous API**

An accepted operation needs durable ownership beyond the request lifetime.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 8.0.4 **payment state-machine diagram**

Uncertain is not synonymous with declined; refunds form a separately tracked lifecycle.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Model resources and messages

Prerequisites: Introduction to System Design, Networking Foundations for System Design 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A client asks to create a payment. It needs a clear resource identity, valid amount representation, possible states, and bounded ways to read results. Build the contract before selecting request syntax.

8.1.1 · API fundamentals and interface contracts

A client calls create-payment. It needs to know the amount units, currency, permitted caller, returned states, and what to do if no reply arrives. A field schema alone cannot tell it whether retrying might charge twice.

An API is an interface together with a communication contract. Describe inputs, results, effects, errors, timing, operation identity, and supported evolution. Begin with concrete histories: successful creation, invalid input, a repeated request, and a lost reply after submission. These expose decisions a transport framework cannot make for you. Keep a definite provider decline distinct from an operation whose outcome is still uncertain.

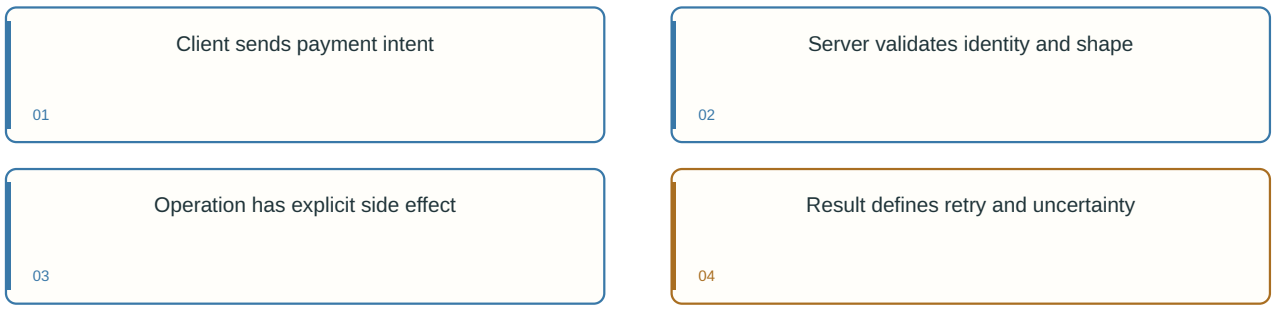


FIG. 8.1.1 **API fundamentals and interface contracts · the example**

An API is a behavioural communication contract, including identity, effects, failures, timing, and supported change.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.1.2 · REST, GraphQL, gRPC, RPC

The same payment capability can be exposed in several ways. REST emphasises resources and HTTP semantics. GraphQL lets clients query a typed graph. gRPC commonly supplies schema-defined remote calls and streaming. RPC more generally models operations called across a boundary.

Choose by what clients need, how the API will evolve, and the cost of serving it. GraphQL can reduce unwanted data transfer but needs query-cost and field-access controls. An RPC name can make an action clear while leaving retries and compatibility to be specified. Compare complete operation histories and tooling, rather than assuming a request syntax establishes reliability or performance.

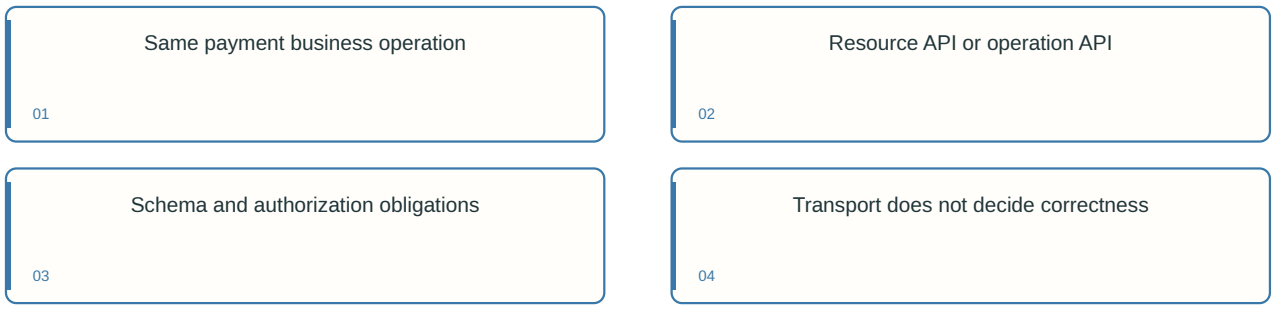


FIG. 8.1.2 **REST, GraphQL, gRPC, RPC · the example**

REST, GraphQL, and RPC styles organise interaction differently; none alone supplies safe retries or compatibility.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.1.3 · Resource modeling

A payment and its refund are related, but they have different identities and lifecycles. A refund may be requested later, authorised separately, and succeed or fail independently.

One resource model uses POST /payments, GET /payments/{id}, and POST /payments/{id}/refunds. The refund then has its own amount, status, and operation identifier. Resource modelling gives these continuing business objects clear identities. Avoid a generic /execute endpoint that hides every meaning, while also avoiding a forced CRUD model that makes domain actions unclear. Let ownership and allowed state transitions guide the interface.

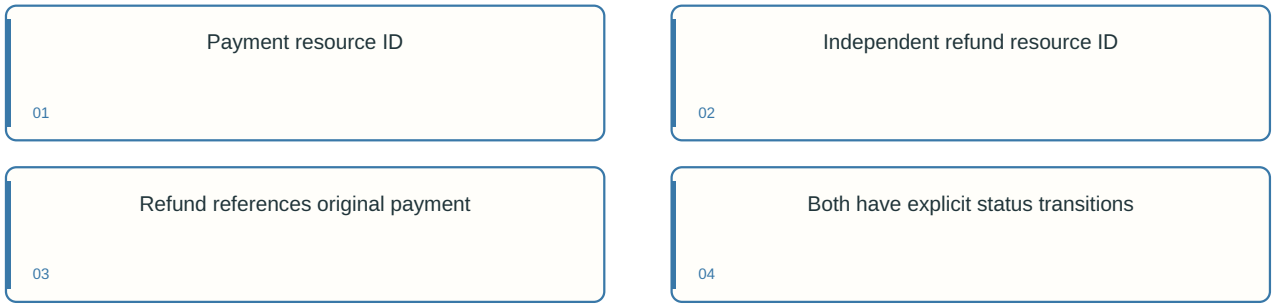


FIG. 8.1.3 **Resource modeling · the example**

Resource modelling gives business objects and operations stable identities and meaningful lifecycles.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.1.4 · Request/response schemas

A request can contain valid JSON yet ask to charge a negative amount. Parsing establishes structural validity; application validation checks business meaning. Check both.

Bound body size, collection length, and supported encodings before expensive work. Decide how unknown fields are handled so clients can evolve predictably. Return a stable machine-readable error code with a useful, safe message and a request identifier. Avoid exposing secrets or raw provider internals. Consider strict clients as well as tolerant ones: a new field can affect a client that rejects unknown input. The schema and its compatibility policy belong together.

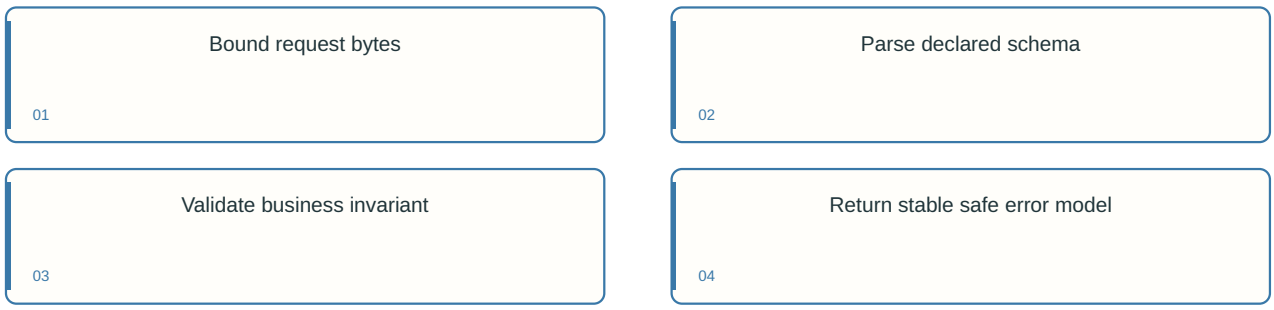


FIG. 8.1.4 **Request/response schemas · the example**

Schemas describe structure; validation also checks business meaning, size limits, and safe error behaviour.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.1.5 · Status codes and error models

The server returns 201 after creating a payment resource whose status is pending. The resource exists; money has not necessarily been captured. Similarly, 202 indicates acceptance for processing rather than final approval.

HTTP status describes the protocol response, while the body can describe the business operation's state. A timeout supplies neither a definitive approval nor a decline. Use application error codes to distinguish invalid input, conflicts, denied access, and uncertain submitted work. This lets clients choose an appropriate next action. Turning every provider problem into the same generic error can hide whether the caller should correct input, wait, or query an existing operation.

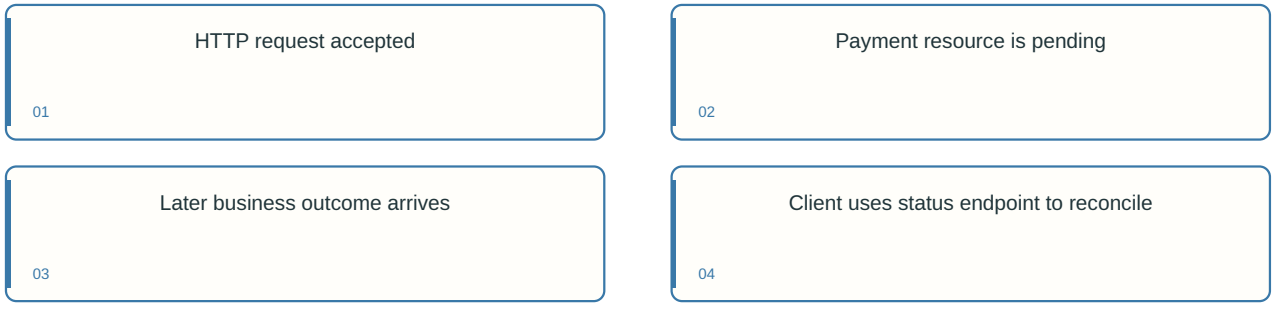


FIG. 8.1.5 **Status codes and error models · the example**

Protocol status and business state can differ: a created resource may still represent pending work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.1.6 · Pagination, filtering, sorting

A request for every payment can return too much data and consume too much memory. Pagination divides the collection into bounded responses. Define maximum page size, permitted filters, ordering, and access checks.

Sorting only by creation time can leave rows with equal timestamps in an unstable order. Add a unique tie-breaker such as the payment identifier. Decide what happens while the collection changes between page requests. Neither offset nor cursor pagination automatically gives the client one frozen snapshot. Enforce tenant access in the query itself, rather than retrieving a large shared collection and hoping the client filters it correctly.

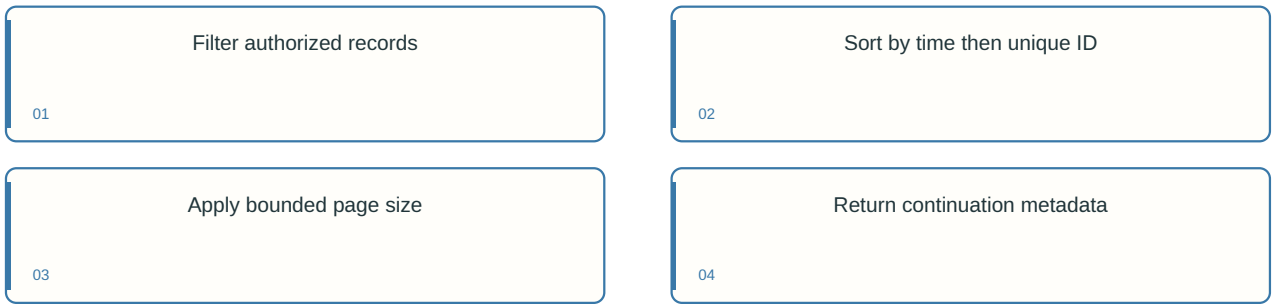


FIG. 8.1.6 **Pagination, filtering, sorting · the example**

Pagination bounds query and response work using defined order, filtering, size, and access rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Can finite storage support a permanent promise to recognise every old idempotency key?

Not for an unbounded set of arbitrary distinct operations unless another constraint supplies a bound or persistent evidence elsewhere. Define key retention, expiry, reuse, and client expectations. A permanent business identity may require a durable record rather than an expiring cache. Make the promise fit the information the system can retain.

Remember: An API describes behaviour and meaning as well as message shape.

Try applying it: Why can creating a payment resource succeed before the provider captures money?

Unit 8 exercises → · Unit 8 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. OWASP API Security Top 10, 2023 edition ↗

   Project documentation. Threat taxonomy for Unit 18.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

Compatibility & idempotency

Prerequisites: 8.1 · Model resources and messages 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A collection changes between pages, and a payment reply disappears before the client receives it. Use stable continuation and operation identities to give repeated calls a defined meaning.

8.2.1 · Cursor vs offset pagination

Suppose a client reads the first ten rows, then asks to skip ten. A new row inserted near the beginning shifts positions, so the second page may repeat or omit an item. This is a difficulty with offset pagination on changing data.

A keyset cursor instead records the last ordered key, such as (created_at,id), and continues after that key. It can reduce skipping cost and these shifting-position problems. Updates to existing rows still need a stated consistency policy. Validate opaque cursors or protect their integrity, and bind them to the applicable filters and tenant. A cursor must not become a way to escape the original query's access rules.

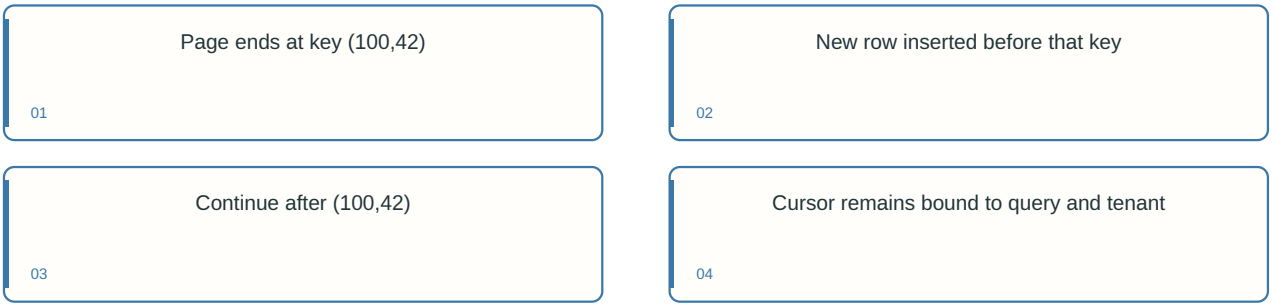


FIG. 8.2.1 **Cursor vs offset pagination · the example**

Keyset cursors continue after ordered keys; offsets skip positions that can shift as data changes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.2.2 · API versioning and compatibility

A payment amount changes from cents to dollars while its JSON type remains numeric. Old clients can now submit a hundred times the intended amount. Compatibility concerns meaning as well as syntax.

Adding an optional field is safer for tolerant clients than changing an existing field's meaning, but strict validators and enum handling may still break. Use an explicit version policy, an overlap period, and evidence about old consumers. Test representative clients. A versioned URL is one tool; it does not migrate callers or update stored events. Plan the transition for every reader that still relies on the earlier contract.

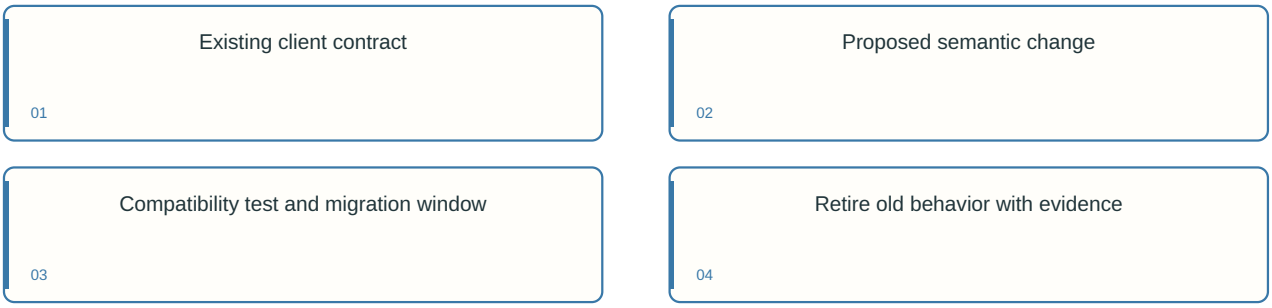


FIG. 8.2.2 **API versioning and compatibility · the example**

Compatibility preserves supported meanings and client behaviour across a planned transition.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.2.3 · Idempotency and idempotency keys

A client retries a payment after losing its reply. It sends the same idempotency key so the server can recognise the original logical operation. The server must bind that key to the authenticated owner, request content, progress, and result.

Enforce uniqueness and the required atomic updates in storage. If the same key arrives with a different amount, return a conflict instead of replaying an unrelated result. Define how long the key is remembered. A provider timeout may require querying or reconciling that provider before another submission. Local key storage alone cannot promise one external charge if the provider does not support the necessary operation identity and recovery behaviour.

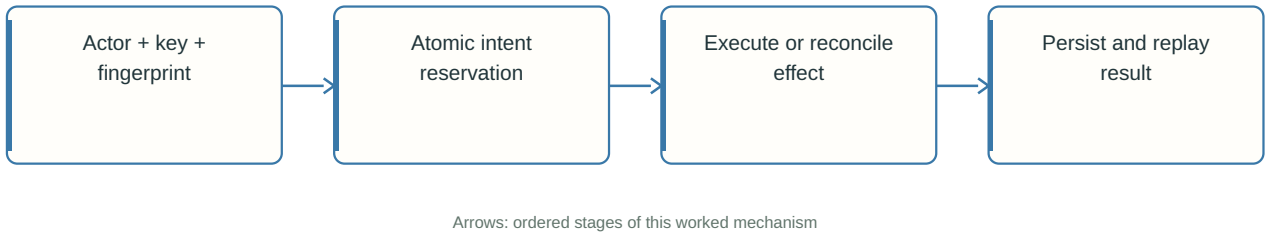


FIG. 8.2.3 **A payment retry follows durable operation identity**

The same key and payload return the established operation; a different payload conflicts. Local identity does not encompass arbitrary provider effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

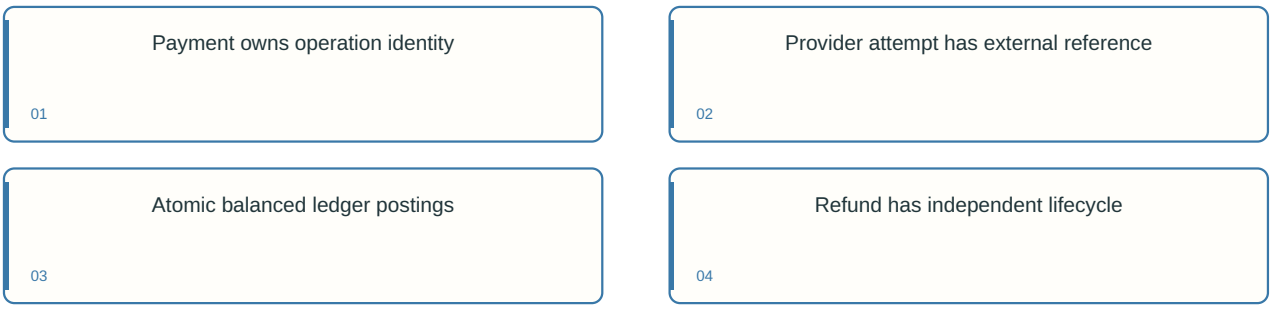


FIG. 8.2.4 **State and ownership in the related design**

Operation identity links provider evidence to payment state and balanced accounting entries.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Can finite storage support a permanent promise to recognise every old idempotency key?

Not for an unbounded set of arbitrary distinct operations unless another constraint supplies a bound or persistent evidence elsewhere. Define key retention, expiry, reuse, and client expectations. A permanent business identity may require a durable record rather than an expiring cache. Make the promise fit the information the system can retain.

Remember: Compatibility preserves meanings; cursors and idempotency keys need scope, validation, and recovery rules.

Try applying it: What should happen when an old idempotency key is reused with a different amount?

Unit 8 exercises → · Unit 8 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. OWASP API Security Top 10, 2023 edition ↗

   Project documentation. Threat taxonomy for Unit 18.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

Control access, load & long operations

Prerequisites: 8.2 · Compatibility & idempotency 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A signed-in client can still ask for another tenant's payment or overload an export. Combine access, resource limits, deadlines, and durable progress with clear acknowledgment boundaries.

8.3.1 · Authentication vs authorization at API boundaries

A user signs in successfully, then requests payment 42. The server still needs to check whether that user owns or may view payment 42. Knowing an identity and granting an action are separate decisions.

Authentication establishes identity under the chosen trust mechanism. Authorisation checks permission for this action on this resource. Derive tenant context from validated membership rather than trusting an arbitrary tenant_id field. A gateway can perform coarse checks, but the service with ownership information must enforce the resource rule. Apply appropriate identity checks to internal service callers too. An obscure identifier reduces guessing; it is not an access-control mechanism.

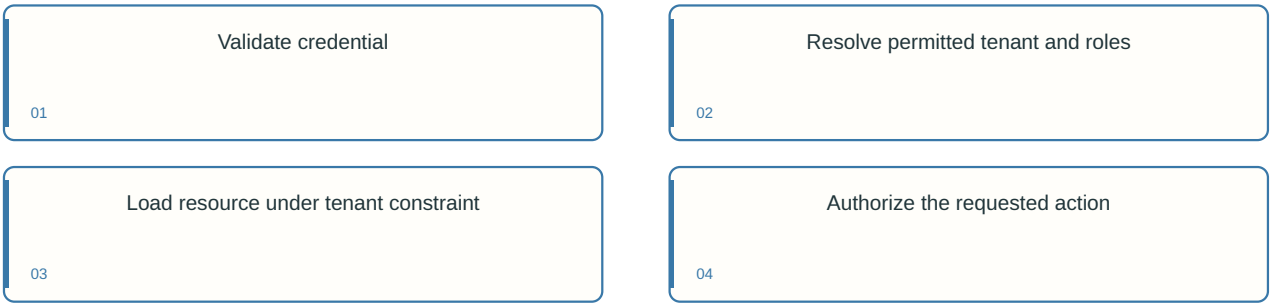


FIG. 8.3.1 **Authentication vs authorization at API boundaries · the example**

Authentication identifies a caller; authorisation checks the permitted action on the particular resource.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.3.2 · Rate limits and quotas

A client makes too many expensive validation requests at once. A rate limit controls that pace, often with a specified burst allowance. A daily export quota instead caps accumulated use over a longer period or resource dimension.

State the identity being limited, window, cost unit, retry guidance, and behaviour if the limiter fails. Per-tenant controls can protect neighbours from one noisy client. Per-user limits alone may miss coordinated abuse. Across regions, a strict global limit needs shared coordination or safely allocated local budgets. If enforcement is approximate, specify the approximation rather than silently giving every region the full global allowance.

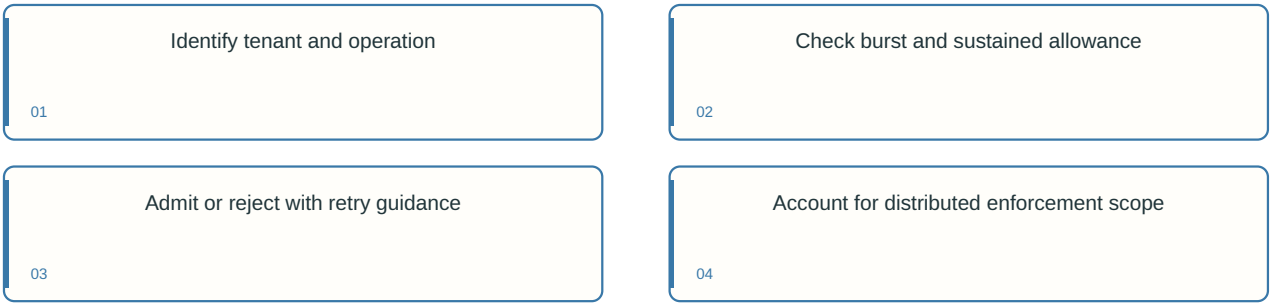


FIG. 8.3.2 **Rate limits and quotas · the example**

Rate limits control pace and bursts; quotas cap accumulated use under defined scope and failure rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.3.3 · Retries and timeouts

A payment request has an overall deadline. Connection acquisition, each attempt, and any backoff must fit inside it. Passing only a fresh full timeout to every downstream call can make the total wait exceed the client's intended limit.

Retry transient work only where repetition is safe. Exponential backoff with jitter spreads attempts but does not add capacity. If four layers each allow three attempts, one intent can cause up to $3^4 = 81$ downstream attempts. Choose a shared retry budget and propagate remaining time. Side effects additionally need a stable operation identity and a way to resolve outcomes that may already have committed.

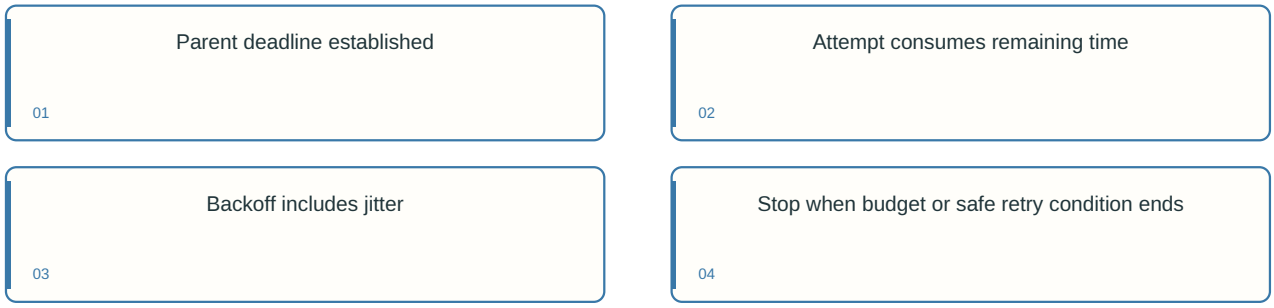


FIG. 8.3.3 **Retries and timeouts · the example**

Retries spend bounded time and capacity and need safe repetition, particularly for uncertain effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.3.4 · Streaming APIs and long-running operations

A report takes minutes to generate. Instead of holding the initial request open, the server can return an operation resource that the client polls or subscribes to. Required progress must be stored beyond the request's memory.

A streaming API returns a sequence of messages and needs framing, buffer limits, cancellation, and possibly a resume policy. Decide whether disconnecting cancels the business job or only the connection. A slow reader should not accidentally hold an unbounded database transaction. Use bounded pages or an explicitly owned streaming resource. The interface must explain how work is tracked after the initiating client has gone away.

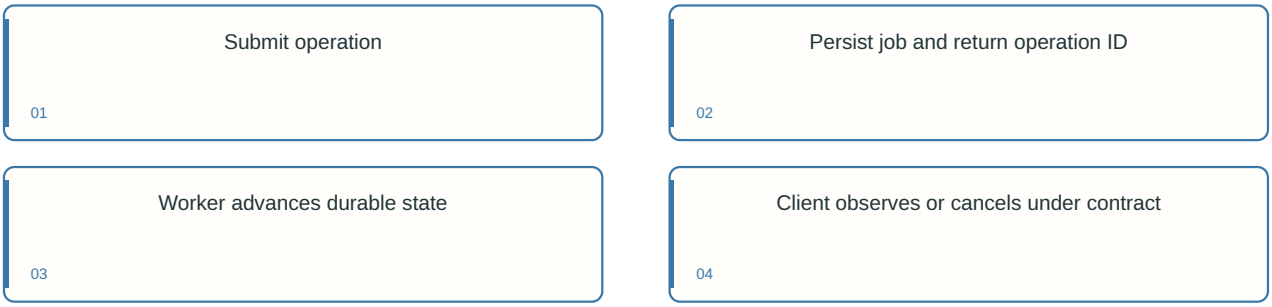


FIG. 8.3.4 **Streaming APIs and long-running operations · the example**

Long-running and streaming APIs need explicit ownership, progress, cancellation, buffering, and recovery contracts.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.3.5 · Webhooks and delivery acknowledgments

A payment service sends a webhook announcing a completed operation. The receiver commits its local effect, but the success reply is lost. The sender retries, so the same event arrives again.

Give webhooks stable event identifiers and a documented signature and timestamp-verification policy. The receiver should persist the required inbox or effect before acknowledging, then recognise redelivery. The sender needs bounded retries and a recovery policy. A 2xx reply establishes only what the receiver's contract promises; it does not mean every later downstream action is complete. Treat the acknowledgment as a named step in the delivery protocol.

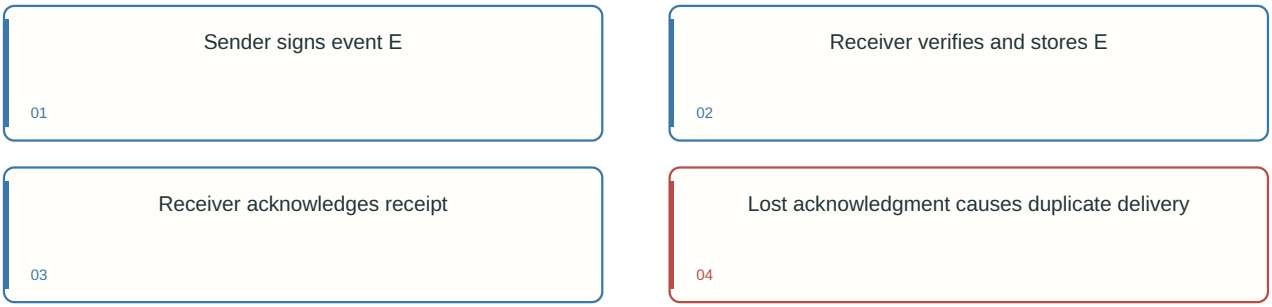


FIG. 8.3.5 **Webhooks and delivery acknowledgments · the example**

Webhooks are asynchronous deliveries requiring authenticity, stable event identity, and defined acknowledgment behaviour.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Can finite storage support a permanent promise to recognise every old idempotency key?

Not for an unbounded set of arbitrary distinct operations unless another constraint supplies a bound or persistent evidence elsewhere. Define key retention, expiry, reuse, and client expectations. A permanent business identity may require a durable record rather than an expiring cache. Make the promise fit the information the system can retain.

Remember: Admission, permission, and completion answer different questions and need separate enforcement.

Try applying it: What can a webhook receiver acknowledge safely if its effect must survive a crash?

Unit 8 exercises → · Unit 8 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. OWASP API Security Top 10, 2023 edition ↗

   Project documentation. Threat taxonomy for Unit 18.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

Evolve a public contract

Prerequisites: 8.3 · Control access, load & long operations 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An event saved today may be replayed a year later. Keep schemas, consumer expectations, gateway roles, and documentation usable across that lifetime.

8.4.1 · Schema evolution

A restored consumer replays last year's events. It encounters fields and status values no current request uses. Deleting the old decoder can therefore break recovery even if today's API traffic works.

Schema evolution applies to events and stored records as well as live requests. Preserve field meanings, defaults, version identity, and handling of unknown values. Test historical samples and any transformations used to read them. An extra field can also affect signatures or uniqueness rules, so “additive” does not automatically mean harmless. Plan compatibility for the full life of the data, including replay, backup restoration, and delayed clients.

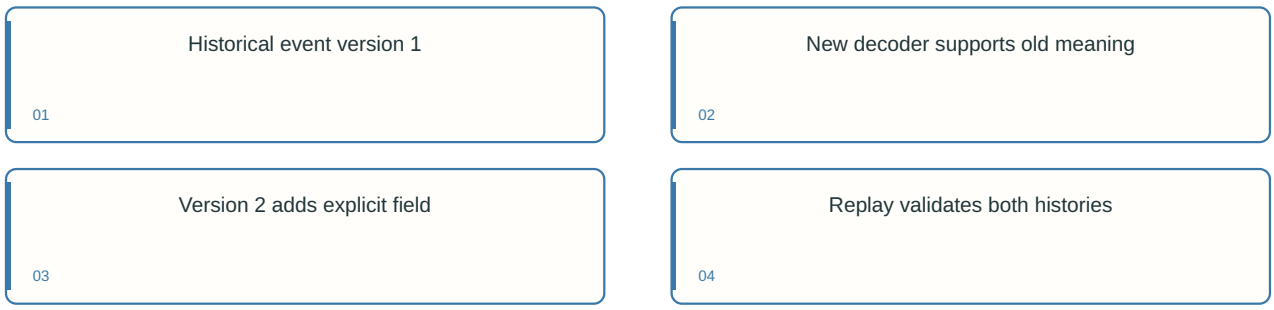


FIG. 8.4.1 **Schema evolution · the example**

Schema evolution must preserve supported interpretation of stored events, historical data, and delayed readers.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.4.2 · Consumer-driven contracts

A consumer expects a payment status to be either pending or final. A provider test that checks only “status is a string” misses that assumption. Consumer-driven contracts record what actual clients need from their provider.

Run relevant provider checks before releasing a change, including representative errors and state behaviour. Keep the contracts focused on meaningful obligations rather than incidental JSON formatting. Schema tests alone cannot establish retry safety, authorisation, or pagination correctness. Use concrete histories for these behaviours. The contract is useful when it protects a real dependency while still allowing the provider to change internal implementation details.

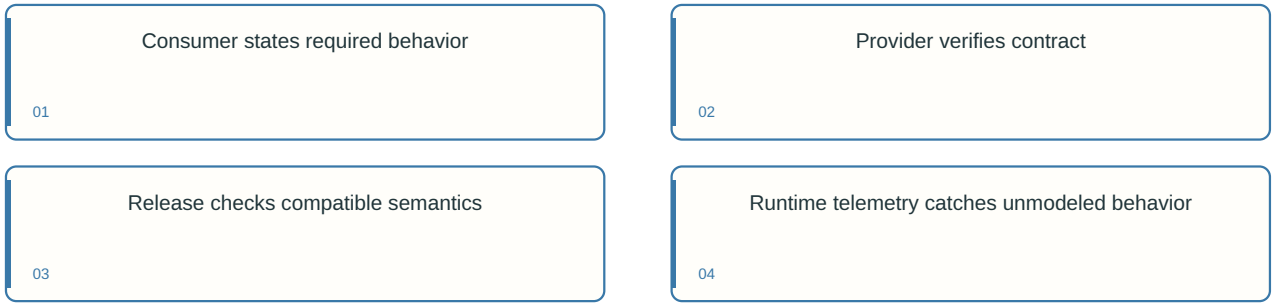


FIG. 8.4.2 **Consumer-driven contracts · the example**

Consumer-driven contracts protect meaningful assumptions of actual clients without freezing incidental formatting.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.4.3 · API gateways and BFF

A mobile screen needs three small pieces of data from several services. A Backend for Frontend, or BFF, can assemble the response around that client's needs. A gateway usually handles shared edge concerns, such as routing and coarse admission.

Both add operating work and can become bottlenecks. Keep ownership checks in services that know the resource rules, and give the aggregation layer a clear owner. A universal BFF that every product must edit can reduce independence. If results are cached, define the key, tenant scope, and freshness. Aggregating private results does not make them safe for a shared public cache.

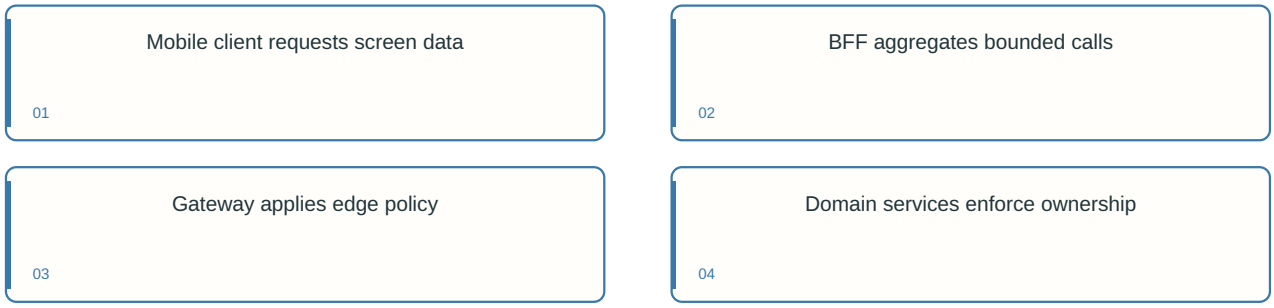


FIG. 8.4.3 **API gateways and BFF · the example**

Gateways provide shared boundary policy; BFFs adapt interactions to particular client needs.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.4.4 · Public vs internal APIs

Two internal services may release at different times and communicate over a network. Their API therefore still has compatibility, trust, load, and failure concerns. “Internal” does not mean that every request is valid or authorised.

A public API usually needs longer compatibility commitments, discoverability, abuse protection, and support processes. State the intended audience and lifetime of both kinds. Internal interfaces can become important dependencies whose accidental behaviour is difficult to change. Document their meaningful guarantees early, and apply proportionate authentication, validation, and resource limits instead of relying solely on location inside a network.

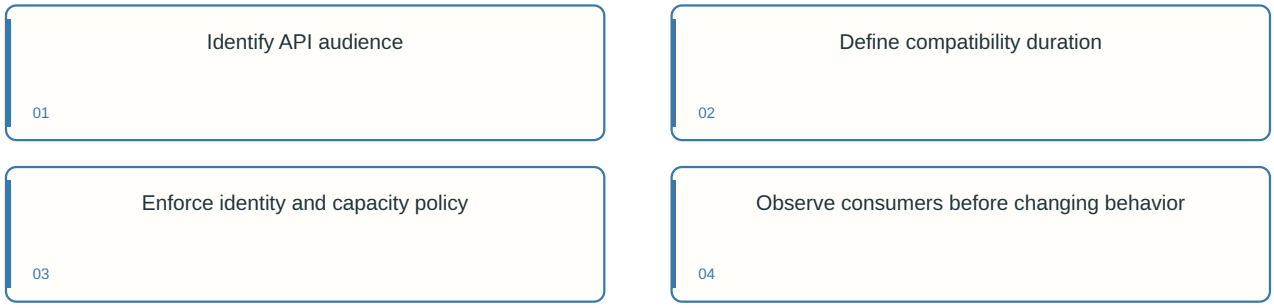


FIG. 8.4.4 **Public vs internal APIs · the example**

Internal APIs still cross trust, version, and failure boundaries; public APIs add explicit audience and support commitments.

Author-created illustration. Numbers are illustrative unless stated otherwise.

8.4.5 · API documentation and discoverability

A developer needs to create a payment and recover if the reply is lost. Documentation that shows only a successful request leaves the most consequential decision unexplained.

Include complete examples for validation errors, access denial, pagination, operation keys, timeouts, and pending states. A machine-readable schema can generate client types, while prose explains the behaviour those types cannot express. Keep examples aligned with the implementation and test the important ones. A sandbox must clearly distinguish simulated effects from real charges. Good discoverability means a reader can find the operation, understand its scope, and choose a supported recovery action when the usual sequence is interrupted.

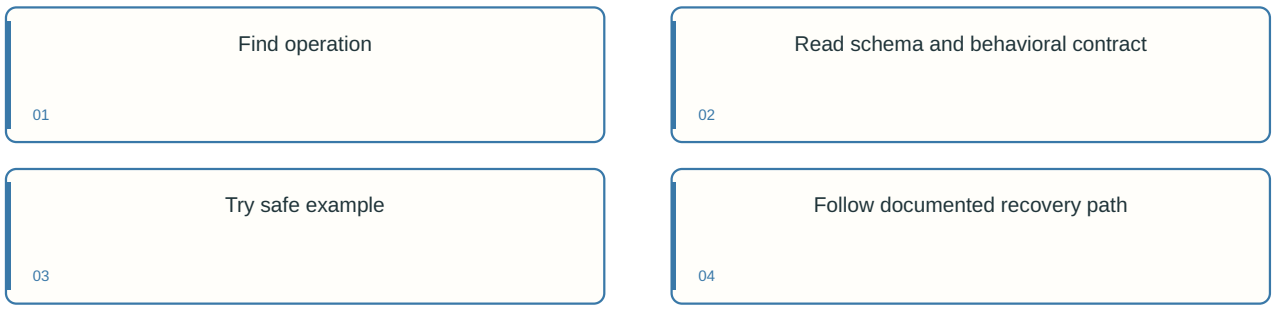


FIG. 8.4.5 **API documentation and discoverability · the example**

Useful API documentation explains complete success and failure histories beyond machine-readable message shape.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · KEYSET CONTINUATION

```
from mechanisms import keyset_page
rows = [(1,1,"A"), (1,2,"B"), (2,3,"C")]
assert keyset_page(rows, after=(1,1)) == rows[1:]
```

The ordered time/ID pair gives equal timestamps a stable tie-breaker. Continuing after (1,1) returns the later rows in this fixed fixture.

A real cursor also needs validation, query and tenant scope, and a rule for changes between pages. Keyset continuation does not itself freeze a changing dataset.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · Can finite storage support a permanent promise to recognise every old idempotency key?

Not for an unbounded set of arbitrary distinct operations unless another constraint supplies a bound or persistent evidence elsewhere. Define key retention, expiry, reuse, and client expectations. A permanent business identity may require a durable record rather than an expiring cache. Make the promise fit the information the system can retain.

Remember: Supported consumers include delayed and recovering readers, not only today's newest client.

Try applying it: Which historical samples would you retain to check that a new decoder still supports recovery?

Unit 8 exercises → · Unit 8 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. OWASP API Security Top 10, 2023 edition ↗

   Project documentation. Threat taxonomy for Unit 18.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

PART 3 / UNIT 09

Data Models and Storage Technologies

A ledger, profile, chat history, and search index do not make the same promises. Compare their reads, changes, and recovery needs before assigning a storage family.

The running example

Choose storage technologies for user profiles, financial transactions, chat messages, event logs, and uploaded PDFs. Justify each decision.

Unit 9 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

Access pattern	Candidate family	Important cost
Relationships and invariants	Relational	Index/write and transaction cost
Nested aggregate	Document	Cross-document rules
Direct key access	Key-value	Secondary query maintenance
Relationship traversal	Graph	Degree and traversal bounds

FIG. 9.0.1 **Database taxonomy**

Families describe useful access patterns; no category label establishes all guarantees.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Layout	Physical grouping	Suitable operation
Row	Fields of one record	Bounded point read
Column	One field across records	Analytical aggregation
Object	Named immutable bytes	Large file transfer

FIG. 9.0.2 **physical storage representations**

Logical data modeling and physical layout are separate decisions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Fact	Normalized source	Legitimate copy
Current profile address	Customer profile	Cache with freshness policy
Purchase-time address	Order history	Historical fact, not stale profile
Display name	User record	Projection with update policy

FIG. 9.0.3 **normalized vs denormalized schema**

Not all duplication is an anomaly: distinguish historical snapshots from copies meant to track current state.

Author-created illustration. Numbers are illustrative unless stated otherwise.

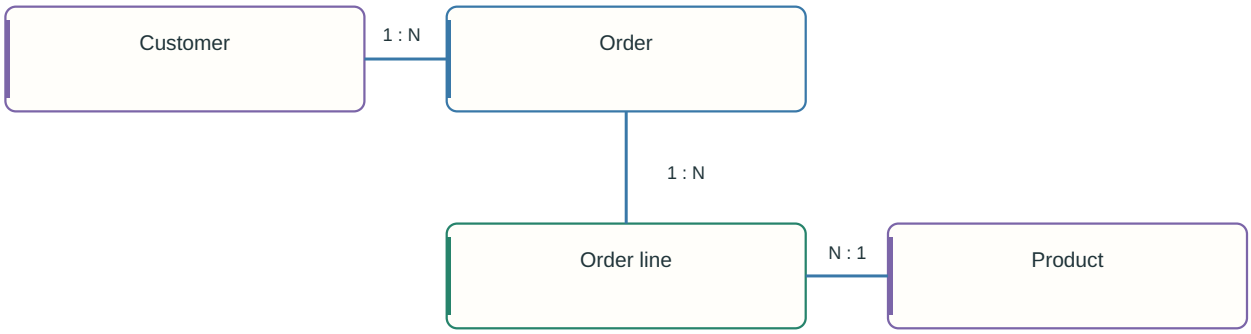


FIG. 9.0.4 **An order schema preserves relationships and history**

Customer has many Orders, Order has many Lines, and each Line references a Product identity while retaining purchase-time values.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Storage promises & database families

Prerequisites: Introduction to System Design 9 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A ledger, profile, chat history, and search index do not make the same promises. Compare their reads, changes, and recovery needs before assigning a storage family.

9.1.1 · What data storage must guarantee

A chat history, a payment ledger, and a thumbnail cache all store data, but losing a copy has different consequences. Before choosing technology, state what is accepted, when it is committed, how callers read it, and which failures it must survive.

Identify the authoritative record and the recovery procedure. A database category does not establish its configured isolation, durability, or backup behaviour. Begin with access patterns and invariants, then compare implementations against them. This prevents a familiar product name from standing in for decisions about the actual service being built.

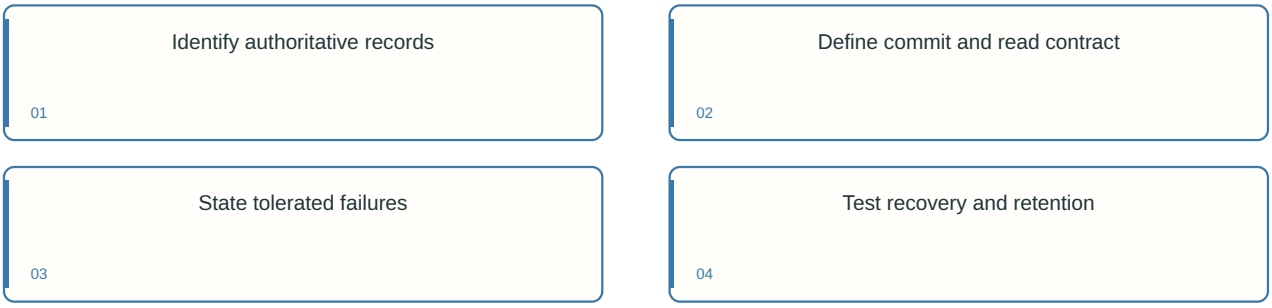


FIG. 9.1.1 **What data storage must guarantee · the example**

Storage guarantees state accepted outcomes, observation rules, ownership, retention, and recovery under specified failures.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.2 · Relational databases and SQL

A ledger can represent accounts and entries as related tables. Keys identify rows, foreign keys protect appropriate relationships, and transactions can group changes. SQL expresses operations on this relational model.

Joins let queries combine related facts. Their cost depends on indexes, row counts, data distribution, and the chosen execution plan; a join is not inherently a scaling mistake. Normalised facts often provide a clear starting point for ownership and consistency. If a measured read path needs a repeated or precomputed representation, add it deliberately and state how it stays correct. The relational model and the physical query cost are related but different questions.

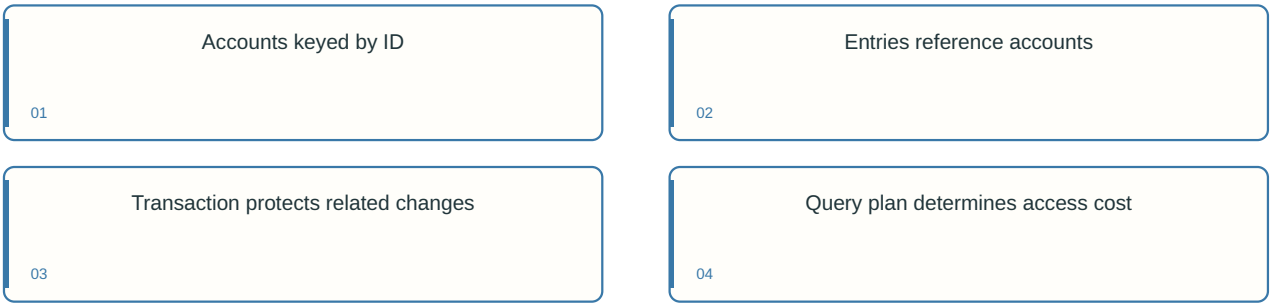


FIG. 9.1.2 **Relational databases and SQL · the example**

Relational storage models facts with keys and constraints; SQL expresses operations whose physical cost needs investigation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.3 · Document databases

A user profile and its preferences can be stored as one nested document. A document database gives that unit an identity and often allows flexible structures. This can suit data usually read or changed together.

Decide which structure is valid and how it evolves. Flexibility moves some checking into application conventions; it does not remove the schema. Also ask how large the document can grow and whether changes across documents need a shared transaction. Placing every chat message in one ever-growing user document mixes different growth and ownership needs. Design the document boundary around actual operations rather than convenient JSON syntax.

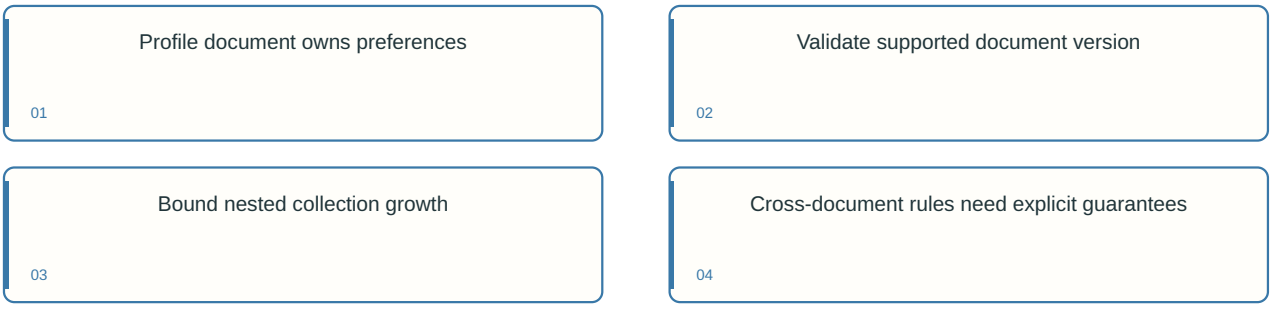


FIG. 9.1.3 **Document databases · the example**

Document storage groups nested data by identity, with explicit size, schema, index, and cross-document rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.4 · Key-value databases

A session lookup already knows its identifier. A key-value store can retrieve the corresponding value directly, making this a natural access pattern. A key such as tenant:42:session:7 also makes the intended identity scope visible.

Other queries, such as finding values by an arbitrary field, may need separately maintained structures. Check the selected store's read, write, expiry, and durability contract; key-value products do not all behave alike. A session's expiry and revocation policy must also be explicit. Retaining an old value after permissions change can be a security problem even when the key lookup itself is correct.

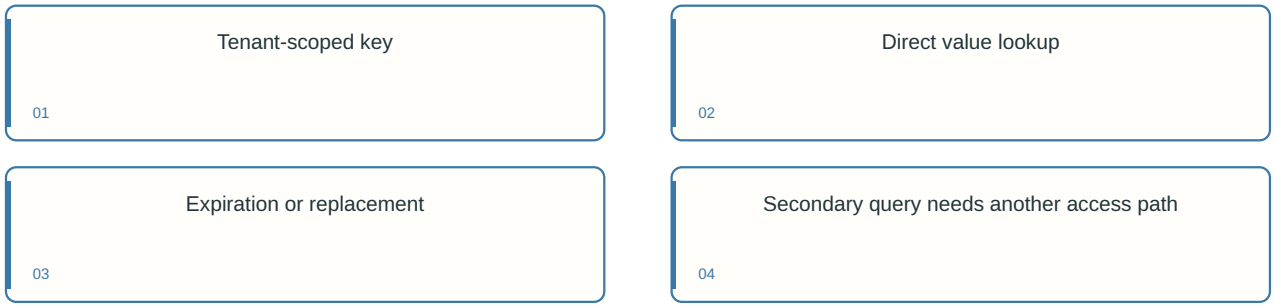


FIG. 9.1.4 **Key-value databases · the example**

Key-value storage serves keyed operations; other queries and expiry need their own supported structures and policy.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.5 · Wide-column stores

A conversation history is often read by conversation identifier and message order. A wide-column design can use the conversation as a partition key and a sequence as an ordering dimension, supporting bounded history reads.

Work through the largest room as well as the typical one. One huge or popular partition can dominate traffic despite many storage nodes. Model the queries first, and duplicate representations for different queries only with a maintenance plan. State which joins or integrity rules the application must enforce. The common storage pattern is useful guidance; exact guarantees still depend on the implementation.

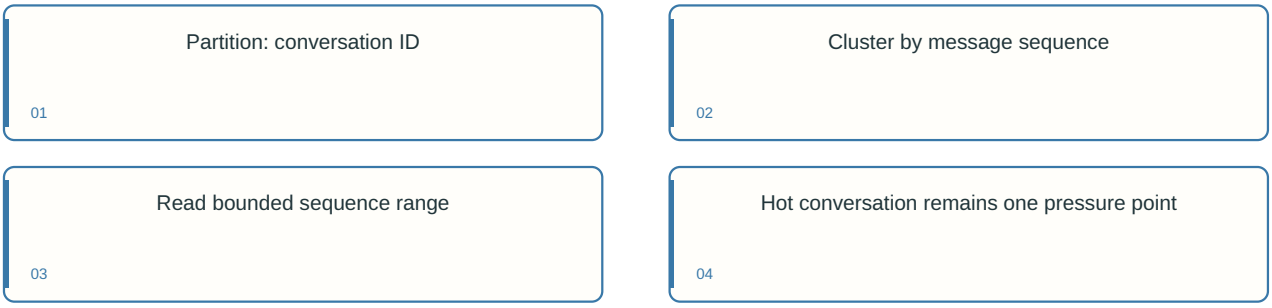


FIG. 9.1.5 **Wide-column stores · the example**

Wide-column designs commonly combine partition keys with ordered dimensions, requiring bounded partitions and query-driven modelling.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.6 · Graph databases

A fraud investigator may follow transfers from an account to devices, other accounts, and further transfers. When variable-depth relationships are central to the query, graph storage can make traversals convenient.

A relational model can also represent edges, so compare actual operations rather than assuming relationships require a graph database. Bound traversal depth and fan-out: one high-degree node can create enormous work. Apply access checks during expansion, not only after collecting results. A traversal that crosses tenant boundaries can expose data even if its starting node was authorised. Query usefulness, cost, and permission rules must be designed together.

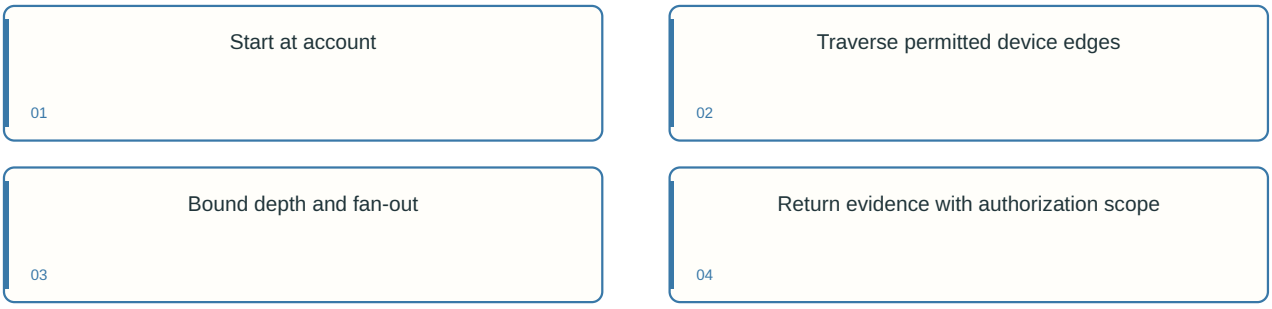


FIG. 9.1.6 **Graph databases · the example**

Graph storage supports relationship traversal whose depth, degree, cost, and access scope need limits.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.7 · Time-series databases

A temperature measurement has a time, value, and dimensions such as device identity. Time-series storage can exploit nearby timestamps, compression, and time-based retention for these measurements.

Choose dimensions carefully. A unique request identifier on every metric can create too many distinct series. Also decide how to handle late readings, clock errors, corrections, and downsampling. A device timestamp records what its clock reported; it does not automatically establish causal order. Preserve ingestion time or a sequence where investigation needs that distinction. Time-based organisation helps when it matches the measurements and queries, rather than simply because a row contains a date.

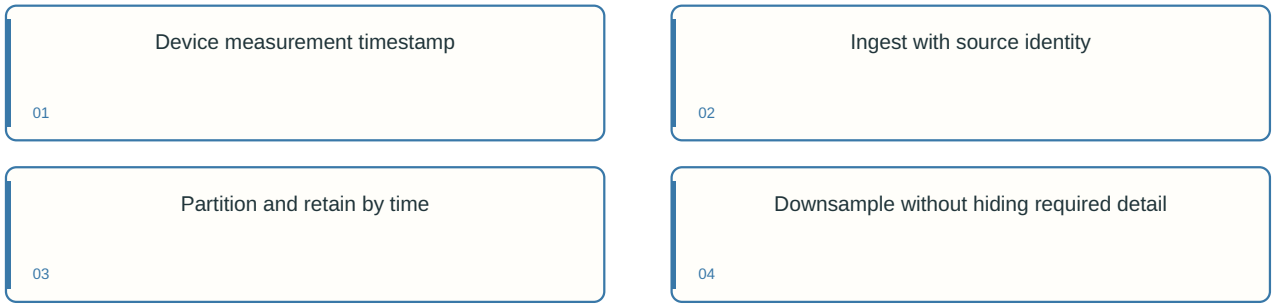


FIG. 9.1.7 **Time-series databases · the example**

Time-series data combines measurements, times, and dimensions with explicit cardinality, lateness, and retention handling.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.8 · Search indexes

A search query needs matching and ranking rather than a transaction over the original record. An inverted index maps terms to document references and supports this retrieval path. It is commonly built from another authoritative store.

If indexing is asynchronous, a new document may appear late and deleted content may remain in the index temporarily. Permission enforcement must remain correct during that lag. Keep source identity and version information so updates can be ordered and the index rebuilt. A search result is a derived serving representation; treating it as the only recoverable copy creates different and usually much harder ownership obligations.

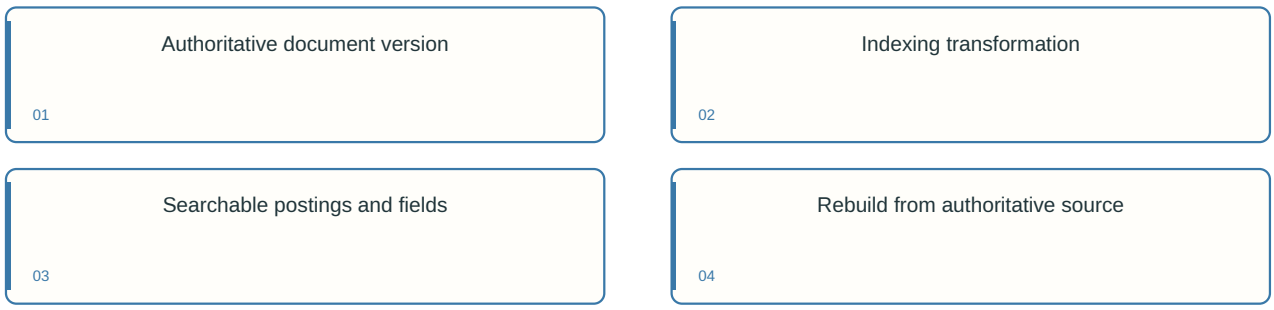


FIG. 9.1.8 **Search indexes · the example**

Search indexes are retrieval representations with source-version, freshness, access, and rebuild obligations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.1.9 · Vector indexes: introductory architecture only

An embedding converts content into a numerical vector. A vector index finds nearby vectors, often approximately, to retrieve candidate documents with similar representations. Its architecture must balance recall, memory, updates, distance calculations, and filtering.

Preserve the source document and version that produced each embedding. The index is derived and needs a rebuild path. Approximate search can miss relevant items, and a similarity score establishes neither factual truth nor access permission. Apply the required tenant and resource checks to candidates. Choose this structure for a demonstrated similarity-search need, rather than as a replacement for authoritative records or exact constraints.

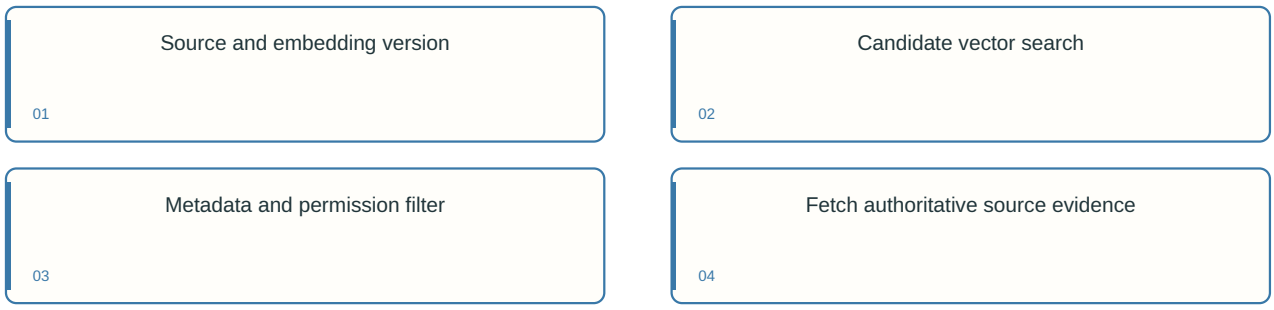


FIG. 9.1.9 **Vector indexes: introductory architecture only · the example**

Vector indexes retrieve similarity candidates from embeddings; scores establish neither truth nor access permission.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How should restoration from a historical backup respect later deletion decisions?

Restore into a controlled path, apply relevant deletion decisions, and reconcile raw and derived copies before serving. Rebuild indexes from the permitted current state rather than blindly expose an older snapshot. Keep deletion evidence narrow and purpose-limited. The backup and live-data lifecycle need a combined recovery procedure.

Remember: Choose storage from operations and invariants, then verify the selected implementation's guarantees.

Try applying it: What would make a document model suitable for a profile but unsuitable for an unbounded chat history?

Unit 9 exercises → · Unit 9 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Carnegie Mellon Database Group: courses ↗

   University courses. Database internals reading.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

Files, objects & analytical layouts

Prerequisites: 9.1 · Storage promises & database families 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A PDF, a checkout row, and an analytical revenue scan read different shapes of data. Separate storage interfaces and physical layouts from the logical facts they represent.

9.2.1 · Object storage vs block vs file storage

Object storage offers named blobs and metadata. Block storage offers addressable blocks. File storage offers a filesystem namespace and operations on files. These interfaces suit different callers.

A PDF upload often fits object storage with separate application metadata. A database engine may require a filesystem backed by suitable block storage. Check the actual protocol and implementation rather than assuming category-wide guarantees. An uploaded object and a database reference are also separate changes: either can exist without the other after a failure. Track incomplete uploads, orphaned objects, and deletion so the application knows when a file is actually usable.

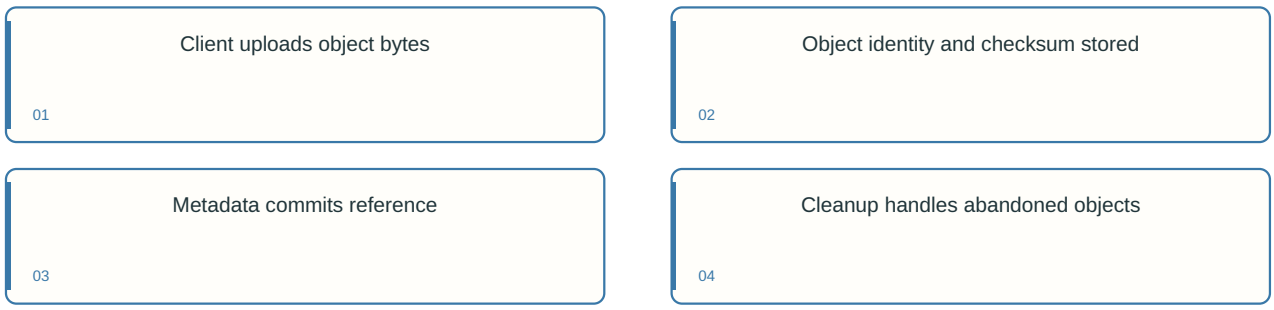


FIG. 9.2.1 **Object storage vs block vs file storage · the example**

Object, block, and file interfaces expose different byte-access and lifecycle contracts.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.2.2 · OLTP vs OLAP

Checkout performs short transactions over current orders. A revenue report may scan years of payments. OLTP describes the first kind of transactional workload; OLAP describes the second kind of analytical work.

A large report on the checkout database can compete for memory and I/O. Copying or transforming data into an analytical store can reduce that interference, but adds lag, lineage, and reconciliation requirements. One engine may serve both uses adequately at a smaller scale. Measure their interaction before introducing another system, and decide how current analytical results need to be. The labels explain workload differences, not mandatory product assignments.

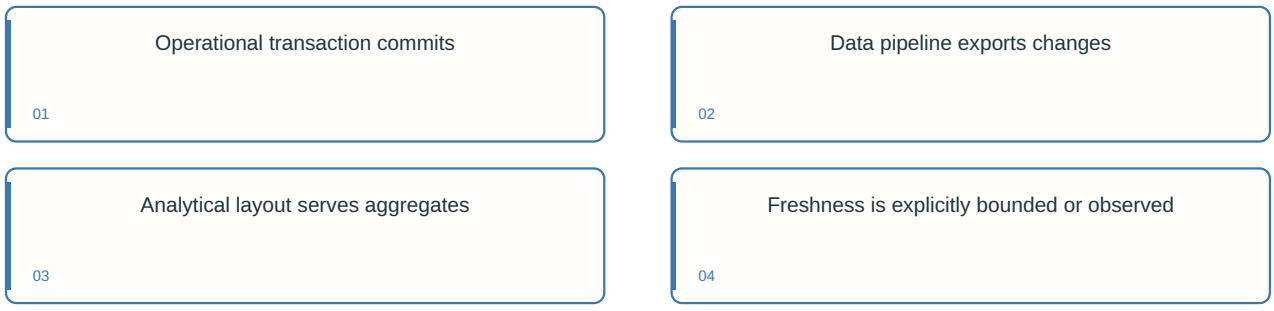


FIG. 9.2.2 **OLTP vs OLAP · the example**

OLTP serves short operational transactions; OLAP serves analytical scans and aggregation, often with different interference costs.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.2.3 · Row-oriented vs column-oriented storage

To display one customer's full record, it is convenient to read fields stored together in a row. To sum revenue over millions of records, reading only an amount column can avoid unrelated fields and support compression.

Row-oriented storage groups fields by record; column-oriented storage groups values by field. Each changes locality and the cost of reads and updates. Modern engines can combine techniques, so inspect the actual access path. Storage layout does not replace a sound logical model or useful indexes. Choose it according to which bytes the important queries need to read and how frequently those values change.

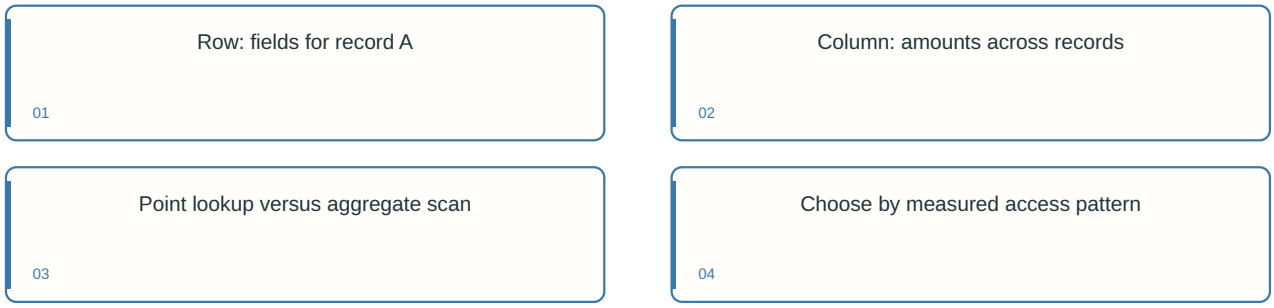


FIG. 9.2.3 **Row-oriented vs column-oriented storage · the example**

Row and column layouts group physical values differently to support different read and update patterns.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.2.4 · Normalization vs denormalization

A customer's current address belongs in their profile. A completed order may also keep the address used for that shipment. This repetition preserves a historical fact; changing the profile should not rewrite the old order.

Normalisation reduces duplicated facts and update anomalies. Denormalisation deliberately repeats or derives data for a purpose, often a faster read. State which meaning a copy has. A cached author name might be a historical snapshot or a value that should follow future edits. Without that distinction, developers cannot know whether a difference is correct history or stale data. Every maintained copy needs an update and recovery rule.

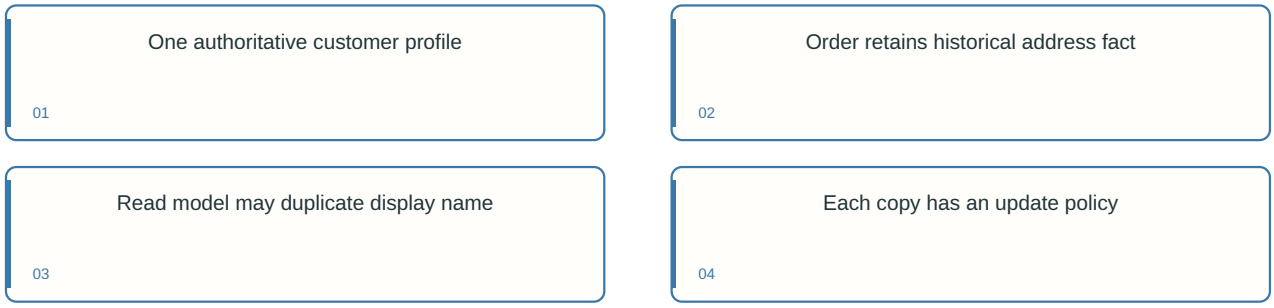


FIG. 9.2.4 **Normalization vs denormalization · the example**

Normalisation reduces repeated facts; deliberate copies require a clear historical or maintained-current meaning.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How should restoration from a historical backup respect later deletion decisions?

Restore into a controlled path, apply relevant deletion decisions, and reconcile raw and derived copies before serving. Rebuild indexes from the permitted current state rather than blindly expose an older snapshot. Keep deletion evidence narrow and purpose-limited. The backup and live-data lifecycle need a combined recovery procedure.

Remember: Group bytes for the access pattern while preserving the meaning of current and historical facts.

Try applying it: Why should changing a customer's address leave the shipping address of an old order unchanged?

Unit 9 exercises → · Unit 9 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Carnegie Mellon Database Group: courses ↗

   University courses. Database internals reading.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

Model relationships and ownership

Prerequisites: 9.2 · Files, objects & analytical layouts 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An order line refers to an order and a product, but those records have different lifecycles. Model keys, cardinalities, and ownership so data stays meaningful as it changes.

9.3.1 · ER diagrams and relational modeling

An order has several lines. Each line belongs to one order, refers to a product identity, and retains the purchase-time values needed for its history. An entity-relationship model makes those identities and cardinalities explicit.

Show keys, optional and required links, and the constraints that protect them. Also consider lifecycle: deleting a product must not remove the financial meaning of an old order. A diagram is more useful when it states these rules rather than merely joining table names. Translate the relevant constraints into the database or application so they are actually enforced.

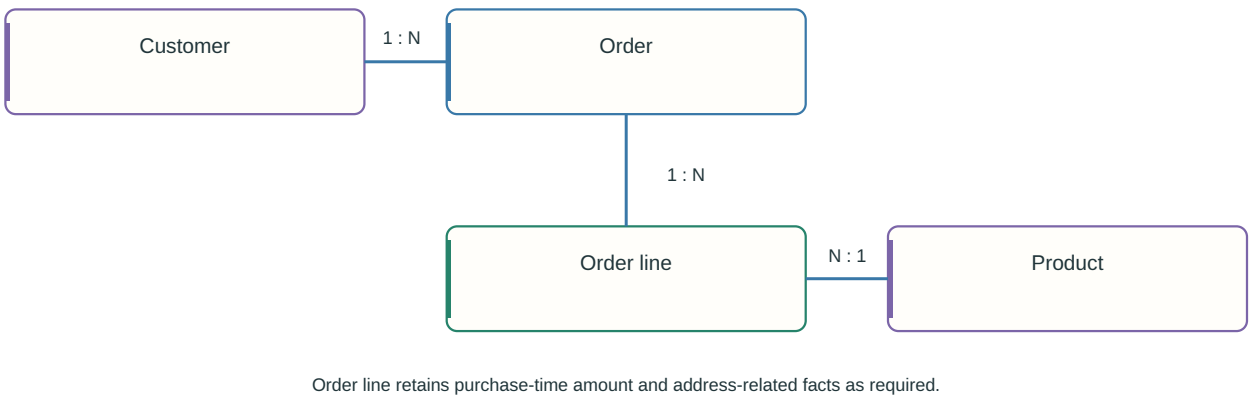


FIG. 9.3.1 **An order schema preserves relationships and history**

Customer has many Orders, Order has many Lines, and each Line references a Product identity while retaining purchase-time values.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.3.2 · Referential integrity

An order line should not point to an order that does not exist. A suitable foreign key can enforce that relationship in a relational database. This is referential integrity.

Decide deletion behaviour deliberately: automatically deleting dependent records may be wrong for historical payments. An application check before insertion can also race with deletion unless an atomic mechanism protects the relationship. Across separate services, one local foreign key cannot enforce a remote record's lifetime. Use stable identifiers, explicit ownership contracts, and reconciliation appropriate to the business rule. The integrity requirement stays important even when its enforcement crosses different storage boundaries.

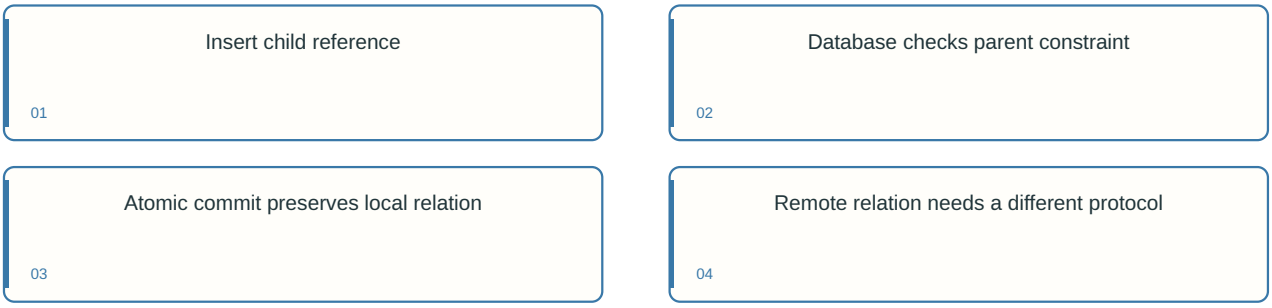


FIG. 9.3.2 **Referential integrity · the example**

Referential integrity protects valid relationships using mechanisms appropriate to the ownership and transaction boundary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.3.3 · Data ownership

Payments records whether a charge was accepted. Analytics may keep a copy for reports, but should not rewrite that financial outcome. Data ownership identifies who may define and change each fact.

If several services directly edit one table, they share transaction and release assumptions even if their source repositories are separate. Provide controlled read and change contracts, with only the permissions each consumer needs. Ownership also includes schema changes, incident response, retention, and recovery. Naming a table owner is a start; the design must give that owner a way to preserve its rules while others use the data.

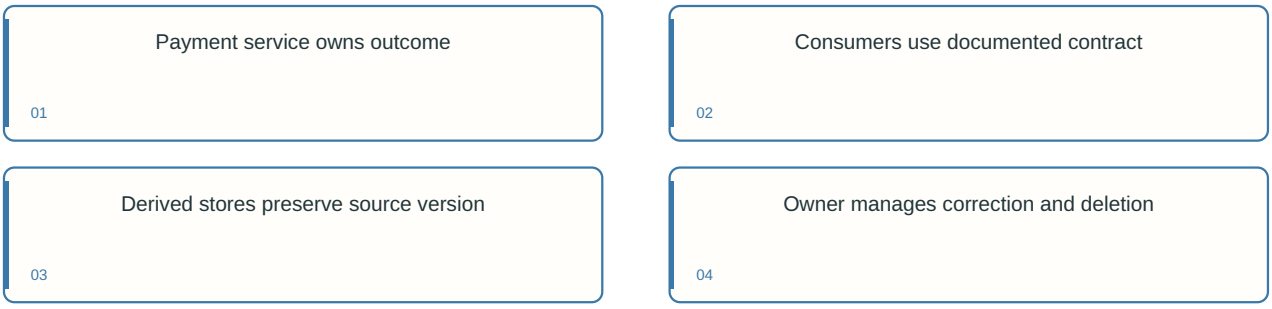


FIG. 9.3.3 **Data ownership · the example**

Data ownership includes who changes a fact and who maintains its access, schema, retention, and recovery.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.3.4 · Choosing storage based on access patterns

List the operations before assigning an engine. Profiles need particular lookups and edits. Financial entries need transaction and audit rules. Chat history needs bounded conversation/time reads. PDFs need large-byte storage and lifecycle management.

Several of these needs may be met by one system. A relational or document model could both serve a profile, depending on its actual constraints. Add another engine when its capability solves a demonstrated problem. Each additional store introduces backup, permissions, migration, monitoring, and expertise costs. Prefer a storage choice justified by the operations it supports over one chosen merely to include every database category in the diagram.

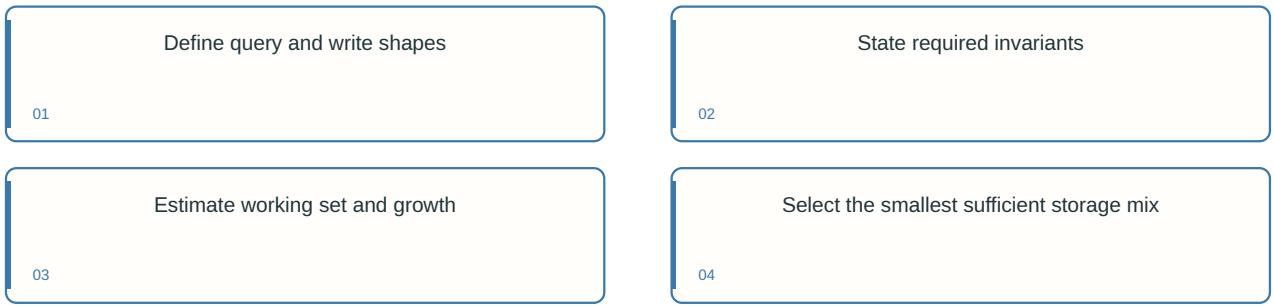


FIG. 9.3.4 **Choosing storage based on access patterns · the example**

Storage selection follows actual queries and invariants, including the complete cost of another engine.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How should restoration from a historical backup respect later deletion decisions?

Restore into a controlled path, apply relevant deletion decisions, and reconcile raw and derived copies before serving. Rebuild indexes from the permitted current state rather than blindly expose an older snapshot. Keep deletion evidence narrow and purpose-limited. The backup and live-data lifecycle need a combined recovery procedure.

Remember: Relationships need both an expressed rule and a mechanism that preserves it.

Try applying it: What must happen to completed order history when a product is removed from sale?

Unit 9 exercises → · Unit 9 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Carnegie Mellon Database Group: courses ↗

   University courses. Database internals reading.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

Access patterns, retention & operational cost

Prerequisites: 9.3 · Model relationships and ownership 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Adding a search engine also adds another recovery and deletion path. Follow raw and derived copies through retention and operation before deciding that another store is worth maintaining.

9.4.1 · Data retention and lifecycle

A document's lifecycle includes its original bytes, extracted fields, indexes, archives, and recoverable copies. Deleting only the live row may leave other copies usable.

Define purpose, retention, expiration, archival, and deletion behaviour from the start. Dropping time partitions can efficiently expire old data; deleting one user's records needs identity-aware propagation. A minimal deletion marker may be needed to prevent old events from recreating removed data, but that marker can itself retain personal information. Keep only what its purpose needs. Test restoration from an old backup so required deletion decisions are reapplied instead of silently reversed.

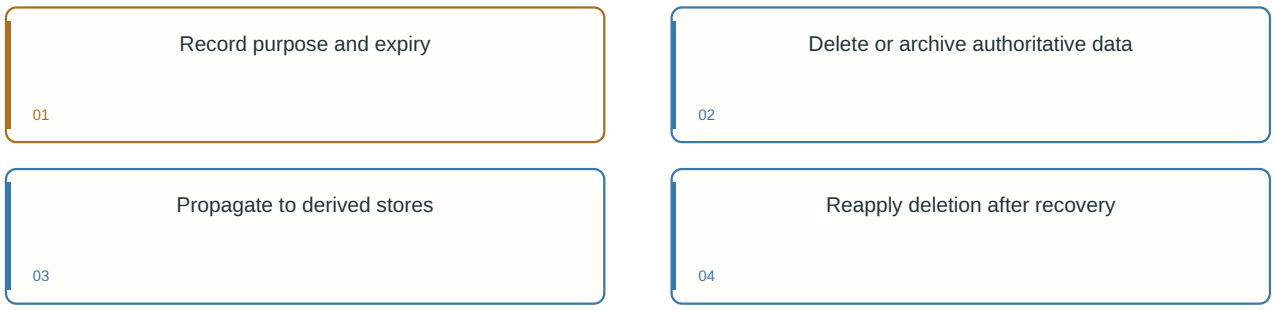


FIG. 9.4.1 **Data retention and lifecycle · the example**

Retention follows raw and derived copies through expiry, archival, deletion, replay, and recovery.

Author-created illustration. Numbers are illustrative unless stated otherwise.

9.4.2 · Polyglot persistence and its operational costs

A shop adds a search engine because its current queries cannot meet a demonstrated relevance need. It now uses different stores for different workloads: polyglot persistence.

The extra capability comes with indexing lag, rebuilds, permissions, schema changes, and another recovery procedure. A change across two stores is not automatically one transaction. Identify which record is authoritative and which representation can be rebuilt, then monitor progress between them. Compare the complete operating cost with the benefit of the new query path. Keeping fewer engines can be an advantage when one already meets the requirements.

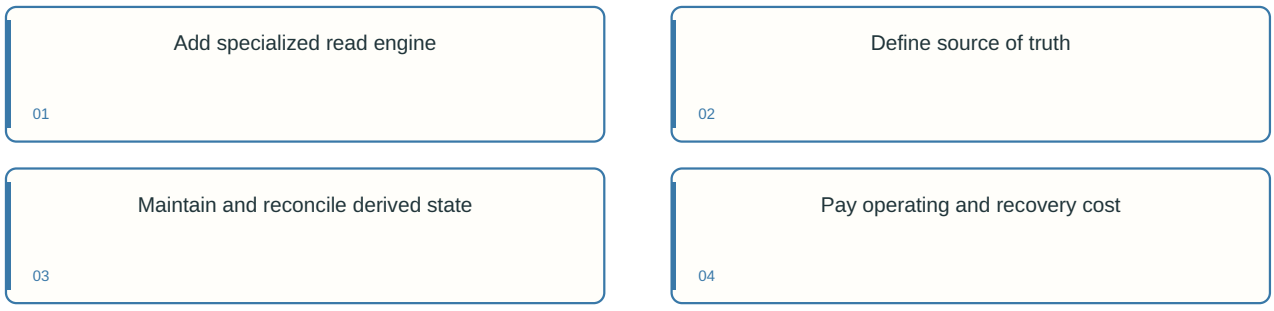


FIG. 9.4.2 **Polyglot persistence and its operational costs · the example**

Polyglot persistence combines engines for useful capabilities while adding operating and cross-store coordination work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

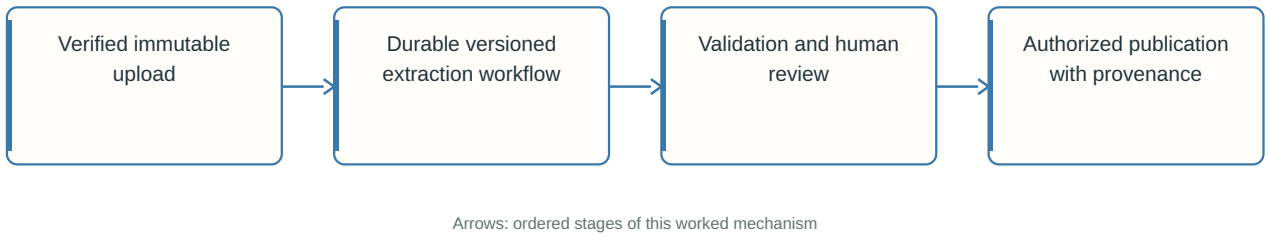


FIG. 9.4.3 **Follow the related complete design**

Ingestion, extraction, review, and publication have distinct durable progress.

Author-created illustration. Numbers are illustrative unless stated otherwise.

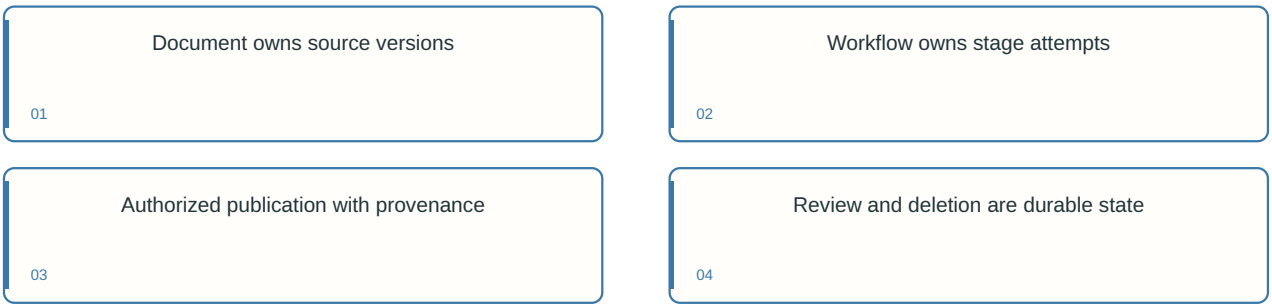


FIG. 9.4.4 **State and ownership in the related design**

Fields and decisions retain tenant, input, transform, and evidence relationships.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PSEUDOCODE · BYTES AND METADATA

```
upload = create_authorized_upload(tenant, expected_size)
object = store_and_verify_bytes(upload)
with transaction():
    insert_document(tenant, object.version, object.digest)
    insert_processing_intent(object.version)
# Orphan-byte cleanup handles incomplete metadata handoff.
```

The metadata points to a verified immutable object version and records the processing obligation. If metadata commit fails, the object can remain orphaned and needs reconciliation or cleanup.

The local transaction does not include the earlier separate object upload. Track that handoff explicitly instead of describing it as one cross-service commit.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · How should restoration from a historical backup respect later deletion decisions?

Restore into a controlled path, apply relevant deletion decisions, and reconcile raw and derived copies before serving. Rebuild indexes from the permitted current state rather than blindly expose an older snapshot. Keep deletion evidence narrow and purpose-limited. The backup and live-data lifecycle need a combined recovery procedure.

Remember: Additional engines earn their cost through a demonstrated capability and a complete lifecycle.

Try applying it: How could restoring an old backup bring deleted content back into a search index?

Unit 9 exercises → · Unit 9 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Carnegie Mellon Database Group: courses ↗

   University courses. Database internals reading.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

PART 3 / UNIT 10

Database Internals, Transactions, and Concurrency Control

A task lookup is quick for one identifier but slow for another. Both requests reach the same database, so adding another application server does not yet tell us how to help.

The running example

Two users attempt to purchase the last available ticket. Demonstrate the lost-update race, show why naive reads fail, and implement valid alternatives using transactions and concurrency control.

Unit 10 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

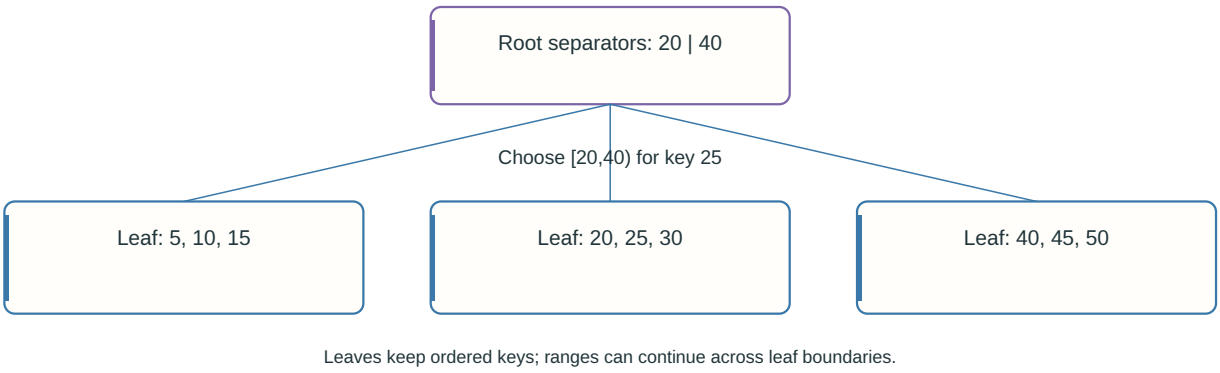


FIG. 10.0.1 **B+tree lookup narrows an ordered key range**

Illustrative keys and leaves demonstrate lookup and range locality. Exact database page layout is implementation-specific.

Author-created illustration. Numbers are illustrative unless stated otherwise.

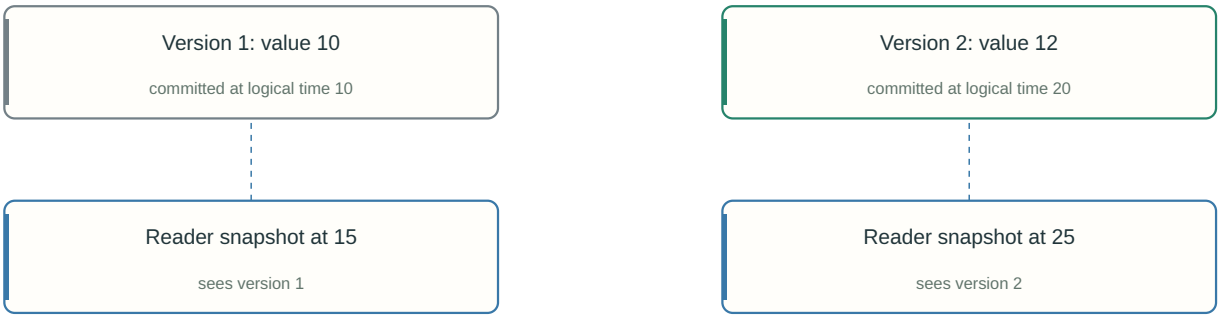
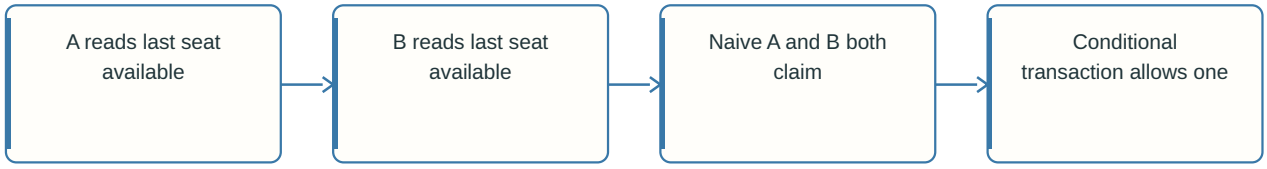


FIG. 10.0.2 **Two snapshots can legitimately see different versions**

A reader whose snapshot precedes version two can keep seeing version one while a later reader sees version two. MVCC alone does not imply serializability.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 10.0.3 **concurrent transaction interleavings**

The observable acknowledgment history, not only the final row, reveals overselling.

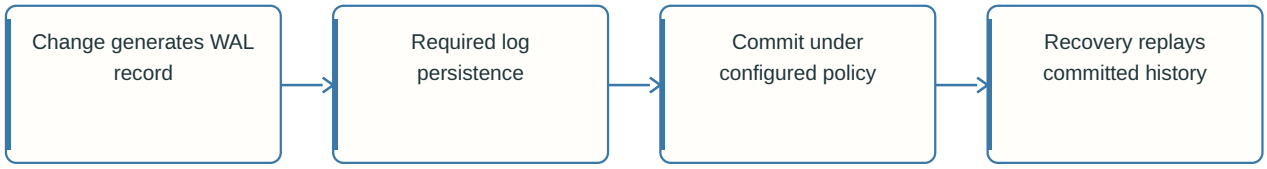
Author-created illustration. Numbers are illustrative unless stated otherwise.

Holder	Owns	Waits for
Transaction A	Row X	Row Y
Transaction B	Row Y	Row X
Result	Circular wait	Abort/retry or consistent ordering

FIG. 10.0.4 **lock wait graph**

A lock-wait graph contains a cycle when each transaction waits on a resource held by the other.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 10.0.5 **WAL and recovery**

Data pages can lag the log under a correct recovery protocol; a checkpoint is not a backup.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Pages, indexes & query plans

Prerequisites: Runtime, Processes, Threads, and Concurrency, Data Models and Storage Technologies 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A task lookup is quick for one identifier but slow for another. Both requests reach the same database, so adding another application server does not yet tell us how to help.

We need to follow the work behind the query: how records are grouped into pages, how indexes locate candidates, and when storage must supply data that memory does not hold. The planner's choices connect these structures to the SQL we write.

In this chapter, we will trace a lookup and a range scan, examine the later work caused by LSM writes, and use query plans to judge an index. The aim is to identify the physical work a proposed improvement actually avoids.

10.1.1 · How databases store and retrieve records

A lookup by task identifier looks simple to its caller. Inside the database, the engine must choose a way to find the task, obtain the stored data, and decide which row version the caller may see. Following these steps helps us explain where the time goes.

An index is a separate search structure that helps locate records by selected values. For a lookup using an index, the engine searches its entries and follows the resulting record reference. It may need to bring parts of both the index and the table from storage into memory. Other queries may scan the table instead. The planner chooses an access path: the sequence of scans, searches, and other operations used to obtain the result.

Two queries can therefore wait for different reasons. One needs storage reads; another spends CPU time filtering records; a third waits for a lock held by another transaction. Inspect the plan and resource behaviour before adding servers. The useful change is the one that addresses the expensive step.

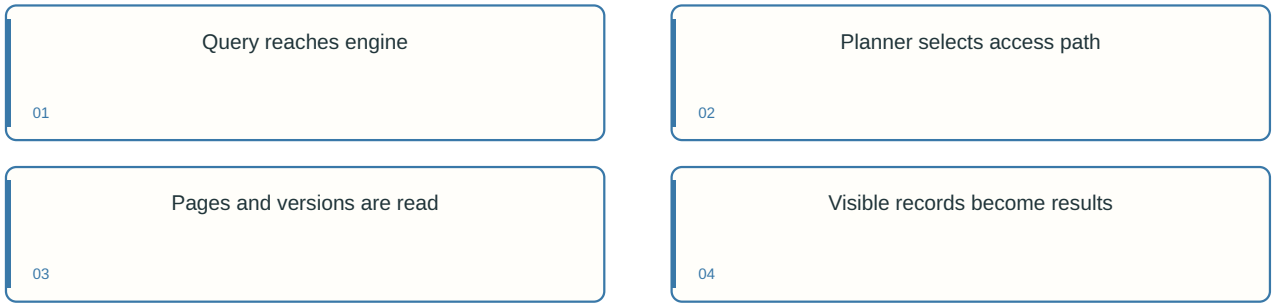


FIG. 10.1.1 **How databases store and retrieve records · the example**

A record lookup performs physical and visibility work through indexes, pages, memory, and storage.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.1.2 · Pages, records, indexes, and buffer pools

Many database engines organise storage into pages. A page is a unit of storage the engine reads and manages together; it can contain several records or index entries. A buffer pool keeps useful pages in memory so later operations can reuse them.

Suppose an index identifies the page containing a task. If that page is already in the buffer pool, the engine can use its memory copy. Otherwise, it must obtain the page from lower storage layers, which may require device I/O. Even a memory hit still involves work, such as finding the record and checking its visibility. The operating system may also cache data, so a database buffer miss need not mean a physical device read.

The relevant memory question is which actively used pages fit. A query that visits a million scattered pages can put heavy demand on storage even when it returns few records. Extra web servers may only send more of those reads. Examine the pages touched and their locality: whether related accesses reuse nearby or already cached data. Page layout and replacement rules vary by engine.

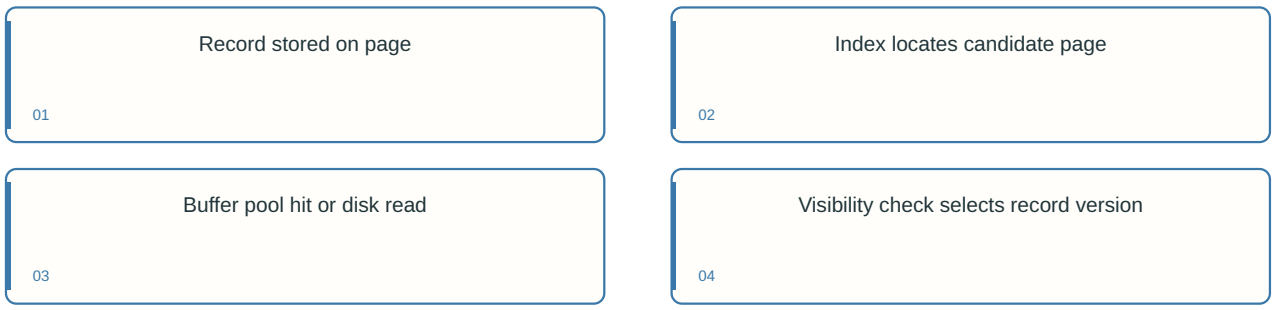


FIG. 10.1.2 **Pages, records, indexes, and buffer pools · the example**

Pages organise records; buffer pools cache pages, while query locality determines which work can be avoided.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.1.3 · B-tree/B+tree and hash indexing

Suppose a query asks for keys 20 through 35. An ordered index can locate the beginning of that range and continue through its entries, rather than checking every record. B-tree-family structures organise keys into a tree of pages, with separators that direct a search toward a smaller range.

In the figure, the root has separators 20 and 40. Key 25 belongs to the middle range, [20,40), so the search follows that branch to the leaf containing 20, 25, and 30. B+tree designs commonly keep records or record references in leaves, and allow a range scan to continue through neighbouring leaves. Larger trees add internal pages between the root and leaves; each step narrows the range.

A hash index instead uses a hash calculated from a key to find equality candidates. It does not provide the same ordered walk from 20 to 35. Choose an index using the query's conditions, ordering, engine support, and update cost. A short tree path can still require page reads, and a range returning many rows still takes time to read and deliver.

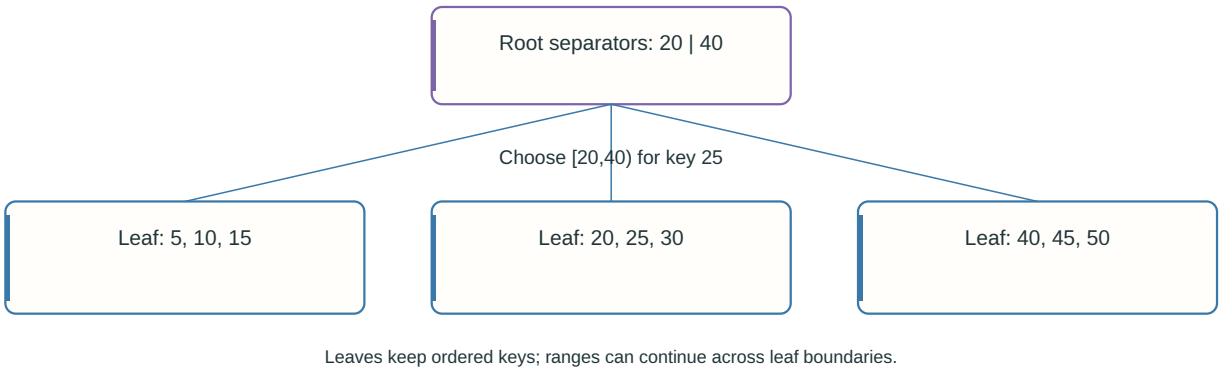


FIG. 10.1.3 **B+tree lookup narrows an ordered key range**

Illustrative keys and leaves demonstrate lookup and range locality. Exact database page layout is implementation-specific.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.1.4 · LSM trees and write amplification

An LSM tree, short for log-structured merge tree, organises updates by collecting them in memory and writing sorted groups to storage. Such a group is called a sorted run. Later, compaction merges runs and removes versions that the engine no longer needs.

In the illustrated path, the engine first records the write in a recovery log under its durability policy, then updates the in-memory structure. When that structure is flushed, it becomes an immutable sorted run: later writes go elsewhere rather than editing that run in place. After several flushes, the same key may appear in several runs with different values. A read must find the relevant version; compaction combines this scattered history when permitted.

Batching can turn many small changes into more sequential storage work. But compaction reads and writes existing data again. Write amplification is the physical bytes written divided by the bytes of logical changes. Its value depends on the layout and workload. Observe sustained ingestion and tail latency, because compaction can compete with requests for bandwidth or fall behind and cause stalls. A short burst that fits in memory conceals this later work.

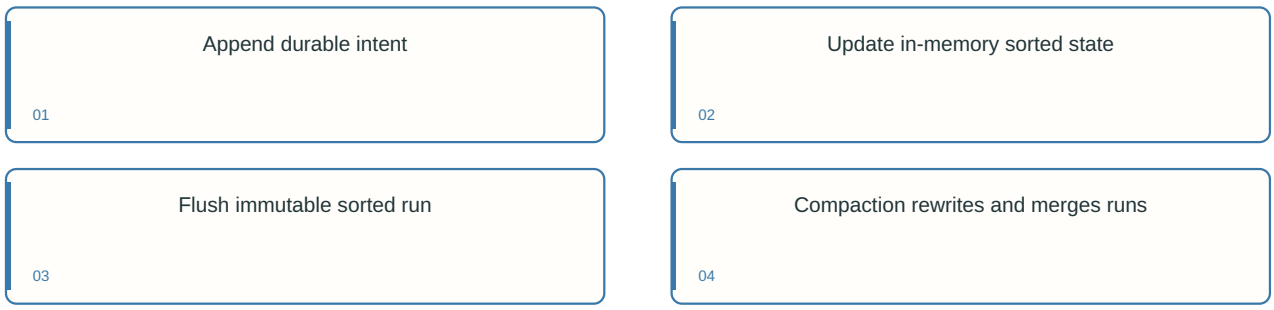


FIG. 10.1.4 **LSM trees and write amplification · the example**

LSM organisation batches sorted writes and compaction, trading ingestion behaviour against read work and amplification.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.1.5 · Query planning and EXPLAIN

A SQL query describes the desired result. The planner chooses how to obtain it, using statistics about the data and estimates of the work required. EXPLAIN shows that selected plan. An execution-analysis form can also report actual work; it may run the statement, including its side effects.

Suppose the query asks for yesterday's orders. An index search may be useful if few orders match. If most of the table matches, a scan may be cheaper than many separate record fetches. The estimate of how many rows will match is called a cardinality estimate, and it influences later choices such as how to join the result to another table.

Now suppose the planner expects ten rows and the operation actually produces a million. A join strategy chosen for the small estimate may perform much more work than expected. Compare estimated and actual row counts, filters, sorts, scans, and buffer activity. Updated statistics or a query rewrite may address the cause. Check the engine's command behaviour before analysing a statement that changes data.

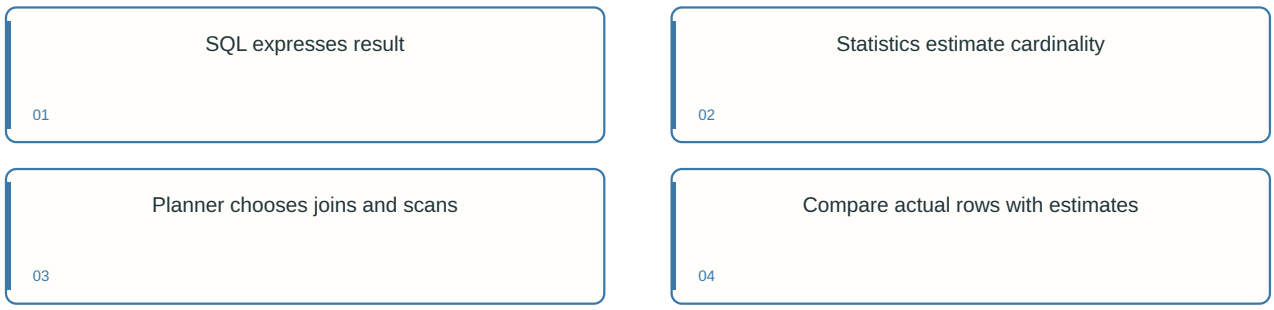


FIG. 10.1.5 **Query planning and EXPLAIN · the example**

A query planner chooses physical execution using estimates; compare plans with actual work where safely supported.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.1.6 · Index selection and query optimization

A tenant-scoped list query filters by tenant and sorts by time and identifier. An ordered index on (tenant_id,created_at,id) may support that access path, depending on the engine and query.

The leading tenant field groups one tenant's entries. Within that group, the time and ID fields provide the requested order; the ID distinguishes records with the same time. A query that fixes the tenant can locate that group and walk a bounded matching range. If it filters or orders by different fields, the same index may fit less well. An index serves particular operations, rather than making every query on the table faster.

Each index also consumes space and must be maintained when relevant data changes. A covering index contains the values needed for a query and may avoid some table-record fetches, subject to the engine's visibility rules. Its larger entries also use more memory and storage. Test representative values and distributions: a small uniform dataset can conceal a tenant holding most of the rows. Inspect both the resulting read plan and the additional write cost.

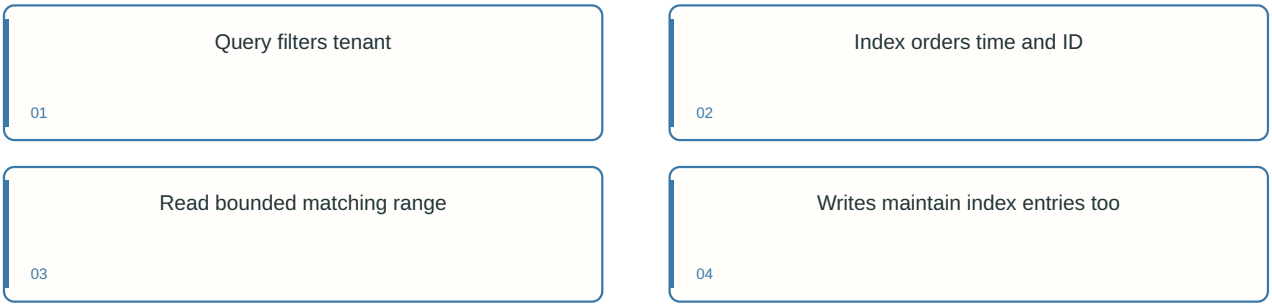


FIG. 10.1.6 **Index selection and query optimization · the example**

Indexes improve selected reads while adding space, maintenance, and write costs.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What changes when one local transaction is divided across independent databases?

The two changes no longer automatically share one commit decision. Suppose the seat update commits in one database but inserting the reservation fails in another. Each store can behave correctly locally while the combined booking is incomplete.

First identify which invariant needs agreement. A distributed transaction may supply a suitable shared decision if the stores and operating requirements support it. Alternatively, a workflow can represent intermediate states explicitly and use retries, reconciliation, or compensation to reach an acceptable outcome. That choice must define what the caller sees while work is incomplete. Separate local commits alone do not preserve the former all-or-nothing business promise.

Remember: Follow the query from its plan through index and data pages. Judge an index by the work it avoids and the write work it adds.

Try applying it: An index makes a tenant list faster, but inserting tasks becomes slower. What additional work does each insert now require?

Unit 10 exercises → · Unit 10 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

3. PostgreSQL 18: Write-Ahead Logging ↗

   Official versioned documentation. Logging and recovery ordering; durability remains configuration-dependent.

   Access reviewed · 21 October 2026

4. RocksDB Overview ↗

   Official implementation documentation · accessed 21 Oct 2026. An LSM-based engine; exact tuning is implementation-specific.

   Access reviewed · 21 October 2026

Commit, logging & versions

Prerequisites: 10.1 · Pages, indexes & query plans 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Our application transfers money between two accounts and reports that it has saved the result. A reader may still see an earlier row version while the writer commits a new one, and a crash may occur before every changed page has reached storage.

These observations raise different questions. Which changes belong together? When has the database accepted them? What information allows recovery, and which version should each reader see?

In this chapter, we will separate the ACID properties, follow commit and a lost reply, and connect the recovery log to delayed page writes. We will then use a version timeline to explain how readers can have different permitted views of the same row.

10.2.1 · ACID properties

Start with account A holding 100 and account B holding 50. A transfer of 10 should leave A=90 and B=60, preserving the total of 150. Saving only the debit would leave the two-account model short by 10. A transaction groups the related database changes into one unit.

ACID names four properties of transactions. Atomicity means the related changes are accepted together or abandoned together. Consistency concerns preserving the required invariants: rules such as preserving this total. The application logic and database constraints must correctly express those rules. Isolation controls the observations and interactions allowed when transactions run concurrently. Durability concerns retaining committed changes under the configured guarantees and failure assumptions.

These properties answer different questions. Atomicity can keep the debit and credit together, but it cannot recognise that the application calculated the wrong amount. Likewise, placing statements in a transaction

does not tell us which isolation level they use. To judge the transfer, check its calculation, constraints, concurrency behaviour, and commit guarantee.

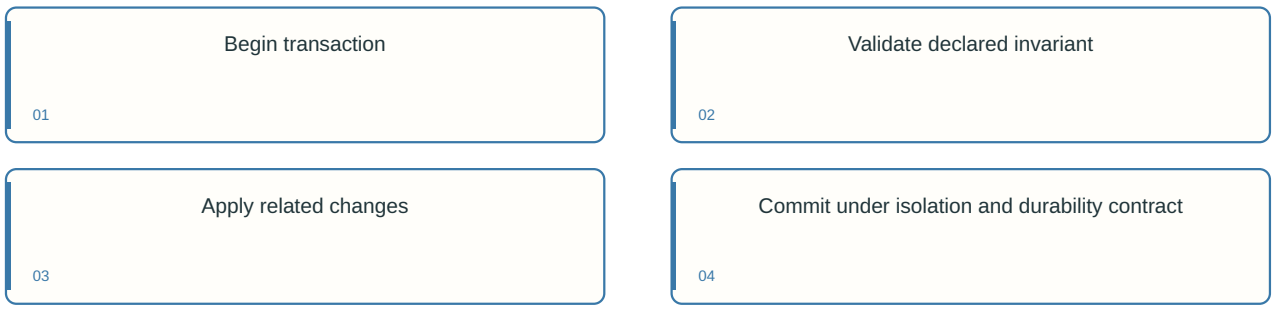


FIG. 10.2.1 **ACID properties · the example**

ACID separates atomicity, invariant preservation, isolation, and conditional durability; application rules still need correct expression.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.2.2 · Transaction commit and rollback

The application changes rows and asks the database to commit. Commit accepts the transaction under the engine's configured guarantees. If the application promises a saved result, it should report success only after the commit succeeds. A rollback abandons the transaction's uncommitted database changes.

The database's decision and the client's receipt of that decision are separate events. Suppose the database commits, then the connection breaks before the reply reaches the application. The rows may already be saved even though the caller has no confirmation. A connection error at this point does not establish rollback. Recovery needs an operation identity stored with the change, or another reliable way to find its outcome before deciding whether to try again.

The transaction boundary also limits what rollback can undo. If the application already charged a payment provider over HTTP, rolling back SQL does not reverse that charge. Replaying the operation without accounting for the earlier charge can repeat an external effect. Trace local changes, commit, and remote actions separately when designing recovery.

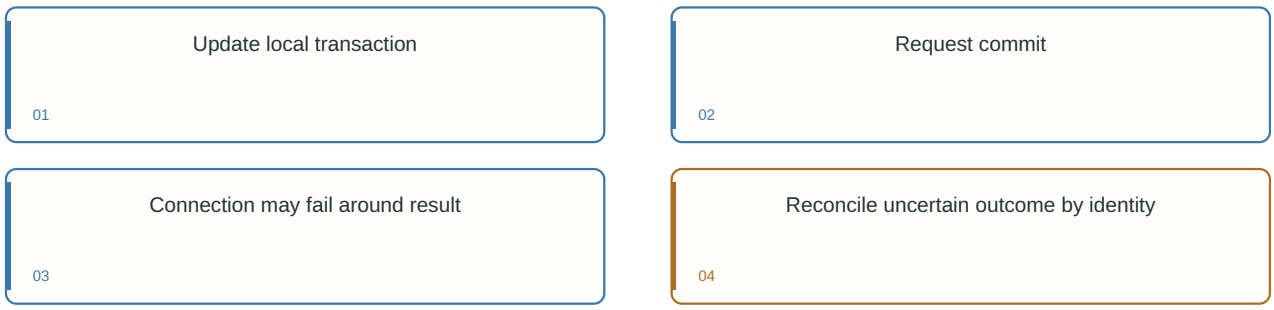


FIG. 10.2.2 **Transaction commit and rollback · the example**

Commit accepts a transaction under the engine protocol; rollback abandons uncommitted work and cannot reverse unrelated remote effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.2.3 · Write-ahead logging

A database may commit while modified data pages are still in memory, awaiting their final storage writes. Write-ahead logging, or WAL, makes this possible by recording recovery information before the corresponding changed pages are written to persistent storage.

Consider a logged change to a task. The engine creates the log records and modifies the page in memory. Under a commit policy that requires durable local WAL, it flushes the required records, including the commit record, before acknowledging success. It can write the data page later. If a crash happens first, recovery uses the persisted log to redo the required work and restore the appropriate database state. Commit therefore need not wait for every modified table and index page to reach its final location.

The exact acknowledgement guarantee depends on synchronization and replication settings. A checkpoint advances the work already reflected in stored pages and limits later recovery work; it is not a backup. If the only device holding both data and log is destroyed, this recovery path is lost too. Independent recovery copies address that different failure.

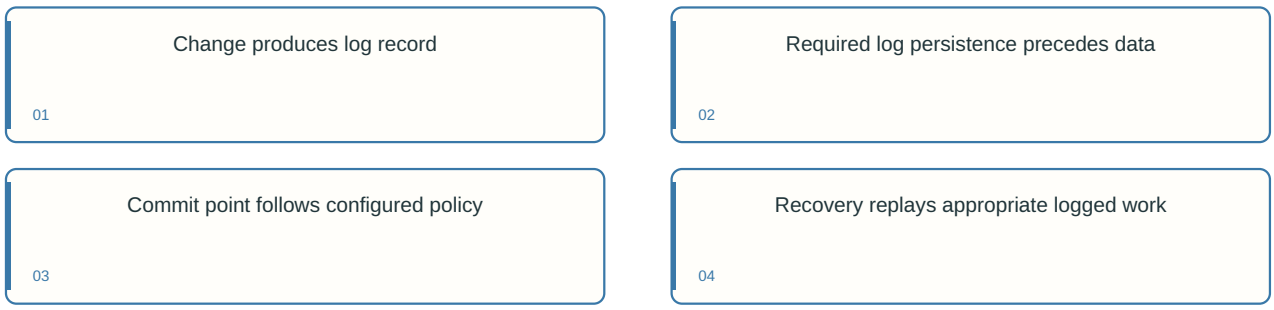


FIG. 10.2.3 **Write-ahead logging · the example**

Write-ahead logging preserves recovery information before corresponding data pages, under configured persistence rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.2.4 · MVCC

Suppose a stored price changes from 10 to 12 while a reader is using it. Multiversion concurrency control, or MVCC, lets the engine retain multiple row versions and select the one the reader is allowed to see. A snapshot defines that reader's view according to the engine's visibility rules.

In the figure, version one, containing 10, commits at logical time 10. A snapshot at 15 sees that version. Version two, containing 12, commits at 20, and a later snapshot at 25 sees it. The older snapshot can continue to use version one even after version two commits. These are illustrative logical times, rather than a requirement that an engine use wall-clock timestamps this way.

Versions reduce some direct interference between readers and writers, but the isolation policy still decides when snapshots are taken and how conflicts are handled. Long readers can keep old versions needed and delay cleanup. Stable snapshots also do not by themselves prevent two transactions from making incompatible decisions about different rows. We examine that history in the next chapter.

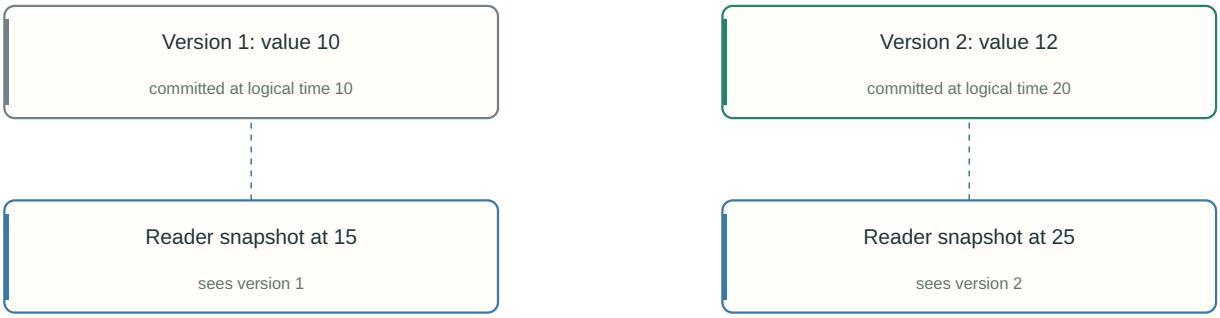


FIG. 10.2.4 **Two snapshots can legitimately see different versions**

A reader whose snapshot precedes version two can keep seeing version one while a later reader sees version two. MVCC alone does not imply serializability.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What changes when one local transaction is divided across independent databases?

The two changes no longer automatically share one commit decision. Suppose the seat update commits in one database but inserting the reservation fails in another. Each store can behave correctly locally while the combined booking is incomplete.

First identify which invariant needs agreement. A distributed transaction may supply a suitable shared decision if the stores and operating requirements support it. Alternatively, a workflow can represent intermediate states explicitly and use retries, reconciliation, or compensation to reach an acceptable outcome. That choice must define what the caller sees while work is incomplete. Separate local commits alone do not preserve the former all-or-nothing business promise.

Remember: Keep related writes in one transaction, wait for the promised commit result, and distinguish recovery information from the versions readers may see.

Try applying it: A price update commits while an older snapshot still reads the earlier price. Which visibility rule should you check before treating this as an error?

Unit 10 exercises → · Unit 10 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

3. PostgreSQL 18: Write-Ahead Logging ↗

   Official versioned documentation. Logging and recovery ordering; durability remains configuration-dependent.

   Access reviewed · 21 October 2026

4. RocksDB Overview ↗

   Official implementation documentation · accessed 21 Oct 2026. An LSM-based engine; exact tuning is implementation-specific.

   Access reviewed · 21 October 2026

Races, anomalies & isolation

Prerequisites: 10.2 · Commit, logging & versions 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Two buyers both read that the last seat is available. Each then tries to reserve it. A transaction around each caller's work helps only if its concurrency rules prevent the conflicting decisions from both being accepted.

A second example is less obvious: two doctors update different rows yet jointly leave nobody on call. Preventing one row from being overwritten does not necessarily protect a rule about several rows.

In this chapter, we will write these histories in event order, compare locks with conditional updates, and distinguish read anomalies, lost updates, and write skew. We will use PostgreSQL 18 to make the isolation rules concrete and identify what must be repeated after a serialization failure.

10.3.1 · Locking and optimistic concurrency control

Two buyers read an available seat at version seven. Each proposes an update that changes the owner and advances the version, but only where version=7. One update succeeds and advances the version to eight. The other can no longer satisfy its version-seven condition.

The condition and change must be one atomic database operation. Reading a version and then writing unconditionally in a separate step leaves time for another buyer to intervene. This guarded change is optimistic concurrency control: do the work on an expected state, then validate that expectation when applying it. For the illustrated competing claims, zero affected rows means the claim did not occur. The application must re-read and make a fresh business decision. Some isolation implementations may instead report a retryable conflict.

Pessimistic locking coordinates access before conflicting work. A transaction might lock the seat, check availability, update it, and commit before releasing protection. Another claimant waits and then checks the resulting state. Locks can also deadlock: A holds row X while waiting for Y, and B holds Y while waiting for X. Consistent lock ordering helps avoid this cycle; the engine may abort a participant to resolve it. Choose the method for the complete invariant and expected contention.

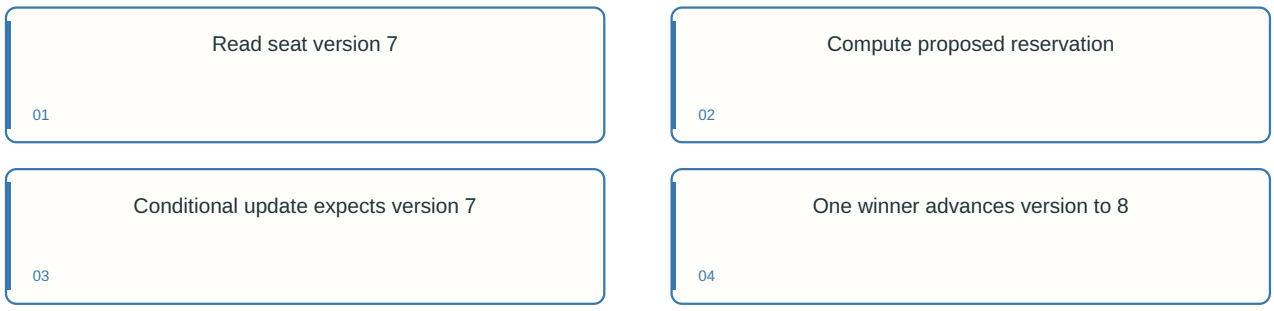


FIG. 10.3.1 **Locking and optimistic concurrency control · the example**

Locks coordinate conflicting access; optimistic checks validate expected state before applying changes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.3.2 · Dirty reads, nonrepeatable reads, phantom reads

Different concurrent histories produce different read anomalies. For a dirty read, transaction A changes a value from 10 to 20 without committing; B reads 20; A then rolls back. B has used another transaction's uncommitted value, which never became committed state.

For a nonrepeatable read, B first reads 10. A then changes the same row to 20 and commits. B reads it again and gets 20. Both values were committed when observed, but the repeated read did not stay stable.

A phantom concerns the rows matching a condition. B asks for reservations for a flight and finds three. A inserts a fourth matching reservation and commits. B repeats the same query and finds four. A new matching row appeared, even though none of the original three had to change.

These are histories to check against an isolation contract, rather than predictions for every database. For example, PostgreSQL 18 prevents dirty reads even when Read Uncommitted is requested, and its Repeatable Read also prevents phantoms. Other implementations can differ. Testing one repeated row read cannot establish all the guarantees needed for a query over a set of rows.

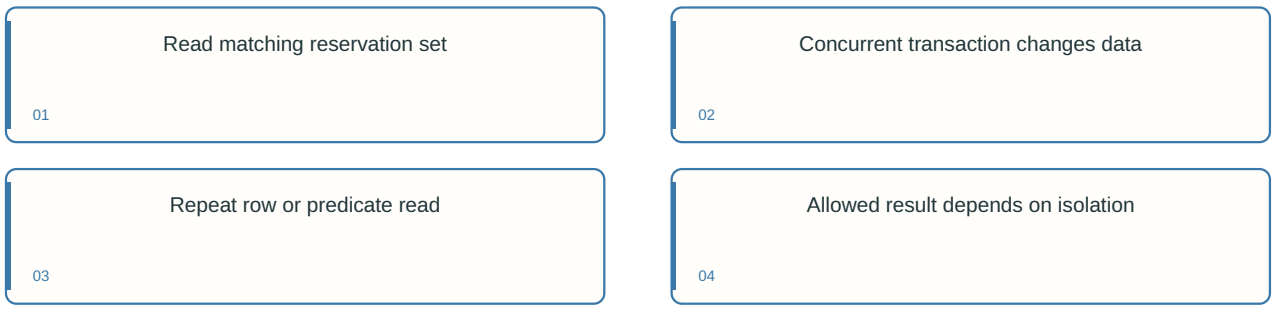


FIG. 10.3.2 **Dirty reads, nonrepeatable reads, phantom reads · the example**

Dirty, nonrepeatable, and phantom reads describe different concurrent histories whose occurrence depends on isolation implementation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.3.3 · Lost updates, write skew, serialization anomalies

A lost update can arise when two callers calculate from the same old value. Both read a counter at 10. A calculates 11 and writes it; B also calculates 11 and writes it later. If both writes commit, the final value is 11, although two increments were intended. B's stale replacement lost A's increment. This history depends on the statement form and conflict rules; an atomic increment or a checked version can change the outcome.

Write skew has a different shape. Two doctors are on call. Transaction A sees doctor B on call and turns A off. Transaction B sees the earlier state with A on call and turns B off. If both commit, nobody remains on call. They changed different rows, so neither needed to overwrite the other's write. Each decision depended on a read that the other transaction made obsolete.

The on-call rule spans both rows. With a correct check in serial execution, the second doctor would see that the first had left and would stay on call. The concurrent outcome has no equivalent serial ordering; it is a serialization anomaly. Protect the full rule with suitable serializable transactions, coordinated locking, or a shared conflict point. Protecting only each doctor's own row is insufficient.

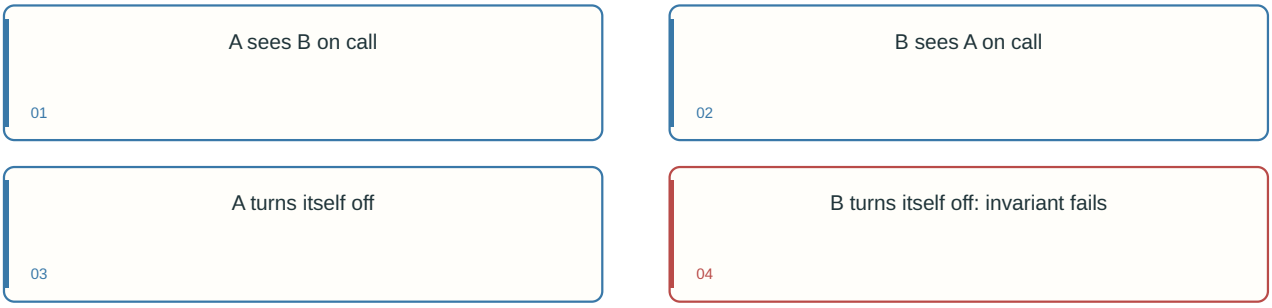


FIG. 10.3.3 **Lost updates, write skew, serialization anomalies · the example**

Lost updates overwrite work; write skew can violate a multi-row rule even when separate rows are protected.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.3.4 · Isolation levels and implementation differences

An isolation level states which interactions concurrent transactions may have. Its implementation is part of the application's contract, so name the engine and version when relying on a particular behaviour.

In PostgreSQL 18, ordinary Read Committed SELECT statements each use a new snapshot. Another transaction can commit between two SELECTs, and the second can see that change. Repeatable Read uses a snapshot established by the first non-transaction-control statement and excludes later concurrent commits from that view. It prevents nonrepeatable and phantom reads, but still permits serialization anomalies such as suitable write-skew histories. A stable view and a serially explainable outcome are different requirements.

PostgreSQL Serializable adds checks for dependencies that could prevent a serial equivalent. The database may reject a transaction with a serialization failure; Repeatable Read can also require retries after concurrent updates. Restart the whole failed transaction, including its reads and decisions, with bounded attempts. Repeating only the last write retains assumptions from the failed attempt. Make external effects safe under replay before placing them inside retrying application code. Other engines and statement forms need their own documented analysis.

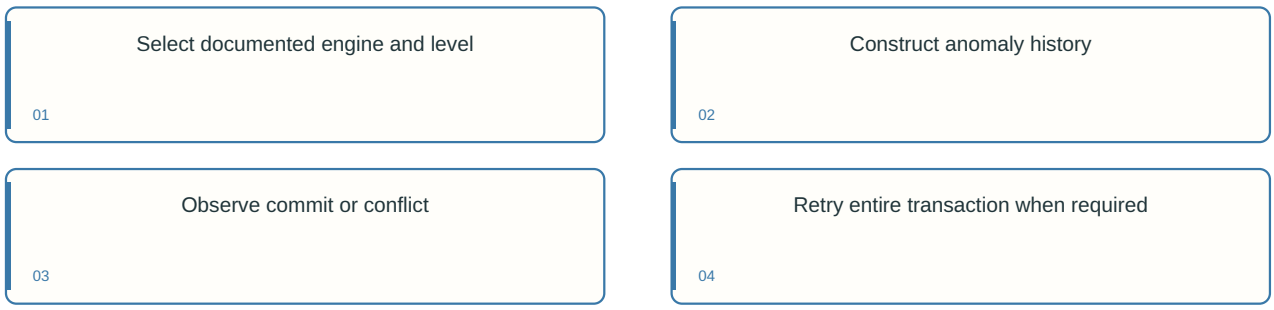


FIG. 10.3.4 **Isolation levels and implementation differences · the example**

Isolation behaviour depends on engine, version, and statement form; retries require fresh whole-transaction reasoning.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.3.5 · Serializable execution and serializability

Serializable execution allows transactions to overlap while requiring their committed effects to match some execution of those transactions one at a time. It is a property of the permitted histories; it does not require the engine to run only one transaction at a time.

Return to the doctors. If A runs first and leaves, B must see A off and stay. If B runs first, A must stay. An outcome with both off matches neither serial order. A serializable implementation must prevent that committed outcome, for example by rejecting a transaction that the application then restarts with fresh reads.

Serializability alone does not add a real-time ordering requirement between separate callers. Linearizability describes operations as taking effect at a point between their invocation and reply, while respecting real-time precedence: an operation completed before another begins must come first. Strict serializability combines this real-time requirement with transaction serializability. State which property the application needs. Even a serial execution of a wrongly calculated transfer gives the wrong amount, and a local database guarantee does not automatically include an external payment call.

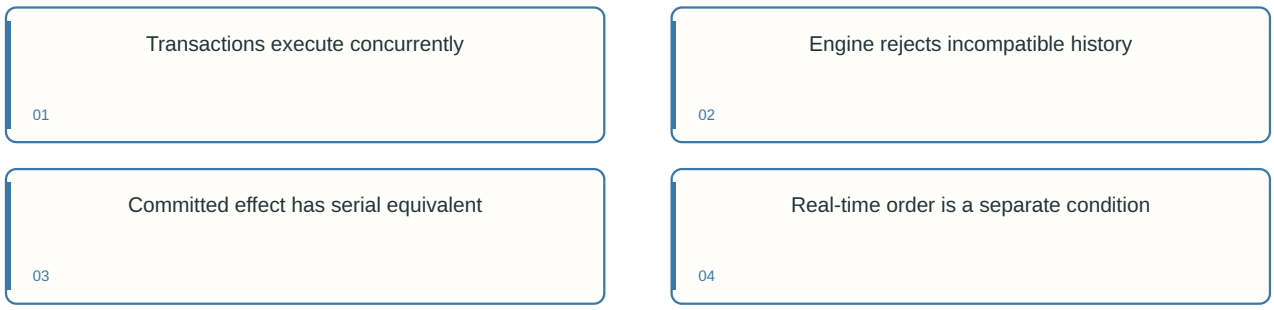


FIG. 10.3.5 **Serializable execution and serializability · the example**

Serializability concerns serial transaction equivalence; linearizability also concerns operation real-time precedence.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What changes when one local transaction is divided across independent databases?

The two changes no longer automatically share one commit decision. Suppose the seat update commits in one database but inserting the reservation fails in another. Each store can behave correctly locally while the combined booking is incomplete.

First identify which invariant needs agreement. A distributed transaction may supply a suitable shared decision if the stores and operating requirements support it. Alternatively, a workflow can represent intermediate states explicitly and use retries, reconciliation, or compensation to reach an acceptable outcome. That choice must define what the caller sees while work is incomplete. Separate local commits alone do not preserve the former all-or-nothing business promise.

Remember: Trace concurrent decisions in order and protect the complete invariant. A retry must repeat the reads and decisions of the failed transaction.

Try applying it: Two transactions each lock only the doctor they will turn off. Why can that leave the shared on-call rule unprotected?

Unit 10 exercises → · Unit 10 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   Access reviewed · 21 October 2026

3. PostgreSQL 18: Write-Ahead Logging ↗

   Official versioned documentation. Logging and recovery ordering; durability remains configuration-dependent.

   Access reviewed · 21 October 2026

4. RocksDB Overview ↗

   Official implementation documentation · accessed 21 Oct 2026. An LSM-based engine; exact tuning is implementation-specific.

   Access reviewed · 21 October 2026

Pools, recovery & ticket correctness

Prerequisites: 10.3 · Races, anomalies & isolation 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

The booking service becomes busy. Requests wait for a database connection, a seat claim must agree with its reservation record, and the backup job reports success. Each situation needs evidence about a different part of the operation.

A pool limit tells us how many connections can be borrowed, but not how quickly they return. A save call tells us little about a shared transaction boundary. A completed backup job does not show that the application can use the restored data.

In this chapter, we will calculate a simple pool bound, trace a multi-statement reservation transaction, and design a meaningful restore check. The conditional seat-claim example will show how an application distinguishes a successful local change from a claim that did not occur.

10.4.1 · Database connection pooling

A connection pool keeps a bounded set of database connections available for reuse. A request borrows one, performs its database work, and releases it. When all connections are held, later requests must wait, time out, or be rejected according to the pool's policy.

Assume ten connections, each held for an average of 100 ms, or 0.1 seconds, per operation. In a simplified fully busy model, each connection completes 1/0.1 = 10 operations per second. Ten connections then give 10/0.1 = 100 operations per second before other constraints and variability. This is a modelled service bound, not a measured database capacity. Time waiting to borrow a connection adds to the caller's latency.

Holding a connection while waiting for an unrelated HTTP service occupies capacity that another request could use. Shorten that interval where the transaction permits it. Measure acquisition time, connection hold

time, lock waits, and transactions left open without work. Increasing the pool size may add database contention instead of throughput. Include pool acquisition in the request deadline so queued work cannot wait indefinitely before its query even starts.

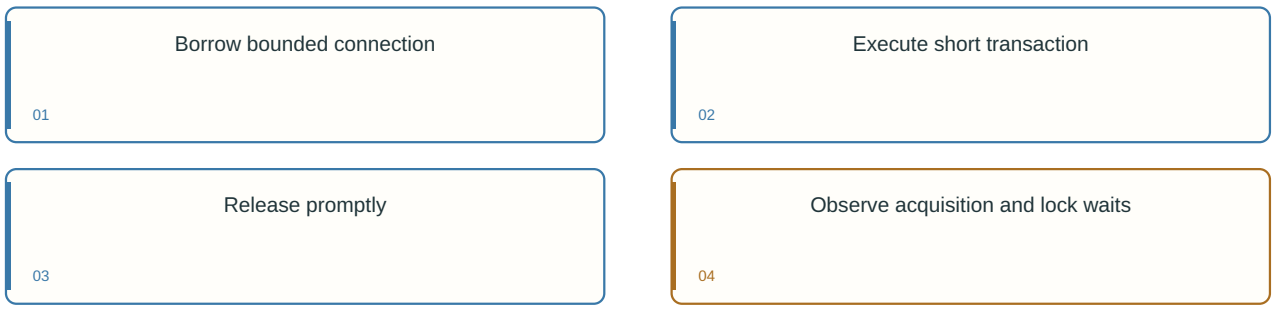


FIG. 10.4.1 **Database connection pooling · the example**

Database pools reuse bounded connections, with acquisition, hold time, and downstream contention included in deadlines.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.4.2 · Transactions across multiple operations

Reserving a seat and inserting its local reservation record must agree. If the seat change commits but the record fails, the stored data no longer describes one valid outcome. Put both statements inside the same actual database transaction.

Begin the transaction, conditionally claim the available seat or inventory, and check that the change succeeded. Then insert the matching reservation and commit. If insertion fails, roll back the seat change too. If the claim did not occur, the application must not insert a successful reservation. The shared commit boundary makes both local changes appear together or neither appear.

Check how the database library manages that boundary. An object-relational mapper, or ORM, maps application objects to database records, but calling save on two objects does not establish that both calls share one transaction. They must use the same transaction owner and commit scope. Also release resources deliberately: a lazy result that reads records later may still need its connection. Across independent stores or remote services, local rollback cannot provide the same common boundary; choose an explicit coordination and recovery protocol.

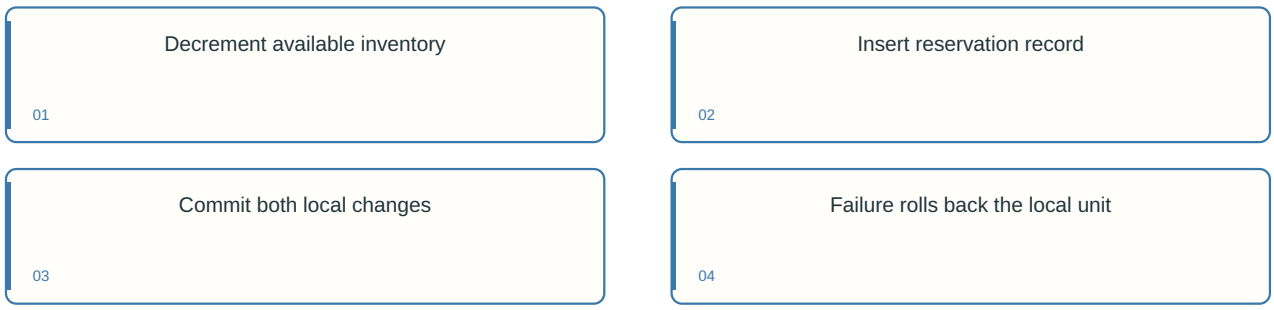


FIG. 10.4.2 **Transactions across multiple operations · the example**

Related statements need an explicit common transaction when their invariant requires one all-or-nothing change.

Author-created illustration. Numbers are illustrative unless stated otherwise.

10.4.3 · Backup, recovery, and durability

A backup job reports success. We still need to know whether its data can be decrypted, restored, and used by the application. A recovery procedure must include the compatible schema, keys, credentials, and any log history required to reach the desired state.

Restore into a clean environment and open the recovered database through the application. For a booking service, find a committed reservation and check that its seat ownership agrees. Measure the full time until the service is usable, and identify how much recent history is missing. These checks give evidence about recovery time and recoverable data. A file in backup storage alone does not answer either question.

Replication maintains live copies and may copy an accidental deletion to them. Backups retain recoverable history according to their retention policy. Point-in-time recovery can start from a suitable backup and replay the required logs to a selected point. State which acknowledged writes the procedure can recover after each failure, and keep suitable independent copies. Test that procedure, including the application checks, before relying on its recovery promise.

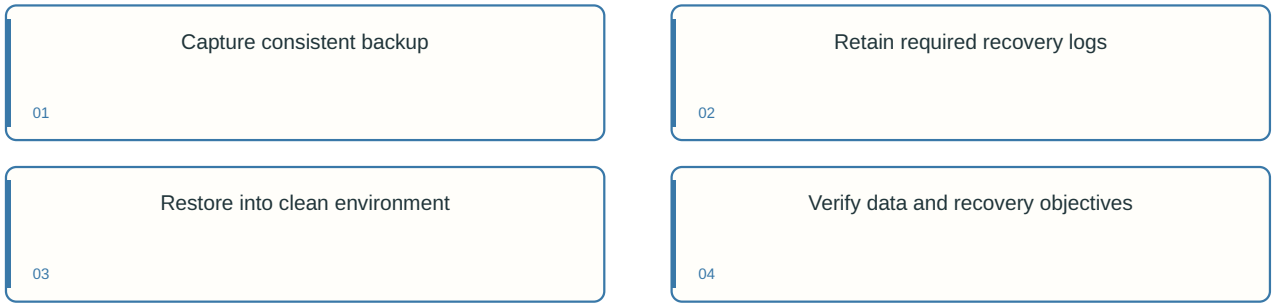


FIG. 10.4.3 **Backup, recovery, and durability · the example**

Recovery needs tested backups, logs, keys, compatible applications, and an explicit acknowledged-data policy.

Author-created illustration. Numbers are illustrative unless stated otherwise.

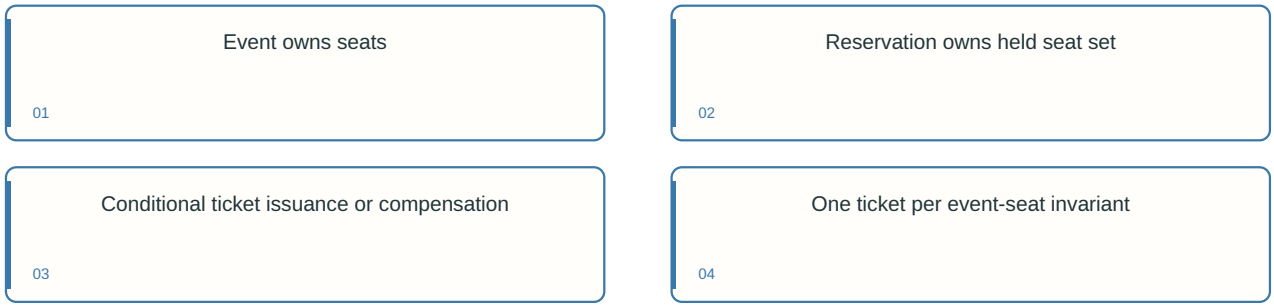


FIG. 10.4.4 **State and ownership in the related design**

Seat, hold, payment, and ticket records must preserve their defined ownership relationships.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · CONDITIONAL SEAT CLAIM

```
from mechanisms import open_demo_store, claim_seat
db = open_demo_store()
db.execute("INSERT INTO seats VALUES (1,NULL)")
db.commit()
assert claim_seat(db, 1, "A")
assert not claim_seat(db, 1, "B")
db.close()
```

The store begins with seat 1 available: its owner is NULL. A's call executes an UPDATE guarded by owner IS NULL. It changes one row and commits, so claim_seat returns true. B's later call checks the same condition, now finds the seat owned by A, and changes zero rows. Its false result tells the application that B did not acquire the seat.

This SQLite example checks availability directly; it does not use the version-seven column from the earlier optimistic-control example. Both place the check and change in one database operation. The executable mechanism collection also checks claims through two connections to the same file-backed store. These calls demonstrate the local ownership result; they are not a test of every overlapping schedule.

The protected rule is that only an available seat can be claimed. A complete booking also needs deliberate rules for its reservation record, payment, hold expiry, and stale callbacks. This local transaction cannot atomically perform an external charge.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · TRY CHANGING THE INPUTS

Let two callers claim the last seat

1. A reads available.
2. B reads available.
3. A writes owner A and reports success.
4. B overwrites owner B and reports success.

Two callers believe they won. Reading the final row would show B, but would miss the earlier success reported to A.

Compare the two policies using the same order of steps. Does either caller learn that the seat has already been taken? This display replays a fixed example. The downloadable SQLite example separately tests the atomic conditional update.

▼ Think further · What changes when one local transaction is divided across independent databases?

The two changes no longer automatically share one commit decision. Suppose the seat update commits in one database but inserting the reservation fails in another. Each store can behave correctly locally while the combined booking is incomplete.

First identify which invariant needs agreement. A distributed transaction may supply a suitable shared decision if the stores and operating requirements support it. Alternatively, a workflow can represent intermediate states explicitly and use retries, reconciliation, or compensation to reach an acceptable outcome. That choice must define what the caller sees while work is incomplete. Separate local commits alone do not preserve the former all-or-nothing business promise.

Remember: Keep connection ownership deliberate, give related local writes one commit boundary, and verify recovery by restoring and using the application.

Try applying it: A restored reservation row exists. What related data and application operation would you check before declaring the booking service recovered?

Unit 10 exercises → · Unit 10 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   `Access reviewed · 21 October 2026`

2. PostgreSQL 18: Index Types ↗

   Official versioned documentation. Supported access structures and their purposes.

   `Access reviewed · 21 October 2026`

3. PostgreSQL 18: Write-Ahead Logging ↗

   Official versioned documentation. Logging and recovery ordering; durability remains configuration-dependent.

   `Access reviewed · 21 October 2026`

4. RocksDB Overview ↗

   Official implementation documentation · accessed 21 Oct 2026. An LSM-based engine; exact tuning is implementation-specific.

   `Access reviewed · 21 October 2026`

PART 3 / UNIT 11

Caching and Content Delivery

A product is read repeatedly between edits. Work out which cost a cache can avoid, how much active data fits, and which layer should hold the reusable representation.

The running example

A popular product page receives enormous traffic. Introduce caching gradually and examine stale prices, thundering herds, inventory correctness, and failure behavior.

Unit 11 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

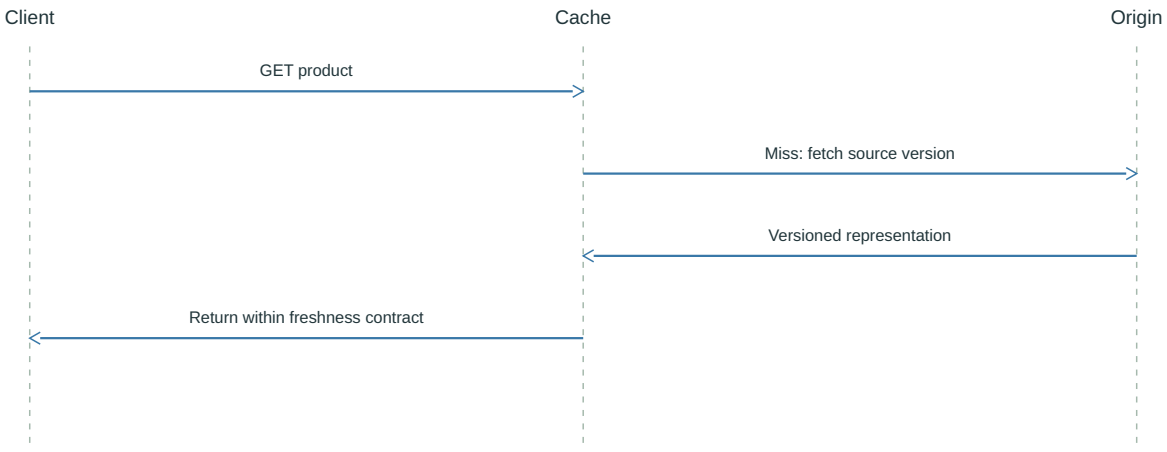
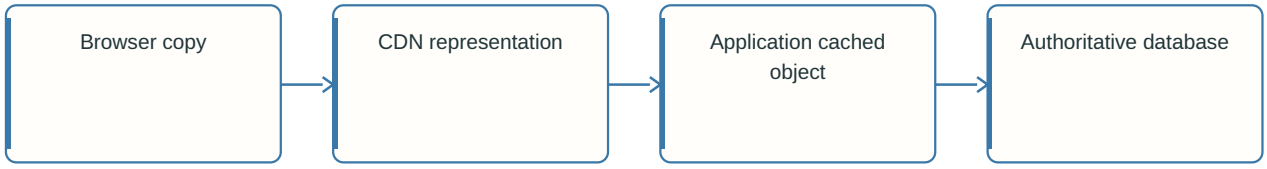


FIG. 11.0.1 **Cache hit/miss sequence**

On a hit, the origin exchange is omitted. Cache-aside population still needs race reasoning.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 11.0.2 **cache hierarchy**

These are distinct copies with separate freshness, identity, and invalidation policies.

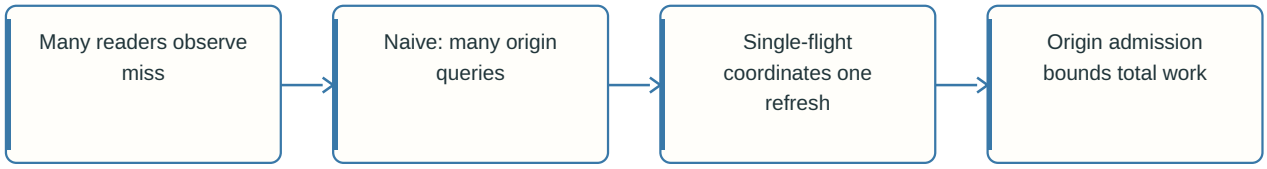
Author-created illustration. Numbers are illustrative unless stated otherwise.

Policy	After A,B,C,A	Insert D: evicted
Ideal LRU	Order B,C,A	B
FIFO	Insertion A,B,C	A

FIG. 11.0.3 **LRU and FIFO disagree after the same accesses**

Capacity is three. Both policies saw the same requests; policy metadata changes the decision.

Author-created illustration. Numbers are illustrative unless stated otherwise.

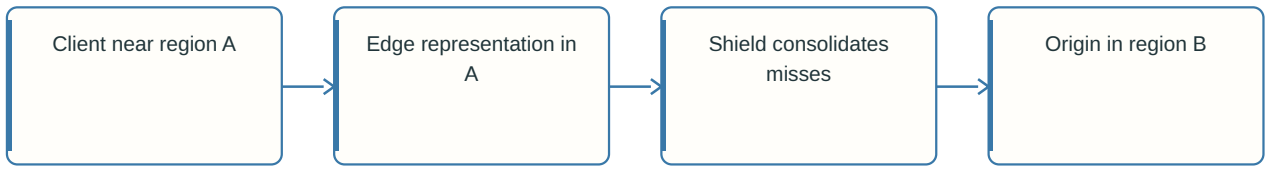


Arrows: ordered stages of this worked mechanism

FIG. 11.0.4 **stampede timeline**

A cache outage or synchronized expiry must not create an unbounded origin reconstruction storm.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 11.0.5 **geographic CDN delivery**

Geographic delivery reduces some distance and origin work while adding copies and freshness obligations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Locality and cache placement

Prerequisites: Networking Foundations for System Design, Data Models and Storage Technologies 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A product is read repeatedly between edits. Work out which cost a cache can avoid, how much active data fits, and which layer should hold the reusable representation.

11.1.1 · Why caching works

A product description is read thousands of times between edits. Reusing a stored response can avoid repeating the database lookup or calculation. This reuse is why caching can help.

Assume a cache hit takes 1 ms, a miss takes 50 ms, and 90% of requests hit. The simple average service time is 0.9×1 + 0.1×50 = 5.9 ms, excluding queueing and other work. Compare this saving with memory, lookup, and maintenance costs. A result used only once may be a poor cache candidate. Hit ratio also does not establish tail latency: slow misses and queues can still dominate some users' waits.

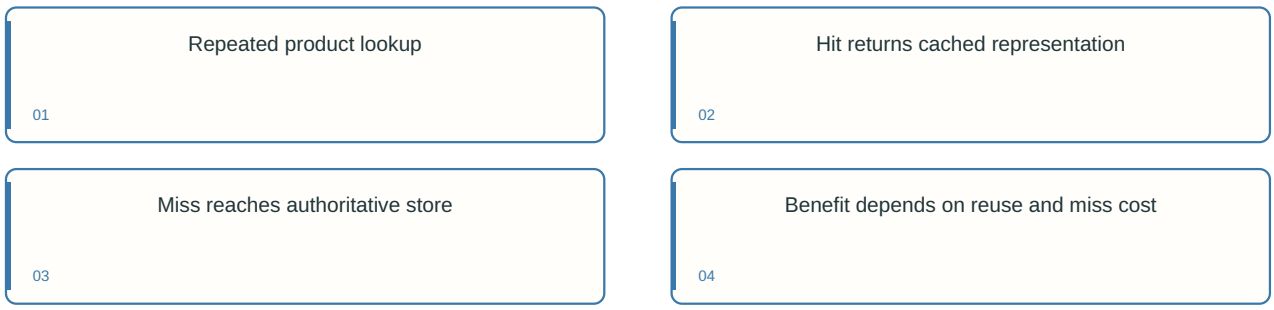


FIG. 11.1.1 **Why caching works · the example**

Caching saves repeated work when reuse outweighs access, storage, and maintenance costs under the required semantics.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.1.2 · Cache locality and the working set

A user repeatedly opens the same product: that is temporal locality. Reading neighbouring records together illustrates spatial locality. A working set is the data actively needed during a relevant interval.

A cache much smaller than that set may continually evict entries just before they are reused. A large scan can also displace useful entries under some policies. Measure bytes and object sizes as well as key counts, and distinguish frequently used data from the full historical collection. Preloading every object can fill memory with cold content. Admission rules decide what deserves a place before eviction decides what must leave.

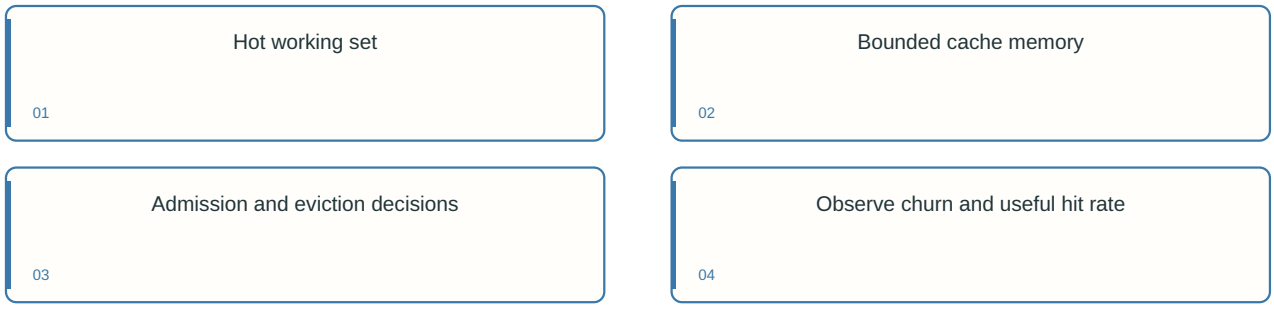


FIG. 11.1.2 **Cache locality and the working set · the example**

Locality and the active working set determine which data can usefully fit and remain in a cache.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.1.3 · In-process vs distributed caches

Two application instances each keep a price in memory. One has price 10 and another price 12. Their in-process caches provide fast access but create separate copies whose freshness needs coordination.

A distributed cache provides a shared network-accessible layer, adding encoding, connection, and failure costs. Choose by reuse patterns, duplicated memory, and required behaviour when a copy is stale or unavailable. Ordinary cached stock counts should not become the sole authority for exclusive allocation. Placing the cache elsewhere changes its access and failure boundaries; it does not by itself resolve the application's consistency rule.

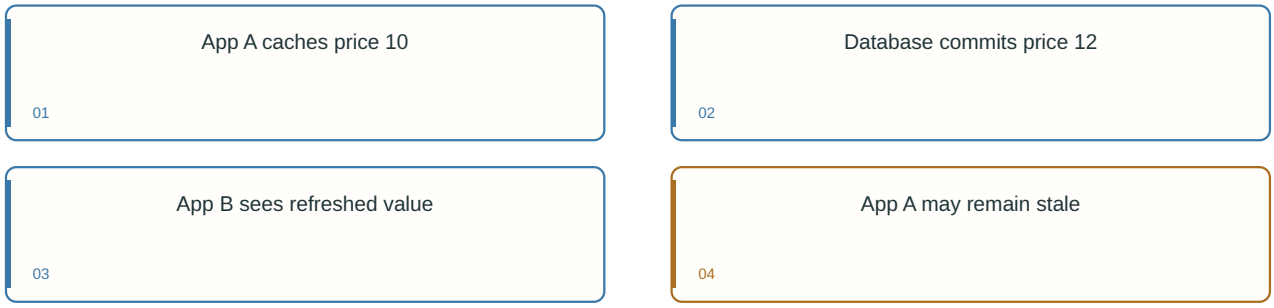


FIG. 11.1.3 **In-process vs distributed caches · the example**

In-process caches duplicate local copies; distributed caches add a shared network boundary and its failure behaviour.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.1.4 · Browser, CDN, application, and database caches

A page can be answered by a browser cache, a CDN, an application cache, or a database buffer pool. Each stores a different kind of copy. Database pages and a rendered account response also have different keys and update rules.

Draw the layers actually used and specify freshness at each. Invalidating an application object does not clear a browser's response. A public edge cache with a poorly scoped key can expose personal data. When diagnosing a stale result, identify which layer answered and which version it held. “The cache was cleared” is incomplete unless it names the particular copy involved.

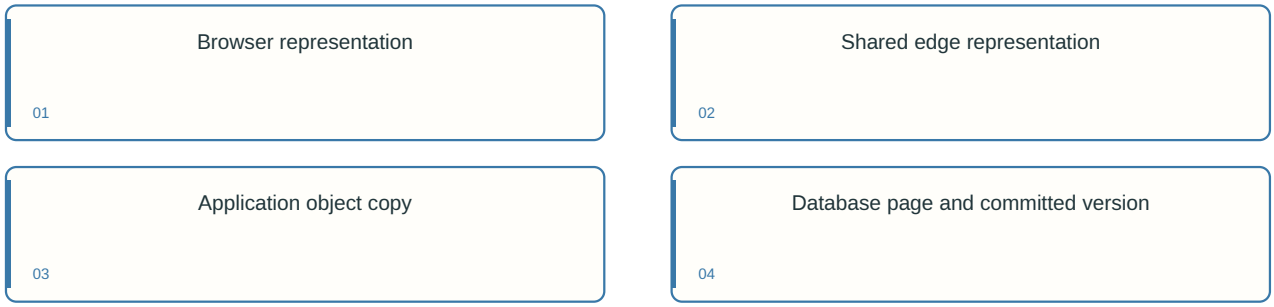


FIG. 11.1.4 **Browser, CDN, application, and database caches · the example**

Browser, edge, application, and database caches hold different copies with independent freshness and trust requirements.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · When should a product stop serving rather than return a stale cached result?

Stop when the operation requires current authority or the permitted stale interval has ended. An old description may be allowed, while allocating scarce stock or deciding current access may not be. State which actions degrade and which wait or reject, with visible status and a recovery path.

Remember: Cache value comes from useful reuse under a valid freshness and access policy.

Try applying it: Why can a high hit ratio coexist with slow or incorrect user responses?

Unit 11 exercises → · Unit 11 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   Access reviewed · 21 October 2026

2. Redis: Key eviction ↗

   Official documentation · accessed 21 Oct 2026. Implementation policies can approximate ideal LRU/LFU.

   Access reviewed · 21 October 2026

3. Content delivery networks — Cloudflare Learning Center ↗

   Official engineering explanation. CDN role; atlas delivery and traffic numbers are illustrative.

   Access reviewed · 21 October 2026

Copies, freshness & eviction

Prerequisites: 11.1 · Locality and cache placement 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A delayed reader inserts yesterday's price after a writer invalidates it. Follow cache population, expiry, versioning, and eviction as separate decisions.

11.2.1 · Cache-aside, read-through, write-through, write-behind

With cache-aside, application code loads from the source after a miss and populates the cache. Read-through places that loading behind the cache interface. Write-through includes backing-store work in the write path; write-behind delays it. Exact ordering varies by implementation.

Trace failures explicitly. A volatile write-behind cache can lose acknowledged changes before backing persistence. Cache-aside also has a race: a reader fetches version one, a writer commits version two and invalidates, then the delayed reader inserts version one. Where freshness requires stronger behaviour, use an appropriate version or coordination protocol. A pattern name alone does not resolve these event sequences.

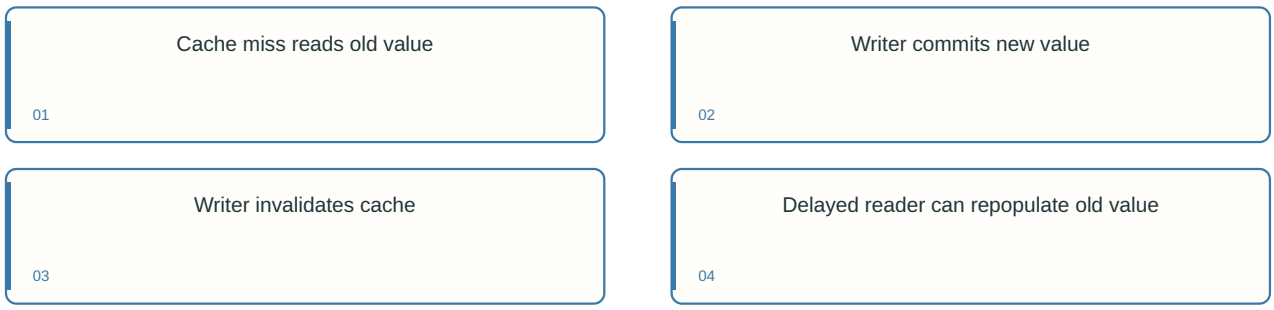


FIG. 11.2.1 **Cache-aside, read-through, write-through, write-behind · the example**

Cache population and write patterns specify ordering and failure gaps; their names alone do not remove races.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.2.2 · TTL and cache expiration

A cache receives a value that was already five minutes old and gives it a one-minute TTL. The copy may then remain six minutes behind its source. TTL describes residence after insertion, rather than necessarily the age of the fact.

Expiration also differs from eviction due to memory pressure. Choose lifetimes using tolerated staleness and refresh behaviour, and consider client-held copies. Adding jitter can spread expirations that would otherwise happen together. For a correctness-critical action, revalidate against the authority when required. A short display-cache TTL is useful only if it matches the promise made about that display.

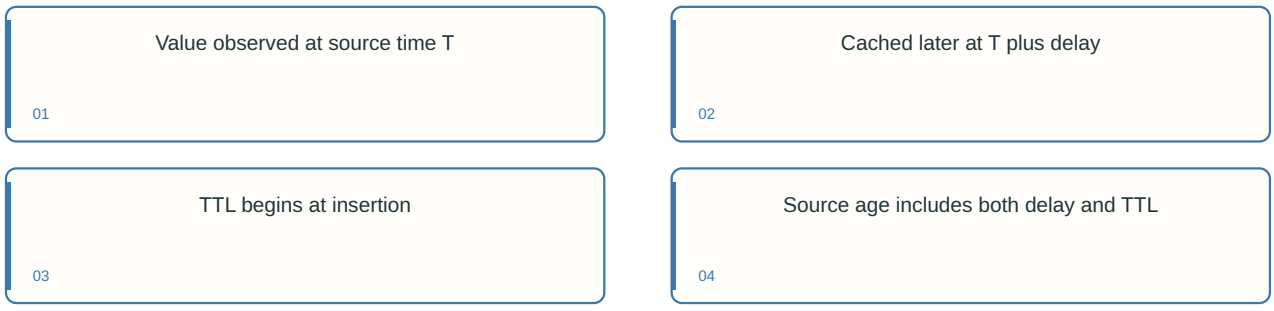


FIG. 11.2.2 **TTL and cache expiration · the example**

TTL limits intended residence after insertion, not necessarily the source age of the inserted fact.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.2.3 · Cache consistency and invalidation

A writer updates the source and deletes a cache entry. A delayed reader can still repopulate it from an earlier read, leaving an old value after the invalidation. Simply deleting after writing does not prevent every race.

Versioned entries, conditional population, or source-version checks can support a stronger protocol. Observe delayed or repeated invalidation events and their progress. Choose freshness per operation. Showing an older price may be acceptable if checkout verifies the actual price; allocating the last item from an old count can oversell it. Invalidation must serve the business rule, rather than merely keep copies approximately similar.

Location	Version and value	Authority
Database	v2: price 12	current authoritative value
App cache A	v1: price 10	stale representation
Checkout	revalidates database	does not trust display cache

FIG. 11.2.3 **A stale cache is a separate state copy**

Invalidation and propagation policy decide when copies agree. The critical checkout operation uses the required authoritative contract.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.2.4 · Eviction: LRU, LFU, FIFO, and alternatives

At capacity three, access A,B,C,A,D. Ideal LRU treats the second access to A as recent and evicts B. FIFO preserves insertion order and evicts A. LFU favours frequently used entries but also needs a tie-breaking rule.

Real cache policies may approximate these ideals. Test the actual policy with representative object sizes and request patterns. Eviction decides what leaves; admission decides what enters. Neither can make an arbitrarily large working set fit into memory. A policy that performs well on one tiny fixture may behave poorly during scans or changing popularity, so measure the workload it is meant to serve.

Policy	After A,B,C,A	Insert D: evicted
Ideal LRU	Order B,C,A	B
FIFO	Insertion A,B,C	A

FIG. 11.2.4 **LRU and FIFO disagree after the same accesses**

Capacity is three. Both policies saw the same requests; policy metadata changes the decision.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · When should a product stop serving rather than return a stale cached result?

Stop when the operation requires current authority or the permitted stale interval has ended. An old description may be allowed, while allocating scarce stock or deciding current access may not be. State which actions degrade and which wait or reject, with visible status and a recovery path.

Remember: Residence time and eviction policy do not by themselves establish current source data.

Try applying it: How can a one-minute TTL serve a value that is more than one minute behind its source?

Unit 11 exercises → · Unit 11 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   Access reviewed · 21 October 2026

2. Redis: Key eviction ↗

   Official documentation · accessed 21 Oct 2026. Implementation policies can approximate ideal LRU/LFU.

   Access reviewed · 21 October 2026

3. Content delivery networks — Cloudflare Learning Center ↗

   Official engineering explanation. CDN role; atlas delivery and traffic numbers are illustrative.

   Access reviewed · 21 October 2026

Hot keys, stampedes & negative results

Prerequisites: 11.2 · Copies, freshness & eviction 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A cache restarts just as a product becomes popular. Control duplicate loading, absent lookups, warming, and permitted stale results before the origin becomes overwhelmed.

11.3.1 · Hot keys

A single product becomes extremely popular. Even a large cache cluster can overload the one owner serving that key. Adding nodes mainly gives other keys more capacity unless the design also distributes this key's traffic.

For reads, local copies, replicas, or coalescing repeated work may help, with an explicit freshness policy. For writes, determine whether the value can be split or combined safely. An aggregate counter may allow distributed increments; exclusive stock allocation needs a stronger rule. Identify whether the pressure is transfer volume, recomputation, object size, or serialised updates before selecting a remedy.

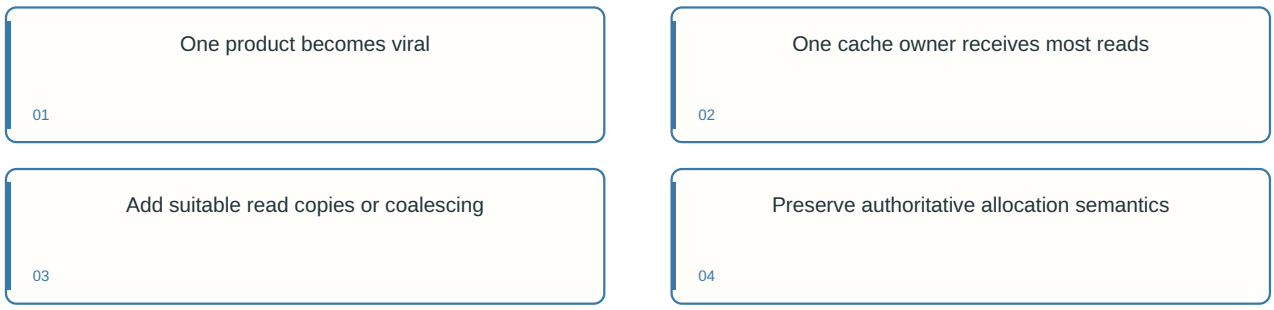


FIG. 11.3.1 **Hot keys · the example**

A hot key concentrates work on an owner and needs a remedy suited to its actual read or write pressure.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.3.2 · Cache stampedes, penetration, and avalanche

Many requests miss the same key and all recompute it: a cache stampede. Repeated requests for nonexistent keys are often called penetration. Broad expiry or cache loss can cause an avalanche of origin traffic.

Use controls suited to the cause. Single-flight loading combines concurrent work for one key. Negative caching can remember a genuine absence. TTL jitter spreads expiry, while bounded refresh and origin admission protect the source. A refresh lock needs its own failure and expiry behaviour. Test cache loss: allowing every application instance to rebuild everything at once can overload the database that remains responsible for the real data.

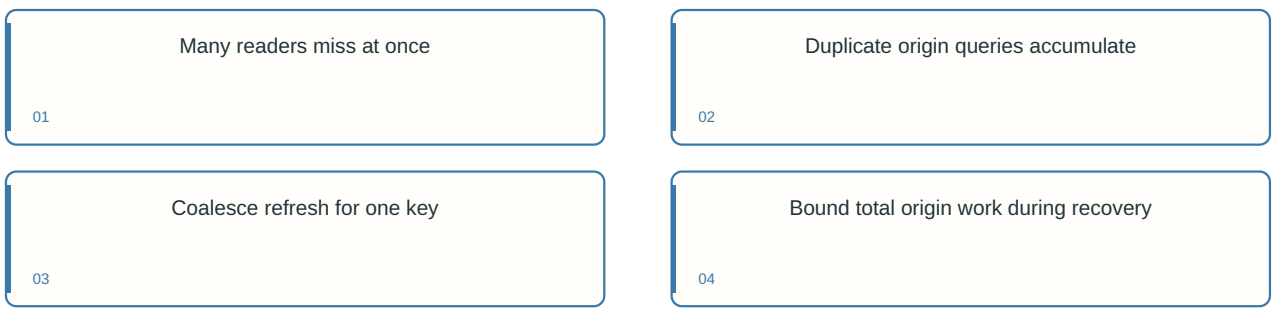


FIG. 11.3.2 **Cache stampedes, penetration, and avalanche · the example**

Stampedes, absent-key traffic, and broad expiry need controlled loading and origin protection.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.3.3 · Negative caching

A client repeatedly asks for a product that does not exist. A short-lived “not found” entry can prevent repeated source lookups. This is negative caching.

Distinguish absence from failure. A temporarily unreachable database has not established that the product is missing. Likewise, one user's denied access must not poison another user's cache entry. Scope keys correctly and use deliberate lifetimes. If the product is created just after its absence is cached, it may stay invisible until expiry or invalidation. The policy should state whether that delay is acceptable and how newly created records become visible.

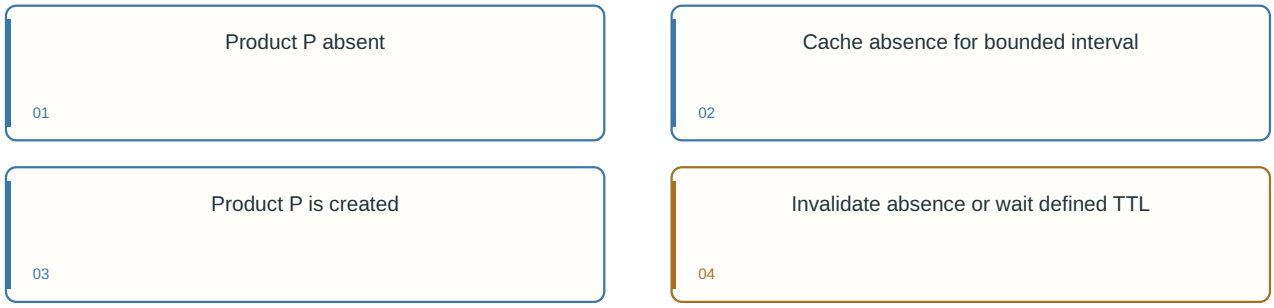


FIG. 11.3.3 **Negative caching · the example**

Negative caching stores a scoped absence or failure meaning for a deliberate lifetime.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.3.4 · Stale-while-revalidate

A cached product description expires while its source is slow. Stale-while-revalidate can return an allowed older copy and refresh it in the background. The user avoids the immediate wait, and combined refresh work can protect the origin.

Define separate ages for fresh data, permitted stale data, and unusable data. Repeated refresh failures should not silently extend stale serving forever. A description may tolerate this arrangement; an old payment reply cannot authorise a new charge. Make degradation visible where appropriate and preserve authoritative checks for operations whose correctness depends on current state. Faster responses have value only within the agreed freshness rule.

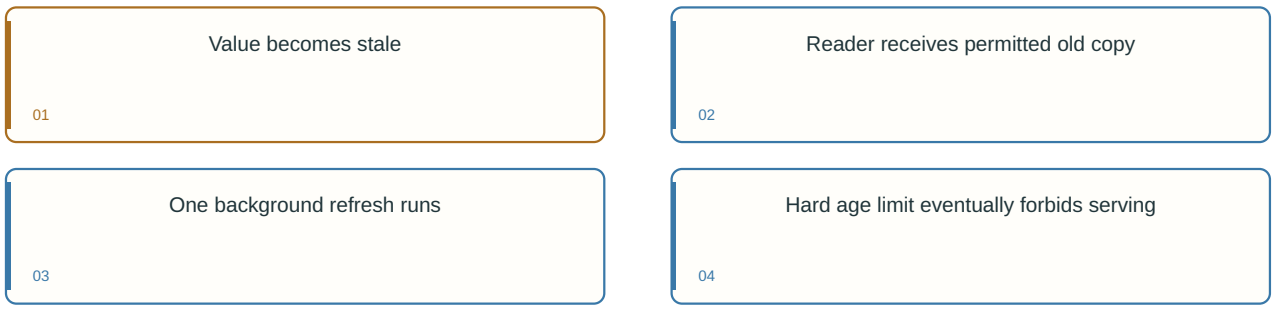


FIG. 11.3.4 **Stale-while-revalidate · the example**

Stale-while-revalidate serves an allowed older value while refresh proceeds, with a limit on unusable age.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.3.5 · Cache warming and prefetching

New application instances start with empty caches. Warming loads likely useful entries before normal demand; prefetching predicts what will be needed soon. Both spend bandwidth and memory before a user actually asks.

Load a measured working set gradually rather than scanning the entire database from every new instance. Check whether prefetched values are used and whether they evict more valuable entries. Staggering a rollout, sharing a cache, or limiting admission may solve a cold-start problem more simply. Stop obsolete speculative work. The useful question is how much future work was avoided, rather than how many entries were loaded.

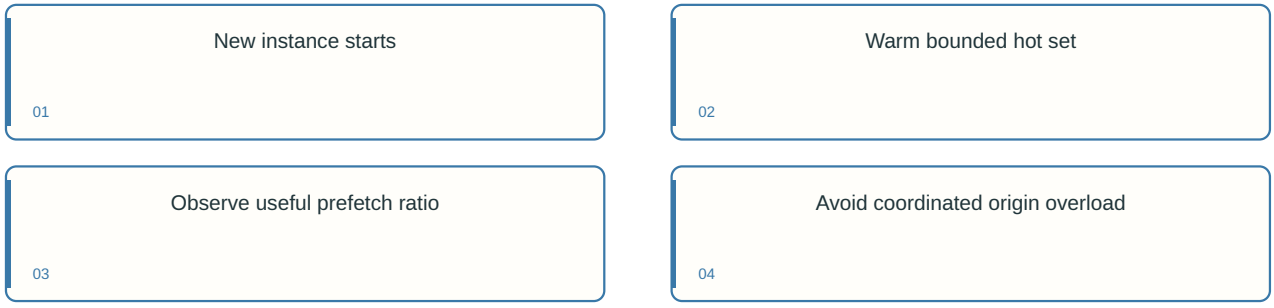


FIG. 11.3.5 **Cache warming and prefetching · the example**

Warming and prefetching spend speculative work that should be justified by actual future reuse.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · When should a product stop serving rather than return a stale cached result?

Stop when the operation requires current authority or the permitted stale interval has ended. An old description may be allowed, while allocating scarce stock or deciding current access may not be. State which actions degrade and which wait or reject, with visible status and a recovery path.

Remember: Bound refresh and fallback work while keeping the accepted stale and absent meanings explicit.

Try applying it: What should happen if background refresh keeps failing past the allowed stale age?

Unit 11 exercises → · Unit 11 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   `Access reviewed · 21 October 2026`

2. Redis: Key eviction ↗

   Official documentation · accessed 21 Oct 2026. Implementation policies can approximate ideal LRU/LFU.

   `Access reviewed · 21 October 2026`

3. Content delivery networks — Cloudflare Learning Center ↗

   Official engineering explanation. CDN role; atlas delivery and traffic numbers are illustrative.

   `Access reviewed · 21 October 2026`

Edges, partitions & the price of a cache

Prerequisites: 11.3 · Hot keys, stampedes & negative results 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A cache cluster and CDN add more copies and another failure path. Compare their operations with source correctness and recovery capacity, including whether the cache should exist at all.

11.4.1 · Redis-like data structures

A set can track identifiers, a sorted set can organise scores or timestamps, and a hash can group fields. Redis-like systems expose such structures with operations suited to particular tasks.

Choose by the operations needed, then examine their atomic scope. One atomic command does not make an arbitrary sequence of commands atomic. A script or transaction may protect the local sequence, subject to the selected system's rules and key placement. Expiry, persistence, and replica behaviour remain separate decisions. A convenient list or stream is not automatically a durable queue with the required acknowledgment and recovery semantics.

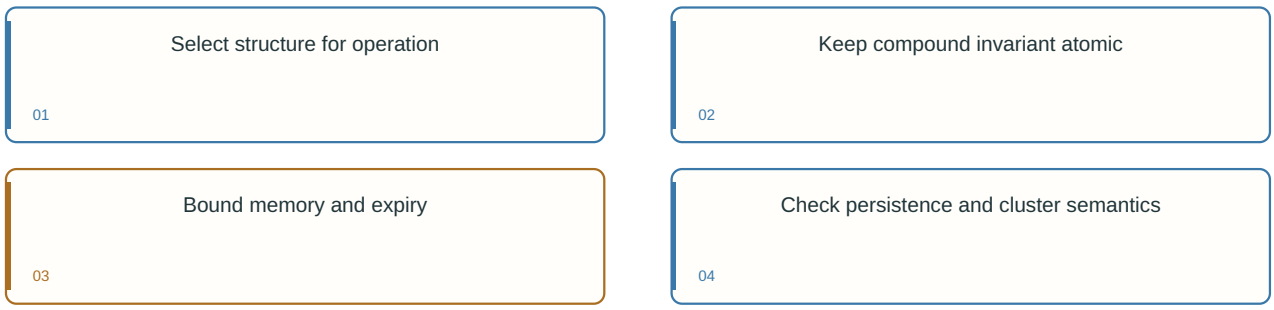


FIG. 11.4.1 **Redis-like data structures · the example**

Redis-like data structures offer specific operations; local command atomicity does not automatically cover a larger workflow.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.4.2 · CDN architecture and origin shielding

An origin shield collects edge misses before they reach the source, reducing duplicate origin work. A CDN places permitted representations near users. Both depend on correct keys and sharing rules.

Decide how query parameters, headers, identities, and content versions affect the representation. A signed URL is a capability with a scope and lifetime, not permission to ignore private-data policy. Purge propagation follows the provider contract and may not remove copies already kept by clients. Immutable versioned URLs can simplify public asset updates. Trace every remaining copy when immediate revocation matters; proximity and bandwidth savings do not establish access correctness.

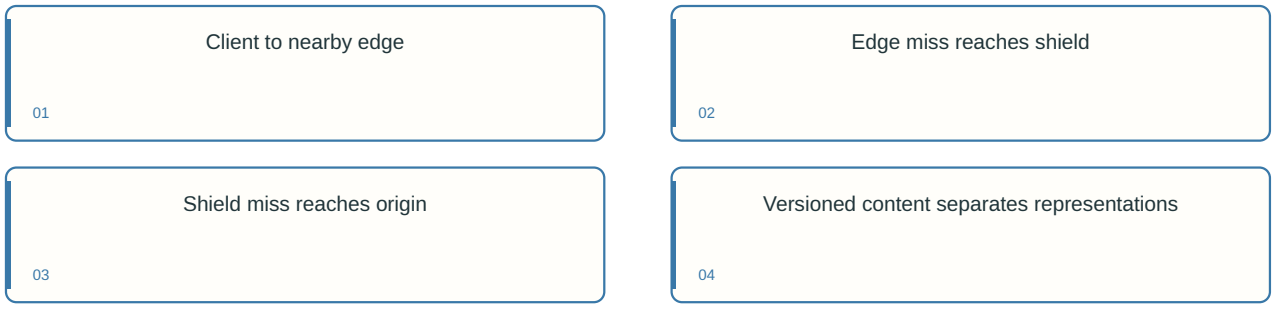


FIG. 11.4.2 **CDN architecture and origin shielding · the example**

CDN edges and shields distribute suitable representations under explicit key, capability, freshness, and purge rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.4.3 · Cache partitioning and replication

Partitioning assigns different keys to different owners. Replication makes additional copies under a protocol. A distributed cache may use both, but failover can still lose, evict, or expose delayed entries.

Specify whether the application falls back, serves permitted stale data, or rejects when the cache fails. Stable hashing can reduce key remapping during membership changes, while leaving hot-key traffic uneven. Keep source and cache recovery capacity explicit. The real peak may arrive when a cache cluster restarts and every missing value needs reconstruction. Test that scenario rather than sizing the origin only for a warm-cache hit ratio.

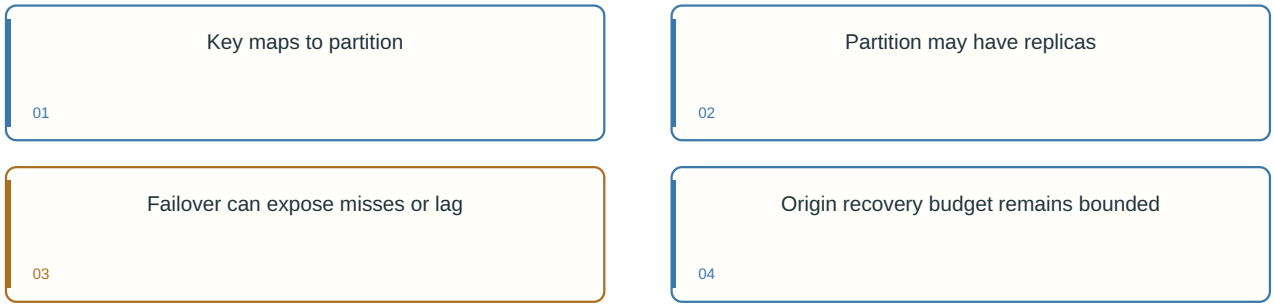


FIG. 11.4.3 **Cache partitioning and replication · the example**

Cache partitions and replicas need failure, remapping, fallback, and rebuild-capacity policies.

Author-created illustration. Numbers are illustrative unless stated otherwise.

11.4.4 · When caching is harmful

A cache adds a lookup, encoding, memory use, and copy-maintenance work. If results are rarely reused, those costs can exceed the work saved. If its key omits authorisation scope, the cache can also serve the wrong user's result.

Test with the cache empty and unavailable. A warm cache can hide insufficient source capacity until the day it disappears. Sometimes a better query or index removes the bottleneck with fewer freshness and recovery obligations. Keep caching where measured reuse and the agreed correctness policy justify it. Removing an unhelpful cache can simplify both performance and reasoning.

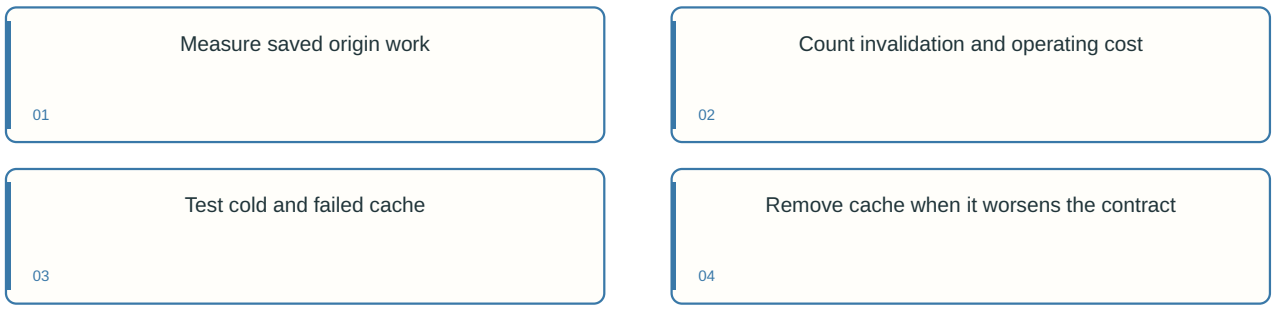


FIG. 11.4.4 **When caching is harmful · the example**

Caching can be harmful when it saves little, weakens scope, hides origin limits, or adds excessive lifecycle work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · IDEAL LRU

```
from mechanisms import LRU
cache = LRU(3)
for key in "ABC": cache.put(key, key)
cache.get("A")
assert cache.put("D", "D") == ("B", "B")
```

After A,B,C,A, ideal LRU considers B the least recently used. Adding D evicts B, as the assertion checks.

The fixture models entry count and exact recency. Actual caches can approximate the policy and use byte or expiry limits. Eviction alone says nothing about whether a loaded value is current.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · TRY CHANGING THE INPUTS

Follow which cache entry is used least recently

1. A: miss · oldest → newest: A
2. B: miss · oldest → newest: A, B
3. C: miss · oldest → newest: A, B, C
4. A: hit · oldest → newest: B, C, A
5. D: miss, evict B · oldest → newest: C, A, D

The cache holds a fixed number of entries. Each access loads a missing key or moves an existing key to the most recently used end. Real caches may use approximate eviction, byte limits, and expiry. Choosing an entry to evict does not tell us whether its value is fresh.

▼ Think further · When should a product stop serving rather than return a stale cached result?

Stop when the operation requires current authority or the permitted stale interval has ended. An old description may be allowed, while allocating scarce stock or deciding current access may not be. State which actions degrade and which wait or reject, with visible status and a recovery path.

Remember: Evaluate each cache by saved work, maintained semantics, and the cost of losing it.

Try applying it: Which workload would show that removing a cache improves the whole system?

Unit 11 exercises → · Unit 11 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9111: HTTP Caching, 2022 ↗
   Original standard. HTTP cache semantics; application cache races are separate examples.
   Access reviewed · 21 October 2026

2. Redis: Key eviction ↗

   Official documentation · accessed 21 Oct 2026. Implementation policies can approximate ideal LRU/LFU.

   Access reviewed · 21 October 2026

3. Content delivery networks — Cloudflare Learning Center ↗

   Official engineering explanation. CDN role; atlas delivery and traffic numbers are illustrative.

   Access reviewed · 21 October 2026

PART 3 / UNIT 12

Scalability, Load Balancing, and Data Distribution

Our URL shortener begins at 100 requests per second. We want to understand how it could grow towards 100,000 requests per second, adding components only when a demonstrated limit calls for them. The first tempting change is to run more application instances. But what happens when the next request reaches a different instance, or when all the new instances contact the same database?

The running example

Scale a URL shortener from 100 requests per second to 100,000. Introduce new components only when justified by a demonstrated bottleneck.

Unit 12 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

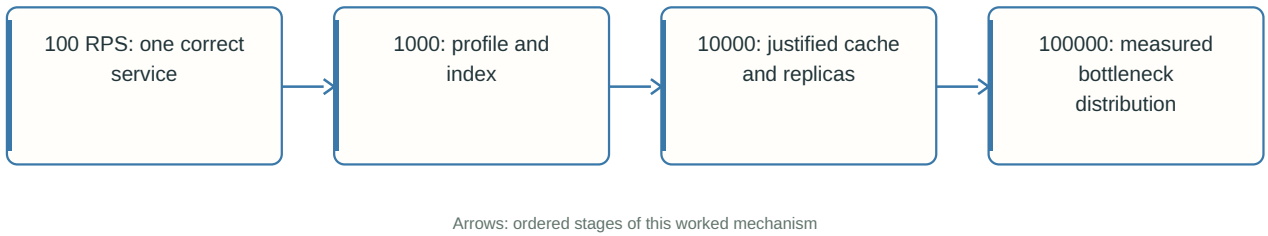


FIG. 12.0.1 **Architecture evolution across four load levels**

These are conceptual teaching checkpoints, not hardware benchmark thresholds.

Author-created illustration. Numbers are illustrative unless stated otherwise.

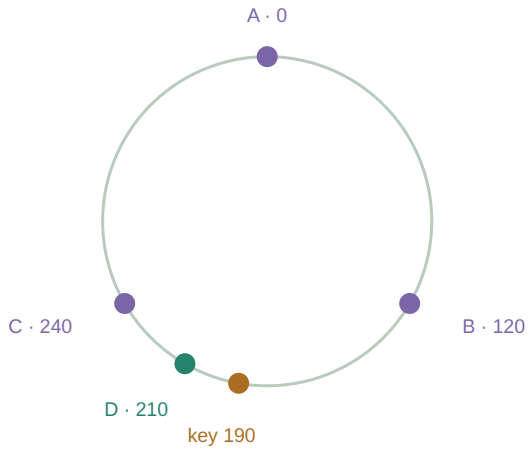


FIG. 12.0.2 **A new owner takes only part of the hash space**

With clockwise-successor ownership, key 190 belongs to C at 240. Adding D at 210 moves it to D; the affected interval is (120,210]. Hash coordinates are illustrative.

Author-created illustration. Numbers are illustrative unless stated otherwise.

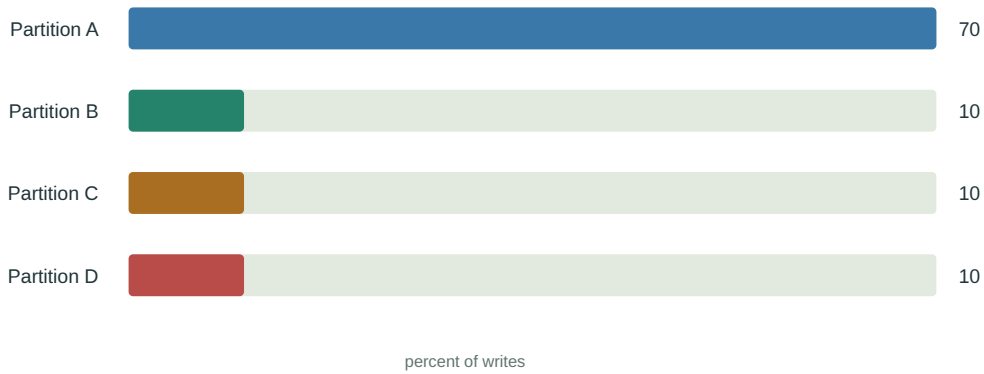
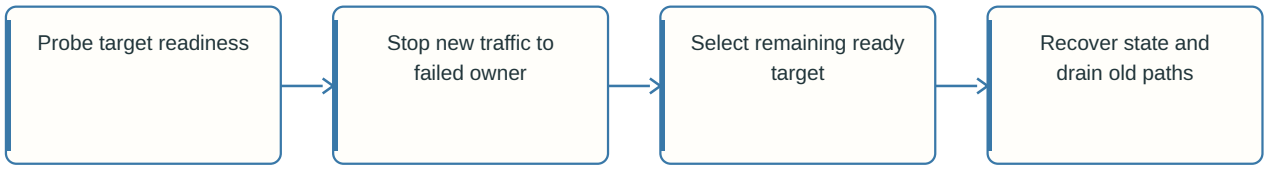


FIG. 12.0.3 **Averages hide a hot partition**

Seventy percent of writes go to A. Adding cold capacity does not split one hot ownership key.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 12.0.4 **load-balancer failover**

Routing failover does not by itself establish fresh state or preserve uncommitted work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Scale compute & route requests

Prerequisites: Requirements, Estimation, and Engineering Trade-offs, Networking Foundations for System Design, Data Models and Storage Technologies, Database Internals, Transactions, and Concurrency Control, Caching and Content Delivery

6 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Our URL shortener begins at 100 requests per second. We want to understand how it could grow towards 100,000 requests per second, adding components only when a demonstrated limit calls for them. The first tempting change is to run more application instances. But what happens when the next request reaches a different instance, or when all the new instances contact the same database?

We will begin with the work performed by one request and decide which parts can be shared across machines. That requires knowing where sessions and accepted work live, how a destination is selected, and what makes an instance ready to receive traffic. We will then examine the delay between detecting demand and making new capacity available.

By the end of the chapter, you should be able to explain when more compute helps, which conditions make an application instance replaceable, and which measurements can guide growth without overwhelming a shared dependency.

12.1.1 · Vertical vs horizontal scaling

Suppose our URL shortener receives 100 requests per second. Over time, the target rises to 100,000 requests per second. These numbers describe the work we want to support; they do not yet tell us how many machines to buy. We first need to find which resource limits the existing service while it meets its response-time and correctness requirements.

One option is to give the current machine more CPU, memory, or other resources. This is vertical scaling. Another is to add machines and distribute suitable work among them. This is horizontal scaling. A machine or independently managed server in such an arrangement is often called a node.

Consider a web application that spends most of its time performing independent calculations. More CPU on one node may let it perform more calculations at once. Several application nodes may also share that work, provided requests can reach them and each can obtain the state it needs. By contrast, if every request waits for the same database lock, adding application nodes mainly adds more callers waiting for that lock. A larger machine also cannot remove a rule that requires those updates to happen one at a time.

Choose the change that addresses the measured limit, then measure completed useful work again. Include startup time, cost, and failure consequences in the comparison. A larger database machine can postpone the need to distribute data, while several application nodes can make replacement easier. Neither arrangement promises a proportional increase in throughput. After one limit is relieved, another resource may become the next constraint.

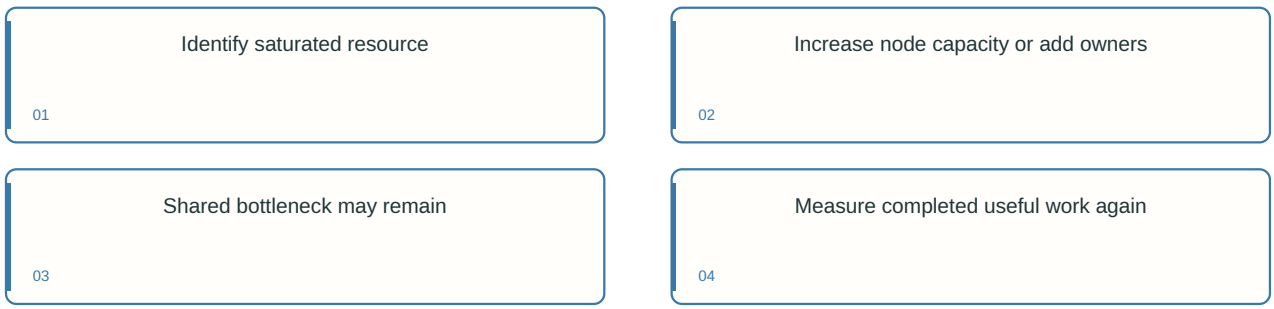


FIG. 12.1.1 **Vertical vs horizontal scaling · the example**

Vertical scaling enlarges a node; horizontal scaling adds nodes, with gains limited by the work that can divide.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.1.2 · Stateless vs stateful services

A user signs in through application instance A. The next request reaches instance B. To serve it correctly, B must establish the same identity, check the current permissions, and find the required application data. If the only record of the user's session exists in A's memory, B cannot simply continue where A stopped.

A stateless service avoids keeping required per-client conversation state only in a particular application instance between requests. For example, a request can carry a session identifier, and either instance can use that identifier to retrieve the session from its designated store. The application still uses state. The useful property is that serving the next request does not require returning to the same application process.

A stateful service keeps some continuing context that affects how later work must be handled. Local sessions are one example. A worker that has accepted a job into memory may also hold work that a replacement cannot reconstruct. Moving the session to a shared store does not, by itself, preserve that accepted job. Each kind of required state needs an owner and a recovery procedure.

Before freely distributing requests, check that every eligible instance has compatible configuration, can access the required state, and follows the same contracts. Also preserve the handling of repeated operations: if a reply disappears, a retry reaching B must not accidentally repeat an effect already performed by A. The word stateless describes a useful arrangement of state; the replacement behaviour still has to be established.

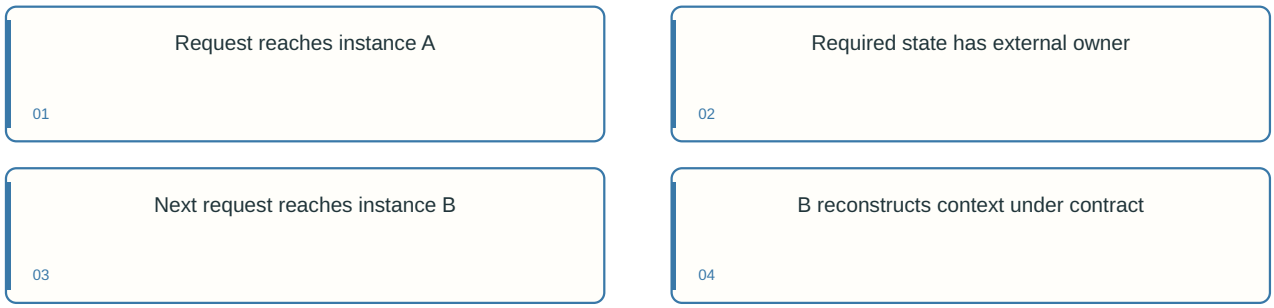


FIG. 12.1.2 **Stateless vs stateful services · the example**

Stateless instances keep no required per-client conversation only locally, while using appropriate external state owners.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.1.3 · Load distribution algorithms

With several application instances available, someone must select the destination for each new unit of traffic. A load balancer performs this selection according to a policy. The policy should distribute work in a way that makes useful capacity available, rather than merely producing an even count of selections.

Round-robin selects eligible backends in turn: A, then B, then C, then A again. It is simple and can work well when instances and requests have similar costs. Weighted selection gives a larger share to an instance expected to handle more work. For example, a higher-capacity instance can receive a greater proportion of new traffic. The weights need to reflect usable capacity under the actual workload.

Least-connections uses the number of active connections as a clue to current load. This can help when some connections last much longer than others. It is less informative when connections carry very different amounts of work. Similarly, even request counts can hide a few expensive requests that keep one instance

busy. Hash-based selection uses a key, such as a session identifier, to keep related traffic together; this can improve locality but can also concentrate a heavy user's traffic.

The selection can happen in an intermediary, which is server-side balancing, or in a caller that knows the available destinations, which is client-side balancing. In either case, decide who discovers backends, excludes unhealthy targets, and performs retries. Observe per-instance queues and saturation to check whether the policy is working. Redistributing requests cannot increase the capacity of a database that all those instances already share.

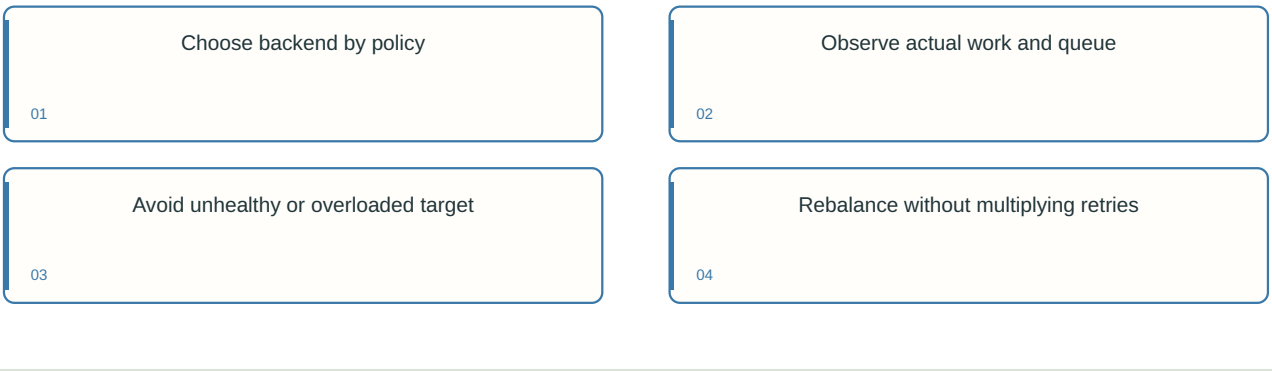


FIG. 12.1.3 **Load distribution algorithms · the example**

Balancing algorithms distribute selections using different load clues whose usefulness depends on request and connection behaviour.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.1.4 · Health checks and failover

An application process answers a ping, yet every real request fails because it cannot initialise the state needed to serve users. A different process has lost an optional recommendation service but can still display the requested page. Both cases show why a health check must answer a particular operating question.

A liveness check asks whether the process should be restarted. A readiness check asks whether the instance should receive traffic under the service's chosen policy. These decisions need different evidence. A process that is still starting may be alive but not ready. An instance that can give an acceptable reduced response may remain ready even when an optional dependency is unavailable.

Suppose every instance checks the same optional dependency and declares itself unready whenever that dependency fails. One shared outage now removes the whole application from routing, including responses it could still provide. Choose checks that reflect the required response, give the checks bounded waiting times, and decide in advance which reduced behaviour is acceptable.

Failover directs work to another eligible instance after the current one is removed or lost. A health check helps decide when to change routing, but the replacement must also have access to valid state and enough capacity. New traffic can be redirected while existing connections or in-flight requests still need handling. Work held only in the failed process's memory does not reappear because a routing table changed. Recovering that work requires its own state and retry rules.

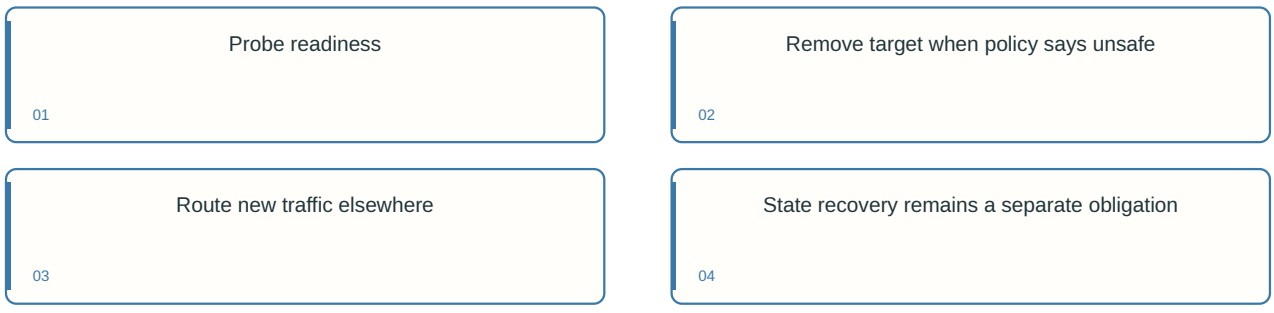


FIG. 12.1.4 **Health checks and failover · the example**

Readiness, liveness, and failover make different decisions and require state and capacity beyond a probe response.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.1.5 · Sticky sessions

Suppose a load balancer selects instance A for a user's first request. Instead of selecting freely on later requests, it continues sending that user's traffic to A. This is a sticky session, also called session affinity. The routing arrangement may use a session identifier or another agreed way to recognise related traffic.

Affinity can help when A holds local session state. It can also be an optimisation when A has already cached data likely to be needed by that user. These uses have different consequences. Losing a disposable cache entry may only make the next request slower; losing the only session record may force the user to sign in again or interrupt the operation.

Now let A fail or leave service during maintenance. Sending the user to B changes the destination, but B still needs a way to recover the required context. Decide whether the session is recoverable elsewhere, whether the client must reconnect, and what happens to in-flight work. Keeping the user attached to A during normal operation does not answer these failure questions.

Sticky routing also limits balancing. A few heavy users assigned to A can overload it while B remains lightly used. Externally owned sessions can remove this requirement for replaceable web instances. Where affinity remains useful, state whether it protects required state or merely improves reuse, and define how assignment changes when an instance fails or drains.

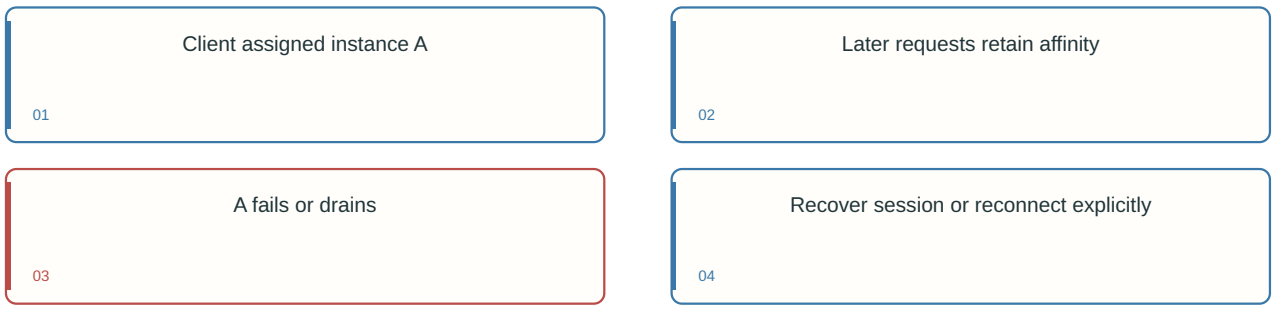


FIG. 12.1.5 **Sticky sessions · the example**

Sticky sessions preserve affinity while adding load-skew and recovery consequences for locally held state.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.1.6 · Autoscaling metrics

A worker queue's oldest unfinished job is getting older, although CPU use remains moderate. The workers may be waiting for storage rather than computing. A rule that adds workers only when CPU is high will miss the growing delay that users experience.

Autoscaling is a control process that changes the number or size of running instances in response to observed signals. Choose a signal that relates to the constrained work. CPU can be useful for computation. Queue age can expose delayed jobs. Connection use may reveal pressure in a workload that spends much of its time waiting. A request count is easier to interpret when requests require comparable work; a changing mix of cheap and expensive requests weakens that assumption.

The controller cannot create ready capacity immediately. It observes the signal, requests new instances, waits for them to start, and may then wait for caches or other resources to warm. Traffic arriving during this interval still reaches the old capacity. Keep headroom and admission limits for that interval, rather than assuming that an instruction to scale has already increased throughput.

Set minimum and maximum capacity and use stabilisation rules so short fluctuations do not repeatedly add and remove instances. The maximum should reflect cost and downstream limits as well as the application's own resources. Adding workers that all wait on one overloaded database can worsen the delay. After scaling, inspect completed jobs and their age to establish whether useful capacity actually increased.

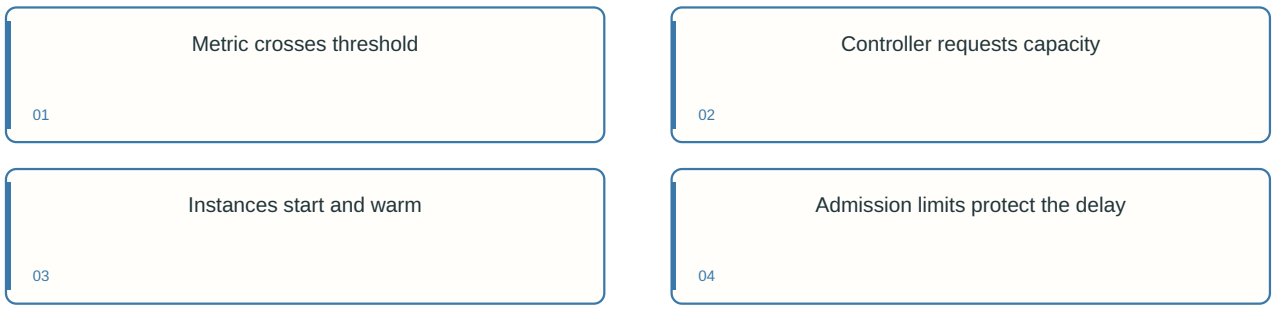


FIG. 12.1.6 **Autoscaling metrics · the example**

Autoscaling reacts to workload signals with delay, bounds, warmup, and downstream-capacity constraints.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Every caller competes for one globally hot resource that can have only one exclusive owner. Can adding many partitions make these ownership decisions independent? Explain what a valid improvement would have to preserve.

Extra partitions can distribute unrelated records, but they do not divide the decision about this one exclusive resource. Two partitions independently granting ownership could accept incompatible winners and break the rule. The hot decision still needs a protocol that establishes the permitted owner.

Read copies or batching may reduce suitable surrounding work, and a different allocation model may be possible if the requirements permit it. For any proposal, explain which work it removes or distributes and how it prevents conflicting grants. Measure the hot owner's actual work instead of inferring progress from the number of partitions.

Remember: Scale the measured limiting work. Freely replacing application instances requires accessible state, compatible behaviour, suitable routing, and bounded demand on shared dependencies.

Try applying it: A signed-in user sends one request to A and the next to B. Which state and behavioural conditions must hold for B to serve correctly, and what changes if A fails after performing a write but before replying?

Unit 12 exercises → · Unit 12 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Application best practices — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Scaling and application boundary review.

   Access reviewed · 21 October 2026

2. Dynamo: Amazon’s highly available key-value store, 2007 ↗

   Original research · author organization. Replication, partitioning, and reconciliation ideas; capstones are original designs, not Dynamo deployments.

   Access reviewed · 21 October 2026

Replicate and partition data

Prerequisites: 12.1 · Scale compute & route requests 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Several application instances can now serve requests, but suppose the database is approaching a measured storage or write limit. We could copy its data to another database, or divide the data between owners. These changes look similar on a diagram, yet they solve different problems and create different obligations.

In this chapter, we will distinguish replication from partitioning, then follow how a key selects its owner. We will compare ordered ranges with hash-based placement and examine what must happen when that placement changes. Moving data while users continue writing will let us see why calculating a destination is only one part of distribution.

The aim is to choose placement from the actual queries and invariants, and to explain how a handover preserves accepted work. We will also examine a partition receiving 70% of writes, because evenly distributed storage does not establish evenly distributed traffic.

12.2.1 · Replication

Suppose a database primary accepts a new record and sends the change to a second database. Keeping additional copies of data under a defined protocol is replication. The primary is the current write authority in this example; the other copy is a replica. What the caller can rely on depends on which events must happen before the primary reports success.

In asynchronous replication, the primary may reply before the replica receives the change. Consider the sequence: the primary commits, the caller receives success, and the primary becomes unavailable before the replica catches up. If that replica is made the new primary, the acknowledged record may be missing. The second copy exists, but its progress matters as much as its existence.

A synchronous arrangement waits for a specified replica milestone before replying. That milestone might concern receipt or durable storage, according to the selected system's protocol; receiving a record and applying it for reads are different steps. State which copies must reach which milestone. Waiting for remote acknowledgment adds delay and may prevent completion when the required communication is unavailable.

Replication can serve appropriate reads or help continue service after a failure. To assess those benefits, examine copy placement, lag, and the promotion procedure together. Two copies on one lost disk do little for a disk-loss requirement. Likewise, promoting a convenient but insufficiently current copy can violate the intended durability promise. The protocol and tolerated failures give meaning to the copy count.

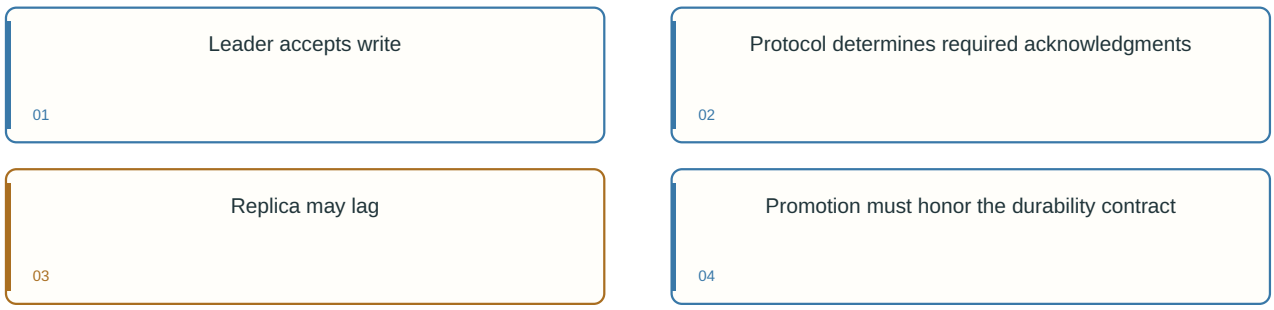


FIG. 12.2.1 **Replication · the example**

Replication maintains copies under explicit acknowledgment, authority, lag, and recovery rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.2.2 · Partitioning and sharding

Suppose one database owns every short-link record. We can divide the records into portions, each containing some of the keys. Such a portion is a partition. When different storage owners manage those portions, the arrangement is commonly called sharding. Unlike replication, which makes additional copies, sharding divides responsibility for different data.

A partition key determines the placement. If records are assigned by tenant identifier, a request for one tenant's links may reach just one owner. A query across all tenants may have to contact several owners and combine their results. The same key that simplifies one query can therefore make another query more expensive. Check the important reads and writes before selecting it.

Ownership also determines where rules can be enforced. A shard may enforce uniqueness among its own records, but that alone does not establish uniqueness across all shards. A change involving several owners may require coordination that a local transaction previously supplied. Joins, transactions, and globally unique values need explicit treatment when their scope crosses the new boundaries.

Keep a way to route requests to the current owner and a procedure for moving data when ownership changes. A large tenant can still dominate a tenant-based shard, and movement must preserve accepted updates. Introduce sharding when a measured storage or write limit justifies these obligations. A large user count, without a workload and resource diagnosis, does not identify either the need to shard or the correct key.

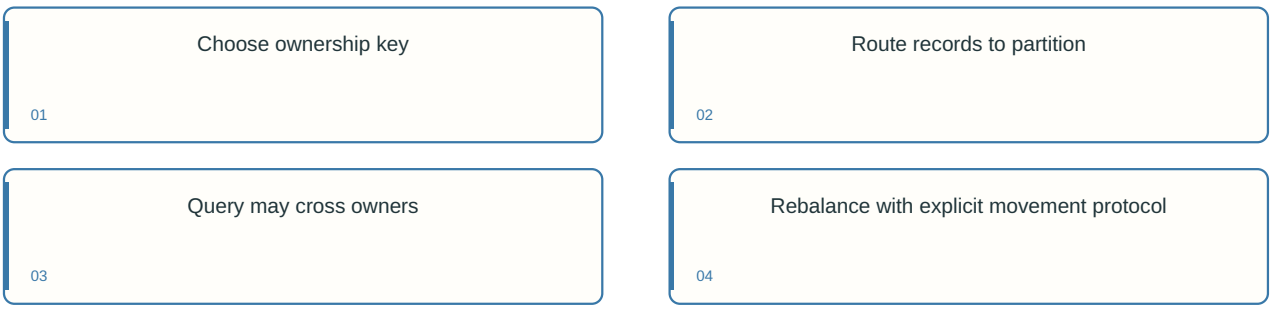


FIG. 12.2.2 **Partitioning and sharding · the example**

Sharding distributes partitions among owners, adding routing, query, transaction, and movement requirements.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.2.3 · Range partitioning vs hash partitioning

Suppose events have increasing time keys. Under range partitioning, each owner holds an interval of those keys. A query for a particular day's events can then visit the relevant intervals. Retiring an old interval can also fit time-based retention. The arrangement preserves useful order between the key and its location.

That same order can concentrate writes. If all new events belong to today's interval, most incoming traffic reaches its owner, even while owners of older intervals are idle. Dividing historical data evenly does not divide current ingestion evenly. Splitting the range helps only if the new traffic can actually fall into different resulting ranges.

Hash partitioning first transforms a key with a hash function and uses that result to choose a partition. Different keys can then spread across owners even when their original values are adjacent. The cost is reduced locality for ordered queries: retrieving one time interval may require visiting several partitions and combining their results. This extra set of requests is often called query fan-out.

A design can combine a time bucket with a spread dimension, assigning today's events to several owners. This distributes suitable writes but makes a read of today's data combine several portions. Neither range nor hash partitioning automatically divides requests for one extremely popular key. Compare write distribution, read locality, retention, and the measured skew together; evenly placed keys are only one part of the problem.

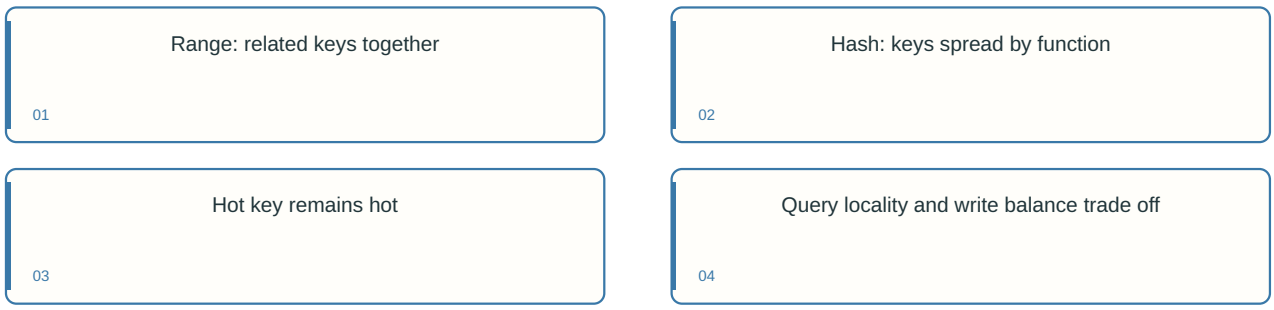


FIG. 12.2.3 **Range partitioning vs hash partitioning · the example**

Range placement supports ordered locality; hashing spreads keys, with different scan and hot-key trade-offs.

Author-created illustration. Numbers are illustrative unless stated otherwise.

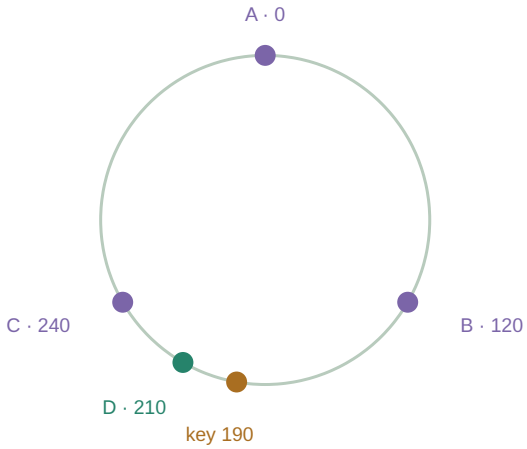
12.2.4 · Consistent hashing and rendezvous hashing

A simple placement rule hashes a key and takes the remainder after division by the number of owners. This is modulo routing. It is easy to calculate, but changing the owner count changes the divisor. Many keys can then map to a different owner, creating a large movement task when capacity is added or removed.

Consistent hashing uses a placement scheme designed to keep much of the old assignment when membership changes. In a common ring arrangement, both keys and owner positions are placed on a circular ordered space. A key is assigned to the next eligible owner position around the ring. Adding a position takes responsibility for part of an existing interval, so keys elsewhere can retain their owners. An owner can have several virtual positions rather than one, giving placement more opportunities to spread key ranges.

Rendezvous hashing reaches a similar goal through scores. For a given key, calculate a stable score for each eligible owner and select the owner with the highest score. When an owner is added, a key moves to it only if the new score beats its previous winner. When an owner is removed, keys assigned to other winners can retain their assignment.

These schemes calculate placement; they do not transfer records or preserve updates during a transfer. Callers also need agreement about membership and hash behaviour. Changing the hash function can change the mapping and therefore requires migration planning. Finally, spreading key assignments does not guarantee equal traffic: one frequently accessed key still has one selected owner unless another mechanism distributes its work.



Clockwise successor: key 190 moves from C to new owner D.

FIG. 12.2.4 **A new owner takes only part of the hash space**

With clockwise-successor ownership, key 190 belongs to C at 240. Adding D at 210 moves it to D; the affected interval is (120,210]. Hash coordinates are illustrative.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.2.5 · Rebalancing

A partition is being moved from owner A to owner B while users continue changing its records. Copying the records once is insufficient. An update accepted by A after its record was copied would be missing from B. Rebalancing must account for both the copied starting state and the changes made during the copy.

Begin with a defined snapshot, which represents the source at a known point. Copy that snapshot to B and capture the subsequent changes. Apply those changes to the destination until it has reached the point required for handover. Check that the records and relevant invariants agree; a destination that merely contains some copied data is not ready to own new writes.

Next establish the ownership transition. A routing epoch is a version of the placement information that lets participants distinguish the old assignment from the new one. The handover procedure must say when B may accept writes and how a request using the old assignment is rejected or redirected. Without such coordination, A and B could both accept incompatible changes, or a caller could continue writing to a copy scheduled for removal.

Copying and applying changes consume the same resources that serve users, so throttle the move to leave serving capacity. Retain the old copy until validation and recovery conditions permit removal. Define rollback and reconciliation for an interrupted handover. Safe rebalancing is complete when data and request routing agree about the current owner, rather than when the initial copy finishes.

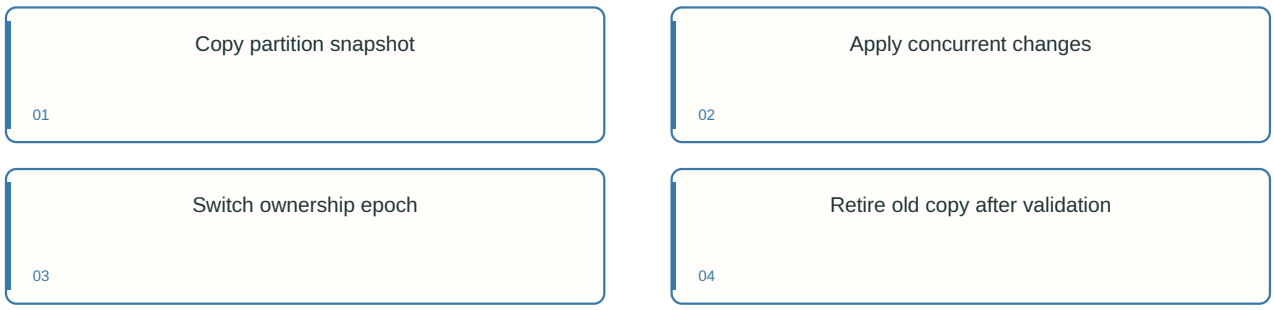


FIG. 12.2.5 **Rebalancing · the example**

Rebalancing transfers current data and authority while preserving concurrent writes, routing, validation, and recovery.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.2.6 · Hot partitions and skew

A cluster's average load looks comfortable, yet one partition receives 70% of writes. That owner develops a queue while the other partitions remain mostly idle. This uneven distribution is skew. A partition carrying a disproportionate share of the work is a hot partition, and its delay can dominate the requests that depend on it.

First identify what is uneven. One partition may contain more bytes, receive more requests, or perform more expensive operations per request. These require different remedies. Inspect traffic, bytes, resource use, and waiting by partition. A fleet average hides the relationship between the affected requests and their particular owner.

If the hot partition contains many independently writable keys, splitting its range or changing placement may divide the work. If almost all requests change one entity, creating more cold partitions does not help that entity. Read copies may distribute allowed reads, and batching may reduce some write overhead. A write rule that needs one authoritative decision still needs a way to make that decision correctly.

Spreading append operations with an extra key dimension can help when the records are independent, but later reads may have to combine several partitions. Before choosing that trade-off, ask whether the pressure comes from data volume, popularity, or a rule requiring updates to occur one at a time. The useful remedy changes the specific pressure that caused the hot partition.

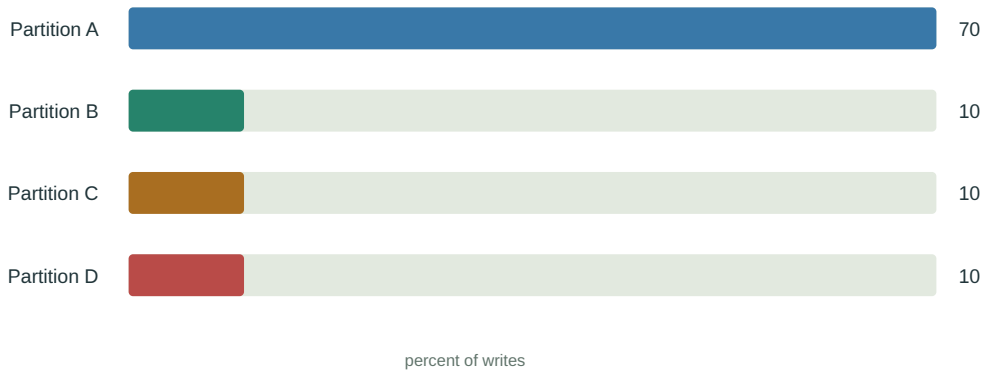


FIG. 12.2.6 **Averages hide a hot partition**

Seventy percent of writes go to A. Adding cold capacity does not split one hot ownership key.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Every caller competes for one globally hot resource that can have only one exclusive owner. Can adding many partitions make these ownership decisions independent? Explain what a valid improvement would have to preserve.

Extra partitions can distribute unrelated records, but they do not divide the decision about this one exclusive resource. Two partitions independently granting ownership could accept incompatible winners and break the rule. The hot decision still needs a protocol that establishes the permitted owner.

Read copies or batching may reduce suitable surrounding work, and a different allocation model may be possible if the requirements permit it. For any proposal, explain which work it removes or distributes and how it prevents conflicting grants. Measure the hot owner's actual work instead of inferring progress from the number of partitions.

Remember: Replication makes copies; partitioning divides ownership. Placement must support the important queries and invariants, while rebalancing must preserve updates and establish one current assignment.

Try applying it: A new owner is added to a hash-based layout, but one key still dominates write traffic. Why might the new owner remain mostly idle, and what must a later ownership move preserve while writes continue?

Unit 12 exercises → · Unit 12 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Application best practices — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Scaling and application boundary review.

   Access reviewed · 21 October 2026

2. Dynamo: Amazon’s highly available key-value store, 2007 ↗

   Original research · author organization. Replication, partitioning, and reconciliation ideas; capstones are original designs, not Dynamo deployments.

   Access reviewed · 21 October 2026

Lag, proxies & saturated pools

Prerequisites: 12.2 · Replicate and partition data 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A user changes a profile, receives success, and then sees the old value on the next page. At the same time, another request waits for a database connection even though the web fleet's CPU looks comfortable. Adding more application instances could leave the stale read unchanged and increase connection pressure.

To understand these outcomes, we will follow a write from the primary to a replica, distinguishing receipt from application. We will then follow a request through a database proxy and a connection pool, identifying which component owns its session state and where it waits.

By the end of the chapter, you should be able to connect routing to a read's freshness requirement and distinguish a connection shortage from the work holding connections busy. That evidence will help us choose a change that affects the actual bottleneck.

12.3.1 · Read replicas and read/write splitting

A user changes a profile through the primary database and receives success. The next page reads a replica that has not yet applied the change. The old profile appears. Both requests may have completed as their databases allowed, yet the sequence fails the user's expectation of seeing their own update.

Read replicas can distribute suitable queries. Read/write splitting sends writes to their authority and sends selected reads to other copies. To choose those reads, define the required freshness. A public report may permit delayed data, while the profile page may require read-your-writes: the next relevant read must include the user's completed update.

One approach is to route the critical read to the primary under the database's required read semantics. Another, where supported, is to record the write's progress marker and use a replica only after it has

applied at least that far. If no suitable replica is ready, the policy needs to say whether to wait, use another destination, or report that it cannot meet the requirement. The routing decision follows the observation the operation needs.

SQL syntax alone cannot establish that requirement. A SELECT may acquire a lock or belong to a transaction whose decisions depend on current state. Splitting such a transaction across unrelated copies can change its meaning. Read replicas add capacity for appropriate reads; they do not make all reads interchangeable or remove the separate authority for writes.

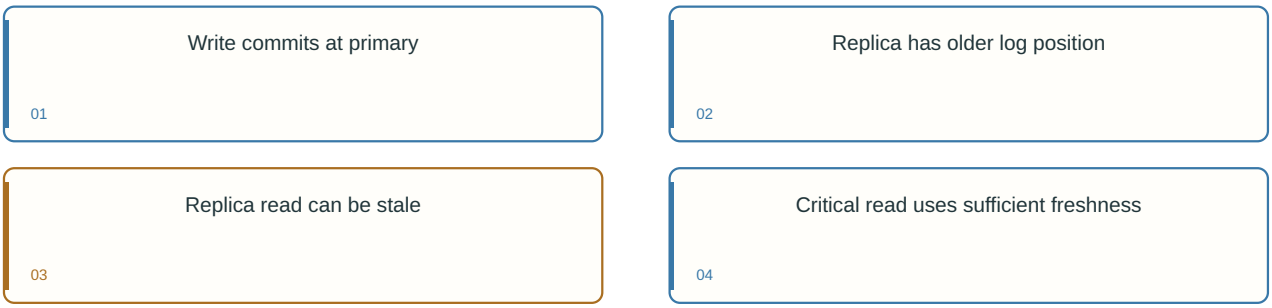


FIG. 12.3.1 **Read replicas and read/write splitting · the example**

Read/write splitting must respect observation and transaction needs, not merely classify by SQL verb.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.3.2 · Replication lag

Replication usually has several stages. The source accepts a change, the change travels to a replica, and the replica applies it so reads can observe it. A replica can therefore have received a record without yet making its effect visible. Replication lag describes how far progress at one of these stages trails the source, and the stage being measured must be named.

Suppose the source has accepted changes through position 100 in an ordered change history, while a replica has applied only through position 90. Its applied state is behind the source. Later it reaches position 95. In the trace, the source is still at 100 and the replica at 95, so a read there may miss the effects at positions 96–100. This is a comparison of progress markers, rather than proof that each position represents one second or one user request.

Where the database supports them, examine source, receipt, and apply progress separately. A delayed transport and a replica that receives changes quickly but applies them slowly need different investigation. Time-based estimates also require interpretation: variable traffic means that a given position gap does not imply a fixed duration. An average across replicas can hide one that has stopped progressing.

Use the relevant progress in operating decisions. A read-your-writes request needs a copy that includes its required change, not merely a reassuring fleet average. A promotion decision must honour the acknowledgment and possible-data-loss contract. Choosing a replacement quickly is useful only if its actual state is acceptable for the work it will take over.

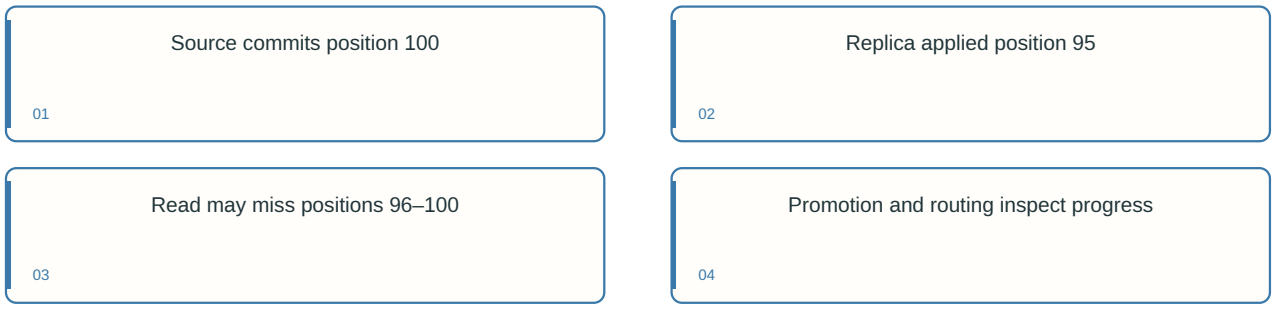


FIG. 12.3.2 **Replication lag · the example**

Replica receipt, persistence, and apply progress can differ and support different freshness or recovery claims.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.3.3 · Database proxying

Several application instances need database connections, and database destinations may change during failover. A database proxy sits between clients and database servers. Depending on its features, it can choose a destination, manage backend connections, or let several client sessions share a smaller set of database connections.

Sharing requires care because a connection can carry state as well as requests. A session variable, a temporary table, or an open transaction may belong to one particular backend session. If a later operation is sent to a different backend without preserving the required context, the application may observe different behaviour even though its SQL text is unchanged.

A proxy may keep a client attached to one backend while that state is needed. This attachment is often called pinning. For example, the transaction must remain on the session that owns it until the transaction ends. Check which features the proxy can safely share and which require pinning, and define when the backend can be released for another caller.

The proxy itself also needs capacity, identity handling, routing, and failure procedures. Reducing connection count does not remove transaction contention inside the database. Nor can a connection switch always repair a failed operation: a write may have committed before its reply was lost. Repeating that write requires the application's operation and retry contract, rather than an assumption that a fresh connection means nothing happened.

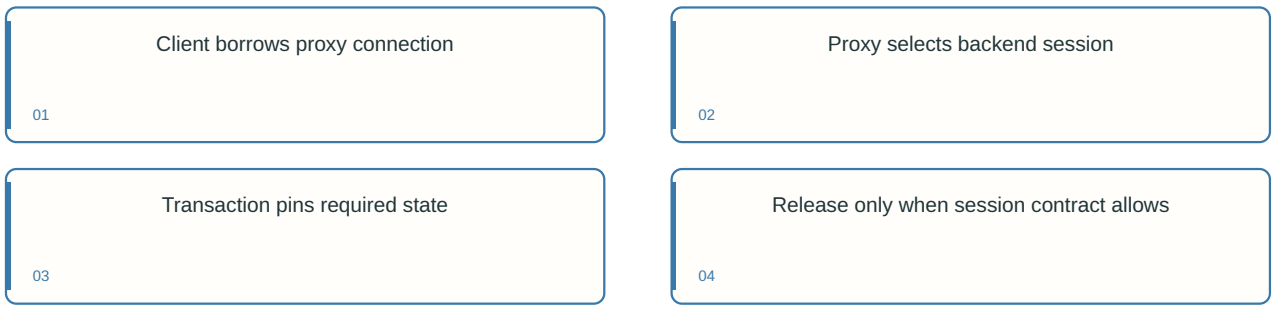


FIG. 12.3.3 **Database proxying · the example**

Database proxies add routing or pooling facilities whose session-state and retry contracts need verification.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.3.4 · Connection pool saturation

An application uses a pool of database connections so requests can borrow and reuse them. When every connection is in use, another request must wait or fail according to the pool's policy. Pool saturation means this borrowing capacity has been exhausted; it does not, by itself, explain why connections remain busy.

Suppose ten application instances each permit twenty database connections. Together they can demand 10 × 20 = 200 connections. Add ten more application instances with the same configuration, and potential demand becomes 20 × 20 = 400 connections. The shared database has not gained capacity merely because more pools now point to it.

There can be waiting at several boundaries. A request first waits to acquire a pool connection. After acquisition, its query may wait for a database lock while still holding that connection. CPU can remain modest throughout the lock wait, even as other callers queue for connections. Measure acquisition time, connection hold time, query time, and contention to locate the cause.

Set an aggregate connection budget across the expected instance count, including capacity needed for recovery and administration. Include acquisition waiting in request deadlines and bound the number of waiting callers. Shorten unnecessary ownership of a connection where possible. Raising every pool limit can transfer a queue into an already constrained database; the useful goal is more timely completed work, rather than more simultaneous borrowers.

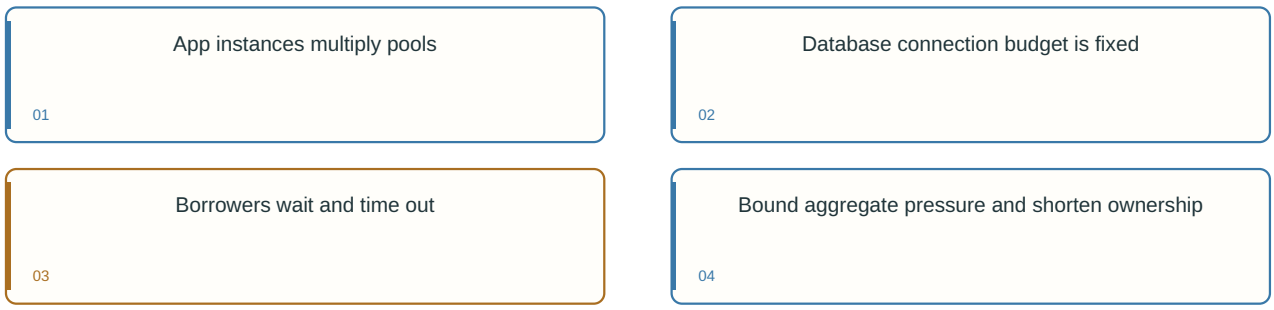


FIG. 12.3.4 **Connection pool saturation · the example**

Pool saturation includes waiting and held connections; aggregate demand across application replicas must remain bounded.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.3.5 · Bottleneck identification

A request takes a long time to complete. Adding web instances seems plausible, but most of the delay may occur after the application has sent a database query. To choose an effective change, follow the request through the stages that determine when its response can be returned. This sequence of dependent work is its critical path.

Separate execution from waiting at each stage. The request may wait for a worker, run application code, wait for a connection, execute a query, and wait for another service. The stage holding up the required outcome is the bottleneck for that workload and target. Its cause may be insufficient processing capacity, a slow access path, lock contention, or admission of more work than the stage can handle.

Compare offered traffic with useful completions, queue growth, and resource evidence. Inspect CPU, memory, disk, network, connections, and locks where relevant. High utilisation helps form a hypothesis, but does not establish which change will improve the user's result. A busy resource serving unrelated background work may have a different effect from the query every user must await.

If one query dominates the path, a justified index or query change may help more than extra web instances. If independent CPU work dominates, added compute may help. Change a material factor, then measure useful throughput and the slower end of the latency distribution again. This checks the proposed explanation and reveals whether the limit has moved to another stage.

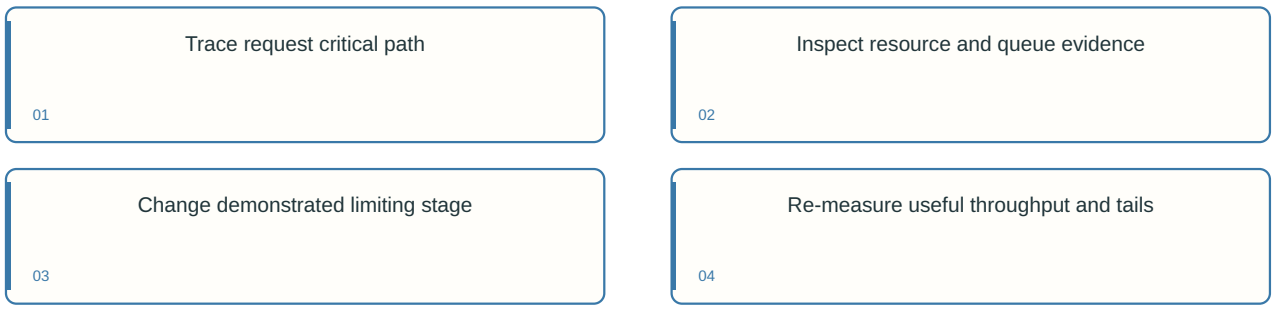


FIG. 12.3.5 **Bottleneck identification · the example**

Bottleneck diagnosis connects useful outcomes and critical-path waits to constrained resources.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · Every caller competes for one globally hot resource that can have only one exclusive owner. Can adding many partitions make these ownership decisions independent? Explain what a valid improvement would have to preserve.

Extra partitions can distribute unrelated records, but they do not divide the decision about this one exclusive resource. Two partitions independently granting ownership could accept incompatible winners and break the rule. The hot decision still needs a protocol that establishes the permitted owner.

Read copies or batching may reduce suitable surrounding work, and a different allocation model may be possible if the requirements permit it. For any proposal, explain which work it removes or distributes and how it prevents conflicting grants. Measure the hot owner's actual work instead of inferring progress from the number of partitions.

Remember: Route reads according to their required progress, and trace connection acquisition and hold time before changing pool sizes or adding application instances.

Try applying it: Ten instances each allow twenty database connections. Ten more instances are added while requests wait on database locks. How does potential connection demand change, and which measurements would help distinguish the cause from the resulting pool wait?

Unit 12 exercises → · Unit 12 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Application best practices — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Scaling and application boundary review.

   Access reviewed · 21 October 2026

2. Dynamo: Amazon’s highly available key-value store, 2007 ↗

   Original research · author organization. Replication, partitioning, and reconciliation ideas; capstones are original designs, not Dynamo deployments.

   Access reviewed · 21 October 2026

Regions, write paths & scale economics

Prerequisites: 12.3 · Lag, proxies & saturated pools 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A burst reaches the service while workers are already busy. The queue grows, and the team proposes both more nodes and a second region. Before accepting either proposal, we need to ask which work can finish in time, which failures must be tolerated, and who may authorise a write when regions cannot communicate.

This chapter follows those questions from controlled waiting to regional ownership. We will compare a copied product description with a claim to the last seat, so the difference between read capacity and write coordination becomes concrete. We will also examine dependencies that can stop recovery even when several application replicas exist.

Finally, we will compare the complete cost of designs that meet the same service requirements. The goal is to justify added capacity by useful outcomes, including behaviour during failure, rather than by the size of the topology.

12.4.1 · Queueing and overload

A queue holds work until a worker can process it. During a brief burst, requests may arrive faster than workers complete them. If arrivals later fall below completion capacity, the workers can drain the backlog. The queue has moved some work into a later interval; it has not increased the workers' processing rate.

If arrivals remain above completions, the backlog continues to grow. Waiting consumes memory or storage and also consumes the caller's time. A request can become useless before the queue runs out of space. For example, a request with a two-second deadline cannot finish on time if it is silently accepted behind ten seconds of required work.

Bound both the amount of waiting work and its age. Admission control decides whether to accept new work when capacity is limited. It can reject work that cannot meet its deadline, shed lower-priority work under the agreed policy, or reserve capacity for required operations. These decisions should happen before waiting callers consume the resources needed to complete existing work.

Make the overload response clear enough for callers to react sensibly. Immediate repeated retries can add work to the very queue that caused the failure. Asynchronous acceptance is also a distinct contract: if the system promises to finish later, it needs to preserve and track that accepted work. A queue is useful for controlled waiting within a capacity and deadline policy; sustained excess demand still requires less admitted work or more useful completion capacity.

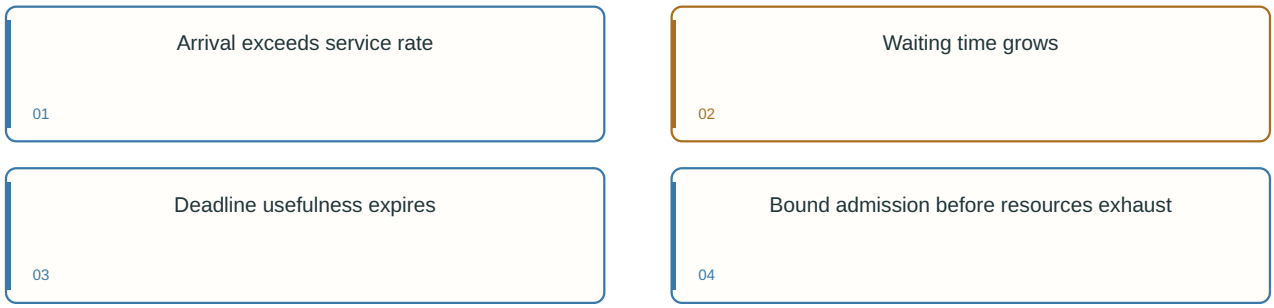


FIG. 12.4.1 **Queueing and overload · the example**

Queues absorb temporary mismatch but sustained overload requires admission, capacity, or workload changes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.4.2 · Single points of failure

Two application containers run on one host. If one process crashes, the other may continue. If the host fails, both disappear. Counting two application boxes therefore answers a process-failure question, but does not establish that service survives a machine failure.

A single point of failure is a dependency whose loss defeats the required service. It may be a machine, a router, a credential source, or a procedure that only one unavailable operator can perform. The requirement matters: a component can be dispensable for an already running request yet essential for replacing instances after an outage.

A failure domain groups resources that can be lost together through a common cause. Place redundant capacity across the failure domains the service must tolerate, then inspect what remains shared. Two instances on different hosts can still depend on one unavailable state owner. Redundancy needs valid state, failure detection, a workable handover, and enough surviving capacity, as well as separated processes.

Trace recovery as carefully as normal serving. Suppose existing instances can run with cached configuration, but every new instance must contact one configuration service during startup. Losing that service may leave present traffic working while preventing all replacement capacity. Include these startup and recovery dependencies when looking for single points of failure. The useful question is which required journey stops when a dependency disappears.

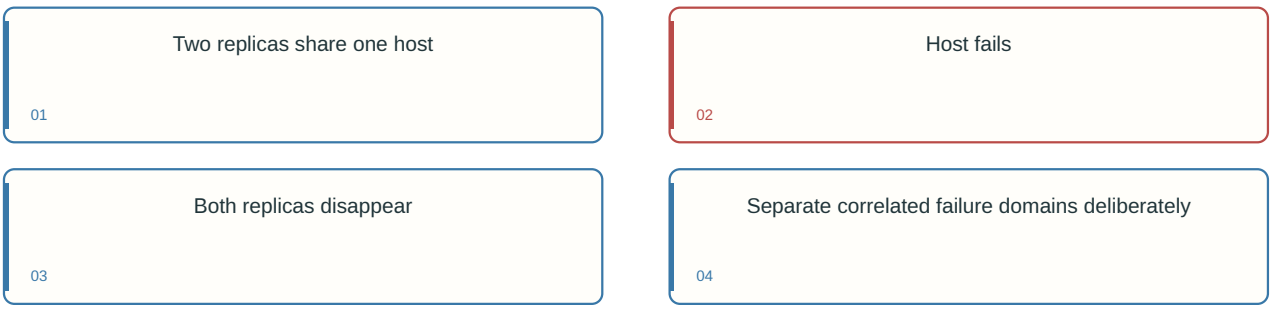


FIG. 12.4.2 **Single points of failure · the example**

A single point of failure can be any shared dependency defeating the required service or recovery.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.4.3 · Multi-region strategies

A service in one region is considering a second region to reduce distance to users or continue through a regional failure. Before choosing a topology, state the tolerated interruption, possible data loss, and location requirements. Then decide where each kind of write is authoritative and which copies can serve each kind of read.

In an active-passive arrangement, one side normally owns the authoritative activity and another is prepared to take over. Promotion needs a rule for deciding when the old authority has stopped and whether the replacement has sufficiently current state. Recovery must also say how the former primary returns without resuming an incompatible role.

In an active-active arrangement, more than one region handles normal activity. This still requires precise ownership. Some operations may have separate regional owners; others may coordinate before accepting an update or accept concurrent changes under a defined conflict policy. During a communication partition, the regions need a rule for which operations can continue and what each caller may observe.

Waiting for a required remote acknowledgment adds propagation time to the write path. Asynchronous copying can avoid that immediate wait, but a promoted copy may lack recent acknowledged changes if the source is lost. Apply the chosen trade-off to actual operations: a delayed public page and an exclusive

reservation need different contracts. A second region is useful when its authority, freshness, and recovery behaviour meet the requirement, rather than merely because another copy exists far away.

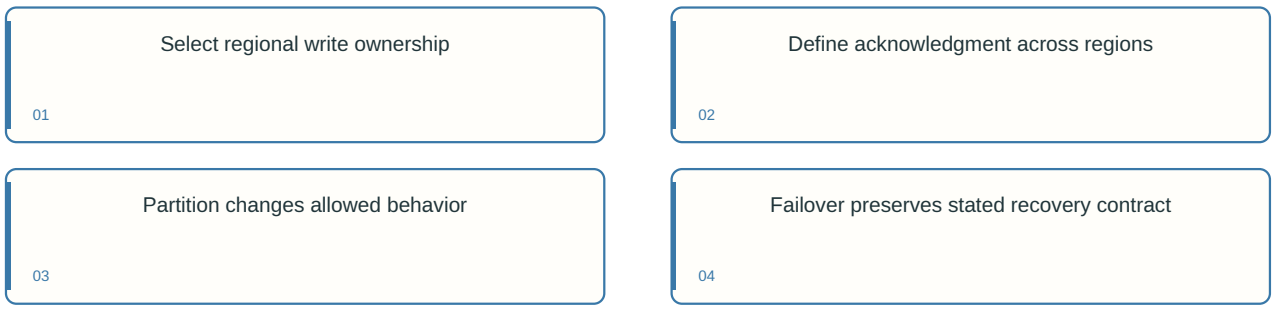


FIG. 12.4.3 **Multi-region strategies · the example**

Multi-region operation needs write authority, read freshness, loss, latency, and rejoin policies.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.4.4 · Scaling writes vs scaling reads

Many users can read copies of a product description if the service permits a freshness delay. Two callers cannot both receive an exclusive claim to the last seat. Reads and writes place different demands on a design because a read observes a representation, while a write changes authoritative state under the application's rules.

For suitable reads, replicas or caches let several owners serve reusable representations. The design still needs freshness, permission, and invalidation rules. Those copies can reduce pressure on the source, but serving more copies does not decide who wins a conflicting state change.

To scale writes, first identify what must be coordinated. Independent tenants or records may be assigned to different owners. Batching can combine suitable work and reduce repeated overhead. Removing unnecessary coordination or maintaining fewer unnecessary indexes may also reduce the cost of each write. Each change must preserve the invariants the original operation enforced.

One hot account or a global sequential allocator can remain a serial point even when thousands of unrelated keys are distributed. Some counter updates can be combined because their effects commute: applying them in different orders gives the same combined value. An exclusive seat claim has a different rule, because incompatible winners cannot both be accepted. The write design must distribute or reduce work without losing that rule; adding read replicas alone cannot supply it.

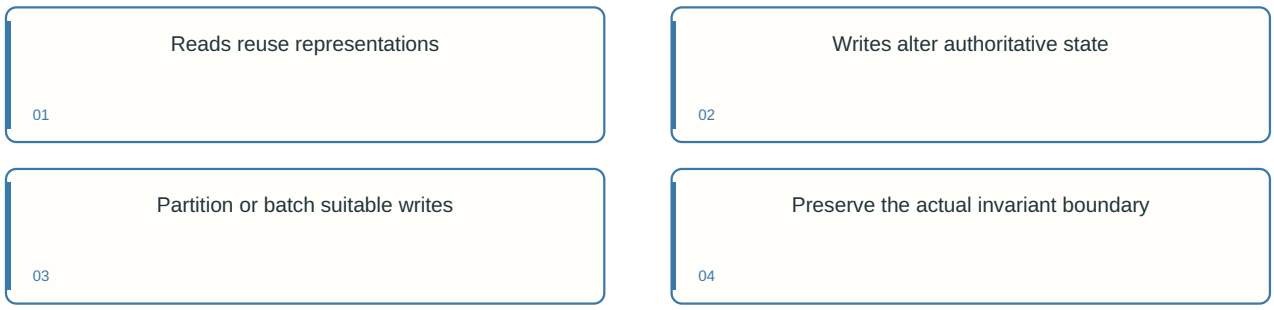


FIG. 12.4.4 **Scaling writes vs scaling reads · the example**

Read scaling can reuse copies; write scaling must divide or reduce ownership work while preserving invariants.

Author-created illustration. Numbers are illustrative unless stated otherwise.

12.4.5 · Scale-out economics

Suppose two candidate designs meet the same workload target: one uses a larger node, and another uses ten smaller nodes. The smaller machine's individual price does not tell us which arrangement costs less. Compare the complete systems at equal response-time, correctness, and failure requirements.

Each node can repeat baseline memory, connection pools, monitoring, and operating work. Distribution can add transfer and coordination. On the other hand, added nodes may let the service add capacity in smaller steps or keep useful capacity through a failure. Count both the added work and the benefit under the actual workload, rather than assuming that scale-out is automatically cheaper or more expensive.

Include spare capacity for the same failures in both candidates. Also account for startup, cache warming, migration, and recovery. A design that meets the target only when every node is available is not equivalent to one sized to meet it after the required failure. Managed services can exchange provider charges for some operating work, while the application team still owns its correctness and recovery decisions.

Finally, relate cost to useful outcomes. Cost per attempted request can look low when many attempts fail or repeat the same operation. Cost per successfully completed business outcome makes those retries and failures visible. Purchase capacity where evidence shows it can increase the required completions. Extra nodes beyond a shared bottleneck can raise the bill while leaving the service's useful throughput almost unchanged.

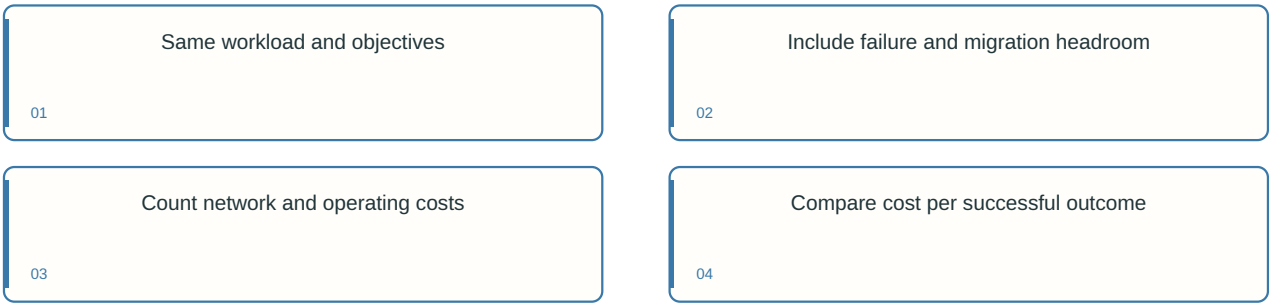


FIG. 12.4.5 **Scale-out economics · the example**

Scale economics includes surviving capacity, coordination, transfer, baseline resources, and operating effort per useful outcome.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · STABLE RENDEZVOUS MAPPING

```
from mechanisms import rendezvous
for key in range(1000):
    old = rendezvous(key, ["A","B","C"])
    new = rendezvous(key, ["A","B","C","D"])
    assert old == new or new == "D"
```

The loop examines the 1,000 integer keys from 0 through 999. For each key, the first call selects its rendezvous owner among A, B, and C. The second call includes D as an additional candidate. The variables old and new hold those two selected owners.

The assertion permits two outcomes: the key keeps its old owner, or its new owner is D. This follows from stable scores for each key-owner pair. A, B, and C retain their previous ordering, so their earlier winner cannot be displaced by another of those same owners merely because D joins. Only D can introduce a new winning score. The loop checks this property over the fixture; it does not assert that any particular fraction of keys moves.

The code calculates assignments without moving stored data or synchronising clients' membership information. A real handover still needs those procedures. Also, every key has one selected owner in this model. If most requests use one key, they still reach that owner even when the counts of assigned keys look well spread.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · TRY CHANGING THE INPUTS

Compare owners before and after adding D

Key 190: old owner **B**; new owner **B**.

Across keys 0–999, 246 change owners. Counts after adding D: A=228, B=252, C=274, D=246.

Every changed key moves to D; keys never move between A, B, and C here. This assignment says nothing about traffic per key or how data is copied.

Change the key and compare its owner before and after D joins. The same key always receives the same scores in this example. Choosing a new owner does not copy data, agree on membership, or spread the traffic of a single popular key.

▼ Think further · Every caller competes for one globally hot resource that can have only one exclusive owner. Can adding many partitions make these ownership decisions independent? Explain what a valid improvement would have to preserve.

Extra partitions can distribute unrelated records, but they do not divide the decision about this one exclusive resource. Two partitions independently granting ownership could accept incompatible winners and break the rule. The hot decision still needs a protocol that establishes the permitted owner.

Read copies or batching may reduce suitable surrounding work, and a different allocation model may be possible if the requirements permit it. For any proposal, explain which work it removes or distributes and how it prevents conflicting grants. Measure the hot owner's actual work instead of inferring progress from the number of partitions.

Remember: Queues, redundancy, regional authority, and cost must be judged against the same completion and recovery requirements. Added topology helps when it changes the actual limit without breaking the required behaviour.

Try applying it: A team proposes more shards and a second region for an exclusive reservation service. Which part of the write rule still needs an authoritative decision, and how should the proposal define behaviour when the regions cannot communicate?

Unit 12 exercises → · Unit 12 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Application best practices — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Scaling and application boundary review.

   `Access reviewed · 21 October 2026`

2. Dynamo: Amazon’s highly available key-value store, 2007 ↗

   Original research · author organization. Replication, partitioning, and reconciliation ideas; capstones are original designs, not Dynamo deployments.

   `Access reviewed · 21 October 2026`

PART 3 / UNIT 13

File Systems, Search Infrastructure, and Data Pipelines

A user starts uploading a large PDF and loses the connection before the application reports success. Some bytes may have reached storage, some may still be missing, and a document record may already exist. Before retrying, the application needs to know which upload and which file those states belong to.

The running example

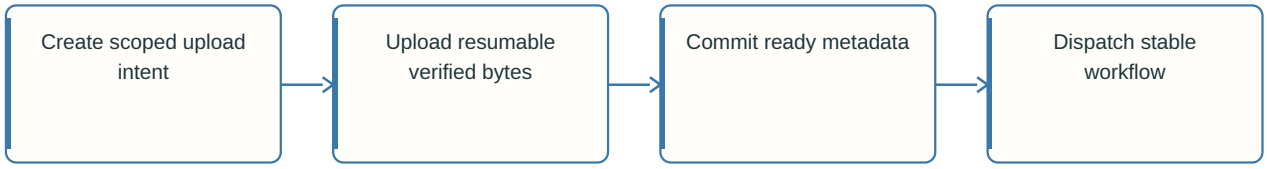
Design a document-processing system accepting millions of files, tracking processing state, extracting metadata, maintaining provenance, and exposing searchable results.

Unit 13 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.



Arrows: ordered stages of this worked mechanism

FIG. 13.0.1 **Upload pipeline**

Input bytes and control metadata have different owners and failure windows.

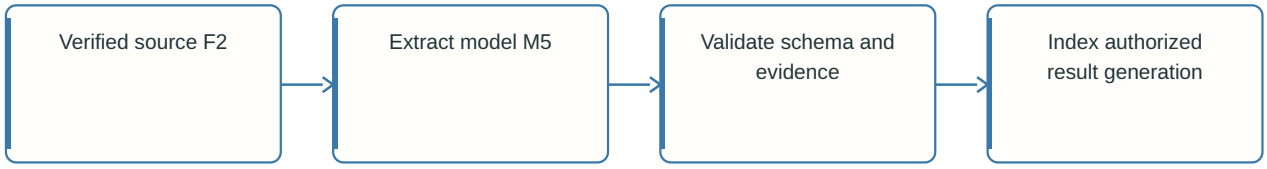
Author-created illustration. Numbers are illustrative unless stated otherwise.

Source / term	Representation	Meaning
D1	blue car	document text
D2	blue bike	document text
blue	D1, D2	posting list
car	D1	posting list

FIG. 13.0.2 **An inverted index is a term-to-document map**

Token analysis creates the lookup representation. Source versions and permissions remain separately owned.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 13.0.3 **ingestion DAG**

A stage's versioned artifact and durable transition make replay inspectable.

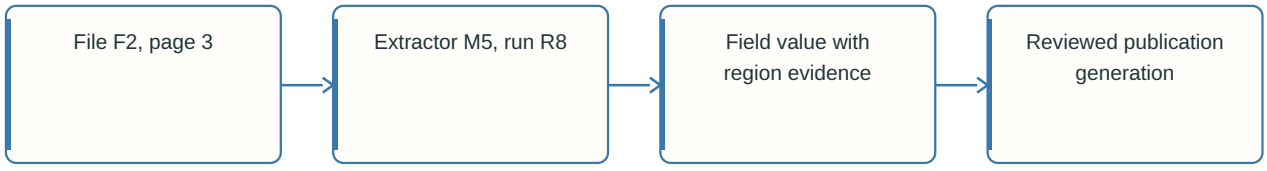
Author-created illustration. Numbers are illustrative unless stated otherwise.

Property	Batch	Streaming
Input	Bounded set	Continuing events
Freshness	Scheduled recomputation	Progress and lateness policy
Recovery	Replay bounded batch	Restore state and offsets

FIG. 13.0.4 **batch vs stream timeline**

Neither mode is automatically correct; output semantics and replay identity must be explicit.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 13.0.5 **lineage graph**

Lineage connects exact source and transform identities through derived artifacts.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Upload bytes without losing identity

Prerequisites: API Design and Communication Contracts, Data Models and Storage Technologies, Database Internals, Transactions, and Concurrency Control

5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A user starts uploading a large PDF and loses the connection before the application reports success. Some bytes may have reached storage, some may still be missing, and a document record may already exist. Before retrying, the application needs to know which upload and which file those states belong to.

In this chapter, we will separate binary content from processing metadata, follow a direct upload into storage, and resume a transfer from confirmed progress. We will then use checksums and stable file versions to decide when stored bytes are ready to become processing input.

13.1.1 · Binary/object storage

A document-processing service receives a PDF. The worker needs the file's bytes, while the application needs to know who owns it, whether the upload finished, and how processing is progressing. These two kinds of data have different jobs. The PDF may remain unchanged while its review status changes several times.

An object store provides operations on named objects containing bytes and associated metadata. The application can place the PDF there and keep its mutable processing record in a database. That record should name a particular object version, along with the owner, expected size, and integrity information needed to recognise the input. Treating completed input versions as immutable means that replacing a file creates another version rather than silently changing the bytes beneath an existing reference.

Follow the handoff between the two stores. First upload the bytes, then verify their identity and completeness, and finally commit the metadata reference that makes them available for processing. The

object upload and the database commit are separate operations. If the commit fails, the bytes may exist without a usable document record. If the record is published too early, a worker may see a reference to an incomplete file. Track these states and reconcile abandoned objects instead of treating either storage operation as completion of the whole workflow.

The term object storage describes an interface category. The chosen service and configuration still determine its read, write, versioning, and recovery behaviour. State those guarantees separately from the application's promise that a document is ready.

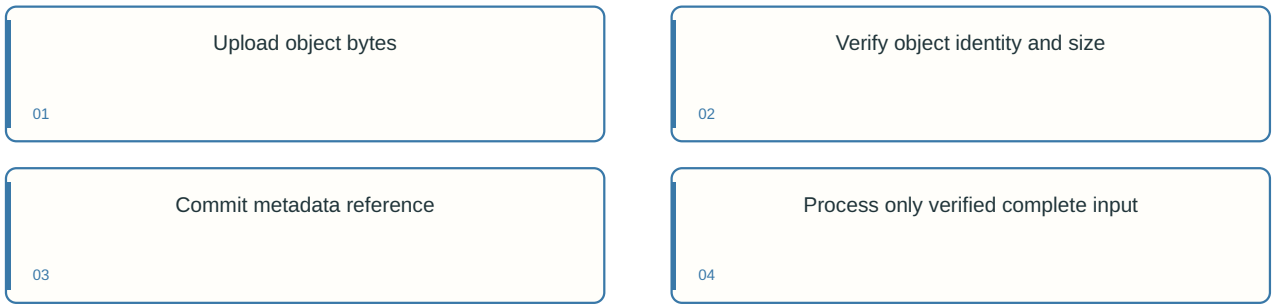


FIG. 13.1.1 **Binary/object storage · the example**

Large binary objects and mutable workflow metadata need stable identities and an explicit lifecycle handoff.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.1.2 · Large-file upload architecture

Suppose a user uploads a large scanned report. If the application server receives every byte and forwards it to storage, that transfer occupies the server's network connections and other resources. An alternative is to let the client send the bytes directly to storage while the application coordinates the operation.

The application first checks the user's permission and quota, then creates an upload intent: a record identifying the owner, intended object, and expected limits. It can issue a narrowly scoped authorisation, such as a signed upload URL, for the client to use. Such a URL is a capability: possessing it grants the operation described by its scope and lifetime. Limit it to the intended upload rather than allowing general access to the storage collection.

After the client sends the bytes, the server verifies that the intended object actually completed and satisfies the required size, ownership, and integrity checks. A client message saying “finished” is a request to check completion, not evidence that all the expected bytes were stored. Only after successful verification should the application make a durable processing job ready.

A client can disconnect, abandon an intent, or complete the transfer just before its response is lost. Keep enough state to recognise a repeated completion request. Expire unused intents and clean up unreferenced objects through a process that distinguishes abandoned uploads from uploads still in progress. Direct transfer changes where the bytes travel; the application still owns admission, permissions, and the ready decision.

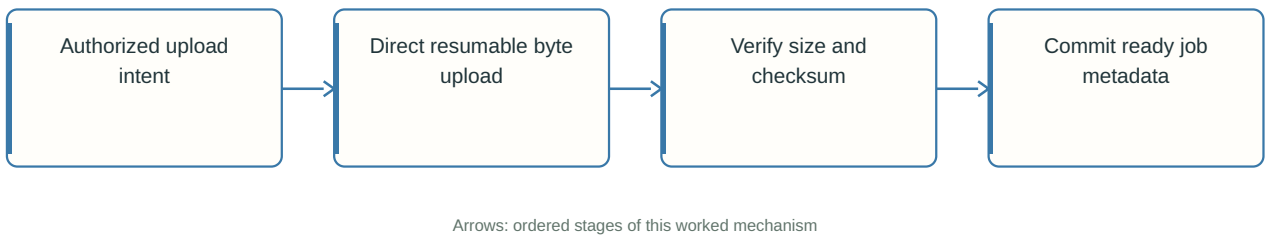


FIG. 13.1.2 **The upload pipeline separates bytes from control**

Incomplete uploads never enter processing as verified input. Metadata failure leaves cleanup/reconciliation work rather than a fictitious cross-service transaction.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.1.3 · Multipart and resumable uploads

A connection fails near the end of a large upload. Sending the whole file again wastes work that storage has already accepted. A multipart upload divides the file into identified parts and assembles them in a separate completion operation. A resumable upload lets the client continue from confirmed progress, which may be represented by parts or by byte offsets, depending on the protocol.

Consider a session whose manifest expects parts 1, 2, 3, and 4. Storage confirms parts 1, 2, and 4 before the connection fails. On reconnecting, the client checks the session's recorded progress and sends the missing part 3. Retries must use the correct session and part identity so that they replace or confirm the intended part rather than add unrelated data. The manifest specifies the order of assembly; arrival order alone is insufficient.

The client must also keep the source file consistent across attempts. If it sends the first parts of one revision and the final part of a later revision, every part may be present while the assembled file is wrong. Bind the session to the intended source and verify the completed object's size and integrity under the chosen protocol. A provider's entity tag is not universally a hash of the complete file, so do not use it as one without checking its definition.

Confirmed parts are intermediate progress. The application should expose a usable file only after assembly and final verification. Incomplete sessions also need an expiry and cleanup policy because their stored

parts consume space even though no completed file appears in the document list.

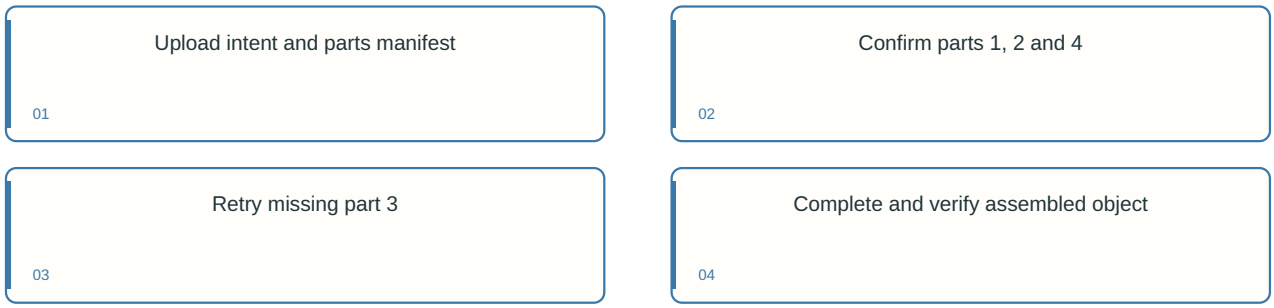


FIG. 13.1.3 **Multipart and resumable uploads · the example**

Resumable or multipart upload tracks confirmed pieces and separately verifies the completed object's identity and integrity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.1.4 · Checksums and content addressing

Suppose a file is uploaded and later downloaded. Comparing their lengths can detect missing bytes, but two different files can have the same length. A checksum computes a value from the bytes so that a later computation can help detect changes. The strength of that check depends on the algorithm and the kinds of corruption or interference it is intended to detect.

A suitable cryptographic digest can also be used as a content identity. The application computes the digest of the actual bytes and uses it to name or locate that content. This is content addressing. Under the algorithm's collision assumptions, matching digests provide evidence that the bytes match. The same mechanism can support deduplication: several references can point to one stored copy of identical content.

The check must cover the bytes that were actually received or stored. Recording a digest supplied by the client, without verifying it, records a claim rather than establishing integrity. If the application has an independently expected digest, compare the computed value against it before accepting the input.

Content identity answers which bytes a reference describes. Permission answers whether a caller may access them. A user who knows another tenant's file hash has not gained permission to download that file. Nor does a matching hash show that the file is safe to process. Authorise each reference separately, and consider the scope of deduplication: an endpoint that reveals whether a hash already exists can disclose another tenant's content even without returning the bytes.

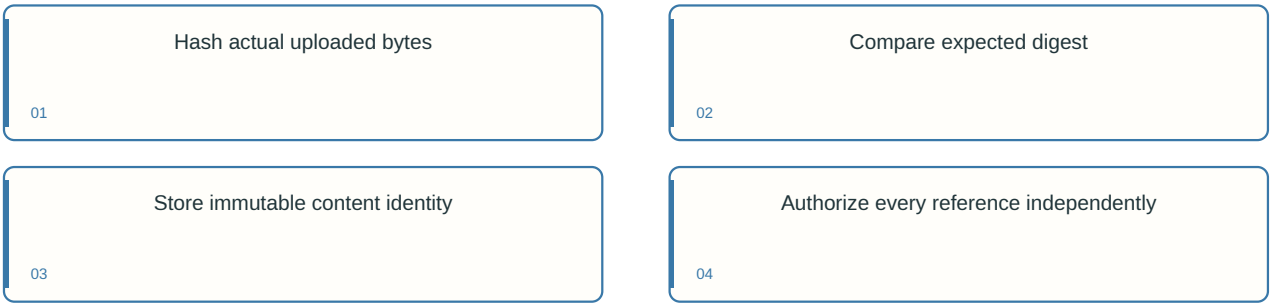


FIG. 13.1.4 **Checksums and content addressing · the example**

Checksums detect corruption under algorithm assumptions; content addressing identifies bytes without establishing permission or safety.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.1.5 · Metadata vs binary data

An extractor reads a PDF and records an invoice amount. A week later, the user replaces the PDF. If the earlier result merely names “invoice.pdf,” which bytes did it describe? A mutable filename is insufficient evidence.

Give each binary version a stable identity: F1 for the original bytes and F2 for the replacement. Metadata describes ownership, annotations, classification, processing attempts, and review status while explicitly naming the relevant input. An extraction result should record its file version and the extractor or model version used. Changing review status need not change the original bytes.

When F2 arrives, deliberately update the current-document reference and decide which outputs need recomputation. An F1 output may remain valid history, but it must not silently become an F2 output. A late F1 worker must also be prevented from replacing the current result. This separates “What were the input bytes?” from “What is happening to the document now?” Stable identity makes the relationship explainable; the retention policy determines how long the historical evidence remains available.

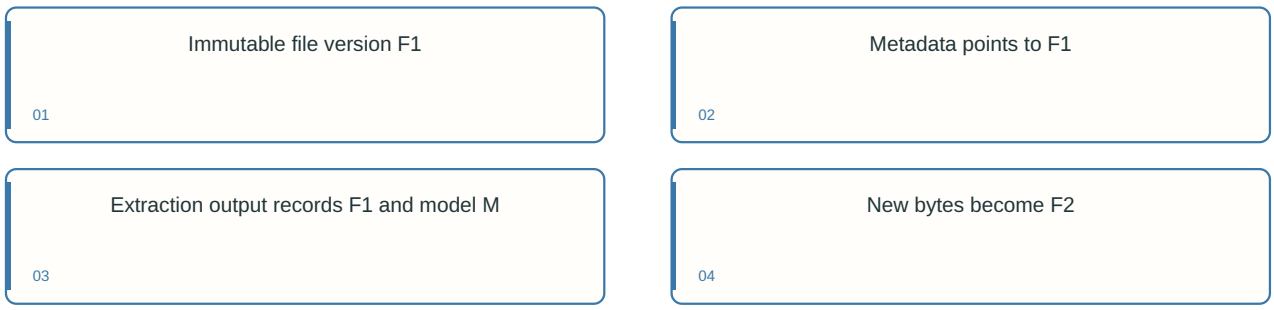


FIG. 13.1.5 **Metadata vs binary data · the example**

Immutable source versions keep changing annotations and processing attempts traceable to the bytes they used.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How would you investigate an extracted field after both source and model have changed?

Locate the disputed output's own identity first. Follow its provenance to the exact source version and region, the model or transform version, relevant parameters, and any review decision. Compare the field with that retained source evidence. Running the current model on the current document examines different inputs and cannot establish what produced the old value. Even rerunning the recorded version may not reproduce a nondeterministic execution exactly. State that limitation separately from missing provenance, and retain the evidence the investigation requires within the applicable lifecycle policy.

Remember: Connect each upload intent to verified, stable binary content. Confirmed transfer progress becomes usable input only after completion checks and the metadata handoff succeed.

Try applying it: A client finishes uploading a PDF, but the metadata transaction fails. What state should keep the file from being processed, and how could cleanup distinguish this abandoned object from an upload still in progress?

Unit 13 exercises → · Unit 13 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OpenLineage: core model and documentation ↗

   Official specification documentation · 1.54.0 at access. Dataset, job, and run lineage concepts.

   Access reviewed · 21 October 2026

2. Elasticsearch: Reindex documents ↗

   Official documentation · accessed 21 Oct 2026. Implementation reindex controls; full generation-switch protocol in the atlas is an original design.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Build and rebuild a search index

Prerequisites: 13.1 · Upload bytes without losing identity 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A PDF has been accepted, but searching for a phrase from it returns nothing. The search engine needs a representation of the file's text, and building that representation is another operation that can fail or fall behind.

We will begin with two short documents to see how an inverted index finds candidates. Then we will follow extraction and indexing as distinct stages. Finally, we will build a replacement index while documents continue changing, and identify the evidence needed before searches can safely switch to it.

13.2.1 · Search engines and inverted indexes

Consider two documents: D1 contains “blue car,” and D2 contains “blue bike.” To answer a query for car, we could read both documents and inspect their text. With millions of documents, repeating that scan for every query would do a great deal of unnecessary work.

An inverted index organises the lookup in the other direction. For each term, it records the documents containing that term. Here, blue points to D1 and D2, car points to D1, and bike points to D2. Each such collection is a posting list. A query for both blue and car can find their common document, D1. More elaborate search systems also use stored fields, positions, and ranking information, but the term-to-document relationship is the starting point.

Before terms enter the index, an analyser divides text into tokens and may normalise them. For example, its case-handling rule determines whether Blue and blue become the same indexed term. The query must use a compatible interpretation of text. Changing that interpretation can require rebuilding the representation of existing documents.

The index is derived from recoverable source content. Keep document identity, source version, and transformation information so it can be updated or rebuilt. An older extraction must not overwrite the indexed version of a newer file. Search freshness and access also need their own rules: a correctly stored original does not prevent an index from serving an outdated or newly private representation.

Source / term	Representation	Meaning
D1	blue car	document text
D2	blue bike	document text
blue	D1, D2	posting list
car	D1	posting list

FIG. 13.2.1 **An inverted index is a term-to-document map**

Token analysis creates the lookup representation. Source versions and permissions remain separately owned.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.2.2 · Indexing pipelines

The application reports that a PDF was uploaded, but a search for its title finds nothing. Several events may lie between those outcomes. A worker must read the file, extract text, validate the result, perhaps enrich it, and write a searchable representation. An indexing pipeline makes these stages and their progress explicit.

Start a job from a verified source version. Each stage should record the identified input it used and the output or failure it produced. If extraction succeeds but the index write fails, recovery should be able to retry the unfinished step rather than lose the job or assume the whole document is searchable. Malformed files need a bounded retry policy and an actionable review or quarantine state, so one repeatedly failing input does not consume work indefinitely.

A retry should name the same intended document and operation. A stable document ID lets an index update the intended representation rather than insert a second searchable document on every attempt. Version checks are also necessary. A stable ID alone cannot stop an old worker from replacing newer text. A completion must still match the current input version and authorised attempt before its result is accepted.

Finally, distinguish storing an indexed representation from publishing it for users. The application should report the source and index version whose required stages completed, and enforce current access policy

when serving it. A copied “public” flag can become stale while processing is under way. Stage progress is useful only when the final published result still belongs to the right version and audience.

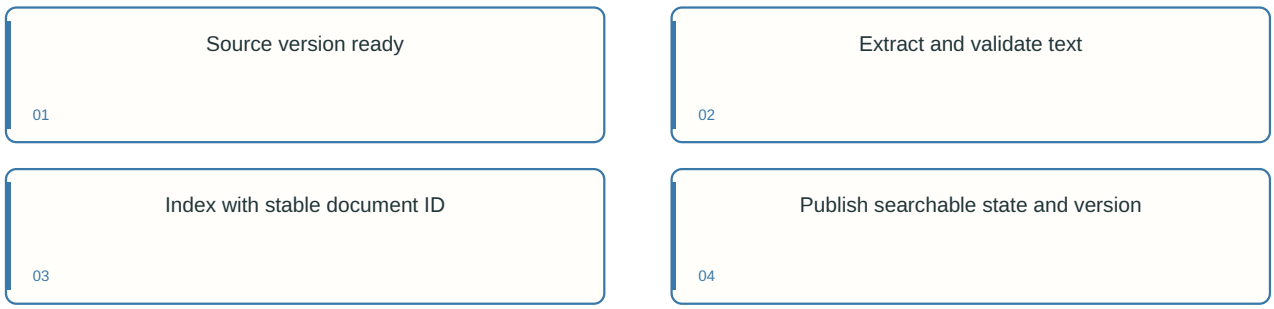


FIG. 13.2.2 **Indexing pipelines · the example**

Indexing stages need identified inputs, repeat-safe output, validated completion, and an owner for failed progress.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.2.3 · Search ingestion and reindexing

Suppose the search analyser changes. Existing index generation A was built using the old analyser, so the application builds generation B while A continues serving searches. Reindexing creates a new searchable representation; it does not stop the source documents from changing during the build.

Begin with a defined source snapshot or change position. That boundary tells us which source state the initial copy represents. While the copy runs, retain a supported way to capture later inserts, updates, and deletions. Then apply those changes to B in a way that preserves the intended versions. If a document is copied and later deleted, the deletion must reach B too. Otherwise the new index can bring removed content back into search.

Before switching, establish a catch-up boundary and validate B against the relevant source state. Check identities, versions, contents, access scope, and deletion handling. Equal counts are weak evidence: B can contain one obsolete document and miss one new document while its count still matches A. Also make sure that changes arriving at the moment of the switch have a defined destination and cannot fall between the build and live-ingestion paths.

A serving alias or routing pointer can select B once these conditions hold. Keep A while the rollback plan remains useful, but recognise that an old generation needs appropriate updates if it is to serve current results again. The safe transition consists of the copy, change capture, validation, and handoff together; changing the pointer is only its visible final step.

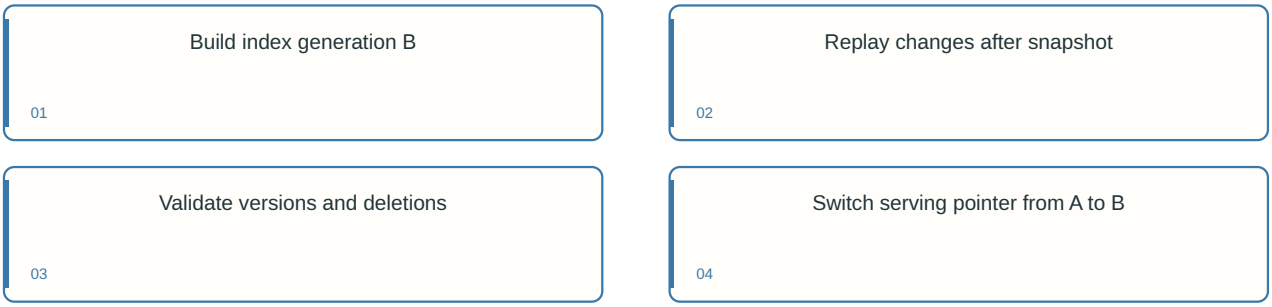


FIG. 13.2.3 **Search ingestion and reindexing · the example**

Reindexing builds and validates a new generation while catching up edits and deletes before serving switches.

Author-created illustration. Numbers are illustrative unless stated otherwise.

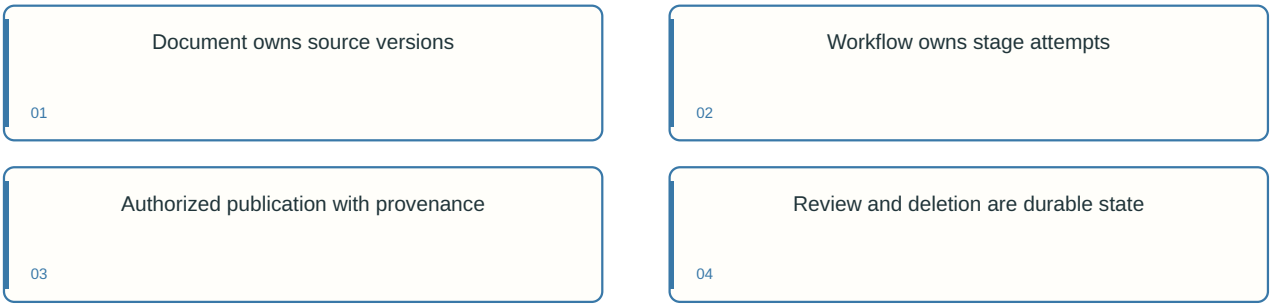


FIG. 13.2.4 **State and ownership in the related design**

Fields and decisions retain tenant, input, transform, and evidence relationships.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How would you investigate an extracted field after both source and model have changed?

Locate the disputed output's own identity first. Follow its provenance to the exact source version and region, the model or transform version, relevant parameters, and any review decision. Compare the field with that retained source evidence. Running the current model on the current document examines different inputs and cannot establish what produced the old value. Even rerunning the recorded version may not reproduce a nondeterministic execution exactly. State that limitation separately from missing provenance, and retain the evidence the investigation requires within the applicable lifecycle policy.

Remember: An index is a versioned representation of source data. Its update and rebuild paths must preserve the intended content, deletions, and access rules through the serving handoff.

Try applying it: Generation B contains the same number of documents as generation A. Give an update-and-deletion sequence that could leave B incorrect despite that matching count. What would you check before switching searches to B?

Unit 13 exercises → · Unit 13 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OpenLineage: core model and documentation ↗

   Official specification documentation · 1.54.0 at access. Dataset, job, and run lineage concepts.

   `Access reviewed · 21 October 2026`

2. Elasticsearch: Reindex documents ↗

   Official documentation · accessed 21 Oct 2026. Implementation reindex controls; full generation-switch protocol in the atlas is an original design.

   `Access reviewed · 21 October 2026`

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   `Access reviewed · 21 October 2026`

Move data through batch and streams

Prerequisites: 13.2 · Build and rebuild a search index 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

The document service produces a report each morning. It also needs to flag certain arrivals within minutes. Both tasks use incoming data, but they require different freshness and recovery behaviour.

This chapter compares bounded batch jobs with continuing streams. We will examine where transformations run, how committed source changes reach another store, and how raw datasets become analytical inputs. A late event will then help us distinguish event time from processing time and decide whether a published result can still change.

13.3.1 · Batch vs streaming pipelines

A team wants yesterday's document totals each morning. It can collect a bounded set of records, process them, and publish one report. This is batch processing. Another team wants an alert soon after each document arrives. Its input keeps arriving, so it uses continuous, or streaming, processing.

The distinction concerns how work and progress are organised. A batch has a defined input boundary and can finish. A stream processor handles an ongoing sequence and records how far it has progressed. Streaming can reduce the delay between an arrival and its result, but the delay still includes ingestion, waiting, computation, and publication. Calling a pipeline streaming does not make its output instantaneous.

Ask what happens when input arrives late. A daily batch can be rerun with a corrected input set. A stream may need to revise a previously published total. In both cases, consumers need to know whether a result is provisional and how corrections will be identified. Reprocessing also needs stable input and output identities so that repair does not create duplicate results.

Choose according to the required freshness, input volume, and recovery work. A scheduled batch may be sufficient when hours of delay are acceptable. Streaming may be justified when later results would miss the purpose of the service, but it adds continuously maintained state, checkpoints, and late-event policies. Batch work has its own resource and failure costs, especially when many jobs compete to finish before the same deadline.

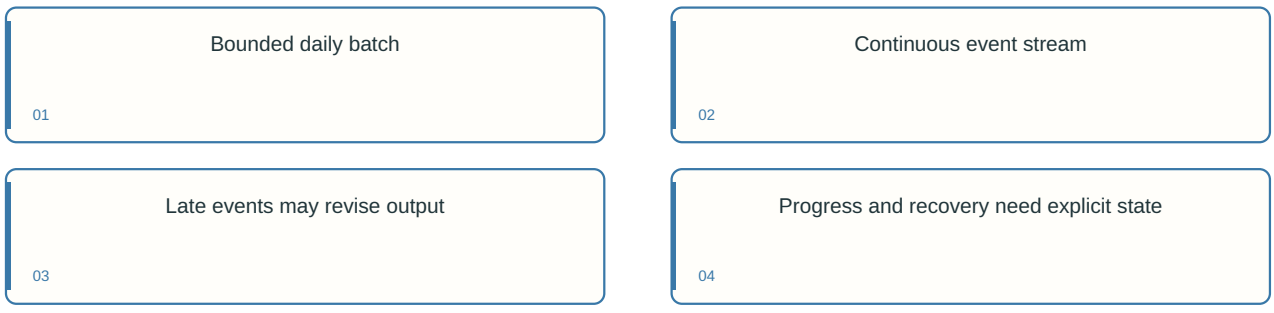


FIG. 13.3.1 **Batch vs streaming pipelines · the example**

Batch works on bounded input; streaming maintains continuing progress, with different freshness and recovery costs.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.3.2 · ETL vs ELT

An analytical report needs invoice totals but does not need every personal field extracted from the PDFs. Where should the pipeline produce its analytical input? ETL and ELT describe two possible orders.

ETL means extract, transform, load. The pipeline extracts source data, transforms it, then loads the result into the target. It might validate currencies and remove unnecessary personal fields before writing an analytical table. ELT means extract, load, transform: extracted data enters the target environment first, and transformations run there. Loading first can support several later analyses, but the target must then govern the broader raw data it receives.

Choose the placement by processing capabilities, privacy requirements, and query needs. Both arrangements require versioned transformations, input and output checks, and lineage. If a currency conversion rule changes, the team must identify which outputs used the earlier rule and which inputs remain available for repair. The acronym describes step order; the surrounding contracts establish quality and repeatability.

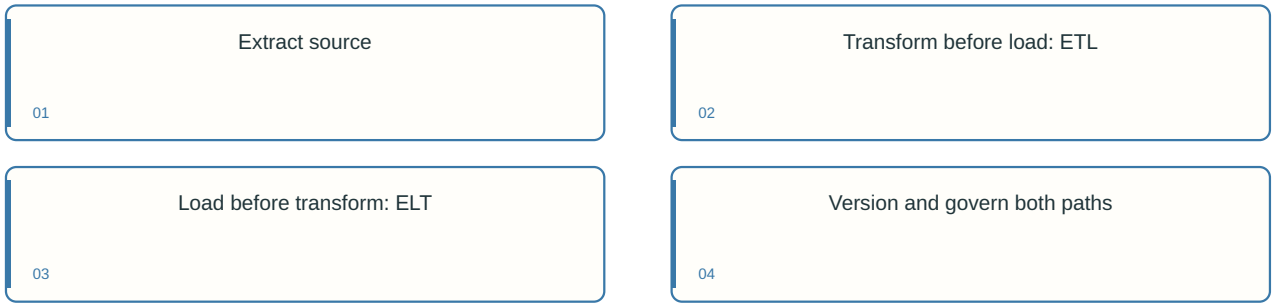


FIG. 13.3.2 **ETL vs ELT · the example**

ETL transforms before target loading; ELT transforms after loading, changing raw-data placement and governance work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.3.3 · Change Data Capture (CDC)

A document's title changes in the source database. Search and analytics need the change too. Repeatedly scanning every source row can spend substantial work rediscovering records that did not change. Change Data Capture, or CDC, uses a supported mechanism to make source changes available to downstream consumers, commonly from the database's record of committed changes.

A new destination usually needs both existing data and subsequent changes. These must meet at a defined boundary. Imagine copying a row before an edit and beginning the change stream only after that edit. If the edit lies between those two observations, the destination can miss it. A snapshot and stream therefore need a coordinated handoff using the source mechanism's supported position or protocol.

Preserve source identities and enough ordering, version, and schema information to apply changes correctly. A repeated event should not create another business record, and an older update should not undo a newer one. Include deletes and schema changes in the plan. Record durable progress and monitor how far the destination is behind; receiving changes is different from making them visible to queries.

The source may commit several row changes in one transaction while the destination applies them separately. A destination query could then see only part of that transaction unless the application or connector supplies the required grouping. CDC transports changes. The destination's application and serving rules determine what readers can observe.

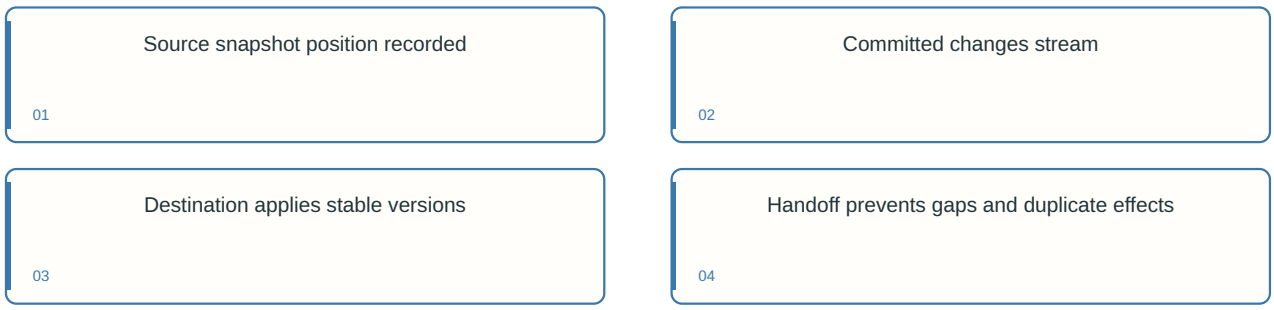


FIG. 13.3.3 **Change Data Capture (CDC) · the example**

CDC transports committed source changes through a supported handoff and ordering mechanism.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.3.4 · Data lakes and warehouses

A document service retains PDFs, extracted text, and analytical tables. A worker needs one exact PDF; an analyst scans a year's validated invoice totals. These operations need different representations of the same underlying information.

A data lake retains datasets in flexible forms, often using object storage. A warehouse provides organised analytical structures and querying facilities. Product boundaries overlap, so compare supported operations rather than assigning capabilities from the label. Separate raw source versions from curated outputs whose fields and meanings have been checked. Catalog information should identify each dataset's owner, schema, transformation, and update state. Files alone do not tell a reader whether two datasets use compatible units or versions.

Choose file formats and partitions for actual queries and recovery tasks. Time partitions can reduce unrelated reads for a monthly report, while doing little for a lookup across one user's scattered records. Retained raw inputs can enable replay, but they also preserve sensitive information. Raw and curated datasets alike need a purpose, access policy, lineage, and retention rule.

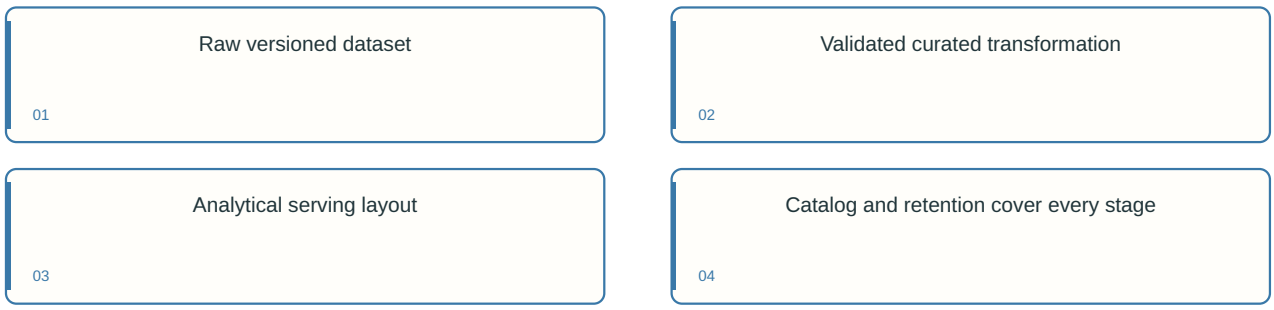


FIG. 13.3.4 **Data lakes and warehouses · the example**

Lakes and warehouses organise raw and curated analytics, requiring ownership, catalog, quality, and lifecycle rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.3.5 · Stream processing fundamentals

An event records a document arriving at 10:00, but a processor receives it at 10:03. The source occurrence has an event time of 10:00. The processor handles it at a processing time of 10:03. These times answer different questions, and the reported event time also depends on the source's clock and timestamp rules.

Suppose a report groups arrivals into one-minute event-time windows. This event belongs to the interval from 10:00 up to 10:01 even though it reaches the processor later. If that interval's count has already been published, the processor needs a policy: revise the count, emit a correction, or treat the event as too late for that result. Consumers must know what “final” means under the selected policy.

Stream processors maintain state such as counts per tenant and window. A watermark represents an estimate of progress through event time under specified assumptions about delay. It can help decide when to publish or close a window, but it cannot prove that no older event will ever arrive. Define allowed lateness and the treatment of events outside that allowance rather than treating the watermark as a statement about every possible future input.

Recovery needs both processing state and input progress. A checkpoint records information used to resume after failure. If a processor restores an earlier count and rereads input, its publication rules must handle repetition. Saving a checkpoint does not by itself make an external write or notification part of the same atomic operation. State which results can be repeated, corrected, or recomputed after restart.

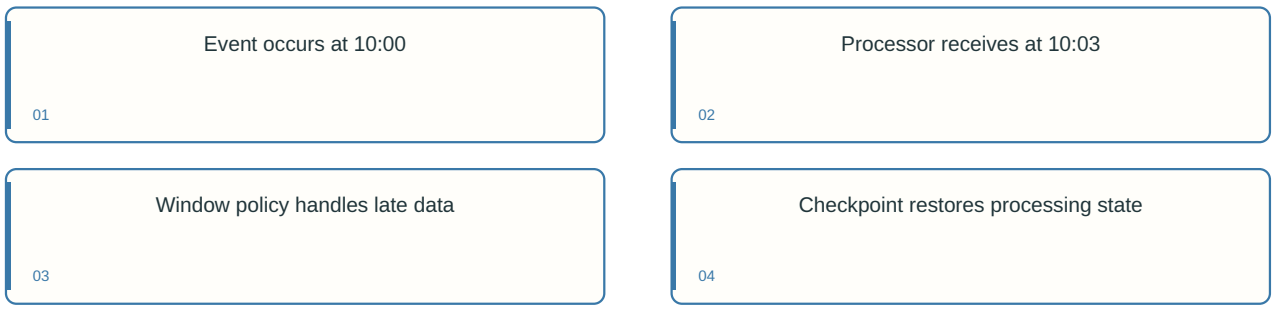


FIG. 13.3.5 **Stream processing fundamentals · the example**

Stream processing needs keyed state, event/processing time, lateness, corrections, and recoverable progress.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How would you investigate an extracted field after both source and model have changed?

Locate the disputed output's own identity first. Follow its provenance to the exact source version and region, the model or transform version, relevant parameters, and any review decision. Compare the field with that retained source evidence. Running the current model on the current document examines different inputs and cannot establish what produced the old value. Even rerunning the recorded version may not reproduce a nondeterministic execution exactly. State that limitation separately from missing provenance, and retain the evidence the investigation requires within the applicable lifecycle policy.

Remember: Choose batch or streaming by the required freshness and recovery behaviour. Define the source-to-destination handoff, the meaning of time, and the treatment of repeated or late input.

Try applying it: A 10:00 event arrives at 10:03 after its one-minute window's count has been published. If consumers require corrected event-time totals, what must the processor communicate about the old result and the late arrival?

Unit 13 exercises → · Unit 13 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OpenLineage: core model and documentation ↗

   Official specification documentation · 1.54.0 at access. Dataset, job, and run lineage concepts.

   Access reviewed · 21 October 2026

2. Elasticsearch: Reindex documents ↗

   Official documentation · accessed 21 Oct 2026. Implementation reindex controls; full generation-switch protocol in the atlas is an original design.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Quality, provenance & replay

Prerequisites: 13.3 · Move data through batch and streams 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A reviewer disputes an invoice amount after both the PDF and the extractor have changed. Rerunning the current pipeline will answer a new question: what does today's transformation produce from today's file? It may not explain the earlier result.

We will identify the source evidence and transformation history needed to investigate that result. We will then distinguish structural checks from checks of meaning, replay repaired work without accepting stale attempts, and follow deletion through derived copies and recovery. These steps bring the unit's separate mechanisms together into one document-processing design.

13.4.1 · Data provenance and lineage

A reviewer disputes an extracted invoice amount. The current PDF may differ from the one used earlier, and the current model may differ too. We need evidence about the particular output being examined.

Provenance records where a result came from. This field might name file F version 2, page 3 or a specific region, extractor M version 5, relevant parameters, and any review decision. Stable references should locate the evidence actually used. Lineage connects artifacts through transformations: the PDF feeds an extraction run, its fields feed a reviewed dataset, and that dataset feeds search and a report. These links also identify which downstream outputs may need correction.

Such records support investigation without establishing that the value is correct. A recorded model version does not guarantee an identical rerun, especially for nondeterministic execution. Preserve the evidence the intended investigation needs and state reproducibility limits. Provenance records also need a lifecycle: copying personal text into a permanent audit record can undermine deletion of the original.

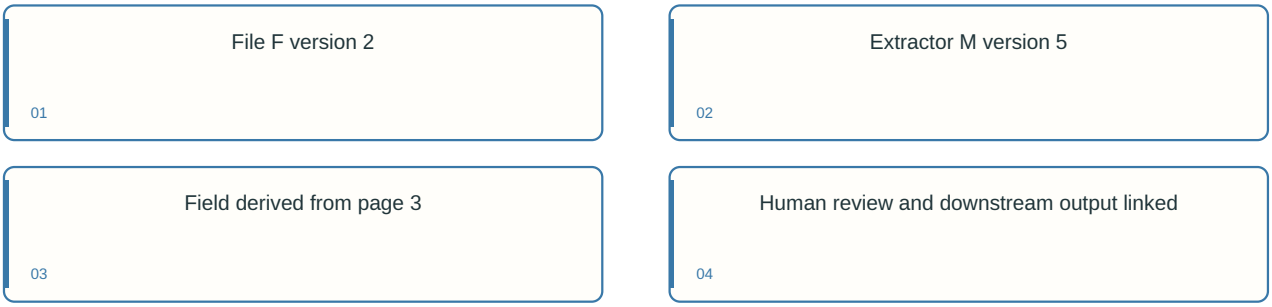


FIG. 13.4.1 **Data provenance and lineage · the example**

Provenance identifies origins; lineage links transformations so outputs can be investigated and corrected.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.4.2 · Data quality, schema validation, and replay

An extracted record contains an amount and a currency, and both have the expected data types. It passes a structural schema check. Yet the number 100 might mean 100 dollars in one version and 100 cents in another. The fields are present, but the interpretation is wrong.

Schema validation checks the supported shape of a record, such as required fields, types, and schema version. Semantic validation checks meaning: units, permitted ranges, relationships, source identity, and business rules. Completeness matters as well. A dataset with valid individual records can still be missing half the expected documents. Compare useful invariants and source versions alongside row counts.

When an input fails validation, quarantine it with a clear reason and enough source evidence to investigate. Quarantine is a controlled state for repair or review, not an instruction to retry the same impossible transformation forever. Once the cause is corrected, replay applies the repaired transformation to the identified inputs. Keep the transform version visible so the new output is distinguishable from the old one.

Replay must preserve effect identity. Reprocessing one field should update or replace the intended result rather than send another payment, email, or other external action. It must also avoid publishing obsolete work. The worked Workflow example rejects epoch 7 when epoch 8 owns the attempt, and rejects file-version-1 when file-version-2 is required. Only the current combination is accepted. In a real store, checking those conditions and publishing the result must be atomic; the small example demonstrates the predicate, not a complete durable pipeline.

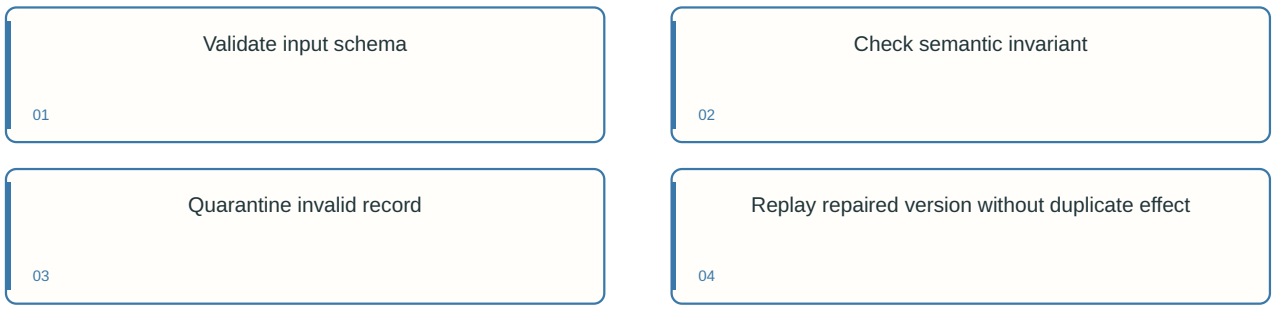


FIG. 13.4.2 **Data quality, schema validation, and replay · the example**

Quality checks validate meaning as well as shape, while replay needs stable versions and safe effect handling.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.4.3 · Lifecycle policies and retention

A user deletes a PDF and can no longer download it. However, a search still returns its extracted text, and a thumbnail remains visible. Deleting the original bytes has removed one artifact while leaving others that contain the same information.

A lifecycle policy states how long data is needed and what happens as it expires, is archived, or is deleted. Follow the document through metadata, extracted fields, previews, search entries, caches, logs, and recoverable copies. Identify which policy applies to each. Keep references that let the deletion workflow find the relevant artifacts without repeatedly scanning all stored files.

Record a durable deletion intent and the progress of its required steps. Failed removals can then be retried. Also consider work already in flight: a worker that read the PDF before deletion may finish afterwards and attempt to publish fresh extracted text. Completion checks must reject work for a deleted document. A minimal deletion marker may be needed to stop old events or replay from recreating it; that marker should retain only the information its purpose requires.

Recovery can reopen the same problem. An older backup may contain a document that was deleted after the backup was taken. Before restored data is exposed, apply the relevant deletion decisions and reconcile its derived copies. Retention limits, recoverable copies, and deletion history must be designed together, so restoring service does not silently restore content that should remain removed.

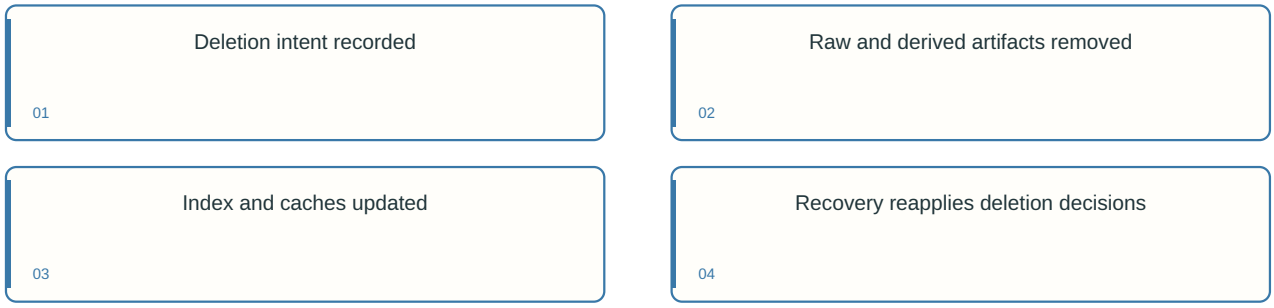


FIG. 13.4.3 **Lifecycle policies and retention · the example**

Lifecycle rules follow originals, intermediates, indexes, logs, and recovery copies through required deletion.

Author-created illustration. Numbers are illustrative unless stated otherwise.

13.4.4 · Designing data-intensive applications

Follow one document through the service. Its upload produces a verified binary version. A durable job refers to that version. Extraction produces identified fields, review may correct them, and indexing produces a searchable representation. The user experiences one service even though each step has its own state and can fail separately.

For each handoff, ask what proves that the input is ready, who owns the next step, and what identity a retry must reuse. Record the source and transformation versions. Preserve deterministic transformations where appropriate, and record the relevant configuration and evidence when a transformation cannot be reproduced exactly. A worker should publish only while its input version and attempt remain current; a deleted or replaced document must not be revived by late completion.

Then trace failure at the connections. Bytes may exist before metadata commits. A job may complete just before its reply disappears. A document may change while reindexing runs. Each case needs durable progress, validation, and a recovery rule that reconciles the resulting states. Retain inputs needed for rebuilding within their lifecycle policy, rather than assuming that either infinite history or immediate deletion suits every artifact.

Finally, observe the outcome users need. Queue throughput alone cannot show that documents are searchable, that the oldest accepted job is advancing, or that every result belongs to the correct tenant and source version. Track progress and unresolved failures across stages. A data-intensive application's correctness comes from the connected contracts governing its data, transformations, and effects, as well as the individual storage and messaging components.

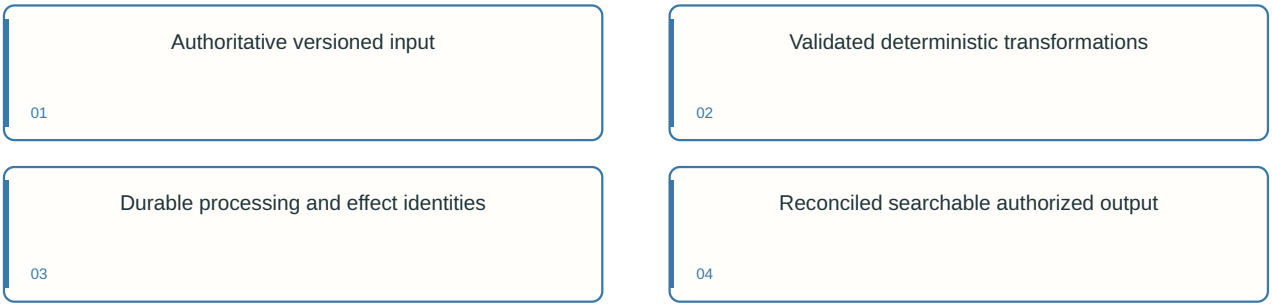


FIG. 13.4.4 **Designing data-intensive applications · the example**

Data-intensive design connects versioned transformations with explicit authority, validation, progress, and recovery.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · SOURCE AND ATTEMPT FENCING

```
from mechanisms import Workflow
run = Workflow("file-version-2", epoch=8)
assert not run.complete(7, "file-version-2", "old")
assert not run.complete(8, "file-version-1", "wrong-input")
assert run.complete(8, "file-version-2", "current")
```

The workflow starts with file-version-2 as its required input and epoch 8 as the current attempt. The first call supplies epoch 7. Although its input version matches, complete returns False and leaves the result unset because the attempt is old. The second call supplies epoch 8 but names file-version-1, so it is rejected for using the wrong input. The third call supplies both current values and stores the artifact named current.

These checks prevent the two illustrated stale completions from replacing the intended result. Workflow also rejects completion when its deleted flag is set, although the displayed calls do not exercise that condition.

This is a local state model. In a real store, checking authority and publishing the result must happen atomically, or deletion or replacement could occur between the check and the write. The artifact itself still needs validation. Provenance should name the transformation version and relevant source evidence as well as the output location.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · TRY CHANGING THE INPUTS

Should this worker be allowed to finish?

Reject completion

The run now belongs to epoch 8. A worker from epoch 7 can no longer save its completion.

Choose an attempt and check its ownership epoch, input version, and document state. A real database must check these conditions and save the result atomically. This step does not include an external model call or an object-store write.

▼ Think further · How would you investigate an extracted field after both source and model have changed?

Locate the disputed output's own identity first. Follow its provenance to the exact source version and region, the model or transform version, relevant parameters, and any review decision. Compare the field with that retained source evidence. Running the current model on the current document examines different inputs and cannot establish what produced the old value. Even rerunning the recorded version may not reproduce a nondeterministic execution exactly. State that limitation separately from missing provenance, and retain the evidence the investigation requires within the applicable lifecycle policy.

Remember: A useful output names its source and transformation, passes checks of structure and meaning, and can be repaired within a lifecycle that also covers derived copies and restored data.

Try applying it: A worker reads a PDF, then the user deletes it before the worker finishes. Which completion check and deletion evidence would stop that worker—and a later replay or backup restore—from republishing its extracted text?

Unit 13 exercises → · Unit 13 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OpenLineage: core model and documentation ↗

   Official specification documentation · 1.54.0 at access. Dataset, job, and run lineage concepts.

   Access reviewed · 21 October 2026

2. Elasticsearch: Reindex documents ↗

   Official documentation · accessed 21 Oct 2026. Implementation reindex controls; full generation-switch protocol in the atlas is an original design.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

PART 4 / UNIT 14

Distributed Systems Fundamentals

An order service sends a payment request and then loses contact with the provider. The order still says pending. The provider may already have charged the card. We will begin by tracing this uncertainty, then ask what failures our design covers and how to order events when different machines show different times.

The running example

A social application writes a new post in one region while a friend reads it from another. Examine different consistency contracts and their user-visible effects.

Unit 14 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

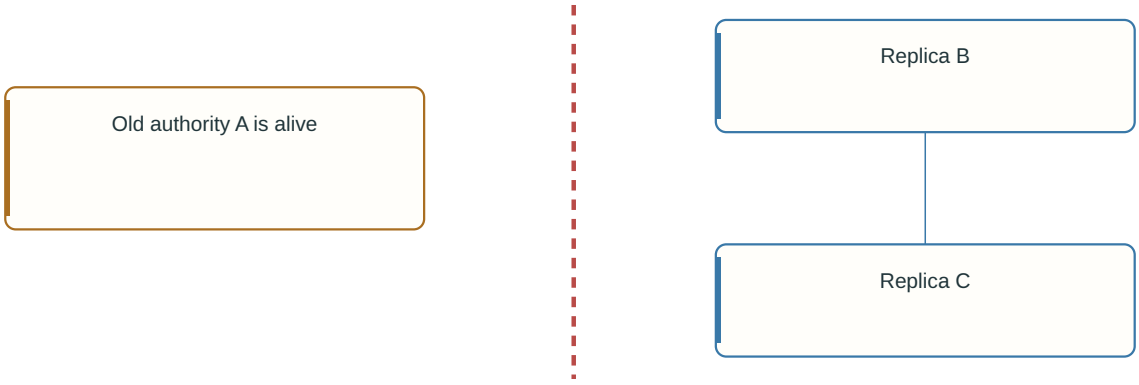


FIG. 14.0.1 **A partition can leave two live but disconnected sides**

Region A remains running while B and C form another reachable side. Correct authority and fencing determine who may make which writes; a missing heartbeat cannot stop A.

Author-created illustration. Numbers are illustrative unless stated otherwise.

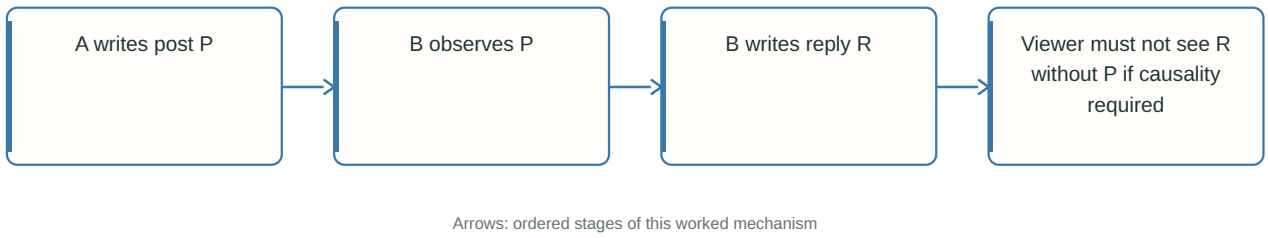


FIG. 14.0.2 **causality timeline**

Causality follows program and message dependencies, not arbitrary wall-clock ordering.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Model	Required constraint	What is not implied
Linearizable	Real-time atomic operations	External-effect transaction
Serializable	Equivalent serial transactions	Real-time order
Causal	Preserve dependencies	One total global order
Eventual	Converge under assumptions	Bounded freshness by itself

FIG. 14.0.3 **consistency model comparisons**

Name the model rather than treating strong and eventual as complete application descriptions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Set	Members	Intersection
Write W=2	A, B	B
Read R=2	B, C	B
Fixed N=3	R + W = 4 > 3	one shared member

FIG. 14.0.4 **Intersecting quorums still need a protocol**

Intersection preserves a possible witness. Version, membership, persistence, and concurrency rules determine how that witness is used.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Partial failure & the meaning of time

Prerequisites: Networking Foundations for System Design, Runtime, Processes, Threads, and Concurrency, Database Internals, Transactions, and Concurrency Control

7 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An order service sends a payment request and then loses contact with the provider. The order still says pending. The provider may already have charged the card. We will begin by tracing this uncertainty, then ask what failures our design covers and how to order events when different machines show different times.

14.1.1 · What makes a distributed system difficult?

An order service asks a payment provider to charge a card. The provider records the charge and sends a reply. Before the order service receives it, the connection breaks. The customer has made one purchase, but the two services now know different things about it.

A distributed system has participants that communicate and can fail independently. One machine can remain healthy while another stops, and both machines can remain healthy while their messages are lost. Each participant sees its own state and the messages that reach it. It cannot directly inspect everything happening elsewhere.

This changes how we design an operation. We must decide who owns the order, who owns the payment, what each success response promises, and how the services discover an outcome after losing contact. A protocol supplies the rules for sending messages, changing state, acknowledging work, and recovering from interruption.

Additional machines can provide capacity or keep service running through some failures. They also create more combinations of delayed messages and incomplete work. Begin with the requirement that needs

distribution, then follow one operation through both its ordinary path and an interrupted path. The connected boxes in a diagram become a design when we can explain these rules.

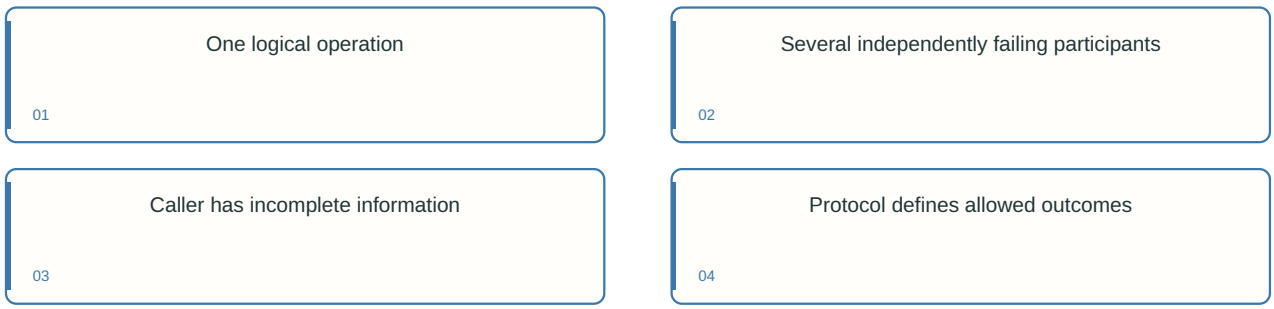


FIG. 14.1.1 **What makes a distributed system difficult · the example**

Distributed participants observe only parts of an operation, so legal outcomes need explicit message and recovery protocols.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.1.2 · Partial failures

Follow the payment more closely. First, the order service sends a charge request. Next, the provider commits the charge. Finally, the provider sends a success response, which is lost. The payment succeeded, but the order service has not learned that result.

This is a partial failure: one part of the overall work has happened while another part has failed or remains uncertain. The order service should distinguish “we have not attempted payment” from “we attempted payment but do not yet know its outcome.” Treating both states as a definite failure loses information needed for recovery.

Give the payment a stable operation reference and retain the local intention to charge. If the provider supports looking up that reference, the order service can ask what happened and update the order accordingly. Comparing the local record with the provider's record is reconciliation. Repeating a submission also requires the provider's documented duplicate-handling rules; a reference is useful only when the relevant participants use it consistently.

Rolling back a local database transaction does not remove a charge already stored by the provider. A refund, where supported, is a further business action called compensation. It has its own possible failures and may have different fees or timing. Recovery begins by finding the actual outcome, rather than assuming that a missing reply erased it.

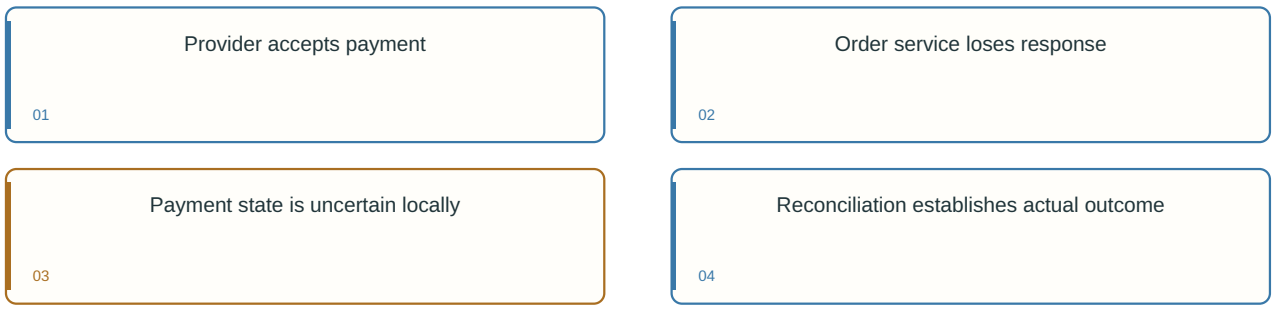


FIG. 14.1.2 **Partial failures · the example**

Partial failure permits one participant's effect to succeed while another fails to observe or record it.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.1.3 · Network uncertainty

A service receives a short message from its leader every second. Such a message is called a heartbeat. Five seconds pass without one. The leader may have crashed, but it may also be paused, overloaded, or separated from the observer by a slow network.

From the missing messages alone, these situations can look the same. In a network with no known upper bound on delivery time, waiting longer still cannot establish with certainty that the leader has stopped. A timeout says that the observer is no longer willing to wait. It supplies a practical suspicion of failure.

The distinction matters when the observer starts a replacement. The earlier leader may resume, and a request sent to it may still arrive. Similarly, retrying a timed-out payment can overlap the original attempt. A correctness rule must remain valid when the timeout's suspicion is wrong.

Use deadlines to bound waiting and release resources. Use the ownership or duplicate-handling protocol to decide which effects may occur. Later in this unit, we will see fencing: a resource checks an authority number and rejects work from an earlier owner. While valid authority cannot be established, an operation that needs exclusive ownership may have to wait or reject the request.

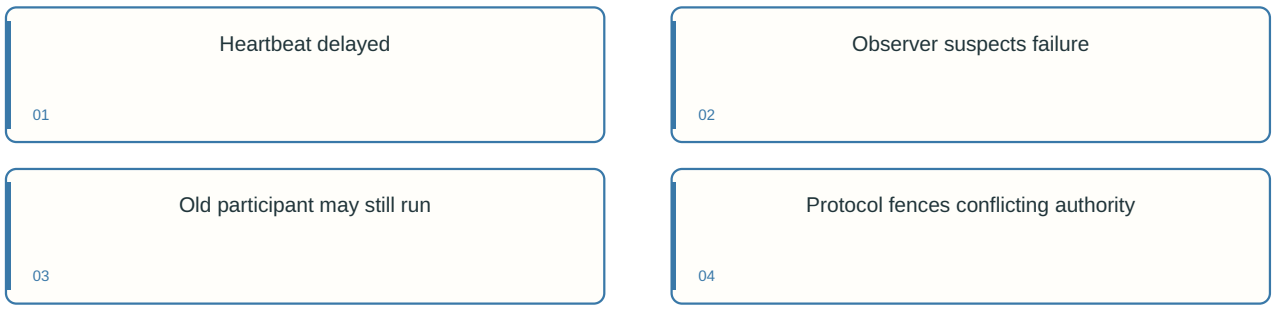


FIG. 14.1.3 **Network uncertainty · the example**

Network delay and failure can appear alike; timeouts bound waiting but cannot prove a remote process stopped.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.1.4 · Fault models

Suppose a replication protocol tolerates a machine that stops responding. Does that also mean it tolerates a machine that sends fabricated answers? These failures give the other participants very different problems. A fault model states which failures the design assumes can occur.

In a crash-stop model, a failed participant stops and does not return. A crash-recovery model allows it to restart. Recovery introduces another question: which information survives? A protocol may require the participant to reload a durable record of its earlier decisions before responding again. If that record disappears, the protocol's assumptions may no longer hold.

A Byzantine failure allows arbitrary behaviour, including sending different, contradictory histories to different recipients. A protocol designed only for crashes need not remain correct in this model. The network model matters too: messages may be delayed, lost, or cut off between groups of machines. A network partition is this loss of communication between otherwise potentially functioning groups.

State these conditions before choosing how many copies, or replicas, to keep. Three replicas sharing one storage device can all lose their data in one device failure. Separately placed replicas face different common causes. Replica counts become meaningful when paired with the failures, storage guarantees, and placement assumptions they are meant to cover.

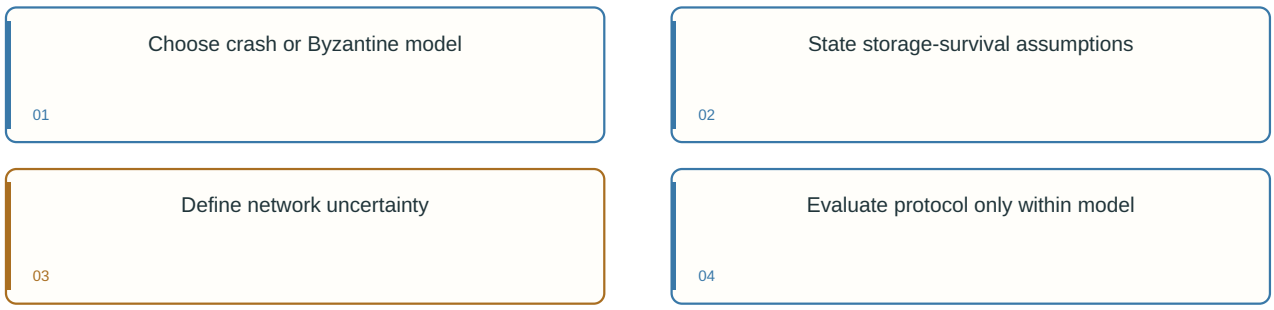


FIG. 14.1.4 **Fault models · the example**

A fault model specifies failures and persistence assumptions a protocol claims to tolerate.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.1.5 · Time, clocks, and event ordering

Machine A sends a message stamped 10:01. Machine B receives it when its own clock reads 10:00. The receive event must follow the send event, even though sorting these two timestamps would put them in the opposite order. The machines disagree about the time.

A wall clock gives a date and time of day. Its reading may differ from another machine's because the clocks run at slightly different rates, called drift, or because synchronization adjusts a reading. These readings are useful for user-visible dates and diagnosis, but unknown clock error limits the conclusions we can draw from them.

For event ordering, we can instead follow dependencies. An earlier event in one process precedes a later event in that process; sending a message precedes receiving it. We can also chain these relations: if a send precedes a receive, and that receive precedes another send, the first send precedes the second. This relation is called happens-before. Events with no such relation are concurrent in this model, even if their physical occurrence times differ.

Measuring a local request duration is another task. Use a suitable monotonic clock, whose reading does not move backward, rather than a wall clock that may be adjusted. Check its behaviour across suspension and restart. For expiry, state which authority judges the time. For a required shared order, use an ordering protocol or log. These choices answer different questions that one timestamp cannot settle by itself.

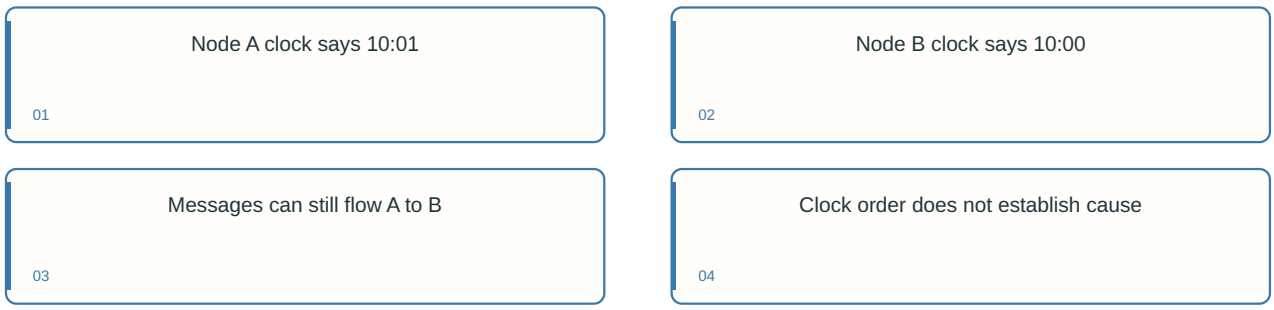


FIG. 14.1.5 **Time, clocks, and event ordering · the example**

Physical timestamps, elapsed durations, and causal order answer different questions and need suitable clocks or protocols.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.1.6 · Physical vs logical clocks

A post can carry both “published at 10:03” and “ordering counter 27.” The first field estimates when publication happened in real time. The second records progress according to an ordering rule. The numbers serve different purposes.

A physical clock attempts to measure real time, usually relative to a common time standard. Synchronizing physical clocks reduces their disagreement, but the remaining uncertainty matters. If two timestamps differ by less than the possible clock error, their numerical order may not establish which event occurred first. Subtracting timestamps recorded on different machines can also give a misleading duration.

A logical clock records ordering information. For example, a process can advance a counter whenever an event occurs and include that counter in outgoing messages. A receiving process then updates its own counter using a defined rule. The resulting values describe relationships among events without measuring elapsed seconds. Counter 27 followed by counter 28 does not mean that one second passed.

Use physical time to display dates, a suitable local clock to measure durations, and logical ordering where the protocol needs dependency information. Some schemes combine physical and logical values, but their interpretation depends on their particular update rules. Giving stored fields precise names and meanings helps prevent later code from treating an ordering counter as a time interval.

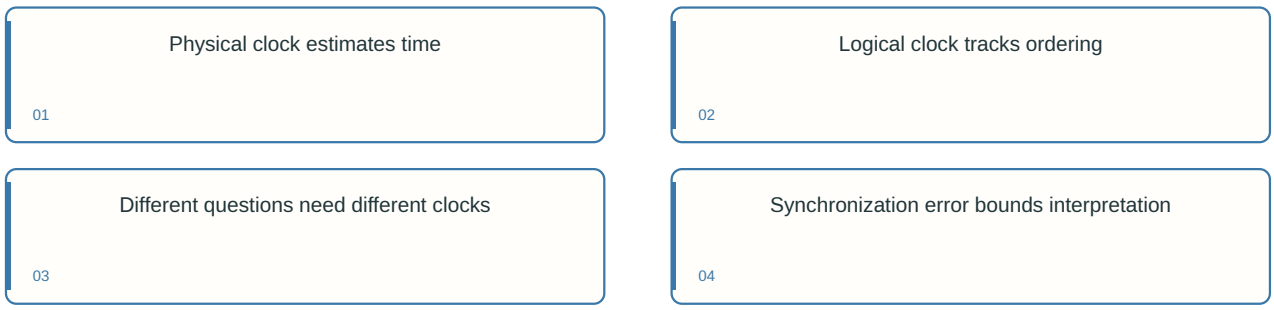


FIG. 14.1.6 **Physical vs logical clocks · the example**

Physical clocks estimate time; logical clocks represent ordering relationships without necessarily measuring seconds.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.1.7 · Lamport clocks and vector clocks

Let process A keep one integer counter. Before each local event, including sending a message, it increases the counter. A sent message carries that value. On receiving a message, a process sets its counter to one more than the larger of its current value and the message's value. These are Lamport clock rules.

If the receiver's value is 5 and the message carries 9, the new value is max(5,9)+1 = 10. Under these rules, an event that happens before another has a smaller Lamport value. The converse does not follow: two independent events can receive values 5 and 9 without either depending on the other. The counter preserves a known dependency but does not reveal every dependency from numerical comparison alone.

A vector clock keeps one counter for each participant in a specified participant set. With participants A and B, the first entry records A's progress and the second records B's. Each participant advances its own entry for an event. On receiving a vector, it combines the entries by taking each component's maximum and advances its own entry for the receive event.

Starting from [0,0], an independent event at A has [1,0], while an independent event at B has [0,1]. Compare corresponding entries. Neither vector is at least as large as the other everywhere, so these events are concurrent. If B then receives A's message, the componentwise merge is [1,1], and incrementing B's entry gives [1,2]. The earlier [1,0] is no greater in any entry and smaller in at least one, so it precedes [1,2]. The code example uses exactly these comparisons.

Vector metadata grows with the participant set, and changing that set needs care. Neither kind of clock chooses how two conflicting edits should be merged. They describe event relationships; a business rule and a storage protocol still have to decide which outcomes are valid.

Event or merge step	Vector	Relation
A only	[1,0]	not before B
B only	[0,1]	not before A
Merge before increment	[1,1]	dominates both

FIG. 14.1.7 **Vector comparison exposes concurrent events**

The componentwise maximum combines known progress. A receive event then advances the receiver's own entry; neither step chooses a business merge.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · During a partition, exclusive seat allocation cannot obtain valid authority. Which other operations could remain useful, and how should their promises and recovery differ?

A public event description can be served from a cached copy if its permitted age and access rules allow it. A feed read may also continue under an explicitly weaker freshness contract. These responses do not grant a seat, so they can have different serving rules from allocation. The allocation request must wait or fail explicitly where authority cannot be established. Make any stale view clear enough that it is not mistaken for a confirmed booking. When communication returns, reconcile pending allocations against the authoritative decision and resume eligible work without issuing a second winner.

Remember: A missing reply leaves a remote outcome uncertain. State the failures being covered, then choose clocks and message rules that support the ordering question you actually need to answer.

Try applying it: A leader misses five heartbeats and is replaced. It then resumes and sends a delayed write. Explain what the missing heartbeats established, what they did not establish, and which resource check could stop the earlier owner's write.

Unit 14 exercises → · Unit 14 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Jepsen: Consistency models ↗

   Primary practitioner research reference. Formal-history distinctions and references to underlying literature.

   Access reviewed · 21 October 2026

2. Time, Clocks, and the Ordering of Events in a Distributed System — Lamport, 1978 ↗

   Original research. Ordering reference for Unit 14.

   Access reviewed · 21 October 2026

3. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services — Gilbert and Lynch, 2002 ↗

   Original research · university-hosted paper. CAP assumptions and impossibility; PDF text extraction is imperfect, original page inspected.

   Access reviewed · 21 October 2026

Consistency as a reader-visible contract

Prerequisites: 14.1 · Partial failure & the meaning of time 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

You publish a post in one region and immediately read your feed from another. The post is missing. Later it appears, then disappears when another replica serves the next request. Each observation raises a different question about what the service promised. This chapter develops precise consistency rules by following short histories of writes and reads.

14.2.1 · Consistency models

A user publishes post P in one region. A friend reads from another region a moment later and does not see it. Is the system behaving correctly? We cannot answer until we know what the read operation promises.

A consistency model defines which histories of operations and observations are allowed. A history records actions such as invoking a write, receiving its success response, starting a read, and returning a value. By examining that order, we can ask whether a result satisfies the chosen model.

Write a small history before choosing a database setting. If publication has completed before the friend's read starts, must P appear? If the friend has already seen P, may a later read omit it? If someone writes a reply after reading P, may another reader see the reply before P? These are different requirements, and different models answer them differently.

Also state the scope: one key, several records in a transaction, one user's session, or the whole service. A delayed popularity count and an exclusive seat allocation can need different rules. Stronger ordering requirements can require communication before a response, and may limit which requests can complete during lost communication. Naming the permitted histories makes those costs easier to evaluate.

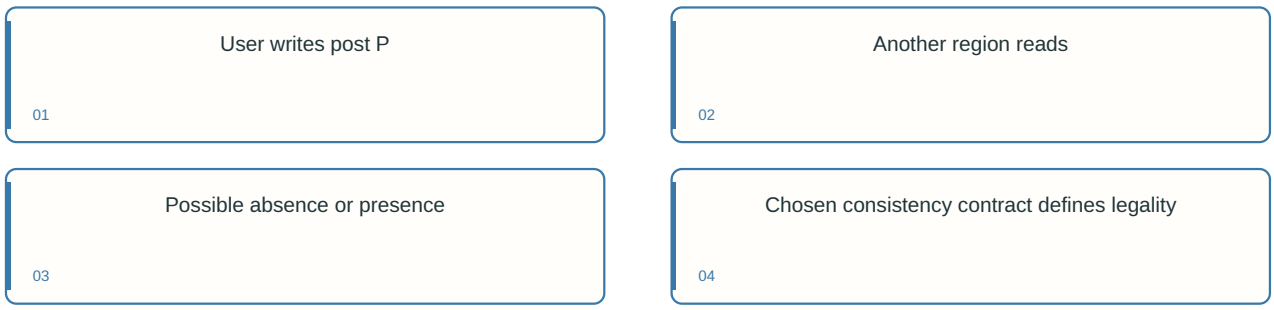


FIG. 14.2.1 **Consistency models · the example**

A consistency model defines permitted observations across operations, sessions, and state copies.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.2.2 · Strong consistency and eventual consistency

Suppose region A accepts a post while region B still has the earlier feed. Replication later carries the update to B. If updates stop, communication resumes, and the required repair continues successfully, the copies eventually reach an agreed result. This convergence is the central promise of eventual consistency.

The promise alone gives no maximum waiting time. It also does not choose what to do if both regions accepted different edits to the same record. A system needs a conflict rule, such as retaining both versions for resolution or applying an appropriate merge. Reaching one common value does not establish that the chosen value preserves everything the application needs.

“Strong consistency” is a less precise phrase. It often means linearizability, which we examine next, but speakers and products may use it differently. Ask which operations, observation rules, and failure conditions the claim actually covers.

We can combine requirements deliberately. A friend's feed may allow propagation delay while the author must see their own completed post immediately. That session requirement needs support beyond eventual convergence alone. Freshness and durability also need separate statements: taking time to propagate an update is different from losing an acknowledged update after a failure. Specify both how reads may lag and which failures committed data must survive.

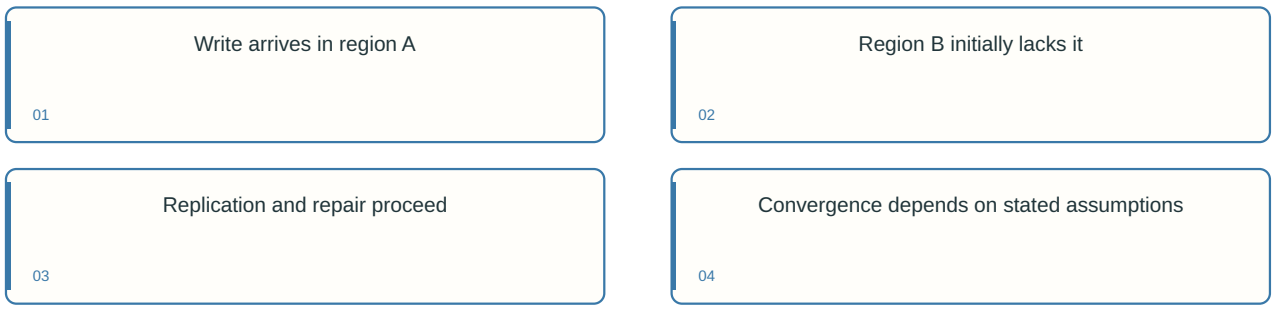


FIG. 14.2.2 **Strong consistency and eventual consistency · the example**

Eventual convergence needs stated repair assumptions and conflict semantics; “strong” needs a named precise model.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.2.3 · Linearizability

Consider a shared value initially equal to 1. Client A writes 2 and receives success. Client B then begins a read. If no other write intervenes, a linearizable implementation must return 2.

Linearizability means that every operation appears to take effect at one instant between its invocation and response, with the resulting order obeying the object's rules. It also preserves real-time precedence: when one operation finishes before another begins, the first must come earlier in that order. The imaginary instant is called its linearization point.

If a read overlaps the write of 2, either 1 or 2 may be valid, depending on where the operations can be placed. All operations must still fit one legal history. A read cannot observe 2 and then, with no intervening write, have a later read return 1.

A timed-out write has an unknown result and may still take effect. Linearizability does not turn lost responses into confirmed failures. Providing the property requires suitable write, read, and recovery rules; comparing timestamps or waiting for a majority of replies does not by itself establish those rules.

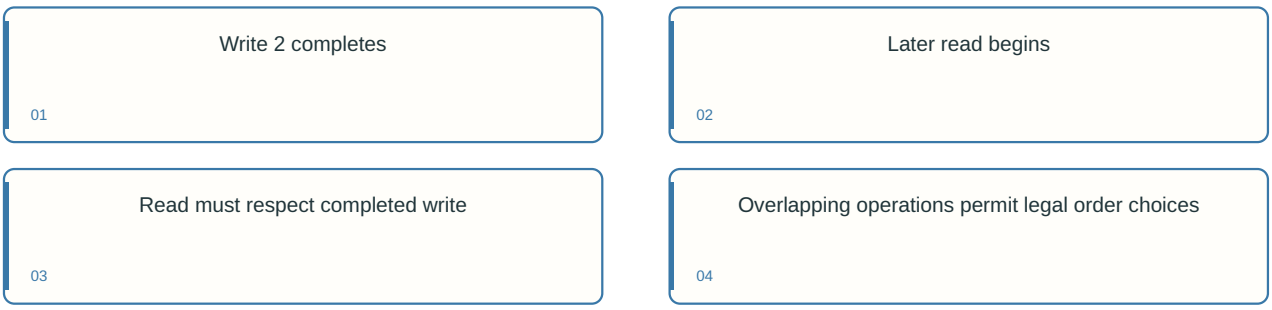


FIG. 14.2.3 **Linearizability · the example**

Linearizability gives operations atomic points between invocation and response while respecting real-time precedence.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.2.4 · Serializability vs linearizability

A bank transfer subtracts 10 from one account and adds 10 to another within a database transaction. Other transactions may run at the same time. Serializability requires their results to match some execution in which whole transactions ran one after another. It lets us reason about the transfer as one transaction rather than every possible interleaving of its individual statements.

This requirement does not, by itself, make the chosen serial order follow real time. Suppose transaction T1 writes x=2 and completes. A separate client later runs T2, which reads x=1. With initial x=1 and no other dependencies, this can fit the serial order T2 then T1. That order explains the values but reverses the observed completion-before-start order.

Strict serializability adds that real-time constraint to serializability. It requires T1 to precede T2 in this example, ruling out the old read. Linearizability applies the corresponding real-time reasoning to operations on an object; strict serializability applies it to transactions that may access several objects.

Check the actual transaction boundary. A local database transaction can group the account updates it controls. It cannot automatically include a payment provider's remote charge. Even with strict serializability inside the database, communication with that provider still needs rules for uncertain replies and recovery.

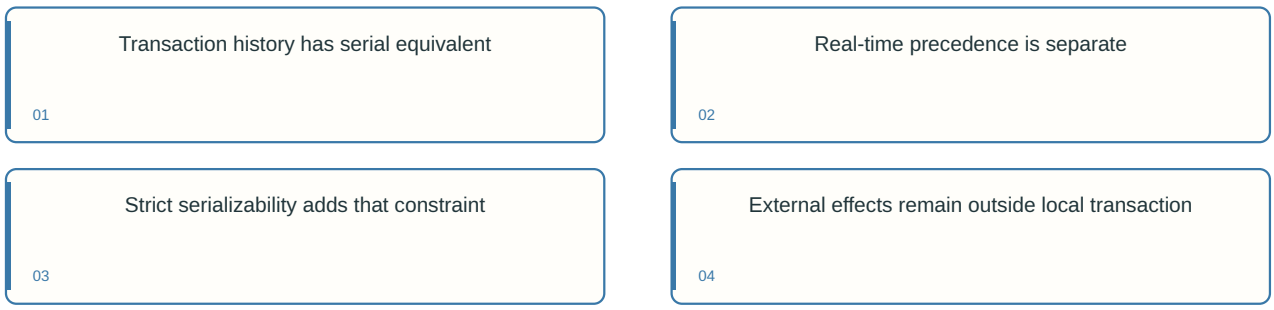


FIG. 14.2.4 **Serializability vs linearizability · the example**

Serializability concerns transaction equivalence; strict serializability additionally respects real-time constraints.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.2.5 · Sequential and causal consistency

Suppose process A writes x=1 and then x=2. Process B reads x=0 and then x=2, with x initially 0. One possible combined order is B's first read, A's first write, A's second write, and B's second read. It preserves the order of each process's own operations.

Sequential consistency requires one such common order that obeys the object's rules and each participant's program order. It does not require that order to preserve real-time precedence across different participants. Thus B's first read can appear before A's writes in the common order even if it began later in real time. Linearizability would add the missing real-time constraint.

Causal consistency focuses on dependencies. Suppose a user reads post P and then writes reply R to it. R depends on P. In an application that retains both and tracks this dependency, a view exposing R must also satisfy the visibility requirement for P. The replica may need to fetch P or delay showing R until its dependency is available.

Two unrelated posts have no such dependency. Causal consistency can allow readers to observe them in different orders, whereas sequential consistency requires a common order. Causal tracking alone also leaves conflict resolution to the specified system. Choose the relationships the application needs to preserve, and check that replication actually carries the information needed to preserve them.

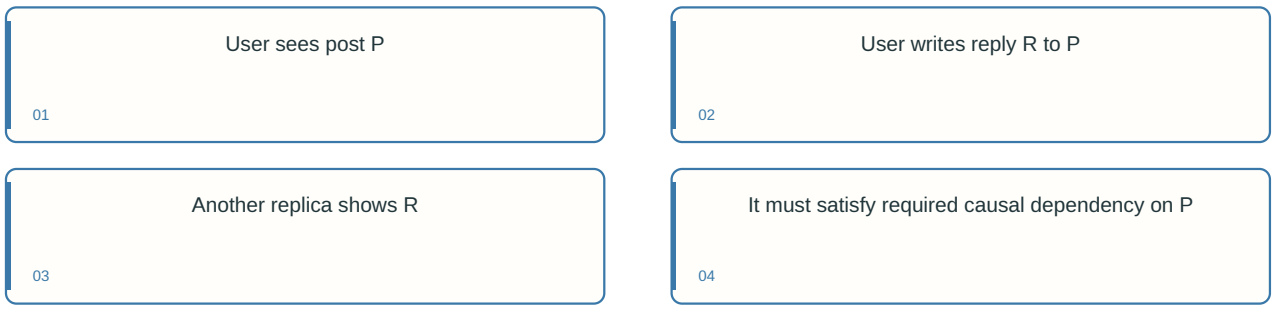


FIG. 14.2.5 **Sequential and causal consistency · the example**

Sequential consistency preserves participant program order; causal consistency preserves dependencies while allowing unrelated orders to differ.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.2.6 · Read-your-writes and monotonic reads

You change your profile name and receive success. Your next profile read should reflect that write, or an update that legitimately supersedes it. This is read-your-writes: a guarantee about what your subsequent reads observe within the supported session.

Now suppose you have seen profile version 10. A later read takes you to a replica containing only version 9. Returning version 9 makes your view move backward. Monotonic reads rule out this loss of previously observed progress. Monotonic refers to the progress of versions, not the numerical value of a field: a later, valid edit may change an age from 30 to 29.

One implementation can attach a required replication position to the session. When a read reaches another replica, that replica checks whether it has reached the required position. If it has, it can serve the read. Otherwise, it must catch up, route the request to a sufficiently current owner, or report that it cannot currently provide the promised result.

These are session guarantees, so another user's session may still see older data. They do not alone provide global linearizability. Define how the session's progress survives failover or follows the user to another device. Sending a user repeatedly to one replica can help in ordinary operation, but that routing choice alone says nothing about what happens when the replica is replaced.

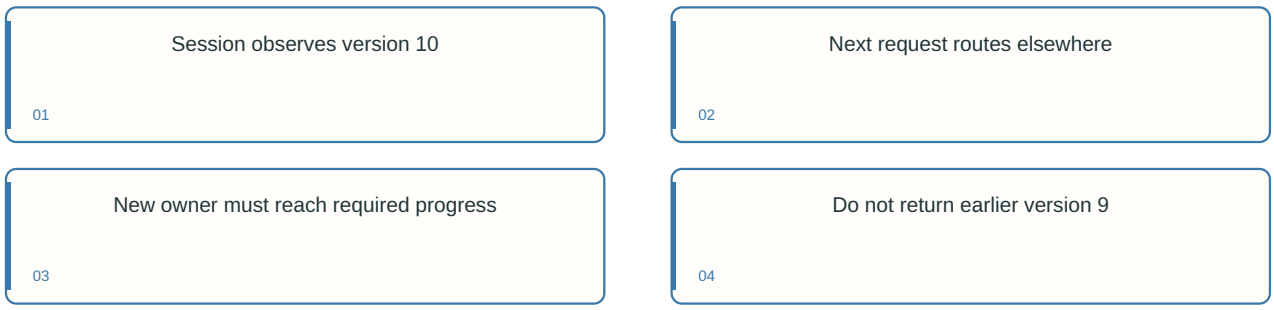


FIG. 14.2.6 **Read-your-writes and monotonic reads · the example**

Read-your-writes and monotonic reads preserve particular session observations across supported routing and recovery paths.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · During a partition, exclusive seat allocation cannot obtain valid authority. Which other operations could remain useful, and how should their promises and recovery differ?

A public event description can be served from a cached copy if its permitted age and access rules allow it. A feed read may also continue under an explicitly weaker freshness contract. These responses do not grant a seat, so they can have different serving rules from allocation. The allocation request must wait or fail explicitly where authority cannot be established. Make any stale view clear enough that it is not mistaken for a confirmed booking. When communication returns, reconcile pending allocations against the authoritative decision and resume eligible work without issuing a second winner.

Remember: A consistency model specifies permitted histories. Identify the operation and scope, then check completed writes, overlapping operations, dependencies, and session progress against that promise.

Try applying it: An author publishes P and sees it, then reads from a replica that omits P. A friend who has never seen P also reads from that replica. Which result violates a promised session guarantee, and what additional rule would make the friend's later read illegal too?

Unit 14 exercises → · Unit 14 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Jepsen: Consistency models ↗

   Primary practitioner research reference. Formal-history distinctions and references to underlying literature.

   Access reviewed · 21 October 2026

2. Time, Clocks, and the Ordering of Events in a Distributed System — Lamport, 1978 ↗

   Original research. Ordering reference for Unit 14.

   Access reviewed · 21 October 2026

3. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services — Gilbert and Lynch, 2002 ↗

   Original research · university-hosted paper. CAP assumptions and impossibility; PDF text extraction is imperfect, original page inspected.

   Access reviewed · 21 October 2026

Partitions, CAP, PACELC & quorums

Prerequisites: 14.2 · Consistency as a reader-visible contract 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Three replicas are running, but a network fault leaves one unable to talk to the other two. A customer can still reach each side. We will examine which requests can complete, what quorum overlap tells us, and how a replacement leader prevents an earlier owner from writing when it resumes.

14.3.1 · CAP theorem, with its actual assumptions

Two replicas store x=1. A network partition cuts communication between them while both remain running. Suppose A accepts x=2 and acknowledges it. A client then asks B to read x. B cannot learn whether A accepted 2, 3, or no write at all.

CAP formalizes this difficulty for an atomic, or linearizable, read/write object in a model that permits communication failures. Its availability requirement says every request reaching a nonfailed participant must eventually complete with the operation's response. It is stronger than a measured uptime percentage. A partition may prevent messages between groups from arriving indefinitely.

B cannot guarantee the correct latest value from its local information. Waiting indefinitely sacrifices availability; answering may violate atomic consistency. Refusing the operation also gives up the required availability for that request. Thus the protocol cannot guarantee both properties in all executions allowed by this model.

Use this result to specify partition behaviour. Exclusive allocation may wait or reject where authority cannot be established. A feed read may serve an explicitly weaker view. CAP does not supply a permanent “choose any two” product classification or determine ordinary-operation latency.

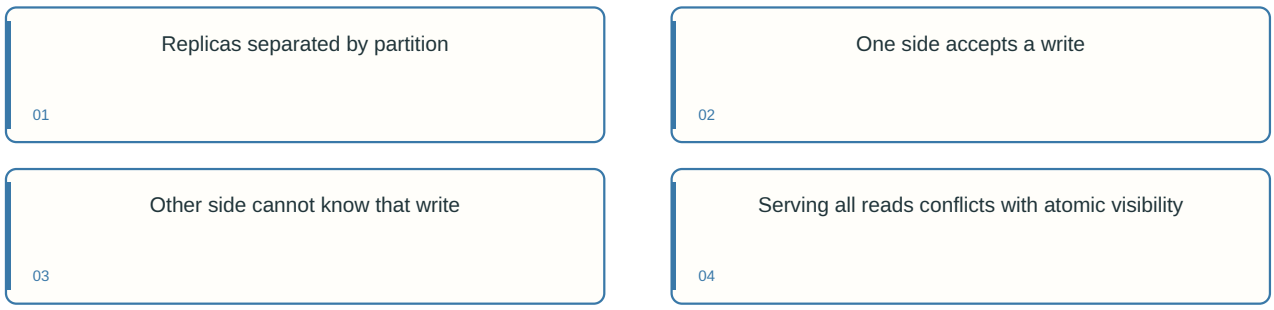


FIG. 14.3.1 **CAP theorem, with its actual assumptions · the example**

CAP concerns the incompatibility of defined atomic consistency and availability guarantees under its partition assumptions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.3.2 · PACELC as a useful extension

Even when both regions are healthy, a message takes time to travel between them. Suppose a write protocol waits for acknowledgment from a distant replica before reporting success. That communication lies on the response path, so the client must wait for it.

PACELC names two design questions. If there is a Partition, how does the system trade Availability against Consistency? Else, while communication is functioning, how does it trade Latency against Consistency? The letters help us remember to examine both failure behaviour and everyday response time.

For example, replying after a local commit and copying the update later can shorten the write path. A nearby replica may then serve an older value. Requiring a coordinated write or routing the read to a current owner can preserve a stronger observation contract, with additional communication on one of those paths. The exact effect depends on the protocol; merely waiting for any remote acknowledgment is insufficient.

Review each important operation and configuration. The same service may coordinate allocations while allowing delayed feed reads. Measure the distances, waits, and workload with stated assumptions. PACELC helps expose a trade-off to investigate; its letters do not establish a protocol's correctness or predict a latency number.

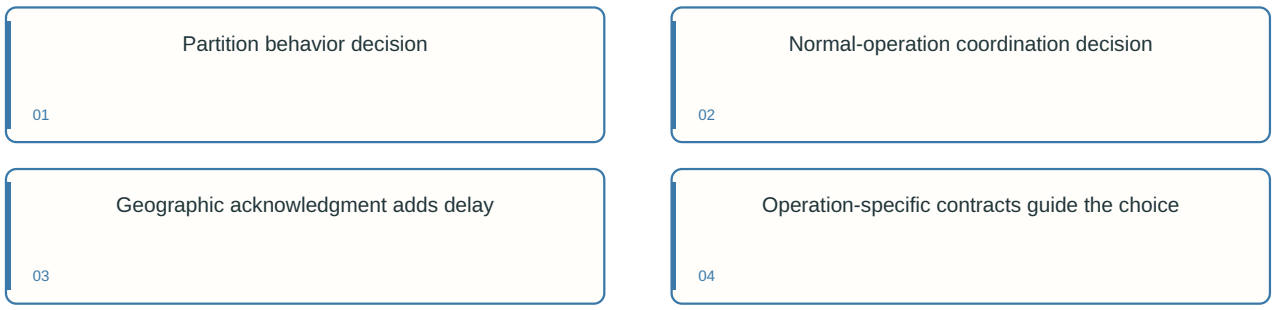


FIG. 14.3.2 **PACELC as a useful extension · the example**

PACELC highlights partition choices and ordinary latency/consistency costs as a design lens, not a complete protocol.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.3.3 · Quorums and read/write trade-offs

Keep a fixed set of three replicas: A, B, and C. Suppose a write waits for acknowledgments from A and B, and a later read asks B and C. B belongs to both sets. More generally, any two groups of two chosen from these three replicas must overlap.

A quorum is a group large enough to satisfy a protocol's participation rule. Let N be the number of replicas in the fixed membership, W the number required for a write, and R the number consulted for a read. If R+W>N, the two sets cannot be disjoint: there are too few replicas to fill both sets separately. If W>N/2, any two write quorums also intersect. For N=3, R=2, W=2, both conditions hold.

This gives the reader access to at least one participant in a completed write's set. It still needs to interpret that participant's evidence correctly. Did an acknowledgment mean the data was durably stored? How are two writers' versions ordered? Can a reader distinguish an accepted version from an unfinished attempt?

Consider an unfinished write that reaches only A. A naive read of A and C returns that new value. A later read of B and C returns the old value because neither has the unfinished write. Both reads contacted two replicas, but the value moved backward. A linearizable protocol needs additional steps or rules to prevent such a history; quorum arithmetic alone does not do so.

Increasing W can reduce the size R needed for intersection, but makes writes wait for more participants. A quorum operation also needs enough reachable participants to finish. Membership changes and temporary substitute replicas require their own rules: the fixed-set proof cannot simply be reused when the sets no longer belong to the same N replicas.

Set	Members	Intersection
Write W=2	A, B	B
Read R=2	B, C	B
Fixed N=3	R + W = 4 > 3	one shared member

FIG. 14.3.3 **Intersecting quorums still need a protocol**

Intersection preserves a possible witness. Version, membership, persistence, and concurrency rules determine how that witness is used.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.3.4 · Split-brain situations

Leader A pauses while holding responsibility for writes. Other replicas stop hearing from it and elect B. Later A resumes, still carrying its earlier view of ownership, and sends a write. If the resource accepts both A and B as authorities, the system has split brain.

Electing B did not physically stop A. Nor does a lock's expiry force a paused process to stop executing when it resumes. Correct ownership therefore needs enforcement at the place where protected effects are accepted.

One approach uses a fencing epoch: an increasing number identifying a period of authority. Suppose A holds epoch 7 and B receives epoch 8 from the ownership protocol. A storage receiver records the highest accepted epoch and atomically checks that number with each protected write. Once it has accepted 8, it rejects a write carrying 7, including a delayed request from A.

The transition must establish the required fence before relying on the new owner's exclusive effects. The receiver must retain and enforce the authority information according to the recovery contract; an unchecked number in a request provides no protection. Test the sequence “pause, replace, resume,” together with delayed messages. It exposes a competing-owner history that a clean crash test never exercises.

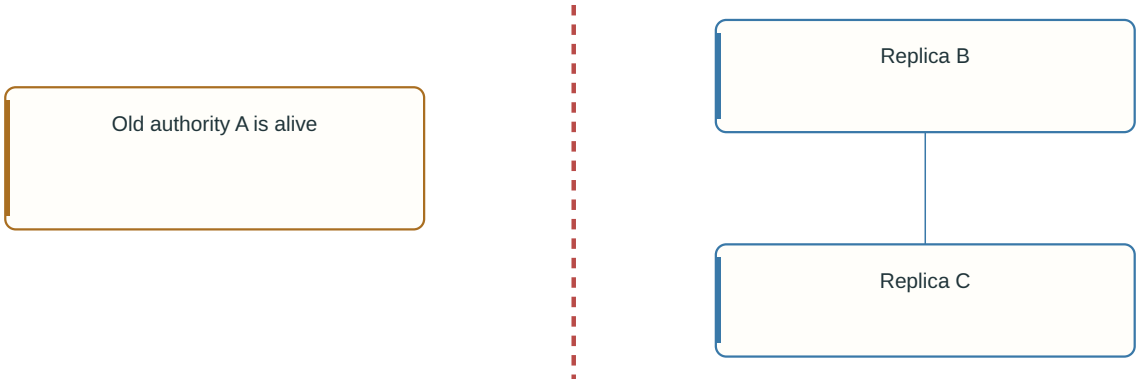


FIG. 14.3.4 **A partition can leave two live but disconnected sides**

Region A remains running while B and C form another reachable side. Correct authority and fencing determine who may make which writes; a missing heartbeat cannot stop A.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · During a partition, exclusive seat allocation cannot obtain valid authority. Which other operations could remain useful, and how should their promises and recovery differ?

A public event description can be served from a cached copy if its permitted age and access rules allow it. A feed read may also continue under an explicitly weaker freshness contract. These responses do not grant a seat, so they can have different serving rules from allocation. The allocation request must wait or fail explicitly where authority cannot be established. Make any stale view clear enough that it is not mistaken for a confirmed booking. When communication returns, reconcile pending allocations against the authoritative decision and resume eligible work without issuing a second winner.

Remember: During lost communication, an operation may need to wait to preserve its promised result. Quorum overlap provides shared participants; version rules, durable evidence, and enforced authority determine what those participants can safely establish.

Try applying it: A pending write reaches only A. A read of A and C returns it; a later read of B and C returns the old value. Both reads used two of three replicas. Explain why this history can violate linearizability despite quorum overlap, and identify what the read protocol must prevent.

Unit 14 exercises → · Unit 14 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Jepsen: Consistency models ↗

   Primary practitioner research reference. Formal-history distinctions and references to underlying literature.

   Access reviewed · 21 October 2026

2. Time, Clocks, and the Ordering of Events in a Distributed System — Lamport, 1978 ↗

   Original research. Ordering reference for Unit 14.

   Access reviewed · 21 October 2026

3. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services — Gilbert and Lynch, 2002 ↗

   Original research · university-hosted paper. CAP assumptions and impossibility; PDF text extraction is imperfect, original page inspected.

   Access reviewed · 21 October 2026

Coordination, safety & liveness

Prerequisites: 14.3 · Partitions, CAP, PACELC & quorums 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Two workers both believe they should own the same partition. Stopping both would avoid conflicting writes, but useful work would stop too. We will connect shared decisions to their enforcement, examine assumptions hidden in remote calls, and distinguish preventing bad outcomes from making progress under stated conditions.

14.4.1 · Distributed coordination

Suppose several workers need to agree which one owns a data partition. Each can read a configuration file, but independently deciding “the old owner seems absent, so I am now the owner” can produce two owners. They need a shared decision with rules for concurrent proposals and failures.

Distributed coordination establishes decisions of this kind. Consensus is the agreement problem underlying many coordination services: participants must not choose conflicting decisions, and a chosen decision must satisfy the protocol's validity rules. A replicated decision log can record an owner and its epoch so participants recover the same authoritative history.

Follow the complete path. The coordination protocol records worker B as owner for a new epoch. B obtains that decision and presents its epoch when acting. The protected resource checks the authority before accepting an effect. Agreement about ownership and enforcement of ownership are separate steps; both are needed.

Coordination also has a progress cost. If participants cannot exchange the messages required for a decision, a safe protocol may be unable to issue new ownership. Define what clients can still do: reading a previously approved, sufficiently fresh configuration may be acceptable, while inventing another exclusive owner is not.

Use shared decisions where an invariant requires them. Two independent updates commute if applying them in either order gives the same result. Where the domain and protocol safely permit such updates, every update may not need one common ordering decision. The choice follows from the state rule being protected, rather than from a general preference for more or less coordination.

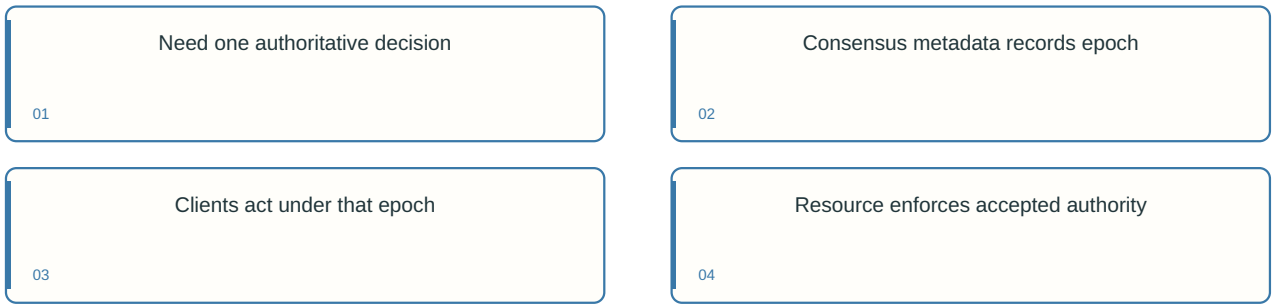


FIG. 14.4.1 **Distributed coordination · the example**

Coordination establishes shared decisions under an explicit failure and progress protocol.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.4.2 · The fallacies of distributed computing

Imagine replacing a local function call with a call to another team's service. The function used to run in the same process. The new call needs a connection, transfers bytes, may wait in another queue, and can lose its reply after the remote work succeeds. Keeping the old assumptions would hide much of the new operation.

The fallacies of distributed computing name common mistaken assumptions: the network is reliable and secure, latency is zero, bandwidth is unlimited, topology stays fixed, there is one administrator, transport costs nothing, and the network is homogeneous. Here topology means which participants and routes are connected; homogeneous means that the communicating environment behaves uniformly.

Turn the list into questions about this call. How long will the caller wait? What happens after an uncertain result? How is the peer authenticated and the action authorized? Can old and new message versions communicate during a rollout? If the caller requests fifty responses instead of one, how much data and connection capacity does it consume?

Also identify who operates each side and who can change it. A remote team's deployment, certificate rotation, or address change may affect the caller without a local code change. The list provides useful review prompts. The answers still need concrete contracts, resource limits, and recovery rules in the actual design.

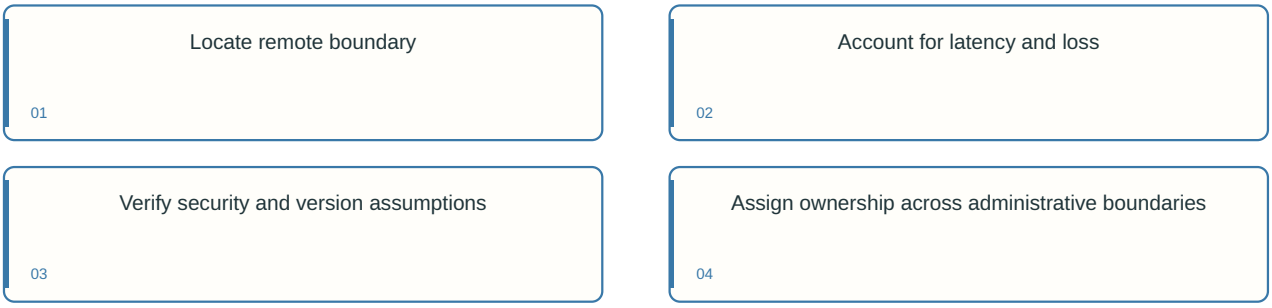


FIG. 14.4.2 **The fallacies of distributed computing · the example**

Distributed-computing fallacies expose hidden assumptions about networks, cost, topology, trust, and administration.

Author-created illustration. Numbers are illustrative unless stated otherwise.

14.4.3 · Safety vs liveness

A booking service promises that one seat will never be assigned to two buyers. If a history ever contains two incompatible successful assignments, the promise is broken. This is a safety property: it excludes a bad outcome at every point in execution.

The service also promises that every accepted booking eventually reaches a recorded outcome, such as confirmed or rejected. This is a liveness property: it requires eventual progress. A booking that remains pending forever can satisfy the no-double-sale rule while failing this progress requirement. A timeout on the customer's screen does not resolve the accepted booking's stored state.

Progress needs assumptions about the environment. We might require that enough participants eventually recover, communication becomes sufficiently timely, and pending work continues to be scheduled. While a partition prevents establishing authority, the service may preserve safety by waiting. It should then resume or recover the operation when the required conditions return. “Eventually” alone supplies no response-time bound; a deadline objective is an additional requirement.

Rejecting every request can also preserve safety while making the service useless. It even satisfies “every accepted booking resolves” when nothing is accepted. To rule this out, state useful-service requirements as well: for example, valid requests for available seats should complete when the required participants are healthy and reachable.

Monitor old pending work and provide a recovery procedure. Operators need a way to discover the existing decision and advance the operation without creating a second owner. A useful design explains both which outcomes must never occur and under which conditions work must move forward.

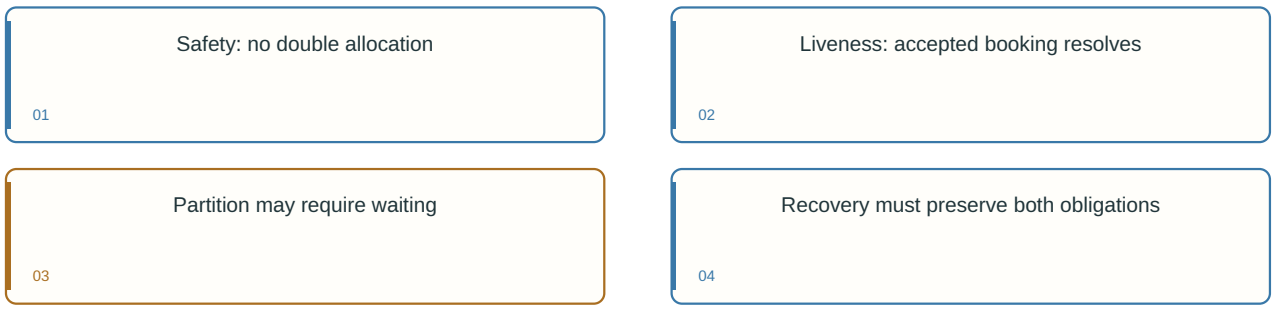


FIG. 14.4.3 **Safety vs liveness · the example**

Safety excludes bad outcomes; liveness requires progress under specified environmental conditions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

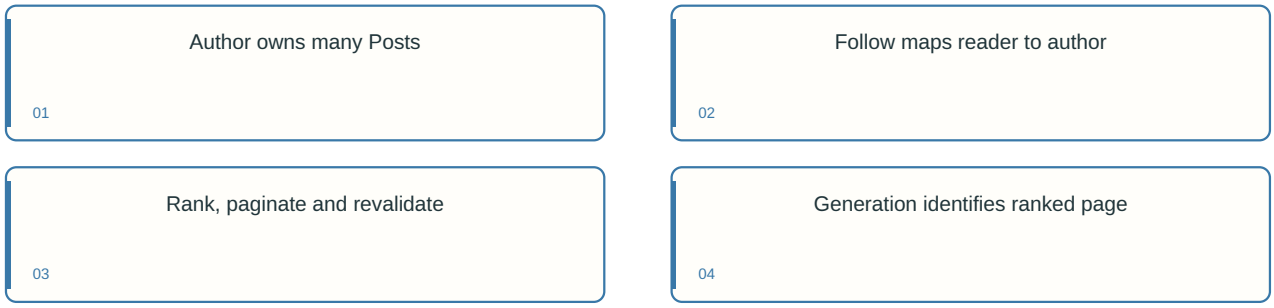


FIG. 14.4.4 **State and ownership in the related design**

The post and access relationships are authoritative; feed candidates are derived references.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · ORDERING MODELS

```
from mechanisms import vector_relation, lamport_receive
assert vector_relation([1,0], [0,1]) == "concurrent"
assert vector_relation([1,0], [1,2]) == "before"
assert lamport_receive(5, 9) == 10
```

The code first compares [1,0] with [0,1]. The first vector is ahead in A's entry and behind in B's entry. Neither contains all the progress recorded by the other, so vector_relation returns "concurrent". The first

assertion checks that result.

Next it compares [1,0] with [1,2]. The A entries are equal, and the B entry has advanced from 0 to 2. Every component of the first vector is no greater, with one strictly smaller, so the function returns "before". Finally, lamport_receive(5,9) takes the larger value, 9, and adds one, giving 10.

These assertions check ordering calculations for the supplied values and a common participant set. They do not simulate message delivery, persistence, or agreement among replicas. A smaller Lamport number alone does not prove a dependency, and a vector relation does not choose how conflicting account edits should be merged. Those behaviours need their own application and protocol rules.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · TRY CHANGING THE INPUTS

When must a read set meet a write set?

Write set example: A, B. Read set example: B, C. Shared members: B.

Every such fixed-set read/write pair must intersect. R + W = 4; N = 3.

A shared replica gives the read a place to encounter a written version. Version selection and membership rules still determine whether the whole protocol is linearizable.

Keep the same N replicas, then vary the number read and written. The calculation shows whether the sets must share a member. To decide which value a read can return, we also need rules for versions, concurrent writes, persistence, repair, and membership changes. Intersection alone does not prove linearizability.

▼ Think further · During a partition, exclusive seat allocation cannot obtain valid authority. Which other operations could remain useful, and how should their promises and recovery differ?

A public event description can be served from a cached copy if its permitted age and access rules allow it. A feed read may also continue under an explicitly weaker freshness contract. These responses do not grant a seat, so they can have different serving rules from allocation. The allocation request must wait or fail explicitly where authority cannot be established. Make any stale view clear enough that it is not mistaken for a confirmed booking. When communication returns, reconcile pending allocations against the authoritative decision and resume eligible work without issuing a second winner.

Remember: Safety excludes forbidden outcomes. Liveness requires progress under stated conditions. Shared decisions, resource checks, and recovery procedures must work together to provide useful service.

Try applying it: A booking service never assigns a seat twice, but some accepted bookings remain pending forever. Another version rejects every request immediately. Which stated progress requirement does the first version fail, and what additional requirement is needed to rule out the second?

Unit 14 exercises → · Unit 14 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Jepsen: Consistency models ↗

   Primary practitioner research reference. Formal-history distinctions and references to underlying literature.

   Access reviewed · 21 October 2026

2. Time, Clocks, and the Ordering of Events in a Distributed System — Lamport, 1978 ↗

   Original research. Ordering reference for Unit 14.

   Access reviewed · 21 October 2026

3. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services — Gilbert and Lynch, 2002 ↗

   Original research · university-hosted paper. CAP assumptions and impossibility; PDF text extraction is imperfect, original page inspected.

   Access reviewed · 21 October 2026

PART 4 / UNIT 15

Replication, Leader Election, and Consensus

Our task database now has copies on three machines. The primary reports that a task is saved, then stops before one copy has caught up. Can that copy take over without losing the task?

The running example

Three database replicas, one leader, and a network partition. Walk through elections, writes, acknowledgement rules, recovery, and potential data loss.

Unit 15 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

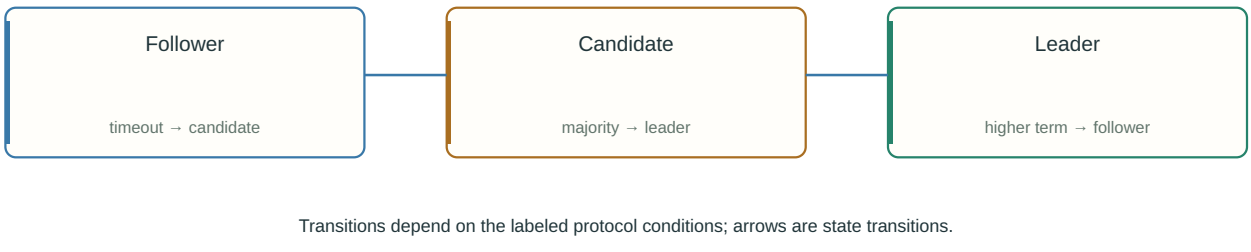
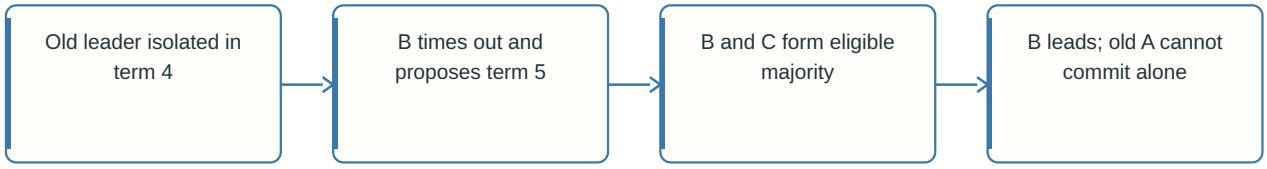


FIG. 15.0.1 **Raft election changes authority under protocol rules**

Election timeout can make a follower a candidate; a valid majority can elect an eligible leader. Observing a higher term returns a participant to follower. This is the role state machine, not the complete protocol.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 15.0.2 **election timeline**

The example assumes correct persistent voting/log rules and fixed non-Byzantine membership.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Participant	Durable log suffix	Meaning
Leader term 5	8:t4, 9:t5	current-term entry 9
Follower A	8:t4, 9:t5	required durable replica
Follower B	8:t4	lagging replica

FIG. 15.0.3 **Presence in a log is not the same as commitment**

Within the stated fixed-membership Raft protocol, majority replication of the current-term entry can advance commit and protect the prefix. This table is not a generic quorum proof.

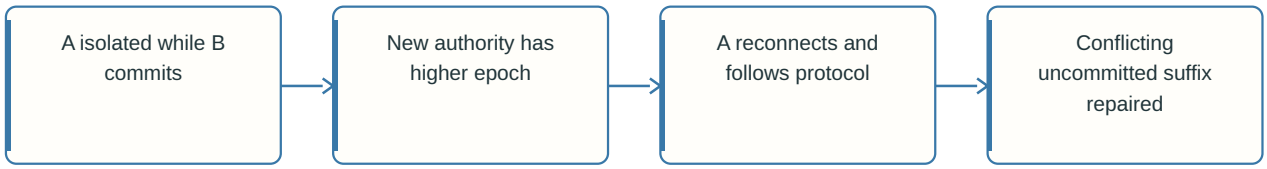
Author-created illustration. Numbers are illustrative unless stated otherwise.

Majority	Members	Shared witness
Election quorum 1	A, B	B
Election quorum 2	B, C	B
Required rules	Voting and durable log evidence	Intersection alone is insufficient

FIG. 15.0.4 **quorum intersection**

Safety comes from protocol rules applied to intersecting decision sets.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 15.0.5 **network partition recovery**

Recovery does not authorize an old leader to resume independent writes or ignore current term and committed history.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Choose a replication model

Prerequisites: Distributed Systems Fundamentals 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Our task database now has copies on three machines. The primary reports that a task is saved, then stops before one copy has caught up. Can that copy take over without losing the task?

We will compare who may accept writes, what a replica's acknowledgment means, and how several replicas combine evidence. These choices explain what replication protects and what still needs a protocol.

15.1.1 · Primary/replica architectures

Suppose our task database runs on machine A, with copies on B and C. A accepts a new task and sends the change to the other two machines. These copies can help us recover if A stops, and they may also serve reads. Keeping copies of a dataset on several participants is called replication. In a primary/replica architecture, the primary is the participant currently authorised to accept writes for that dataset.

Now let A disappear just after telling the user “Saved.” If B has not received the task, promoting B may make a saved task disappear from the user's view. If A later returns and still accepts writes, there may be two competing histories. The arrangement of three machines has not, by itself, settled either problem.

We need a contract for acknowledgment: what must happen before a write receives success? We also need rules for promotion, meaning the transfer of write authority to another replica. These rules must preserve the history the contract promises, restrict which replicas are eligible to become primary, and prevent the old primary from making unauthorised changes. A read from a lagging replica may return an earlier state even while the primary is healthy. Replica count is useful information, but persistence, ownership, and recovery rules determine what those copies let us promise.

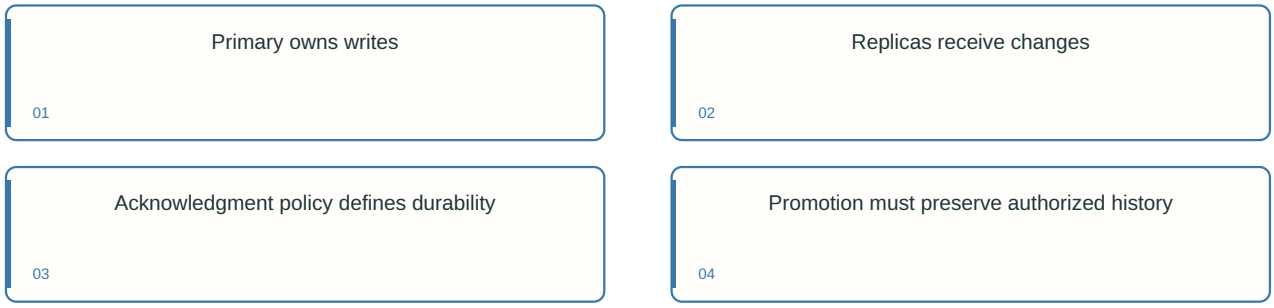


FIG. 15.1.1 **Primary/replica architectures · the example**

Primary/replica topology needs explicit write authority, acknowledgment, promotion, and stale-owner protection.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.1.2 · Leader/follower replication

Let the primary record “create task 7” followed by “complete task 7.” A follower must reproduce this order; applying completion before creation would not have the same meaning. In leader/follower replication, one leader sends an ordered stream of records to followers, which reproduce the changes.

Follow one record through a follower. First, the follower receives its bytes into memory. Next, it writes them to the persistent storage required by the protocol. Later, it applies the record to its task collection. Receipt, durable storage, and application are different milestones. A crash after receipt can lose a memory-only record. A follower that has stored a record but has not applied it can still serve an older task list.

Replication lag describes how far a follower trails the relevant leader position. The relevant position depends on the question: stored records matter for recovery, while applied records matter for what a local read can show. Some protocols also distinguish records that are present from records that have been committed, or accepted into the authoritative history. We will examine that distinction with Raft.

When comparing implementations, ask which milestone a follower acknowledges, what reads it may serve, and which history it may continue after promotion. Saying that a follower “has the data” leaves these questions unanswered.

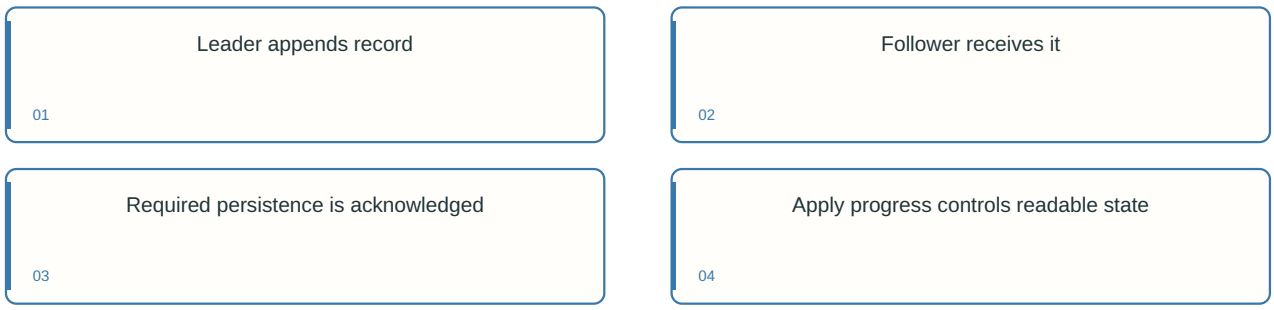


FIG. 15.1.2 **Leader/follower replication · the example**

Leader/follower replication orders updates, while receipt, persistence, commitment, and application can mark different progress.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.1.3 · Multi-leader replication

Imagine two regions, each with a writable copy of a task. Their connection fails. In region A, a user changes its title to “Read chapter.” In region B, another user changes the same title to “Review notes.” Neither region has seen the other change. When communication returns, both versions must be handled.

Multi-leader replication allows more than one leader to accept writes for the same replicated data. It can make local writes possible during a regional disconnection, but the system must define the result of concurrent changes: changes made without either side having observed the other. One option keeps both versions for review. Another merges changes where the application's data model permits it.

A last-writer-wins rule selects one version using an ordering such as a timestamp with a defined tie-breaker. This can make replicas converge, meaning they eventually agree, while still discarding a meaningful edit. A clock ahead of the others may make an earlier real-world edit appear later. The rule therefore needs a business justification as well as an unambiguous ordering.

Also specify how deletions propagate and how a record already replicated through one leader is recognised, so it does not circulate as a new update indefinitely. Multiple writers are suitable when the conflict rules fit the application. Exclusive allocation, such as selling the final item, may still require coordination.

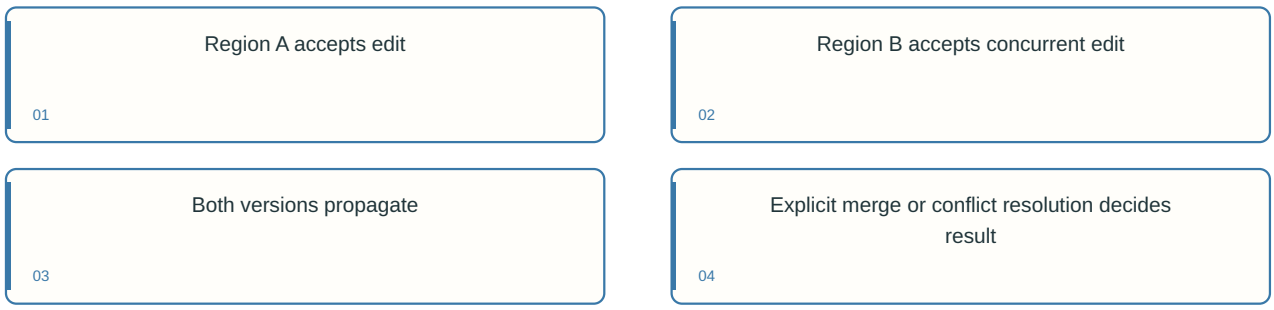


FIG. 15.1.3 **Multi-leader replication · the example**

Multiple write leaders need valid conflict semantics, not just deterministic convergence that discards business meaning.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.1.4 · Leaderless replication

Suppose a coordinator sends a task update to several replicas and waits for the responses required by its write rule. A later read asks several replicas for their versions. One may have an earlier value; others may report the update. The read protocol must interpret these versions and decide which result to return.

Leaderless replication uses an arrangement of this kind instead of routing every write through one permanent leader. A coordinator handles a particular request, but that role need not give it permanent authority over the dataset. The absence of one permanent leader does not remove the need for rules about concurrent updates, membership, durable acknowledgments, or version comparison.

Replicas also need ways to repair missing data. Read repair updates stale replicas discovered during a read. Hinted handoff temporarily keeps information intended for an unavailable replica and forwards it when possible. Background anti-entropy compares replicas and repairs differences outside individual requests. Each mechanism has its own scope; a stored hint is not automatically equivalent to acknowledgment by the original replica.

Trace two concurrent writes and the return of a long-absent replica when evaluating such a system. Two responses out of three provide a count. Whether they establish a fresh, linearizable result depends on the full protocol, including how it handles versions and overlapping requests.

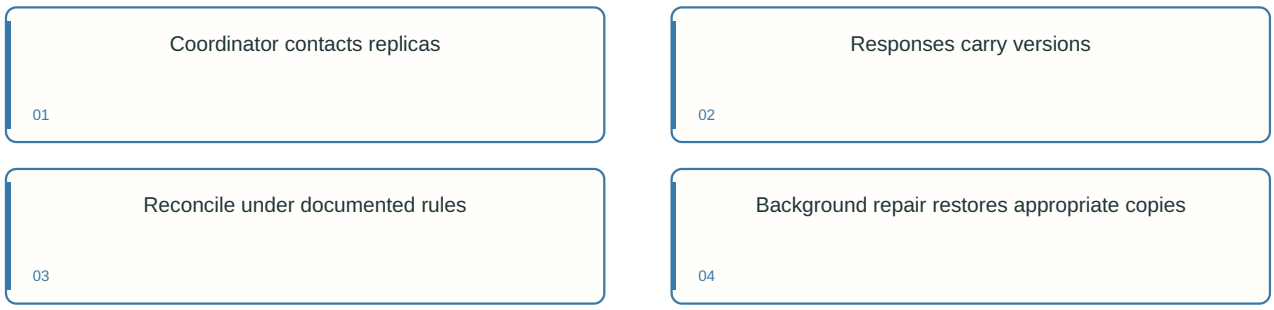


FIG. 15.1.4 **Leaderless replication · the example**

Leaderless protocols reconcile several replicas with implementation-specific version, repair, and membership rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.1.5 · Synchronous vs asynchronous replication

A primary stores a task and immediately reports success. Its replica will receive the update later. This is asynchronous replication: the caller's success does not wait for the specified replica work. If the primary is lost before another eligible replica durably stores the task, failover can lose an acknowledged write.

Now change the order. The primary sends the record, waits for the required replica acknowledgment, and then reports success. This is synchronous replication. We still need to ask what that acknowledgment means. “Received in memory,” “stored durably,” and “applied to readable state” are distinct promises. Waiting for receipt alone does not provide the same protection against a replica process restart as waiting for durable storage.

The required replicas matter too. Two durable copies on independent machines protect against a different set of failures from copies that share the same failed storage. A promotion rule must continue a history compatible with the acknowledgment contract; promoting any reachable replica can defeat the intended protection.

Waiting for remote work adds latency. If a required replica cannot communicate, writes may have to wait or fail rather than receive success. Choose this boundary from the application's data-loss and availability requirements, then test the same boundary through failover. The words “synchronous” and “asynchronous” describe the waiting arrangement; the stated milestones determine its meaning.

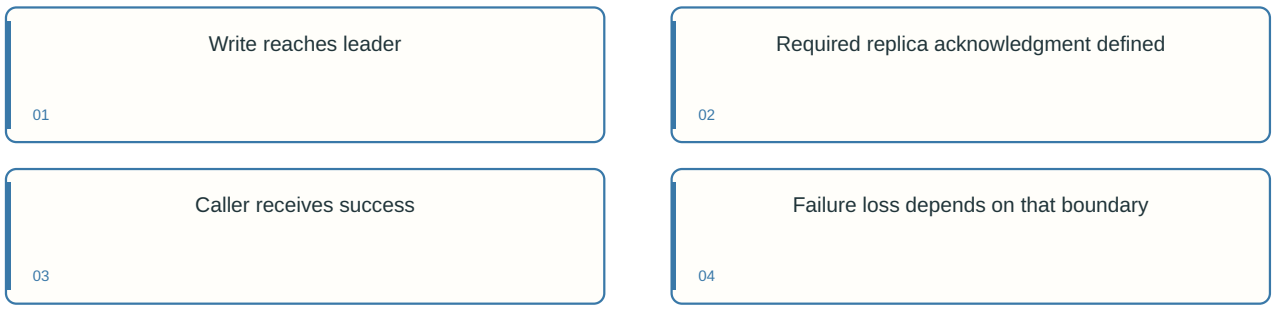


FIG. 15.1.5 **Synchronous vs asynchronous replication · the example**

Synchronous replication waits for named milestones; asynchronous acknowledgment can precede another durable copy.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.1.6 · Quorum-based systems

In a fixed group of three voters A, B, and C, a majority contains two members. The group A+B overlaps B+C at B. Any two majorities in this group share at least one member. This intersection gives a later decision a possible witness to an earlier one.

A quorum is a set of participants whose responses satisfy a protocol's decision rule. The shared participant is useful only if it remembers the required evidence and obeys rules that carry that evidence into later decisions. For example, a voter may need to persist a vote before replying, so restarting does not let it forget and vote incompatibly. A participant holding an accepted log record may have to reject a candidate whose history is too old.

If B simply agrees with every request, intersection protects nothing: B can help two overlapping groups accept conflicting histories. The arithmetic establishes a shared member, while the protocol explains what that member must preserve and refuse.

This reasoning assumes the same fixed voter group. Changing the group can change which quorums intersect, so membership transitions require their own rules. Use quorum counts to follow a specified protocol, together with its persistence, eligibility, and recovery rules. A majority count on its own is not an algorithm for consistent storage.

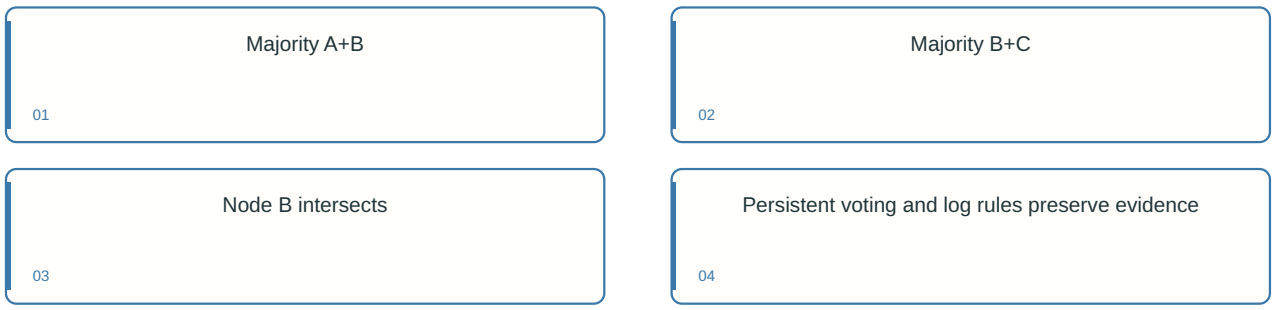


FIG. 15.1.6 **Quorum-based systems · the example**

A quorum is a permitted decision set within a protocol whose members preserve the required evidence.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · A failed region has been repaired and can receive traffic. What evidence would you require before allowing authoritative writes there again?

Reachability shows that communication is possible; it does not establish authority or a safe history. Require the current ownership and voter configuration, a history compatible with committed decisions, and the durable copies required by the acknowledgment policy. An old leader may need rebuilding and must remain fenced while obsolete authority could still act.

Also verify that routing directs writes to the authorised participants and that the region has enough capacity to resume its assigned work. Apply the supported rejoin procedure and reconcile uncertain external effects. The evidence should cover safe failback under current state, rather than assuming that restoring the old machines restores the old permission to lead.

Remember: To assess replication, trace a write from its authorised receiver through the required durable copies to the success reply. Then remove the primary and ask which replica may continue that history. The number of copies alone does not answer either question.

Try applying it: A follower receives a record in memory and acknowledges it, but crashes before storing it durably. What would the acknowledgment need to establish for the primary to count it as a copy that survives this process loss?

Unit 15 exercises → · Unit 15 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. MIT 6.5840: Distributed Systems, Spring 2026 ↗

   University course. Advanced distributed-systems reading.

   Access reviewed · 21 October 2026

2. In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

   Original research. Raft reference for Unit 15.

   Access reviewed · 21 October 2026

3. Paxos Made Simple — Leslie Lamport, 2001 ↗

   Original research. Conceptual consensus safety; atlas does not present toy code as production Paxos.

   Access reviewed · 21 October 2026

4. A Conflict-Free Replicated JSON Datatype — Kleppmann and Beresford ↗

   Original research · 2017 revision. One formal CRDT model; the miniature tie-break exercise is not this complete implementation.

   Access reviewed · 21 October 2026

Raft and replicated state machines

Prerequisites: 15.1 · Choose a replication model 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

The replicas agree through log index 8. A partition separates the leader from the other two machines, which must decide whether one of them can take over. Choosing a name is only the beginning: the new leader must continue the right history.

We will follow command application, elections, log replication, and commitment. Raft and Paxos show why a later decision must carry evidence from earlier decisions, including while the voter group changes.

15.2.1 · Consensus and replicated state machines

Start with an empty task collection on A, B, and C. Suppose they agree that the first command is “create task 7” and the next is “complete task 7.” Each applies creation to the empty collection, then completion to the resulting task. Their task collections now agree.

This is the replicated state-machine approach. A state machine applies a command to its current state to produce a new state and a result. Replication works when replicas begin from equivalent state and apply the same commands in the same order deterministically: the same state and command must produce equivalent results. Consensus lets participants agree on a choice; repeated decisions can establish the command sequence despite the failures allowed by the protocol.

Agreement must preserve safety, so incompatible commands cannot both become the accepted command at one position. Progress also needs conditions, such as enough reachable participants and sufficiently favourable communication. Consensus does not promise continued decisions through every possible partition or endless delay.

Suppose creation calls a random-number generator independently on each replica. The agreed command could then create different identifiers. Put the chosen identifier into the agreed command, or use another supported way to make the decision consistent. The same concern applies to reading the current time.

Applying a command also differs from causing an external effect. If every replica applies “charge the customer” by calling a payment provider, the log alone does not prevent repeated charges. External work needs explicit ownership, stable operation identity, and recovery at the receiving boundary.

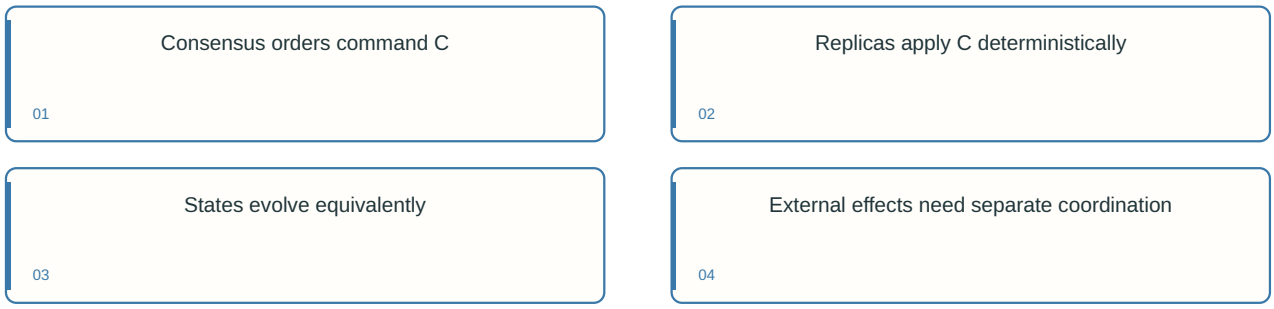


FIG. 15.2.1 **Consensus and replicated state machines · the example**

Replicated state machines apply agreed commands deterministically; external effects remain a separate boundary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.2.2 · Raft: terms, elections, log replication, commit

Return to three replicas A, B, and C. They share a committed history through index 8, where an index is a position in the log. A is leader, but a partition separates it from B and C. B stops receiving the expected leader messages. A timeout can make B a candidate: a participant seeking leadership. The timeout creates suspicion, rather than proving that A has stopped.

Raft divides activity into numbered terms, or election generations. In the worked trace, B seeks leadership in term 5. A voter grants at most one vote in a term and checks the candidate's log. Raft compares the term of the last entry first; a higher last-entry term is more up to date. If those terms match, the longer log wins the comparison. Thus a candidate cannot obtain votes merely by being reachable. It must meet the log-eligibility rule as well. The current term, vote, and required log state are persisted before the replies specified by Raft, so a restart cannot erase the evidence on which safety depends.

If eligible B obtains a majority, it becomes leader. It appends index 9 with term 5, sends it to C, and waits for the required durable replication. A follower checks that the preceding log position and term match before accepting the continuation. If its uncommitted suffix conflicts with the leader's history, Raft repairs that suffix under its log rules.

B can use majority replication of index 9 to advance commitment because that entry belongs to B's current term. Commitment establishes the authoritative prefix through that position; the replicas can apply its commands in order. An entry inherited from an earlier term cannot be declared committed merely by counting its copies on a majority. Raft commits earlier entries indirectly when the required current-term entry commits. This restriction works with election eligibility to preserve history through later leadership changes.

A remains unable to commit a new quorum-backed write alone. On learning a higher term, a participant returns to the follower role; the role diagram shows these transitions, rather than the entire algorithm. Reads need protection too. A leader can be isolated and unaware that a replacement exists. A supported linearizable-read path establishes current authority and the required committed position, then waits for local application before serving the result. Election, replication, commitment, reads, and membership must follow the implementation's complete Raft protocol.

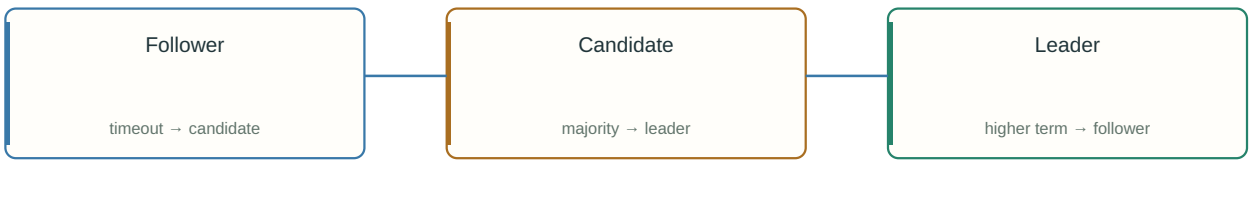


FIG. 15.2.2 **Raft election changes authority under protocol rules**

Election timeout can make a follower a candidate; a valid majority can elect an eligible leader. Observing a higher term returns a participant to follower. This is the role state machine, not the complete protocol.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.2.3 · Paxos: conceptual foundations

Consider one decision, such as which command belongs at a particular log position. A proposer asks acceptors to choose a value. An acceptor is a participant that records promises and accepted proposals. Paxos gives each proposal a unique, ordered number, so an acceptor can distinguish an older attempt from a newer one.

First comes preparation. A proposer chooses a higher proposal number and asks a majority for promises not to accept lower-numbered proposals. Each reply also reports the highest-numbered proposal the acceptor has already accepted, if there is one. If any such evidence is returned, the proposer must use the value from the highest-numbered accepted proposal among those replies. If none is returned, it may choose a new proposed value.

Next comes acceptance. The proposer asks acceptors to accept that proposal number and value. An acceptor can accept if doing so does not break a promise it has made to a higher-numbered attempt. Acceptance by the required majority chooses the value. Promises and accepted evidence must survive the failures covered by the protocol.

Why preserve an earlier value? A new preparation majority intersects any earlier choosing majority. Its replies carry evidence that prevents a later successful attempt from choosing a conflicting value. A new proposer is therefore not free to start the decision again with its favourite value. Competing proposers can still keep interrupting one another and delay progress.

Multi-Paxos uses related decisions to build an ordered command sequence; a stable leader can reduce repeated preparation work. This is a conceptual introduction. Precise persistence, recovery, membership, and progress rules remain necessary in an implementation.

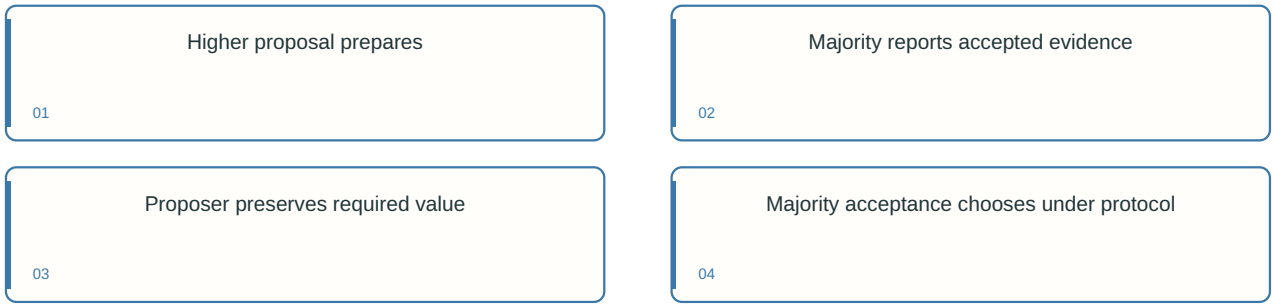


FIG. 15.2.3 **Paxos: conceptual foundations · the example**

Paxos preserves chosen decisions through numbered proposals and required prior evidence across intersecting majorities.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.2.4 · Log ordering and durability

A replica's log can contain a command that the application is not yet entitled to treat as accepted. It can also contain a committed command that has not yet changed its local readable state. To follow recovery, keep four questions separate: where does the record sit in the order, is it durably stored, is it committed, and has it been applied?

The figure shows a leader in term 5 with records 8:t4 and 9:t5. This notation means index 8 was created in term 4 and index 9 in term 5. One follower has durably stored both; another has only index 8. Under the fixed-membership Raft rules, the leader and the first follower supply the majority needed for the current-term entry 9. Its commitment also protects the preceding log prefix. The second follower can catch up later.

Until commitment is established, the presence of index 9 on a particular machine does not by itself justify a success claim. After commitment, a replica still has to apply the entry before its local task state reflects it. A read requiring that position must wait for the appropriate application progress.

Recovery preserves committed history while repairing incompatible uncommitted suffixes. A snapshot, which stores state representing an earlier applied prefix, must identify a history compatible with the remaining log. Monitor commit and apply positions as well as stored positions: each explains a different kind of delay.

Participant	Durable log suffix	Meaning
Leader term 5	8:t4, 9:t5	current-term entry 9
Follower A	8:t4, 9:t5	required durable replica
Follower B	8:t4	lagging replica

FIG. 15.2.4 **Presence in a log is not the same as commitment**

Within the stated fixed-membership Raft protocol, majority replication of the current-term entry can advance commit and protect the prefix. This table is not a generic quorum proof.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.2.5 · Membership changes

Suppose we replace a cluster's voters by editing a configuration file. The old group still believes that it may decide, while the new group begins deciding independently. If their decision majorities can be disjoint, neither group needs to consult a participant carrying the other's evidence. Both may accept incompatible histories.

Membership is therefore part of the agreement protocol. It specifies which participants have voting authority. A safe change must preserve the required decision rules during the transition, including when messages are delayed or the leader fails halfway through it.

Raft's joint-consensus approach introduces an intermediate configuration containing the old and new groups. Decisions in that stage need the required majority from each group. Only through the supported sequence does the new configuration become authoritative by itself. New servers can first receive the log as nonvoting participants, so catching them up does not immediately change who can decide.

Use the reconfiguration procedure provided by the chosen implementation; some implementations expose a different supported procedure. Preserve persistent member identities and test interrupted transitions. Reusing an identity or replacing all members outside the protocol can invalidate assumptions about remembered votes and logs. Adding a server changes more than storage capacity if it also changes the decision group.

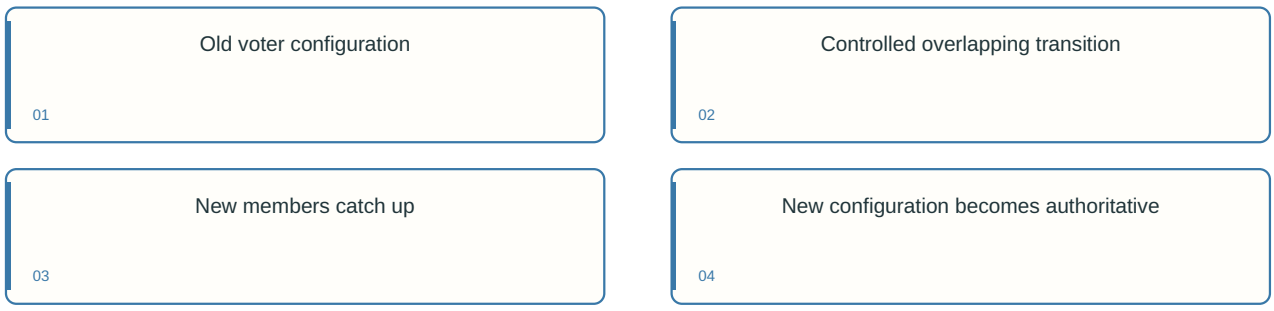


FIG. 15.2.5 **Membership changes · the example**

Membership changes need a supported transition preserving decision safety between old and new configurations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · A failed region has been repaired and can receive traffic. What evidence would you require before allowing authoritative writes there again?

Reachability shows that communication is possible; it does not establish authority or a safe history. Require the current ownership and voter configuration, a history compatible with committed decisions, and the durable copies required by the acknowledgment policy. An old leader may need rebuilding and must remain fenced while obsolete authority could still act.

Also verify that routing directs writes to the authorised participants and that the region has enough capacity to resume its assigned work. Apply the supported rejoin procedure and reconcile uncertain external effects. The evidence should cover safe failback under current state, rather than assuming that restoring the old machines restores the old permission to lead.

Remember: An agreed log is useful because eligible leaders preserve its committed history and replicas apply its commands consistently. Raft and Paxos combine remembered evidence with restrictions on later decisions. Their election, commit, recovery, read, and membership rules must be followed together.

Try applying it: A new Raft leader finds an earlier-term entry on a majority of replicas. Why is that observation alone insufficient for its usual copy-counting commit rule, and what current-term event can commit the preceding history?

Unit 15 exercises → · Unit 15 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. MIT 6.5840: Distributed Systems, Spring 2026 ↗

   University course. Advanced distributed-systems reading.

   Access reviewed · 21 October 2026

2. In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

   Original research. Raft reference for Unit 15.

   Access reviewed · 21 October 2026

3. Paxos Made Simple — Leslie Lamport, 2001 ↗

   Original research. Conceptual consensus safety; atlas does not present toy code as production Paxos.

   Access reviewed · 21 October 2026

4. A Conflict-Free Replicated JSON Datatype — Kleppmann and Beresford ↗

   Original research · 2017 revision. One formal CRDT model; the miniature tie-break exercise is not this complete implementation.

   Access reviewed · 21 October 2026

Fences, leases & failure detection

Prerequisites: 15.2 · Raft and replicated state machines 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Worker A pauses while holding a lease. The lease expires, and worker B takes over. A then wakes and sends the write it had planned earlier. Both processes are alive, but only one should still have authority.

We will separate detecting a possible failure, transferring permission, and checking that permission where an effect is accepted. This will help us see what locks, leases, metadata services, and fencing each contribute.

15.3.1 · Split-brain prevention and fencing tokens

Worker A is allowed to update a resource using token 7. It pauses for long enough that its lease expires. Worker B obtains token 8 and begins work. When A resumes, it still holds its old instructions and can send a write. Updating a coordinator's owner field has not stopped A's process or withdrawn messages it already sent.

A fencing token is an ordered ownership generation carried with a protected request. The receiver checks the generation before accepting the effect. Suppose the resource records 8 as its highest accepted generation. An incoming request with 7 must be rejected. The comparison and the protected update must be atomic: another request must not slip between the authority check and the change.

The point at which the receiver learns the new generation matters. Merely issuing token 8 at the coordinator does not make a receiver that has only seen 7 reject 7. The design must ensure that the newer generation is established where the effect is accepted, according to its ownership contract. The receiver must also retain the authority information across the failures covered by that contract.

Fencing prevents later actions under obsolete authority once the relevant newer generation is established. It does not undo an effect already accepted, and it does not protect a separate external API that ignores the token. Trace every effect boundary when preventing split brain: the condition in which competing participants act as if each has current authority.

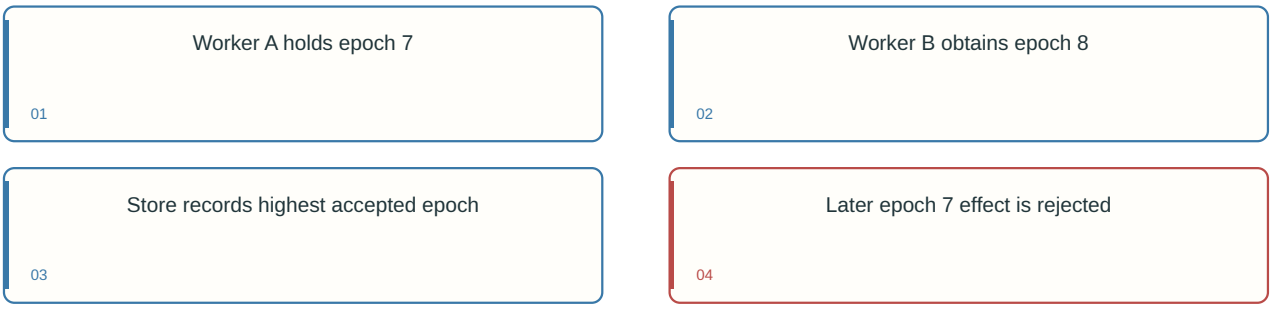


FIG. 15.3.1 **Split-brain prevention and fencing tokens · the example**

Fencing epochs reject stale authority only when receivers enforce them atomically with protected effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.3.2 · Failure detection

A follower expects heartbeats from its leader. None arrives before its waiting limit. The leader may have crashed, but it may also be paused or separated by a congested network. From the follower's local observation, these situations can look the same.

A failure detector turns observations such as missing messages into a suspicion that a participant has failed. Choosing a shorter timeout can start recovery sooner, but it can also produce more false suspicions. Choosing a longer one may avoid some unnecessary elections while making users wait longer after a real failure. These are choices about detection and recovery speed under assumptions about timing.

The ownership protocol must remain safe when the suspicion is wrong. In Raft, a timeout starts an election subject to voting and log rules; it does not grant leadership immediately. For a worker operating on external storage, resource-side fencing may be needed to reject the old worker after authority changes.

A heartbeat arriving successfully proves little about spare capacity or the latency of an unrelated request. Readiness and load checks answer additional questions. Keep these observations distinct: detection supplies evidence for taking action, while the authority protocol determines which decisions and effects may be accepted.

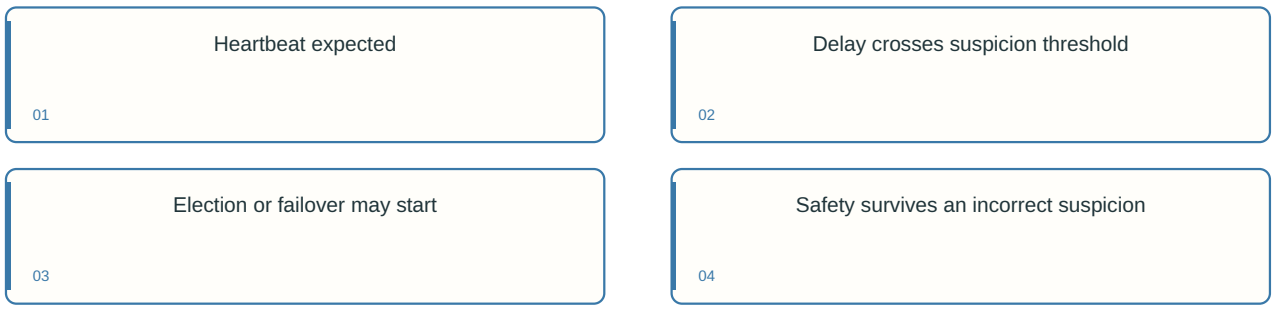


FIG. 15.3.2 **Failure detection · the example**

Failure detection produces suspicions under timing assumptions; authority safety must tolerate false suspicion.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.3.3 · Strongly consistent metadata services

A storage system needs to record which worker currently owns a partition. If two workers read incompatible owner records and both act on them, the payload can be damaged even when the payload store itself is healthy. Small records about ownership, configuration, or generations can therefore need stronger coordination than much larger application data.

A strongly consistent metadata service maintains such control information under a specified consistency contract. For a linearizable ownership operation, a successful update must be observed by later operations that require current state, respecting their real-time order. Consensus can support this contract, but a cached read from any replica does not automatically provide it. Use the service's supported write and fresh-read paths.

This service belongs to the control plane: the part that decides how work is assigned and managed. The data plane performs the ordinary reads, writes, or transfers using those decisions. Existing data-plane traffic may continue with safely usable cached state while the metadata service is unavailable. Acquiring a new lease or changing ownership may have to wait because it requires a fresh decision.

State which cached decisions remain valid, for how long, and how stale authority is rejected. Consulting the metadata service on every payload request creates a capacity and availability dependency, so do so where the contract requires it. Its own storage, quorum, and recovery assumptions remain part of the system design.

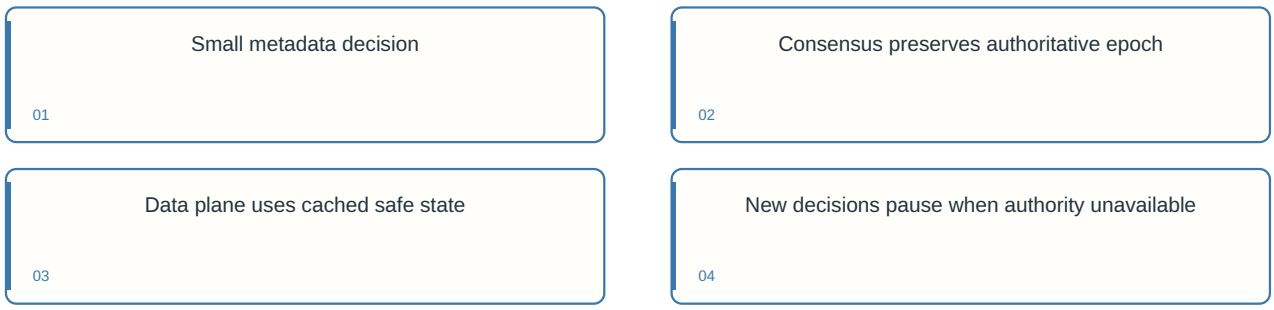


FIG. 15.3.3 **Strongly consistent metadata services · the example**

Consistent metadata services own small control decisions, with explicit consequences when new decisions cannot proceed.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.3.4 · Distributed locks and leases

Suppose only one worker should update a shared file at a time. A lock grants permission to an identified owner. If the owner crashes while holding it, a lock with no recovery rule could block others indefinitely. A lease addresses this problem by granting permission only for a bounded period, with expiry and renewal defined by the lease service's timing rules.

Now let the worker pause beyond that period. Another worker acquires the lease, but the first worker later resumes at the instruction following acquisition. Its old local variable may still say “I own the lease.” Acquisition and the protected action are separate events; time can pass and authority can change between them.

Use fencing at the protected resource when this history threatens correctness. The receiver must reject the obsolete ownership generation under an atomic authority check. Releasing the lock also needs an ownership check: an old holder must not delete the newer holder's lock merely because the lock has the same name. Renewal needs the corresponding identity and validity rules.

Leases rely on stated assumptions about expiry and time; a local clock comparison alone is not a complete distributed ownership protocol. When the entire invariant lies inside one database, a transaction or database-native coordination may protect it more directly. Choose the coordination boundary around the resource and the failures that can interrupt its use.

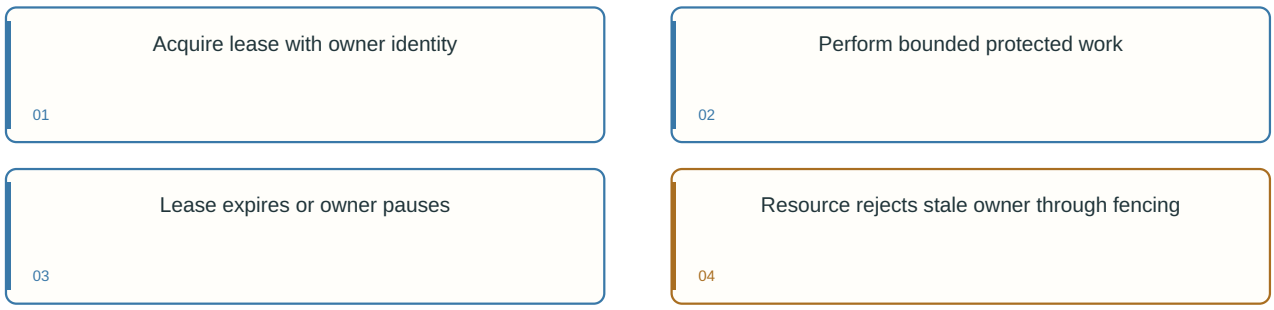


FIG. 15.3.4 **Distributed locks and leases · the example**

Locks and leases grant permission with identity and expiry rules, while resumed stale effects may still need fencing.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · A failed region has been repaired and can receive traffic. What evidence would you require before allowing authoritative writes there again?

Reachability shows that communication is possible; it does not establish authority or a safe history. Require the current ownership and voter configuration, a history compatible with committed decisions, and the durable copies required by the acknowledgment policy. An old leader may need rebuilding and must remain fenced while obsolete authority could still act.

Also verify that routing directs writes to the authorised participants and that the region has enough capacity to resume its assigned work. Apply the supported rejoin procedure and reconcile uncertain external effects. The evidence should cover safe failback under current state, rather than assuming that restoring the old machines restores the old permission to lead.

Remember: A missing heartbeat can justify suspicion, and a lease can expire, while the old process remains able to send requests. Protect the effect by checking current authority at its receiver. A fencing token helps only where that receiver enforces it atomically with the protected operation.

Try applying it: The coordinator has issued token 8, but the protected store has only seen token 7. What must the design establish at the store before relying on it to reject later requests carrying 7?

Unit 15 exercises → · Unit 15 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. MIT 6.5840: Distributed Systems, Spring 2026 ↗

   University course. Advanced distributed-systems reading.

   Access reviewed · 21 October 2026

2. In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

   Original research. Raft reference for Unit 15.

   Access reviewed · 21 October 2026

3. Paxos Made Simple — Leslie Lamport, 2001 ↗

   Original research. Conceptual consensus safety; atlas does not present toy code as production Paxos.

   Access reviewed · 21 October 2026

4. A Conflict-Free Replicated JSON Datatype — Kleppmann and Beresford ↗

   Original research · 2017 revision. One formal CRDT model; the miniature tie-break exercise is not this complete implementation.

   Access reviewed · 21 October 2026

Conflicts, CRDTs & regional trade-offs

Prerequisites: 15.3 · Fences, leases & failure detection 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Two regions can keep serving local writes while disconnected, but their changes may disagree when communication returns. We need to decide whether to merge the changes, preserve a conflict, or require one authority to choose.

We will build a simple convergent set, examine what its merge can guarantee, and then apply the same questions to regional replication and recovery. A merge must preserve the application's intended meaning, not merely make the copies equal.

15.4.1 · Conflict resolution and CRDT introduction

Two disconnected replicas begin with an empty set. Replica A adds X, while replica B adds Y. When they exchange state, each takes the union: the result contains X and Y. Merging again changes nothing. Reversing the exchange order produces the same result. A grow-only set, which permits additions but no removals, gives us a simple example of a conflict-free replicated data type, or CRDT.

CRDTs define update and merge rules so that replicas can converge under stated delivery and state assumptions. For a state-based grow-only set, union is commutative, so merge order does not matter; associative, so grouping does not matter; and idempotent, so repeated receipt of the same state adds no new effect. With the required updates eventually exchanged, these properties make duplicate and differently ordered exchanges manageable. Other CRDT families use different update and delivery conditions.

Removal needs more information. If one replica removes X while another concurrently adds X, simply choosing union cannot express the intended removal. An observed-remove set gives additions distinct identities and tracks which additions a removal actually observed. A concurrent addition that the removal

did not observe can survive. Safe cleanup must preserve the information needed to distinguish these histories.

Convergence says that replicas eventually agree according to the type's rules. It does not guarantee every application invariant. Two regions can each allocate the last item they see; merging both decrements can agree on negative stock. Exclusive allocation needs suitable coordination or a design that assigns separate spending rights. Some edits are better retained as conflicts for people to resolve. Choose the merge's meaning before choosing automatic convergence.

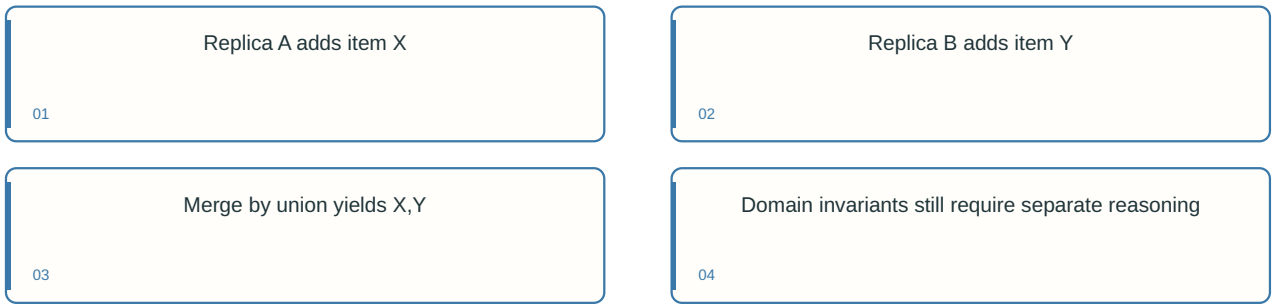


FIG. 15.4.1 **Conflict resolution and CRDT introduction · the example**

CRDTs converge through specified update/merge rules but do not enforce arbitrary business invariants.

Author-created illustration. Numbers are illustrative unless stated otherwise.

15.4.2 · Multi-region replication trade-offs

A user writes in one region while another region holds a recovery copy. If the local primary replies before that remote copy is durable, the request avoids waiting for that remote acknowledgment. It also leaves a period in which loss of the local history can lose an acknowledged write. Waiting for a required cross-region durable quorum changes that exposure, while adding remote communication and making some partitions block progress.

Begin a regional design by stating where each dataset may be written and which locations must acknowledge success. A single-authority design needs safe promotion and stale-owner protection. A multi-leader design needs valid conflict semantics. The copies' geography matters: several replicas inside one region do not provide the same regional-loss protection as required durable copies elsewhere.

During a partition, the chosen contract determines which side, if any, can accept authoritative writes. Some stale reads may remain acceptable even when new writes must fail. State this per operation rather than describing the whole application as simply available or unavailable.

Recovery has a return journey too. A repaired old primary may carry a divergent uncommitted suffix or lack records committed elsewhere. Before it can write, it must rejoin under current authority and catch up, reconcile, or rebuild according to the protocol. Keep stale actions fenced and verify routing and capacity before returning traffic. Test failback, the return to a recovered region, as carefully as failover to the replacement.

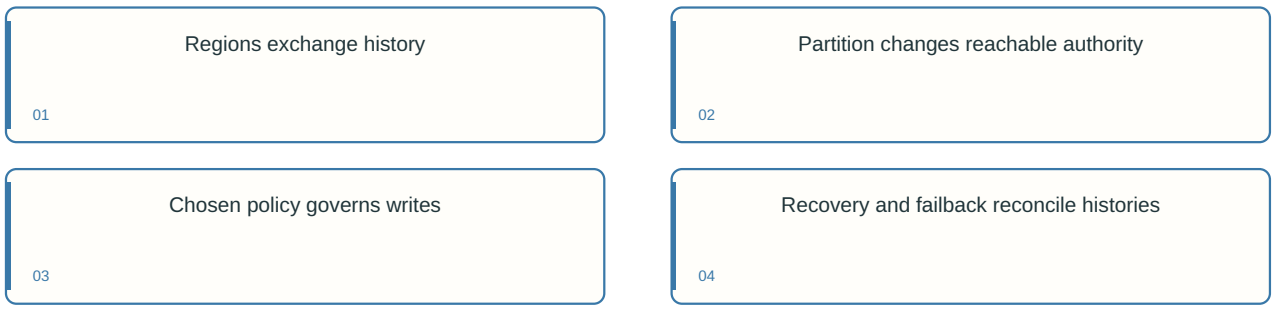
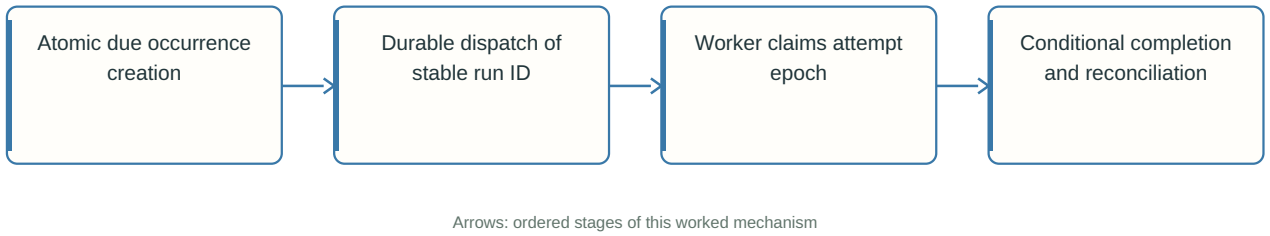


FIG. 15.4.2 **Multi-region replication trade-offs · the example**

Regional replication needs authority, acknowledgment geography, conflict, failover, and safe return-to-service rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 15.4.3 **Follow the related complete design**

Create due work durably before a current worker attempts its effect.

Author-created illustration. Numbers are illustrative unless stated otherwise.

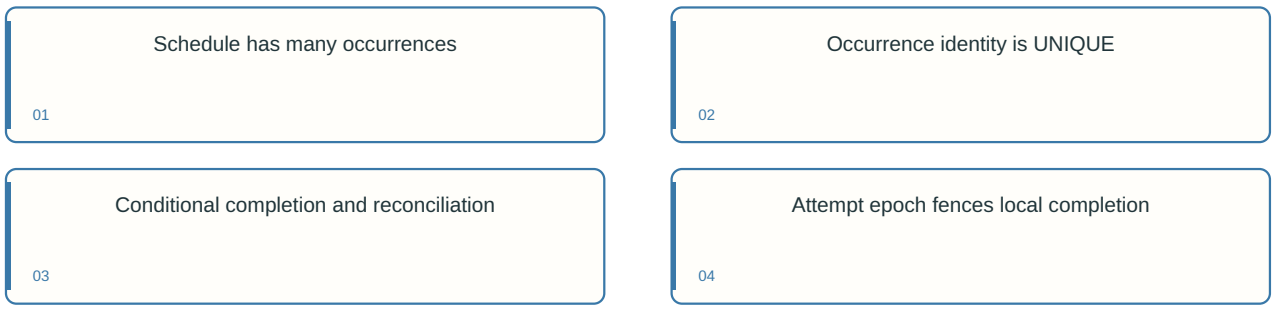


FIG. 15.4.4 **State and ownership in the related design**

One scheduled occurrence can have several attempts, while retaining one logical run identity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PSEUDOCODE · THREE REPLICAS, ONE PARTITION

```
A,B,C contain committed index 8
partition isolates old leader A
B and C elect eligible B in term 5
B appends index 9, term 5
B and C durably replicate index 9
Raft commit rule establishes committed prefix
A cannot acknowledge a new quorum-committed write alone
rejoin: A follows term 5 and repairs uncommitted suffix
```

The trace begins with three fixed voters A, B, and C, all containing committed history through index 8. Assume that participants follow the protocol rather than behaving maliciously, and that required protocol state is durable across the allowed crashes. A partition isolates the old leader A while B and C can still communicate.

B becomes leader in term 5 only after satisfying Raft's election rules, including log eligibility and the required majority of persisted votes. B appends index 9 with term 5. B's durable copy and C's durable copy form a majority of the three voters. Because index 9 belongs to the current term, Raft's commit rule lets B establish commitment through index 9, including the preceding prefix. Replicas then apply committed commands in order; durable storage alone is a different milestone from application.

A has only its isolated vote and cannot acknowledge a new quorum-committed write alone. It may remain alive and retain uncommitted work, which is why isolation must not be treated as proof that the old process

stopped. On rejoining, A observes term 5, follows the current leader, and repairs an incompatible uncommitted suffix under Raft's rules. Committed history must be preserved.

This pseudocode follows one history; it does not implement elections, log matching, recovery, membership changes, or fresh reads. Use the full supported protocol for those operations. In particular, an isolated node must not serve a supposedly current read merely because it still believes it is leader, and changing the three-voter configuration requires the implementation's membership procedure.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · A failed region has been repaired and can receive traffic. What evidence would you require before allowing authoritative writes there again?

Reachability shows that communication is possible; it does not establish authority or a safe history. Require the current ownership and voter configuration, a history compatible with committed decisions, and the durable copies required by the acknowledgment policy. An old leader may need rebuilding and must remain fenced while obsolete authority could still act.

Also verify that routing directs writes to the authorised participants and that the region has enough capacity to resume its assigned work. Apply the supported rejoin procedure and reconcile uncertain external effects. The evidence should cover safe failback under current state, rather than assuming that restoring the old machines restores the old permission to lead.

Remember: Replicas can converge on an outcome that the application cannot accept. Choose update and merge semantics from the business rule, then state where regional writes may proceed, what success requires, and how a recovered region safely rejoins.

Try applying it: Two regions each allocate the last item while disconnected, then merge both decrements consistently. What does their agreement establish, and which allocation rule still needs a different design?

Unit 15 exercises → · Unit 15 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. MIT 6.5840: Distributed Systems, Spring 2026 ↗

   University course. Advanced distributed-systems reading.

   Access reviewed · 21 October 2026

2. In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

   Original research. Raft reference for Unit 15.

   Access reviewed · 21 October 2026

3. Paxos Made Simple — Leslie Lamport, 2001 ↗

   Original research. Conceptual consensus safety; atlas does not present toy code as production Paxos.

   Access reviewed · 21 October 2026

4. A Conflict-Free Replicated JSON Datatype — Kleppmann and Beresford ↗

   Original research · 2017 revision. One formal CRDT model; the miniature tie-break exercise is not this complete implementation.

   Access reviewed · 21 October 2026

PART 4 / UNIT 16

Message Queues, Events, and Distributed Workflows

The order service saves a customer's order and asks a receipt worker to send an email. The worker sends it, but its acknowledgment is lost. When the same message arrives again, should the worker send another receipt? To answer, we need to distinguish the record being delivered from the effect it asks for.

The running example

Design an e-commerce checkout that coordinates payment, inventory reservation, shipment creation, and refund compensation.

Unit 16 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

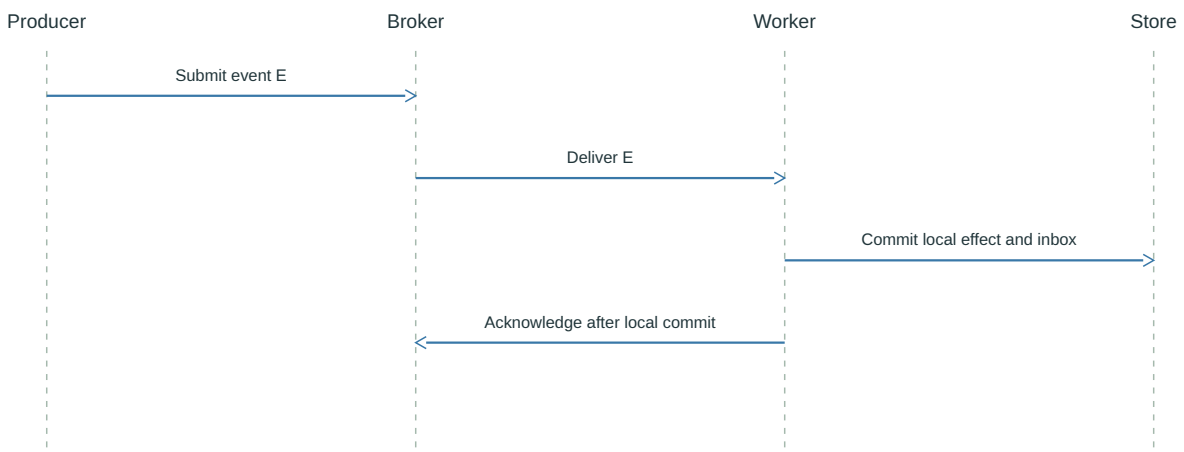


FIG. 16.0.1 **Queue lifecycle**

A lost final acknowledgment can cause redelivery; the local effect contract must handle it.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Partition	Owner before	Owner after rebalance
P0	Consumer A	Consumer B
P1	Consumer B	Consumer B
Progress	Last committed offset	In-flight work may replay

FIG. 16.0.2 **consumer partition assignment**

Ordering is per partition under this model, and rebalancing does not make in-flight side effects disappear.

Author-created illustration. Numbers are illustrative unless stated otherwise.

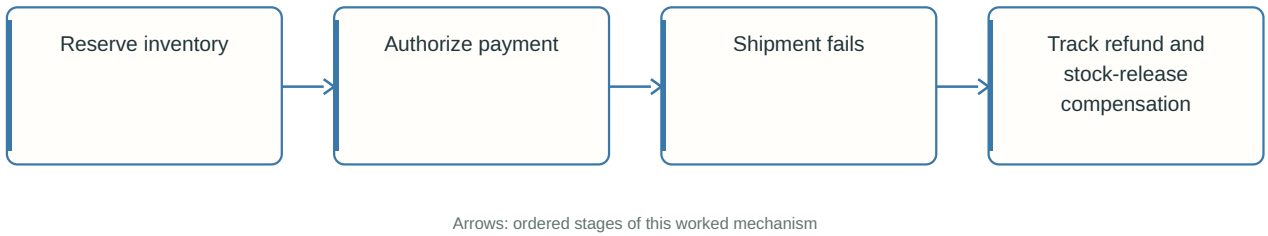


FIG. 16.0.3 **Saga happy/failure paths**

Compensation has its own retry, uncertainty, and final-state obligations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

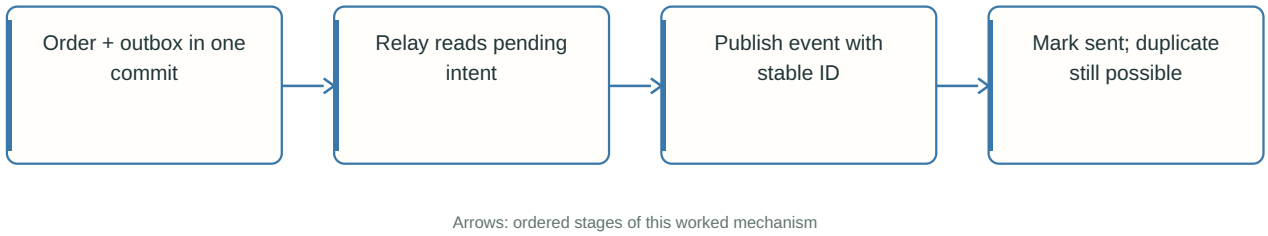


FIG. 16.0.4 **The outbox closes a local publication gap**

The relay can crash after publish but before marking. A consumer must handle the repeated event; the database transaction does not include the entire network.

Author-created illustration. Numbers are illustrative unless stated otherwise.

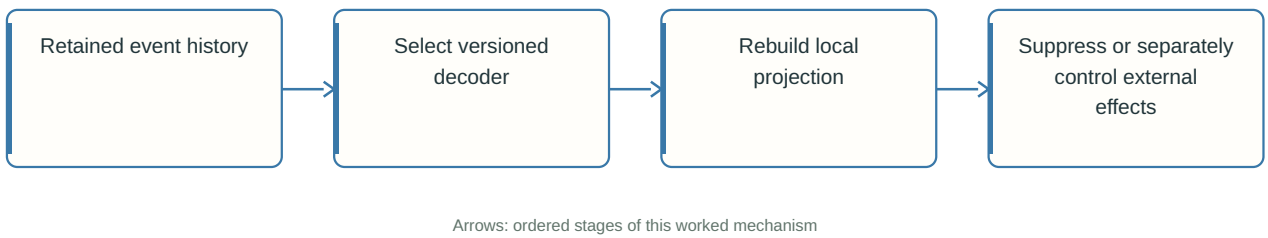


FIG. 16.0.5 **event replay**

Replaying state must not blindly repeat emails, charges, or other external business effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Queues, logs & delivery boundaries

Prerequisites: API Design and Communication Contracts, Database Internals, Transactions, and Concurrency Control, Distributed Systems Fundamentals, Replication, Leader Election, and Consensus

9 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

The order service saves a customer's order and asks a receipt worker to send an email. The worker sends it, but its acknowledgment is lost. When the same message arrives again, should the worker send another receipt? To answer, we need to distinguish the record being delivered from the effect it asks for.

We will follow a message from producer to broker to consumer, comparing queues, retained logs, and publication to subscribers. Then we will use short crash histories to decide what acknowledgments and delivery guarantees actually establish. Partition order and transaction boundaries will help us state precisely which progress can be recovered and which effects still need protection.

16.1.1 · Synchronous vs asynchronous communication

A customer places an order. The order service can save the order and reply while receipt delivery continues later. If the request waits for the receipt result, that part of the communication is synchronous. If submission and observing completion happen separately, it is asynchronous.

There are now two completion points. The first response may mean “your order and the obligation to send a receipt are saved.” It does not yet mean “the receipt was delivered.” The response should provide an order identifier and a way to check progress after the request ends.

For the promise to survive a process crash, the receipt intent must become durable before the service reports that it has accepted this work. Keeping a background task only in request memory leaves nobody

responsible after a restart. A queue lets workers handle a temporary burst later, but it does not create processing capacity: if arrivals continue faster than workers finish, the backlog keeps growing.

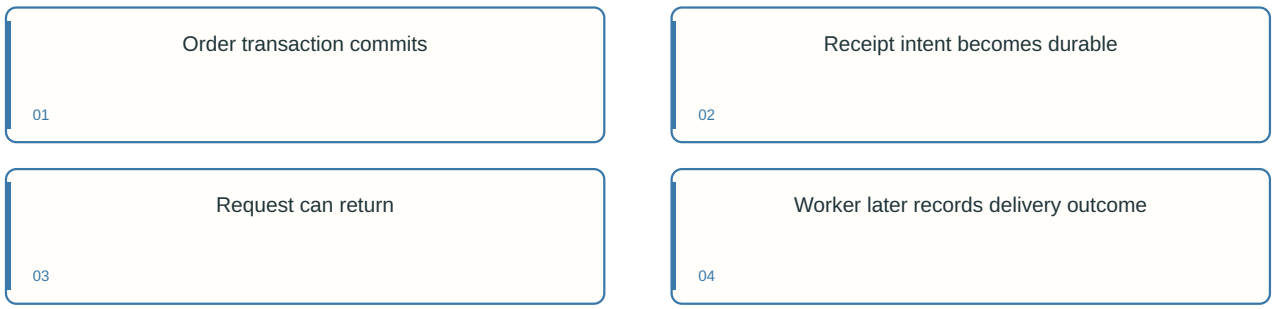


FIG. 16.1.1 **Synchronous vs asynchronous communication · the example**

Asynchronous submission separates acceptance from completion and needs an owner for the intervening work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.2 · Queues vs logs vs pub/sub

Suppose several receipt workers are available. A work queue can assign a receipt job to one worker, then assign another job to a different worker. This arrangement distributes work among consumers. A retained log instead keeps an append history that readers can revisit, each using a stored position to find its next record.

Now suppose both a receipt service and an analytics service should learn about each order. Publication and subscription, or pub/sub, describes distributing publications to subscribers. Whether a subscriber can recover messages missed during an outage depends on the system's retention and acknowledgment policy. An ephemeral publication may retain nothing for an absent subscriber.

A product can combine queue, log, and pub/sub behaviour. Choose by the work that recovery requires. A consumer returning after a long outage needs records retained for long enough and a way to recover its progress position. Calling a service a broker does not, by itself, establish that it provides this durable history.

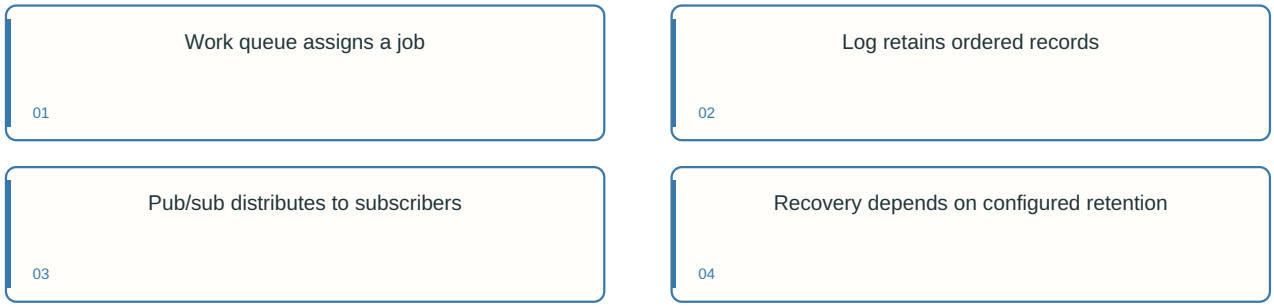


FIG. 16.1.2 **Queues vs logs vs pub/sub · the example**

Queues distribute work, logs retain ordered history, and pub/sub follows its stated subscription and retention contract.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.3 · Producers, consumers, brokers

Follow one receipt event through the system. The order service submits the event; it is the producer. The broker stores or routes it. A receipt worker receives the event and performs the work; it is the consumer.

Several replies can occur along this path. The broker might acknowledge receiving bytes in memory, or it might acknowledge persistence under specified storage and replication assumptions. Later, the consumer can acknowledge receipt or completion of a business effect. These statements establish different facts. A producer that has received a broker reply cannot assume that a customer already has a receipt.

Name what each acknowledgment confirms. Give the message a stable identity, a schema version explaining its format, an owner, and correlation information linking it to the order. Then follow both its age and its business outcome. A broker can be healthy and accepting messages while every consumer rejects their payloads; broker uptime and enqueue rate alone would hide that failure.

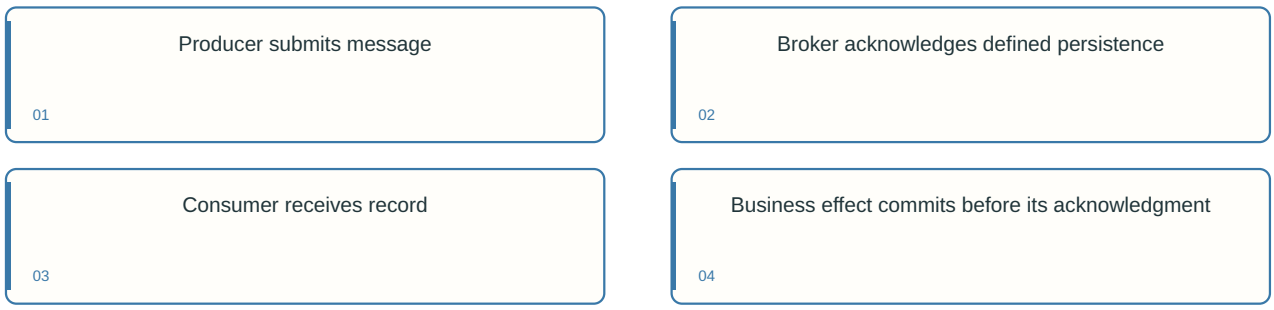


FIG. 16.1.3 **Producers, consumers, brokers · the example**

Producers, brokers, and consumers acknowledge different stages that must be named to reason about ownership.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.4 · Partitions, consumer groups, offsets

A retained log may divide records into partitions. Think of each partition as an ordered lane. A key, such as an account identifier, selects a lane so related records can share its order. An offset identifies a position within that partition; a committed progress position tells a consumer where to resume.

A consumer group distributes responsibility for partitions among its members. Suppose a consumer processes offset 42, then stops before saving its progress. After a membership change, another consumer may receive that partition and process the record again. This transfer of partition ownership is a rebalance. It does not erase effects the earlier consumer already performed.

Each partition has its own order and progress. Reaching a particular offset in P0 says nothing about completion in P1. Choose keys from the ordering the application needs, and decide when progress may safely commit. One heavily used partition can limit the whole group while other consumers are idle. Adding consumers does not automatically divide that lane, and parallel work within a lane needs its own effect-ordering rules.

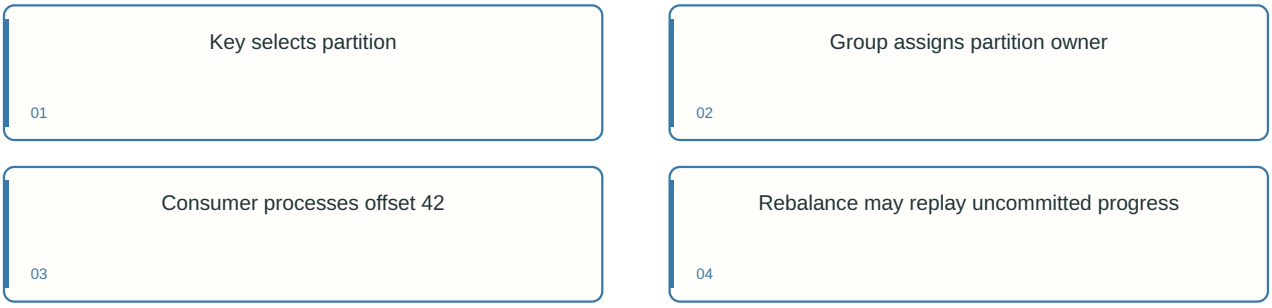


FIG. 16.1.4 **Partitions, consumer groups, offsets · the example**

Partitions provide ordered lanes; consumer groups and offsets track ownership and progress within those lanes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.5 · Delivery guarantees

The broker delivers event E7 twice, but the application increments its local counter only once. These observations can both be correct. A delivery is an attempt to pass the record to the consumer. The counter change is the business effect produced while processing it.

To obtain this result, the consumer recognises E7 and commits that recognition together with the counter change. On redelivery, it finds the saved identity and skips another increment. An external email sent during processing would be a further effect, with its own rules for repetition.

A delivery guarantee therefore needs a stated boundary. Are we counting broker deliveries, execution starts, or committed effects? Follow acceptance at the source, retention and redelivery, effect commit, and progress acknowledgment. State what survives the failures under discussion at each step. A guarantee about one of these steps does not automatically describe the entire path from the original request to every business outcome.

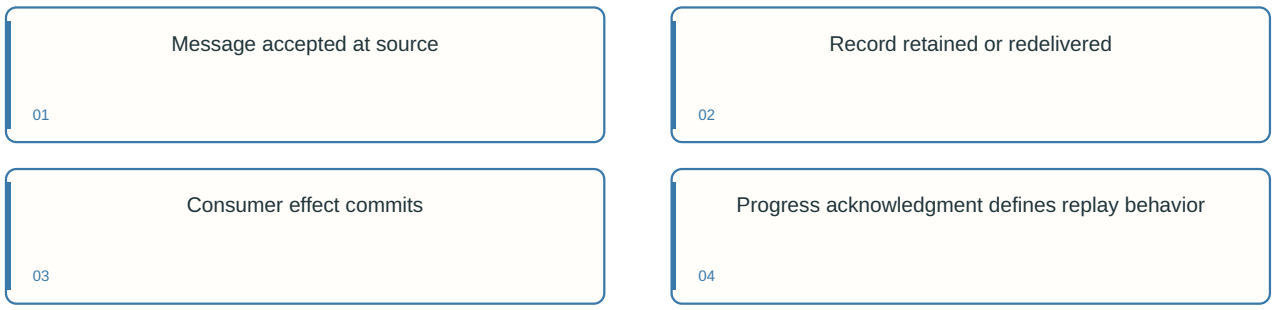


FIG. 16.1.5 **Delivery guarantees · the example**

Delivery guarantees need a stated record or effect boundary and do not automatically compose across external systems.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.6 · At-most-once and at-least-once processing

Consider two possible orders for processing a message. In the first, the consumer acknowledges the message and then updates its counter. If it crashes after acknowledgment but before the update, the broker may consider the work finished and never redeliver it. At this boundary, at-most-once handling allows a missing outcome: zero applications are possible.

In the second order, the consumer updates the counter and then acknowledges. A crash between these events leaves the counter changed but the broker's progress unchanged. Redelivery can start processing again. Under the required retention and recovery assumptions, this supports at-least-once handling, so the application must make repetition safe.

For one local database effect, an atomic inbox can record the processed event identifier and the effect in the same transaction. Either both commit or neither does. Saving them separately leaves another crash gap. We will examine this consumer pattern below. Neither acknowledgment order alone makes an unrelated email or payment call atomic with broker progress.

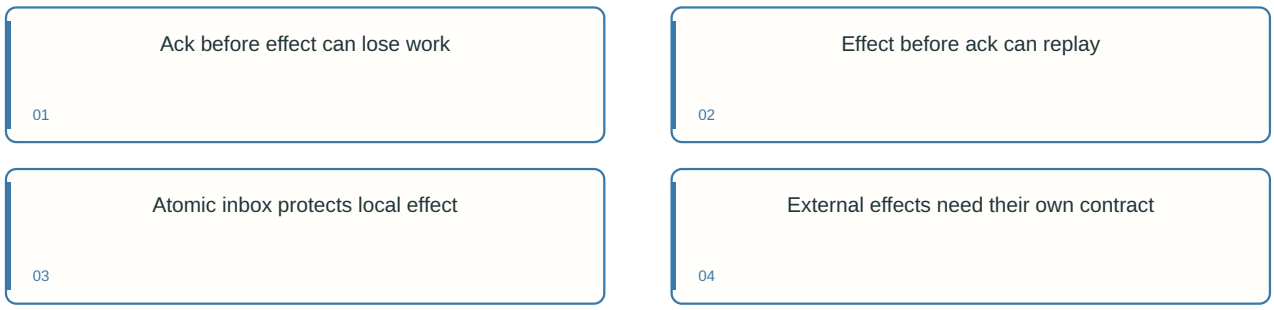


FIG. 16.1.6 **At-most-once and at-least-once processing · the example**

At-most-once can lose required effects; at-least-once can repeat them and therefore needs safe effect handling.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.7 · Exactly-once processing within defined boundaries

Suppose a processor reads an input record, produces an output record, and saves where it has reached in the input. A supported stream-processing transaction can commit the output and consumed progress together. If it aborts, neither becomes committed; if it commits, both do. Recovery can then avoid skipping that input or exposing a second committed output for the same processing.

This is a meaningful exactly-once claim within the records, transaction protocol, and fault model it covers. Now let the processing also call an external email server. The email may be sent before a crash even if the stream transaction does not commit. Recovery can execute the input again and send another email. The external call was outside the transaction.

For an external provider, use the protocol that provider supports. A stable idempotency key may let several attempts produce one relevant outcome within its identity and retention rules, although execution still starts more than once. State which records and effects the guarantee covers, and which failures recovery can handle.

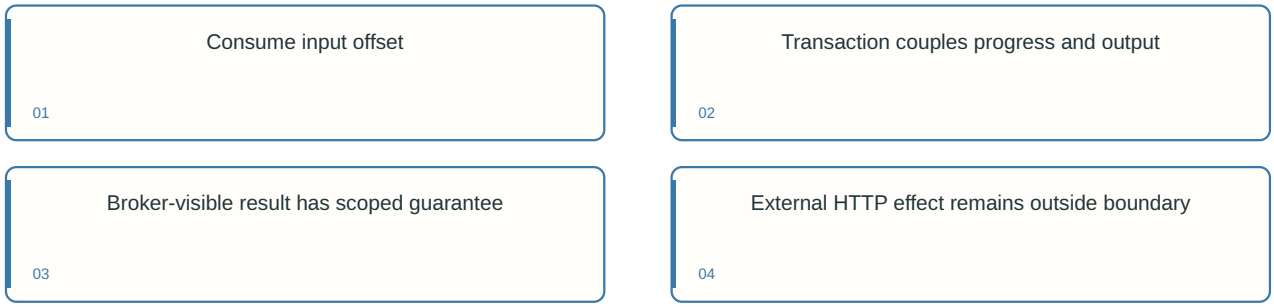


FIG. 16.1.7 **Exactly-once processing within defined boundaries · the example**

Exactly-once guarantees cover only the supported atomic boundary and its identity, retention, and failure assumptions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.8 · Ordering guarantees

Two account events have sequence numbers 10 and 11. The log stores and delivers 10 first. Worker A starts processing 10 but waits for a provider. Worker B starts 11 and finishes first. Log order, delivery order, start order, and effect-completion order are different observations; the first two did not establish the last.

If the account rule requires event 10 to take effect before 11, the consumer must preserve that dependency when applying changes. It can coordinate processing by account or use an equivalent sequencing boundary. A later record must not be reported as applied while an earlier required change remains unresolved.

A permanently failing record makes this choice visible. Skipping event 10 may let processing continue, but can break the account's required sequence. Sequence numbers and gap detection help reveal the missing step. Decide the business ordering rule first, then choose partition keys, processing concurrency, and a deliberate policy for records that block progress.

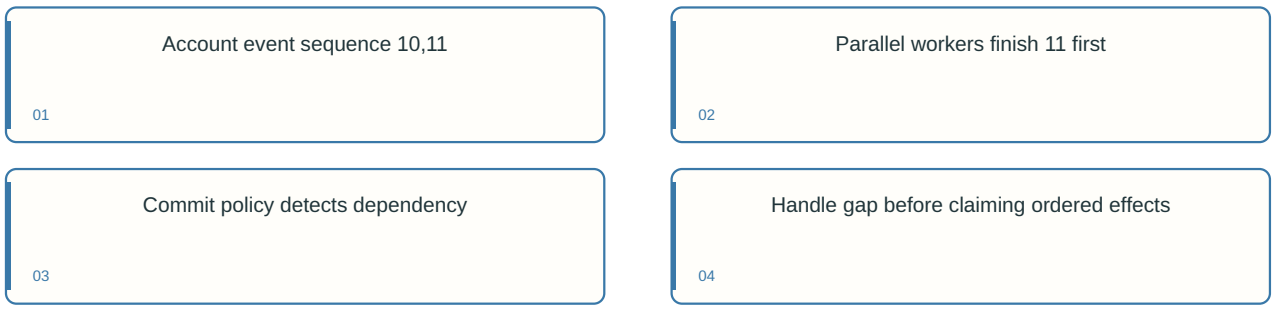


FIG. 16.1.8 **Ordering guarantees · the example**

Publication, delivery, execution, and effect order can differ when work proceeds concurrently.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.1.9 · Consumer acknowledgements

A consumer commits its local update, sends an acknowledgment, and loses contact with the broker. If the broker did not record the acknowledgment, it can redeliver the message. The local update still happened. Moving the acknowledgment before the update would create the opposite risk: a crash could leave the broker finished and the update missing.

An acknowledgment tells the broker that a specified responsibility has advanced. For example, “saved to a durable inbox” can allow later processing to recover from that inbox. “Business effect completed” claims that a later step has finished. In either case, the receiver needs durable evidence supporting the promise through the failures it covers.

Choose the effect boundary before deciding when to acknowledge. Local transactions, inbox/outbox records, stable provider keys, and reconciliation each address particular gaps; acknowledgment alone cannot make a separate external system atomic. Set acknowledgment deadlines or extend processing leases to fit the expected work as well. Otherwise, a healthy but slow worker may repeatedly lose its assignment while it is still running.

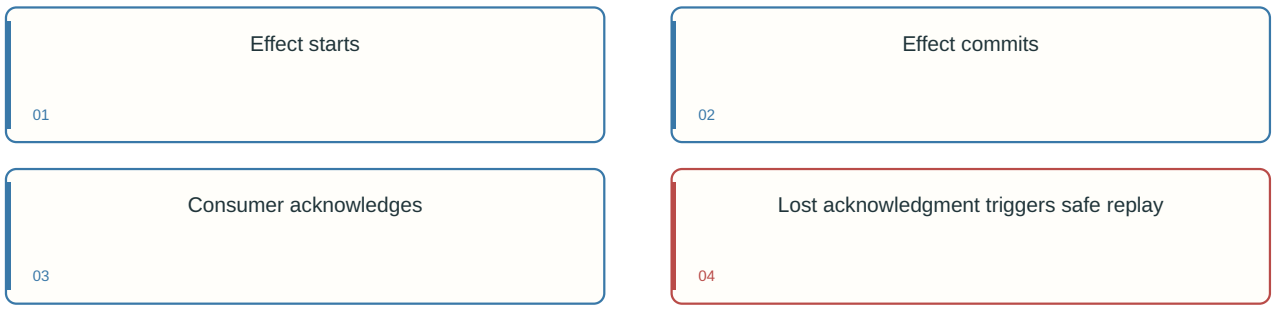


FIG. 16.1.9 **Consumer acknowledgements · the example**

Consumer acknowledgment advances broker progress, with loss or repetition gaps around separately committed effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How could one broker transaction be exactly-once while two emails are sent?

The broker can atomically commit consumed progress and produced records while email remains outside its transaction. A consumer can send an email, lose the completion reply, and send again during recovery. Use the provider's contract, stable identity where supported, and reconciliation. The broker's scoped guarantee does not automatically cover the email effect.

Remember: Follow the accepted message, processing attempt, committed effect, and acknowledgment as separate events. State the failure assumptions and the boundary covered by each delivery or ordering guarantee.

Try applying it: A consumer commits its counter update and crashes before acknowledgment. Explain why redelivery can be safe with an atomic inbox. What further problem appears if processing also sends an email?

Unit 16 exercises → · Unit 16 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Saga distributed transactions pattern — Microsoft ↗

   Official engineering guidance. Local transactions and compensation; compensation is not always a perfect inverse.

   `Access reviewed · 21 October 2026`

2. Transactional Outbox — Chris Richardson ↗

   Pattern author’s explanation. Publication intent and duplicate delivery.

   `Access reviewed · 21 October 2026`

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   `Access reviewed · 21 October 2026`

Retries, consumers & event history

Prerequisites: 16.1 · Queues, logs & delivery boundaries 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Our consumer can recover a message after a crash, but recovery can deliver it more than once. Another record fails every time because its format is no longer understood. Repeating all work indefinitely would waste capacity, while dropping every failure would abandon valid obligations.

We will separate temporary failures from records needing repair, then make a local effect safe under repeated delivery. Finally, we will examine a different use of events: preserving accepted business changes as authoritative history. The goal is to recover unfinished work without confusing transport attempts, completed effects, and the events used to reconstruct state.

16.2.1 · Retries and dead-letter queues

A receipt attempt fails because its provider is briefly unavailable. Waiting before retrying gives the provider time to recover and stops one failing job from immediately consuming another worker. An invalid address is different: sending the unchanged request a thousand times will not repair it. Classify failures and bound both attempts and waiting time.

When the ordinary retry policy ends, a dead-letter queue can hold the unresolved work for repair or review. The job has moved to another recovery path; its business obligation has not thereby been completed. Important unresolved work still needs an owner and alerting.

Keep the original event identity, source version, attempt history, and a safe error reason. After repair, replay should continue the same work under its duplicate and ordering rules. A replay tool that invents fresh identities could repeat established effects, while one that skips a required record could break a sequence. The repair path should restore valid progress.

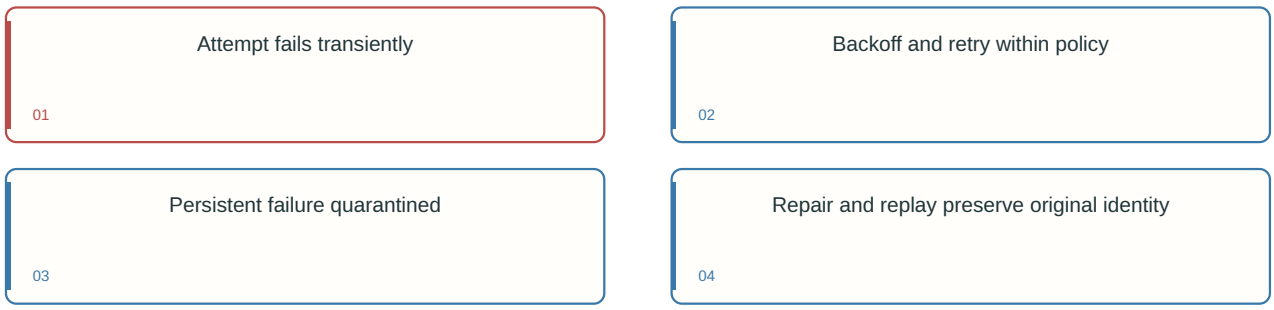


FIG. 16.2.1 **Retries and dead-letter queues · the example**

Retry and dead-letter policies need bounds, useful error context, ownership, and a safe repair/replay path.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.2.2 · Poison messages

One record fails on every attempt while neighbouring records succeed. Its data may be malformed, its schema version unsupported, or its requested business action invalid. Such a record is often called a poison message because ordinary retries keep encountering the same obstacle.

The response depends on the cause. If a corrected decoder can read a legitimate historical version, a controlled replay may recover the work. If the message requests an action the business rules forbid, retrying the unchanged payload will not make it valid. Before discarding records, inspect samples and failure patterns: a software bug can reject an entire valid version.

Quarantine enough identity and error evidence to diagnose and repair the record, while avoiding unnecessary sensitive payloads. Decide whether records depending on it must wait. Bounded retries protect processing resources, and a review path protects the unresolved business work. During repair and replay, preserve the rules that prevent repeated effects and detect gaps in required ordering.

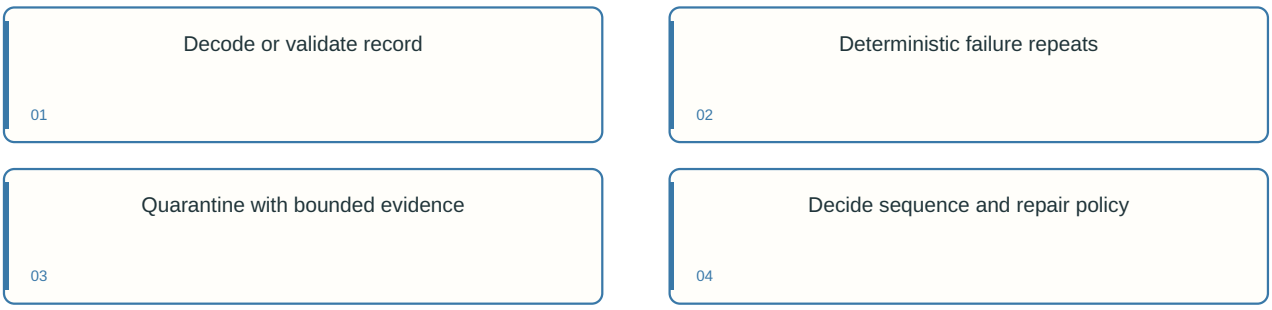


FIG. 16.2.2 **Poison messages · the example**

Poison messages need diagnosis and controlled quarantine rather than unlimited identical retries.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.2.3 · Idempotent consumers

The consumer receives E1 and begins a database transaction. It inserts E1 into a table of processed event identifiers, then increments the orders counter. A uniqueness rule allows only one stored entry for E1. When this transaction commits, both the recognition and the counter change become durable.

Suppose the acknowledgment is lost and E1 arrives again. The attempted insert finds the existing identity, so the consumer skips the increment. If the first transaction had rolled back, neither its identity nor its counter change would be committed, and another attempt could do the work. This is an idempotent consumer for the local effect: repeated deliveries leave the same relevant outcome.

The two writes must share the transaction. Recording E1 first in a separate commit could cause recovery to skip a missing increment; incrementing first could cause recovery to repeat it. Keep the identity for the supported delivery and replay window, and retain it through restart. An external provider is outside this local transaction and needs its own stable key or reconciliation protocol.

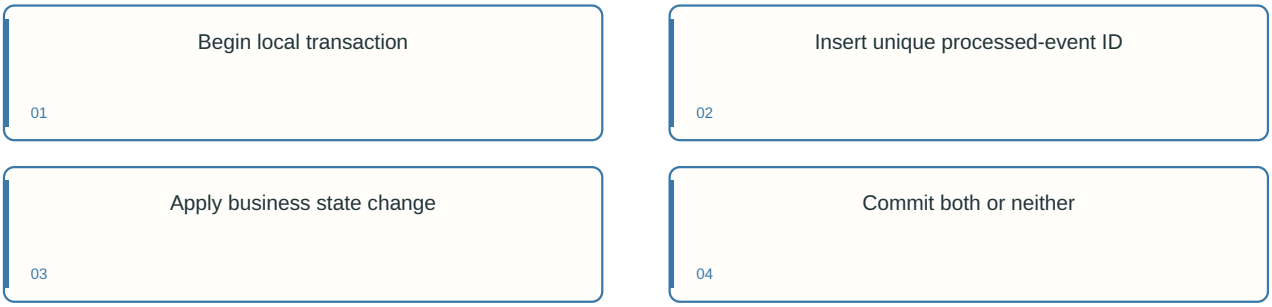


FIG. 16.2.3 **Idempotent consumers · the example**

Idempotent consumers commit recognition and the protected local effect together, retaining identity for supported replay.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.2.4 · Event sourcing

An order can be represented by its latest state, or by a history of the accepted changes that produced it. With event sourcing, that accepted domain-event history is authoritative. For example, applying OrderPlaced and then later accepted changes under the order's rules reconstructs its current state.

Reconstruction applies the events in order. A snapshot can save time by storing an intermediate state together with the event position it represents; replay then continues from the next event. A broker's transport log is not automatically this domain history. It may retain records for a different period and have different ownership and meanings.

The history must remain interpretable as schemas change, so define how corrections, replay, privacy, and retention work. Rebuilding state or a read projection must not resend real charges or emails whenever an old event is read. Auditability also depends on the domain and its access rules: append-only storage alone does not establish a sufficient financial ledger. Choose event sourcing when preserving and interpreting this history serves a real need.

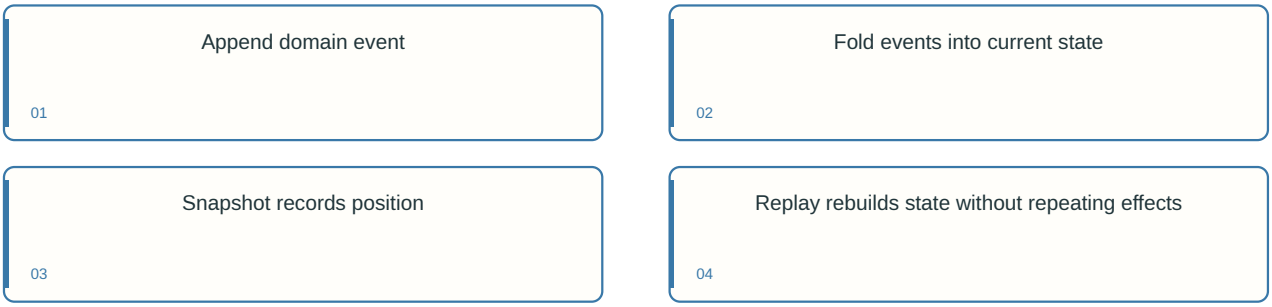


FIG. 16.2.4 **Event sourcing · the example**

Event sourcing makes interpretable accepted events authoritative, with snapshot, correction, privacy, and replay obligations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How could one broker transaction be exactly-once while two emails are sent?

The broker can atomically commit consumed progress and produced records while email remains outside its transaction. A consumer can send an email, lose the completion reply, and send again during recovery. Use the provider's contract, stable identity where supported, and reconciliation. The broker's scoped guarantee does not automatically cover the email effect.

Remember: Classify failures before retrying. Preserve identity through repair, commit local duplicate recognition with its effect, and distinguish transport history from the domain history used to reconstruct state.

Try applying it: A repaired decoder can now read a quarantined event, but that event may already have produced a local effect before failing later. Which identity and completion evidence should replay preserve, and how could skipping it affect dependent events?

Unit 16 exercises → · Unit 16 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Saga distributed transactions pattern — Microsoft ↗

   Official engineering guidance. Local transactions and compensation; compensation is not always a perfect inverse.

   Access reviewed · 21 October 2026

2. Transactional Outbox — Chris Richardson ↗

   Pattern author's explanation. Publication intent and duplicate delivery.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Read models, the outbox & sagas

Prerequisites: 16.2 · Retries, consumers & event history 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An order is saved successfully, but its event is never published because the service crashes. Later, payment succeeds and shipment fails. Both histories leave work owed to another part of checkout, even though a local database transaction has finished.

We will first separate the authoritative command model from a convenient read view. Then we will record publication intent alongside the order and follow how a relay delivers it. Finally, we will coordinate several local outcomes with a saga, treating compensation as further tracked business work. These examples show what a local commit can settle and what requires continuing recovery across owners.

16.3.1 · CQRS

An order command checks whether a requested transition is legal before changing the authoritative order. A customer-history page has another task: show convenient rows of order summaries. Each can be simpler with a model suited to its work. Command Query Responsibility Segregation, or CQRS, separates these command and query responsibilities or models.

A projection builds the read-friendly view from accepted changes. After an order command commits, the projection may still show the earlier state until it processes that change. Define what the customer should see during this interval. The application might read the current authoritative owner, show progress, or accept a stated delay. It also needs a way to rebuild the projection after failure.

CQRS can use one database and one process. Event sourcing, separate databases, and separate services are additional choices. Introduce extra models and synchronisation where they simplify actual validation and query workloads enough to justify maintaining agreement between them.

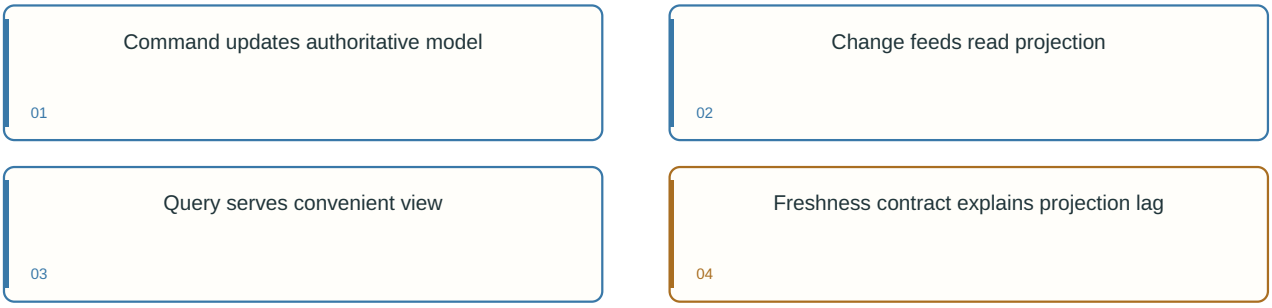


FIG. 16.3.1 **CQRS · the example**

CQRS separates command and query models where their different needs justify projection and freshness work.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.3.2 · Transactional outbox

An order commits, but the process crashes before it publishes OrderPlaced. The database contains the order, while downstream services have no event to act on. A transactional outbox repairs this local gap: write the order change and an outbox row describing the event in the same database transaction. A commit establishes both the change and the obligation to publish; an abort establishes neither.

A relay reads the pending outbox row and publishes its event. Suppose publication succeeds, then the relay crashes before marking the row sent. On recovery, the row still appears pending, so the relay can publish the same event again. The durable intent prevents forgotten publication, but it does not by itself prevent repetition.

Keep a stable event identity so consumers can handle duplicates. Monitor the age of pending intents and retain enough history for the required recovery. The original transaction covers local state and publication intent; subscribers and their external effects keep their own transaction, identity, and recovery rules.

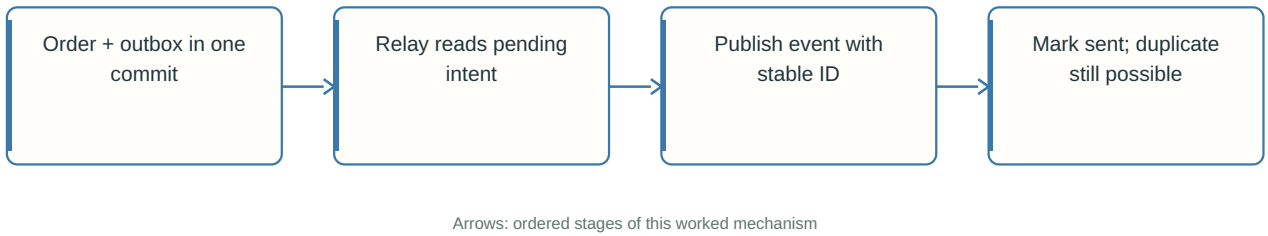


FIG. 16.3.2 **The outbox closes a local publication gap**

The relay can crash after publish but before marking. A consumer must handle the repeated event; the database transaction does not include the entire network.

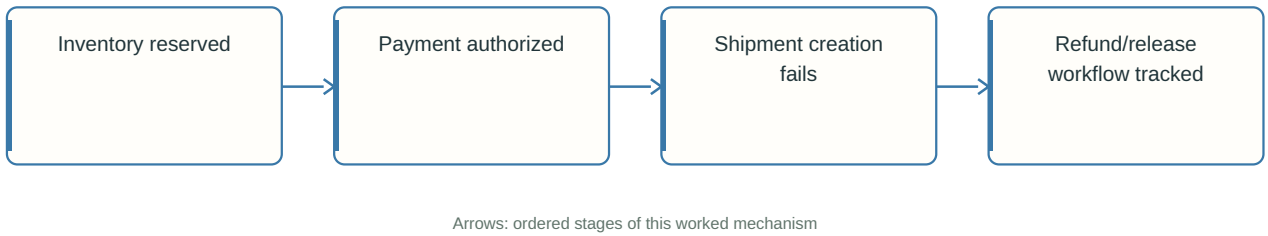
Author-created illustration. Numbers are illustrative unless stated otherwise.

16.3.3 · Sagas and compensating transactions

Checkout reserves inventory, authorises payment, and requests shipment from separate owners. Each step can commit locally before the next begins. A saga records this sequence and the actions required when the overall operation cannot continue.

Suppose inventory is reserved and payment authorised, but shipment creation fails. The workflow now owes compensating actions: release the stock reservation and cancel the authorisation or request the appropriate refund, according to the payment state. The reservation and payment step really occurred. Compensation is another business operation that moves the system toward an acceptable recovery outcome.

Persist forward and compensating progress. If a refund attempt fails or its reply is lost, the saga remains unresolved until retry or reconciliation establishes the result. A refund may happen later, incur different costs, or leave user-visible consequences; compensation is not always a perfect inverse. When any step's reply is missing, check the relevant authority using its stable identity before assuming nothing happened or starting a fresh operation.



Arrows: ordered stages of this worked mechanism

FIG. 16.3.3 **A failed shipment starts tracked compensation**

Each step is a local outcome. Compensation can fail or remain uncertain and must have durable progress and reconciliation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

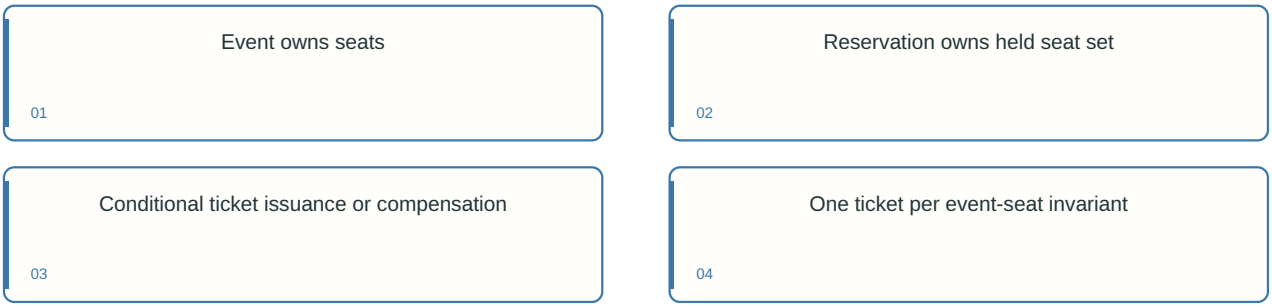


FIG. 16.3.4 **State and ownership in the related design**

Seat, hold, payment, and ticket records must preserve their defined ownership relationships.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How could one broker transaction be exactly-once while two emails are sent?

The broker can atomically commit consumed progress and produced records while email remains outside its transaction. A consumer can send an email, lose the completion reply, and send again during recovery. Use the provider's contract, stable identity where supported, and reconciliation. The broker's scoped guarantee does not automatically cover the email effect.

Remember: An outbox commits local state with the obligation to publish. A saga tracks outcomes and compensation across owners. Each mechanism settles a particular gap while later effects keep their own recovery rules.

Try applying it: The outbox relay publishes OrderPlaced and crashes before marking it sent. What does recovery see, why can publication repeat, and which transaction should protect a subscriber's local update?

Unit 16 exercises → · Unit 16 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Saga distributed transactions pattern — Microsoft ↗

   Official engineering guidance. Local transactions and compensation; compensation is not always a perfect inverse.

   `Access reviewed · 21 October 2026`

2. Transactional Outbox — Chris Richardson ↗

   Pattern author's explanation. Publication intent and duplicate delivery.

   `Access reviewed · 21 October 2026`

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   `Access reviewed · 21 October 2026`

Coordinate durable workflows

Prerequisites: 16.3 · Read models, the outbox & sagas 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Checkout may wait for approval long after the original request has ended. During that wait, a scheduler can restart, a worker can lose its lease, and a provider can finish an operation whose reply never arrives. Recovery needs to know which business intention still exists and which attempt currently has authority.

We will compare participant-driven coordination with an explicit coordinator, then examine how engines and durable job records preserve progress. Stable identities will connect scheduled occurrences, worker attempts, and provider outcomes. We will also budget replay alongside live traffic and reconcile uncertain results, so restarting work advances the original operation safely.

16.4.1 · Choreography vs orchestration

After inventory is reserved, checkout needs to request payment. With choreography, a participant observes the InventoryReserved event and reacts by starting the next action. With orchestration, an explicit coordinator records “inventory reserved” and decides to issue the payment command. Both arrangements can implement the same business sequence.

Follow where an operator would find the current progress and the next obligation. In choreography, that information is spread across participants and their events, which can make an entire stalled workflow harder to see. Orchestration gathers the decisions about retries and compensation into a coordinator, while adding that coordinator and a workflow definition that must evolve safely.

Neither arrangement removes participant failure or the need for durable outcomes. Choose according to the sequence's complexity, audit needs, and teams changing it. Carry a shared correlation identity through

either design so an operator can connect each participant's records to the same business operation and work out where it stopped.

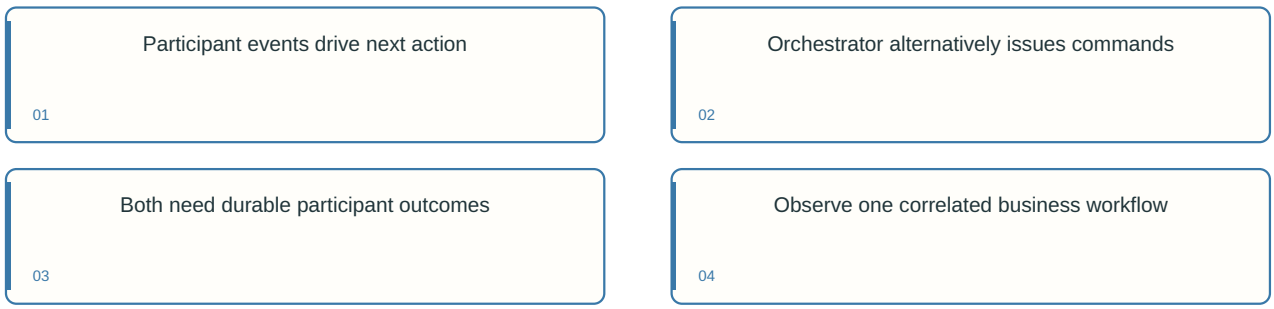


FIG. 16.4.1 **Choreography vs orchestration · the example**

Choreography distributes event reactions; orchestration gives workflow direction and progress an explicit owner.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.4.2 · Workflow engines

A workflow requests human approval and waits. Its executor then restarts. If progress and the outstanding wait are durable, a replacement can continue waiting for the same approval instead of beginning checkout again. A workflow engine provides mechanisms for persisting progress, timers, and retry decisions under its supported programming model.

External activities still have separate failure points. Suppose an activity sends a payment request, the provider accepts it, and the worker crashes before recording completion. The recovered workflow knows that it dispatched the activity, but may not know the provider's result. Running the activity again can repeat the call. A stable activity identity and effect key let recovery relate attempts to the original intention and reconcile with the provider.

Understand the engine's replay, determinism, versioning, activity-identity, and cancellation rules. The application must still define legal transitions and safe external actions. Durable coordination makes long-running recovery easier within that model; it does not automatically establish one external effect for every activity.



FIG. 16.4.2 **Workflow engines · the example**

Workflow engines persist coordination under a programming model while external activities still need safe effect contracts.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.4.3 · Durable job execution

A durable job record keeps the intended work's identity, input reference, progress, attempt history, and replaceable owner. The job is the business intention. An attempt is one worker's try at carrying it out.

Suppose attempt seven stops responding, loses authority, and attempt eight begins. Attempt seven may later resume. Before accepting its result, storage must check that the attempt or fencing epoch is still current and that the result belongs to the expected input version. A conditional completion can then reject the obsolete worker's write. This check protects the result where it is enforced; external effect receivers need suitable protection too.

Store large inputs separately and verify their version before processing. Bound retries and expose terminal or review states when automatic recovery cannot finish. Returning HTTP 202 does not itself give a process-local background task durable progress or a replacement owner. The accepted response must agree with the persisted job and with who will continue it after a crash.

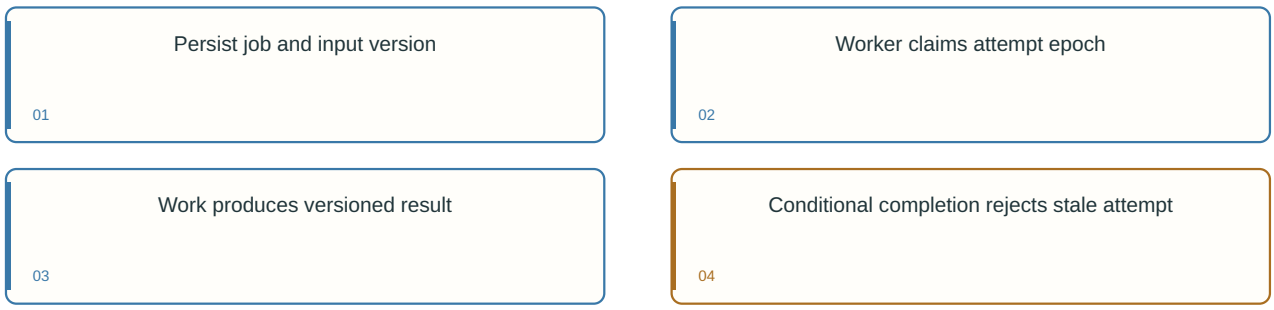


FIG. 16.4.3 **Durable job execution · the example**

Durable jobs retain identity, input, state, and attempt authority beyond the initiating process.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.4.4 · Scheduling, leases, and worker coordination

A report is due at noon. Two schedulers both observe that it is time to run. Give the occurrence a stable identity, such as (schedule_id, occurrence), and use an atomic claim or uniqueness rule to create one durable run for it. Later worker retries are attempts of that run; they are not new noon occurrences.

A worker may hold a lease that grants authority for a limited period. If its lease expires while it is paused, another worker can take over, yet the old worker may resume too. Lease expiry does not stop its code. Protect accepted effects with the appropriate idempotency or fencing checks at the resource that receives them.

Define what happens after missed periods, clock changes, overlapping runs, and scheduler failover. One scheduler process does not by itself make occurrences unique or recoverable across restarts. Deciding that an occurrence is due and safely executing its effects are separate stages, each needing durable identity and recovery rules.

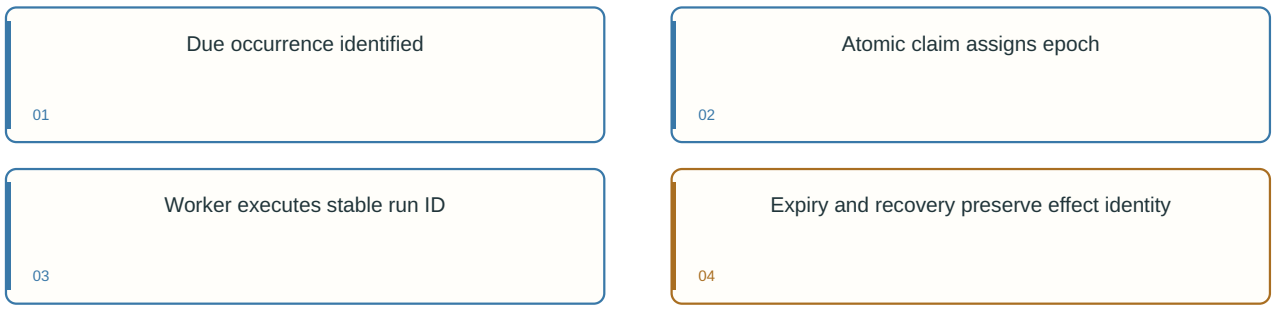


FIG. 16.4.4 **Scheduling, leases, and worker coordination · the example**

Scheduling separates stable due occurrences from attempts, with atomic creation, time, overlap, and stale-worker rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.4.5 · Backpressure and replay

A consumer returns after an outage and replays retained input. Its old records may call the same database or provider that is currently serving live requests. If replay uses all available capacity, current users can wait even though the recovery worker appears productive.

For an illustrative capacity budget, suppose the dependency can handle 100 operations per second and live work needs 70. At most 30 operations per second remain for replay under those assumptions; allocating all 100 to replay leaves no room for the live work. These figures are teaching inputs, not measured capacity. Backpressure limits admission or slows producers when downstream work cannot keep up, while a separate replay budget limits catch-up work.

Preserve source identities and suppress or separately control repeated external effects during replay. Retention must cover the required recovery interval. Monitor oldest unfinished age and blockage by key as well as queue length and throughput. Many completed records can hide one important operation that has remained stuck.

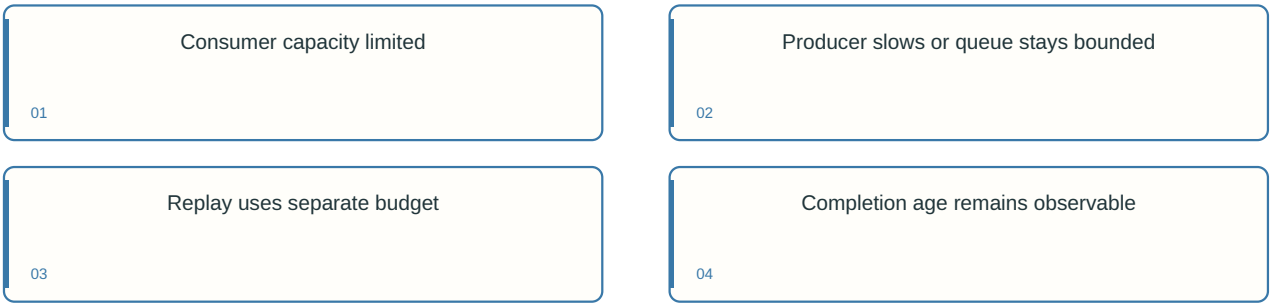


FIG. 16.4.5 **Backpressure and replay · the example**

Backpressure and replay need capacity budgets, retained identity, and useful completion indicators.

Author-created illustration. Numbers are illustrative unless stated otherwise.

16.4.6 · Reconciliation after partial failure

The local payment record says pending, but the provider has charged the card. This can happen when the provider accepts the charge and its reply is lost before local completion is saved. A fresh charge would risk paying twice.

Reconciliation compares the intended local operation with the observed external outcome. Use the original stable operation identity to query the provider. If evidence confirms the charge, apply the matching local transition under the usual uniqueness and state rules, and record the evidence. If the evidence is incomplete, keep a pending investigation state. Recovery is advancing the existing intention rather than inventing another payment to clear a pending row.

The reconciler must be authorised, idempotent, and observable. Run reconciliation periodically and during recovery, and track unresolved discrepancies by age and owner. This is a normal part of handling partial failure. It still depends on correct persistence and validation on the main path: without the original identity and intent, there may be no safe basis for comparison.

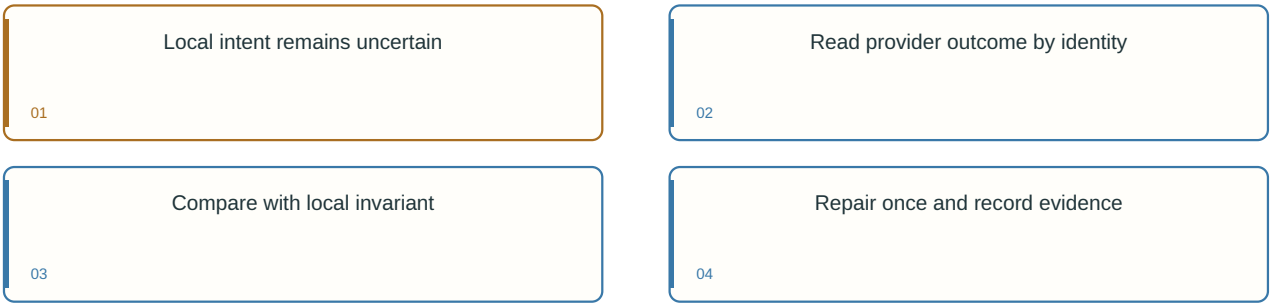


FIG. 16.4.6 **Reconciliation after partial failure · the example**

Reconciliation compares intended and observed outcomes and repairs partial failure through safe identified operations.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · DUPLICATE DELIVERY, ONE LOCAL EFFECT

```
from mechanisms import open_demo_store, consume_local_event
db = open_demo_store()
assert consume_local_event(db, "E1", "orders")
assert not consume_local_event(db, "E1", "orders")
assert db.execute("SELECT value FROM counters").fetchone()[0] == 1
db.close()
```

The first call receives E1 and returns True. Within its transaction, it inserts E1 into the processed-event table and increments the orders counter from zero to one. The second call uses the same E1, finds that identity already present, returns False, and leaves the counter at one. The assertion checks this committed local result.

Event recognition and the counter update share one transaction. If that transaction rolls back, neither change is committed, so recovery can try again. If acknowledgment is lost after commit, repeated delivery can safely find E1. This example protects a local database effect. Replacing the increment with an external email or payment call would require another protocol because the provider is outside that transaction.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · How could one broker transaction be exactly-once while two emails are sent?

The broker can atomically commit consumed progress and produced records while email remains outside its transaction. A consumer can send an email, lose the completion reply, and send again during recovery. Use the provider's contract, stable identity where supported, and reconciliation. The broker's scoped guarantee does not automatically cover the email effect.

Remember: Keep a durable identity for the intention and a separate record of attempts and authority. Recovery can then reject stale completion, limit replay, and reconcile uncertain outcomes against the original operation.

Try applying it: Attempt seven loses its lease and attempt eight starts. Meanwhile, the provider reports that the original payment succeeded although the workflow says pending. Which checks should govern worker completion, and how should recovery use the provider result?

Unit 16 exercises → · Unit 16 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Saga distributed transactions pattern — Microsoft ↗

   Official engineering guidance. Local transactions and compensation; compensation is not always a perfect inverse.

   `Access reviewed · 21 October 2026`

2. Transactional Outbox — Chris Richardson ↗

   Pattern author's explanation. Publication intent and duplicate delivery.

   `Access reviewed · 21 October 2026`

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   `Access reviewed · 21 October 2026`

PART 4 / UNIT 17

Reliability, Fault Tolerance, and Disaster Recovery

Two application replicas run on different processes, so we expect one to keep serving if the other stops. During an incident, both lose access to the same credential source and fail together. Later, a single surviving replica cannot carry the traffic that the pair had been serving.

The running example

A downstream payment provider becomes unavailable during peak traffic. Demonstrate the failure cascade and progressively introduce resilience mechanisms.

Unit 17 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

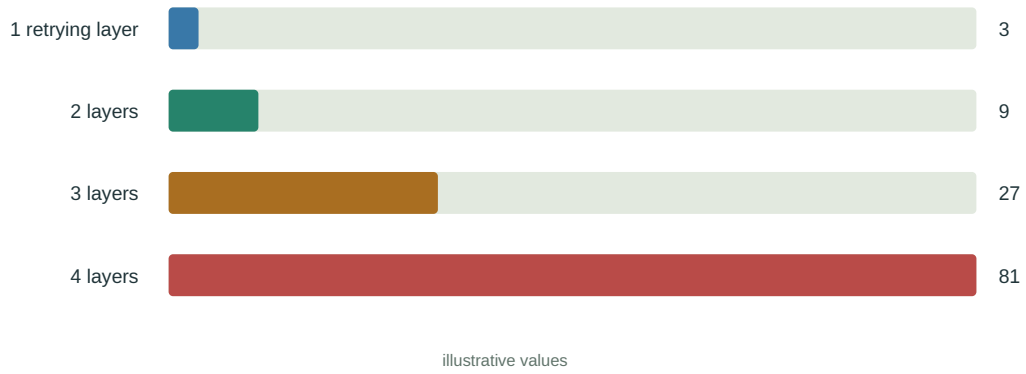


FIG. 17.0.1 **Retry amplification timeline**

Three attempts at every independent layer can amplify an intent geometrically; jitter alone does not reduce the attempt budget.

Author-created illustration. Numbers are illustrative unless stated otherwise.

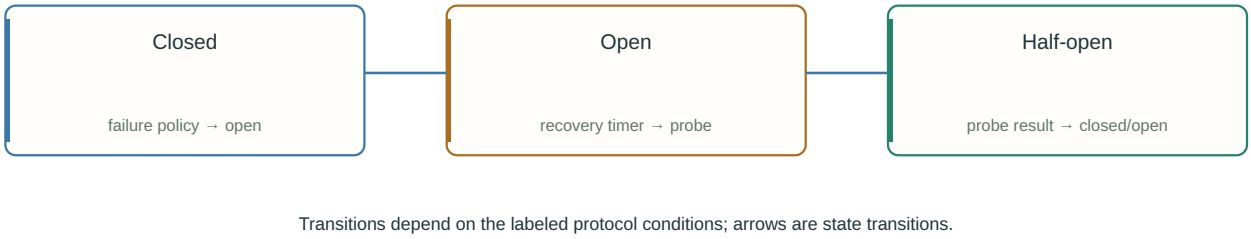


FIG. 17.0.2 **A circuit breaker bounds futile calls**

Closed permits calls, an observed failure policy opens the breaker, and bounded half-open probes test recovery. The breaker does not fabricate a successful business result.

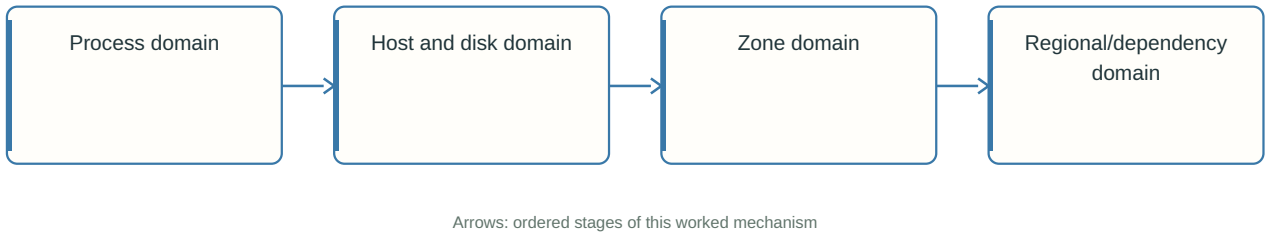
Author-created illustration. Numbers are illustrative unless stated otherwise.

Pool	Criticality	Failure isolation
Payments	Critical	Reserved bounded capacity
Receipts	Optional	Own queue and workers
Shared limits	DB, memory, network	Still require aggregate budgets

FIG. 17.0.3 **bulkhead isolation**

Separate pools limit some failure propagation, but shared resources remain correlated.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 17.0.4 **failure-domain map**

Each larger failure domain can remove several smaller replicas at once; independent placement is deliberate.

Author-created illustration. Numbers are illustrative unless stated otherwise.



FIG. 17.0.5 **Recovery time and recoverable data are different**

Last recoverable data at 11:55, incident at 12:00, service restored at 12:20: five minutes of data exposure and twenty minutes of recovery in this example.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Failure domains & redundancy

Prerequisites: Requirements, Estimation, and Engineering Trade-offs, Scalability, Load Balancing, and Data Distribution, Replication, Leader Election, and Consensus, Message Queues, Events, and Distributed Workflows

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Two application replicas run on different processes, so we expect one to keep serving if the other stops. During an incident, both lose access to the same credential source and fail together. Later, a single surviving replica cannot carry the traffic that the pair had been serving.

These histories raise different questions: which causes can remove several replicas at once, what information survives, and how much work the survivors can complete? In this chapter, we will map failure domains, separate availability from durability, and examine the state, capacity, and authority needed for active-active and active-passive operation.

17.1.1 · Failure domains and fault isolation

Two workers use the same disk and credential source. Their processes are separate, but one disk failure or credential change can stop both. A failure domain groups parts exposed to a common cause.

List those shared causes as well as the separate processes. If both workers run on one host, losing that host removes both workers' CPU and disk access. Even on separate hosts, a credential rotation can deny both access to the same provider. A process boundary protects against some failures while leaving these others shared.

Fault isolation limits how a failure spreads. For example, separating the resources used by payment and receipt delivery can keep a stalled email provider from occupying every payment worker. Map the shared hosts, zones, dependencies, and administrative controls before choosing a boundary. Separation costs

capacity and operating work, so choose it according to the harm and recovery objective, then test the shared dependencies as well as individual processes.

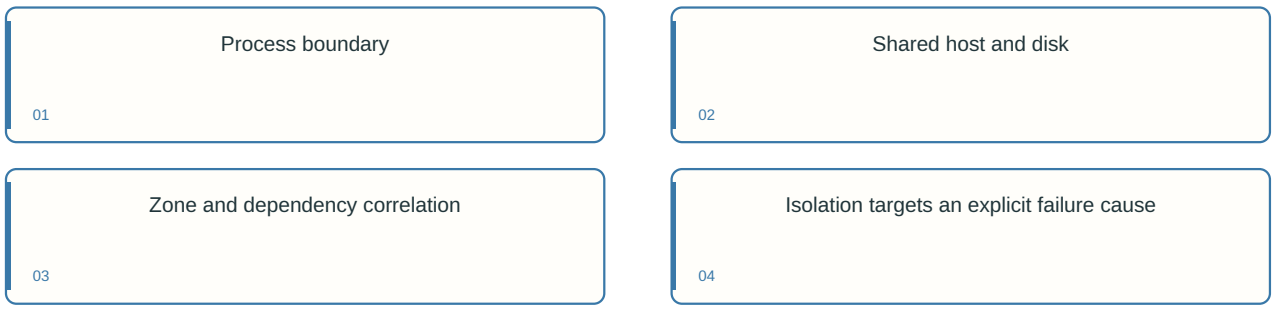


FIG. 17.1.1 **Failure domains and fault isolation · the example**

Failure domains identify shared causes; isolation limits their spread through resources and dependencies.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.1.2 · Availability vs durability

A database refuses requests during recovery but keeps all committed rows. Users cannot obtain service during this period, although their committed data remains recoverable. A web application can show the opposite problem: it keeps answering requests but loses writes that it had accepted into memory before a restart.

Availability concerns whether users can obtain the required service when they need it. Durability concerns whether accepted, committed information remains recoverable through the stated failures. We need separate evidence for these promises. A successful response does not establish that a write will survive a restart; a recoverable database does not establish that it can serve requests now.

Define which requests count and what a useful response means, then state the data-loss tolerance and failures covered. During a careful restore, stopping service may help preserve the correct history. Traffic failover may restore access, while backups may recover data. Neither measure alone establishes the whole reliability contract.

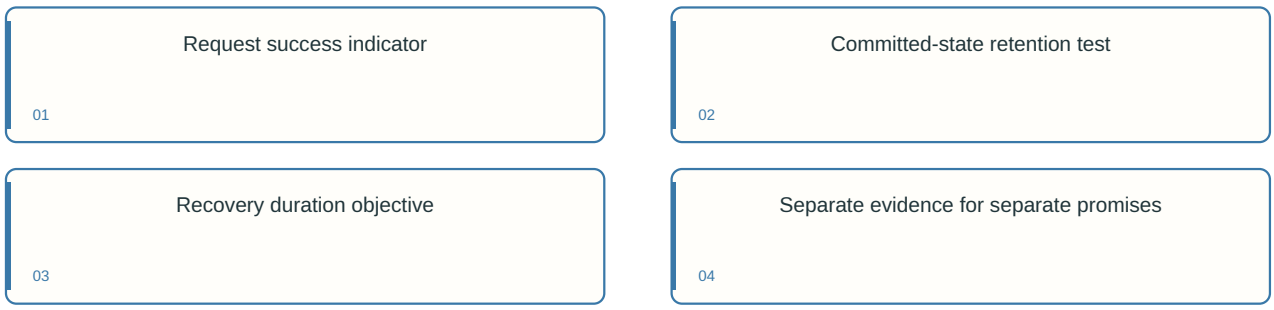


FIG. 17.1.2 **Availability vs durability · the example**

Availability concerns useful serving; durability concerns committed data surviving the specified failure model.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.1.3 · Redundancy

Two instances each operate at 70% of their individual capacity. If one fails, the remaining instance cannot absorb both workloads at the same rate. Redundancy provides additional instances or copies, but it needs enough surviving capacity to carry the promised workload.

For illustration, assume each instance can handle 100 requests per second while meeting the response-time target. Two running at 70% carry 140 requests per second together. After one fails, the survivor can complete only 100 at that assumed capacity. The remaining forty requests per second need a permitted reduced-service or rejection policy. A queue can absorb a short burst, but it will keep growing if this excess continues.

The replacement also needs usable state, independent failure placement, failure detection, and traffic movement. Shared bad configuration can disable every replica at once. Test takeover and restoration under the expected load. Spare resources help when the system can find them, direct work to them, and preserve the required state during the transition.

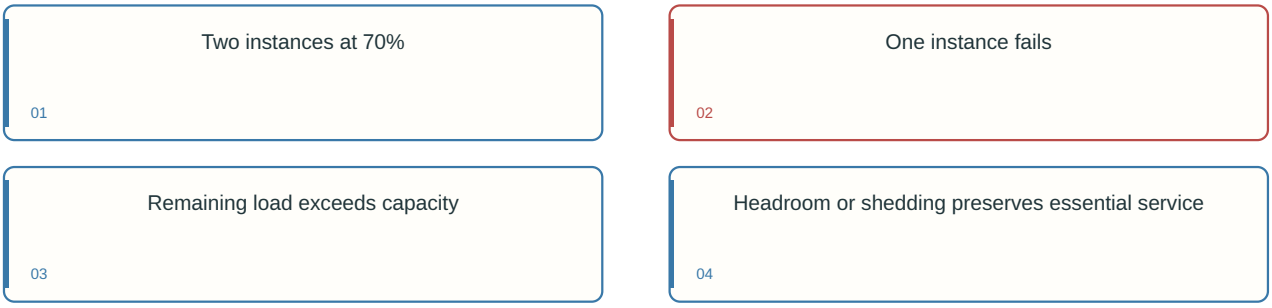


FIG. 17.1.3 **Redundancy · the example**

Redundancy requires independent failure exposure, valid state, usable takeover, and enough surviving capacity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.1.4 · Active-active vs active-passive

In active-active operation, several locations or instances serve the selected function at the same time. Active-passive keeps a standby ready for takeover. The choice can differ by layer: application instances may all serve requests while one database leader owns writes.

A standby may hold a data copy without accepting authoritative writes. To promote it, we must establish whether its history meets the data-loss policy and prevent the earlier owner from writing again. This prevention is called fencing. In an active-active arrangement, define which actions different participants may accept concurrently and how conflicting changes are prevented or resolved.

A passive standby can have configuration drift or warmup problems that appear only during takeover. Multiple active instances need routing and suitable coordination. For either arrangement, define failure detection, surviving capacity, and the return after repair. Test that the replacement can reconstruct usable state and serve real requests. The operating model includes these transitions as well as the normal traffic arrangement.

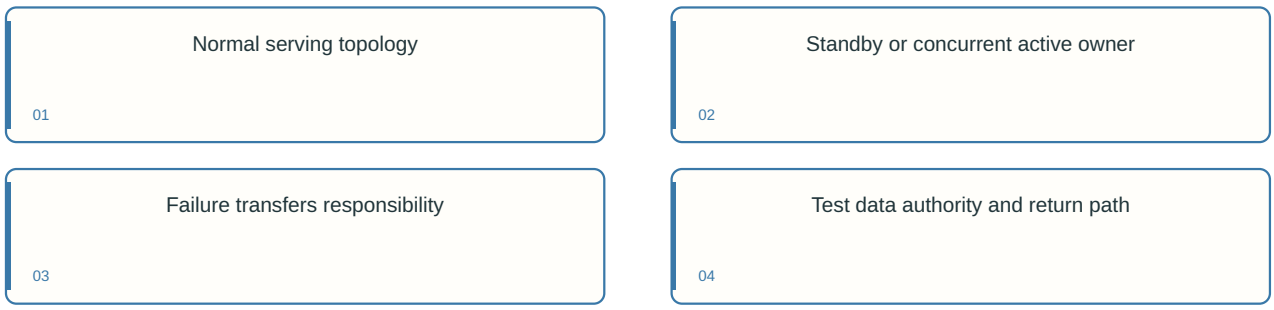


FIG. 17.1.4 **Active-active vs active-passive · the example**

Active-active and active-passive assign serving roles for a particular function, each needing tested transitions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How can a disaster drill restore infrastructure yet fail the business recovery requirement?

Machines can run while restored data is too old for the RPO, backups lack usable keys, accepted jobs are missing, or competing owners can still write. Validation and traffic return may also push useful service beyond the RTO even when machine startup is fast.

Test the full business journey with its required state, identities, authority, credentials, and recovery objectives. Include unresolved effects and catch-up work. A successful drill provides evidence for the scenario it exercised, rather than every possible disaster.

Remember: Redundancy protects the stated service only when survivors have independent failure placement, usable state, sufficient capacity, and a tested path to take over.

Try applying it: Two instances each serve 70% of their measured capacity. After one fails, what must change for the survivor to preserve the required service?

Unit 17 exercises → · Unit 17 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

2. Disaster recovery planning guide — Google Cloud ↗

   Official guidance · accessed 21 Oct 2026. Recovery planning, distinct from a measured achievement of an objective.

   Access reviewed · 21 October 2026

3. Reliability Pillar — AWS Well-Architected ↗

   Official guidance · 2024 revision at access. Reliability review lenses and operations; provider-neutral teaching uses the concepts.

   Access reviewed · 21 October 2026

Retries, circuit breakers & isolation

Prerequisites: 17.1 · Failure domains & redundancy 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

During peak traffic, the payment provider slows down. Calls retain connections while they wait, new requests queue, and callers begin retrying. Soon the application has few resources left even for requests whose dependencies are healthy.

Repeating the calls cannot restore the missing capacity. We need to decide how long work remains useful, which failures justify another attempt, and how much recovery traffic the service can afford. We will follow deadlines, retry budgets, circuit breakers, resource isolation, and rate limits to see where each can interrupt this sequence.

17.2.1 · Timeouts

A client permits two seconds for an operation. Spending three seconds waiting for a database connection would exceed that allowance before the query even starts. An overall deadline marks the time after which the request is no longer useful to its caller.

Record the deadline when the request begins. Before each stage, calculate how much time remains. Waiting for a pooled connection, making remote attempts, backing off between attempts, and returning the reply all consume the same allowance. A stage timeout limits a particular wait; it should fit within the remaining overall time. Giving every stage a fresh full timeout can keep work running long after the caller has stopped waiting.

Pass the remaining time downstream and cancel or clean up owned work when possible so it releases resources. Record which stage timed out to distinguish waiting for a connection from executing a query. A timeout does not prove that an external write failed: the provider may have completed it before its reply was lost. That uncertain effect needs recovery even though the caller's wait has ended.

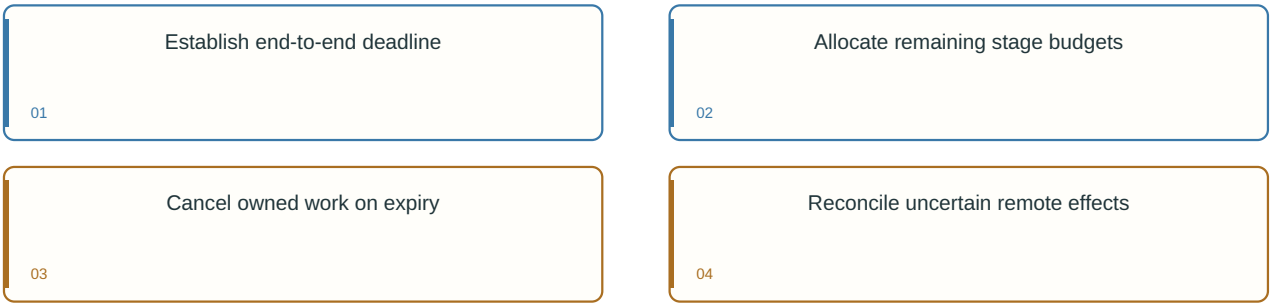


FIG. 17.2.1 **Timeouts · the example**

Timeouts bound waiting; overall deadlines include acquisition and stage waits without proving remote nonexecution.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.2.2 · Retries, exponential backoff, jitter

A dropped connection during a safe lookup may justify another attempt. An invalid payment amount will not improve if the same request is repeated. Before retrying, decide whether the failure is likely to be temporary and whether repeating this operation is safe.

Backoff increases the wait between attempts. With exponential backoff, the wait grows by a factor after each failure, usually up to a chosen maximum. Jitter varies these waits so callers are less likely to retry together. These techniques change timing; a maximum attempt count, an overall retry budget, and a deadline bound the extra work.

If an effect may already have happened, retain a stable operation identity and use a repetition or lookup rule supported by the receiver. An identifier alone cannot make an arbitrary payment call safe to repeat. Respect relevant retry guidance and decide which layer owns retries. Several independent policies can multiply attempts around one request. Include retry traffic in capacity planning, because it competes with new work and recovery.

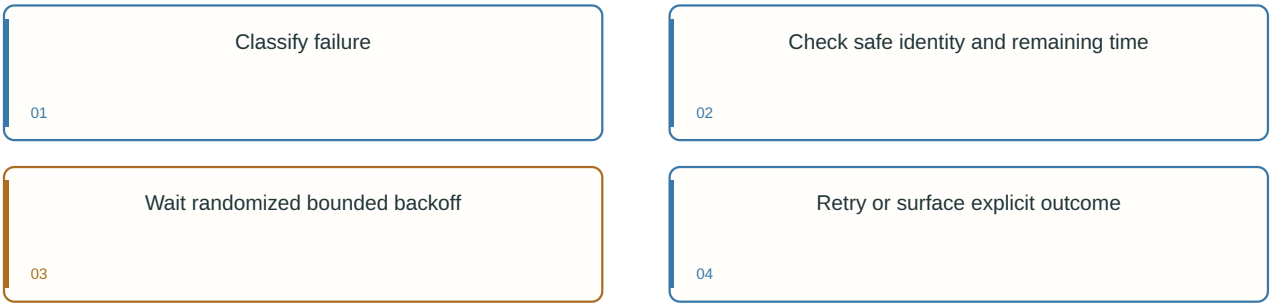


FIG. 17.2.2 **Retries, exponential backoff, jitter · the example**

Retries spend bounded capacity and time, with backoff, jitter, classification, and safe effect identity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.2.3 · Retry storms

Four nested layers each allow three attempts, including the initial attempt. One logical request can then create up to $3^4 = 81$ downstream attempts. During a dependency outage, this amplification can consume the capacity that might otherwise complete useful work.

Draw a call tree. The outer layer can make three attempts. Each can cause three attempts at the next layer, giving nine. Repeating this through four layers gives $3 \times 3 \times 3 \times 3 = 81$ attempts at the bottom. This is a worst-case allowance: success, a deadline, or another limit may stop the tree sooner.

Coordinate retry ownership and enforce a budget across the whole path. Jitter spreads attempts in time but does not reduce their configured total by itself. Bound admission and suppress redundant work where possible. Measure original user intents separately from attempts and useful completions; a rise in traffic during an outage may come from retries rather than new users. Leave capacity for successful work and recovery.

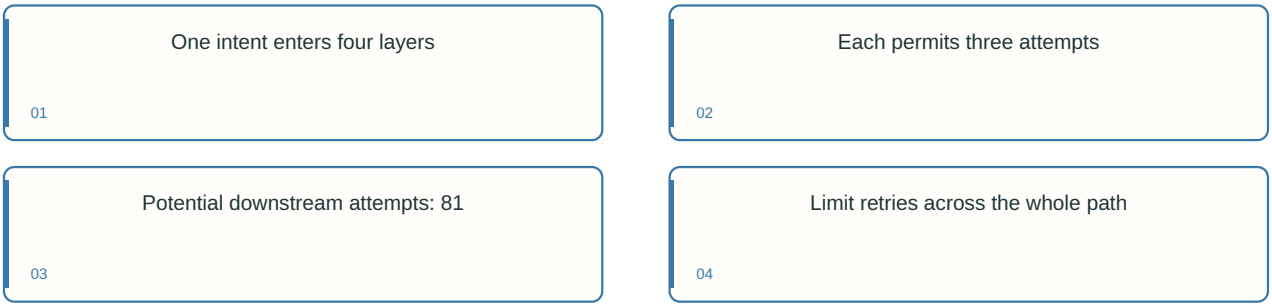


FIG. 17.2.3 **Retry storms · the example**

Retry storms amplify failure traffic; timing randomisation alone does not bound total attempts.

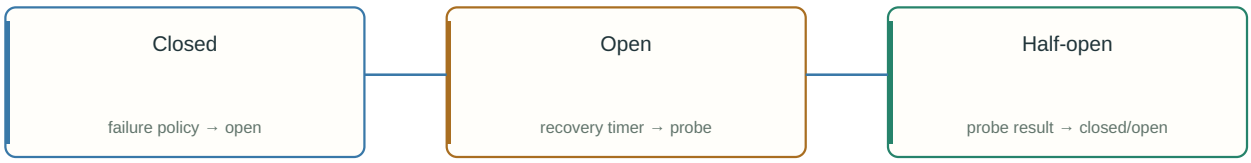
Author-created illustration. Numbers are illustrative unless stated otherwise.

17.2.4 · Circuit breakers

A dependency repeatedly fails. A circuit breaker can temporarily reject new calls, preventing each caller from holding resources while waiting for another likely failure. Its common states are closed, open, and half-open.

While closed, the breaker permits calls and observes their outcomes. When its configured failure rule is met, it opens and rejects selected calls promptly. After a wait, it becomes half-open and permits a limited number of recovery probes. Successful probes can close it again; failed probes can reopen it. Define the outcomes and thresholds that control these transitions.

Bound probes across callers and avoid having all instances probe simultaneously after a shared timer. Otherwise, recovery can trigger another surge. The breaker controls resource expenditure; it does not repair the dependency or choose an acceptable business result. A failed authorisation call must not become a fabricated payment approval. The application still needs a deliberate failure, pending, or degraded response whose meaning fits the operation.



Transitions depend on the labeled protocol conditions; arrows are state transitions.

FIG. 17.2.4 **A circuit breaker bounds futile calls**

Closed permits calls, an observed failure policy opens the breaker, and bounded half-open probes test recovery. The breaker does not fabricate a successful business result.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.2.5 · Bulkheads

Receipt email stalls and occupies every worker. Payment authorisation now cannot obtain a worker even though its provider is healthy. Bulkheads reserve or separate resources so one kind of work cannot consume the entire allowance needed by another.

Give receipt delivery its own bounded workers and connections, with a separate allocation for payment work. If email stalls, it can occupy only its allocation. Bound its waiting queue too, so a growing receipt backlog does not exhaust memory. Payment work can then continue within its reserved capacity, provided the resources it still shares remain available.

Check those shared limits: two worker pools may still exhaust one database, network link, or host. Their combined limits must fit the shared capacity. Too much partitioning can leave resources idle in one pool while another rejects work, so choose boundaries according to workload and harm. Observe saturation and rejection in each pool. Isolation must cover the resource through which the failure would spread.

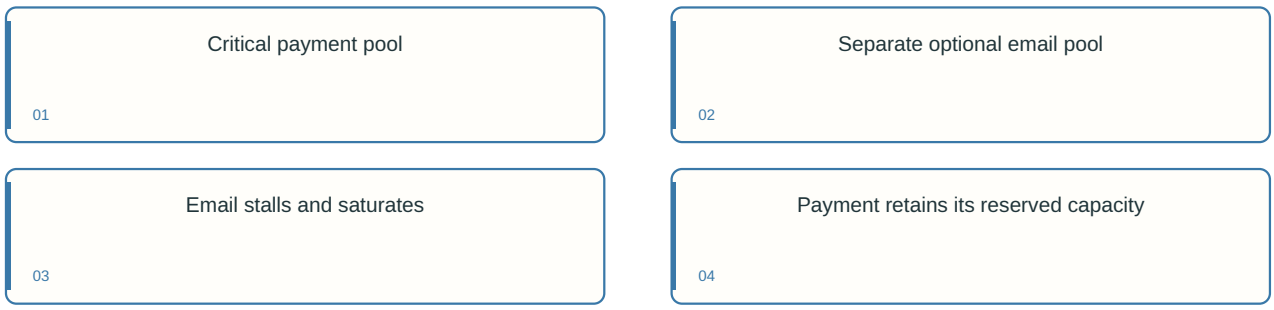


FIG. 17.2.5 **Bulkheads · the example**

Bulkheads partition relevant resources so one workload cannot consume another's entire budget.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.2.6 · Rate limiting

Suppose a caller may send occasional bursts, but we need to limit its continuing request rate. A token bucket records an allowance of tokens. Tokens accumulate at a configured refill rate up to the bucket's capacity, and each admitted request spends tokens.

Start a bucket with ten tokens and a refill rate of two per second. Ten immediate unit-cost requests can use the initial burst. If no more requests are admitted, three seconds later the balance is six tokens; it can never exceed the bucket's capacity. Coordinate the balance update and spending decision so simultaneous callers cannot both spend the last token. Excess requests may be rejected. A shaper may instead delay work to control its pace, using bounded waiting.

Define the caller identity, request cost, burst size, and total service budget. Per-tenant limits can support fairness, but their combined allowances must also protect shared capacity. If the limiter's remote state becomes unavailable, decide whether to reject, allow, or spend a previously allocated local budget. These choices have different availability and abuse consequences, so test that failure path.

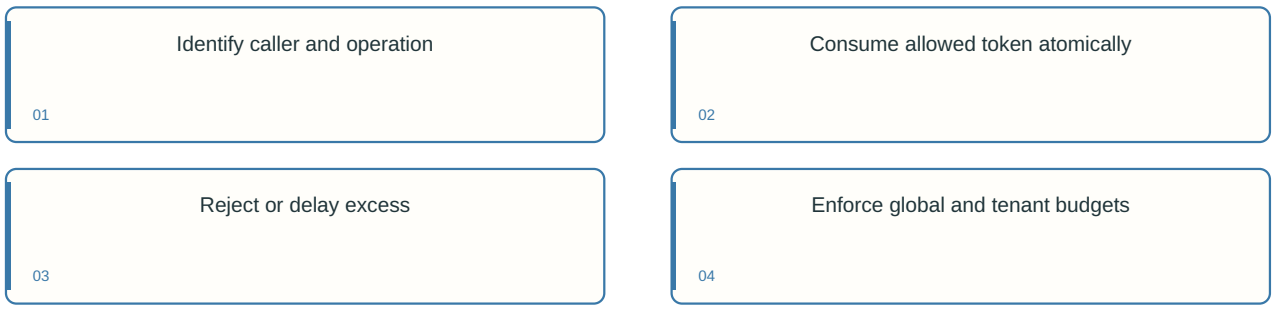


FIG. 17.2.6 **Rate limiting · the example**

Rate limiting applies a scoped pace and burst rule with an explicit state-failure policy.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How can a disaster drill restore infrastructure yet fail the business recovery requirement?

Machines can run while restored data is too old for the RPO, backups lack usable keys, accepted jobs are missing, or competing owners can still write. Validation and traffic return may also push useful service beyond the RTO even when machine startup is fast.

Test the full business journey with its required state, identities, authority, credentials, and recovery objectives. Include unresolved effects and catch-up work. A successful drill provides evidence for the scenario it exercised, rather than every possible disaster.

Remember: Deadlines bound useful lifetime; retry budgets bound repeated work; breakers, bulkheads, and rate limits protect the resources through which failures spread.

Try applying it: Four nested layers each allow three attempts. Why can jitter spread those attempts without solving their total load, and where should the overall limit apply?

Unit 17 exercises → · Unit 17 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

2. Disaster recovery planning guide — Google Cloud ↗

   Official guidance · accessed 21 Oct 2026. Recovery planning, distinct from a measured achievement of an objective.

   Access reviewed · 21 October 2026

3. Reliability Pillar — AWS Well-Architected ↗

   Official guidance · 2024 revision at access. Reliability review lenses and operations; provider-neutral teaching uses the concepts.

   Access reviewed · 21 October 2026

Shed load and degrade deliberately

Prerequisites: 17.2 · Retries, circuit breakers & isolation 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

The recommendation service is unavailable, but the catalog still works. We can show a useful product page without recommendations. If the ledger is unavailable, however, a convenient default balance cannot establish whether a payment is permitted.

The challenge is to define useful reduced service while preserving the meaning of each operation. We will choose work to shed, examine graceful degradation and fallback paths, and distinguish health checks that control restarting from those that control incoming traffic. Each decision must account for the capacity and business rules that remain.

17.3.1 · Load shedding

Under overload, a service may stop generating recommendations while preserving order status and reconciliation. Load shedding rejects selected work early enough to leave resources for higher-priority outcomes.

Rejecting before expensive parsing or remote calls can protect more capacity than rejecting after that work has already been done. Choose priorities from product and correctness requirements. Reconciliation may need reserved capacity to discover the outcomes of already accepted payments, even while optional recommendations are being refused. Return a clear overload result and bound subsequent retries so rejection does not simply create more work.

Shedding can preserve useful completions while demand exceeds capacity; it cannot serve all of an indefinitely excessive workload. More capacity, less work per request, or a changed service target may still be needed. Measure useful accepted outcomes and fairness across callers. Raw throughput can fall while essential work improves, because the service has deliberately stopped spending resources on lower-priority work.

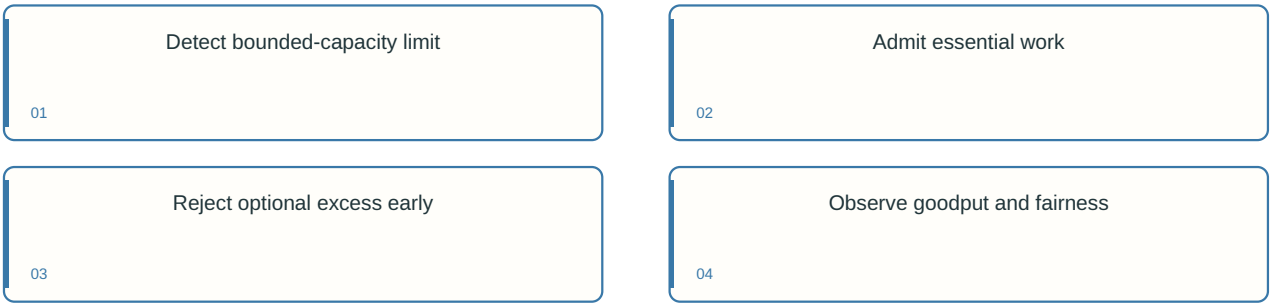


FIG. 17.3.1 **Load shedding · the example**

Load shedding rejects selected work early to preserve useful higher-priority outcomes under overload.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.3.2 · Graceful degradation

A product page can remain useful without recommendations. A sale cannot safely proceed merely because an old cached count says stock exists. Graceful degradation preserves an agreed subset of useful behaviour while keeping the rules that still matter.

Define that reduced result before the incident. If recommendations fail, the page can still show the product description and price. If the system cannot establish permission to allocate stock, it may need to pause purchase confirmation. Showing an optimistic confirmation would create a promise that the system cannot support.

Specify which data may be stale, which actions pause, and what the user sees. Then test the reduced path, including its permissions and capacity. Such paths may be used rarely in ordinary operation and fail when first needed. Catching every error and returning a default does not establish safety; the alternative outcome needs a clear meaning of its own.

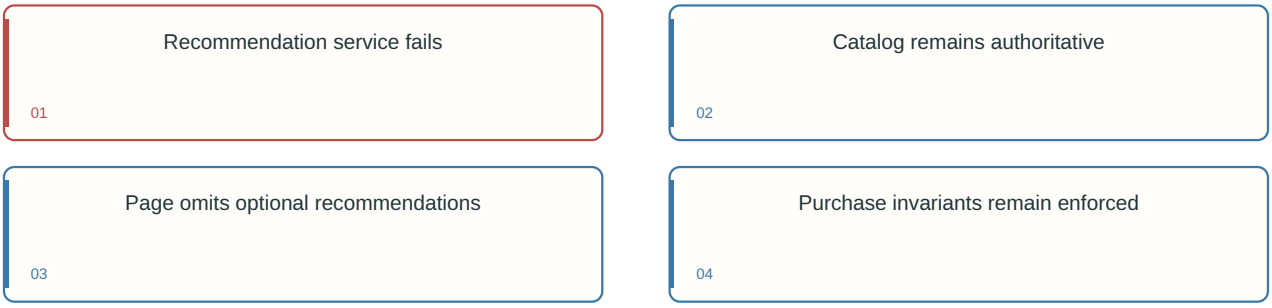


FIG. 17.3.2 **Graceful degradation · the example**

Graceful degradation preserves a permitted subset of behaviour with visible freshness and action limits.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.3.3 · Fallbacks

A cached help page may substitute for an unavailable content service. A default account balance cannot establish the state of an unavailable ledger. A fallback is an alternative behaviour whose meaning must be acceptable for this operation.

The fallback is another path through the system, with its own data age, permissions, and load limit. If every failed cache request goes to one small database, the substitute can become the next outage. Check what all callers will do together, as well as whether one fallback request succeeds. Specify how long the alternative remains acceptable and make degraded operation visible where needed.

Switching payment providers also requires care. If the first provider may already have charged the customer, sending the payment to another provider can create a second charge. The alternative must account for the first attempt's identity and uncertain outcome; changing providers does not settle that history. Evaluate a fallback as a business outcome and a recovery path, rather than treating any returned response as success.

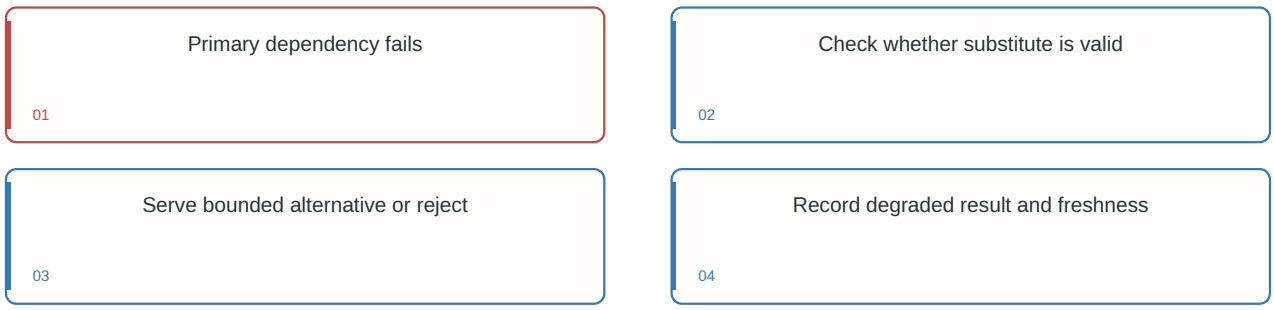


FIG. 17.3.3 **Fallbacks · the example**

Fallbacks substitute defined business outcomes, with their own access, duration, capacity, and recovery risks.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.3.4 · Health checking

A process keeps restarting because its liveness check calls an unavailable shared provider. Restarting cannot repair that provider. If every replica follows the same rule, they can all restart together and add warmup work during the outage.

Keep each probe focused on the decision it controls. Liveness asks whether the process needs restarting, for example because it is stuck. Readiness asks whether traffic should be sent here now. Removing readiness can stop new traffic, while separately arranged draining lets ongoing work finish. A startup check or allowance gives initialisation time before ordinary checks apply.

Bound probe work and avoid making every process restart depend on an external service it cannot repair. Readiness policy also needs care: marking every replica unready for one shared dependency may remove service that could still operate in a permitted reduced mode. A successful probe is evidence about that check, not proof that every critical operation meets its objective. Monitor real useful operations alongside these local signals.

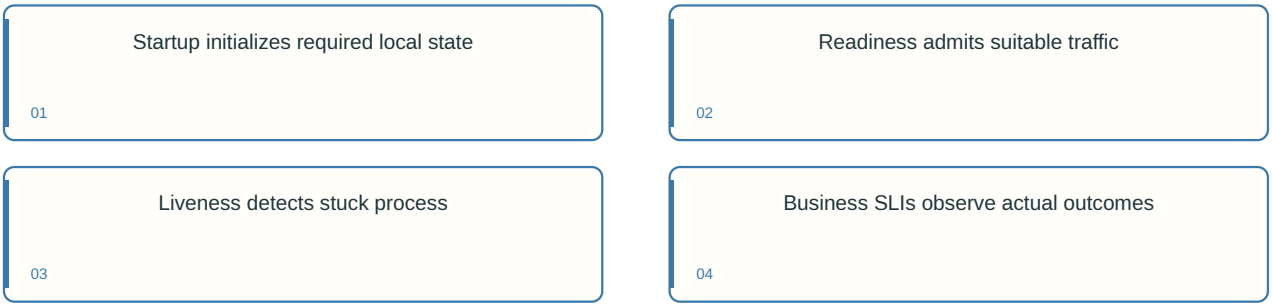


FIG. 17.3.4 **Health checking · the example**

Health probes support bounded local decisions rather than establishing the complete business service guarantee.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How can a disaster drill restore infrastructure yet fail the business recovery requirement?

Machines can run while restored data is too old for the RPO, backups lack usable keys, accepted jobs are missing, or competing owners can still write. Validation and traffic return may also push useful service beyond the RTO even when machine startup is fast.

Test the full business journey with its required state, identities, authority, credentials, and recovery objectives. Include unresolved effects and catch-up work. A successful drill provides evidence for the scenario it exercised, rather than every possible disaster.

Remember: Choose reduced service by its business meaning, and check that its permissions, freshness, and capacity remain acceptable during the failure.

Try applying it: Why can a cached help page remain useful during an outage while a default account balance cannot justify a payment?

Unit 17 exercises → · Unit 17 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

2. Disaster recovery planning guide — Google Cloud ↗

   Official guidance · accessed 21 Oct 2026. Recovery planning, distinct from a measured achievement of an objective.

   Access reviewed · 21 October 2026

3. Reliability Pillar — AWS Well-Architected ↗

   Official guidance · 2024 revision at access. Reliability review lenses and operations; provider-neutral teaching uses the concepts.

   Access reviewed · 21 October 2026

Recover, test & learn from failure

Prerequisites: 17.3 · Shed load and degrade deliberately 8 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

The replacement machines are running, yet checkout still cannot resume. A restored database is too old, an encryption key is missing, or a repaired primary attempts to write using its former authority. Infrastructure can return before the required business service does.

We will set recovery time and data-loss objectives, follow a complete restore and authority handover, and test the failure scenarios that the design promises to cover. We will then use service objectives, controlled fault exercises, and incident evidence to choose changes that reduce the causes and consequences of future outages.

17.4.1 · RTO and RPO

An incident begins at noon. Service returns at 12:20, so the interruption lasted twenty minutes. If recovery restores data only through 11:55, there is five minutes of data-loss exposure relative to the incident point.

The recovery time objective, or RTO, is the target time to restore the defined service. The recovery point objective, or RPO, specifies the acceptable gap between the incident and the recoverable data history, often expressed as time. In our example, twenty minutes and five minutes are observed results to compare with the targets. They do not tell us whether the business considers that recovery acceptable. The five-minute gap describes exposure; actual lost records depend on what was written in that interval.

Set both objectives for the business function. Then measure detection, decision, restoration, validation, and traffic return. Timing only a file copy misses much of the interruption. A faster restore may use older data, so evaluate recovery time and data-loss exposure separately.



FIG. 17.4.1 **Recovery time and recoverable data are different**

Last recoverable data at 11:55, incident at 12:00, service restored at 12:20: five minutes of data exposure and twenty minutes of recovery in this example.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.2 · Backups and restore testing

A backup survives the lost host, but its only decryption key was on that host. The bytes exist, yet the application cannot recover them. A useful backup needs the surrounding restoration resources too.

A restore needs usable bytes, required keys and credentials, compatible software, configuration, and an environment in which the application can run. Document schema versions and any retained logs needed to reach the required recovery point. Keep these resources independent enough from the failed domain to remain usable. A rehearsal can find missing pieces while ordinary service still runs.

Restore actual backups and check business invariants and reconciliation state, rather than relying only on a completed backup job or valid checksums. Live replication may repeat a mistaken deletion or corruption, while a retained historical backup may recover an earlier state. State which failure each copy protects against and test the corresponding procedure. A copy is useful recovery evidence only when the application can use it to restore the required service and history.

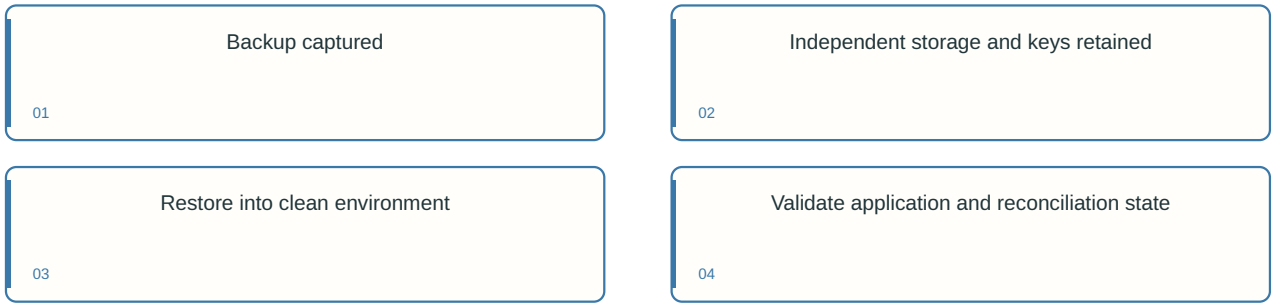


FIG. 17.4.2 **Backups and restore testing · the example**

Usable backups require independent storage, keys, compatible state, and tested application restoration.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.3 · Failover and failback

Failover moves service or write authority to a replacement. Failback returns it after repair. Both transitions need valid history, sufficient capacity, routing, and protection from conflicting owners.

Before moving writes, establish which destination holds the required history under the data-loss policy and which owner may act next. Fence the old owner so that a delayed or restarted process cannot continue writing. After repair, the old primary must catch up or be rebuilt from the new history before regaining authority. Sending traffic back too soon can expose older data or allow two competing writers.

A database promotion alone does not move every user request. Existing connections, cached addresses, and retries may continue using old paths. Test the full sequence: transfer authority, route traffic, validate useful service, and later return after repair. Include the old owner's attempted return in the test. Recovery is incomplete if that return can reintroduce incompatible writes.

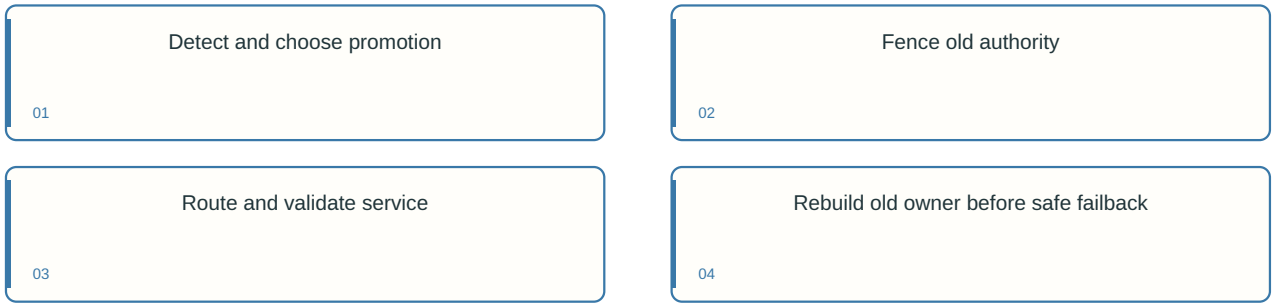


FIG. 17.4.3 **Failover and failback · the example**

Failover and failback transfer traffic or authority while preserving current history, fencing, and capacity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.4 · Multi-zone and multi-region deployments

Copies in separate zones can survive some local incidents. Copies in separate regions can cover other failures, but add propagation delay, transfer cost, data-placement concerns, and recovery work.

Ask which failure each placement should survive. Separate zones can reduce shared local infrastructure risks. A remote region can protect against a regional incident while adding distance between data copies. Waiting for a distant copy can increase write latency; not waiting can leave recent writes unavailable there when failover is needed. The chosen replication and promotion rules must meet the stated recovery objective.

Check dependencies that remain shared. A global credential change or configuration fault can still affect all regions. Plan capacity for surviving traffic and catch-up work, rather than only the normal traffic split. Geographic distance provides part of the independence. The authority protocol, usable data, and operational dependencies determine whether the remaining location can actually provide the promised service.

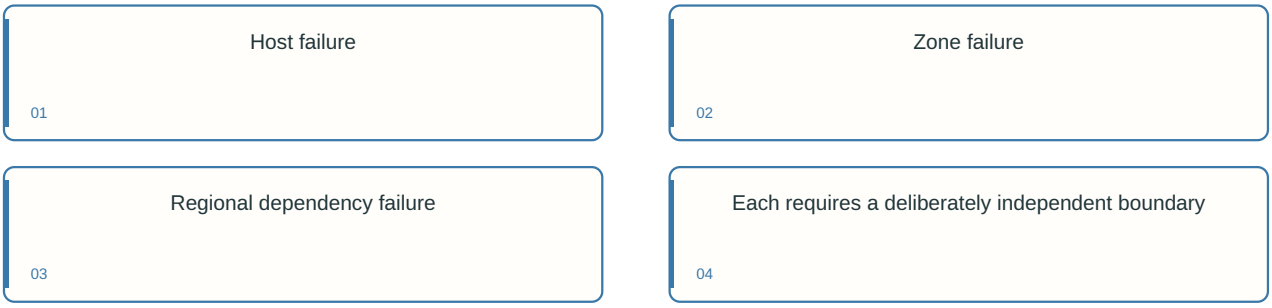


FIG. 17.4.4 **Multi-zone and multi-region deployments · the example**

Zone and region placement address different correlations while adding propagation, cost, and recovery dependencies.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.5 · Chaos engineering

Begin a fault exercise with a question: if a worker pauses past its lease, can its old completion overwrite a new result? A lease gives authority for a limited period; a pause can leave the worker acting on authority that has already expired.

State the expected invariant, introduce a bounded fault, and check the result. Pause the first worker, let a replacement gain authority, then resume the first and attempt completion. The protected store should reject the stale result. Observing that both processes eventually run again would not establish that the invariant held.

Chaos engineering tests such hypotheses by introducing controlled faults. Define the affected scope, stop conditions, observations, recovery owner, and environment appropriate to the risk. Random process killing without a hypothesis may cause harm without clarifying a mechanism. Increase scope when the evidence justifies it. A passing drill supports the tested scenario and configuration; it does not prove every future failure harmless.

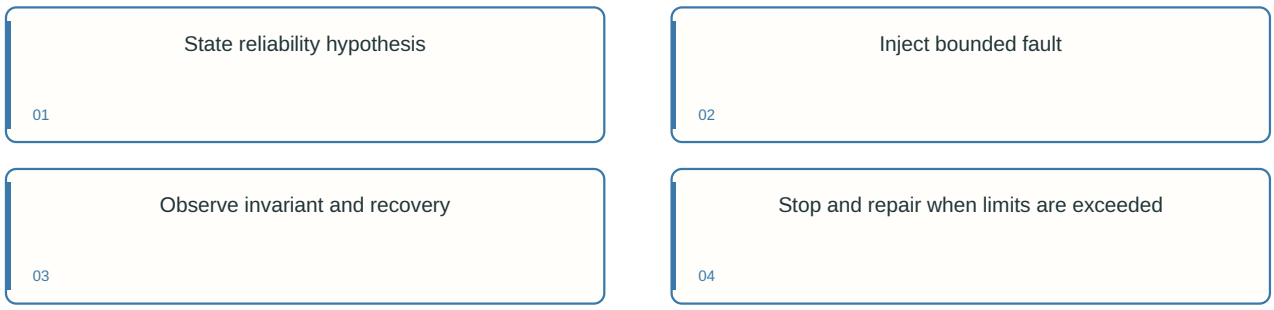


FIG. 17.4.5 **Chaos engineering · the example**

Chaos exercises test explicit hypotheses with bounded scope, observations, stop conditions, and recovery ownership.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.6 · SLO-based reliability decisions

Suppose optional calls consume most of a request's deadline and cause most failures. Removing those calls from the critical path may improve the user experience more directly than adding another region.

Use the service-level objective, or SLO, to identify the required useful outcome. Break down which eligible requests miss it and why. If an optional dependency dominates those failures, removing its wait may address the measured problem. Compare that change with the costs and new failure paths of another region. The error budget—the amount of failure allowed by the objective over its measurement period—helps show how quickly current failures are consuming the allowance.

Keep important low-volume operations visible. Their failures can disappear inside a denominator dominated by cheap successful reads, so separate indicators where the harm differs. More redundancy has cost and complexity; choose it for a failure mechanism it can improve. An error budget helps prioritise work, but a small event count does not make a severe payment or data-loss incident harmless.

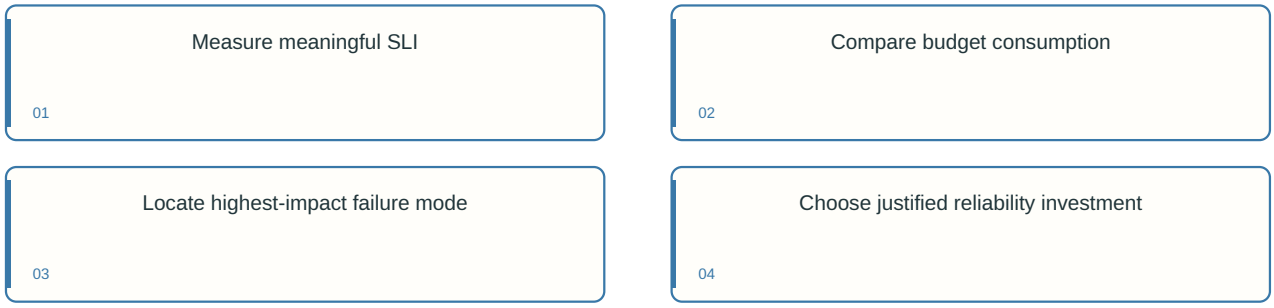


FIG. 17.4.6 **SLO-based reliability decisions · the example**

SLO evidence guides reliability investment without hiding critical low-volume harm inside a broad denominator.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.7 · Cascading failures

A slow provider holds connections. Waiting requests accumulate, callers retry, and the extra work overloads previously healthy components. This feedback creates a cascading failure: an initial problem spreads through the resources and dependencies that connect the system.

The first slowdown reduces useful completions. Waiting work retains resources for longer, so fewer resources remain for new requests. Callers then retry, adding demand to the same reduced capacity. A healthy component can become overloaded even though it had no original fault. Interrupting this loop requires reducing admitted or repeated work and releasing scarce resources, while preserving an explicit outcome for obligations already accepted.

Use deadlines, bounded admission, retry budgets, isolation, and permitted degradation at the resource boundaries involved. Leave capacity for reconciliation and recovery. A breaker that sends every failed request to one fallback can start another cascade. Trace the original and recovery paths under a shared failure; testing dependencies one at a time can miss the interaction that turns a modest slowdown into a widespread outage.

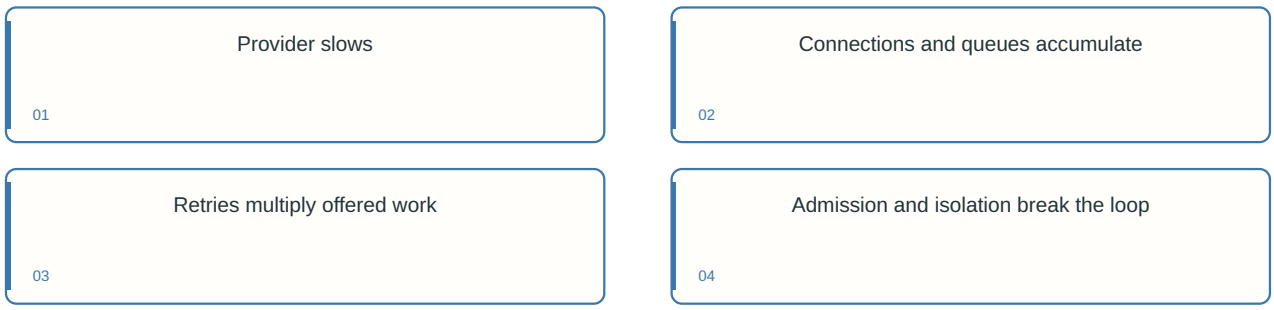


FIG. 17.4.7 **Cascading failures · the example**

Cascading failures spread through resource pressure and retries, including potentially overloaded fallback paths.

Author-created illustration. Numbers are illustrative unless stated otherwise.

17.4.8 · Incident analysis and prevention

After an incident, reconstruct what happened, which conditions allowed it, how it was detected, and what helped or delayed recovery. Stopping at “human error” leaves unanswered why one action could affect every replica.

A timeline separates the initiating event from the conditions that let harm spread. A mistaken command may be the trigger; broad permissions and shared placement may explain its reach. Connect corrective work to those observed mechanisms. Scoped credentials can reduce exposure, a tested restore can improve recovery, and bounded retries can limit amplification. The incident evidence should explain why each action is useful.

Give each action an owner and a way to verify completion. Replaying the relevant failure sequence can show whether the changed system now limits the harm. An added alert helps only when it leads to a useful response. The aim is to reduce recurrence or impact by changing the system and its operating conditions, with evidence that the change addresses what happened.

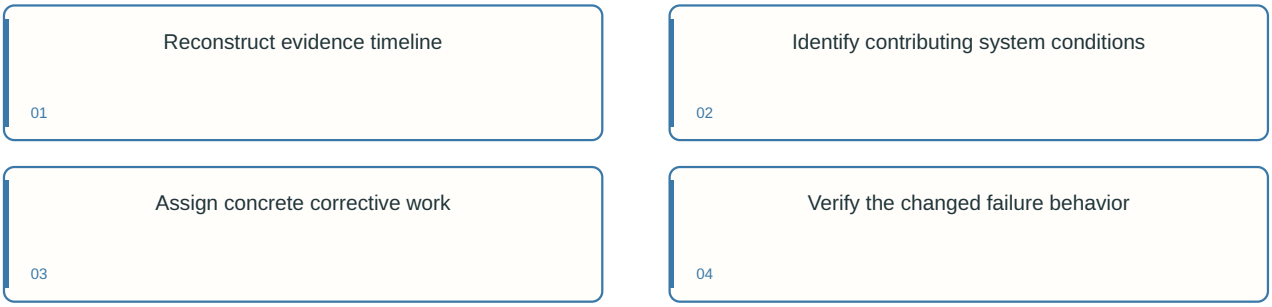


FIG. 17.4.8 **Incident analysis and prevention · the example**

Incident analysis links contributing mechanisms to verified corrective changes without substituting blame for explanation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · RETRY AMPLIFICATION

```
attempts_per_layer = 3
independent_retry_layers = 4
assert attempts_per_layer ** independent_retry_layers == 81
```

The assertion calculates $3^4 = 81$ permitted downstream attempts in the nested worst case. The first layer can make three attempts; each can cause three at the next layer, giving nine, then twenty-seven, then eighty-one. Each layer's three attempts include its initial attempt. Actual execution can stop earlier after success or when another limit is reached.

The code checks the arithmetic, not an outage policy. Use an overall retry budget and deadline to bound the whole path, protect capacity for useful work and recovery, and retain operation identity under a supported repetition rule. Jitter changes when attempts occur; it does not reduce the configured total by itself.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · How can a disaster drill restore infrastructure yet fail the business recovery requirement?

Machines can run while restored data is too old for the RPO, backups lack usable keys, accepted jobs are missing, or competing owners can still write. Validation and traffic return may also push useful

service beyond the RTO even when machine startup is fast.

Test the full business journey with its required state, identities, authority, credentials, and recovery objectives. Include unresolved effects and catch-up work. A successful drill provides evidence for the scenario it exercised, rather than every possible disaster.

Remember: Recovery restores useful service, required history, and valid authority. Measure the whole journey, then use drills and incident evidence to test specific improvements.

Try applying it: A drill brings machines back quickly but restores only old data. Which recovery objective may still be missed, and what else must be checked before declaring recovery?

Unit 17 exercises → · Unit 17 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

2. Disaster recovery planning guide — Google Cloud ↗

   Official guidance · accessed 21 Oct 2026. Recovery planning, distinct from a measured achievement of an objective.

   Access reviewed · 21 October 2026

3. Reliability Pillar — AWS Well-Architected ↗

   Official guidance · 2024 revision at access. Reliability review lenses and operations; provider-neutral teaching uses the concepts.

   Access reviewed · 21 October 2026

PART 5 / UNIT 18

Security Architecture and Privacy

A document service serves two organisations. An administrator from tenant A signs in successfully and changes a document identifier in the request to one belonging to tenant B. The service recognises the administrator, yet must still reject the read.

The running example

Design a multi-tenant SaaS application where an administrator must never gain access to another organization's data.

Unit 18 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

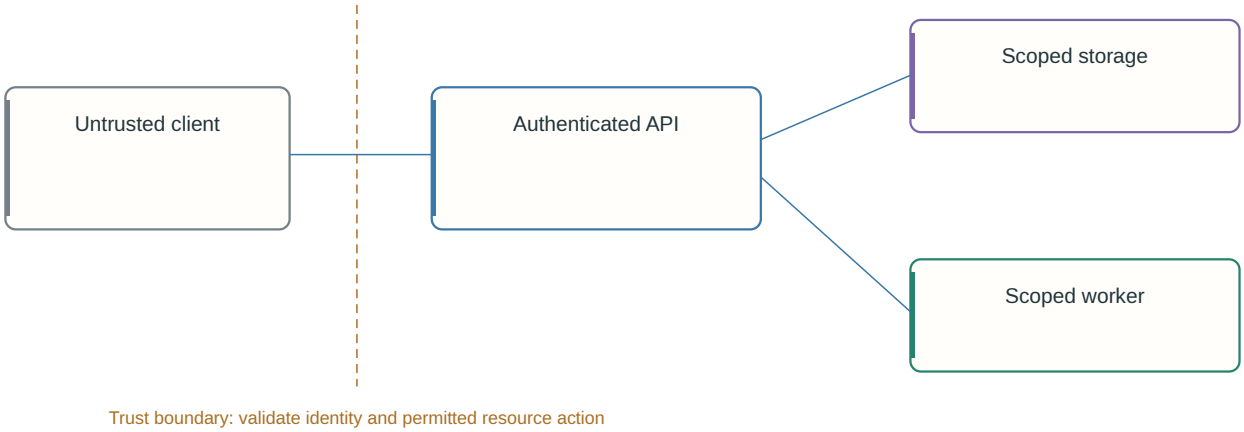


FIG. 18.0.1 **Tenant identity must survive every trust boundary**

An authenticated tenant context scopes storage queries, jobs, object capabilities, and search. Client-provided tenant labels are not independent authority.

Author-created illustration. Numbers are illustrative unless stated otherwise.

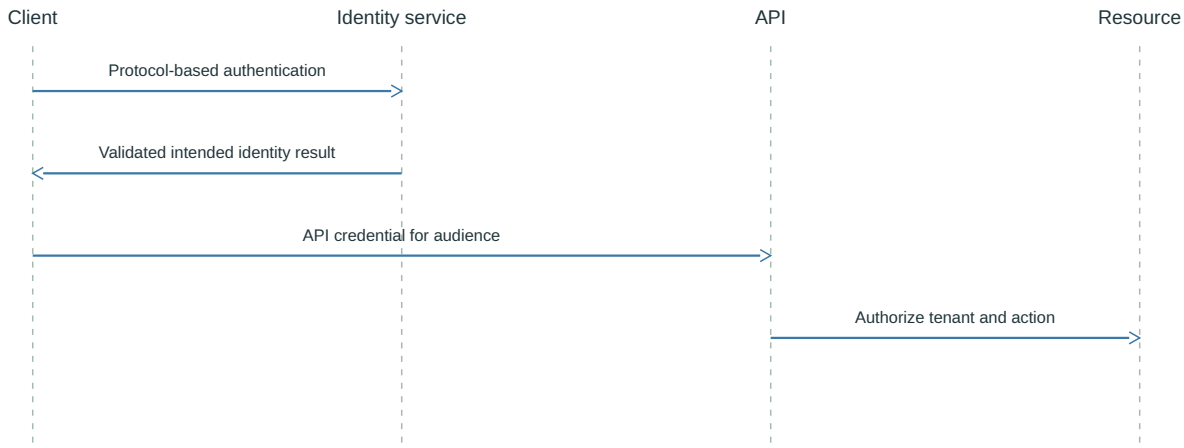


FIG. 18.0.2 **authentication sequence**

Authentication, token validation, and resource authorization are distinct protocol and enforcement decisions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

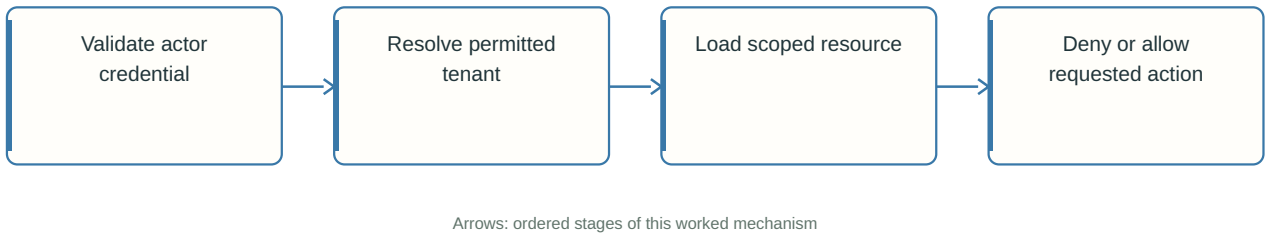


FIG. 18.0.3 **authorization decision flow**

A valid token never grants arbitrary access to a guessed resource identifier.

Author-created illustration. Numbers are illustrative unless stated otherwise.

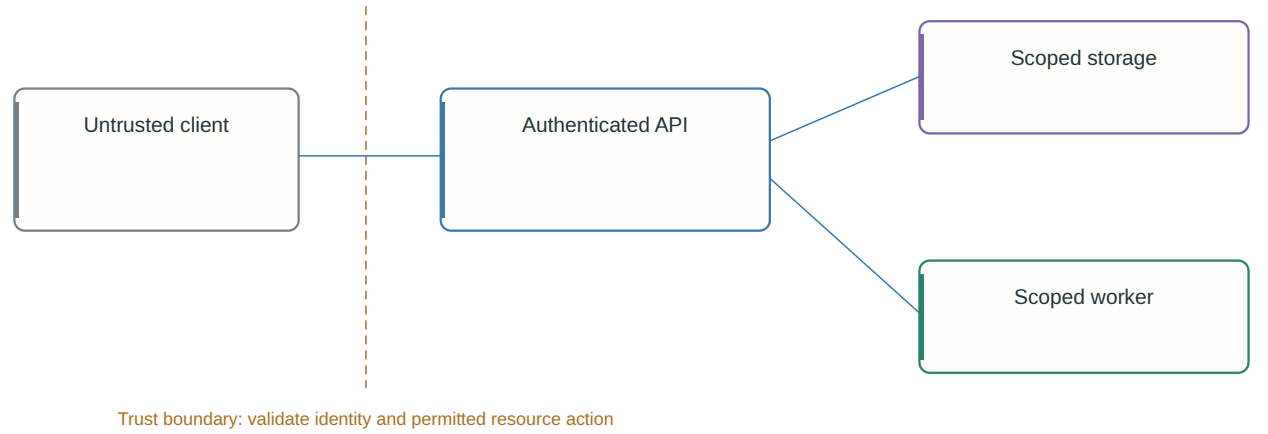


FIG. 18.0.4 **Tenant identity must survive every trust boundary**

An authenticated tenant context scopes storage queries, jobs, object capabilities, and search. Client-provided tenant labels are not independent authority.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Threats, identity & authorization

Prerequisites: Networking Foundations for System Design, Application Architecture and Domain-Driven Design, API Design and Communication Contracts, Data Models and Storage Technologies

7 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A document service serves two organisations. An administrator from tenant A signs in successfully and changes a document identifier in the request to one belonging to tenant B. The service recognises the administrator, yet must still reject the read.

We will follow this request from the browser through identity validation and the resource permission check. Along the way, we will distinguish login from delegated access, connect roles to their tenant scope, and decide how much authority a worker needs. The aim is to explain which evidence permits a particular action, rather than assuming a successful login permits every later request.

18.1.1 · Threat modeling

Begin with a document service's valuable assets: private files, credentials, processing capacity, and trustworthy audit records. The organisations using this shared service are its tenants. Each tenant's administrators should manage that organisation's documents without gaining access to another's.

Take one asset, a private PDF. An unauthorised reader might guess its identifier, reuse an old download link, or find its contents in a worker's logs. An attacker might also submit files that exhaust the extraction workers. These paths cause different harms and need different controls.

Threat modelling connects a possible misuse to the asset at risk, the actor's actual privileges, a control, and a way to check that control. Draw how the document travels through the service and where the assumptions about its caller or contents change. For the guessed-identifier path, require a resource

ownership check and test a request from another tenant. Prioritise plausible harmful paths; the drawing is useful when it helps explain how a particular harm is prevented.

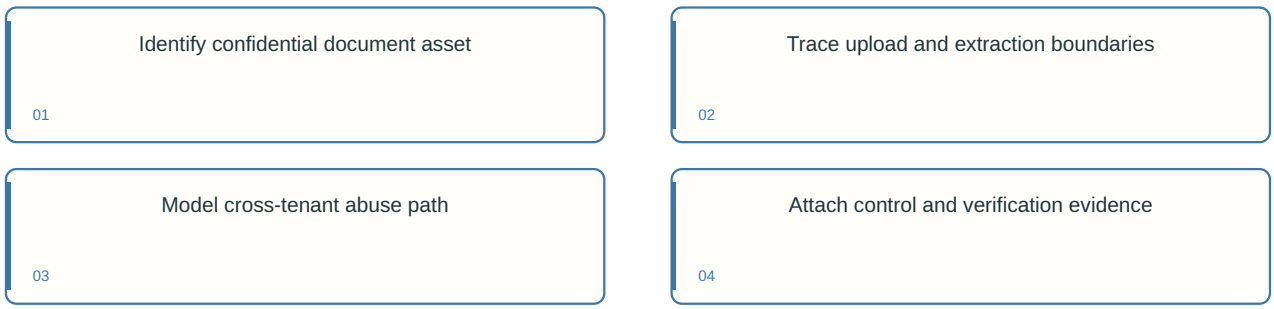


FIG. 18.1.1 **Threat modeling · the example**

Threat modelling connects assets, actors, abuse paths, and impact to controls and concrete tests.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.1.2 · Trust boundaries

A browser submits a document, a worker reads it from storage, and an operator changes production settings. Each crosses a boundary with different expectations about identity, data, and authority. A trust boundary marks where those expectations change.

At the browser boundary, a submitted tenant identifier is a claim made by the caller. The application must establish the caller's identity and permitted tenant before using that claim to select private data. If a trusted gateway forwards identity headers, configure which gateway may supply them and prevent a direct caller from impersonating it.

The worker boundary raises another question. A file may have been uploaded by an authenticated user, but its contents remain untrusted. Parsing it may require restricted execution, limited storage access, and limits on time or memory. An internal job message also needs a trustworthy job identity and scope. State what evidence each receiver relies on and what it must validate before performing its sensitive operation.

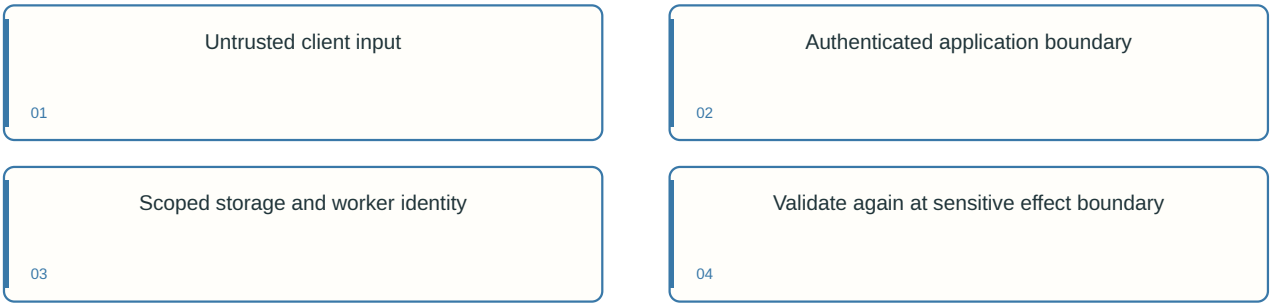


FIG. 18.1.2 **Trust boundaries · the example**

Trust boundaries mark changes in identity, authority, and confidence that require appropriate validation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.1.3 · Authentication vs authorization

An administrator of tenant A signs in and asks for tenant B's document. The identity can be genuine while the requested action is forbidden. Authentication establishes who the actor is. Authorisation checks whether that actor may perform a particular action on a particular resource.

Follow the request in order. Validate the credential, resolve the actor's permitted tenant from trusted membership information, and load the document within that scope. Then check whether the actor may perform the requested action. An “administrator” label is incomplete unless we know which organisation it applies to. A tenant value supplied in the request cannot grant membership by itself.

Deny access unless the policy permits it. Apply the relevant resource and action check to reads, edits, exports, and background work. Difficult-to-guess identifiers can reduce enumeration, but someone who learns an identifier must still be denied another tenant's data. Test both allowed and forbidden operations with two tenants, including documents that share the same local identifier.

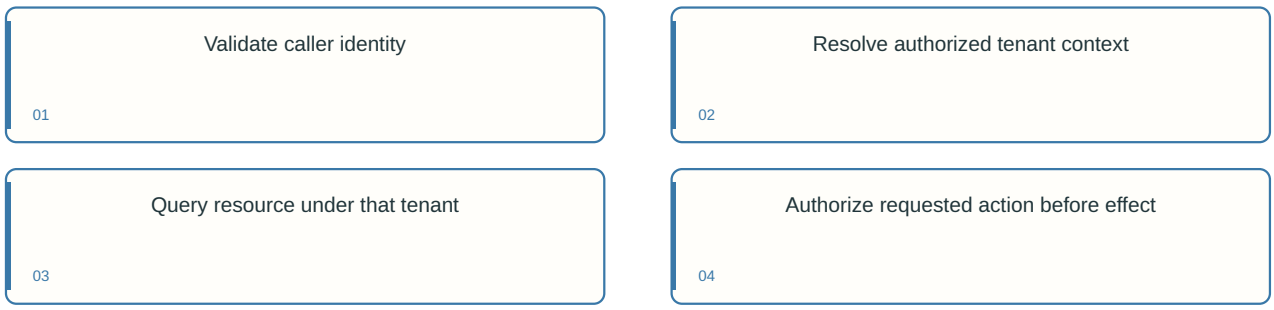


FIG. 18.1.3 **Authentication vs authorization · the example**

Authentication establishes identity; authorisation grants a particular action on a resource under policy.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.1.4 · Sessions, cookies, tokens

After a successful login, the application needs a way to recognise later requests. One design creates a random session identifier and stores the associated identity and expiry on the server. The browser sends that identifier in a cookie, and the server looks up the session on each request.

Another design uses a token carrying claims that the receiver validates, such as who issued it, its intended recipient, and its expiry. A cookie describes browser storage and transport; a token describes a credential or its representation. These are different dimensions: a signed token can also be carried in a cookie. A valid signature alone does not establish that the token is current, intended for this service, or sufficient for this action.

Protect credentials in transit and use suitable cookie attributes and cross-site request forgery (CSRF) controls. CSRF occurs when another site causes the browser to send an unwanted request using credentials it attaches automatically. Decide how logout and revocation take effect: removing a server session and waiting for an independently validated token to expire have different behaviour. Use reviewed libraries and document the chosen lifetime.

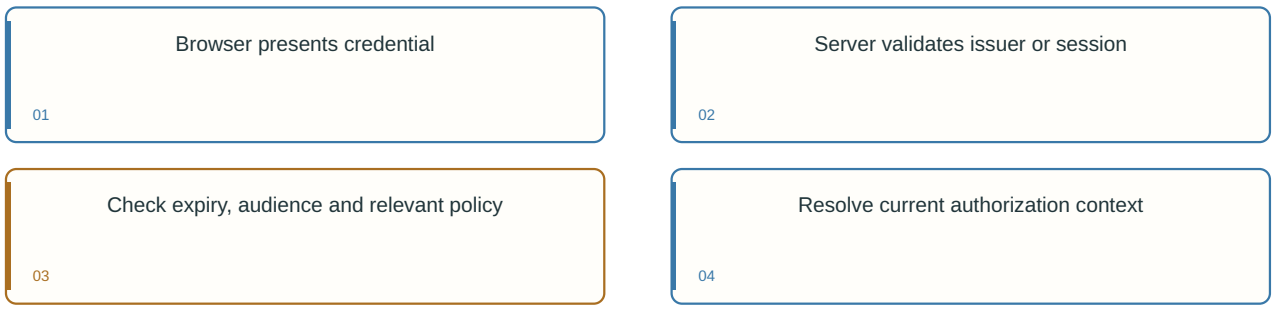


FIG. 18.1.4 **Sessions, cookies, tokens · the example**

Sessions, tokens, and cookies have different state and transport roles with explicit validation and revocation rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.1.5 · OAuth 2.0 and OpenID Connect

Suppose the document application wants permission to read a user's files from another service. OAuth 2.0 supports delegated access: the application receives authority to call a resource API within an allowed scope. Establishing which user signed in is a related but different task. OpenID Connect adds an identity layer to OAuth, including an ID token intended for the client application.

In an OpenID Connect authorisation-code journey, the browser goes to the identity provider, where the user authenticates. The provider returns a code to the registered redirect location. The application exchanges that code for the intended tokens, following the protocol. Proof Key for Code Exchange, or PKCE, binds the exchange to a verifier held by the client, helping prevent misuse of an intercepted or injected code.

The client validates the ID token for its identity purpose. The API validates its own access token for API access; an arbitrary access token is not a login assertion, and an ID token is not a substitute API credential. Use maintained libraries and current OAuth security guidance. The checks include who issued the token (issuer), whom it is intended for (audience), its expiry, and the permitted redirect location. Where the flow requires state or nonce checks, a state value links the returned response to the initiating browser request; a nonce links the ID token to that login attempt. Follow the checks specified for the chosen flow and role.

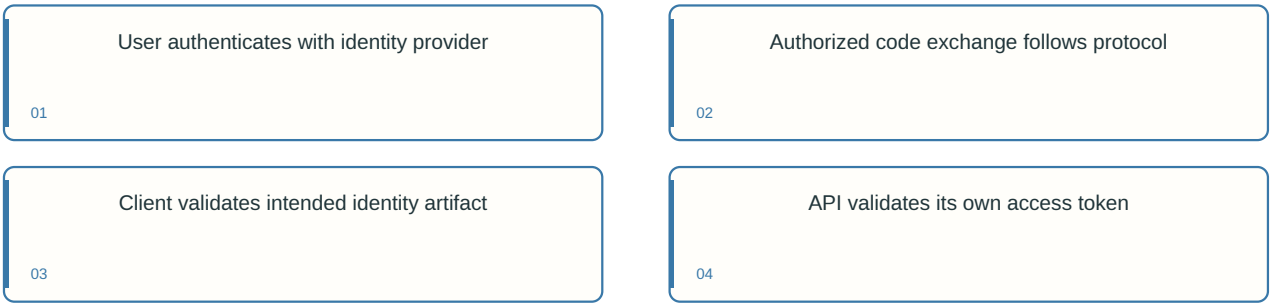


FIG. 18.1.5 **OAuth 2.0 and OpenID Connect · the example**

OAuth delegates access; OpenID Connect adds identity assertions with flow-specific validation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.1.6 · Role-based and attribute-based access control

A tenant administrator may export documents while an ordinary member may only read them. Role-based access control assigns permissions through roles such as administrator or member. Attribute-based access control evaluates facts about the actor, resource, and request, such as tenant membership, document owner, or classification.

The service can combine them. First require a role that permits export. Then require that the document belongs to a tenant in which the actor currently holds that role. “Administrator” without the tenant condition would grant much broader authority. Both the role assignment and the document's tenant must come from trusted state, rather than fields the caller can freely choose.

Broad roles are easy to describe but can grant too much. Detailed attribute rules can express narrower permissions but need understandable inputs and tests. A central policy engine evaluates rules; enforcement points are the places that act on its answer. Every relevant API and worker path must enforce the decision, including the chosen policy for membership changes while work waits in a queue.

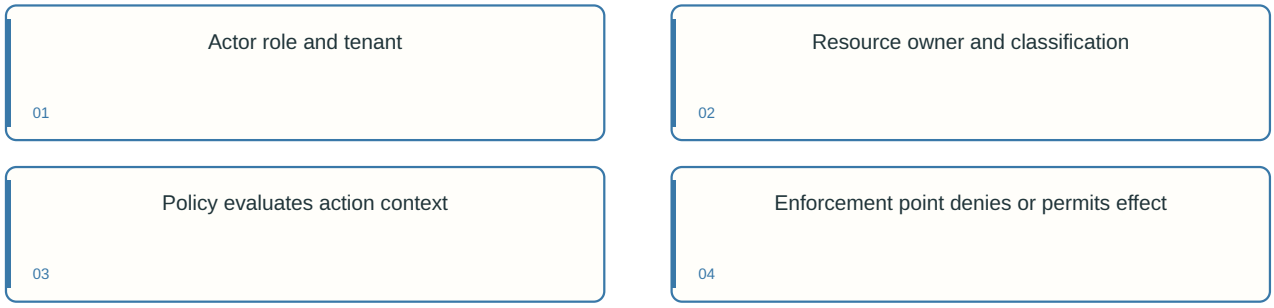


FIG. 18.1.6 **Role-based and attribute-based access control · the example**

RBAC assigns scoped role permissions; ABAC evaluates trusted attributes, with enforcement and testing required for either.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.1.7 · Least privilege

An extraction worker needs to read one input and write its output. It does not ordinarily need to list every tenant's files or alter deployment settings. Least privilege gives an identity the operations and scope required for its work.

For one job, give the worker a narrowly scoped credential or capability. A capability is authority carried by possession of something such as an object access link: whoever can use it may obtain the access it grants. Limit the objects, operations, and lifetime to the job's needs. If that worker is compromised, the reduced authority limits what the attacker can reach.

Review actual inherited permissions as well as the role's description. A narrow application API is insufficient if the same identity can read all objects directly from storage. Separate runtime, deployment, and administrative identities where appropriate. Short-lived credentials need workable refresh and failure behaviour; emergency access needs its own limits and audit. Check what the identity can actually do, then remove authority it no longer requires.

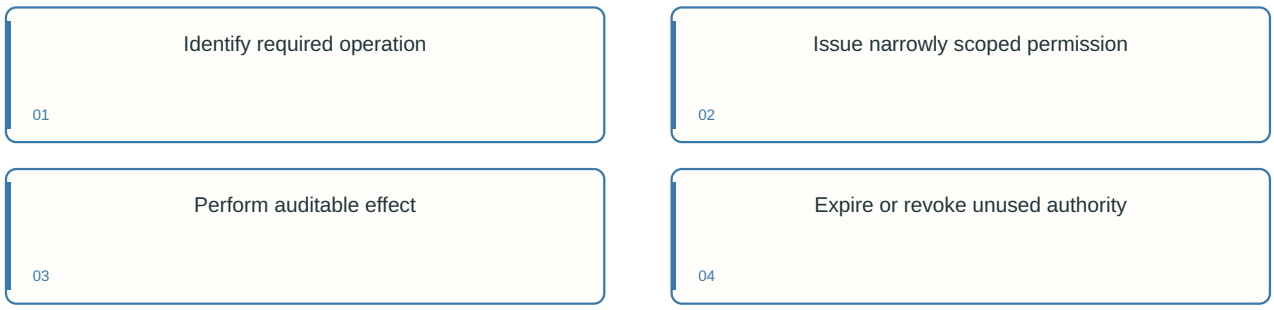


FIG. 18.1.7 **Least privilege · the example**

Least privilege limits effective identity capabilities to the work and resources actually needed.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What should revocation mean for an issued object capability and a queued export?

Define when each form of authority stops working. A queued export can check current membership before the later actions required by policy. An already issued object capability may remain usable until expiry unless the delivery mechanism supports earlier revocation. If immediate download denial is required, choose a delivery design that can enforce it rather than relying on expiry alone. Test revocation before execution, before publication, and after link issuance, and describe the resulting lifetime accurately to users.

Remember: Validate identity first, then decide whether that actor may perform this action on this resource within the permitted tenant. Preserve that distinction in roles, protocol tokens, and worker permissions.

Try applying it: A tenant A administrator has a valid session and knows tenant B document 7. Which identity, tenant, resource, and action checks should the request pass through, and where should it be denied?

Unit 18 exercises → · Unit 18 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

2. OWASP: Password Storage ↗

   Official project guidance · accessed 21 Oct 2026. Password-specific hashing and current deployment parameter guidance.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

4. OpenID Connect Core 1.0, errata set 2 ↗

   Original specification · December 2023. Identity layer and validation requirements; distinct from API authorization.

   Access reviewed · 21 October 2026

Secrets, encryption & trust boundaries

Prerequisites: 18.1 · Threats, identity & authorization 7 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A private file is encrypted, but its processing worker can read every file and obtain every decryption key. If the worker is compromised, the encryption may offer little protection against the actions that identity can already perform.

We will trace the real access paths, then examine complementary controls: limited credentials, protected transport and storage, suitable password hashing, key custody, and network boundaries. We will also rotate a credential and consider restoring old encrypted data. The goal is to identify what each control protects and what must remain available for legitimate recovery.

18.2.1 · Defense in depth

A private document can be protected by tenant-scoped queries, storage permissions, limited object capabilities, worker isolation, and audit records. Defense in depth uses complementary controls so that one failure is less likely to cause the full harm.

Suppose an application query accidentally omits its tenant filter. Independently scoped storage access may still prevent the worker from reading another tenant's object. An audit event can help detect an unexpected attempt, although recording the attempt does not prevent the read. State the role of each control rather than counting them as equivalent barriers.

Check whether the protections can fail together. If every layer accepts the same client-supplied tenant field without validating it, repeating the check offers little independence. Also decide what happens when a policy dependency fails: an error should not silently turn a private operation into an unrestricted one. Trace a misuse path, identify which controls can interrupt it, and verify that their results actually govern the sensitive effect.

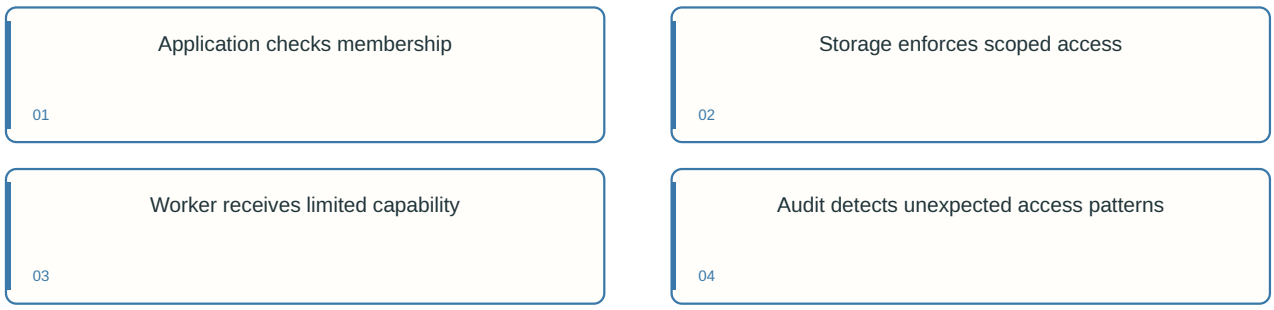


FIG. 18.2.1 **Defense in depth · the example**

Defense in depth combines complementary controls rather than repeating one unverified assumption at several layers.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.2.2 · Secrets management

A secret has a lifecycle: creation, distribution, use, rotation, and revocation. For example, a storage credential must reach the intended workers without appearing in source, logs, client bundles, or command history. Assign an owner and control who may read or replace it.

Rotation introduces a new credential and moves legitimate users of the old credential to it. A planned change may briefly allow both versions while running instances refresh. Observe that the new version works, retire the old one, and verify that the old credential is rejected. Revocation is the step that makes earlier authority unusable under the system's enforcement rules. A suspected compromise may require faster retirement than an ordinary planned change.

Managed identities or short-lived credentials can reduce stored secrets, but the service still needs a way to obtain initial authority and handle renewal failures. A secret-management service provides facilities; the application owns the operating plan. Encrypting a secret file adds little separation if every reader can also obtain its decryption key.

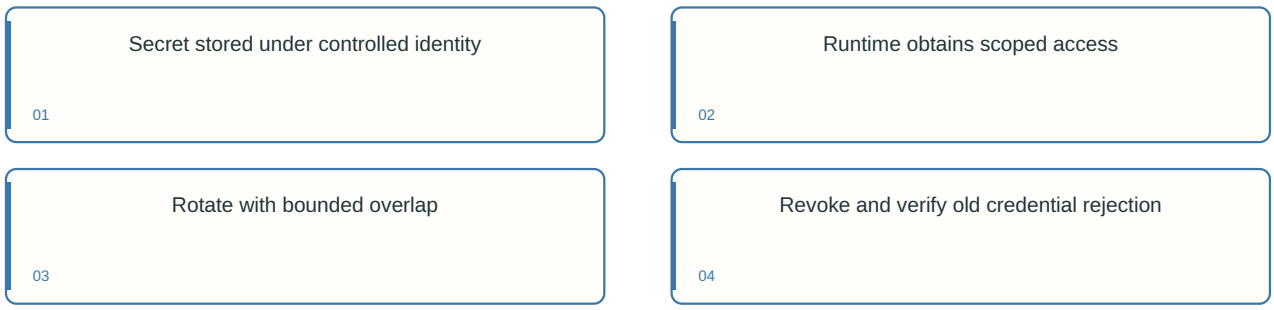


FIG. 18.2.2 **Secrets management · the example**

Secrets need controlled distribution, ownership, rotation, revocation, and protection from accidental exposure.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.2.3 · Encryption in transit and at rest

Trace a private download. Encrypted storage is read by an authorised service, decrypted for processing, and sent to the browser over a protected connection. Plaintext exists at the processing endpoint because the endpoint needs to use it.

Encryption in transit protects bytes while they travel between the endpoints of the protected connection. TLS supplies this protection under its protocol and configuration. If TLS ends at a gateway, examine the gateway-to-service hop separately. Encryption at rest protects stored bytes under particular assumptions about key custody and access. Neither protection makes a service with legitimate decryption rights unable to read the data.

Now follow the copies: backups, extracted text, logs, and exported CSV files may have different storage and delivery paths. Encrypting the original PDF does not automatically protect those copies. Identify where each protection begins and ends, who can obtain plaintext there, and whether the resource policy permits that use. Use reviewed cryptographic protocols and libraries rather than constructing a new scheme for the application.

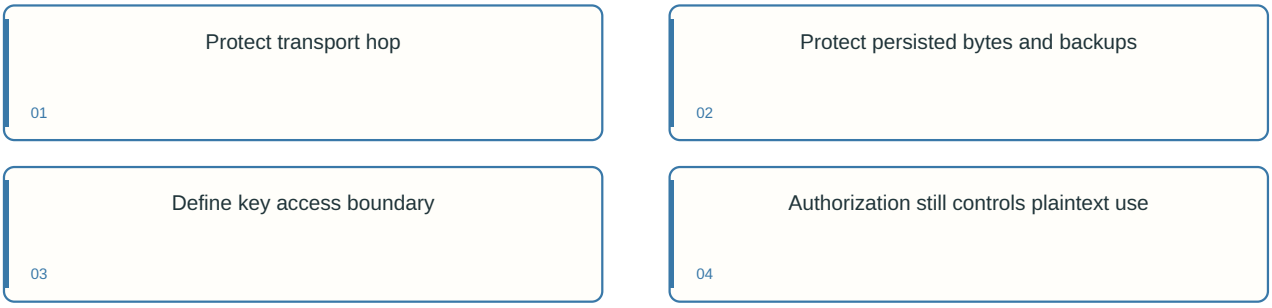


FIG. 18.2.3 **Encryption in transit and at rest · the example**

Transit and storage encryption protect different boundaries under key/access assumptions and do not replace authorisation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.2.4 · Password hashing

If a password database is stolen, an attacker can test candidate passwords without using the login endpoint. Fast general-purpose hashes make these offline guesses inexpensive. Purpose-built password hashing deliberately requires more computation and, for suitable algorithms, memory.

At registration, a maintained library generates a random salt and derives the stored hash using the chosen algorithm and parameters. The salt is stored with the hash; it need not be secret. Different salts prevent one precomputed result from being reused across many users. At login, verification uses the stored settings to check the supplied password without recovering the original password.

Follow current OWASP guidance and select supported parameters after measuring their cost on the deployment's hardware and expected demand. Store the algorithm and parameters so successful verification can trigger an upgrade when needed. Do not invent one universal work setting. Reversible encryption exposes passwords to anyone who obtains the decryption key. Online login and account recovery still need separate abuse controls.

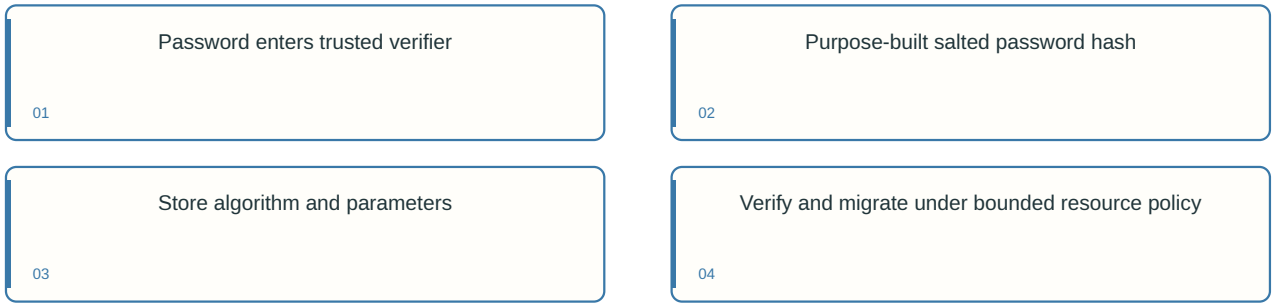


FIG. 18.2.4 **Password hashing · the example**

Password hashing uses a purpose-built salted work mechanism with maintained implementations and deployment-appropriate parameters.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.2.5 · Key management and rotation

Data encrypted with key version one may still be needed after version two becomes active. Key rotation therefore needs a mapping from encrypted data to the key version that can decrypt it. Changing the active key does not re-encrypt earlier data.

New writes can use version two while version one remains available for required old data. Record the key reference with the ciphertext. In envelope encryption, a data key encrypts the content and a separate key-encryption key protects that data key. Recovery must obtain the protected data key and the authority needed to unwrap it.

Key management covers generation, custody, permitted use, rotation, recovery, and destruction. Restore an older backup with its required key versions and credentials before retiring them. Otherwise, a backup can remain intact but unreadable. Map keys to live data, retained copies, and recovery requirements; then decide when older data must be re-encrypted or a key can be destroyed. Keeping every old key indefinitely can conflict with deletion goals, so recovery and retirement need one consistent lifecycle plan.

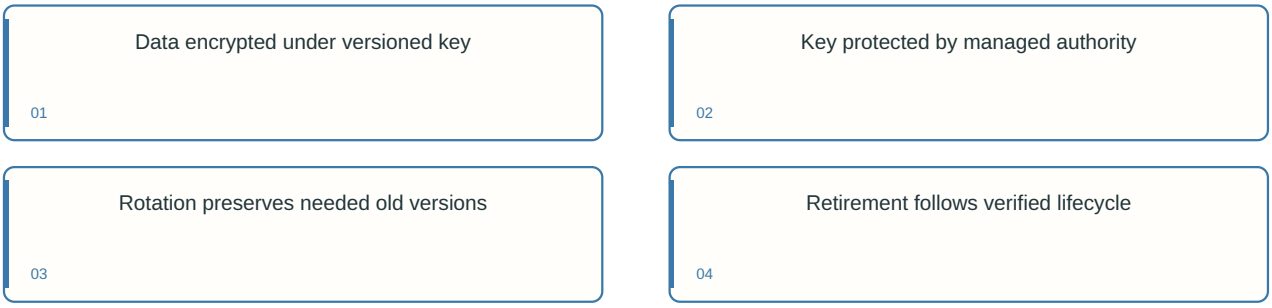


FIG. 18.2.5 **Key management and rotation · the example**

Key management tracks data dependencies through creation, rotation, historical recovery, and destruction.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.2.6 · Network segmentation

An upload worker may need storage access but no route to the production administration interface. Network segmentation divides reachable paths so a compromised component has fewer opportunities to move into unrelated systems. This further movement is often called lateral movement.

Draw the worker's actual communication needs. Permit the required storage and coordination routes, then test that unnecessary administration paths are blocked. An allowed connection still needs authentication and permission for the requested operation. Network reachability establishes that a request can arrive; it does not establish that the caller may read a document.

Outbound paths matter too. A service that fetches a user-supplied URL might be induced to contact a private internal endpoint. Restrict and validate destinations, including what happens after name resolution or a redirect, rather than accepting a superficially acceptable URL. Test real reachability and return paths under the deployed configuration. A firewall-rule description alone cannot show whether the intended boundary is effective.

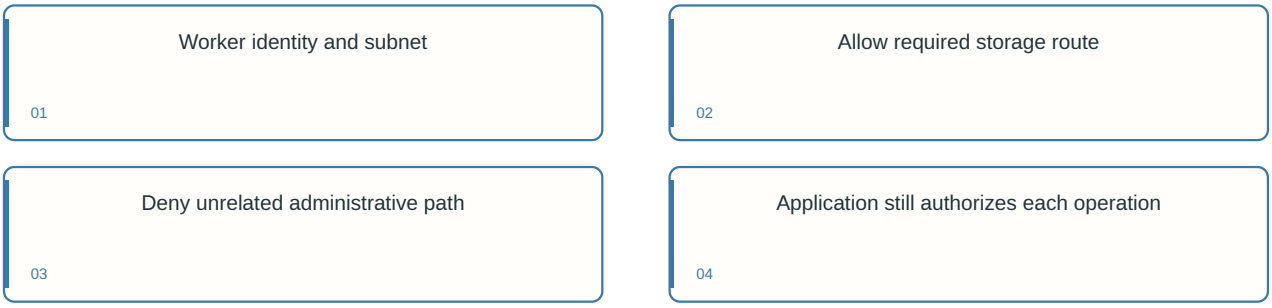


FIG. 18.2.6 **Network segmentation · the example**

Segmentation restricts network reachability while application identity and resource permissions remain necessary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.2.7 · Zero-trust principles

A worker sends a request from an internal address. The destination recognises the address, but the worker may be compromised or may be asking for data outside its job. Network location alone cannot settle either question.

Zero-trust principles make the actor, resource, action, and relevant context explicit instead of granting broad implicit access from location. Authenticate the workload identity, then authorise its requested resource operation. A mesh or VPN may help protect communication and establish identity; the application still needs to decide whether this worker may read this document.

Turn the principle into operating rules. Define credential lifetimes, limited privileges, and how policy changes take effect. Cached credentials or decisions may remain usable for a stated interval, so revocation behaviour must account for that interval. The approach is not a single product and does not imply a new central lookup for every packet. Review the evidence used at each sensitive boundary and test the lifetime of the authority it grants.

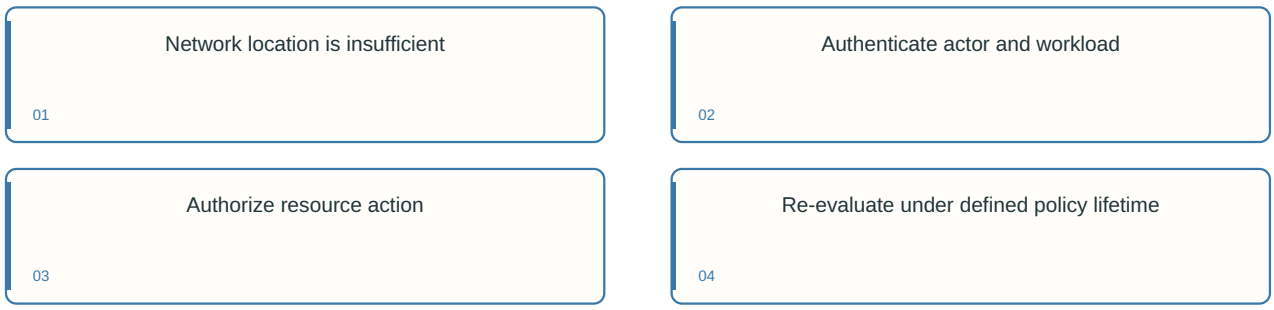


FIG. 18.2.7 **Zero-trust principles · the example**

Zero-trust principles avoid implicit permission from location alone and make trust evidence and limits explicit.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What should revocation mean for an issued object capability and a queued export?

Define when each form of authority stops working. A queued export can check current membership before the later actions required by policy. An already issued object capability may remain usable until expiry unless the delivery mechanism supports earlier revocation. If immediate download denial is required, choose a delivery design that can enforce it rather than relying on expiry alone. Test revocation before execution, before publication, and after link issuance, and describe the resulting lifetime accurately to users.

Remember: Trace credentials, plaintext, and decryption authority through their real lifecycles. Rotation must retire old authority without accidentally destroying the keys and access needed for required recovery.

Try applying it: New files use key version two. Which key references, permissions, and older versions must a restore test exercise before version one can be retired?

Unit 18 exercises → · Unit 18 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

2. OWASP: Password Storage ↗

   Official project guidance · accessed 21 Oct 2026. Password-specific hashing and current deployment parameter guidance.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

4. OpenID Connect Core 1.0, errata set 2 ↗

   Original specification · December 2023. Identity layer and validation requirements; distinct from API authorization.

   Access reviewed · 21 October 2026

Tenant isolation, audit & deletion

Prerequisites: 18.2 · Secrets, encryption & trust boundaries 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Tenant-scoped database queries work correctly, but a cache mixes two documents numbered 7. Later, a deletion removes the database row while an old extraction job recreates its text. Both failures occur outside the original query.

We will carry trusted tenant context through caches, jobs, objects, and exports, then follow private data through diagnostics, audit, retention, and deletion. The goal is to account for the copies and delayed work that can expose or recreate data after the main request has ended.

18.3.1 · API security and common vulnerabilities

A document endpoint uses a parameterised SQL query. Submitted values remain values rather than becoming SQL instructions, which addresses injection through those values. Yet the same safe query can return another tenant's document if it lacks the required ownership condition. Input safety and object authorisation answer different questions.

Follow an accepted request beyond parsing. Check whether the caller may use the selected object, change each submitted property, and receive the fields in the response. A valid JSON request that changes a protected tenant field can still be harmful. An export can also consume excessive CPU, storage, or delivery bandwidth even when its contents are permitted, so bound expensive operations and apply the intended quotas.

Review file paths, callback destinations, and fetched URLs at the point where they are used. Safe syntax does not establish that a destination or operation is allowed. Test cross-tenant identifiers, protected-property changes, injection attempts, privileged operations, and resource limits. Different failure paths require different controls.

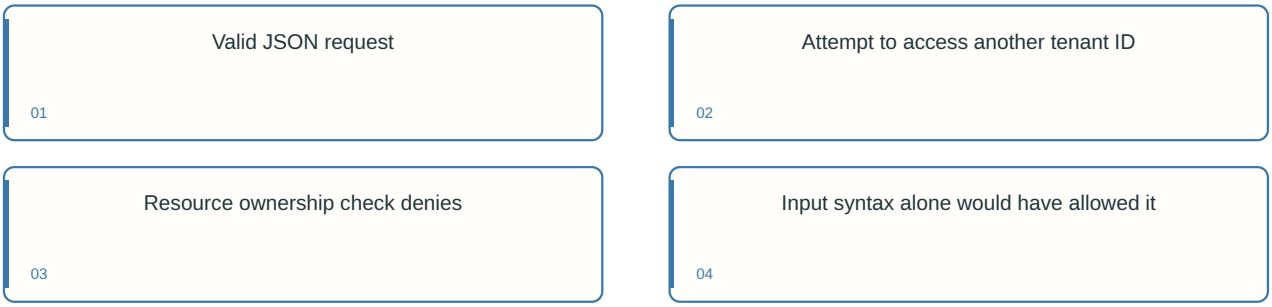


FIG. 18.3.1 **API security and common vulnerabilities · the example**

API security follows how accepted input is used, including ownership, injection, resource, and destination risks.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.3.2 · Multi-tenant isolation

Tenant A and tenant B each have a document numbered 7. The database query includes the tenant, but a shared cache uses only document_id as its key. A result cached for A under key 7 can now be returned on B's path. Isolation failed after the correct query.

Include trusted tenant scope in the cache identity, such as (tenant, document_id), and preserve the same distinction through search indexes, queues, workers, and exports. A worker should resolve the job's actual resource relationships; an unchecked tenant field in a message must not let it select another organisation's input or output. Database policies can add enforcement when correctly configured, but the other stores and paths still need their own checks.

Follow the resulting object to download. A correctly scoped export is exposed if its output is public or its access capability grants too much. Include telemetry and administrative tools in the review. Test two tenants through every route by which their data travels, including overlapping local identifiers and direct object access.

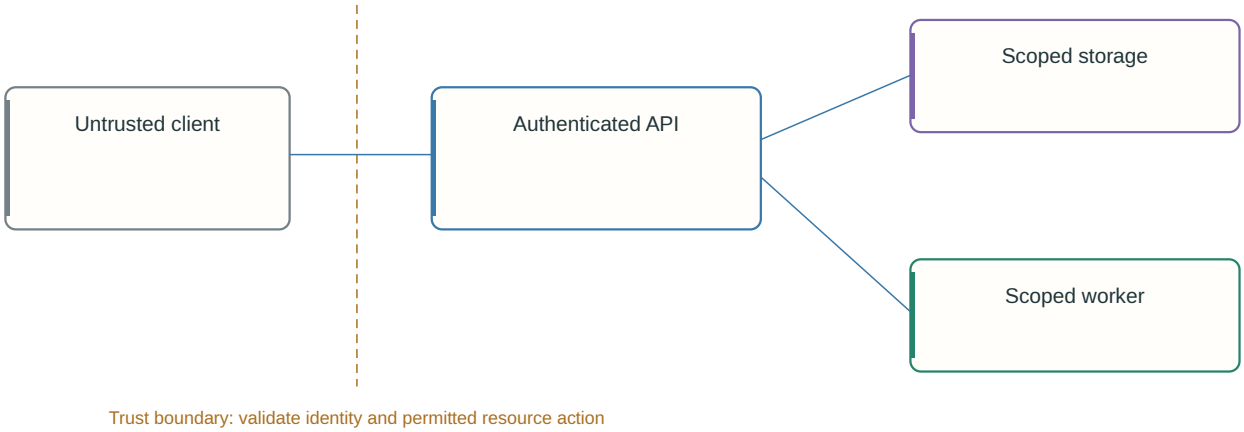


FIG. 18.3.2 **Tenant identity must survive every trust boundary**

An authenticated tenant context scopes storage queries, jobs, object capabilities, and search. Client-provided tenant labels are not independent authority.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.3.3 · Data minimization

A conversion fails, and an operator needs to find out why. An operation identifier and error class may be enough to connect the failure to its processing stage. Copying the whole private document or an access token into the log adds exposure without necessarily helping the diagnosis.

Data minimisation asks which information is necessary for a defined purpose. Make that decision when designing messages, logs, database fields, and derived records. A job message might contain a scoped object reference instead of another full copy of the document, provided the worker can obtain it through the required access checks.

For each retained field, identify its intended use, permitted readers, and retention period. Collecting less sensitive content reduces both disclosure risk and the work needed to remove copies later. Keep enough evidence to operate the service, but do not store a field merely because someone might find it interesting in the future.

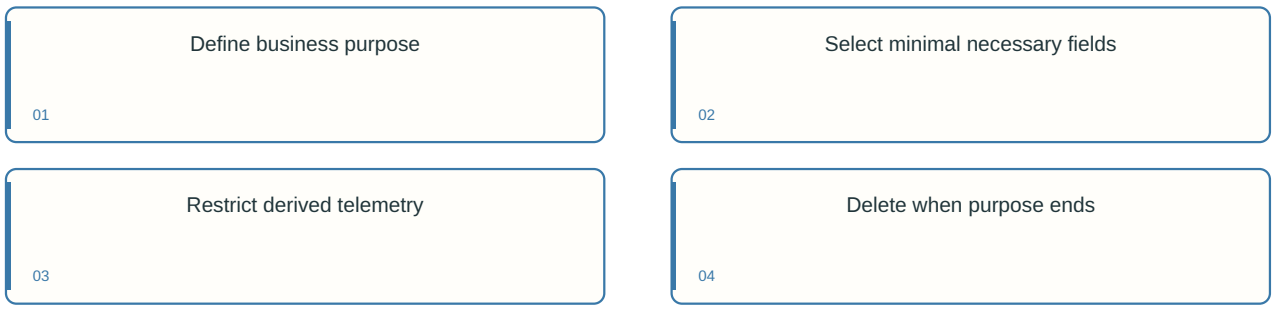


FIG. 18.3.3 **Data minimization · the example**

Data minimisation retains information justified by a stated purpose and lifecycle.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.3.4 · Audit trails

An administrator requests a private export. A useful audit trail connects a trusted actor to the tenant, target, action, and outcome, with time or sequence information. Give the operation an identifier so its request, authorisation decision, and eventual result can be related.

“Request received” proves only that a request reached that recording point. It does not prove that the worker produced an export or that the output was published. Record events whose meanings match those boundaries. If a worker stops after producing the object but before recording completion, recovery should reconcile the operation's durable state and object rather than leave an unexplained success or repeat the action blindly.

Protect the trail against inappropriate alteration, including by people whose actions it records. Restrict its readers and retention too: an audit store can become another source of private data. Necessary identifiers and outcome classes are often more useful than full sensitive payloads. Trustworthy evidence needs clear event meanings as well as protected storage.

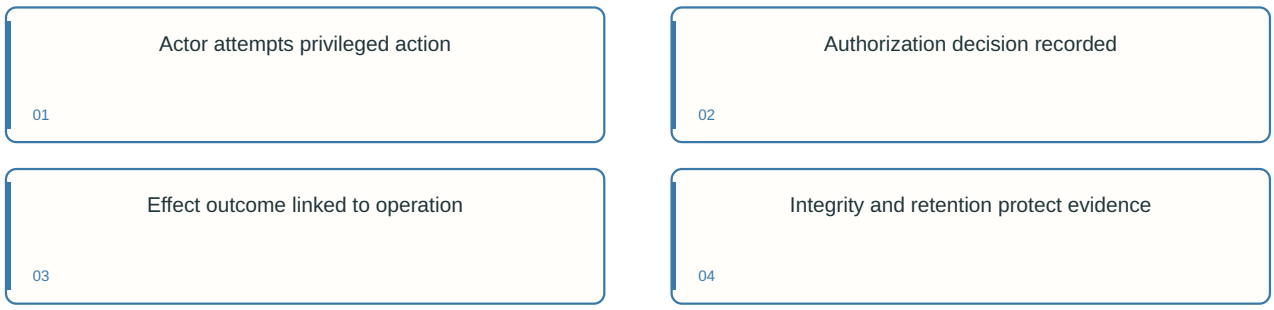


FIG. 18.3.4 **Audit trails · the example**

Audit trails record scoped actors, actions, and actual outcomes while protecting integrity and sensitive context.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.3.5 · Privacy, retention, and deletion

A tenant deletes a document. Removing its main database row may leave the object, extracted text, search entries, cached results, and earlier exports. Begin with a map of the copies and the retention rules that apply to each.

Record a durable deletion request before reporting that work has been accepted. Track its progress through the required stores so a restart can resume unfinished removal. Distinguish acceptance from completion in the API. Workers must also check whether an input is still permitted: an extraction job queued before deletion must not recreate the removed text afterward. Coordinate deletion and publication so a late worker cannot publish a new copy after the deletion process has passed that store.

Treat recovery as another path that can bring data back. When restoring an older snapshot or replaying history, apply the current deletion decisions before serving the recovered data. Document retained copies and any applicable exceptions under the approved policy. Make the promised collection, access, retention, and deletion behaviour match these actual operating steps.

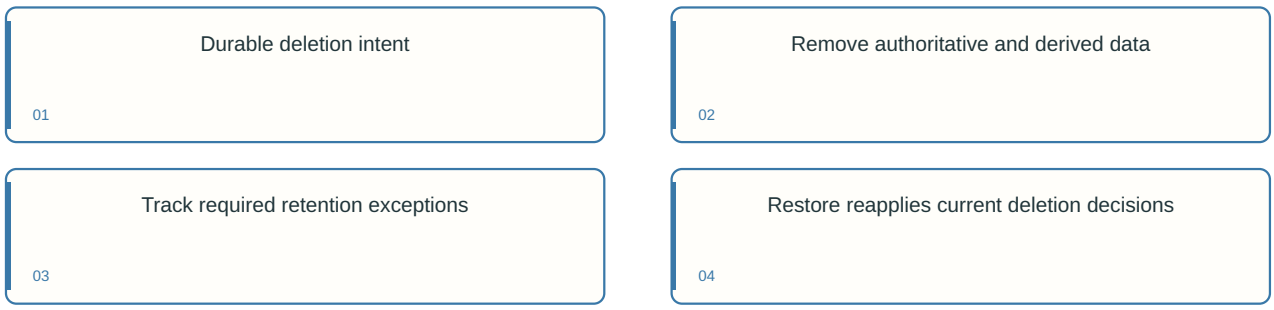


FIG. 18.3.5 **Privacy, retention, and deletion · the example**

Privacy lifecycle spans collection, access, correction, retention, deletion, and historical recovery of copies.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What should revocation mean for an issued object capability and a queued export?

Define when each form of authority stops working. A queued export can check current membership before the later actions required by policy. An already issued object capability may remain usable until expiry unless the delivery mechanism supports earlier revocation. If immediate download denial is required, choose a delivery design that can enforce it rather than relying on expiry alone. Test revocation before execution, before publication, and after link issuance, and describe the resulting lifetime accurately to users.

Remember: Tenant isolation follows data through every store and worker. Deletion must account for those copies, delayed publication, and restore paths, while audit records describe the outcomes actually reached.

Try applying it: A document is deleted while an extraction job waits. Which copies and worker transitions must the deletion process account for so that the document does not reappear?

Unit 18 exercises → · Unit 18 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

2. OWASP: Password Storage ↗

   Official project guidance · accessed 21 Oct 2026. Password-specific hashing and current deployment parameter guidance.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

4. OpenID Connect Core 1.0, errata set 2 ↗

   Original specification · December 2023. Identity layer and validation requirements; distinct from API authorization.

   Access reviewed · 21 October 2026

Supply chains and security reviews

Prerequisites: 18.3 · Tenant isolation, audit & deletion 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A reviewed change becomes a running release through dependencies, builds, stored artifacts, and deployment credentials. Each step can change what reaches production. Users also need recovery paths that are practical without granting authority to the wrong organisation.

We will examine the release path and the user journey, then bring the unit's controls together in a private export review. We will change membership, cancellation, and deletion state while the job waits. The aim is to state what should happen at each later boundary and identify evidence that it actually happens.

18.4.1 · Supply-chain security

A source change passes review, but the deployed executable contains a different dependency or was replaced after the build. Reviewing application source alone does not cover this path. Build tools, dependencies, artifacts, deployment credentials, and update mechanisms form the software supply chain.

Follow one release from its source commit and selected inputs to the built artifact. Record the artifact's identity, such as its digest, and verify that the same artifact is deployed. Provenance is evidence about where and how the artifact was produced; it helps connect a running release to the inputs and process that were checked.

Control dependency resolution and limit who or what can publish artifacts and deploy them. Untrusted contribution builds should not inherit deployment secrets merely because trusted branch builds use them. A valid signature establishes a signing relationship and integrity under the scheme's assumptions; the signed

code or build inputs can still be harmful. Keep a way to identify affected releases and replace or withdraw compromised ones.

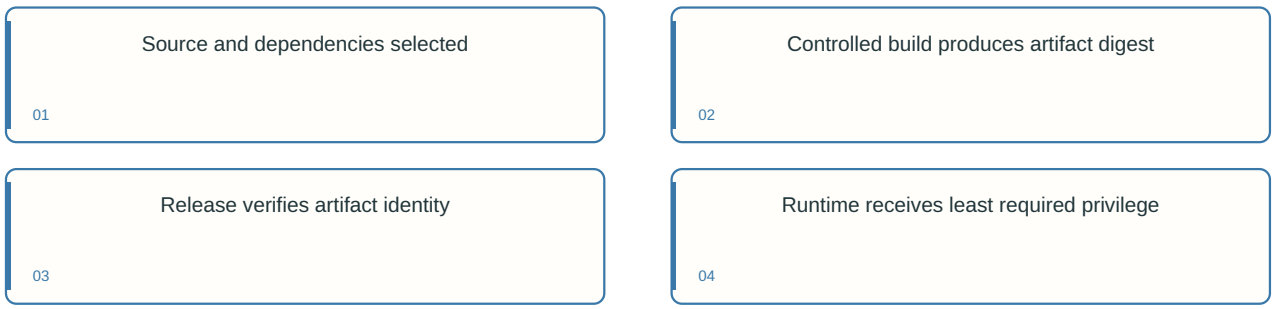


FIG. 18.4.1 **Supply-chain security · the example**

Supply-chain controls protect inputs, builds, artifacts, credentials, deployment, and compromised-release response.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.4.2 · Security vs usability trade-offs

A user loses access to an account and needs recovery. A process that is too difficult may lead legitimate users to share accounts or seek unsupported workarounds. A shortcut that lets any administrator reset any tenant's account creates a different failure: it grants authority across organisation boundaries.

Walk through the same journey as the legitimate user and as someone attempting account takeover. Ask what evidence each can provide, who may approve recovery, and what authority the completed process restores. A fallback should not quietly bypass the identity and tenant restrictions required during normal login.

Safe defaults, scoped support tools, understandable recovery, and proportionate extra verification can improve both protection and usability. Test successful legitimate recovery as well as denial of the abusive path. Mandatory constraints remain explicit: a convenient recovery cannot justify cross-tenant access. The useful trade-off concerns ways to meet those constraints while making the supported path practical for the people who need it.

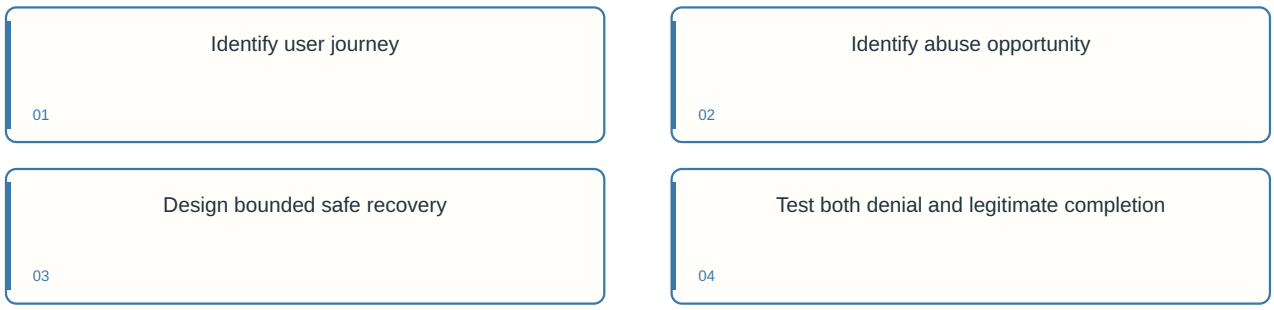


FIG. 18.4.2 **Security vs usability trade-offs · the example**

Security and usability need a workable safe journey without trading away mandatory isolation or correctness.

Author-created illustration. Numbers are illustrative unless stated otherwise.

18.4.3 · Secure-by-design reviews

Follow a sensitive export from request to download. Validate the caller, resolve permitted tenant membership, and select only authorised resources. Record the export operation with trusted scope before placing it in the queue. The worker obtains that operation, checks the policy required at execution, and uses limited authority to read inputs and write a private output.

Before publication, check the operation's current state and the policy required there. A retry after cancellation or deletion must not publish an obsolete export. When a user downloads it, authorise delivery or issue a deliberately scoped, time-limited capability. Record the relevant result in the audit trail. If a capability can be forwarded and used without another identity check, account for that property in its permissions and lifetime.

Now revoke membership after enqueue. Specify whether later execution, publication, and download must stop, and how already issued capabilities are handled. Test those transitions, including deletion racing with publication. Assign an owner to each control and retain evidence that the deployed path enforces it. A checklist helps coverage; the review's conclusion depends on the actual permissions, configurations, and observed outcomes.

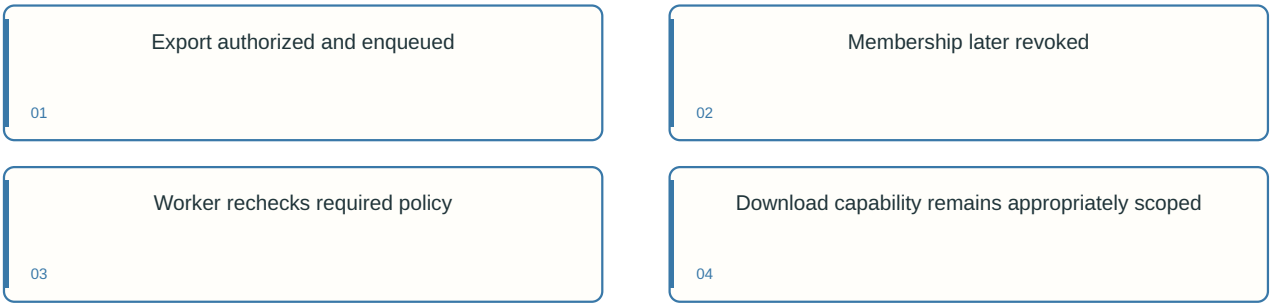


FIG. 18.4.3 **Secure-by-design reviews · the example**

Secure reviews follow sensitive operations and changed conditions through every required enforcement boundary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

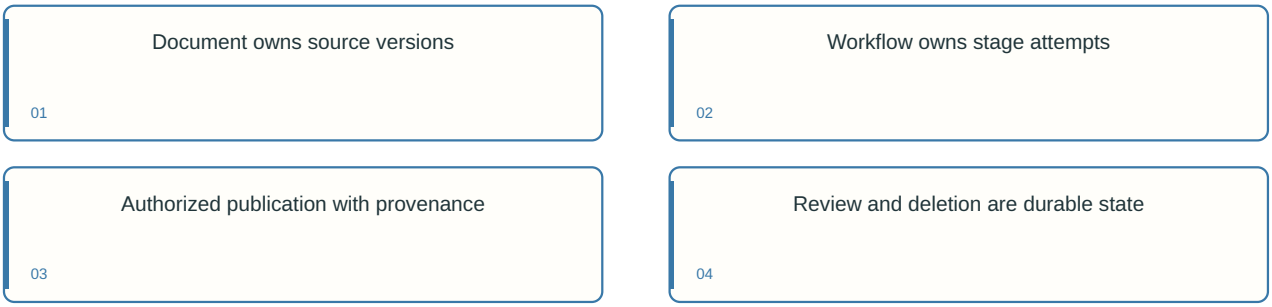


FIG. 18.4.4 **State and ownership in the related design**

Fields and decisions retain tenant, input, transform, and evidence relationships.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · SYNTHETIC TENANT POLICY

```
from mechanisms import permit_demo
actor = {"tenant":"A", "active":True, "actions":["read"]}
assert permit_demo(actor, {"tenant":"A"}, "read")
assert not permit_demo(actor, {"tenant":"B"}, "read")
```

The first assertion returns True: the active actor belongs to tenant A, the resource belongs to A, and read is in the actor's allowed actions. The second returns False because the resource belongs to B. The predicate

also requires active to be exactly True and the action to appear in actions.

The example assumes that actor and resource attributes have already been supplied by a trustworthy source. It does not authenticate credentials, load current membership, or validate malformed identity records. In a real path, establish those facts before evaluating permission, and ensure that a denial stops the effect. Trace the same tenant scope through cache keys, queued jobs, object access, exports, and search; the two assertions alone do not verify those paths.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · What should revocation mean for an issued object capability and a queued export?

Define when each form of authority stops working. A queued export can check current membership before the later actions required by policy. An already issued object capability may remain usable until expiry unless the delivery mechanism supports earlier revocation. If immediate download denial is required, choose a delivery design that can enforce it rather than relying on expiry alone. Test revocation before execution, before publication, and after link issuance, and describe the resulting lifetime accurately to users.

Remember: Connect the reviewed source to the deployed artifact, provide workable bounded recovery, and test sensitive journeys as authority and data state change.

Try applying it: Membership is revoked after an export is queued. What should happen at execution, publication, and download, including a capability issued before revocation?

Unit 18 exercises → · Unit 18 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

2. OWASP: Password Storage ↗

   Official project guidance · accessed 21 Oct 2026. Password-specific hashing and current deployment parameter guidance.

   Access reviewed · 21 October 2026

3. RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

   Original protocol security guidance. OAuth threat and flow guidance; implementation should use maintained protocol libraries.

   Access reviewed · 21 October 2026

4. OpenID Connect Core 1.0, errata set 2 ↗

   Original specification · December 2023. Identity layer and validation requirements; distinct from API authorization.

   Access reviewed · 21 October 2026

PART 5 / UNIT 19

Observability, Monitoring, and Incident Engineering

A user waits five seconds for a payment request. The CPU dashboard looks ordinary, so adding processor capacity has no clear justification yet. We need to find where the time went and whether this request represents a wider problem.

The running example

An API's p99 latency increases from 200 ms to 5 seconds. Use traces, metrics, logs, and database analysis to localize the bottleneck.

Unit 19 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

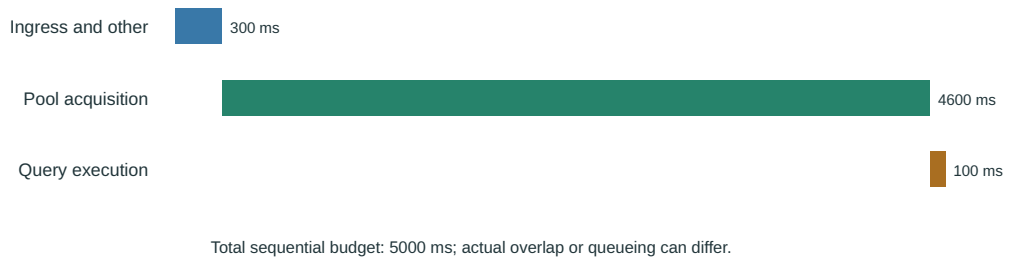


FIG. 19.0.1 **This slow request is dominated by waiting**

The synthetic five-second path spends 92% in pool acquisition. More CPU is not an evidence-based first fix for this execution.

Author-created illustration. Numbers are illustrative unless stated otherwise.

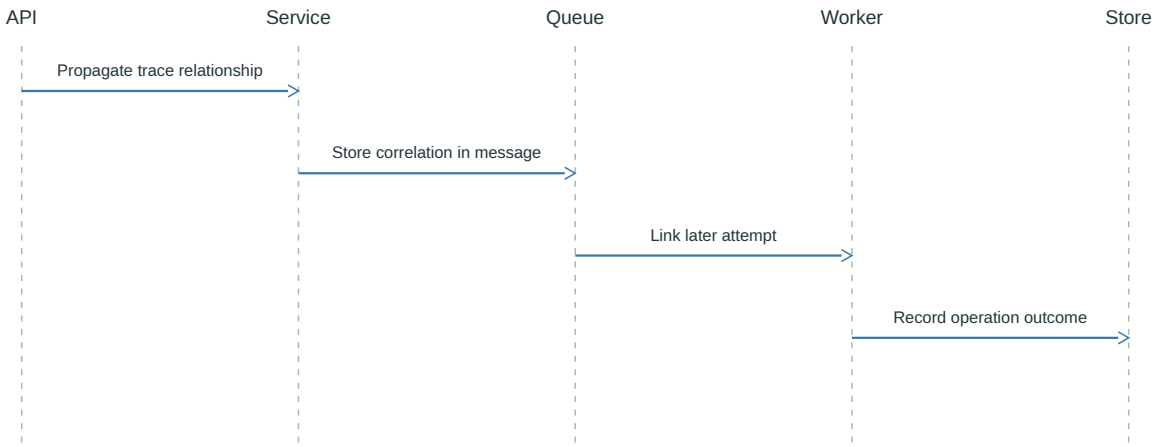


FIG. 19.0.2 **request ID propagation**

Trace IDs relate evidence; they are not authorization or proof that the entire workflow is complete.

Author-created illustration. Numbers are illustrative unless stated otherwise.

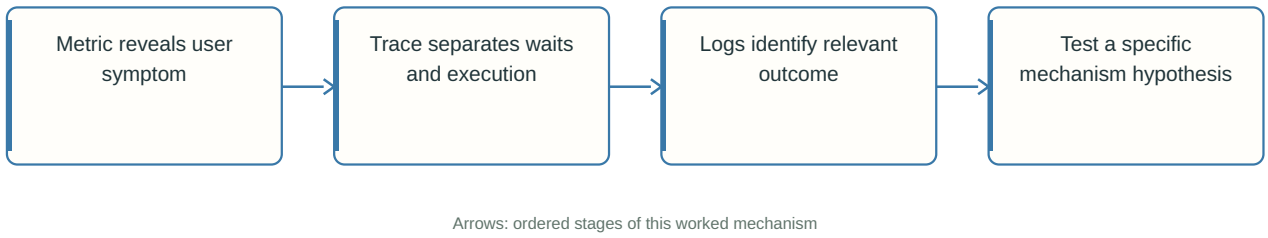


FIG. 19.0.3 **metrics-to-root-cause workflow**

No one signal substitutes for a causal investigation with explicit counterevidence.

Author-created illustration. Numbers are illustrative unless stated otherwise.



FIG. 19.0.4 **error-budget burn chart**

Percentages: at a 99.9% event objective, 1% bad events burns the 0.1% allowance ten times as fast.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Observe behavior with three signals

Prerequisites: Requirements, Estimation, and Engineering Trade-offs, Networking Foundations for System Design, Runtime, Processes, Threads, and Concurrency, Database Internals, Transactions, and Concurrency Control

6 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A user waits five seconds for a payment request. The CPU dashboard looks ordinary, so adding processor capacity has no clear justification yet. We need to find where the time went and whether this request represents a wider problem.

We will combine population measurements, individual request timelines, and structured event records. We will then follow tracing context across calls and messages and relate the user's symptom to traffic, errors, and resource pressure. The aim is to ask a precise question of each signal and connect the answers into an explanation we can test.

19.1.1 · Monitoring vs observability

A user waits five seconds for a payment request while the CPU dashboard looks normal. The dashboard answers one question: how much processor capacity is in use? It does not tell us where the request spent those five seconds. The application could be waiting for a database connection while the processor has little work to do.

Monitoring tracks selected conditions, such as CPU use or a known failure rate. Observability concerns our ability to investigate internal behaviour using the evidence the system produces. These are useful emphases rather than universal boundaries between products.

Begin with the user's operation and follow its work, waits, dependencies, and state changes. We need evidence that can distinguish a slow query from a wait before the query starts. Collecting more unrelated measurements does not necessarily help us make that distinction. The useful question is which observation would support or weaken the explanation we are considering.

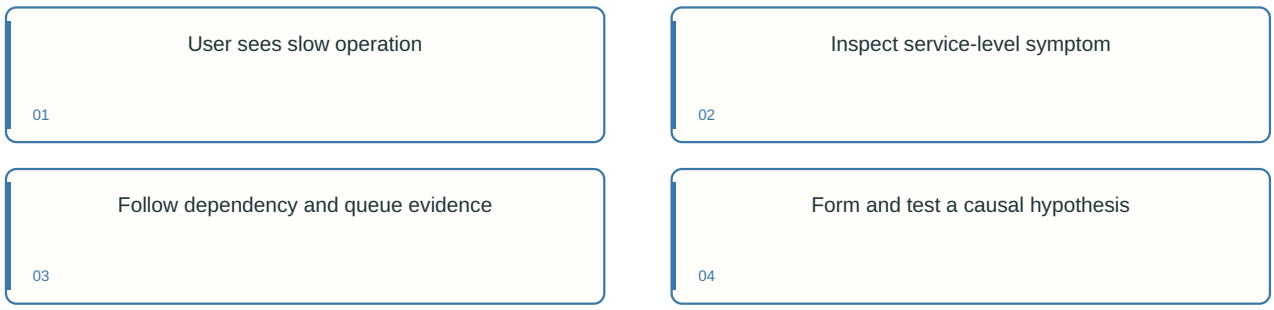


FIG. 19.1.1 **Monitoring vs observability · the example**

Monitoring follows known signals; observability helps explain behaviour through correlated outputs and instrumentation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.1.2 · Logs, metrics, and traces

A latency histogram groups request durations into ranges. It can show that slow requests have become more common, but it cannot tell us where a particular request waited. For that, we can inspect a trace: a record connecting related intervals of work along one execution path. A log records an event, such as a payment attempt ending with a timeout, together with its context.

These signals answer different parts of an investigation. Metrics summarise measurements across a population. A trace lets us follow an individual path. Logs can supply the event details needed to understand what happened on that path. A stable operation identity helps connect the relevant records.

One slow trace does not establish the cause of every slow request. Compare individual histories with population measurements. Sampling, dropped records, clock differences, and privacy limits also affect the evidence available. A missing record may be a gap in collection rather than proof that an event never happened.

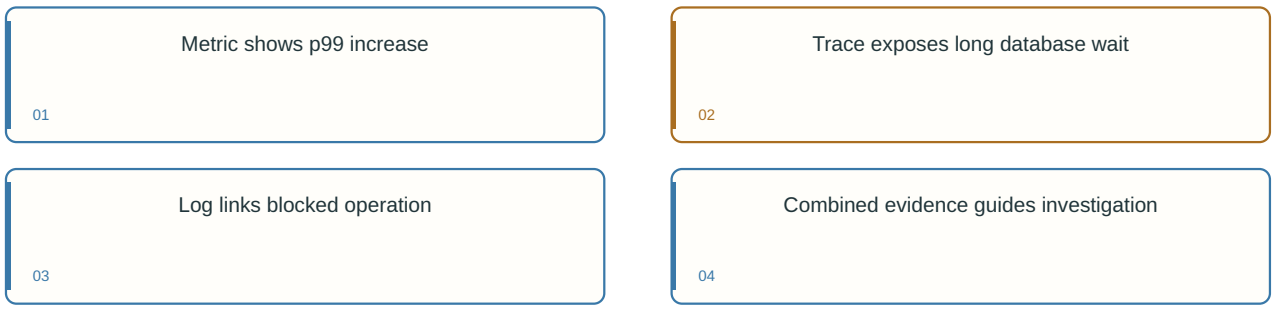


FIG. 19.1.2 **Logs, metrics, and traces · the example**

Logs record events, metrics aggregate populations, and traces connect execution work and waits.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.1.3 · Structured logging

Suppose one instance writes “payment failed” while another writes “provider unavailable.” Finding every timeout now depends on knowing which phrases the code uses. Structured logging records named fields with consistent meanings, such as event_type=payment_attempt, operation_id, outcome_class=provider_timeout, and duration.

We can query the outcome class across instances without interpreting each sentence. The fields still need precise definitions. A payment attempt starting, a provider reply arriving, and a local outcome committing are different events; recording one does not establish the others.

If a finish event is missing, the process may have stopped or the record may have been lost. We still need to recover the provider's result through the payment's supported status or reconciliation path. Keep credentials and raw private documents out of routine logs. Bound record sizes and volume so an error burst does not overwhelm logging resources, and retain records for a defined investigative purpose.

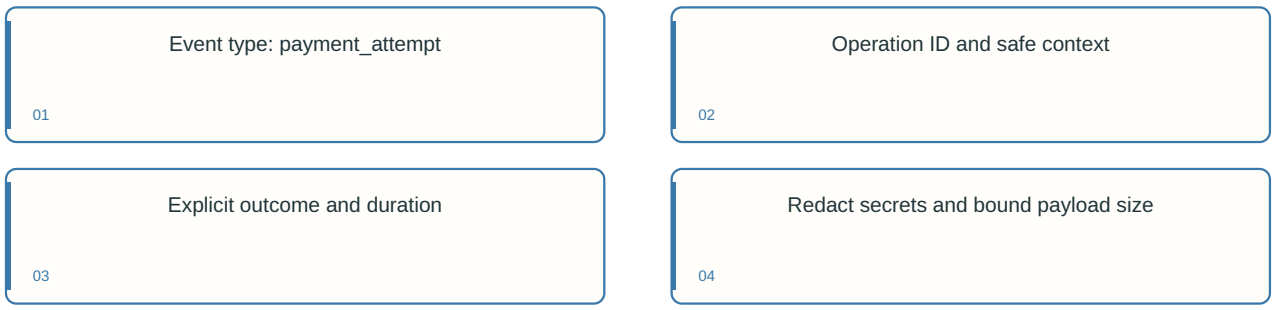


FIG. 19.1.3 **Structured logging · the example**

Structured logs give fields stable meanings, bounded size, safe content, and explicit event boundaries.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.1.4 · Distributed tracing

Consider the constructed five-second request in the figure. It spends 4,600 ms acquiring a database connection, 100 ms executing a query, and 300 ms elsewhere. Most of its elapsed time passes before the query can begin. Faster query execution alone would leave that long wait largely intact.

A span records an interval such as handling a request, acquiring a connection, or executing a query. Parent and child relationships connect spans into a trace. Recording acquisition separately from execution makes the wait visible; without that boundary, it may appear as unexplained application time. Distributed tracing carries these relationships across components.

For later asynchronous work, a span link can preserve the causal connection even when processing continues after the original request ends. A trace remains evidence about one execution and may be sampled. Compare slow paths with normal paths and latency distributions before extending this example's explanation to the service as a whole.

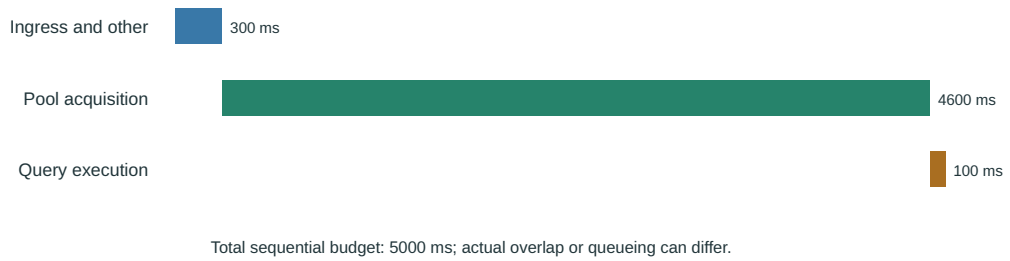


FIG. 19.1.4 **This slow request is dominated by waiting**

The synthetic five-second path spends 92% in pool acquisition. More CPU is not an evidence-based first fix for this execution.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.1.5 · Trace IDs, span IDs, context propagation

A trace identifier groups related spans, while a span identifier names one interval within that trace. To connect work across a network call, the sender must carry the relevant identifiers and relationship information to the receiver. This is context propagation.

Suppose a request starts trace T7. The service records its database work and publishes a message with tracing context. The worker can use that context to relate its later processing to the request. If the context is lost at the queue boundary, both sides may produce valid spans that the investigator cannot follow as one connected history.

Check each transport rather than assuming a standard format instruments every library automatically. Preserve a durable business operation identity as well: progress must remain recoverable even when traces are not retained. Incoming tracing context may be untrusted, so validate or replace it according to policy. A trace ID supplies diagnostic correlation, not identity or permission, and context fields should not become a way to pass unnecessary private data.

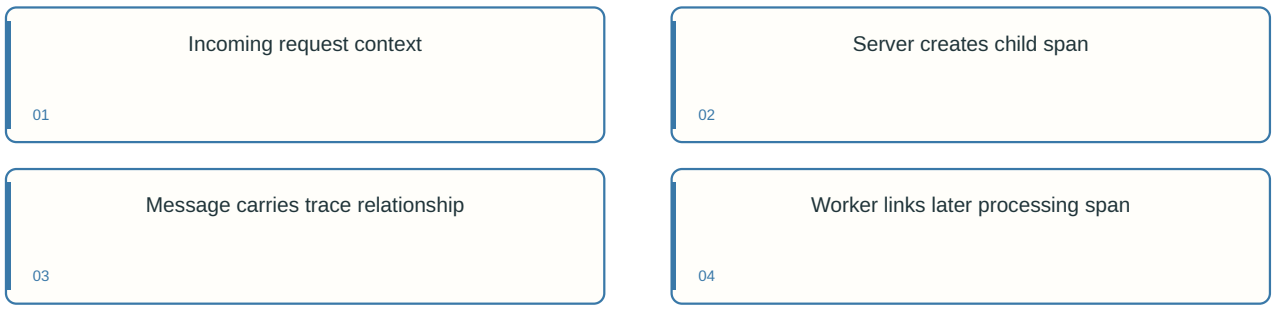


FIG. 19.1.5 **Trace IDs, span IDs, context propagation · the example**

Trace context links observations across calls or messages and must not be treated as authorisation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.1.6 · Golden signals

Traffic rises, the connection queue grows, and responses slow down while errors initially remain low. Looking only at failure counts would miss this developing problem. Four commonly used golden signals bring together latency, traffic, errors, and saturation.

Latency describes how long the selected operations take. Traffic describes the amount of work arriving or completing. Errors describe failed outcomes. Saturation describes pressure on a constrained resource, such as requests waiting behind an occupied connection pool. Together, these signals connect demand, resource pressure, and what users receive.

Define the outcomes being counted. An HTTP 200 does not necessarily establish that a payment completed, and many successful cheap reads can hide failures in a less frequent critical operation. Examine important operations separately where needed. For work continuing after a request ends, add domain measures such as the age of the oldest unresolved payment; request-serving signals alone may miss that harm.

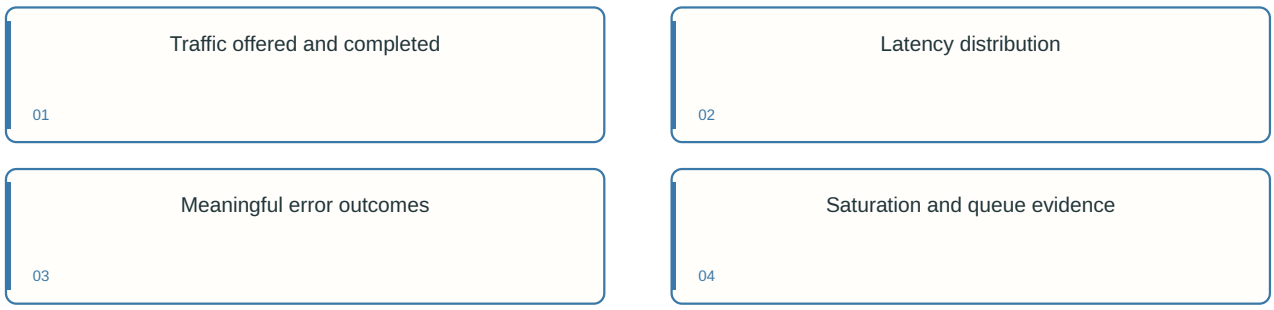


FIG. 19.1.6 **Golden signals · the example**

Golden signals connect latency, traffic, errors, and saturation using meaningful eligible outcomes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What evidence would support a latency explanation beyond correlation with a release?

Locate the extra wait or execution on affected paths and compare them with normal paths and population measurements. If the hypothesis is that a release holds connections longer, inspect changed request behaviour, connection hold times, occupancy, and acquisition waits in the timeline. Consider competing explanations and test a focused intervention under comparable conditions. Support is stronger when the mechanism predicts the symptom and changing that mechanism changes the observed behaviour; a release marker alone establishes timing.

Remember: Use metrics to establish the population symptom, traces to locate work and waits, and logs to explain relevant events. Correlate their evidence without treating one history as the whole population.

Try applying it: A latency histogram shows a worsening p99 and one trace shows a long connection wait. What does each establish, and what further evidence would support applying that explanation to the wider slowdown?

Unit 19 exercises → · Unit 19 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OpenTelemetry: Signals ↗

   Official documentation · accessed 21 Oct 2026. Complementary observability signals.

   Access reviewed · 21 October 2026

2. OpenTelemetry: Context propagation ↗

   Official documentation · accessed 21 Oct 2026. Trace context relationships; trace identity does not authorize access.

   Access reviewed · 21 October 2026

3. Alerting on SLOs — The SRE Workbook ↗

   Engineering book. Budget burn and actionable alerts.

   Access reviewed · 21 October 2026

Choose indicators, budgets & alerts

Prerequisites: 19.1 · Observe behavior with three signals 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

The connection pool is full and request durations rise, but the on-call engineer is already receiving pages about harmless CPU fluctuations. Collecting and alerting on every available measurement has made it harder to recognise the condition that needs action.

We will connect request measurements to resource measurements, define eligible outcomes and a service objective, and calculate how quickly failures consume its allowance. We will then choose alerts, sampling policies, and dashboard views for the decisions they must support. By the end, we should be able to explain both what an urgent signal means and which useful response should follow it.

19.2.1 · RED and USE methods

For the payment API, we can ask how many requests arrive, which fail, and how long they take. This is the RED view: rate, errors, and duration. For its database connection pool, we ask how much capacity is in use, how much work waits, and which resource errors occur. This is the USE view: utilisation, saturation, and errors.

The two views help connect a symptom to a possible explanation. RED may show rising request duration. USE may show that all connections are occupied and more requests are waiting. CPU can remain low because those waiting requests are not executing processor work.

The pool measurement narrows the investigation; it does not explain why connections remain occupied. Inspect their holders and compare the timing with affected requests. Use enough time detail and per-owner views to expose bursts or hot shards hidden by averages. Keep domain indicators too: uncertain charges can still require attention after ordinary request durations recover.

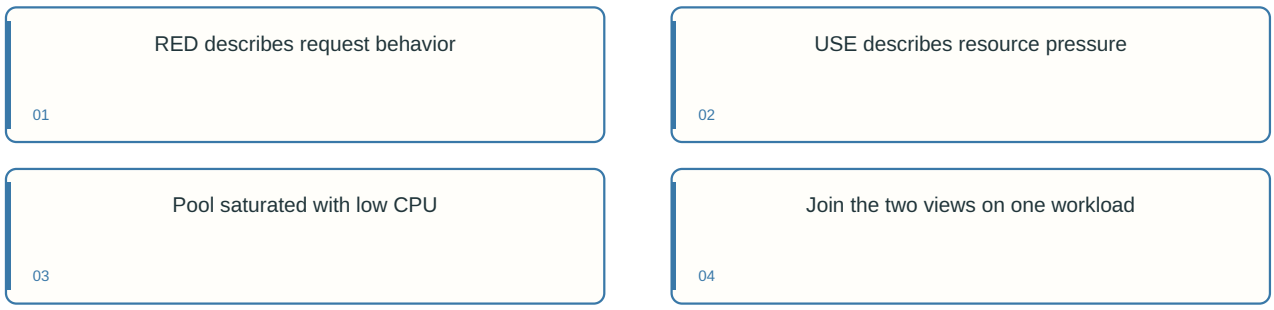


FIG. 19.2.1 **RED and USE methods · the example**

RED focuses on serving rate, errors, and duration; USE examines resource utilisation, saturation, and errors.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.2.2 · SLIs, SLOs, error budgets

First define what counts as a good event and which events are eligible. For example, a success fraction divides good eligible outcomes by all eligible outcomes. This measured behaviour is a service-level indicator, or SLI. A service-level objective, or SLO, sets a target for that indicator over a stated window. Its error budget is the allowed shortfall.

For a 99.9% success target, the allowed bad fraction is 100% − 99.9% = 0.1%, or 0.001. Suppose 1% of eligible events are bad during an observed interval. The burn rate is 0.01 / 0.001 = 10: the observed bad fraction is ten times the allowance's fraction. Sustained behaviour like this uses the budget rapidly.

Keep the eligible population and windows explicit. This ratio alone does not tell us the remaining budget or how long it will last; event volume and earlier consumption also matter. A large denominator can hide harm to critical operations, while a few events can make a low-volume indicator fluctuate sharply. The objective describes the stated population and window, rather than guaranteeing every request's outcome.

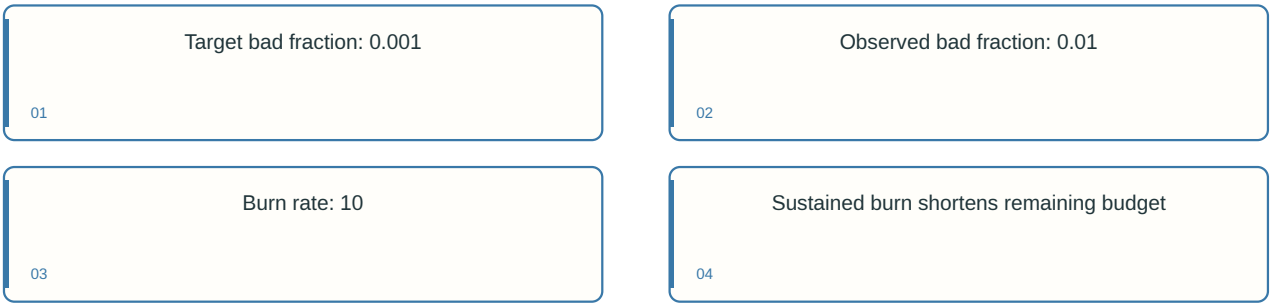


FIG. 19.2.2 **SLIs, SLOs, error budgets · the example**

SLIs measure behaviour; SLOs target it; error-budget burn compares observed shortfall with its allowance.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.2.3 · Alert design and alert fatigue

A brief CPU spike pages the on-call engineer, who finds that useful service never degraded. Repeated pages like this consume attention needed when users are actually affected. An alert should connect a meaningful condition to a useful response.

For an urgent page, state which outcome is threatened, who owns the response, and where investigation should begin. Sustained rapid error-budget burn may justify immediate attention; a condition needing planned capacity work may belong in a ticket. Match the urgency to how soon someone must act and what that action can achieve.

Choose thresholds and windows for the objective and response time. Include context and a runbook, meaning a procedure for investigation, action, and verification. Group related alerts where they would otherwise demand the same action during one incident. Review unnecessary pages and incidents that were missed. The aim is a paging channel that helps responders recognise harm and begin mitigation promptly.

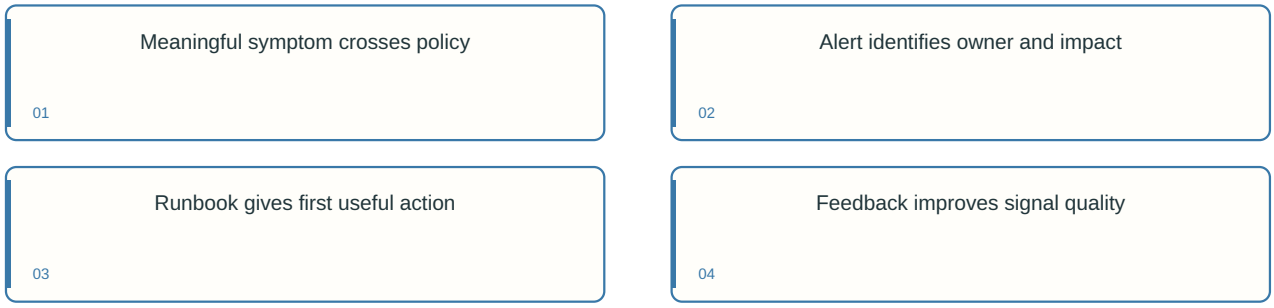


FIG. 19.2.3 **Alert design and alert fatigue · the example**

Useful alerts identify actionable harm or impending objective loss with an owner and response.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.2.4 · Sampling and telemetry cost

Keeping every trace may exceed the available storage and processing budget. Sampling retains a selected subset. Head sampling makes the selection early, before the whole trace is known. It cannot choose an initially discarded trace because that request later fails. Tail sampling waits for later observations and can select traces by duration or error, but needs capacity to buffer and process the arriving data.

The selection policy changes what the retained evidence represents. A collection favouring errors is useful for diagnosis, but its error fraction cannot be read as the failure rate of all requests. Late or dropped spans can also leave an incomplete history.

Metrics have a related cost boundary. Each combination of label values identifies a time series. Giving every request its own label can create a growing number of series. Use bounded dimensions for population metrics and keep detailed identities in suitable logs or traces. Choose sampling, retention, privacy limits, and ingestion capacity together, including behaviour during failure bursts when telemetry volume may rise.

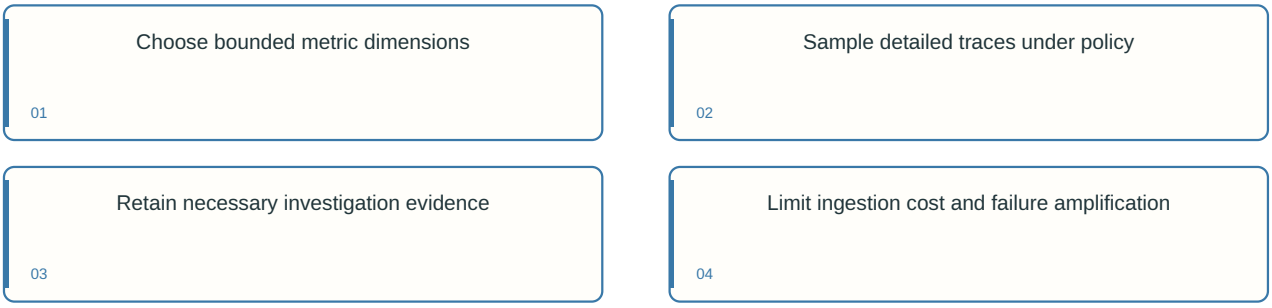


FIG. 19.2.4 **Sampling and telemetry cost · the example**

Sampling and retention balance diagnostic questions, privacy, cardinality, and telemetry capacity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.2.5 · Dashboards

Begin a dashboard with the decision it should support. For “Are payments completing?”, show accepted operations, required completions, failure classes, and the age of unresolved work. These measurements establish the service outcome before the responder examines resources that might explain a change.

A useful path then leads from a changed outcome to the relevant workload, deployment, dependency, or resource. For rising request latency, that might mean opening a duration distribution and then examining connection-pool wait. This arrangement reduces the number of unrelated graphs the responder must interpret before forming a hypothesis.

Include units, denominators, measurement windows, deployment markers, and visible data gaps. A flat line caused by missing telemetry has a different meaning from a measured zero. Provide drill-downs and ownership, and check the dashboard against known examples or earlier incidents. Its value comes from helping someone make a supported decision, rather than from the number of panels it contains.

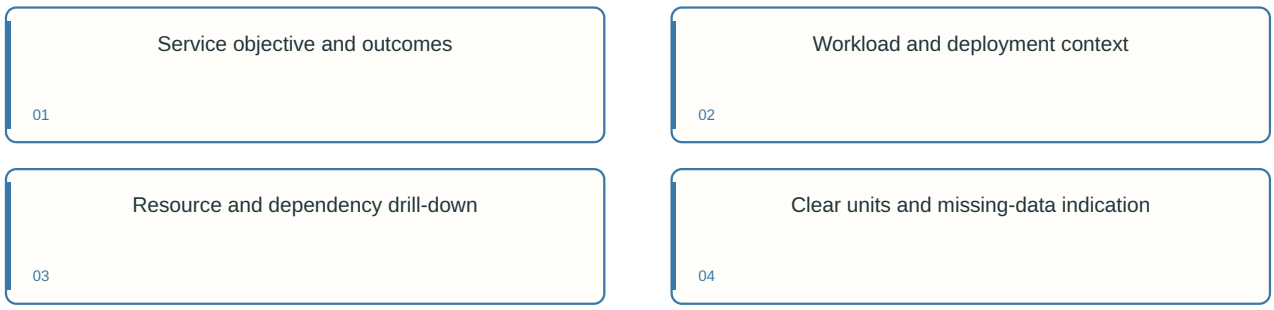


FIG. 19.2.5 **Dashboards · the example**

Dashboards organise evidence for decisions with clear units, denominators, windows, gaps, and ownership.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What evidence would support a latency explanation beyond correlation with a release?

Locate the extra wait or execution on affected paths and compare them with normal paths and population measurements. If the hypothesis is that a release holds connections longer, inspect changed request behaviour, connection hold times, occupancy, and acquisition waits in the timeline. Consider competing explanations and test a focused intervention under comparable conditions. Support is stronger when the mechanism predicts the symptom and changing that mechanism changes the observed behaviour; a release marker alone establishes timing.

Remember: Define eligible outcomes and the objective before calculating budget burn. Choose bounded telemetry, dashboards, and alerts that support a clear decision and a useful response.

Try applying it: A 99.9% success objective has an observed 1% bad-event fraction. How do you calculate burn rate, and what population, window, and response information would you need before turning it into a page?

Unit 19 exercises → · Unit 19 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OpenTelemetry: Signals ↗

   Official documentation · accessed 21 Oct 2026. Complementary observability signals.

   Access reviewed · 21 October 2026

2. OpenTelemetry: Context propagation ↗

   Official documentation · accessed 21 Oct 2026. Trace context relationships; trace identity does not authorize access.

   Access reviewed · 21 October 2026

3. Alerting on SLOs — The SRE Workbook ↗

   Engineering book. Budget burn and actionable alerts.

   Access reviewed · 21 October 2026

Profile and investigate production

Prerequisites: 19.2 · Choose indicators, budgets & alerts 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A deployment precedes a slowdown. One responder proposes faster SQL, another proposes more CPU, and a third wants to restart the service. The shared timing is a clue, but it does not show which part of the request became slower or which intervention would help.

We will distinguish execution from waiting, use profiles and traces to examine specific mechanisms, and compare the affected paths with normal ones. We will also coordinate incident actions and write procedures that connect observations, supported mitigations, and recovery checks. The goal is to restore useful service while preserving the evidence needed to explain the incident.

19.3.1 · Profiling

A CPU profiler samples where the processor executes code. If many samples fall in parsing under a representative workload, parsing is a candidate for investigation. Other profiling modes can examine allocations or blocking. Choose the mode for the resource question being asked.

Return to the request waiting for a connection. A CPU-only profile may contain little evidence of its long delay because waiting uses little processor time. Even the function consuming the largest share of CPU need not account for the largest share of elapsed request time. A trace that records waits helps us see the difference.

Use a profiling method with acceptable overhead and privacy behaviour, and state which workload it represents. Compare the profile with request traces and useful completion rates. Make a focused change, then measure again under comparable conditions. An improvement in CPU consumption is useful evidence

about that resource; separately check whether the change improved the latency or throughput users needed.

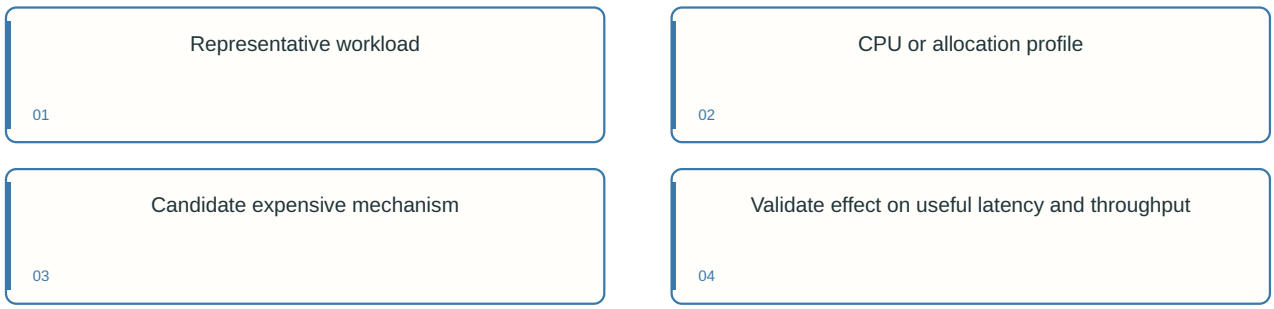


FIG. 19.3.1 **Profiling · the example**

Profiling locates runtime work or waits under measured conditions, which must be related to user outcomes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.3.2 · Query and application performance diagnosis

To investigate a slow request, first separate time spent waiting from time spent executing. Relevant intervals may include an admission queue, connection acquisition, lock wait, query execution, network transfer, and response encoding. These boundaries give competing explanations a place to appear.

In our five-second example, acquisition takes 4.6 seconds and the query takes 0.1 seconds. Halving query time saves 0.05 seconds, leaving a 4.95-second path if everything else stays the same. The arithmetic points us toward the acquisition wait. We still need to inspect why connections remain occupied: for example, long transactions or slow work by existing holders could require different repairs.

If p99 rises from 200 ms to five seconds with modest CPU, compare normal and slow requests, pool occupancy and hold times, lock evidence, query plans where relevant, and recent changes. Preserve useful evidence before restarting components, and assess whether a restart would erase diagnostic state or add load. Test the explanation against observations rather than choosing a cause from the symptom alone.

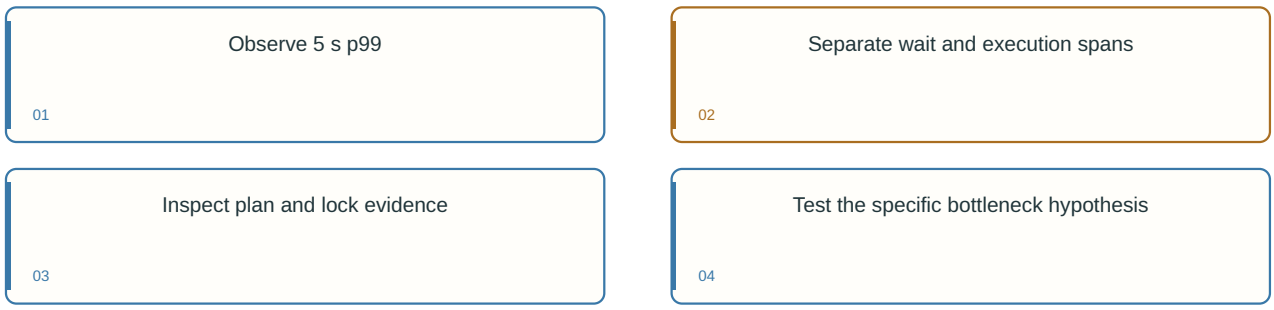


FIG. 19.3.2 **Query and application performance diagnosis · the example**

Diagnosis separates waiting from execution and tests proposed bottleneck mechanisms against evidence.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.3.3 · Incident response

During a complex incident, several responders may investigate at once. If one restarts instances while another changes pool settings without coordination, the team can lose track of which action produced which result. Assign someone to coordinate the response and make decision ownership clear.

Keep a timeline of observed impact, hypotheses, interventions, and results. Others can investigate separate questions while the coordinator maintains the changing picture and arranges communication. Limit unrelated changes so that new observations remain interpretable. Restoring safe useful service can come before a complete causal explanation, provided the unresolved investigation has an owner.

Choose mitigation with the current state in view. Rolling back code may be unsafe after an incompatible data migration. When serving recovers, inspect accepted business operations as well: uncertain provider charges still need reconciliation. A stopped alert establishes only that its condition cleared. Verify the required service outcomes and assign remaining recovery work before declaring the incident resolved.

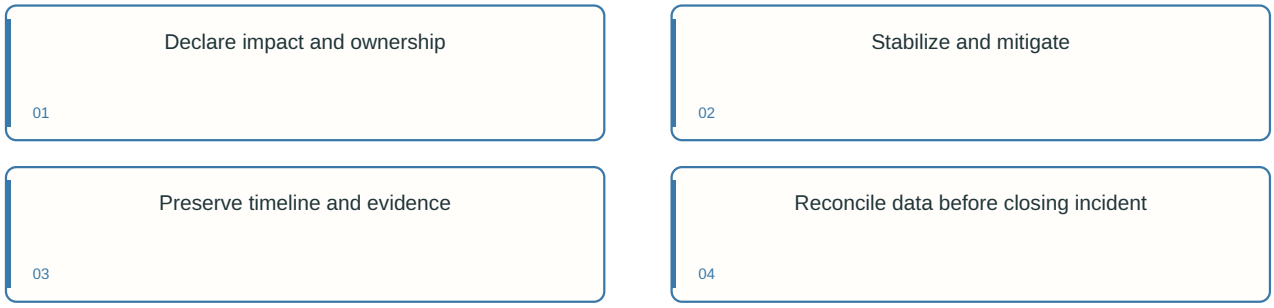


FIG. 19.3.3 **Incident response · the example**

Incident response coordinates mitigation, decisions, communication, and remaining uncertain business outcomes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.3.4 · On-call and runbooks

A runbook saying only “restart the service” leaves the responder to decide when that action is appropriate, what it risks, and how to recognise recovery. A useful runbook explains the observations that justify an action and the result expected afterward.

For the pool-wait investigation, a procedure might first inspect acquisition delay and long transactions. If a known optional workload is holding scarce connections, the supported mitigation might be to stop that workload, then verify required completions and wait times. The procedure must also say when to stop or escalate if its conditions do not match the incident.

On-call needs working access, escalation, rehearsal, and sustainable ownership as well as written instructions. Audit emergency access. Repeated manual repair is operating toil: necessary work that keeps recurring without removing the underlying problem. If nightly reconciliation or cache rebuilding becomes routine, give service owners time to change the failure mechanism. Extending the instructions alone will not remove that burden.

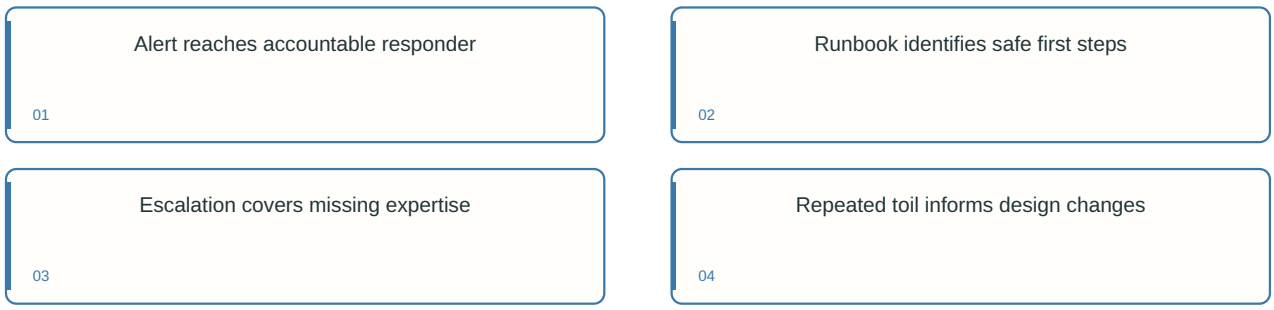


FIG. 19.3.4 **On-call and runbooks · the example**

On-call needs useful alerts, access, escalation, rehearsed runbooks, and service ownership that reduces recurring toil.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.3.5 · Root-cause investigation

A slowdown begins after a deployment. The timing makes the release a useful lead, but it does not explain how requests became slow. The deployment may be a trigger acting on another condition, such as limited pool capacity or an unbounded queue.

Consider a hypothesis: the release increases database calls per request, connections stay occupied longer, the pool fills, and later requests wait to acquire one. Each step predicts evidence we can inspect. Compare changed request behaviour, connection hold times, occupancy, and acquisition waits with the affected timeline. A query-execution slowdown would predict a different change within the trace.

Test competing explanations and ask whether a focused intervention changes the proposed mechanism under comparable conditions. This gives stronger support than a shared timestamp alone. Several conditions may contribute, including weak detection or slow recovery. Preserve those distinctions so corrective work addresses how the incident developed and spread, rather than stopping at the label “bad release.”

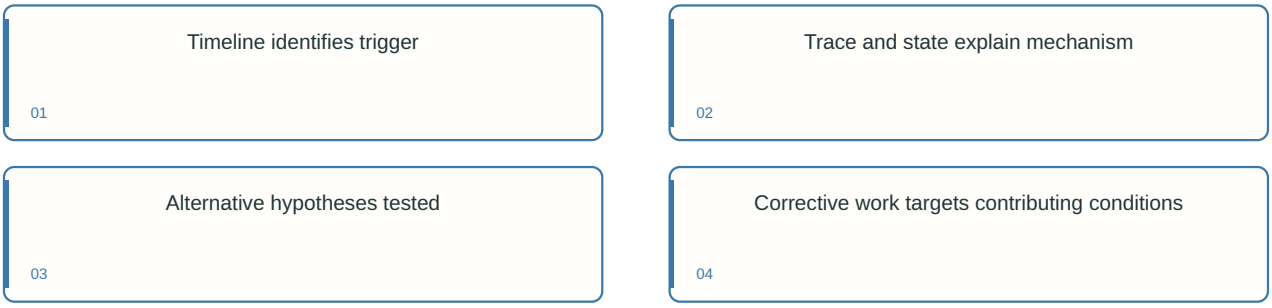


FIG. 19.3.5 **Root-cause investigation · the example**

Cause investigation distinguishes triggers from contributing mechanisms and tests explanations beyond correlation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · What evidence would support a latency explanation beyond correlation with a release?

Locate the extra wait or execution on affected paths and compare them with normal paths and population measurements. If the hypothesis is that a release holds connections longer, inspect changed request behaviour, connection hold times, occupancy, and acquisition waits in the timeline. Consider competing explanations and test a focused intervention under comparable conditions. Support is stronger when the mechanism predicts the symptom and changing that mechanism changes the observed behaviour; a release marker alone establishes timing.

Remember: Locate the extra wait or execution, then test the mechanism that could cause it. Coordinate mitigation, preserve state and evidence, and verify useful service as well as cleared alerts.

Try applying it: The five-second path contains 4.6 seconds of connection acquisition and 100 ms of query execution. Which observations would help explain the acquisition wait, and how should responders coordinate a mitigation and its recovery check?

Unit 19 exercises → · Unit 19 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OpenTelemetry: Signals ↗

   Official documentation · accessed 21 Oct 2026. Complementary observability signals.

   Access reviewed · 21 October 2026

2. OpenTelemetry: Context propagation ↗

   Official documentation · accessed 21 Oct 2026. Trace context relationships; trace identity does not authorize access.

   Access reviewed · 21 October 2026

3. Alerting on SLOs — The SRE Workbook ↗

   Engineering book. Budget burn and actionable alerts.

   Access reviewed · 21 October 2026

Learn from incidents and trace async work

Prerequisites: 19.3 · Profile and investigate production 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

The HTTP service has recovered, yet some accepted extraction jobs remain unfinished. Meanwhile, the team's incident notes say only “bad release,” and a fleet average hides a shard approaching its storage limit. Each view leaves out part of the system's continuing work.

We will turn incident evidence into specific improvements, monitor capacity at the owner that can limit progress, and follow asynchronous operations beyond the original request. We will distinguish the age of unfinished work from the duration of completed work. The aim is to recognise recovery and ongoing harm at the business outcome the system promised.

19.4.1 · Blameless postmortems

After service recovers, the team needs an account of what happened and what should change. A blameless postmortem records impact, the timeline, detection, mitigation, and contributing conditions. It explains decisions using the information people had at the time, including measures that helped them recover.

Suppose an interface hid that one action would affect every replica. Reconstruct what the responder could see before judging the action. Then examine how the interface, permissions, or rollout controls could make its scope clearer or limit the harm. This turns the investigation into changes the system can enforce.

“Be more careful” does not alter the failure mechanism. Assign specific corrective work with an owner and a way to check its effect against the failure scenario. Explain why safeguards did not prevent or contain the incident, and protect unnecessary customer or employee details. Blameless analysis still requires clear ownership of improvements and evidence that the work helps.

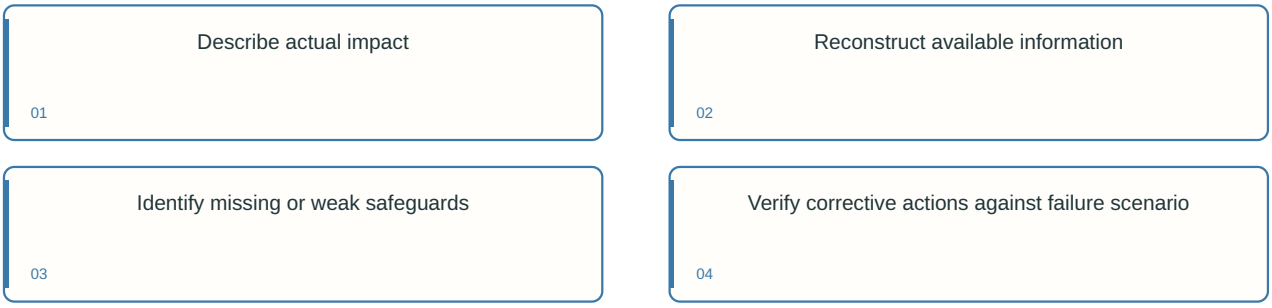


FIG. 19.4.1 **Blameless postmortems · the example**

Blameless postmortems reconstruct impact and conditions, with specific owned improvements and verification evidence.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.4.2 · Capacity monitoring

The fleet's average CPU is low, yet one shard is nearly out of disk and a connection pool remains full. The average combines spare resources with a constrained owner. Capacity monitoring must examine where work accumulates and which resource can stop useful service.

Track current use, growth rate, queue age, and the time needed to add or release capacity. A disk approaching its limit requires action early enough for the available expansion process. A forecast based on recent growth is an estimate; changes in traffic or a release can alter that growth, so revise it as evidence changes.

Include background work such as compaction, backup, and migration, which also consumes capacity. Check what remains when a planned failure removes an instance, disk, or other owner. Spare capacity elsewhere helps only if the work can actually use it. The useful forecast connects an approaching limit to the time and resources needed for a supported response.

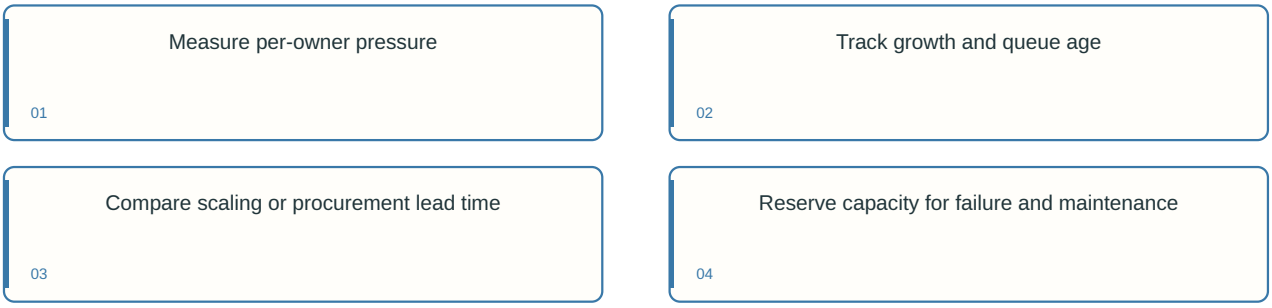


FIG. 19.4.2 **Capacity monitoring · the example**

Capacity monitoring tracks meaningful approaching limits, growth uncertainty, and the time available to respond.

Author-created illustration. Numbers are illustrative unless stated otherwise.

19.4.3 · Observability across asynchronous workflows

An HTTP request returns 202 in a few milliseconds, but its extraction job waits for hours. The response measures acceptance. The user's required outcome arrives only when the later processing finishes, so acceptance latency can look healthy while useful service has stopped.

Give accepted work a durable operation identity and follow it through delivery, worker attempts, stage commits, retries, and the final outcome. Record when stages are entered so queue delay can be separated from processing duration. Attempts need their own identities without losing their relationship to the original operation; record stale or duplicate attempts being rejected as well.

For unfinished work, operation age is the time since acceptance up to now. For completed work, end-to-end duration runs from acceptance to the required outcome. Track useful completions and the oldest unfinished age together: completed-job durations alone omit jobs that remain stuck. Use the operation identity to locate their state across brokers, workers, stores, and providers, even when the original request trace is no longer retained.

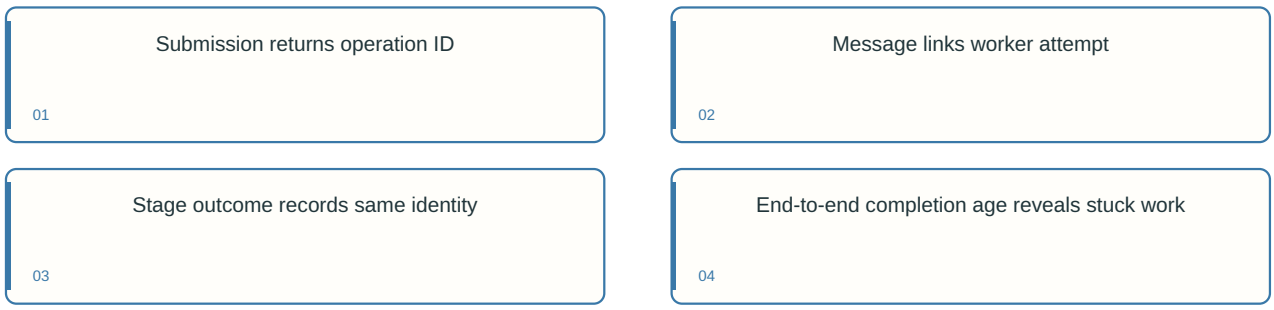


FIG. 19.4.3 **Observability across asynchronous workflows · the example**

Asynchronous observability follows accepted work through later attempts, stage waits, and required final outcomes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

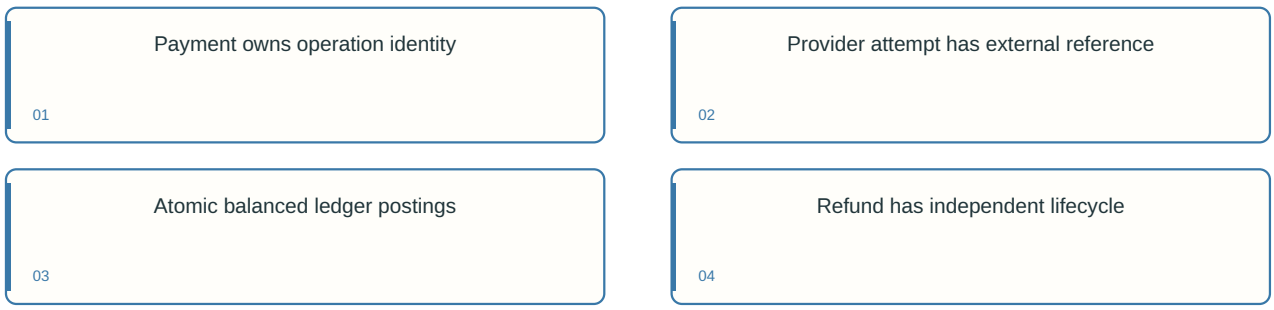


FIG. 19.4.4 **State and ownership in the related design**

Operation identity links provider evidence to payment state and balanced accounting entries.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · ONE TRACE ACCOUNTING MODEL

```
stages_ms = {"other":300, "pool_wait":4600, "query":100}
assert sum(stages_ms.values()) == 5000
assert stages_ms["pool_wait"] / sum(stages_ms.values()) == .92
```

The dictionary assigns durations to three sequential stages of one constructed request. Their sum is 300 + 4,600 + 100 = 5,000 ms, so the first assertion checks the five-second total. The second divides 4,600 ms of

pool acquisition by 5,000 ms overall: 0.92, or 92%. The query accounts for only 100 ms. Halving that query time would save 50 ms if the other stages stayed unchanged.

The model directs investigation toward the acquisition wait, but it neither collects a real trace nor establishes why connections are occupied. Inspect hold times, relevant transactions and locks, and normal as well as slow request populations before attributing the incident to a particular mechanism. The dictionary also assumes sequential accounting; real traces can contain overlapping spans whose durations cannot simply be added.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · What evidence would support a latency explanation beyond correlation with a release?

Locate the extra wait or execution on affected paths and compare them with normal paths and population measurements. If the hypothesis is that a release holds connections longer, inspect changed request behaviour, connection hold times, occupancy, and acquisition waits in the timeline. Consider competing explanations and test a focused intervention under comparable conditions. Support is stronger when the mechanism predicts the symptom and changing that mechanism changes the observed behaviour; a release marker alone establishes timing.

Remember: Use incidents to improve the conditions that shaped them. Monitor limiting owners and follow durable operation identities until the required outcome, including work still unfinished after serving recovers.

Try applying it: Acceptance remains healthy while document completions stop. Which operation ages, stage timings, and durable records would you inspect, and why can completed-job durations alone hide the stuck work?

Unit 19 exercises → · Unit 19 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OpenTelemetry: Signals ↗

   Official documentation · accessed 21 Oct 2026. Complementary observability signals.

   Access reviewed · 21 October 2026

2. OpenTelemetry: Context propagation ↗

   Official documentation · accessed 21 Oct 2026. Trace context relationships; trace identity does not authorize access.

   Access reviewed · 21 October 2026

3. Alerting on SLOs — The SRE Workbook ↗

   Engineering book. Budget burn and actionable alerts.

   Access reviewed · 21 October 2026

PART 5 / UNIT 20

Deployment, Cloud Infrastructure, and Platform Architecture

The same image works locally and fails in production. Compare environments and runtime isolation, keeping persistent state, resources, and platform duties explicit.

The running example

Take a single-process application and deploy it into a reliable, observable, multi-instance production environment with safe releases.

Unit 20 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

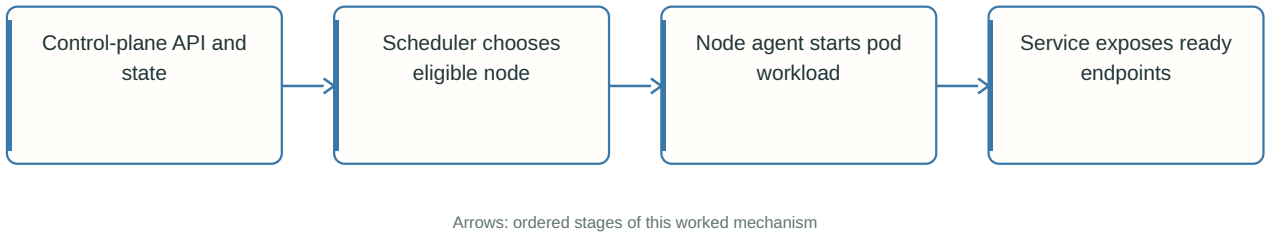
Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

Layer	Virtual machine	Container
Operating environment	Guest OS per VM	Shared host kernel
Process resources	Guest-managed processes	Runtime constraints
Persistent state	Explicit attached owner	Not ephemeral writable layer

FIG. 20.0.1 **VMs and containers isolate at different layers**

These are common architecture distinctions, not a claim that one option universally provides sufficient security or reliability.

Author-created illustration. Numbers are illustrative unless stated otherwise.

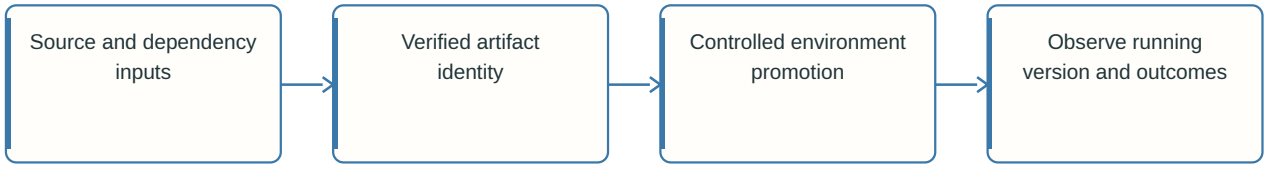


Arrows: ordered stages of this worked mechanism

FIG. 20.0.2 **Kubernetes cluster**

A simplified cluster control/data relationship; scheduling does not supply application data consistency.

Author-created illustration. Numbers are illustrative unless stated otherwise.



Arrows: ordered stages of this worked mechanism

FIG. 20.0.3 **CI/CD pipeline**

CI/CD preserves evidence and identity rather than merely copying files.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Stage	Blue	Green
Before switch	Serving existing version	Candidate verified
During switch	Drain under policy	Receives new traffic
Data	Shared or migrated state	Rollback compatibility required

FIG. 20.0.4 **blue-green deployment**

Parallel runtime environments do not automatically make shared schema changes reversible.

Author-created illustration. Numbers are illustrative unless stated otherwise.

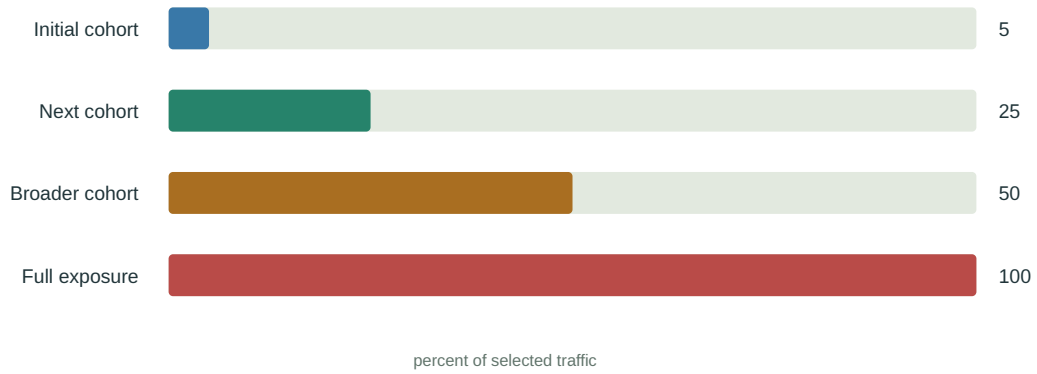


FIG. 20.0.5 **Canary exposure expands only after verification**

Percentages are an illustrative rollout plan, not an automatic safety rule. Each increase depends on meaningful representative checks and stop conditions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Environments, VMs & containers

Prerequisites: Networking Foundations for System Design, Application Architecture and Domain-Driven Design, Reliability, Fault Tolerance, and Disaster Recovery, Security Architecture and Privacy, Observability, Monitoring, and Incident Engineering

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

The same image works locally and fails in production. Compare environments and runtime isolation, keeping persistent state, resources, and platform duties explicit.

20.1.1 · Development vs staging vs production

A service passes development tests but fails in production because its network permissions and connection limits differ. Development, staging, and production have different purposes, yet the relevant behaviours need deliberate compatibility.

Use staging to exercise protocols, identity, migrations, and failure paths with appropriate test data and secrets. Exact production scale may be impractical, so record what staging cannot establish. Separate environment values from business meaning. A local pass is evidence for that environment; it does not prove storage, network, or load behaviour elsewhere. Make the differences visible enough that release checks can address the important ones.

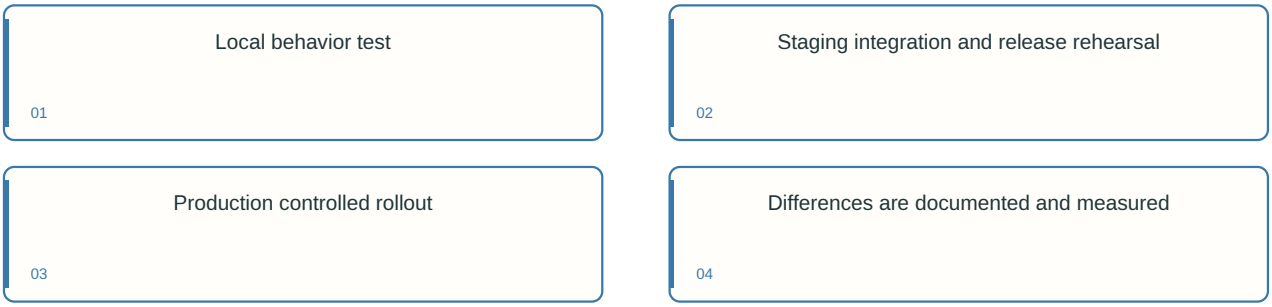


FIG. 20.1.1 **Development vs staging vs production · the example**

Environments have different purposes while relevant protocols, state transitions, and configuration meanings need deliberate compatibility.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.1.2 · Virtual machines vs containers

A virtual machine presents virtualised hardware and a guest operating system. A container commonly isolates processes while sharing the host kernel. Both provide useful boundaries, but their isolation and operating responsibilities differ.

Choose using workload, expertise, startup behaviour, and required separation. Images, identity, networking, storage, and resource limits need design in either case. Several containers on one host still share its failure and capacity. VM boundaries also sit on shared physical and administrative infrastructure. Identify the actual threat and failure model rather than treating either packaging label as complete isolation.

Layer	Virtual machine	Container
Operating environment	Guest OS per VM	Shared host kernel
Process resources	Guest-managed processes	Runtime constraints
Persistent state	Explicit attached owner	Not ephemeral writable layer

FIG. 20.1.2 **VMs and containers isolate at different layers**

These are common architecture distinctions, not a claim that one option universally provides sufficient security or reliability.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.1.3 · Container fundamentals

A container packages an executable environment. Runtime settings then restrict its CPU, memory, files, and network access. Packaging helps repeatability when image and configuration are controlled.

Required persistent data should not live only in an ephemeral writable layer that disappears on replacement. Resource limits can also throttle or terminate work, so preserve necessary progress and handle shutdown. Broad host mounts or privileged execution can weaken the intended isolation. Inspect the actual runtime permissions and persistence path. A reproducible image is one input to a predictable service, alongside configuration, state, and operating policy.

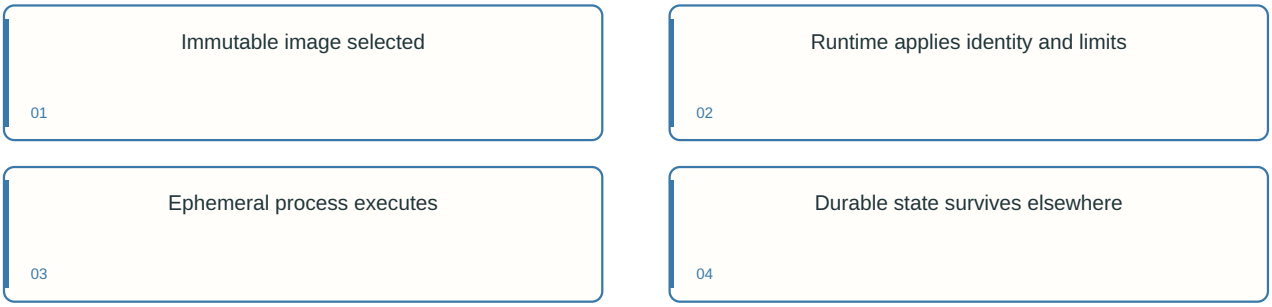


FIG. 20.1.3 **Container fundamentals · the example**

Containers package execution while runtime controls, persistent-state placement, identity, and shutdown still need design.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.1.4 · Kubernetes fundamentals

Kubernetes controllers compare declared objects with observed cluster state and work to reconcile differences. Pods, scheduling, services, and node agents provide primitives for running and locating workloads.

The platform does not infer an application's transaction invariants or create its tested backups and access model automatically. Identify which cluster responsibilities the team operates and which a provider manages. An orchestrator adds a control plane and a substantial configuration surface. If simpler deployment machinery meets the requirements, that can be a reasonable choice. Adopt platform facilities for their concrete benefits while retaining ownership of application behaviour and recovery.

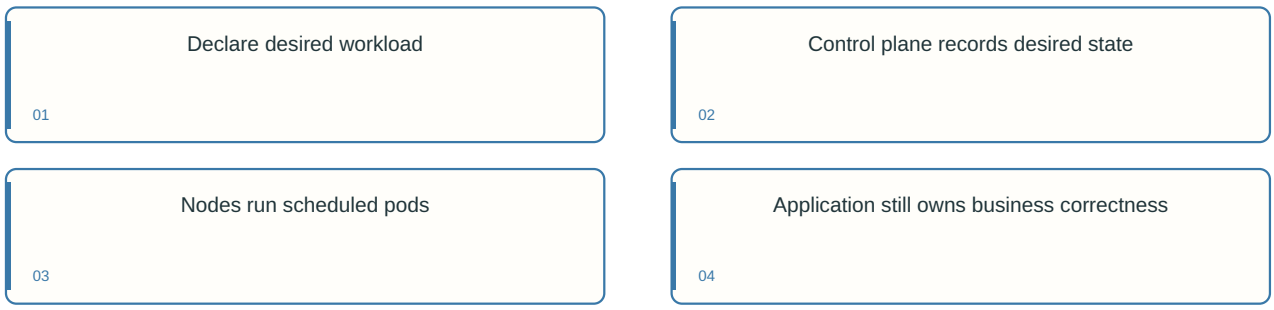


FIG. 20.1.4 **Kubernetes fundamentals · the example**

Kubernetes reconciles workload resources; application transactions, access, backups, and service objectives remain explicit responsibilities.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How can rollback restore old code while leaving incompatible data or effects?

Old code can return while new schemas, values, provider calls, and published messages remain. Maintain compatibility during overlap and identify the point where contraction ends safe rollback. Some outcomes need forward repair or compensation. Restoring an artifact cannot reverse every state transition it previously performed.

Remember: Packaging improves repeatability only with controlled configuration and appropriate state ownership.

Try applying it: Which data would disappear if a container's writable layer were replaced?

Unit 20 exercises → · Unit 20 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. The Twelve-Factor App ↗

   Original methodology. Application deployment conventions.

   Access reviewed · 21 October 2026

2. Kubernetes components ↗

   Official documentation · accessed 21 Oct 2026. Platform reference for Unit 20.

   Access reviewed · 21 October 2026

3. Canarying Releases — The SRE Workbook ↗

   Engineering book. Representative rollout evaluation and stop conditions.

   Access reviewed · 21 October 2026

Control planes, scheduling & configuration

Prerequisites: 20.1 · Environments, VMs & containers 5 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

The scheduler cannot place new work, but existing requests still succeed. Separate control and serving paths, then examine startup, scaling, configuration, and replacement dependencies.

20.2.1 · Control plane vs data plane

The control plane decides placement and configuration. The data plane carries the running application's traffic. Under some conditions, a control-plane outage stops new scheduling while already running workloads continue.

Draw both paths. A request that unnecessarily performs a synchronous configuration lookup can turn a control-plane problem into a serving outage. Cached values need validity and update rules, particularly for permission changes. Recovery also may need a configuration or credential service before replacements can start. A redundant data path is useful only when its required startup and authority dependencies are understood.

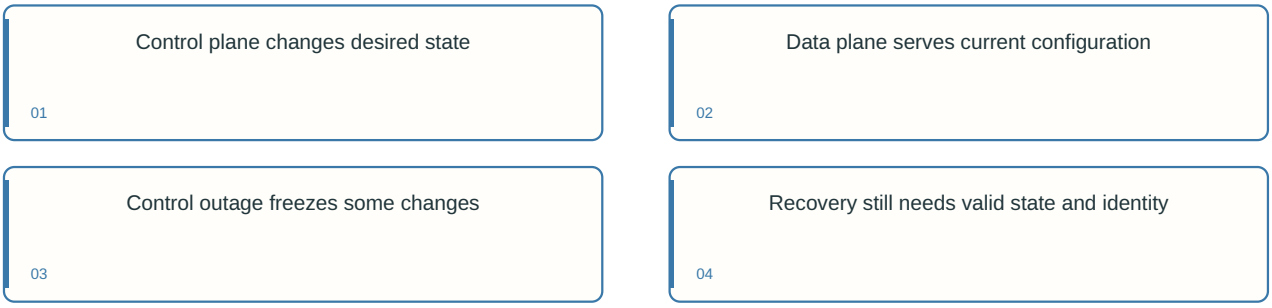


FIG. 20.2.1 **Control plane vs data plane · the example**

Control planes decide configuration and placement; data planes serve traffic with different startup and failure dependencies.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.2.2 · Scheduling and service discovery

A scheduler assigns a workload to eligible resources. Its decisions can consider capacity, affinity, anti-affinity, and failure domains. Service discovery later helps callers locate the running endpoints.

A scheduled process may still be starting and not ready for traffic. Discovery can lag both startup and termination, so clients need bounded handling of stale endpoints. Placement also says nothing about shared-data correctness: two healthy replicas can each point to an unrelated local database. State the storage and readiness contracts alongside the scheduling rule. Successful placement establishes where work runs, rather than that the whole application state is coordinated.

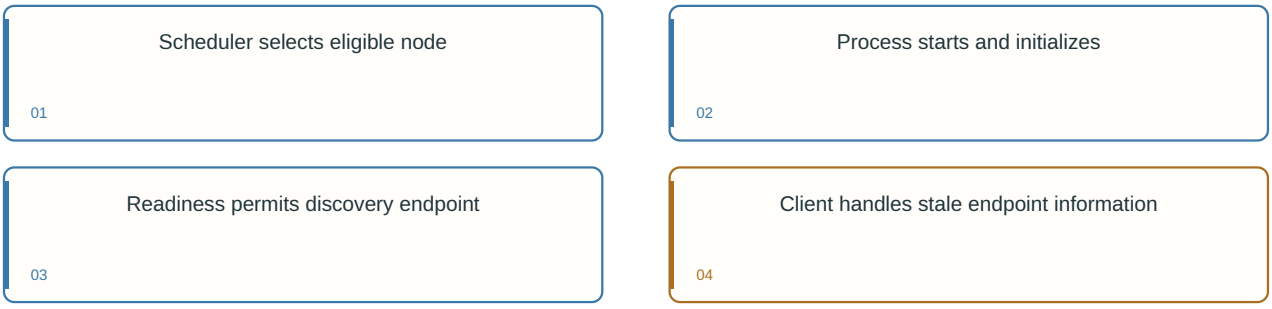


FIG. 20.2.2 **Scheduling and service discovery · the example**

Scheduling selects eligible resources; discovery advertises endpoints whose readiness and shared-state behaviour still need handling.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.2.3 · Autoscaling

Scaling up creates more workers, but each may open additional database connections. If the database is already saturated, the extra concurrency can worsen the queue. Scale useful capacity while respecting downstream limits.

Choose signals appropriate to the workload and include warmup, minimum capacity, and maximum spend. Scaling down needs draining: stop accepting new work, finish or transfer durable jobs, and release resources. Killing an unrecorded background task loses it. Autoscaling during retries also needs care, because attempt traffic can rise while useful demand is unchanged. The platform and application admission rules should act together.

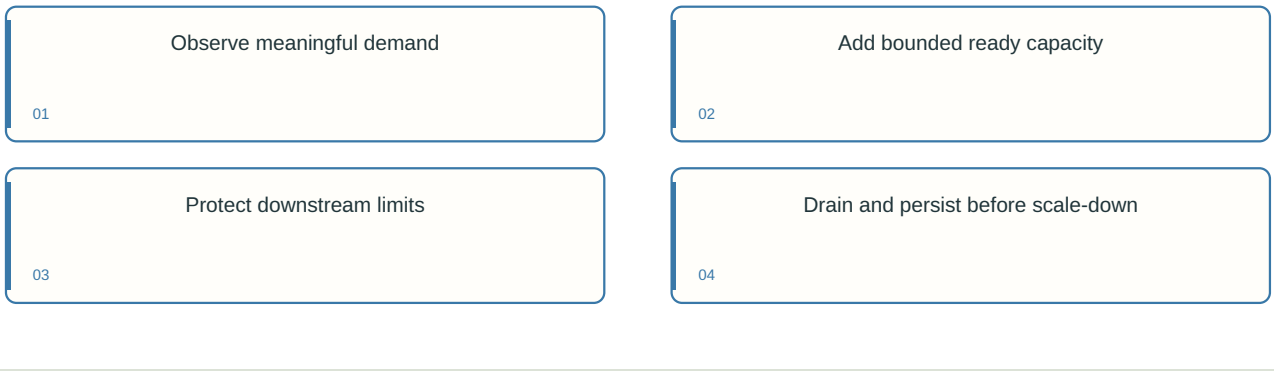


FIG. 20.2.3 **Autoscaling · the example**

Autoscaling must include useful signals, startup, draining, durable work, downstream limits, and spending bounds.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.2.4 · Configuration and secret delivery

A bad configuration can disable every otherwise healthy replica without changing application code. Treat configuration delivery as a versioned, validated operation with a recovery path.

Roll out risky changes in a controlled way and record the effective version in telemetry. Some settings can reload safely; others require replacing connections or processes. Secrets need stricter access and logging controls than ordinary values. Verify that the application actually uses the intended version. This gives an incident investigation something concrete to compare with the time the behaviour changed.

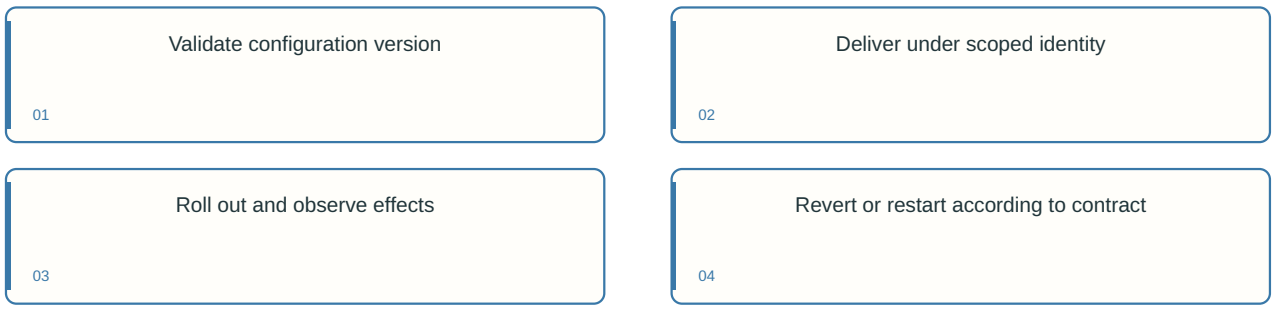


FIG. 20.2.4 **Configuration and secret delivery · the example**

Configuration and secret delivery need validation, versions, rollout/reload semantics, and a recovery path.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.2.5 · Immutable infrastructure

Instead of manually changing a server until it works, immutable infrastructure replaces it from a defined artifact and configuration. Repeatability improves when provisioning and inputs are controlled.

Runtime data still changes, and rollback still depends on compatible state. Keep persistence and migrations explicit. If an emergency edit is needed, capture it in the maintained configuration so replacement does not lose it or recreate drift. Check the running artifact and values as well as the deployment's intention. Rebuilding the same machine definition does not automatically restore its lost data or reverse an external effect.

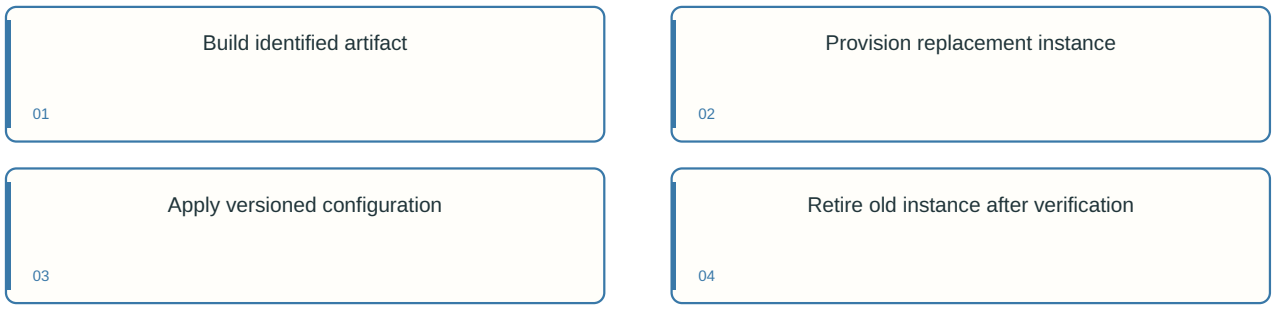


FIG. 20.2.5 **Immutable infrastructure · the example**

Immutable infrastructure replaces defined instances, while runtime state, migrations, and emergency drift remain managed concerns.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How can rollback restore old code while leaving incompatible data or effects?

Old code can return while new schemas, values, provider calls, and published messages remain. Maintain compatibility during overlap and identify the point where contraction ends safe rollback. Some outcomes need forward repair or compensation. Restoring an artifact cannot reverse every state transition it previously performed.

Remember: Running traffic and creating replacement capacity can depend on different paths.

Try applying it: Which dependency could let current instances serve while preventing every replacement from starting?

Unit 20 exercises → · Unit 20 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. The Twelve-Factor App ↗

   Original methodology. Application deployment conventions.

   Access reviewed · 21 October 2026

2. Kubernetes components ↗

   Official documentation · accessed 21 Oct 2026. Platform reference for Unit 20.

   Access reviewed · 21 October 2026

3. Canarying Releases — The SRE Workbook ↗

   Engineering book. Representative rollout evaluation and stop conditions.

   Access reviewed · 21 October 2026

Releases, flags & schema migrations

Prerequisites: 20.2 · Control planes, scheduling & configuration 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A release writes a new field, then a rollback restores a reader that cannot understand it. Keep artifacts, flags, configuration, and migration stages compatible while versions overlap.

20.3.1 · Infrastructure as Code

Infrastructure as Code makes resource definitions and policies reviewable and repeatable. A change plan can reveal proposed replacements, but the real effect depends on provider behaviour, stored state, privileges, and drift.

A resource replacement can be destructive for a stateful service. Understand lifecycle and recovery before applying it. Coordinate concurrent updates and protect secrets outside ordinary tracked values. Verify the resulting state. Also preserve the state information and credentials needed to recover the infrastructure definition itself. Code is a useful source of desired configuration, while restoration remains an operating procedure with dependencies.

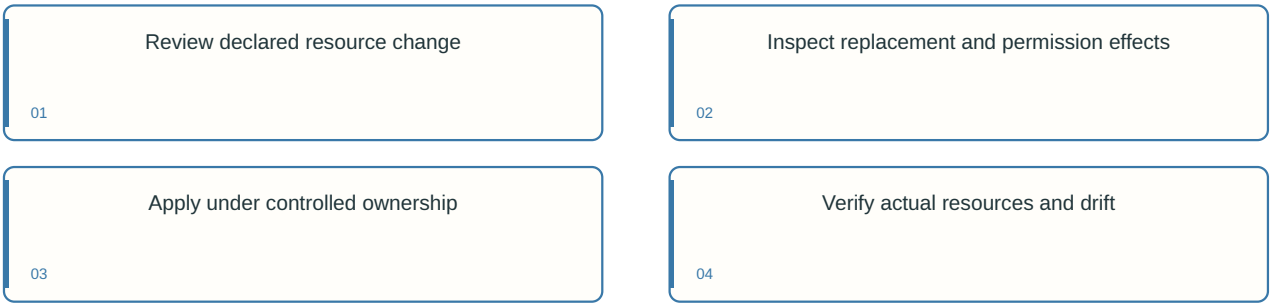


FIG. 20.3.1 **Infrastructure as Code · the example**

Infrastructure as Code makes desired resources reviewable without eliminating state, privileges, drift, or destructive-change risks.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.3.2 · CI/CD

Continuous integration checks changes. Delivery prepares a releasable artifact, and deployment places that artifact into an environment. Preserve identity from the source through the built artifact to the version actually running.

Use scoped pipeline permissions and meaningful checks for the release risks. A test in one environment may miss mixed-version traffic during a database migration. Keep untrusted build input away from release secrets. Automation reduces manual variation, but its own partial failures need a supported recovery path. A green pipeline is evidence of its defined checks, rather than proof of every production assumption.

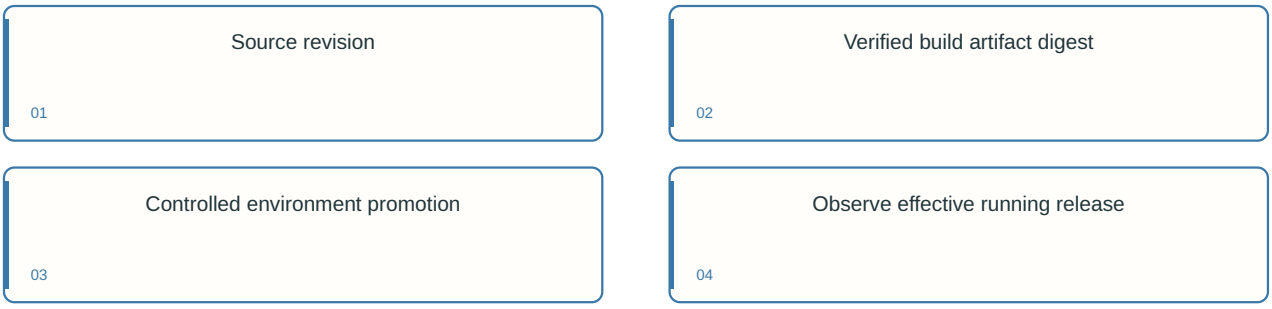


FIG. 20.3.2 **CI/CD · the example**

CI, delivery, and deployment preserve artifact identity and run checks appropriate to actual release risks.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.3.3 · Rolling, blue-green, and canary deployments

Rolling deployment replaces instances gradually. Blue-green prepares another environment before switching traffic. Canary sends a controlled traffic subset to the new version before wider exposure.

Each needs compatible state, draining, spare capacity, and verification. A canary receiving only cheap requests can miss an expensive-path regression. Choose cohorts and stop conditions using the meaningful outcomes. Blue-green can duplicate application capacity while both versions still share one mutable database; switching traffic back does not reverse its changes. Select the release arrangement according to the risk and operating cost, including the data transition.

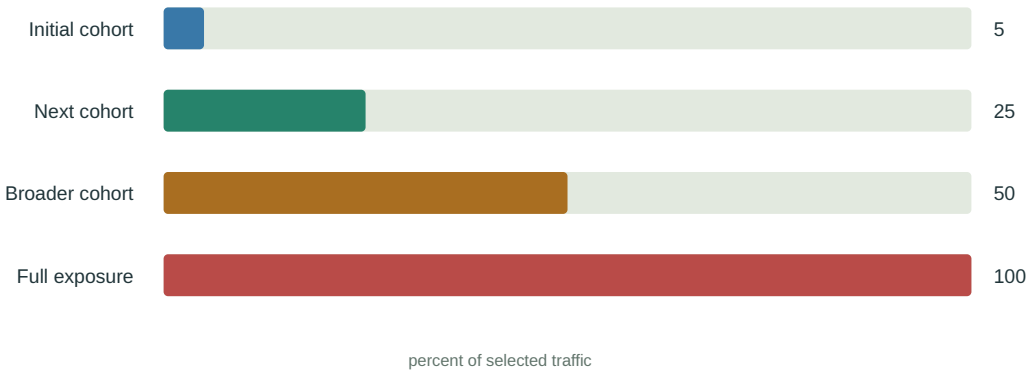


FIG. 20.3.3 **Canary exposure expands only after verification**

Percentages are an illustrative rollout plan, not an automatic safety rule. Each increase depends on meaningful representative checks and stop conditions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.3.4 · Feature flags

A feature flag can release code separately from exposing a behaviour. Give the flag an owner, scope, safe default, observable state, and removal plan. Combinations of flags also create combinations to test.

The flag is not authorisation. A browser-controlled setting must not grant a privileged server operation. Consider disabling after the feature has written a new data format: older code may still need to understand it. Keep state compatibility and migration separate from the exposure switch. Flags offer control over some transitions, while stale flags and hidden configuration can accumulate their own complexity.

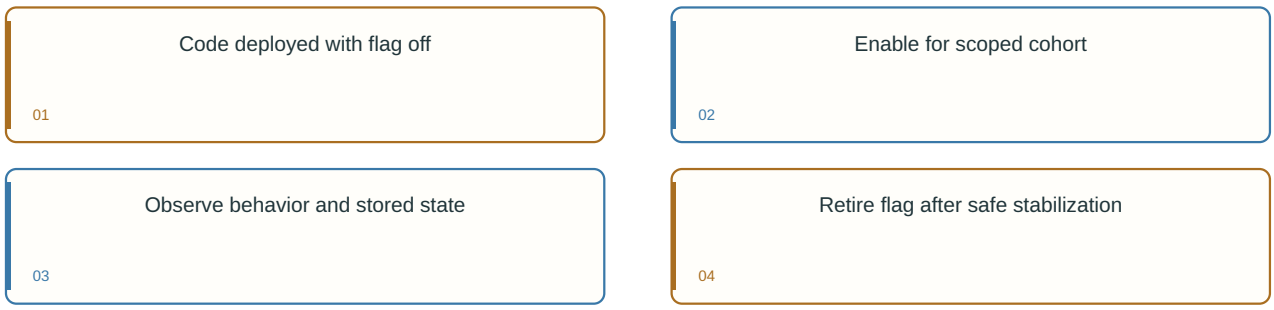


FIG. 20.3.4 **Feature flags · the example**

Feature flags control exposure with ownership and safe defaults, but do not reverse stored effects or replace permission.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.3.5 · Rollbacks and database migrations

A release removes a database column. Rolling back its code now restores a reader that needs the missing column. The old artifact exists, but the state is no longer compatible with it.

Expand-and-contract avoids this abrupt transition. Add compatible structures, deploy code that tolerates the overlap, backfill with bounded resumable work, and verify. Remove old structures only after old readers are gone. Mark when rollback stops being supported. Code, configuration, schema, and external effects have different reversal rules, so a rollback procedure must name the assumptions it relies on.

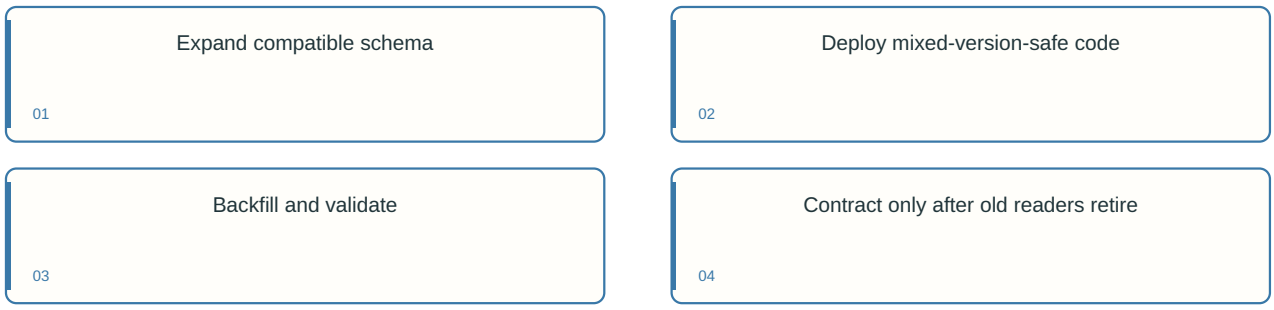


FIG. 20.3.5 **Rollbacks and database migrations · the example**

Rollback depends on compatible state; expand-and-contract maintains a supported overlap before removing old structures.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.3.6 · Multi-environment configuration

Production uses different endpoints and credentials from staging. Those are environment values. The meaning of a payment amount should remain the same unless a deliberate business policy says otherwise.

Validate required fields and types at startup or controlled reload. Use explicit versions and careful defaults for security and retention. An absent value should fail clearly when guessing could route data to the wrong region or tenant. Avoid scattered environment-specific conditionals that no test setup reproduces. Keep the effective configuration understandable so differences between environments are intentional rather than accidental business semantics.

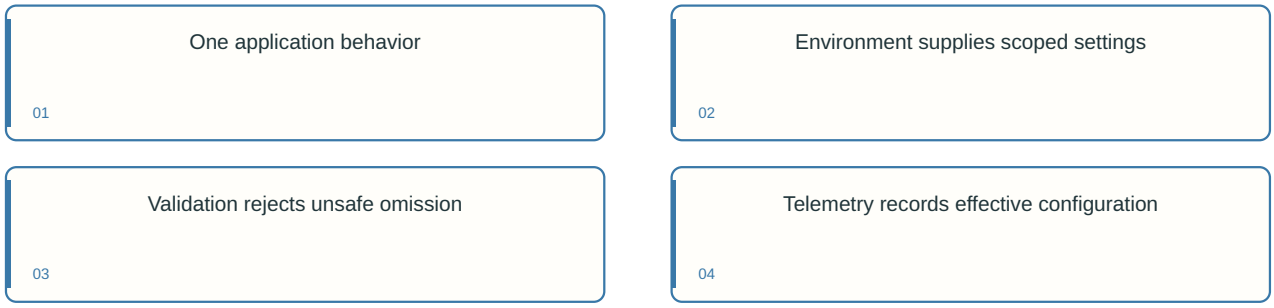


FIG. 20.3.6 **Multi-environment configuration · the example**

Environment values should be validated and explicit without silently changing business meanings.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · How can rollback restore old code while leaving incompatible data or effects?

Old code can return while new schemas, values, provider calls, and published messages remain. Maintain compatibility during overlap and identify the point where contraction ends safe rollback. Some outcomes need forward repair or compensation. Restoring an artifact cannot reverse every state transition it previously performed.

Remember: Release safety includes data and protocol transitions, not only selecting an old executable.

Try applying it: At what point in expand-and-contract does rollback to the old reader become unsafe?

Unit 20 exercises → · Unit 20 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. The Twelve-Factor App ↗

   Original methodology. Application deployment conventions.

   Access reviewed · 21 October 2026

2. Kubernetes components ↗

   Official documentation · accessed 21 Oct 2026. Platform reference for Unit 20.

   Access reviewed · 21 October 2026

3. Canarying Releases — The SRE Workbook ↗

   Engineering book. Representative rollout evaluation and stop conditions.

   Access reviewed · 21 October 2026

Platforms, serverless & ownership

Prerequisites: 20.3 · Releases, flags & schema migrations 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A managed platform offers useful capabilities while leaving application choices in your hands. Compare delegated work, limits, topologies, and internal developer needs under the same service requirements.

20.4.1 · Twelve-Factor App principles

The Twelve-Factor App describes operational practices such as explicit dependencies and configuration, backing-service treatment, process discipline, and logs as event streams. These ideas can improve repeatability for suitable service applications.

They are methodology rather than a universal proof of quality. Stateful work and current platforms still need explicit persistence, identity, limits, and recovery. Apply each practice's purpose to the actual workload. Checking twelve labels does not establish authorisation, transaction correctness, or usable backups. A design remains accountable to its requirements and observed behaviour, even when it follows a well-known set of conventions.

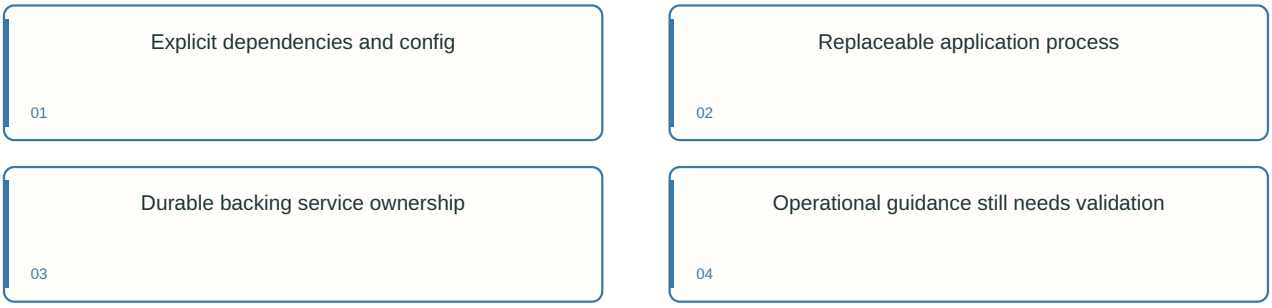


FIG. 20.4.1 **Twelve-Factor App principles · the example**

Twelve-Factor practices guide repeatable service operation while workload-specific correctness and recovery remain required.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.4.2 · Serverless compute

A function platform manages some provisioning and invocation work. An activity can still time out, restart, or be invoked again. Rapid scaling may also create too many concurrent calls to one database.

Externalise required state, bound downstream concurrency, and use operation identities for effects. Compare startup delay, sustained cost, portability, and remaining operating work with container or VM alternatives. The application still needs monitoring and secure access. “Serverless” describes a delegation arrangement, not unlimited capacity or absence of failures. Use it where its execution model suits the work and its recovery requirements.

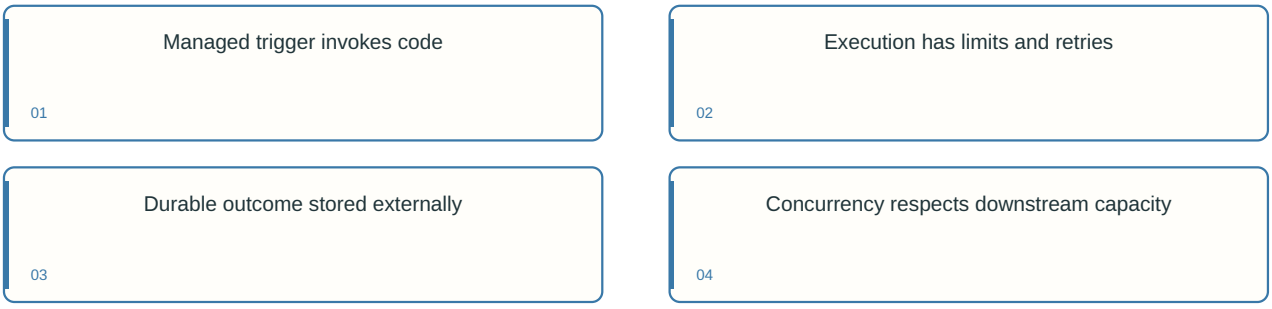


FIG. 20.4.2 **Serverless compute · the example**

Serverless delegates selected platform duties while retaining limits, durable state, identity, and safe retries.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.4.3 · Cloud vs on-premises trade-offs

Cloud and on-premises operation place responsibilities in different hands. Compare procurement delay, expertise, networking, data placement, managed capabilities, recovery, and exit costs for the same workload.

Cloud capacity remains subject to quotas and provisioning time. Predictable owned capacity can be efficient in some circumstances but requires equipment and disaster management. Avoid generic cheapest-provider claims without current rates and full assumptions. The atlas uses hypothetical cost units for arithmetic; a procurement choice needs actual quotations and organisational requirements. The useful comparison is the complete operating arrangement rather than one machine price.

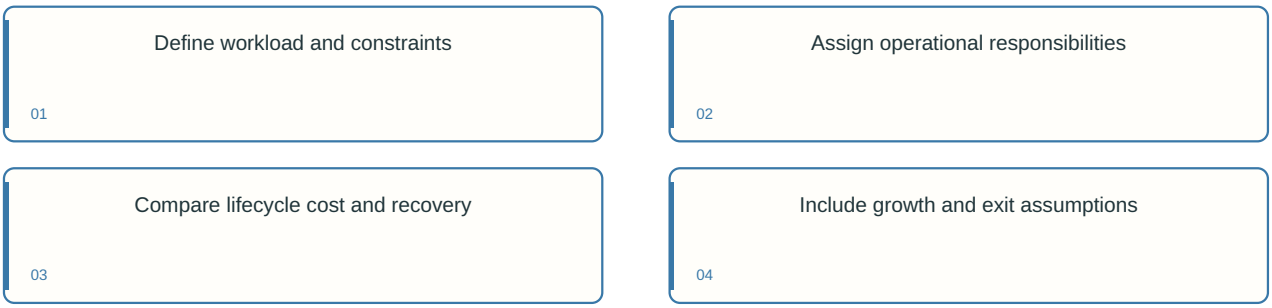


FIG. 20.4.3 **Cloud vs on-premises trade-offs · the example**

Cloud/on-premises comparisons need equal workload, ownership, procurement, recovery, and verified cost assumptions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.4.4 · Infrastructure ownership and managed services

A provider manages database patching and replication. The application team still chooses data access, schema, acknowledgment settings, retention, and how recovery is validated. Managed service means selected responsibilities have moved, not that business responsibility has disappeared.

Read the actual contract and failure boundaries. Self-management offers control while requiring expertise and on-call work. In either arrangement, test client behaviour during maintenance, throttling, failover, and credential changes. List what the team still owns. That makes cost and staffing comparisons more accurate than describing an entire system as simply managed or self-hosted.

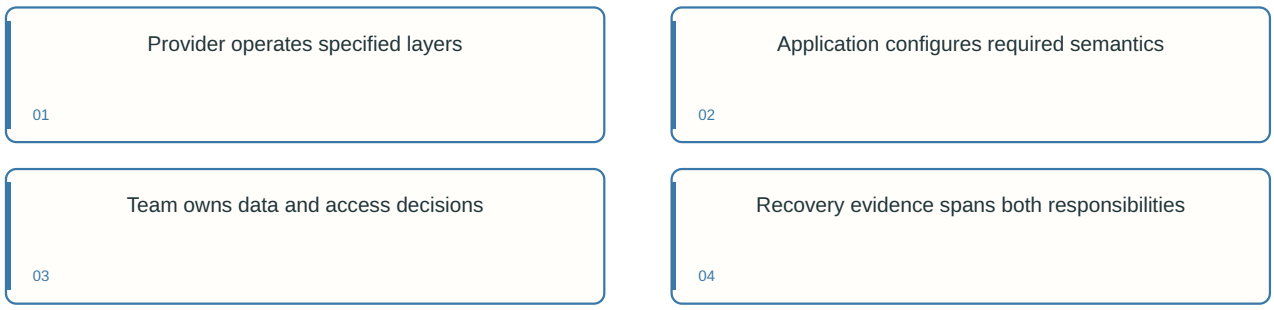


FIG. 20.4.4 **Infrastructure ownership and managed services · the example**

Managed services transfer selected duties while clients retain responsibilities defined by their actual contract.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.4.5 · Deployment topologies and networking

A module diagram shows application responsibilities. A deployment topology shows their processes, machines, zones, networks, data, and trust boundaries. It reveals whether two replicas share the same host or storage dependency.

Include relevant ingress, egress, encryption termination, private routes, and operating access. Follow the loss of one placement: how does a replacement obtain its configuration and state? Label whether arrows represent traffic, control, or another dependency. The purpose is to explain actual communication and failure relationships, rather than redraw logical boxes with infrastructure logos.

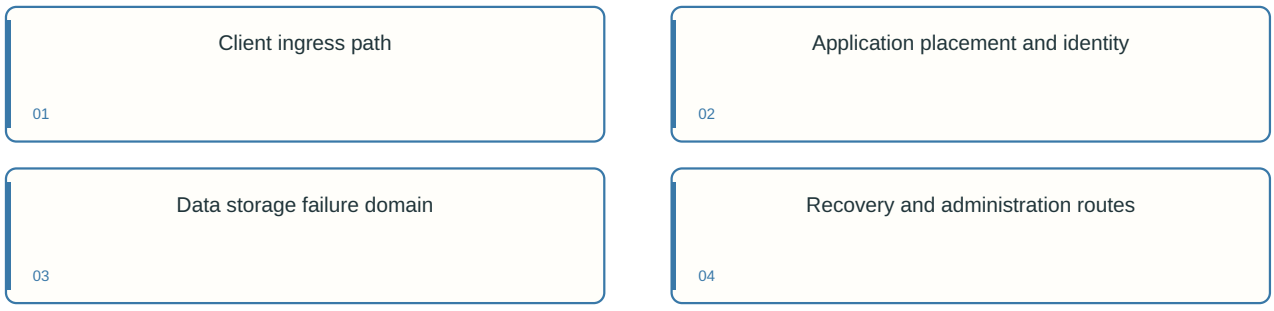


FIG. 20.4.5 **Deployment topologies and networking · the example**

Deployment topology places execution, state, routes, trust, and failure domains on real infrastructure.

Author-created illustration. Numbers are illustrative unless stated otherwise.

20.4.6 · Platform engineering basics

A platform team can offer a reusable way to build artifacts, obtain identity, emit telemetry, and deploy with tested defaults. Platform engineering treats development teams as users of those capabilities.

Provide understandable limits, ownership, and supported escape paths for different workloads. An opaque mandatory framework can hide decisions that application teams still need to make. Measure useful adoption, reliability, and operating burden rather than only the number of internal tools created. A platform abstraction earns its place when it reduces repeated work while exposing the failure and resource behaviour relevant to its users.

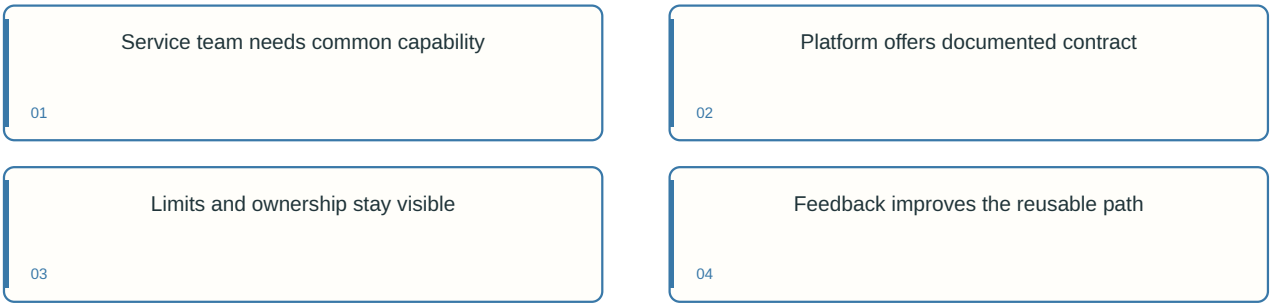


FIG. 20.4.6 **Platform engineering basics · the example**

Platform engineering offers reusable operating capabilities whose limits and ownership remain visible to developer users.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PSEUDOCODE · SAFE RELEASE PROGRESSION

```
add_compatible_schema_field()
deploy_mixed_version_safe_code()
backfill_with_bounded_progress()
verify_old_and_new_readers()
retire_old_readers()
remove_obsolete_schema_only_when_safe()
```

Old and new readers coexist during compatible expansion and backfill. Retire old readers before removing the structures they need.

Rollback is safe only while its state and protocol assumptions still hold. Destructive contraction or external effects may require forward repair. Verify the actual running artifact and configuration too.

Download the executable mechanism collection ↓ · Follow the connected complete design →

▼ Think further · How can rollback restore old code while leaving incompatible data or effects?

Old code can return while new schemas, values, provider calls, and published messages remain. Maintain compatibility during overlap and identify the point where contraction ends safe rollback. Some outcomes need forward repair or compensation. Restoring an artifact cannot reverse every state transition it previously performed.

Remember: A platform abstraction should simplify repeated work while exposing the limits its users must manage.

Try applying it: Which correctness and recovery responsibilities remain with a team using a managed database?

Unit 20 exercises → · Unit 20 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. The Twelve-Factor App ↗

   Original methodology. Application deployment conventions.

   `Access reviewed · 21 October 2026`

2. Kubernetes components ↗

   Official documentation · accessed 21 Oct 2026. Platform reference for Unit 20.

   `Access reviewed · 21 October 2026`

3. Canarying Releases — The SRE Workbook ↗

   Engineering book. Representative rollout evaluation and stop conditions.

   `Access reviewed · 21 October 2026`

PART 5 / UNIT 21

Performance, Cost Optimization, and System Validation

A service misses its response-time goal. Budget the path, locate the limit, and design a benchmark that can test the proposed explanation without hiding waiting or errors.

The running example

Compare three production architectures for a service with variable traffic. Model monthly costs, p99 latency, availability, and operational complexity under explicit assumptions.

Unit 21 assessment → Explained solutions →

Concept coverage

Unit plates · the required visual vocabulary

Figure numbers with .0 denote unit-level reference plates. Chapter figures retain their chapter numbers.

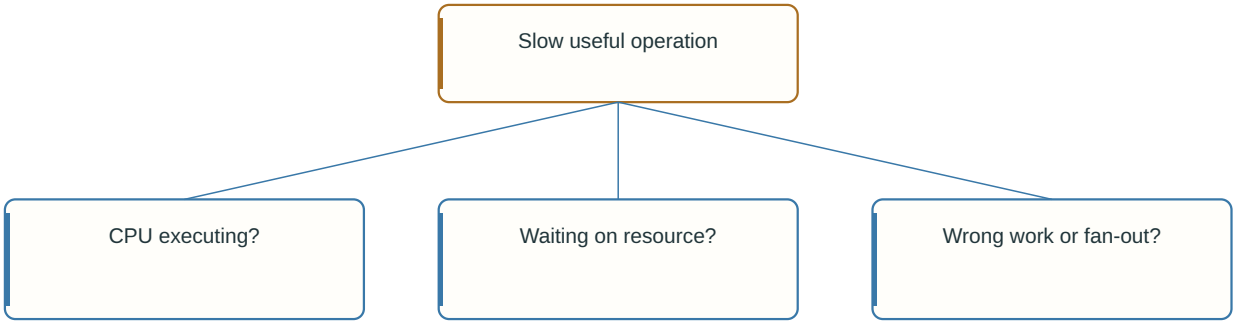


FIG. 21.0.1 **Performance bottleneck tree**

Start with a symptom, distinguish execution from waiting, then inspect the resource on the critical path.

Author-created illustration. Numbers are illustrative unless stated otherwise.

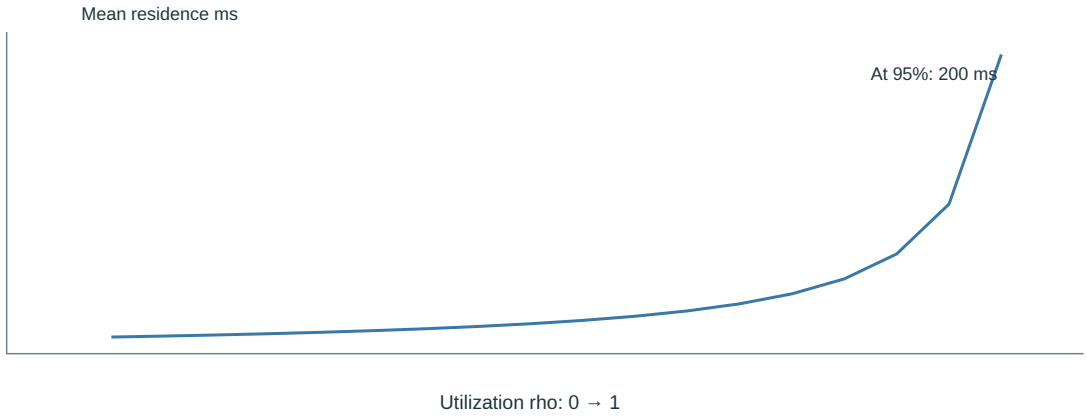


FIG. 21.0.2 **throughput-latency curve**

In this single-server M/M/1 model, mean waiting plus service time grows near full utilization. The model assumes Poisson arrivals and independent exponential service times. Measure real workloads and p99 separately.

Author-created illustration. Numbers are illustrative unless stated otherwise.

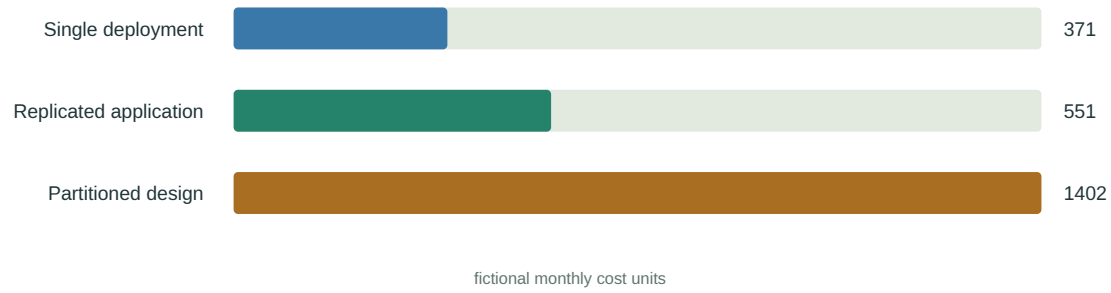


FIG. 21.0.3 **Architecture costs are not capacity guarantees**

The complete cost worksheet uses fictional monthly units. Each candidate still needs independent checks for correctness, performance, and failure requirements.

Author-created illustration. Numbers are illustrative unless stated otherwise.

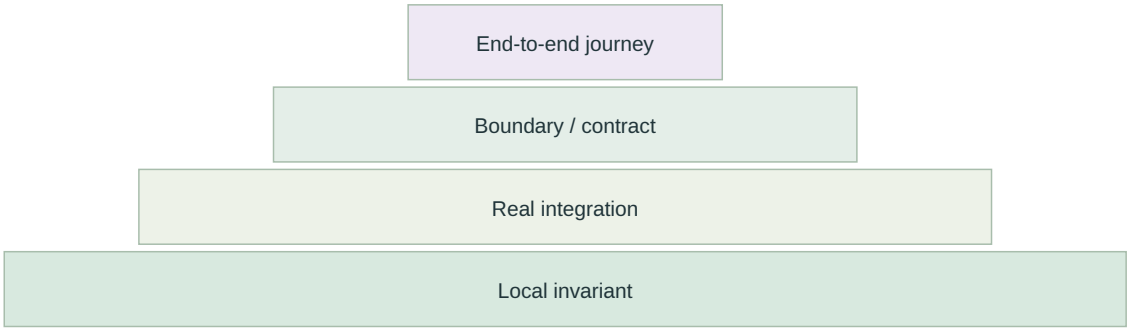


FIG. 21.0.4 **test pyramid**

The conceptual pyramid distinguishes feedback scope and cost, not a mandatory numerical ratio of tests.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Criterion	Hard constraint?	Evidence
No duplicate local posting	Yes	Transactional invariant test
p99 target	Required objective	Representative load distribution
Recovery target	Required objective	Restore/failover drill
Monthly cost	Preference or budget	Explicit unit-cost model

FIG. 21.0.5 **design decision matrix**

Feasibility precedes preference scoring. An unmeasured objective must not be reported as achieved.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Budgets, bottlenecks & honest benchmarks

Prerequisites: Requirements, Estimation, and Engineering Trade-offs, Runtime, Processes, Threads, and Concurrency, Database Internals, Transactions, and Concurrency Control, Scalability, Load Balancing, and Data Distribution, Reliability, Fault Tolerance, and Disaster Recovery, Observability, Monitoring, and Incident Engineering, Deployment, Cloud Infrastructure, and Platform Architecture

6 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A service misses its response-time goal. Budget the path, locate the limit, and design a benchmark that can test the proposed explanation without hiding waiting or errors.

21.1.1 · Performance budgets

Assume a request has a 200 ms budget: 30 ms for edge and network, 20 ms for queues, 100 ms for data access, and 50 ms for computation and response. These allocations add to the end-to-end target.

A performance budget makes the expected costs visible before implementation. Validate the assumed allocations with measurements under the stated workload. Include connection waits, encoding, tails, and failure paths, rather than budgeting only the main query. Work that exhausts the budget before reaching the query still matters to the user. A proposed allocation is an objective to check, not a measured result.

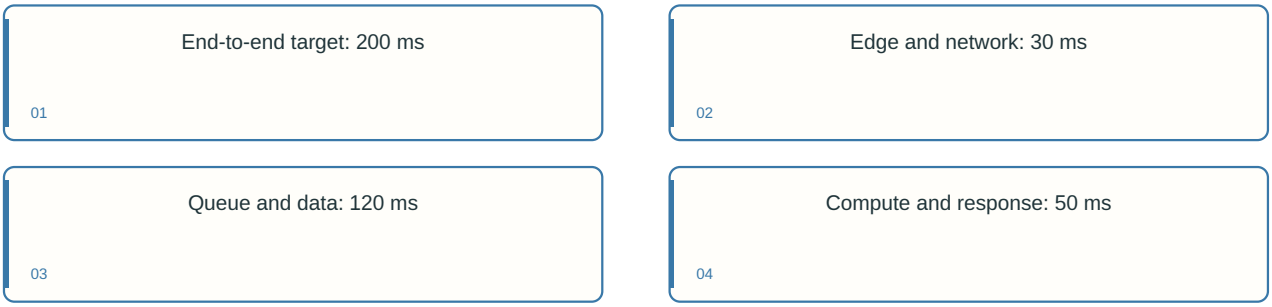


FIG. 21.1.1 **Performance budgets · the example**

Performance budgets allocate acceptable end-to-end costs under a workload and remain targets until measured.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.1.2 · CPU, memory, disk, network, and database bottlenecks

A slow request may compute heavily, wait on disk, wait for a lock, or queue for a database connection. High CPU suggests one set of hypotheses; low CPU does not rule out a severe delay.

Trace the critical path and resource queues. Memory pressure can cause allocation stalls or termination, while transfer volume can saturate a network. A bottleneck tree organises possible explanations so measurements can distinguish them. Change the component implicated by the evidence rather than adjusting everything at once. An improvement can reveal another limit, so repeat the observation after the change.

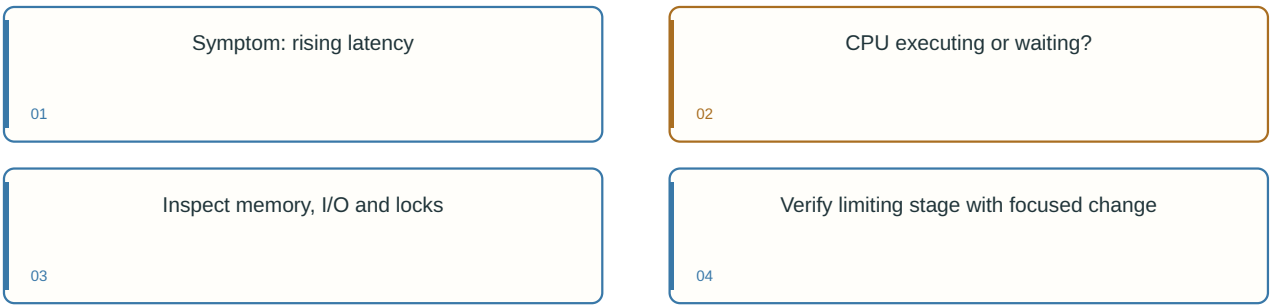


FIG. 21.1.2 **CPU, memory, disk, network, and database bottlenecks · the example**

Bottlenecks can be computation or waiting at CPU, memory, storage, network, locks, or data services.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.1.3 · Profiling and benchmarking

A profile shows where runtime work occurs. A benchmark compares behaviour under chosen conditions. To make a result useful, record the workload, data distribution, capacity, versions, warmup, concurrency, duration, and success criteria.

A fast isolated function does not establish application throughput. Include variability, failures, and resource cost, and compare equivalent service promises. Preserve enough setup to reproduce a change in behaviour after a dependency or schema update. The value of a benchmark is the question it answers under its conditions, rather than the size of its most impressive number.

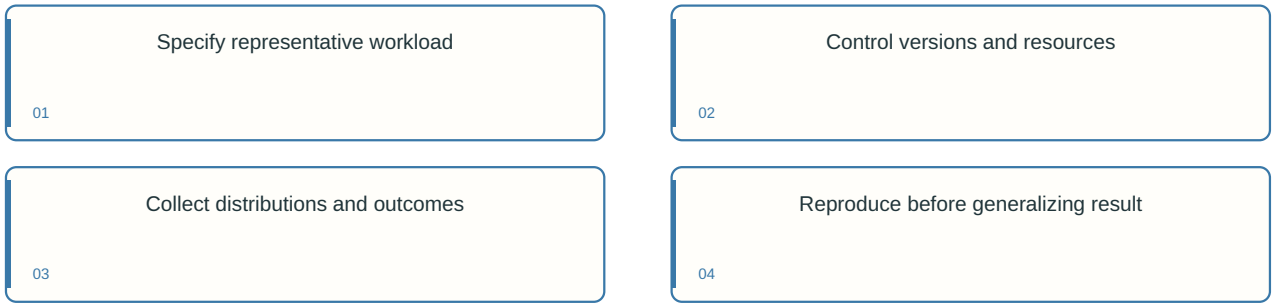


FIG. 21.1.3 **Profiling and benchmarking · the example**

Profiles locate runtime work; benchmarks compare controlled conditions with disclosed workloads, versions, variation, and outcomes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.1.4 · Load, stress, spike, and soak testing

A load test checks expected demand. A stress test explores limits. A spike test examines abrupt changes, while a soak test looks for long-duration effects such as leaks or compaction backlog.

Choose the test for the uncertainty. Also inspect the generator: one that waits for each slow reply may stop issuing traffic that real users would continue to send. Check its own capacity and record errors, abandonment, waiting, and correct business results. High request throughput with lost reservations is a failed correctness test. Each measurement needs a workload and an outcome definition that match the claim being investigated.

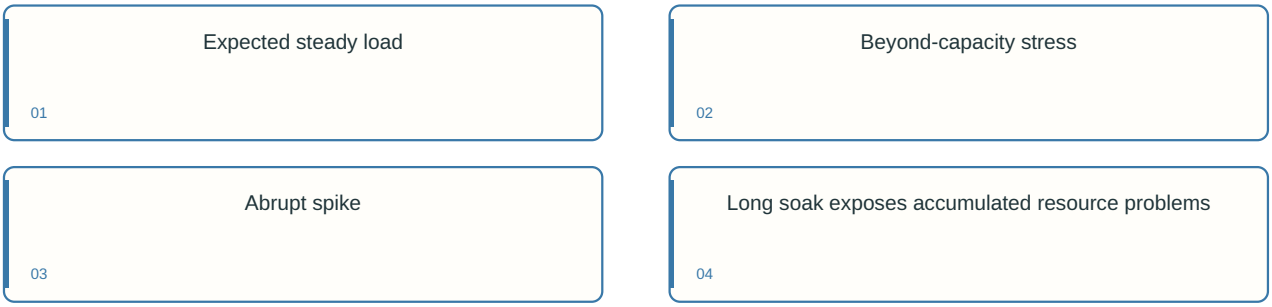


FIG. 21.1.4 **Load, stress, spike, and soak testing · the example**

Load, stress, spike, and soak tests answer different expected-load, limit, abrupt-change, and long-duration questions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.1.5 · Benchmark methodology and statistical pitfalls

A generator waits for a slow response before sending the next request, although real arrivals would continue independently. The test omits some waiting that users would experience. This is a common form of coordinated omission.

Other pitfalls include tiny samples, unrealistic data, undisclosed warm caches, omitted failures, and incompatible versions. Report the method and distributions with appropriate precision. Change one factor or use a justified experimental design. A result supports the tested conditions; a different traffic mix, failure state, or data distribution may produce another outcome. Preserve the context rather than presenting the number as a general property of the system.

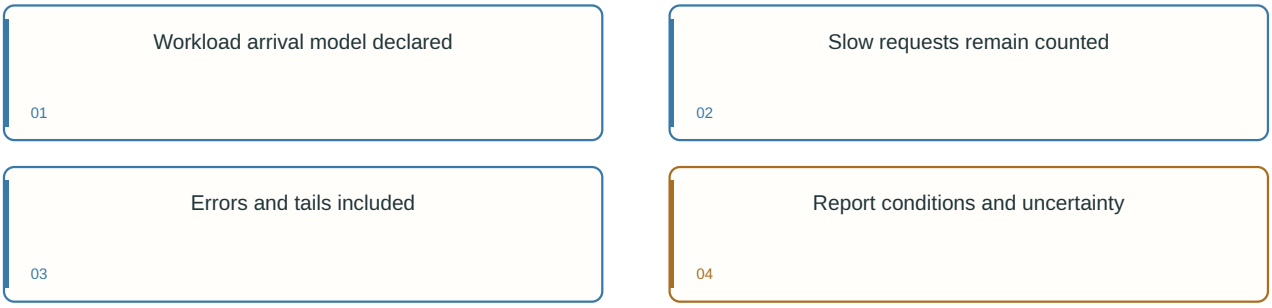


FIG. 21.1.5 **Benchmark methodology and statistical pitfalls · the example**

Benchmark validity depends on realistic input, sufficient observations, honest failures, and a generator that represents arrivals correctly.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.1.6 · Tail latency analysis

A service's p99 rises from 200 ms to five seconds. First ask whether a rare expensive operation changed or whether many operations slowed. Split queueing, execution, network, and required fan-out.

Use population distributions and representative traces together. Averaging individual machines' percentiles does not reconstruct a fleet percentile, and very high percentiles need enough observations to estimate usefully. One slow trace suggests a path to investigate, not a universal explanation. Compare normal and slow cases under the same workload. Tail analysis connects the distribution to its causes instead of assuming the mean describes all users.

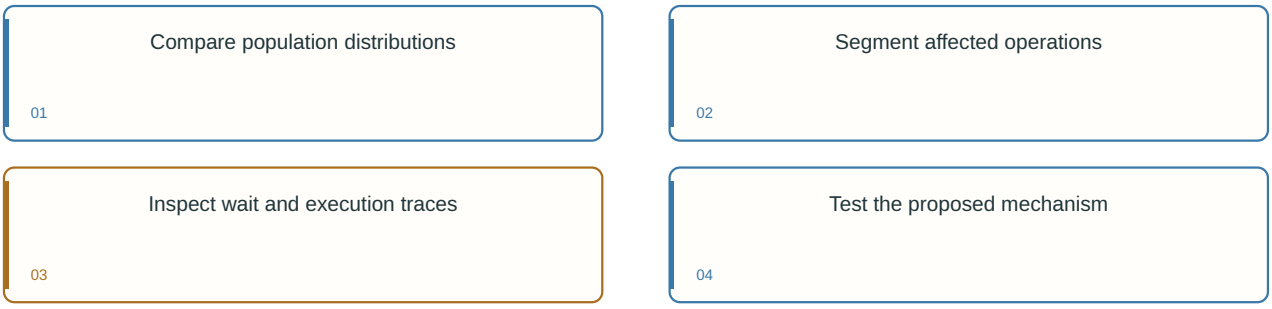


FIG. 21.1.6 **Tail latency analysis · the example**

Tail analysis connects timing distributions and representative paths without incorrectly averaging percentile summaries.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · When could a more expensive architecture be simpler to operate overall?

Compare total work for people as well as the invoice. A managed service may reduce patching, backup, or recovery burden while adding charges and leaving application responsibilities. Evaluate the same correctness and service goals with explicit staffing assumptions. Choose the design the team can understand, change, and recover reliably.

Remember: Performance evidence is meaningful only with its workload, conditions, and outcome definition.

Try applying it: How could a load generator accidentally omit the very delays users would experience?

Unit 21 exercises → · Unit 21 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. The Practical Test Pyramid — Ham Vocke ↗
   Original engineering article. Test scope and feedback trade-offs, not a mandatory test-count ratio.
   `Access reviewed · 21 October 2026`

2. Handling Overload — Google SRE ↗
   Engineering book. Admission, capacity, and overload response.
   `Access reviewed · 21 October 2026`

3. The Tail at Scale — Dean and Barroso, 2013 ↗
   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.
   `Access reviewed · 21 October 2026`

Capacity, architecture & cost per outcome

Prerequisites: 21.1 · Budgets, bottlenecks & honest benchmarks 6 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Extra nodes, caches, and transfer can change both capacity and cost. Compare complete architectures using explicit inputs and useful outcomes rather than one attractive infrastructure rate.

21.2.1 · Queue saturation and throughput ceilings

As offered traffic rises, useful completions eventually stop increasing. Extra work then waits or is rejected. A queue stores that mismatch; it supplies no additional service capacity.

Observe successful outcomes, latency, and saturation to locate the throughput knee, where additional demand sharply worsens experience. Reserve margin for variability, background work, maintenance, and the failures the system promises to tolerate. A short benchmark maximum is not automatically a safe production target. State which amount of useful traffic remains supportable after a node or zone is lost, rather than sizing only the healthy fleet.

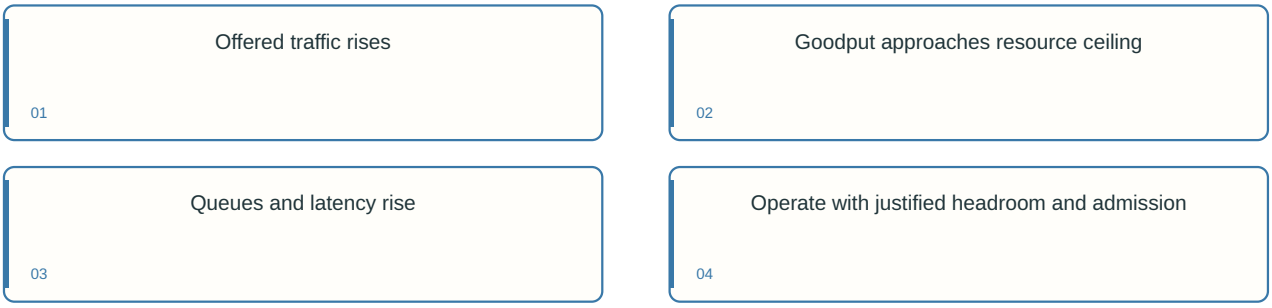


FIG. 21.2.1 **Queue saturation and throughput ceilings · the example**

Saturation limits useful completion; added arrivals become waiting or rejection, requiring operating margin.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.2.2 · Performance tuning vs architecture changes

A query spends most of its time scanning rows. An index may solve the problem without changing data ownership. Sharding that database would introduce routing and migration while possibly leaving the expensive query design intact.

Performance tuning changes an implementation within its arrangement; an architecture change alters major placement, ownership, or interaction. Either may be justified. Measure before and after, and include correctness, operating work, and migration risk. Some central invariants really require a different model to scale. Choose the intervention by the demonstrated limit, rather than assuming the larger architectural change is the more effective one.

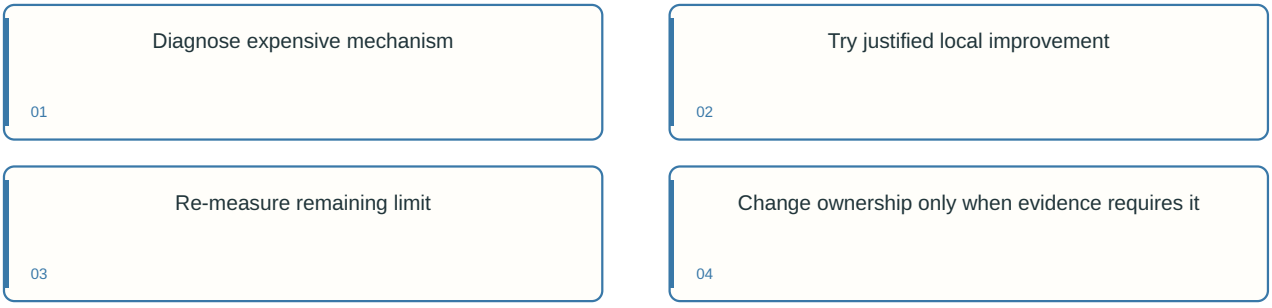


FIG. 21.2.2 **Performance tuning vs architecture changes · the example**

Tuning changes implementation costs; architecture changes alter major ownership or placement and need proportionate evidence.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.2.3 · Horizontal scaling economics

Compare one larger deployment, replicated application instances sharing a data owner, and partitioned storage. More nodes add baseline memory, discovery, coordination, and network work. Their advantage depends on which load can actually be divided.

Use equal requirements and failure headroom. A partitioned design may cost more at small scale while extending a later ceiling, but that is a hypothesis to test. Keep price, operating labour, and traffic assumptions explicit. Hypothetical rates can teach arithmetic; actual spending decisions need current prices. Node count alone cannot establish useful capacity or a lower cost per successful outcome.

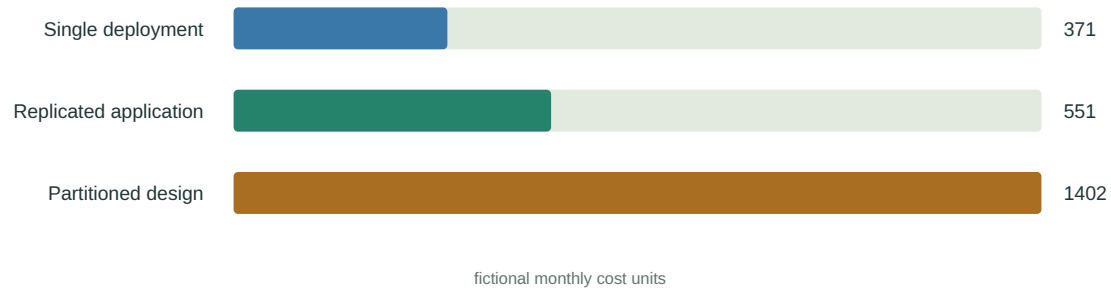


FIG. 21.2.3 **Architecture costs are not capacity guarantees**

The complete cost worksheet uses fictional monthly units. Each candidate still needs independent checks for correctness, performance, and failure requirements.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.2.4 · Compute, storage, networking, and telemetry costs

A cheap computation can produce a large response sent millions of times. Transfer may dominate its total cost. Follow the cost path through compute, requests, storage, indexes, replicas, backups, networking, and telemetry.

Include operational work where relevant. A cache can save source computation while adding memory and invalidation cost. Compression can save transfer while adding CPU. The worked prices in this atlas are fictional inputs, not current provider quotations. Keep the units and accounting period visible so the reader can replace the inputs and understand which part of the cost changes.

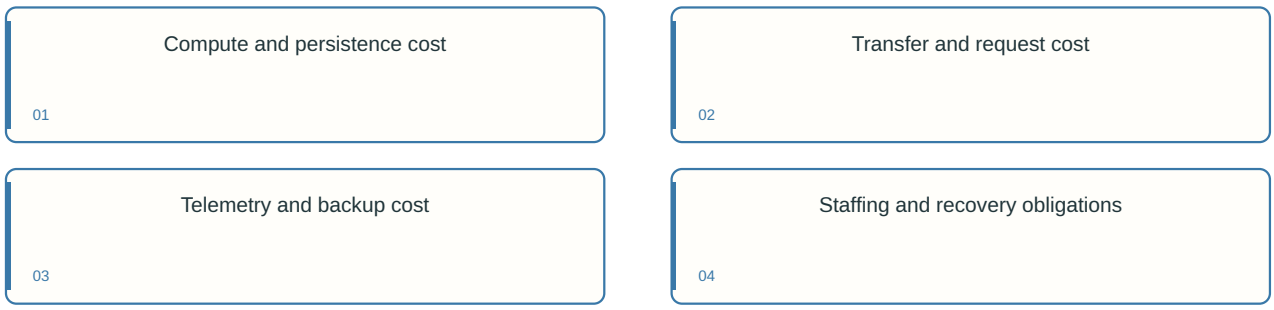


FIG. 21.2.4 **Compute, storage, networking, and telemetry costs · the example**

Total cost follows compute, requests, data, copies, transfer, telemetry, and operating work under explicit rates.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.2.5 · Cost per request or transaction

Assume monthly cost is 3,000 units and the system completes 150,000 useful business outcomes. The unit cost is 3,000/150,000 = 0.02 units per outcome.

Dividing by every attempt can conceal waste if many attempts fail or repeat. Choose a meaningful denominator such as completed orders or delivered documents, while retaining technical breakdowns for diagnosis. State the period and included expenses. Compare successful work under equivalent requirements; an operation that returns quickly but loses its result is not the same outcome. This keeps cost optimisation tied to the service users actually receive.

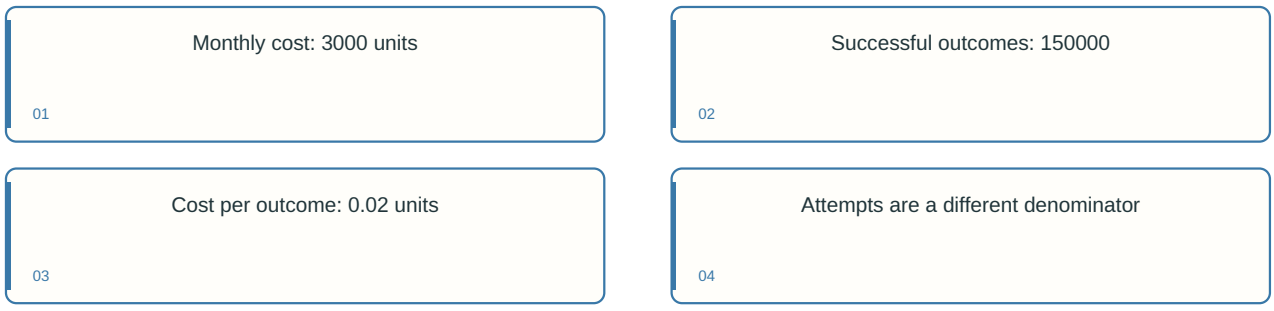


FIG. 21.2.5 **Cost per request or transaction · the example**

Cost per useful outcome avoids denominators inflated by failed or repeated attempts.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.2.6 · Capacity reservation vs elastic capacity

A steady baseline can suit committed capacity, while occasional bursts may favour capacity added when needed. Reservations trade commitment and idle-risk for particular commercial terms. Elastic resources still need provisioning time and may have different rates.

Compare baseline utilisation, burst duration, growth uncertainty, and failure needs. Buying for a rare peak can leave resources idle; waiting to scale after a short burst begins can arrive too late. A mixed approach may fit, but the balance needs measured traffic and current terms. Keep these commercial assumptions separate from the technical proof that surviving capacity meets the workload.

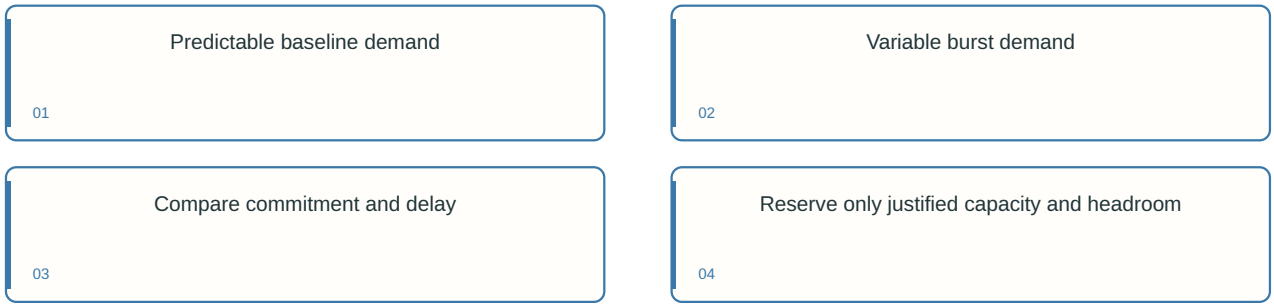


FIG. 21.2.6 **Capacity reservation vs elastic capacity · the example**

Reserved and elastic capacity trade commitment, idle risk, startup delay, and commercial terms against measured demand.

Author-created illustration. Numbers are illustrative unless stated otherwise.

A complete cost worksheet with fictional inputs

Monthly rates below are invented teaching inputs, not provider quotations. Capacity, p99, and availability are evaluation dimensions whose results must be measured; this cost table cannot establish them.

```
Fictional monthly cost units; compare only after checking requirements
```

Architecture	Compute	Data	Cache	Storage	Transfer	Telemetry	Total
Single deployment	40	300	0	1	20	10	371
Replicated application	160	300	50	1	20	20	551

Architecture	Compute	Data	Cache	Storage	Transfer	Telemetry	Total
Partitioned design	320	900	100	2	40	40	1402

Compare latency, availability, and operating work under stated assumptions

Construct a 100-request trace for each option: single deployment has 98 observations at 190 ms, one at 900 ms, and one at 1,400 ms; replicated application has 98 at 140 ms, one at 450 ms, and one at 800 ms; partitioned design has 98 at 110 ms, one at 600 ms, and one at 1,600 ms. Nearest-rank p99 is observation 99 after sorting. These synthetic traces deliberately show that lower common-case latency can coexist with worse tail latency. They predict no real deployment.

For a separate steady-state availability model, assume independent app-instance availability of 0.99 and data-owner availability of 0.999. An interchangeable pool of k apps is available with probability 1 − 0.01^k. The single and replicated options require one data owner. The partitioned operation in this example requires all three independent owners, so multiply by 0.999^3. Shared outages, overload, failover time, repair, and correlated failures are omitted; these probabilities are not SLO commitments.

```
Illustrative comparison; latency traces, availability probabilities, and operating judgments are assumptions
```

Architecture	Monthly cost units	Constructed p99	Modeled availability	Operational complexity judgment
Single: 1 app, 1 data owner	371	900 ms	98.901%	Lower: one app release and one data owner
Replicated: 4 apps, 1 data owner	551	450 ms	99.899999%	Medium: rolling releases, shared cache, pool capacity and failover
Partitioned: 8 apps, 3 required data owners	1,402	600 ms	99.700300%	Higher: routing, rebalancing, multiple owners and partial recovery

For a per-key request touching only one shard, the all-three-owners availability expression does not apply. Replication with an actual quorum or failover protocol also needs a different model. The calculation is executable as architecture_comparison() in the mechanism collection. Choose a candidate against the

workload and targets, then test tail latency, degraded capacity, restoration, and operating burden. Cost arithmetic alone cannot establish those properties.

▼ Think further · When could a more expensive architecture be simpler to operate overall?

Compare total work for people as well as the invoice. A managed service may reduce patching, backup, or recovery burden while adding charges and leaving application responsibilities. Evaluate the same correctness and service goals with explicit staffing assumptions. Choose the design the team can understand, change, and recover reliably.

Remember: Filter by required behaviour before ranking cost, and keep modelled quantities distinct from measured results.

Try applying it: Why can a lower cost per attempt coexist with a higher cost per completed order?

Unit 21 exercises → · Unit 21 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. The Practical Test Pyramid — Ham Vocke ↗
   Original engineering article. Test scope and feedback trade-offs, not a mandatory test-count ratio.
   `Access reviewed · 21 October 2026`

2. Handling Overload — Google SRE ↗
   Engineering book. Admission, capacity, and overload response.
   `Access reviewed · 21 October 2026`

3. The Tail at Scale — Dean and Barroso, 2013 ↗
   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.
   `Access reviewed · 21 October 2026`

Efficiency and a useful test portfolio

Prerequisites: 21.2 · Capacity, architecture & cost per outcome 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An optimisation saves CPU while adding transfer, or a test suite adds checks without covering failures. Evaluate the whole effect and choose evidence for the particular risk.

21.3.1 · Cost-performance optimization

A smaller instance is cheaper but misses the latency target under the same traffic. It is not a comparable successful design. Cost-performance optimisation first requires feasible correctness, service, security, and maintenance behaviour.

Then vary factors such as cache size, batching, compression, placement, or capacity shape with measurements. Check downstream effects: saving write bytes may add repeated expensive decompression on reads. Record the workload and objective that make the choice useful. An optimisation can become unsuitable after the traffic mix changes, so retain the conditions and a way to reconsider it.

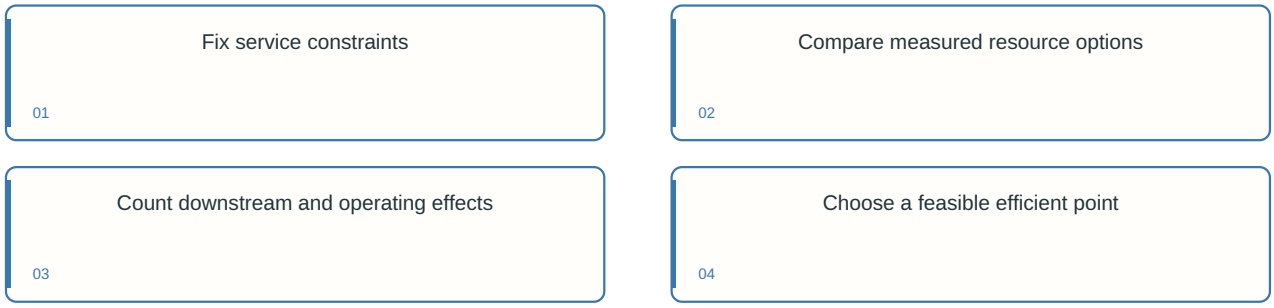


FIG. 21.3.1 **Cost-performance optimization · the example**

Cost-performance optimisation compares feasible equivalent service outcomes and checks downstream effects.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.3.2 · Sustainability and resource efficiency

Removing unused retained data saves storage work. Bounding retries avoids repeated computation and transfer. Resource efficiency begins by finding work the service does not need to perform.

Right-size capacity and batch suitable background operations without breaking deadlines. Evaluate complete effects: less CPU in one component can cause much more network or storage work elsewhere. Environmental claims need actual evidence about the relevant hardware, place, time, and lifecycle. Do not invent energy or carbon measurements. Useful engineering improvements can be explained by the resources they reduce while keeping broader sustainability conclusions within the available evidence.

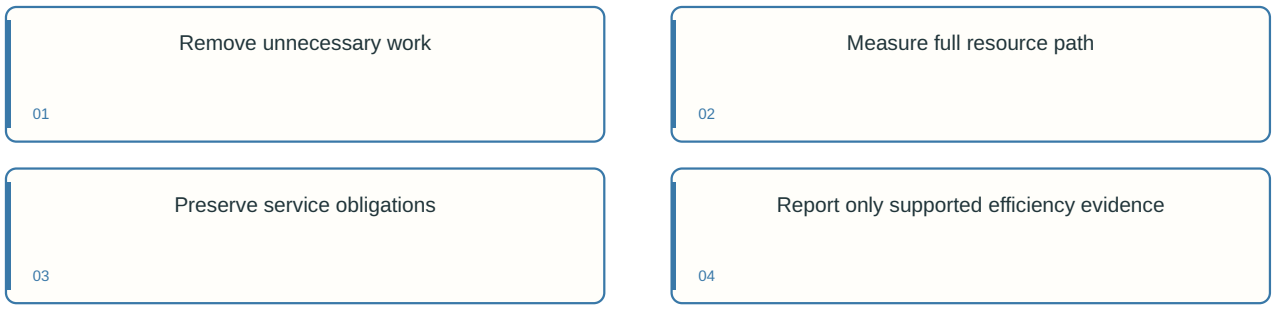


FIG. 21.3.2 **Sustainability and resource efficiency · the example**

Resource efficiency removes unnecessary work and transfer while broader sustainability claims require actual supporting evidence.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.3.3 · Unit, integration, contract, end-to-end, and performance tests

A unit test can check fee arithmetic with controlled inputs. A database integration test can check a transaction. A contract test checks boundary expectations, and an end-to-end test follows a complete journey. Performance tests examine behaviour under load.

Choose each test for the failure it can detect. A fake provider helps exercise declines, but cannot establish real serialization or commit behaviour. A suite that asserts every private call may hinder harmless refactoring while missing a lost reply. The test pyramid is guidance about feedback cost and scope, not a mandatory numerical ratio. Useful evidence matters more than raw test count.

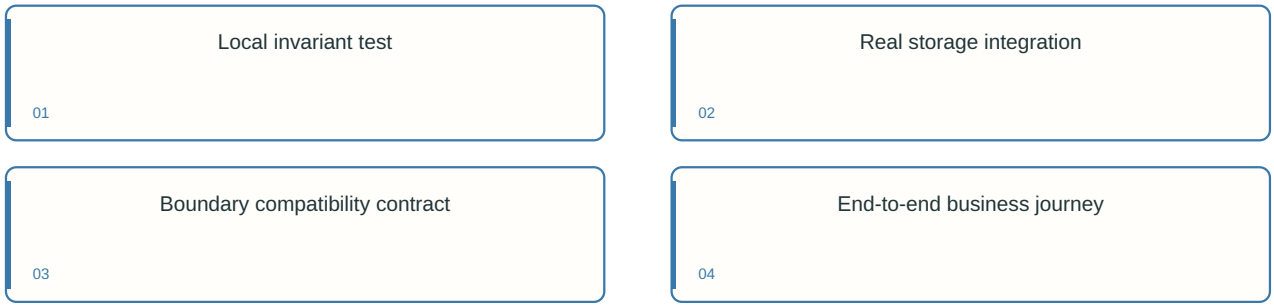


FIG. 21.3.3 **Unit, integration, contract, end-to-end, and performance tests · the example**

Test scopes provide different evidence; choose checks by failures they can detect rather than a mandatory count ratio.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.3.4 · Fault-injection tests

A process killed before it starts work says little about a crash after commit but before acknowledgment. Place faults at the boundary whose guarantee is in question.

Inject delays, unavailability, interruptions, or invalid responses with explicit acceptable outcomes and stop conditions. Test a resumed worker after lease expiry and an interrupted data move where relevant. Combine serving observations with data checks: the application can start responding again while a promised job remains lost. A fault test passes the business contract only when both the recovered service and its required state are accounted for.

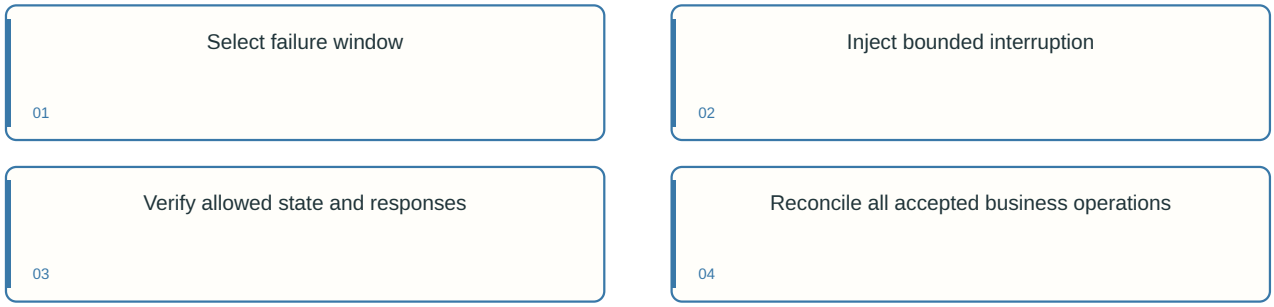


FIG. 21.3.4 **Fault-injection tests · the example**

Fault injection targets meaningful interruptions and validates both useful recovery and required state.

Author-created illustration. Numbers are illustrative unless stated otherwise.

▼ Think further · When could a more expensive architecture be simpler to operate overall?

Compare total work for people as well as the invoice. A managed service may reduce patching, backup, or recovery burden while adding charges and leaving application responsibilities. Evaluate the same correctness and service goals with explicit staffing assumptions. Choose the design the team can understand, change, and recover reliably.

Remember: Efficiency and validation should reduce real work or uncertainty without weakening the required contract.

Try applying it: Which failure window would a process kill before work begins leave untested?

Unit 21 exercises → · Unit 21 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. The Practical Test Pyramid — Ham Vocke ↗

   Original engineering article. Test scope and feedback trade-offs, not a mandatory test-count ratio.

   Access reviewed · 21 October 2026

2. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   Access reviewed · 21 October 2026

Fitness functions and design validation

Prerequisites: 21.3 · Efficiency and a useful test portfolio 4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A migration has matching row counts but wrong tenant ownership. Validate identities and invariants, preserve important regressions, and return to the requirements that justified the design.

21.4.1 · Data consistency validation

Source and destination each contain a thousand rows, but some rows belong to the wrong tenant. Equal counts missed the error. Data consistency validation must compare identities, versions, and domain rules as well as totals.

For a ledger, check balanced postings and operation uniqueness. For a search index, check the source version and deletion state. Run suitable checks after migration, replay, and restoration. Sampling can reduce cost but supports a weaker claim than examining every relevant record. State the coverage of the evidence so a useful sample is not presented as complete equivalence.

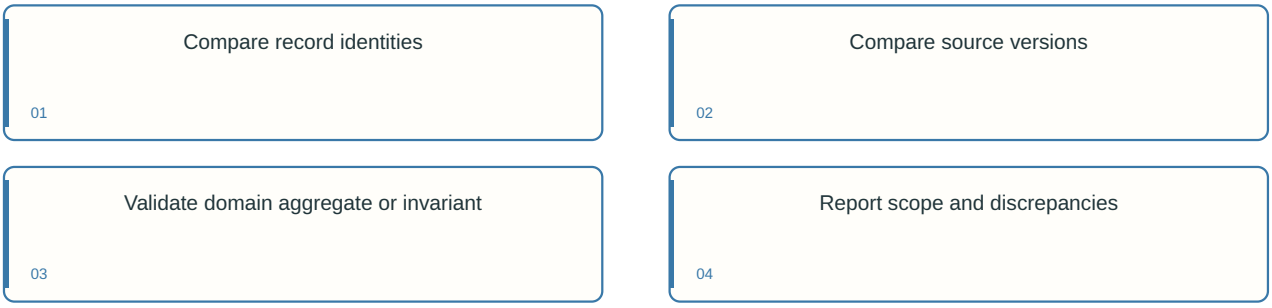


FIG. 21.4.1 **Data consistency validation · the example**

Data consistency checks compare identities, versions, relationships, and invariants beyond equal counts.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.4.2 · Architecture fitness functions

An import rule can prevent the orders module from depending on private payment storage. A runtime test can check tenant-scoped queries. These are architecture fitness functions: repeatable checks for properties the architecture needs to preserve.

Protect the reason for the boundary instead of freezing every incidental class relationship. Some qualities require drills or manual review rather than static inspection. Update a check when a legitimate decision changes the required property. Passing the set provides evidence for its covered behaviours, not a general certificate of architectural correctness. Choose checks that can reveal a meaningful erosion of the intended design.

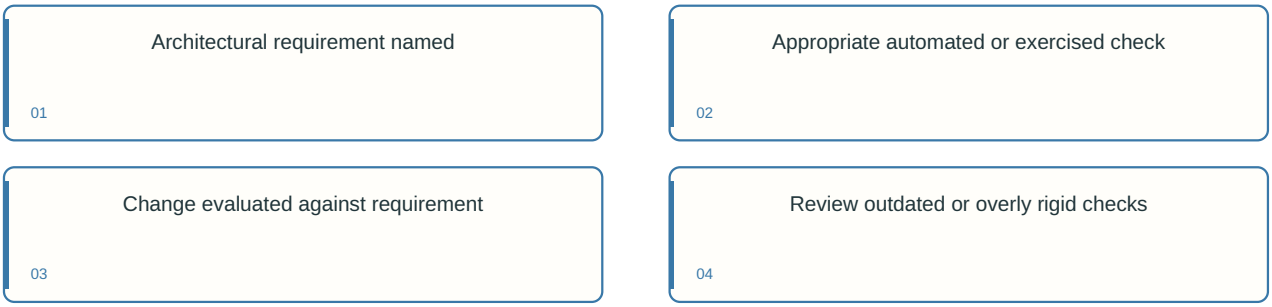


FIG. 21.4.2 **Architecture fitness functions · the example**

Architecture fitness checks protect required properties without freezing irrelevant implementation details.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.4.3 · Regression testing

A regression test keeps an important behaviour visible while the implementation changes. Include relevant errors, concurrency, compatibility, and recovery sequences, especially those that previously caused harm.

Use representative data and preserve the test's purpose through refactoring. Performance checks need sufficiently stable conditions and tolerances to avoid treating noise as a defect. If a requirement changes, revise the expected outcome explicitly. Deleting a failing test without naming the changed contract loses the reason it existed. The suite should preserve useful obligations while allowing the code that fulfils them to improve.

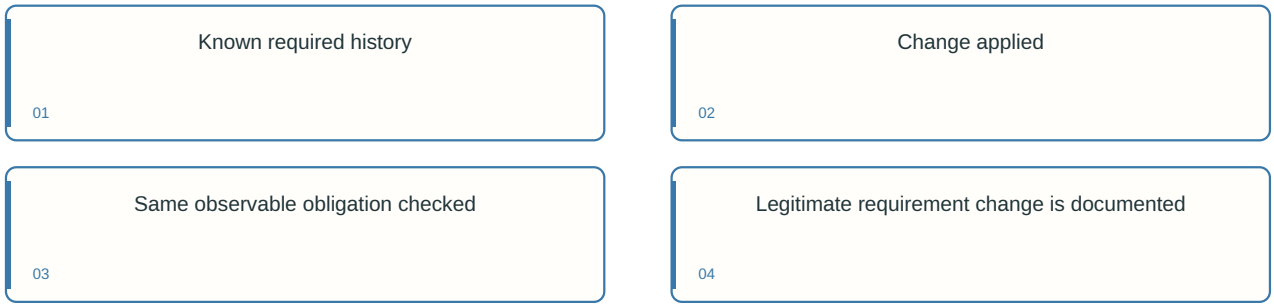


FIG. 21.4.3 **Regression testing · the example**

Regression tests preserve important behaviour while allowing deliberate contract changes and tolerating measured noise appropriately.

Author-created illustration. Numbers are illustrative unless stated otherwise.

21.4.4 · Evaluating a design against its original requirements

Return to the original brief. For each promised behaviour, identify its test, measurement, review, or still-unresolved assumption. This creates a traceable link from the design choice to its evidence.

Check function, timing distributions, capacity, recovery, access, cost, and operating burden under the stated workload. A diagram review and a load test answer different questions. If a target has not been measured, keep calling it a target. Record the remaining uncertainty rather than treating completed coverage as completed validation. The final evaluation asks whether the chosen system meets the needs that justified building it.

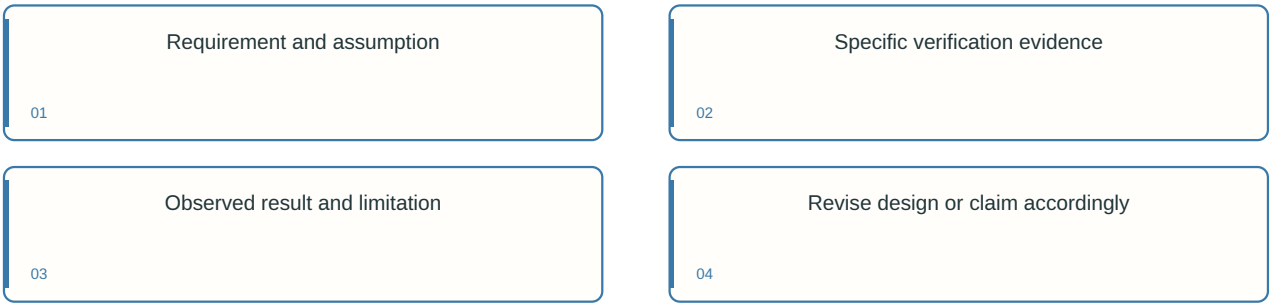


FIG. 21.4.4 **Evaluating a design against its original requirements · the example**

Design validation traces original requirements to tests, measurements, reviews, and clearly unresolved assumptions.

Author-created illustration. Numbers are illustrative unless stated otherwise.

Try the mechanism in code

PYTHON · FICTIONAL COMPARISON

```
from mechanisms import cost_options
totals = {name:sum(parts.values()) for name,parts in cost_options().items()}
assert totals == {"single":371, "replicated_app":551, "partitioned":1402}
```

The monthly totals are 371, 551, and 1,402 fictional units. They show how component costs add, rather than how the deployments actually perform.

Check equal requirements before ranking price. The additional latency and availability models have explicit synthetic inputs; real capacity, tails, and recovery require corresponding tests.

Download the executable mechanism collection ↓ · Follow the connected complete design →

THE WORKBENCH · TRY CHANGING THE INPUTS

Calculate the cost of one successful outcome

- Single deployment: 371 monthly units / 150,000 successful outcomes = **0.002473** units/outcome.
- Replicated application: 551 monthly units / 150,000 successful outcomes = **0.003673** units/outcome.
- Partitioned design: 1402 monthly units / 150,000 successful outcomes = **0.009347** units/outcome.

Compare costs only after checking that the options meet the same requirements.

These invented monthly costs are 371, 551, and 1,402 units. Change the number of successful outcomes to see the cost per outcome. Before comparing the options, test whether each can meet the required capacity, latency, availability, and correctness. These figures are not provider prices.

▼ Think further · When could a more expensive architecture be simpler to operate overall?

Compare total work for people as well as the invoice. A managed service may reduce patching, backup, or recovery burden while adding charges and leaving application responsibilities. Evaluate the same correctness and service goals with explicit staffing assumptions. Choose the design the team can understand, change, and recover reliably.

Remember: Pair each requirement with evidence and keep unresolved targets clearly labelled.

Try applying it: What additional evidence would show that an equal-count migration preserved the right data?

Unit 21 exercises → · Unit 21 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. The Practical Test Pyramid — Ham Vocke ↗

   Original engineering article. Test scope and feedback trade-offs, not a mandatory test-count ratio.

   Access reviewed · 21 October 2026

2. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

3. The Tail at Scale — Dean and Barroso, 2013 ↗

   Original research. Tail-latency considerations; atlas probabilities and latency examples are author-calculated.

   Access reviewed · 21 October 2026

PART 6 / UNIT 22

End-to-End Design, Architecture Reviews, and Evolution

We can now bring the earlier ideas together. Each design starts with a concrete workload, follows its main operation, and examines what happens when part of the system fails.

The running example

Twelve complete architectural designs with a common 22-point review protocol.

Unit 22 assessment → Explained solutions →

The complete review protocol

1. Functional requirements
2. Nonfunctional requirements
3. Explicit assumptions and workload numbers
4. Naive initial architecture
5. Core API contracts
6. Data model and schema
7. Low-level component organization
8. High-level system architecture
9. Data and control flows
10. Capacity estimates
11. End-to-end latency estimates
12. Failure scenarios
13. Scalability improvements
14. Consistency requirements
15. Security model
16. Deployment approach
17. Observability design
18. Infrastructure cost considerations
19. Alternative architectures and trade-offs
20. Architecture evolution from small scale to large scale
21. Design review identifying weaknesses and improvements
22. A short implementation exercise

Each of the twelve designs above contains every section, including the implementation challenge. Optional next subjects: Search engine. · Cloud storage service. · Distributed cache. · API gateway. · Metrics collection platform. · Multi-tenant SaaS. · Real-time fraud detection. · IoT ingestion platform..

URL Shortener

Prerequisites: API Design and Communication Contracts, Data Models and Storage Technologies, Caching and Content Delivery, Scalability, Load Balancing, and Data Distribution

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A visitor opens a short link and expects to reach the right page. Behind this small action, the service must find the original URL and decide whether its owner still permits the redirect.

We will follow one link from creation to use and revocation. This will help us decide which records need an authoritative owner, which reads may use a cache, and which work can wait until after the visitor receives a reply.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.1.1 · Functional requirements

A user submits a long URL and receives a short code. Later, a visitor opens that code and the service redirects the visitor to the stored target. The owner can manage the link through an authenticated interface, set its expiry, and revoke it.

Keep the original target and the identity of the creation request. Record clicks for analytics, but let this secondary work proceed independently: an analytics failure should not make a valid redirect wait indefinitely. Checking the target and handling abuse still belong to the main service.

22.1.2 · Nonfunctional requirements

For this example, aim for redirect p99 below 100 ms at the supported peak. This means that at least 99% of the measured redirects should finish within that time under the stated test conditions. A mapping reported as saved must survive the selected single-storage-node failure, and management requests must preserve ownership.

We also need to say how soon revocation takes effect. If a cache keeps an old target, a fast reply may still be wrong. Choose the cache freshness rule together with the latency target and storage acknowledgment policy, then test the three together.

22.1.3 · Explicit assumptions and workload numbers

Assume 100 link creations/s and an average of 10,000 redirects/s. Let the redirect peak be ten times the average, or 100,000/s. Assume each mapping occupies 200 bytes before indexes and extra copies.

There are 86,400 seconds in a day, so raw daily growth is 100 × 86,400 × 200 = 1,728,000,000 bytes, or 1.728 GB. For later transfer estimates, assume a 500-byte redirect response. These are teaching inputs. Real record sizes, traffic durations, and transport overhead must be measured before choosing capacity.

22.1.4 · Naive initial architecture

Begin with one web application and one relational mapping table. Give the code column a uniqueness constraint, so the database can reject a second mapping with the same code. A background job handles analytics.

This arrangement makes creation and lookup easy to follow. It also exposes the first limits: the application host can fail, and all cache misses eventually reach one storage path. Measure that path before dividing the data into shards. Sharding introduces routing and uniqueness decisions, and may leave the actual bottleneck untouched.

22.1.5 · Core API contracts

POST /links accepts a target, expiry, and idempotency key. GET /{code} returns the permitted redirect or the documented expired or revoked result. DELETE /links/{id} checks the owner before changing state.

An idempotency key identifies one intended creation. Retrying the same content with that key recovers its original result; changing the content under the same key conflicts. Validate supported URL schemes and

syntax without fetching arbitrary destinations. State whether redirects are permanent or temporary, because clients may retain a permanent response after the stored link changes.

22.1.6 · Data model and schema

A link record contains id, unique code, target, owner_id, creation time, expiry, state, and version. An operation record binds the owner and operation key to a request fingerprint and the resulting link. Make that key unique within its owner scope.

The code has a wider obligation: it must be unique throughout its intended namespace. If data is later sharded, allocation or routing must preserve this rule across owners. Separate uniqueness constraints on unrelated shards cannot do so by themselves. Expiry and revocation remain part of authoritative, versioned link state.

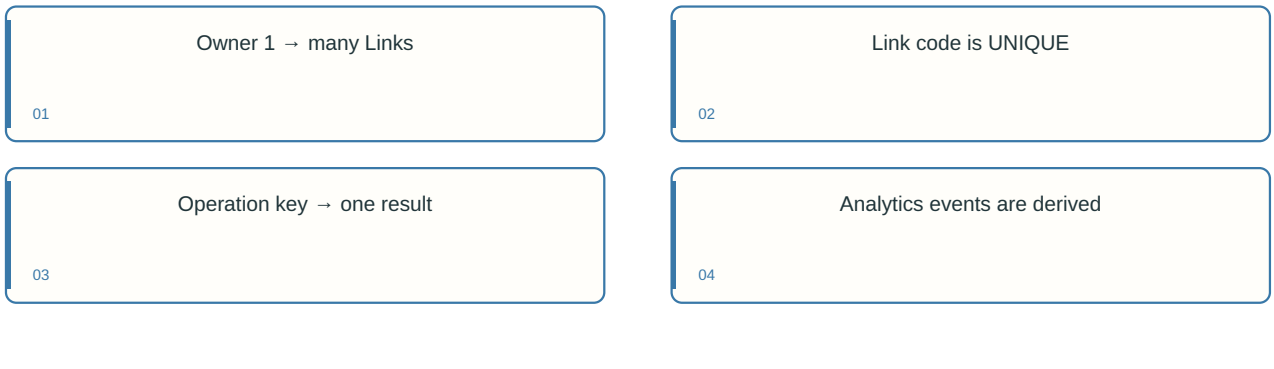


FIG. 22.1.1 **Data ownership and relationships**

A link has one owner and a unique code. The stored operation identifies its creation result.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.1.7 · Low-level component organization

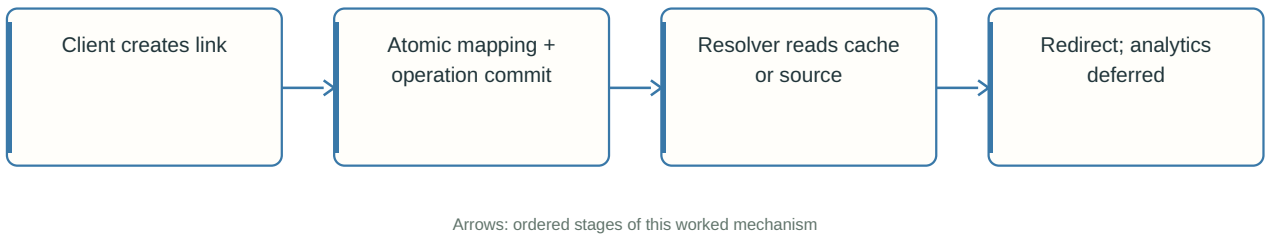
LinkService checks the caller and target. CodeAllocator proposes a short code. LinkRepository saves the mapping and operation result in one transaction. RedirectResolver checks whether a link is usable, while AnalyticsPublisher handles click events.

The allocator may encode a suitably unique integer in base62, using 62 symbols to represent the number compactly. It may instead propose random codes and retry collisions. Either way, the repository's uniqueness decision protects the mapping. A random proposal alone proves nothing about uniqueness. Translate storage failures into the service's contract rather than exposing provider details to every caller.

22.1.8 · High-level system architecture

Place an edge or reverse proxy before replaceable application instances. A cache can answer reads when the expiry and revocation policy permits reuse. A replicated authoritative store retains mappings; a separate queue or log carries analytics work.

The cache is a copy that makes some reads cheaper. It must not be the only recoverable record of a link. If it fails, many more requests can reach storage, so bound fallback traffic. Additional application instances preserve some serving capacity; the storage placement and acknowledgment rules determine which saved mappings survive a storage failure.



Arrows: ordered stages of this worked mechanism

FIG. 22.1.2 **Follow the main operation**

Creation and redirect use authoritative mappings; analytics is a separate obligation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.1.9 · Data and control flows

On creation, validate the request, commit the mapping and deduplication result together, and only then return success. On redirect, determine whether the link is usable, obtain a cache or source value permitted by the freshness policy, and send the redirect. Record analytics independently of that required result.

When many callers miss the same code, combine their source lookup where useful. Keep private management data out of the public redirect cache. These choices let the write, redirect, and analytics paths have different workloads while keeping each responsibility clear.

22.1.10 · Capacity estimates

With a 90% cache hit ratio, 10% of redirects reach the source. At 10,000 redirects/s, that is 1,000 source reads/s; at the 100,000/s peak, it is 10,000 reads/s, before retries or combined misses. Peak response traffic is 100,000 × 500 = 50,000,000 bytes/s, or 50 MB/s, before transport overhead.

A cold or unavailable cache removes the assumed protection. Test that workload separately and define how origin admission limits or rejects excess work. Success with a warm cache does not establish cold-cache capacity.

22.1.11 · End-to-end latency estimates

Suppose a warm redirect spends 20 ms at the edge and network, 10 ms in the application, 5 ms accessing the cache, and 20 ms returning the result. The proposed path totals 55 ms.

This is a budget to test. A miss adds source lookup and possibly queueing, and individual average stage times do not establish end-to-end p99. Measure the complete distribution under hot links, cache loss, storage failover, and the revocation checks required by the contract before claiming the 100 ms objective.

22.1.12 · Failure scenarios

Suppose two creations propose the same code. The first insert commits. The second must fail its uniqueness check without changing the first target, then try another code. If a creation commits but its reply disappears, a retry must recover the saved operation result instead of creating a new link.

Other failures cross different boundaries. Cache loss can overload the source unless admission is bounded. An old cache entry can serve a revoked link unless freshness rules stop it. Analytics failure should delay analytics progress. Test each history, because a normal create-and-open demonstration does not exercise these repairs.

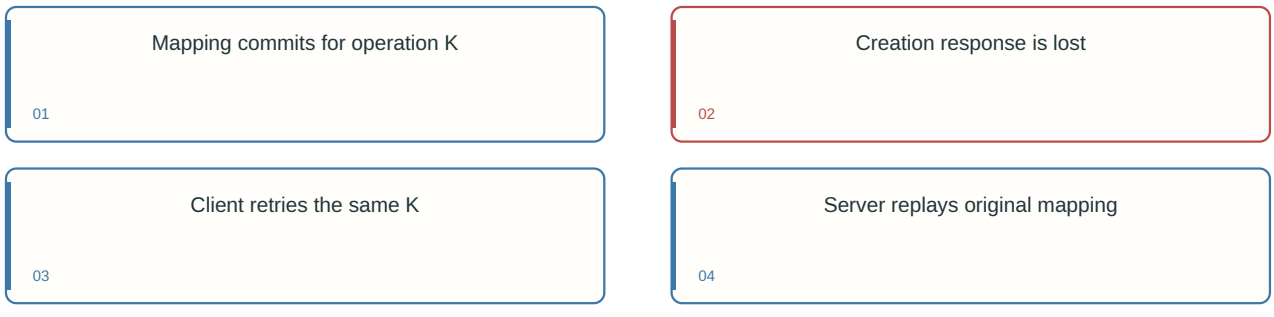


FIG. 22.1.3 **Follow the failure and its correction**

Test a collision, a lost reply, and a stale cached target at their different boundaries.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.1.13 · Scalability improvements

Add application instances when application work is the measured limit. Add shared or edge caching when repeated permitted reads justify it. Partition mappings when the authoritative storage path needs to be divided.

A very popular code may still overload its one partition. More permitted read copies can help that case, whereas more partitions containing unrelated codes cannot. Every new copy inherits the expiry and revocation obligation. When data moves, routing, versions, and recovery must preserve global code ownership and the identities of existing operations.

22.1.14 · Consistency requirements

Creating a code requires one atomic uniqueness decision: two different links cannot both win the same name. Choose an owner-facing read-after-write rule so an owner knows when the newly created mapping will appear. Public redirects may observe delayed changes only within the stated expiry and revocation contract.

Click events can be aggregated later using stable event identities. Their temporary delay is different from assigning a code twice. Describe the observable rule for each operation; a single label such as “eventually consistent” would hide these differences.

22.1.15 · Security model

Check ownership on management requests and limit abusive creation. Accept only supported target schemes. If a preview feature fetches destinations, restrict its outgoing connections and validate redirect hops; otherwise, a public URL field could be used to reach private services.

Protect credentials and avoid exposing private targets in logs and analytics. A difficult-to-guess short code still grants no complete permission policy for confidential content. If the product supports restricted destinations, specify authentication and access checks for those redirects as well.

22.1.16 · Deployment approach

Start in one region with a stated failure contract. Use compatible rolling application releases, a suitably replicated authoritative store, and backups whose restoration has been tested. Drain connections where needed and retain operation identities when instances are replaced.

New code must understand existing mapping versions. If a second region becomes necessary for locality or recovery, decide where writes are authoritative and which acknowledged changes could be lost during promotion. Geographic separation is useful only when these rules support the required recovery outcome.

22.1.17 · Observability design

Measure redirect outcomes and latency separately for hits and misses. Track origin admission, hot codes, code conflicts, storage progress, and the delay before revocation reaches permitted readers. Observe analytics lag on its own path.

A high hit ratio says little about whether the cache returns a newly revoked target. Alert on useful request failures and rapid error-budget consumption. Use traces or sampled request histories to find the responsible path, while keeping private targets out of routine telemetry.

22.1.18 · Infrastructure cost considerations

Estimate compute from instance-hours × unit rate, storage from retained bytes × copy factor × rate, and transfer from response bytes × transfer rate. State the duration used in each calculation.

For example, 30 days contain 2,592,000 seconds. Sustaining 100,000 redirects/s at 500 bytes for that whole period produces 129.6 TB before overhead. A brief peak at the same rate produces much less traffic. Use observed durations and current prices for spending decisions; the example only identifies the quantities a cost model needs.

22.1.19 · Alternative architectures and trade-offs

Sequentially allocated identifiers simplify code generation, provided the allocator preserves uniqueness, but they can reveal creation order. Random codes reduce that predictability and require collision checks. Neither choice supplies access control.

Permanent redirects allow clients to reuse responses more aggressively, which can make later target changes or revocation harder to enforce. Temporary redirects retain more server control at a traffic cost. Compare these choices using the required mutation, revocation, privacy, and load behaviour rather than choosing a code-generation technique in isolation.

22.1.20 · Architecture evolution from small scale to large scale

Use traffic levels as checkpoints for asking questions. Around 100 requests/s, one application and transactional store may be a useful starting point. Around 1,000, inspect query plans and indexes. Around 10,000, examine measured application and cache needs. At 100,000, expand the actual bottleneck and verify degraded and cold-cache behaviour.

These numbers are not hardware promises; a particular deployment can reach its limit earlier or later. Throughout the changes, keep code and operation identities stable so that existing links and retries retain their meaning.

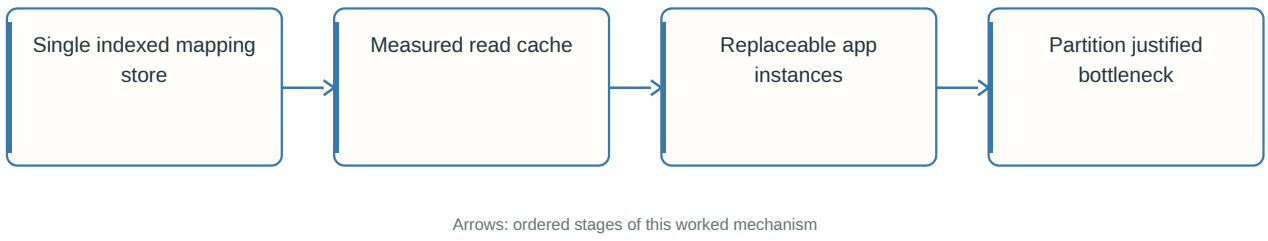


FIG. 22.1.4 **How the design can grow**

Add capacity for a measured limit while preserving code uniqueness and revocation.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.1.21 · Design review identifying weaknesses and improvements

Three uncertainties deserve particular attention: how quickly revocation takes effect, how the source behaves after cache loss, and how code uniqueness survives partitioning. Revoke a link under load, disable or empty the cache, and force a collision across the real allocation boundary.

For each check, record the permitted degraded outcome and the evidence still missing. Extra cache replicas may reduce some outages, but they do not answer all three questions. A useful review ends with a specific supported failure behaviour and a test capable of exposing a mistaken assumption.

22.1.22 · A short implementation exercise

Implement base62 encoding and a repository with an atomic unique-code constraint. Force two proposals to use the same code. Check that the first target remains unchanged and that the second creation retries safely.

Next, stop execution after commit but before the creation reply. Retry with the same owner and operation key. The existing mapping should return, with no second logical creation. Change the target under that key and verify the documented conflict. This distinguishes code collisions from repeated requests: each needs its own rule.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
def allocate(target, operation_key):
    prior = operations.find(operation_key)
    if prior: return prior
    while attempts_remaining():
        code = propose_code()
        try:
            with transaction():
                link = insert_unique_code(code, target)
                save_operation_result(operation_key, link.id)
            return link
        except CodeConflict:
            continue
        except OperationConflict:
            return load_and_validate_prior_result(operation_key, target)
    raise AllocationUnavailable
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

2. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   Access reviewed · 21 October 2026

3. Application best practices — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Scaling and application boundary review.

   Access reviewed · 21 October 2026

Rate Limiter

Prerequisites: Requirements, Estimation, and Engineering Trade-offs, Runtime, Processes, Threads, and Concurrency, Scalability, Load Balancing, and Data Distribution, Reliability, Fault Tolerance, and Disaster Recovery

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Suppose three application instances each start with ten tokens for the same customer. Each instance follows its local arithmetic correctly, yet together they may admit thirty requests at once.

Before building a rate limiter, we must decide what the allowance covers. We will begin with one bucket, trace its refill and spending, and then examine what changes when several processes or regions share the policy.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.2.1 · Functional requirements

A request asks to perform work with a known cost. The limiter checks whether the caller's policy permits that work now, then allows it or supplies useful waiting or rejection guidance.

Separate three requirements. A continuing rate limits sustained work; a burst allowance permits some work to arrive together; a cumulative quota limits a total over a longer period. Also state the enforcement scope. An exact global allowance and a bounded regional approximation are different policies. Giving every region a full copy of the global bucket multiplies the permitted work.

22.2.2 · Nonfunctional requirements

Admission should add little delay while protecting downstream resources and preserving tenant fairness. Specify what happens when the limiter cannot reach its authoritative state.

During a partition, an exact global cap may require waiting or rejection because a region cannot safely know what others have spent. Continuing to allow all requests, often called failing open, cannot preserve that cap unless previously allocated safe budgets cover the work. Choose latency, availability, and precision together, with the protocol and assumptions needed to support them.

22.2.3 · Explicit assumptions and workload numbers

Assume 100,000 checks/s and one million active keys. Our sample bucket holds at most ten tokens and refills at two tokens/s. A token represents an amount of admitted work.

At an assumed 64 bytes per key, raw state is 1,000,000 × 64 = 64 MB. Engine allocations, expiry metadata, replicas, and network work add to this. These are teaching assumptions rather than benchmarks. A small state record can still be expensive if many requests contend to update it.

22.2.4 · Naive initial architecture

Begin with a dictionary in one process. A suitable lock protects the entire refill-and-spend transition for one bucket, so two callers cannot spend its last token together.

This protects only callers sharing that owner. An unrelated second process starts another allowance, and restarting may reset the first. The local design is suitable when that scope and reset behaviour meet the requirement. To enforce a shared cap across instances, we must add a shared authority or an explicit allocation protocol.

22.2.5 · Core API contracts

check(key,cost,policy_version) returns the decision, remaining allowance, and a meaningful retry delay. Derive the key from trusted caller and resource identity so callers cannot escape a limit by inventing new keys. Require a positive, bounded cost.

Authorise policy changes and define when a new version takes effect. Distinguish overload from invalid input, and specify whether rejected attempts consume a separate abuse allowance. These meanings let callers decide whether to wait, correct the request, or stop retrying.

22.2.6 · Data model and schema

A bucket record contains scope_key, policy_version, tokens, last_refill, and expiry. Refill and spending belong to one atomic update. A daily cumulative quota may need separate state because it has a different lifecycle.

Use integer or fixed-point units where useful to avoid numeric drift. Define how policy changes treat existing tokens and capacity. Trust the server's time basis rather than client timestamps. Expiring a record may recreate a full bucket, so expiry must be chosen as part of the protection contract.

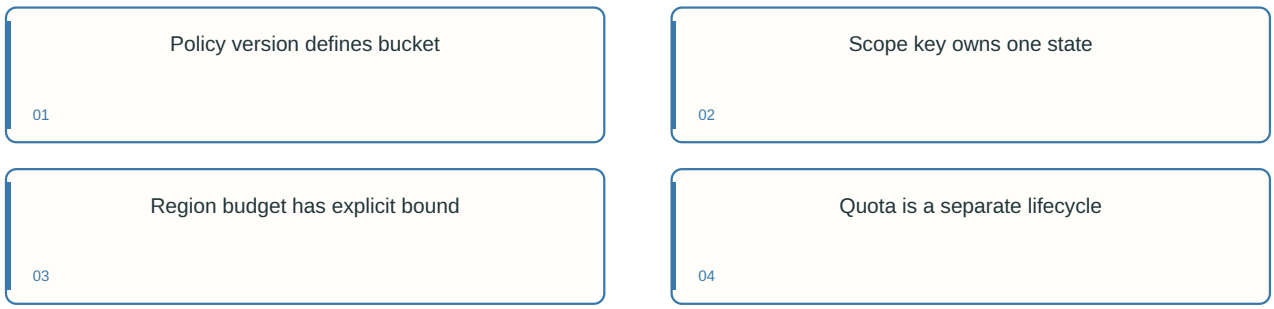


FIG. 22.2.1 **Data ownership and relationships**

A bucket groups scope, policy, tokens, and refill time in one coordinated record.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.2.7 · Low-level component organization

KeyResolver establishes trusted scope, PolicyResolver loads a validated policy, and Clock provides the time basis. BucketStore owns the coordinated state transition. AdmissionService turns its result into the caller-facing decision.

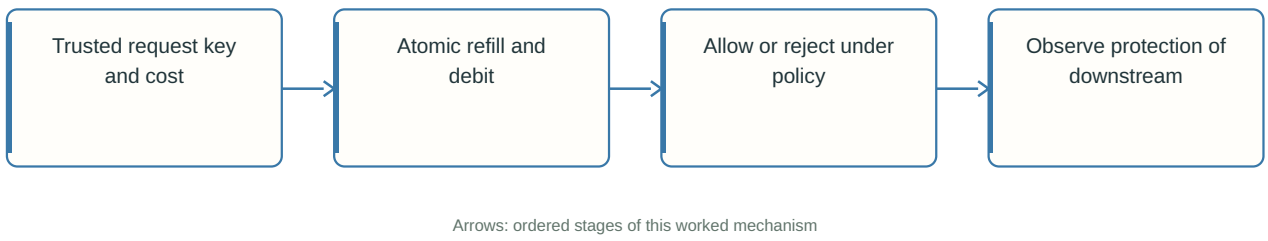
These boundaries make deterministic tests easier: a test can supply time and policy directly. They do not create concurrency safety by themselves. Two correct calculations can both start from one old token count. Keep load, refill, comparison, and debit under one owner or supported transaction so only one decision can spend the final token.

22.2.8 · High-level system architecture

Use coarse admission near ingress and more specific checks before expensive resources. Independent keys can have different storage owners. A strict shared bucket still needs one atomic authority or a

reviewed distributed protocol.

Regions can receive bounded portions of an allowance in advance and spend locally. This reduces remote checks but introduces allocation and replenishment rules. State the outstanding-budget or overshoot bound. Protect the limiter's own capacity too: otherwise, the service intended to absorb excessive traffic may become the first overloaded dependency.



Arrows: ordered stages of this worked mechanism

FIG. 22.2.2 **Follow the main operation**

Derive trusted scope before making the refill-and-spend decision.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.2.9 · Data and control flows

First derive trusted scope and the current policy. Under coordinated access, load the bucket and calculate elapsed time, never allowing a negative interval to add tokens. Refill by elapsed time × refill rate, capped at capacity. Compare the available tokens with cost, then debit only an allowed request.

If a meaningful wait can be computed, divide the token deficit by refill rate. Record bounded metrics and return the decision. The code keeps the whole transition atomic; unprotected reads followed by writes would let two requests spend the same token.

22.2.10 · Capacity estimates

One million 64-byte records need 64 MB of raw state. If every incoming request makes a remote check, the assumed workload also creates 100,000 limiter requests/s before retries and replica work.

Measure update rate, active state, contention, and replication cost separately. Evenly distributed records do not help a single key receiving most requests: its transitions still need coordination. Batching and local budgets can reduce remote traffic, but may change decision timing or precision. Include those changes when comparing capacity.

22.2.11 · End-to-end latency estimates

Suppose the application has a 100 ms response target and assigns 5 ms to admission. That allocation is a proposed budget to validate under contention and failover.

A strict cross-region check adds network propagation and queueing and may exceed it. Preallocated local budgets can shorten the request path, while adding replenishment work. Measure the complete path, including policy lookup and store replacement, against the required protection rule. A faster design that grants a different allowance needs to be described as a changed contract.

22.2.12 · Failure scenarios

Imagine two requests reading one remaining token. Each sees enough allowance, and both approve the work. Combining refill, comparison, and debit atomically repairs this race because the second request then observes the first debit.

Also test backward time, record expiry, forged keys, and conflicting policy versions. These can create extra allowance even when the refill formula is correct. During owner loss, apply the documented rejection, safe local budget, or bounded fail-open rule. Protection depends on identity, time, and deployment scope as well as arithmetic.

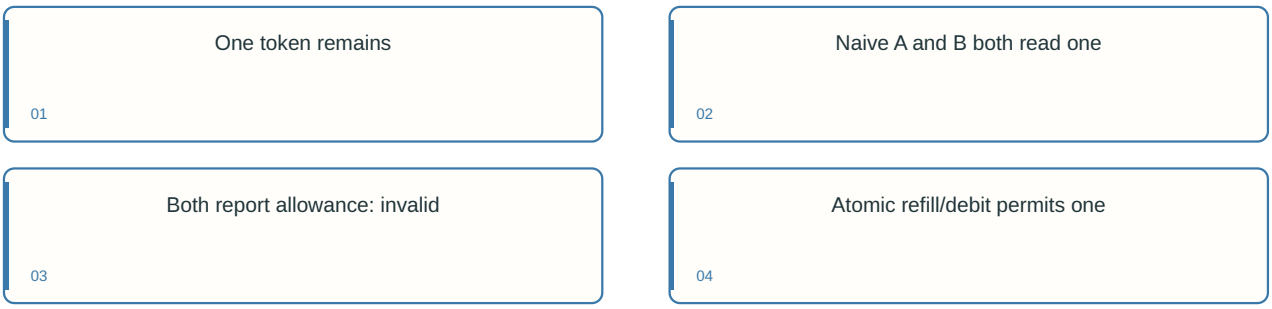


FIG. 22.2.3 **Follow the failure and its correction**

Two callers must not independently spend the same last token.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.2.13 · Scalability improvements

Partition independent keys while keeping each bucket's decisions coordinated. Versioned policy definitions can be cached where suitable, but token spending still needs the required authority.

A strict hot bucket cannot be divided into unrelated full buckets without changing its cap. Hierarchical limits or allocated local allowances can distribute some work when their semantics fit the resource. Give expensive operations appropriate costs. Expand the part that limits protected work or coordinated checks; adding space for more cold keys will not relieve a hot decision.

22.2.14 · Consistency requirements

A strict token bucket preserves its refill-and-debit rule under the chosen time model. A distributed approximation needs an explicit bound, such as the total allowance independently outstanding in regions.

The algorithm also defines the policy: token buckets permit bursts, fixed windows have boundary effects, and sliding windows describe recent traffic differently. Usage counters that agree later cannot undo excessive work already admitted. State the invariant required when a decision is made, then choose a mechanism that preserves it.

22.2.15 · Security model

Bind scope keys to authenticated tenant or service identity. Authorise policy administration and limit creation of new keys, since unlimited keys could exhaust state memory. Validate tenant headers, policy names, and time inputs.

Limit what remaining-allowance responses reveal about other callers. Admission and permission answer different questions: having enough tokens does not permit reading another tenant's document. The protected operation must still enforce its resource access rule.

22.2.16 · Deployment approach

Deploy policy and bucket owners with stated state-loss and failover behaviour. If replacement grants every caller a new full bucket, the system may admit another burst that the contract did not allow.

Roll out policy versions compatibly and test older callers, clock behaviour, and owner failure. A fallback must have an acceptable bound. Preserve state or allocate safe replacement allowance according to the design, even though each individual bucket record is small.

22.2.17 · Observability design

Measure allowed and rejected cost, check latency, owner errors, active keys, hot-key pressure, and policy-version disagreement. Compare these signals with the load and health of the protected resource.

Many rejections can mean successful protection or a mistaken policy denying legitimate work. Few rejections can hide fail-open behaviour during state-store failure. Observe both unsafe admission and unnecessary denial. Keep metric dimensions bounded so one label per arbitrary key does not overwhelm monitoring.

22.2.18 · Infrastructure cost considerations

Cost includes state memory, check requests, replicated updates, transfer, and operating work. A remote strict check on every inexpensive business operation can cost more than the work it protects.

Compare strict and local-budget designs with their guarantees stated, using symbolic or explicitly fictional rates for teaching. If a cheaper design allows more overshoot or behaves differently during failure, include that consequence. A smaller infrastructure bill alone does not establish equal protection.

22.2.19 · Alternative architectures and trade-offs

A token bucket admits bounded bursts. A leaky-bucket shaper smooths output by making work wait. Fixed-window counters are simple, but traffic can cluster on both sides of a window boundary. Sliding approaches track recent traffic at additional state or computation cost.

Regional budgets add allocation work in exchange for shorter local decisions and some independence during failure. Choose using the protected resource's needs. Include the rejection response and expected retry behaviour, because callers can otherwise recreate the same burst immediately.

22.2.20 · Architecture evolution from small scale to large scale

First build a deterministic local bucket with one atomic owner. Introduce shared state when multiple processes need one cap. Partition keys when measured load requires it.

Add regional allocation for a specific latency or failure requirement and state its bound. At every step, preserve the original policy meaning or explicitly version the changed contract. Increasing the number of enforcement instances should not silently increase the customer's allowance.

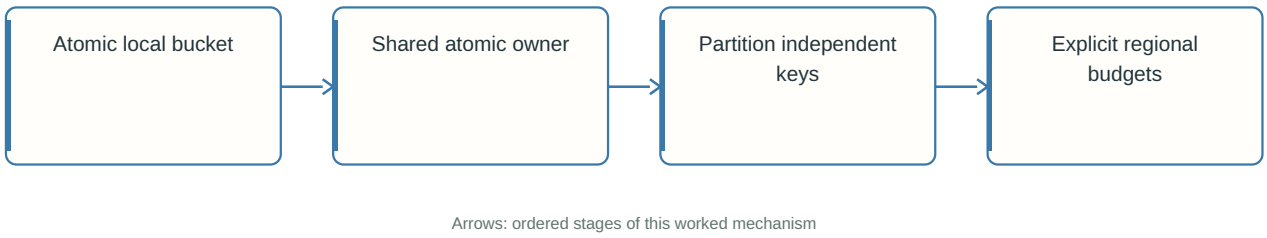


FIG. 22.2.4 **How the design can grow**

Move from local to shared admission only with an explicit scope and allocation rule.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.2.21 · Design review identifying weaknesses and improvements

Review whether the advertised scope matches actual enforcement. Ask what a policy update does to existing tokens and how the hottest key behaves. These questions cannot be answered by the refill formula alone.

Test simultaneous spending of the last token, backward time, expiry, policy replacement, owner loss, and regional reconnection. Full independent buckets do not support an exact-global claim. Record unresolved evidence about the real protocol rather than assuming that a shared storage product automatically establishes every required behaviour.

22.2.22 · A short implementation exercise

Build the sample bucket with an injected monotonic clock. Spend its ten initial tokens, reject an immediate eleventh one-token spend, then advance three seconds. At two tokens/s, six tokens should return, within capacity ten.

Reject invalid costs and check that backward time adds no allowance. Reproduce the race in which two unprotected readers spend one token, then coordinate the transition and confirm that only one wins. Keep both histories so the test explains why the repair works.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
def spend(bucket, now, cost):
    require(cost > 0)
    elapsed = max(0, now - bucket.last)
    available = min(bucket.capacity,
                    bucket.tokens + elapsed * bucket.rate)
    allowed = available >= cost
    bucket.tokens = available - cost if allowed else available
    bucket.last = max(bucket.last, now)
    return allowed
# Entire transition executes atomically for this bucket.
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

THE WORKBENCH · TRY CHANGING THE INPUTS

Use a burst of tokens, then let the bucket refill

Virtual time: 0s · tokens: **10** of 10 · admitted: 0 · rejected: 0.

Each second adds two tokens, up to ten. Advancing three seconds adds at most six tokens.

The bucket holds at most ten tokens and gains two per second. Spend tokens, then advance the simulated clock. One browser owns this bucket. A shared implementation must update the balance atomically; several independent owners need a protocol to enforce a global limit.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 2697: A Single Rate Three Color Marker, 1999 ↗

   Original protocol description · Informational RFC. Token-based traffic metering example; atlas admission bucket is a simpler model.

   Access reviewed · 21 October 2026

2. Handling Overload — Google SRE ↗

   Engineering book. Admission, capacity, and overload response.

   Access reviewed · 21 October 2026

Chat Application

Prerequisites: Networking Foundations for System Design, Data Models and Storage Technologies, Distributed Systems Fundamentals, Message Queues, Events, and Distributed Workflows

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A chat message appears on the sender's screen. The recipient's phone has just disconnected. What should “sent” mean while that phone is unreachable?

We will store the conversation before relying on live delivery, then let a returning device recover what it missed. Following those two paths will help us distinguish saved history, device delivery, human reading, and approximate presence.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.3.1 · Functional requirements

Two people exchange messages in an authorised conversation. Online devices receive updates; disconnected devices later fetch missing history. Support bounded groups and a defined retention policy as well.

A message moves through several stages: the service accepts it, storage persists it, a device receives it, and a person may read it. Give each receipt the meaning its evidence supports. Presence is an approximate indication of connectivity. A WebSocket supplies a live communication path, but opening it proves neither durable history nor human reading.

22.3.2 · Nonfunctional requirements

Retain acknowledged messages under the selected storage-failure contract and protect conversation privacy. Define ordering within each conversation, or state a deliberately weaker rule. For the supported workload, target online delivery below one second.

An offline device has a different delivery timeline and catches up later. Approximate presence expiry need not satisfy the strict rules used for exclusive allocation. Specify retention and visibility for moderation, deletion, and returning devices so recovery does not reveal history they may no longer access.

22.3.3 · Explicit assumptions and workload numbers

Assume 100,000 concurrent connections and 10,000 accepted messages/s. Each stored message averages 500 bytes and produces five recipient deliveries in this simplified workload.

Ingestion is 10,000 × 500 = 5 MB/s. Over 86,400 seconds, that is 432 GB/day. Five deliveries multiply body traffic to about 25 MB/s before framing and overhead. Model multiple devices, large groups, connection memory, and reconnect bursts separately. These assumptions describe work to test; they do not establish gateway or database capacity.

22.3.4 · Naive initial architecture

Begin with one connection server and a transactional history store. Commit the message before returning a persistence receipt, then try to deliver it to online devices.

If the gateway crashes, clients lose their connections, but committed history can survive under the storage contract. They reconnect and fetch missing messages. This separates the server's availability limit from message persistence. A database commit says the message was retained; only later evidence can establish delivery to a particular device.

22.3.5 · Core API contracts

A send includes conversation_id, client_message_id, and bounded content. Fetch history after a sequence with an operation such as GET /conversations/{id}/messages?after=sequence. Check membership on both reads and writes.

Delivered and read receipts identify the relevant device or user and the highest sequence covered by the product's policy. A repeated client_message_id in the sender's uniqueness scope returns the original message. A client can therefore recover from a lost send reply without adding another logical message.

22.3.6 · Data model and schema

Store conversations and membership separately from messages. Each message contains conversation_id, sequence, sender, client_message_id, body, and version. Receipt records hold delivered and read progress for the appropriate user or device; heartbeat records provide expiring presence.

Enforce the intended client-identity uniqueness scope. A sequence authority, or a supported conflict model, must establish conversation order. Client-clock timestamps alone cannot supply a stable total order. Keep the identity used for retries valid across reconnects.

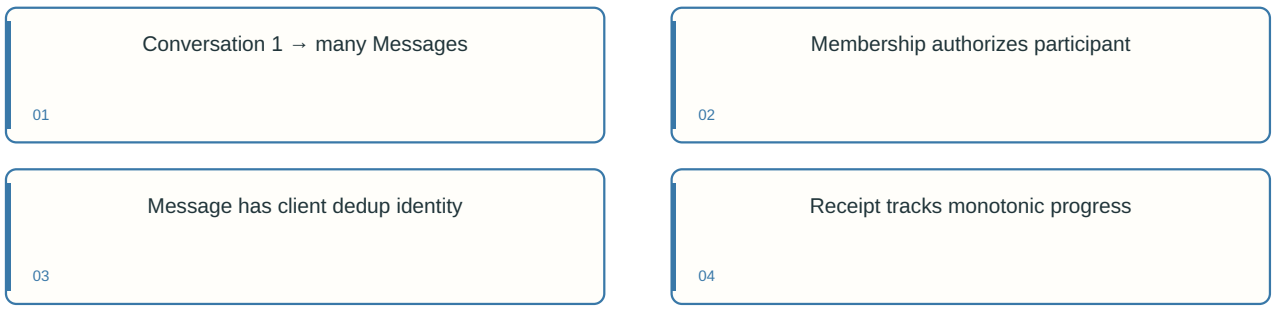


FIG. 22.3.1 **Data ownership and relationships**

History, membership, and receipt progress have different owners and lifetimes.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.3.7 · Low-level component organization

ConnectionGateway owns live connections. MembershipPolicy checks access. MessageService coordinates identity and persistence, SequenceOwner assigns conversation order, and HistoryRepository reads bounded pages. DeliveryWorker distributes committed messages.

ReceiptAggregator combines progress without moving backward under the chosen device policy. Large groups need bounded fan-out work rather than one long loop inside the message transaction. These components can initially run together, but their contracts should distinguish durable history from routing information that may disappear during a gateway restart.

22.3.8 · High-level system architecture

Use replaceable connection gateways and route durable message work by conversation. Commit delivery intent through a log or outbox, then let workers find currently connected recipient devices. An outbox is a

durable record of work to publish, saved with the message so a crash cannot lose the obligation between the two writes.

Connection routes may be stale. A missed frame is therefore repaired from history. If the broker is unavailable, committed intent can wait while acknowledged history remains retained. The gateway is a route to the device, rather than the only record of its messages.

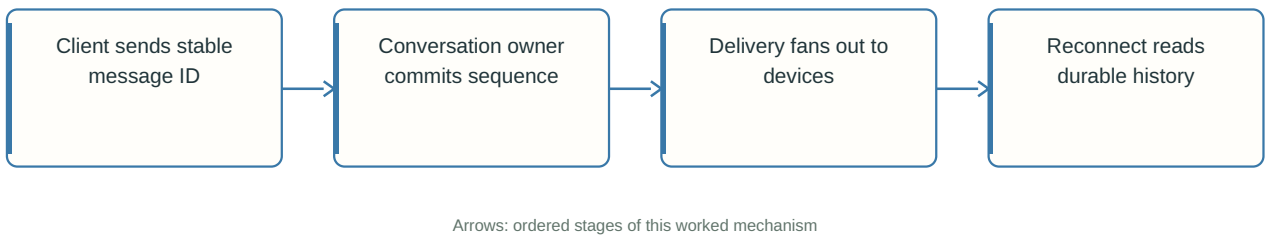


FIG. 22.3.2 **Follow the main operation**

Store the message before reporting persistence, then deliver and recover by identity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.3.9 · Data and control flows

The client sends a stable message identity. The service checks membership, assigns a sequence, and commits both message and delivery intent before returning the persistence result. The code illustrates this durable boundary; device delivery and read receipts follow later.

Workers send frames and apply valid receipts. On reconnect, a device supplies its last observed sequence, fetches bounded missing pages, and detects gaps. It recognises repeated frames by identity. This lets the retained conversation and the device's current display recover independently after an interrupted live connection.

22.3.10 · Capacity estimates

The assumptions give 5 MB/s of raw storage growth, or 432 GB/day. Five recipients per message produce 50,000 deliveries/s and 25 MB/s of message bodies.

Measure connection memory separately. At an illustrative 20 kB each, 100,000 connections need 2 GB before other process memory or queued content. Bound outbound buffers. A slow device may receive less than a sender produces, allowing its queue to grow until the gateway runs out of memory even though storage remains healthy.

22.3.11 · End-to-end latency estimates

Suppose ingress takes 20 ms, commit coordination 30 ms, delivery queues 20 ms, and the recipient path 80 ms. The budget totals 150 ms before rendering.

This does not establish online p99. Test hot conversations, congestion, failover, and slow-device buffers. Measure persistence-receipt delay, device-delivery delay, and human-read delay separately. Offline delivery follows a different timeline, and a successful transport reply cannot establish every later stage.

22.3.12 · Failure scenarios

If the send reply disappears after commit, the repeated client identity recovers the saved message. If a gateway stops, devices reconnect and catch up from history. If a device consumes messages too slowly, disconnect or reduce its service before its buffer exhausts the gateway.

Membership removal can race with cached access state, so enforce the documented visibility rule on live delivery and history. Reconnect storms need admission controls and staggered retries. Test these event sequences directly; two permanently connected clients do not exercise them.

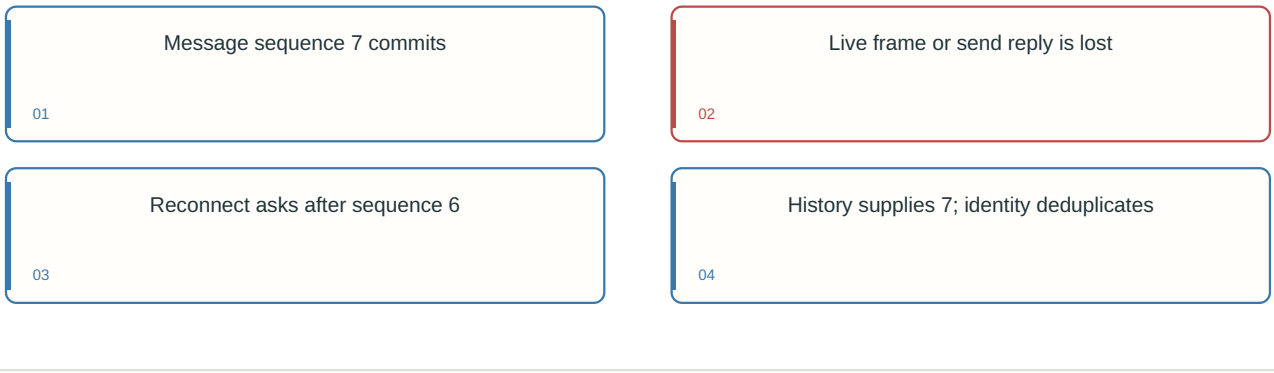


FIG. 22.3.3 **Follow the failure and its correction**

A missed live frame is recovered from history; repeating a send retains its original identity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.3.13 · Scalability improvements

Distribute independent gateway connections and partition history and ordering by conversation where useful. Let bounded workers perform group fan-out.

One popular conversation can still concentrate ordering and delivery work. It may require explicitly different large-channel semantics instead of including every recipient in a low-latency acknowledgment chain.

Preserve sequences and deduplication identities when ownership moves. Include history catch-up in capacity tests, because many devices returning after a gateway failure produce a different workload from ordinary message averages.

22.3.14 · Consistency requirements

Define a per-conversation order, and make the sender's confirmation agree with the saved message. A history cursor must resume without silently skipping required records. Cross-conversation total order is unnecessary unless the product requires it.

Live frames may repeat, so devices deduplicate by identity. Presence and receipts may propagate later, while delivered and read progress follow the chosen monotonic model: old updates must not move progress backward. These are separate guarantees; using WebSockets does not by itself provide exactly-once messaging.

22.3.15 · Security model

Enforce membership on sends, history, fan-out, search, and media access. Limit abusive sending, payload size, and group size. Keep message bodies out of routine telemetry.

If end-to-end encryption is required, specify device keys, recovery, membership changes, and the resulting limits on server features. TLS protects a connection but does not supply that full product guarantee. Attachments also need access and retention rules. An authorised chat page does not automatically protect every derived content URL.

22.3.16 · Deployment approach

Drain gateways during rollouts and support reconnects from older clients. Evolve message and receipt formats compatibly. Preserve history through the chosen replication and promotion rules.

Test reconnect and catch-up alongside ordinary sending. Restarting every gateway together may create a much larger burst than normal traffic. A new region needs explicit ordering and membership behaviour. Retain client identities and progress during replacement so uncertain sends do not become duplicated history records.

22.3.17 · Observability design

Measure persistence delay, live-delivery lag, sequence-gap recovery, reconnect rates, fan-out backlog, and hot conversations. Observe slow-device limits and membership denials too.

Correlate client and server identities in suitable detailed records without adding one unbounded metric label per message. Synthetic clients can send, disconnect, reconnect, and check both history and forbidden access. This follows the user's journey. A healthy connection count alone cannot establish that accepted messages are retained or revoked devices are blocked.

22.3.18 · Infrastructure cost considerations

Separate costs for connections, retained history, fan-out transfer, indexes, replicas, media, and telemetry. Thirty days of the assumed raw history need 432 GB/day × 30 = 12.96 TB before indexes and copies.

Media can dominate storage and transfer, so give it a separate quota and lifecycle. Use explicit or symbolic rates rather than assumed provider prices. Cost per retained message describes a different burden from cost per delivered body, especially when recipients and devices multiply the delivery path.

22.3.19 · Alternative architectures and trade-offs

WebSockets support bidirectional communication with low live delay, while requiring connection management. Server-Sent Events (SSE) can carry server updates alongside HTTP sends for suitable clients. Polling is simpler but creates requests during idle periods.

A single conversation sequence authority simplifies order and creates a hot-key boundary. Concurrent regional writing needs an explicit merge and ordering model and changes the contract. Compare alternatives using offline recovery, receipts, and privacy; transport features alone do not decide the architecture.

22.3.20 · Architecture evolution from small scale to large scale

First make durable history and reconnect recovery correct. Add gateway replicas with explicit connection routing. Introduce durable fan-out for growing volume or groups, then partition conversation ownership when measured storage or ordering work requires it.

Regional placement introduces another authority and failure decision. Throughout these changes, retain message identity: a retry after a shard move or gateway replacement should still name the same logical send. Increasing connection capacity while losing that identity can create duplicate messages.

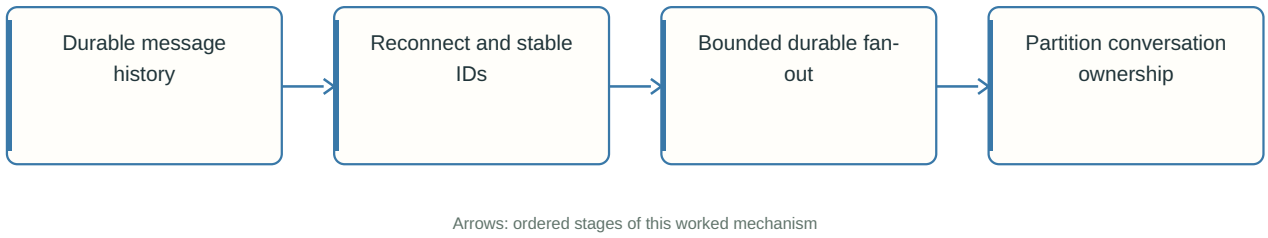


FIG. 22.3.4 **How the design can grow**

Scale connections and fan-out without losing per-conversation progress.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.3.21 · Design review identifying weaknesses and improvements

Review the meanings of “sent” and “delivered,” the hottest conversation, slow-device memory, and membership revocation. For each, describe an event sequence and its required result.

Test a commit followed by a lost reply, a dropped frame, gateway loss, stale routing, and a history request after membership removal. Check both the resulting list and receipts. These checks examine recovery and privacy, which a demonstration with two always-online clients cannot establish.

22.3.22 · A short implementation exercise

Build a client model that recognises message identities and detects sequence gaps. Save a message but suppress its live frame. Reconnect from an earlier sequence and confirm that history adds the message once to the visible list.

Then repeat its client_message_id and verify that storage creates no second logical message. Deduplicating displayed frames and deduplicating history creation protect different boundaries. Test both so a neat display does not hide duplicated durable records.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
with transaction():
    existing = find_by_client_id(sender, client_message_id)
    if existing: return existing
    require_member(sender, conversation)
    seq = allocate_conversation_sequence(conversation)
    msg = insert_message(conversation, seq, body)
    insert_outbox(msg.id, "deliver")
return persisted_receipt(msg)
# Device delivery and read receipt happen later.
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 6455: The WebSocket Protocol, 2011 ↗

   Original standard. Bidirectional transport framing; not durable chat delivery.

   Access reviewed · 21 October 2026

2. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

3. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

Notification Platform

Prerequisites: Design Patterns, Dependency Injection, and Object Lifecycles, API Design and Communication Contracts, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A password-reset message reaches a user hours after it was requested. The sender may have completed its work, yet the notification has missed its purpose.

We will follow a durable notification intent through channel selection, provider submission, and later evidence. The design must keep accepted work recoverable while recognising that provider acceptance, device delivery, and human viewing are different outcomes.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.4.1 · Functional requirements

An application records an intent to notify someone. The platform chooses permitted email, SMS, or push channels, renders a versioned template, submits the message, and tracks the outcome. Some intents may be scheduled or cancelled within a defined window.

Keep message purpose in the policy: operational and product messages may follow different preference rules. A channel adapter handles delivery mechanics; it should not silently decide which messages the user may receive. Provider acceptance, device delivery, and human viewing need distinct states and evidence.

22.4.2 · Nonfunctional requirements

Preserve accepted intents and make delay, expiry, and permanent failure visible. Give each message a business expiry. A password-reset message delivered hours late may be useless or harmful even if

submission eventually succeeds.

Reserve resources for critical work so a marketing batch cannot occupy every worker. Avoid duplicate external effects where the provider contract supports it and expose uncertainty where it does not. The promised outcome must match what the service can establish.

22.4.3 · Explicit assumptions and workload numbers

Assume ten million intents/day. Dividing by 86,400 seconds gives about 115.7 intents/s; a tenfold peak gives about 1,157/s. At 1 kB of initial intent and attempt metadata each, raw growth is 10 GB/day.

Retries, retained content, indexes, and copies add work. Treat channel quotas as separate configured inputs to check with the actual provider. The daily service average does not establish how quickly one SMS or email route can submit its share.

22.4.4 · Naive initial architecture

A first version calls a provider directly after a user action. The user waits for it, and a crash after the business commit but before sending can lose the notification obligation.

Save required intent in the business transaction or its outbox, then let a worker submit it. This closes that local loss window and lets a provider outage delay delivery without erasing accepted work. The repair introduces queued state, attempts, and recovery ownership; an untracked background function would not supply these guarantees.

22.4.5 · Core API contracts

POST /notifications accepts recipient, template version, variables, channel policy, expiry, and idempotency key. GET /notifications/{id} reports pending, submitted, delivered, expired, failed, or cancelled according to defined transitions.

Cancellation cannot recall a message already accepted by a provider. Callback endpoints verify the sender and recognise repeated events. A delivered state requires the relevant provider evidence, rather than simply an exception-free worker call. State which later outcomes remain outside the platform's knowledge.

22.4.6 · Data model and schema

Store the intent, template version, relevant preference information, channel/recipient tasks, attempts, and a provider-event inbox. Stable identities and uniqueness rules bind each intended effect to its task. Retain

provider references, including uncertain outcomes.

Separate sensitive content from minimal progress and audit records. Decide whether a preference change affects already queued work. Where the policy requires current consent or cancellation checks, a submission-time snapshot alone is insufficient.

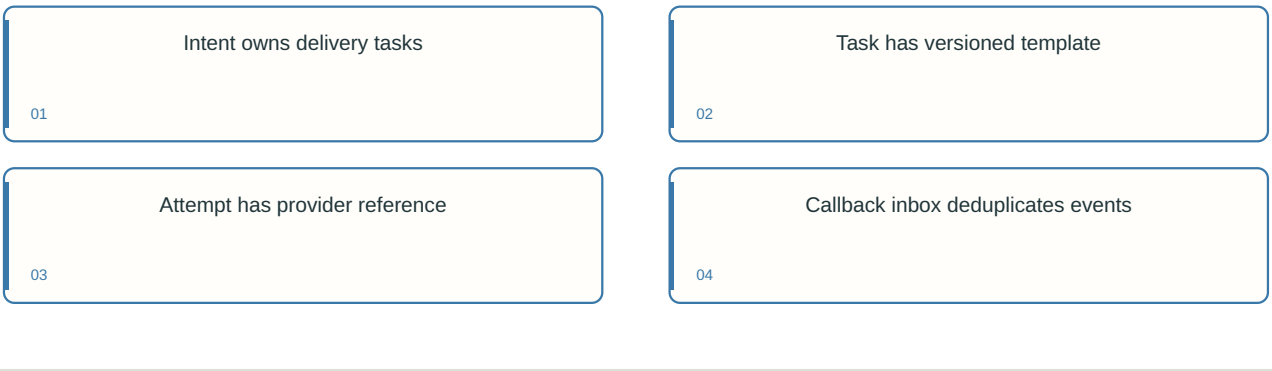


FIG. 22.4.1 **Data ownership and relationships**

Intent, delivery tasks, attempts, and callbacks record different stages of the notification.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.4.7 · Low-level component organization

NotificationService records intent, PreferencePolicy selects allowed channels, and TemplateRenderer creates content. SenderStrategy selects channel behaviour; ProviderAdapter translates that provider's protocol and outcomes. DeliveryRepository owns attempts and durable progress.

Factories can construct genuine channel variations, and decorators can measure calls, provided their lifetimes and result meanings stay clear. A retry wrapper must preserve operation identity and handle an uncertain prior submission. Sharing a method signature does not make two providers' recovery contracts interchangeable.

22.4.8 · High-level system architecture

Use durable ingress, preference and template access, channel queues, bounded workers, callback inboxes, and an outcome store. Separate critical and bulk capacity where their priorities require it.

Limit how much capacity one failing provider can occupy. For recovery, intent, attempt state, queue acknowledgment, and progress must agree about ownership. Draw those durable boundaries with the normal send path so a worker or broker restart has a clear starting point.

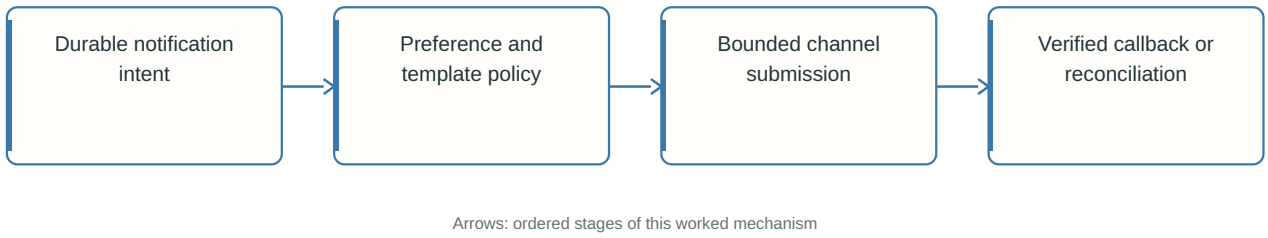


FIG. 22.4.2 **Follow the main operation**

Durable intent reaches a bounded channel worker and a provider with explicit outcome meanings.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.4.9 · Data and control flows

Deduplicate and save the intent, choose permitted content and channel, then persist a delivery task. Before submitting, the worker checks current attempt authority, expiry, and the required current-preference policy.

Use a stable provider operation identity where supported. Save either the submission evidence or an uncertain outcome before acknowledging according to the task protocol. Verified callbacks may arrive before or after the attempt reply. Apply them by identity and allowed transition, so an old event cannot incorrectly reverse later progress.

22.4.10 · Capacity estimates

The overall average is about 115.7 intents/s, but channels have separate limits. Suppose SMS tasks arrive at 100/s while the route submits 40/s. The queue gains 100 − 40 = 60 tasks/s. If both rates persist for ten minutes, 60 × 600 = 36,000 submissions accumulate.

Include retries, recipient fan-out, and expiry. Some waiting messages may become useless before the queue drains. Admission, priority, or a supported alternate route must respond to that channel limit, regardless of the service-wide average.

22.4.11 · End-to-end latency estimates

Suppose durable API acceptance takes 50 ms, queue wait takes 2 seconds, and provider submission takes 500 ms. Provider acceptance then occurs around 2.55 seconds after the API request.

These are illustrative budgets, not measured percentiles. Device delivery and human viewing require further events and evidence. Measure the end event that matters for the message's purpose as well as API latency. A quick acknowledgment only establishes quick acceptance of the obligation.

22.4.12 · Failure scenarios

A worker submits a message and dies before recording the reply. The provider may already have accepted it. Recover using a supported status query or provider idempotency; if neither resolves the outcome, retain the duplicate risk explicitly.

Switching providers can create a second message while the first is still in flight. Also handle malformed templates, bounded outage retries, expiry, and forged callbacks. Reconcile old submitted or uncertain tasks rather than leaving their outcome permanently implied by an empty worker queue.

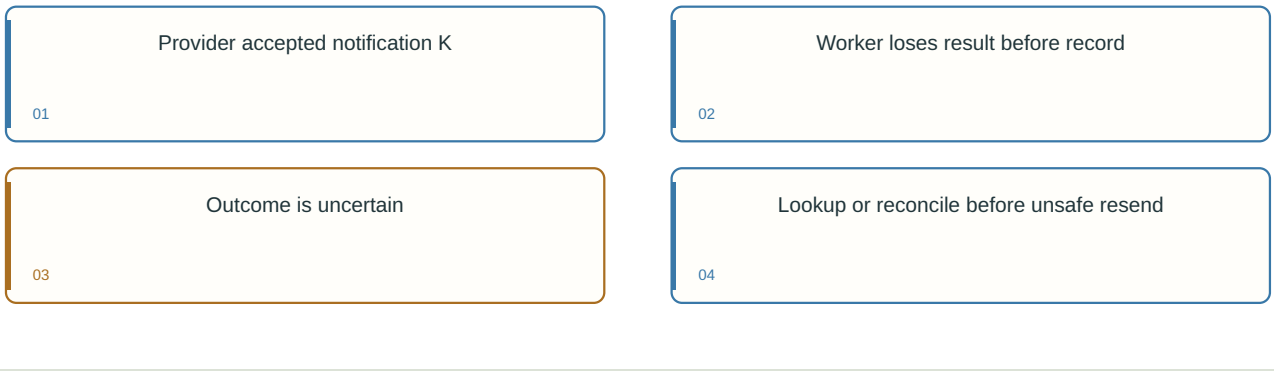


FIG. 22.4.3 **Follow the failure and its correction**

A sent message with a lost reply is uncertain, so recovery needs provider evidence or a stated duplicate policy.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.4.13 · Scalability improvements

Distribute independent recipients and channel tasks, preserving any required per-recipient order and concurrency bounds. More workers help only while the downstream route has capacity.

Add providers with explicit selection, quota, and uncertainty rules. An alternate route may improve some outages, but timeout followed by switching can submit twice. Keep critical capacity and business expiry visible. Useful, permitted progress is a better scaling objective than the largest possible number of parallel submissions.

22.4.14 · Consistency requirements

Commit accepted intent and its deduplication result atomically. Task delivery may repeat, so local transitions recognise stable identities. Provider acceptance and final delivery each require their own evidence.

Retain the template version so a retry does not change content under an old identity, and apply current preferences at the boundary the policy requires. Email, SMS, and push contracts differ. Name the local record or external effect protected by each mechanism instead of assigning a general exactly-once guarantee.

22.4.15 · Security model

Authorise which actors may send particular templates to particular recipients. Validate variables and encode or escape them for their channel, including safe header and content construction. Protect recipient data, credentials, and callback authenticity.

Bound intent creation to prevent abuse. Keep message bodies and tokens out of routine logs. Links inside messages need an appropriate audience and lifetime; rendering or delivery must not accidentally broaden the access they grant.

22.4.16 · Deployment approach

Deploy workers with durable attempt state and graceful draining so scale-down does not abandon unrecorded work. Template changes create new versions rather than altering content behind an existing retry identity.

Evolve task schemas compatibly and plan credential rotation. Test replacement during an outage and recovery: accumulated work must not flood a returning provider. Verify that acceptance and critical delivery still work when another channel is saturated.

22.4.17 · Observability design

Track accepted intents, queue age by channel, expiry, submission delay, uncertain outcomes, callbacks, and duplicate recognition. Separate critical and bulk outcomes so a good aggregate does not hide delayed important messages.

Alert on old unresolved tasks and rapid error-budget consumption. A queue can age into an incident without any worker exception. Correlate task, attempt, and provider evidence without exposing private content.

Enqueue success is one observable stage of the journey.

22.4.18 · Infrastructure cost considerations

Include intent and attempt storage, provider-submitted units, workers, callbacks, transfer, and operating effort. A retry can be billable even when it produces no useful delivery. Model channel_unit_price × submitted_units using explicit inputs.

Content length and character encoding may affect SMS billing under the selected provider's contract, which must be checked. Cost per accepted intent and cost per relevant successful delivery describe different work; keep their denominators separate.

22.4.19 · Alternative architectures and trade-offs

Direct synchronous sending uses little infrastructure and ties the user's request to the provider. Queued delivery adds durable state, recovery, and delay. An orchestrator can make escalation across channels explicit, while independent event reactions can make overall progress harder to inspect.

A backup provider adds both a recovery option and an uncertainty boundary. Compare alternatives using expiry, criticality, permissions, and available evidence. Eventually submitting something may fail the original notification purpose.

22.4.20 · Architecture evolution from small scale to large scale

Start with one sender behind a clear contract. Add durable intent and a worker when required work must survive failure or leave the user's request path. Introduce channel strategies and factories for real variation.

Separate critical and bulk resources as volume and harm justify it. Add failover and escalation after defining uncertain-outcome recovery. Preserve task, template, and attempt identities through these changes so scaling does not accidentally create fresh effects for the same intent.

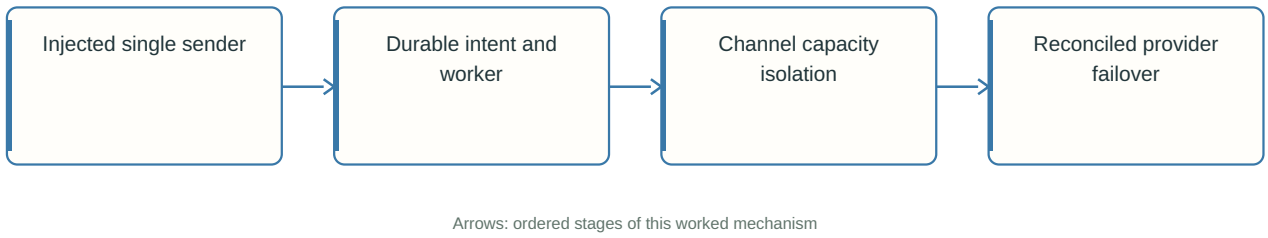


FIG. 22.4.4 **How the design can grow**

Add channels and routes after defining their expiry, priority, and retry behaviour.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.4.21 · Design review identifying weaknesses and improvements

Review whether “delivered” matches the evidence, whether expiry and current preferences are checked, and whether failover can duplicate a message. These questions involve product meaning as well as implementation.

Test send-then-crash, forged and repeated callbacks, preference revocation after enqueue, template changes, and channel saturation. Late events must not incorrectly reverse terminal progress. Successful submission requests alone cannot establish timely delivery to the intended people.

22.4.22 · A short implementation exercise

Build a recording sender that stores a submission but returns uncertainty. Process the task again and inspect its attempt identities and local state. It must not invent delivery evidence.

Add expiry and current-preference checks. Then switch providers after the uncertain submission and demonstrate how a second external effect can result. Implement only the recovery your mock provider supports, and make the remaining duplicate risk visible.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
def deliver(task, now):
    require_current_attempt(task)
    if now >= task.expires_at: return mark_expired(task)
    if not permitted_now(task): return mark_cancelled(task)
    sender = sender_factory.for_channel(task.channel)
    result = sender.send(task.stable_provider_key, render(task))
    if result.uncertain:
        return persist_uncertain(task, result.reference)
    return persist_submission_result(task, result)
# A provider acknowledgment is not a human-read receipt.
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

   Original engineering article. Dependency injection and wiring.

   Access reviewed · 21 October 2026

2. Transactional Outbox — Chris Richardson ↗

   Pattern author's explanation. Publication intent and duplicate delivery.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Social Media Feed

Prerequisites: Data Models and Storage Technologies, Caching and Content Delivery, Scalability, Load Balancing, and Data Distribution, Message Queues, Events, and Distributed Workflows

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A popular author publishes a post. Preparing a copy for every follower can make later reads quick, but many followers may never open their feeds. Waiting until each read avoids that unused work and makes the read more expensive.

We will compare both journeys and build a hybrid where it helps. Throughout, finding and ranking candidates must preserve the viewer's permission to see the underlying posts.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.5.1 · Functional requirements

Users publish posts, follow accounts, and read a ranked home feed. Provide bounded continuation while enforcing privacy, blocking, moderation, and deletion.

Ranking chooses among permitted posts. A high score cannot grant visibility. Candidate lists and engagement totals can lag their source, but the product must define their acceptable freshness. Separate finding interesting posts from deciding which posts the viewer may receive.

22.5.2 · Nonfunctional requirements

Assume a feed p99 target of 300 ms at the supported workload. Retain accepted posts, define how promptly access changes take effect, and give pagination understandable duplicate and omission

behaviour.

Ranking may change between refreshes. State whether a refresh begins a new generation and what its continuation cursor preserves. Because candidate caches can be old, specify which source and permission properties are rechecked on every served page.

22.5.3 · Explicit assumptions and workload numbers

Assume ten million daily readers make twenty feed reads each. That is 200 million reads/day, or about 2,315/s. Assume one million posts/day.

For ordinary authors, use a simplified average of one hundred followers and a 32-byte candidate reference. Preparing all those references creates 100 million/day, or 3.2 GB raw. A celebrity with millions of followers belongs to a different workload class: this average hides both the publication burst and work spent for inactive followers.

22.5.4 · Naive initial architecture

Start with indexed posts and follow relationships. When a reader requests a feed, retrieve posts by followed authors and rank the candidates. This avoids maintaining a prepared copy for every follower.

As follow sets grow, retrieval and ranking can make each read expensive. Measure that work and the follower distribution before materialising feeds. Materialisation means storing a derived result in advance; it saves repeated reads while adding publication work, storage, and refresh responsibilities.

22.5.5 · Core API contracts

POST /posts creates a versioned post. POST /follows changes a relationship. GET /feed returns a bounded page, continuation cursor, and relevant ranking generation. DELETE /posts/{id} records authoritative removal.

Bind cursors to the viewer and applicable filters, validate them, and define their snapshot or continuation meaning. A cursor locates a page; it does not preserve permission after access changes. Reapply the required visibility policy whenever another page is served.

22.5.6 · Data model and schema

Posts contain identity, author, version, visibility, state, and creation time. Follows connect reader and author. Optional candidate records connect reader to post and source version, with ranking information. Retain

model version or generation where reproducibility needs it.

Posts and access relationships are authoritative; candidates are derived references. Stable source identity supports deduplication, removal, and rebuilds. State whether copied features are snapshots or are expected to follow later updates.

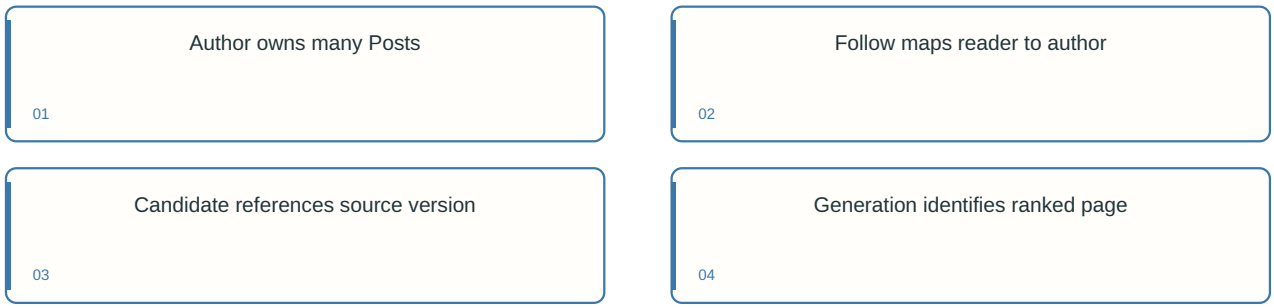


FIG. 22.5.1 **Data ownership and relationships**

The post and access relationships are authoritative; feed candidates are derived references.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.5.7 · Low-level component organization

PostService owns publication and FollowService owns relationship changes. CandidateGenerator finds or prepares references, FeatureFetcher obtains bounded ranking inputs, and Ranker scores them. VisibilityFilter enforces access; FeedAssembler merges, deduplicates, and pages results.

Keep visibility independent of desirability. Missing features may change ranking, but must not broaden permission. These boundaries let us improve candidate or feature performance without moving access rules into a model score.

22.5.8 · High-level system architecture

Commit publications durably with outbox intent. Workers prepare candidates for ordinary authors, while high-fan-out authors can be retrieved at read time. Candidate storage, ranking compute, and permitted caches support serving.

This hybrid avoids immediately writing every celebrity post into every follower's feed. It introduces a rule for choosing and changing strategies, plus reconciliation work. Keep model deployment decisions distinct from the data path, and preserve source state and access checks whenever derived copies lag.

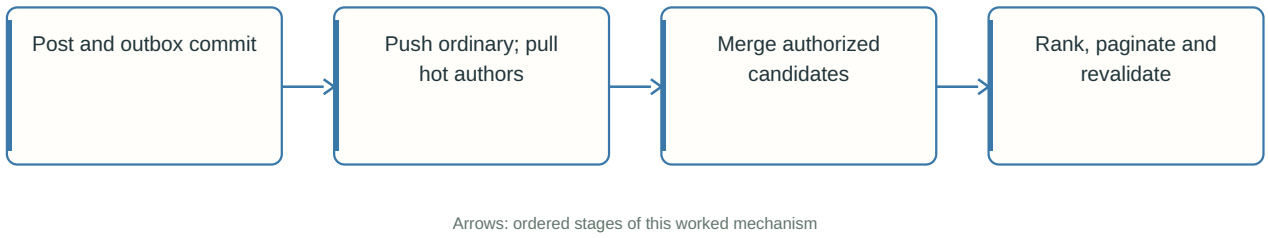


FIG. 22.5.2 **Follow the main operation**

Combine candidates, enforce visibility, rank, and return a bounded page.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.5.9 · Data and control flows

Publication commits the post and fan-out intent. Workers insert candidate references under stable identity. A read combines prepared candidates with posts pulled from high-fan-out authors, checks required current access and post state, fetches bounded features, ranks, deduplicates, and returns a continuation.

Track the generation. A deleted reference may remain physically stored while serving rejects it, if the deletion contract permits that delay. Permission to expose a post and eventual storage cleanup have separate completion boundaries.

22.5.10 · Capacity estimates

The assumed 200 million daily reads average about 2,315/s; a fivefold peak is about 11,574/s. One million posts × one hundred followers creates 100 million candidate writes/day.

A post reaching ten million followers creates its own burst, regardless of the average. Also estimate candidates ranked per request and feature lookups per candidate. A modest API rate can multiply into much larger storage and compute demand through these internal fan-outs.

22.5.11 · End-to-end latency estimates

Suppose the path spends 20 ms at the edge, 50 ms retrieving candidates, 80 ms fetching features, 70 ms ranking, and 30 ms preparing a response. The proposed total is 250 ms before further tail and queue allowance.

Measure end-to-end p99 against the 300 ms objective. One required slow feature can dominate completion, so define safe missing-feature behaviour. Measure publication freshness separately: a fast page can still contain old or empty candidates.

22.5.12 · Failure scenarios

A worker may repeat fan-out after a crash. Stable candidate identity prevents the repeat from adding duplicate effects. A celebrity burst needs its selected pull or bounded distribution path to avoid unrestricted queue growth.

Privacy changes can precede candidate cleanup, so current serving checks remain essential. If ranking fails, a permitted chronological feed may be an acceptable fallback. Test access on that fallback too; its simpler ordering does not remove deletion or visibility obligations.

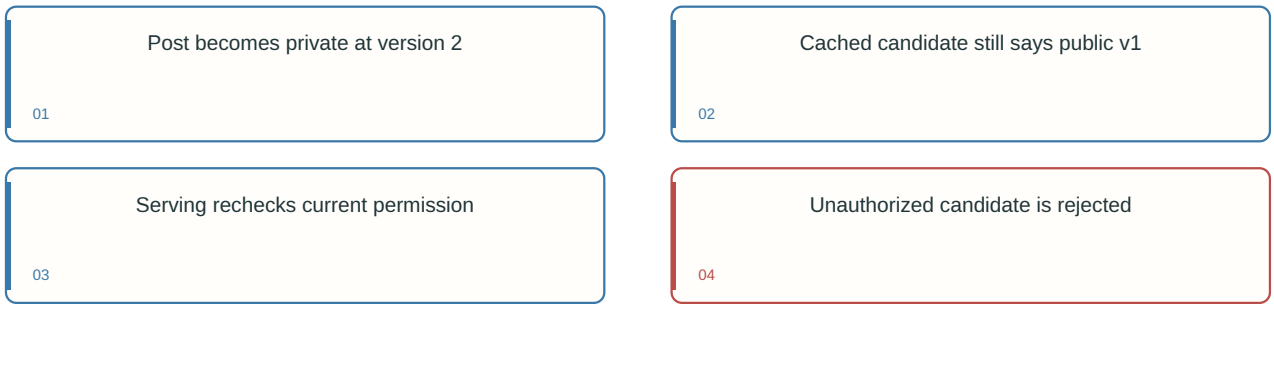


FIG. 22.5.3 **Follow the failure and its correction**

Repeated fan-out and stale candidates must not create duplicate or forbidden visible posts.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.5.13 · Scalability improvements

Partition candidate work by a measured ownership key, batch feature reads, and cache within the allowed freshness scope. Bound candidates retrieved and ranked per read. Give rebuilds separate budgets so they cannot consume all live-serving resources.

Choose the push/pull threshold from write amplification, reader activity, and read cost. Moving an author across that threshold needs a transition that prevents missing and duplicate candidates. More workers help only while the stores they update have remaining capacity.

22.5.14 · Consistency requirements

Post identity, updates, and deletion need a suitable transactional authority. Candidate projections and engagement totals may propagate later. An author who expects an immediate view of their own post may need a direct-source overlay or another supported read path.

Access still follows its own freshness rule even when candidates are stale. Specify which page properties belong to a ranking snapshot and which are revalidated. The feed contains several observable consistency requirements.

22.5.15 · Security model

Enforce author permissions, viewer visibility, blocking, and moderation on every relevant path. Validate cursors and bound scraping and feature-query amplification. Client scores must never grant access.

Ranking records may expose private behaviour. Limit retention and access, and propagate required deletion into features and analytical copies. Apply tenant and content-scope rules to model inputs and outputs as well as the final page.

22.5.16 · Deployment approach

Version rankers, use representative canary cohorts, and check rollback compatibility with stored features. Mixed-version readers must understand fan-out formats during rollout.

Separate rebuild capacity from live reads and publication. A region-local feed needs a defined route for source and access-policy updates before promising privacy and local reads together. Observe model releases alongside code and schema releases because each can change the served result.

22.5.17 · Observability design

Track publication-to-candidate delay, candidate counts, visibility denials, read latency, feature tails, model versions, duplicates, and hot-author backlog. Measure relevance and useful content separately from infrastructure timing.

Low p99 does not establish correct, varied, current pages. Pair operational measures with content and privacy checks. Detailed evidence can retain source versions and generations without adding unbounded per-user or per-post labels to metrics.

22.5.18 · Infrastructure cost considerations

Include candidate writes, feature storage and reads, ranking CPU, response transfer, and rebuilds. The simplified 3.2 GB/day of raw references excludes indexes and other metadata.

Compare push and pull using the fraction of followers who actually read. Preparing feeds for inactive readers can spend work never used. Use explicit fictional rates or verified current quotes, and compare designs with equal visibility, retention, freshness, and recovery requirements.

22.5.19 · Alternative architectures and trade-offs

Fan-out-on-write performs work when a post arrives to make later reads cheaper. Fan-out-on-read avoids unused preparation and adds work to each active read. A hybrid assigns different traffic classes to each and adds transition complexity.

Precomputed ranking can save serving CPU while adding freshness delay. Compare these choices with active-reader ratios and author skew. Include deletion and privacy requirements in the comparison, since a faster page with weaker visibility is a different service contract.

22.5.20 · Architecture evolution from small scale to large scale

Begin with indexed pull reads, then add bounded ranking and useful caching. Prepare ordinary-author candidates when repeated read work becomes limiting. Introduce a distinct high-fan-out path after measuring author skew.

Regional copies need source-version and privacy-update rules. Revisit thresholds as reading, posting, and follow distributions change. Preserve stable candidate identity and explicit generations so each stage retains the intended continuation and duplicate behaviour.

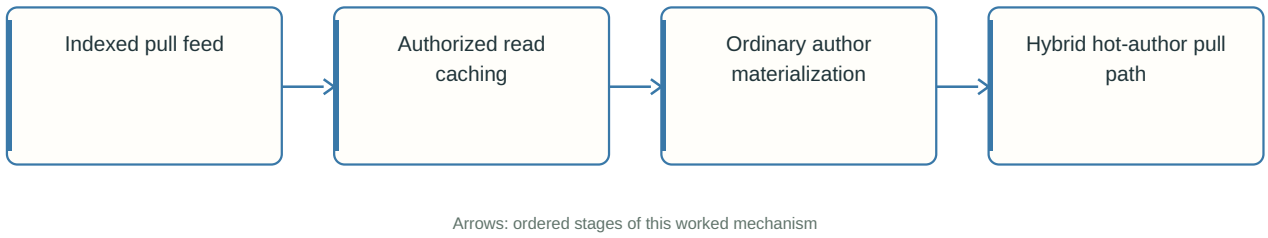


FIG. 22.5.4 **How the design can grow**

Prepare common feeds only where measured reading saves more work than publication adds.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.5.21 · Design review identifying weaknesses and improvements

Review inactive-reader assumptions, high-degree authors, and access revocation. Put a post into cached candidate pages, then delete it or make it private. Verify that later pages obey the visibility rule.

Replay fan-out, pause a worker, and disable a feature dependency. Check candidate identities, source versions, and fallback access. An empty queue establishes progress through a worker, but does not prove that each intended reader obtained one usable, authorised reference.

22.5.22 · A short implementation exercise

Build push and pull generation over a small follow graph containing one very high-degree author. Count work for explicit user journeys so inactive-reader costs remain visible.

Delete a materialised post and test the serving filter. Replay a fan-out event and check for visible duplicates. Explain which strategy spends work before anyone reads and which moves work into active reads, while enforcing the same privacy rules in both.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
def feed(reader, cursor):
    candidates = materialized(reader) + pull_hot_authors(reader)
    unique = deduplicate_by_post_id(candidates)
    permitted = [p for p in unique if visible_now(reader, p)]
    features = fetch_bounded_features(permitted)
    ranked = rank(permitted, features, model_version)
    return page(ranked, validated_cursor(cursor, reader))
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   Access reviewed · 21 October 2026

2. Application best practices — Microsoft Architecture Center ↗

   Official guidance · accessed 21 Oct 2026. Scaling and application boundary review.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Video Streaming Platform

Prerequisites: Networking Foundations for System Design, Caching and Content Delivery, File Systems, Search Infrastructure, and Data Pipelines, Deployment, Cloud Infrastructure, and Platform Architecture

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

An encoder has written two of the three segments required for a video. If the service now displays “Ready,” a viewer may start playback and encounter a missing piece.

We will follow uploaded bytes through validation, conversion, and publication. A playable generation must name a verified set of outputs, and the delivery path must enforce the video's access and deletion rules.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.6.1 · Functional requirements

An owner uploads a video, the service validates and converts it, and an authorised viewer receives a playback manifest and media segments. Support resumable upload, processing progress, and deletion.

Adaptive playback lets a client switch among available renditions as conditions change. A rendition is an encoded version suitable for a particular playback profile. Switching cannot repair missing media. We need both a transformation workflow and a deliberate decision that a complete generation is ready to play.

22.6.2 · Nonfunctional requirements

Preserve sources and required derived media under stated durability and retention rules. Choose playback-start and smoothness objectives for supported clients and measure the delivery journey.

Asynchronous processing must expose failed and review states. Protect manifests, segments, thumbnails, and download links as well as the player page. A private metadata row cannot protect media bytes that remain publicly fetchable.

22.6.3 · Explicit assumptions and workload numbers

Assume 1,000 uploads/day averaging 1 GB and thirty minutes each. That is 1 TB/day of source bytes and 500 source-hours/day. If selected renditions total five output GB per source GB, they add 5 TB/day.

Assume 10,000 simultaneous viewers average 3 Mb/s each. Delivery is 30 Gb/s; dividing by eight gives 3.75 GB/s before overhead. Encoding, retention, and playback impose different work. These teaching ratios are not codec or infrastructure benchmarks.

22.6.4 · Naive initial architecture

A prototype accepts an upload and lets one worker create one playback file. This is easy to trace, but large transfers and computation depend on fragile local execution, and one file adapts poorly to varying networks.

Direct upload to storage keeps large bytes out of application workers. Durable encoding tasks retain required progress. Start with a small supported rendition set; producing many unused formats adds compute and storage without necessarily improving the intended viewers' experience.

22.6.5 · Core API contracts

POST /videos creates an upload intent. Scoped multipart authorisation permits transfer; completion validates the stored input. A status endpoint reports progress, and playback returns an authorised manifest reference. Deletion tracks required cleanup.

Distinguish upload complete, processing, playable, and failed. Define playback-capability scope, lifetime, and cache behaviour so an old URL does not grant indefinite access. Accepted bytes are input to processing, rather than proof of usable media.

22.6.6 · Data model and schema

Store video identity, owner, source version, and state. Upload sessions track unfinished transfers. Encoding jobs reference input digest, profile version, and attempt epoch. Renditions identify profile, version, object key, and checksum. A manifest generation names its required artifacts.

Publish only after verifying that set. Keep published references immutable, introducing a new version when bytes change. Source and owner relationships let deletion, repair, and inventory reconciliation find every derived object.

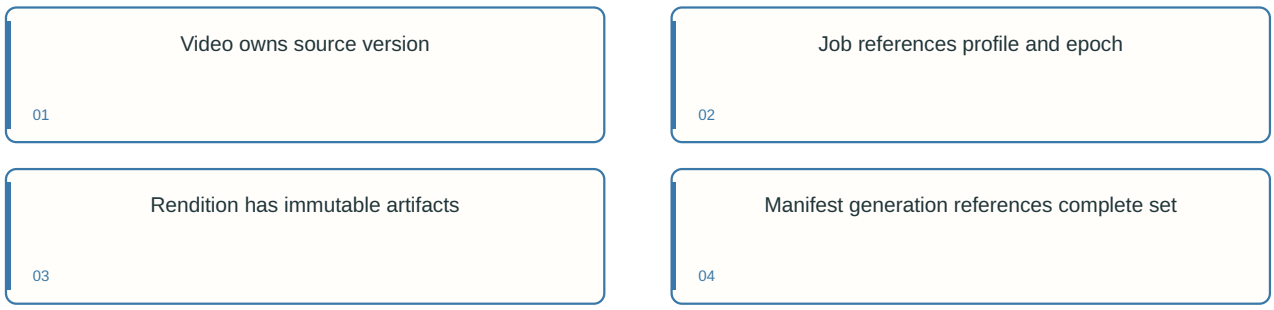


FIG. 22.6.1 **Data ownership and relationships**

A published manifest generation names identified, verified outputs from a particular input and profile.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.6.7 · Low-level component organization

UploadService grants upload permission, MediaValidator checks supported bounded input, and EncodingPlanner chooses profiles. WorkerLease manages attempts; TranscoderAdapter performs isolated conversion. ArtifactPublisher advances published metadata only after required outputs are verified.

PlaybackService checks access and issues scoped references. Separate writing artifacts from publishing a generation: finishing one file says nothing about the remaining required files. Publication needs evidence about the whole set.

22.6.8 · High-level system architecture

Use object storage for immutable sources and segments, durable workflow state and a task queue, isolated transcoders, metadata storage, and edge delivery. The application handles control and access while bytes flow through storage and CDN facilities.

Give untrusted parsers limited resources, permissions, and egress. Keep source and derived retention explicit. The queue schedules work; source storage, workflow state, and published metadata still have distinct recovery responsibilities.

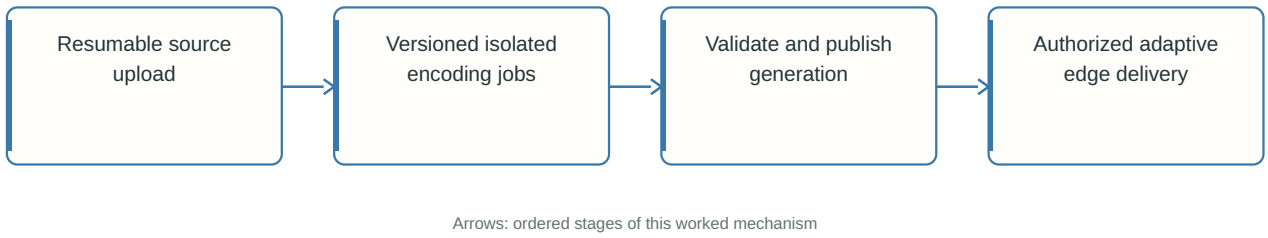


FIG. 22.6.2 **Follow the main operation**

Uploaded bytes become playable only after required transformation and publication checks.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.6.9 · Data and control flows

Validate the completed upload's identity and choose a versioned plan. An attempt produces artifacts, then verifies the required segments and manifest. Atomically publish the playable metadata generation only when that set is valid.

Failed attempts may leave unpublished objects for cleanup or safe reuse. Viewers obtain authorised references and fetch supported renditions. Publishing after only the first output is valid only under an explicitly supported progressive-publication contract. Otherwise, it exposes an incomplete generation as ready.

22.6.10 · Capacity estimates

Thirty days of the assumed input need 30 TB of originals and 150 TB of renditions before copies and overhead. Viewer transfer at 30 Gb/s is much larger than metadata traffic.

Encoding capacity depends on measured content and profile costs. For illustration, two compute-hours per source-hour would turn 500 source-hours/day into 1,000 compute-hours/day. Measure that ratio with representative media before sizing workers; the calculation provides a workload assumption, not a performance result.

22.6.11 · End-to-end latency estimates

Suppose playback authorisation takes 50 ms, manifest delivery 30 ms, and initial buffering 800 ms. The example path totals 880 ms.

A cache miss or constrained client changes this path. Measure startup, rebuffering, rendition switches, and failures separately from metadata API p99. Segment delivery and player behaviour are part of the experience. A quick permission check provides little evidence about a missing or slowly arriving first segment.

22.6.12 · Failure scenarios

An interrupted upload, corrupt source, partial encoding, and missing published object are different failures. Give each an explicit state and repair path. Keep partial generations unpublished and verify the final set.

If edge delivery fails, fallback can send enormous traffic to the origin, so protect it. Revocation and deletion must affect playable references and copies under the selected capability and cache rules. Test through actual media requests rather than checking only metadata state.

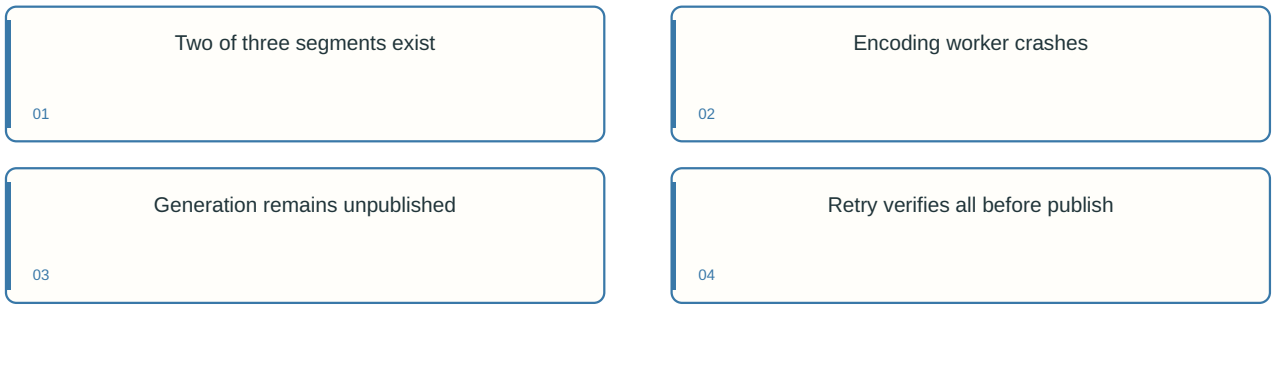


FIG. 22.6.3 **Follow the failure and its correction**

Partial segment output stays unpublished until the required generation is complete.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.6.13 · Scalability improvements

Scale upload transfer, encoding, and playback separately. Bound encoding concurrency and prioritise jobs by deadlines. Cache immutable segments where access policy permits reuse.

A viral video creates hot-object delivery pressure, which may leave metadata storage mostly unaffected. Plan origin protection and regional-copy cost during cache loss. More profiles and transcoders also create backfill and storage work. Expand the measured limiting path rather than increasing every component together.

22.6.14 · Consistency requirements

A published generation must name a usable artifact set. Attempts may repeat, but stable output identities and conditional completion prevent an old worker from replacing a newer generation. An epoch is an attempt-authority number; publication rechecks it at the authoritative transition, as the code illustrates.

Declare cache reuse intervals and separate privacy and deletion requirements. A recovery region must possess the objects required by its publication metadata. Copying only some bytes does not make a complete failover source.

22.6.15 · Security model

Authorise upload and playback, bound sizes and supported formats, isolate parsers and encoders, and give workers narrow storage permissions. Protect signed capabilities and keep them out of routine logs.

Check manifests, segments, and thumbnails directly. Test capability expiry, cache keys, membership removal, and deletion end to end. Privacy must cover every route to content, including objects fetched without reopening the player page.

22.6.16 · Deployment approach

Version encoding profiles and retain usable older outputs until a validated transition permits removal. Workers need durable claims and graceful shutdown. Give expensive backfills budgets that preserve live processing and playback.

Test restoring metadata to an older point while source bytes survive. Source identities and artifact inventories must let the application reconcile what exists. Preserving running containers alone cannot repair a lost relationship between media and metadata.

22.6.17 · Observability design

Observe upload completion, oldest jobs, failed profiles, compute-to-media ratios, and ready-generation integrity. Track edge hits, origin transfer, startup, rebuffering, and access outcomes for playback.

Correlate source, profile, and generation in detailed evidence. Alert on missing segments and stuck videos even when workers report few exceptions. A synthetic private-video journey should include successful playback followed by revocation, covering both delivery and access changes.

22.6.18 · Infrastructure cost considerations

Include originals, renditions, repeated encoding, compute, delivery requests, transfer, metadata, and telemetry. Sustaining 3.75 GB/s for one hour transfers 3.75 × 3,600 = 13,500 GB, or 13.5 TB, before overhead.

Viewer-demand duration matters alongside peak concurrency. Compare profile count and retention with quality needs, using labelled fictional rates or verified quotes. A smaller rendition set saves work while changing the client's playback choices.

22.6.19 · Alternative architectures and trade-offs

A single file is simple to manage and offers little adaptation. Segmented adaptive delivery adds manifests and a larger artifact lifecycle. Pre-encoding makes outputs ready early while spending resources on profiles some viewers never use.

On-demand encoding reduces unused work and adds startup or burst risk. Managed conversion delegates selected operations, while publication, access, and reconciliation remain design decisions. Compare alternatives under equal client, privacy, and recovery requirements.

22.6.20 · Architecture evolution from small scale to large scale

Start with resumable upload and durable processing identity. Add a small tested adaptive set with versioned publication. Introduce suitable edge delivery and grow encoding independently from playback.

Add regional objects or profiles for demonstrated locality, recovery, or quality needs. Retain exact source and profile identities so each result remains explainable and reproducible. Increased capacity must preserve what playable, private, and deleted mean for existing videos.

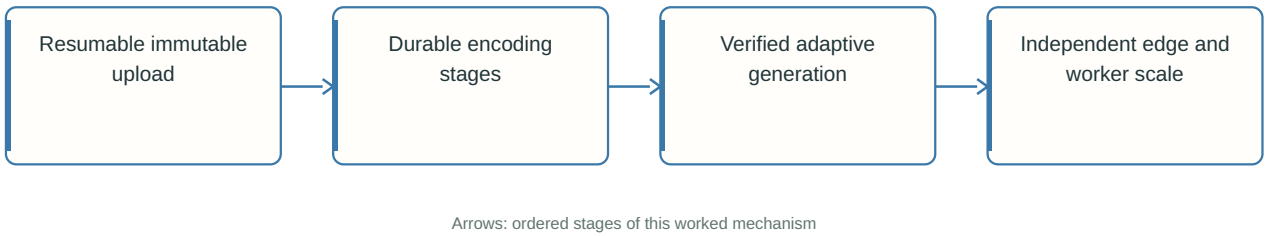


FIG. 22.6.4 **How the design can grow**

Scale encoding separately from delivery and preserve versioned playable sets.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.6.21 · Design review identifying weaknesses and improvements

Review three claims separately: every ready generation is playable, private artifacts remain protected, and origin survives edge failure. Each needs its own evidence.

Test corrupt input, interruption between output and publication, repeated completion, missing segments, expired access, and deletion after caching. One successful warm-cache playback leaves these partial-processing and private-recovery paths unexamined.

22.6.22 · A short implementation exercise

Build a publisher that requires three segments. Write two and stop. The manifest must stay unpublished. Resume with the same job identity, write the third, validate checksums, and atomically advance one metadata generation.

Next remove a published segment deliberately and run an integrity check. It should identify the broken set. Explain the difference between the boundary that established readiness and the later failure that damaged an already published generation.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
def publish(job):
    require_current_epoch(job)
    artifacts = expected_artifacts(job.source, job.profile)
    require(all(exists_and_valid(a) for a in artifacts))
    with transaction():
        assert_current_epoch(job)
        generation = record_complete_manifest(artifacts)
        mark_playable(job.video, generation)
# Partial worker output is never silently declared ready.
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. RFC 8216: HTTP Live Streaming, 2017 ↗

   Original protocol description · Informational RFC. One adaptive streaming protocol; do not confuse Informational RFC with Standards Track.

   Access reviewed · 21 October 2026

2. RFC 9111: HTTP Caching, 2022 ↗

   Original standard. HTTP cache semantics; application cache races are separate examples.

   Access reviewed · 21 October 2026

3. OpenLineage: core model and documentation ↗

   Official specification documentation · 1.54.0 at access. Dataset, job, and run lineage concepts.

   Access reviewed · 21 October 2026

Ride-Hailing System

Prerequisites: Data Models and Storage Technologies, Scalability, Load Balancing, and Data Distribution, Distributed Systems Fundamentals, Message Queues, Events, and Distributed Workflows

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Two riders request a trip, and both matchers choose the same nearby driver. The search may be accurate, yet it has not decided who receives the assignment.

We will separate recent-location search from the authoritative claim on a driver. That distinction lets us improve matching and distribute location traffic while preserving one valid active assignment.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.7.1 · Functional requirements

A driver reports location and availability. A rider requests a trip; the service finds candidates, offers work, records an accepted assignment, and tracks progress or cancellation.

A nearby driver is a candidate. Receiving an offer also does not establish assignment. Define offer expiry and what the rider sees while matching is unresolved. Two matchers observing the same old availability must not create incompatible active trips for one driver.

22.7.2 · Nonfunctional requirements

Set a bounded machine-side dispatch target for the assumed region and preserve exclusive active assignments. Treat position age and accuracy as uncertainty rather than ground truth, and protect precise location information.

Trip transitions must survive retries and temporary disconnection. A missing heartbeat does not prove a trip has ended. Human offer acceptance and payment-provider outcomes follow different timelines. Keep those states visible so every delay does not become an unsupported failed-trip result.

22.7.3 · Explicit assumptions and workload numbers

Assume 100,000 online drivers report every five seconds. This produces 100,000 / 5 = 20,000 updates/s. At 100 raw bytes each, the stream is 2 MB/s, or 172.8 GB/day if every update is retained.

Assume 100 new trip requests/s. Estimate map requests, candidate evaluation, regional skew, and retention separately. The high-volume location projection and lower-volume assignment authority have different jobs and need not use the same storage or coordination mechanism.

22.7.4 · Naive initial architecture

Begin with a service holding recent positions and selecting a nearby available driver. Sequential requests are easy to follow. With concurrent requests, two matchers may select the same driver before either changes availability.

Keep search positions as a derived projection and perform the exclusive claim against authoritative assignment state. Improve spatial lookup after this boundary is correct. Faster nearest-neighbour search can produce better candidates, but cannot decide between simultaneous claims. Stale locations can still harm match quality without violating exclusive assignment.

22.7.5 · Core API contracts

Location updates include position, source sequence, and observed time. POST /trips creates a stable rider request. POST /offers/{id}/accept attempts a valid claim; status and cancellation expose explicit trip transitions.

Derive driver identity from authentication and validate plausible updates. Expired or superseded offers return a documented conflict. Repeated acceptance recovers the existing result. A caller-supplied driver identifier must not bypass ownership checks or replace another assignment.

22.7.6 · Data model and schema

driver_state holds availability, assignment_id, and version. A trip holds rider, state, driver, and version. An offer identifies trip, driver, expiry, and status. Latest-position records include driver sequence, spatial cell,

and observation time; retained location history has a separate restricted lifecycle.

Protect active assignment through conditional transitions or suitable uniqueness constraints. A spatial index helps find candidates; it must not independently decide whether a driver is free.

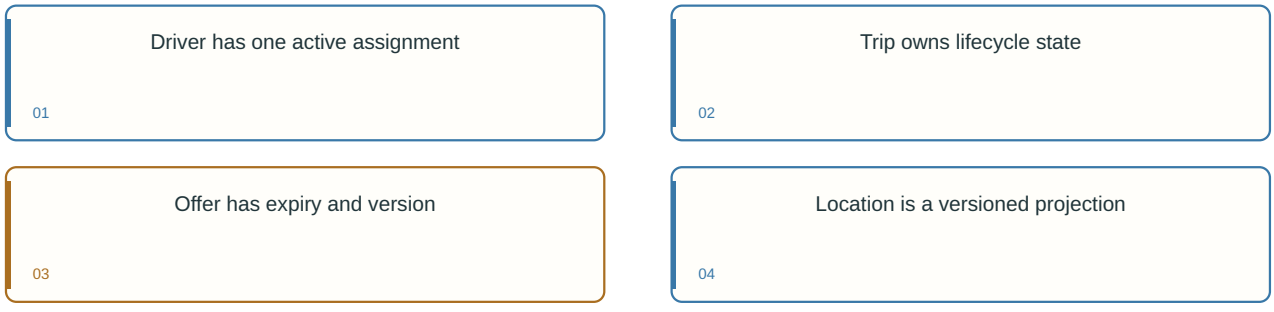


FIG. 22.7.1 **Data ownership and relationships**

Latest position helps search; driver and trip state decide exclusive assignment.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.7.7 · Low-level component organization

LocationIngestor validates update order. SpatialIndex finds candidates, Matcher ranks a bounded set, and OfferService checks offer validity. AssignmentRepository owns the claim; TripStateMachine controls later transitions.

Map and ETA adapters have separate deadlines so provider failure cannot corrupt assignment. Stable trip and acceptance identities make retries recover the same work. Many matchers may choose candidates, while the final availability decision stays at its atomic owner.

22.7.8 · High-level system architecture

Use a regional location stream and spatial projection, durable trip and assignment storage, matching workers, offer delivery, and trip events. Geographic partitioning can keep searches local, provided driver movement has a defined handoff.

Configuration may arrive from a control plane, but two regions with stale availability copies must not both assign one driver. Assignment requires one valid authority or a transfer protocol. Draw local search and exclusive ownership as distinct paths.

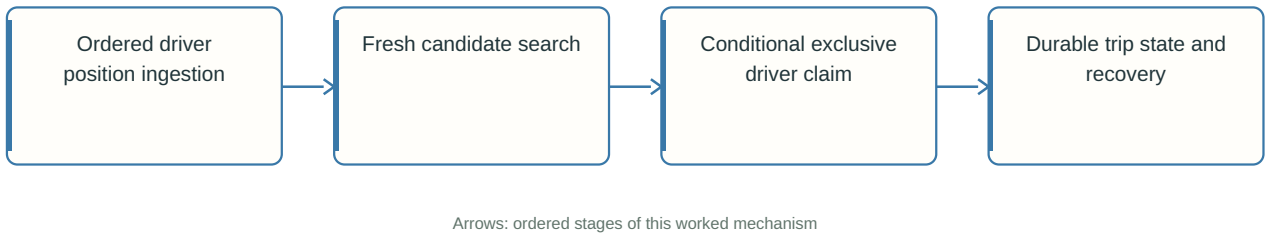


FIG. 22.7.2 **Follow the main operation**

Candidate search precedes a valid conditional offer acceptance.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.7.9 · Data and control flows

Commit the rider request identity. Search sufficiently fresh candidates, recheck authoritative availability, and create a bounded-lifetime offer. On acceptance, verify that the offer is still pending, unexpired, and compatible with current ownership.

Atomically bind driver and trip under the chosen transaction model. Repeated acceptance recovers its result; stale acceptance conflicts. If the reply disappears, trip status resolves uncertainty. The conditional acceptance, rather than delivery of the offer, prevents incompatible winners.

22.7.10 · Capacity estimates

The workload produces 20,000 position updates/s and 100 dispatches/s. Evaluating twenty candidates per dispatch adds 100 × 20 = 2,000 candidate evaluations/s before retries and ETA requests.

Do not automatically call a routing service for every position update. Estimate hot cells, city-centre peaks, and boundary crossings. Candidate pruning can lower map cost while leaving the exclusive claim unchanged. Global averages can hide the one area where most work is concentrated.

22.7.11 · End-to-end latency estimates

Suppose ingress takes 30 ms, candidate lookup 40 ms, bounded ETA evaluation 80 ms, and assignment work 30 ms. The machine-side planning budget totals 180 ms.

Human acceptance is a separate interval. Missing candidates, communication delays, expiry, and retries extend the whole matching journey. Measure time in those states and validate the budget against

representative cities, traffic, and providers. An optimistic machine sum does not establish the rider's total wait.

22.7.12 · Failure scenarios

An older position must not move the serving projection backward under its source-sequence rule. If two requests choose one driver, only one authoritative claim may win. A paused worker must not accept an offer superseded while it was stopped.

Recover lost offers or replies through status. A map outage may allow a justified simpler ranking or require paused matching, but cannot bypass exclusive ownership. Exercise each failure at its state boundary so a search problem cannot become conflicting trip commitments.

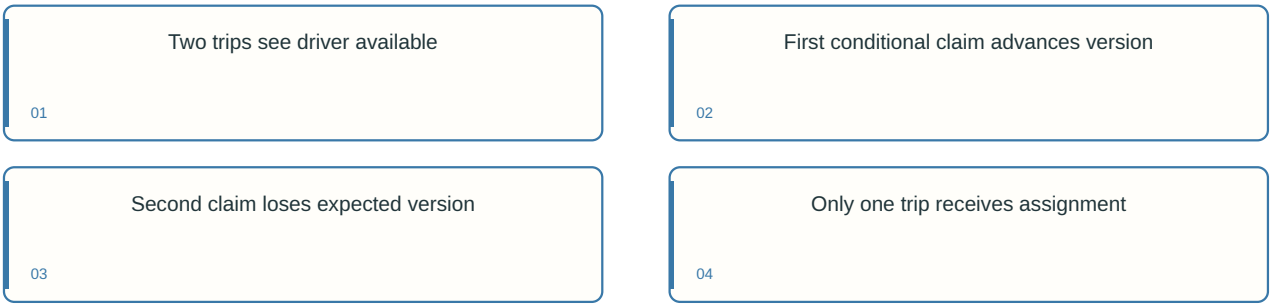


FIG. 22.7.3 **Follow the failure and its correction**

Two riders can choose one candidate, but only one authoritative claim may win.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.7.13 · Scalability improvements

Scale location ingestion and spatial serving separately from assignment transactions. Prune or batch ETA calls. When splitting regions or cells, transfer the driver's authority explicitly.

Candidate caches need age information. Extra matchers cannot automatically divide a hot claim owner whose decisions remain serialised. Finer cells add boundary queries and movement coordination, so choose them from measured concentrated work while retaining one active-assignment rule.

22.7.14 · Consistency requirements

Location can propagate through a delayed projection with a declared age policy. Assignment requires a coordinated conditional change linking driver and trip. Crossing a regional boundary requires authority transfer, rather than independent acceptance on each side.

Give status reads the required read-after-accept behaviour. Observation time helps judge position age; sequence rules reject old updates, and fencing rules reject old owners. These mechanisms protect different kinds of state and should be explained separately.

22.7.15 · Security model

Authenticate driver updates and limit rider and driver views to the relevant trip. Restrict precise history, including support and administrator paths, retain only justified data, and audit privileged use.

Bound geographic scraping and costly map requests. Choosing an arbitrary driver identifier does not confer claim authority. Keep raw coordinates out of broad telemetry where unnecessary. Apply privacy rules to streams, caches, exports, and logs as well as the displayed map.

22.7.16 · Deployment approach

Use regional placement with a stated failure model and durable assignment state. Keep releases compatible with older clients and in-progress offers. Drain new work while retaining existing trips and offers.

Recovery must reconcile active assignments and uncertain provider outcomes. An old snapshot must not reoffer a driver still assigned elsewhere. Establish valid history and fencing before replacement authority acts. Test recovery with active trips, since an empty-system restart cannot reveal conflicting ownership.

22.7.17 · Observability design

Track location age, stale-update rejection, cell skew, candidate counts, claim conflicts, offer expiry, matching outcomes, and time in each dispatch state. Observe map-provider errors separately.

Correlate stable trip-creation and acceptance identities. Alert on unresolved assignments and invariant violations as well as HTTP failures. Simulated drivers can cross cells, disconnect, and accept near expiry while riders compete, testing fresh search and exclusive outcomes together.

22.7.18 · Infrastructure cost considerations

Include ingestion, recent-location state, map and ETA calls, assignments, offer attempts, history, transfer, and operations. Route calls for every update could dominate cost despite efficient position storage.

Model calls per candidate and dispatch, then compare pruning and batching using explicit fictional rates or verified terms. Keep the accuracy and latency effects of cheaper ranking visible. The model makes no claim about a real company's internal architecture or bill.

22.7.19 · Alternative architectures and trade-offs

Straight-line distance is inexpensive and may ignore road travel. Route-based ETA can improve ranking while adding delay and a provider dependency. Sequential offers reduce simultaneous acceptance opportunities; multiple offers require a controlled single winner.

Fine spatial cells can improve locality and add handoffs and boundary queries. Compare fairness, rider wait, candidate quality, and machine cost. Each choice must preserve exclusive assignment.

22.7.20 · Architecture evolution from small scale to large scale

Start with a correct regional claim and basic proximity search. Add streamed positions and spatial indexes when lookup work requires them, then distribute matching with bounded ETA evaluation.

Split geographic ownership after measuring hot areas and defining a safe transfer procedure. Regional expansion adds a failure and partition policy for active trips. Preserve request identities and claims throughout: replicating matcher processes does not replicate the authority on which they depend.

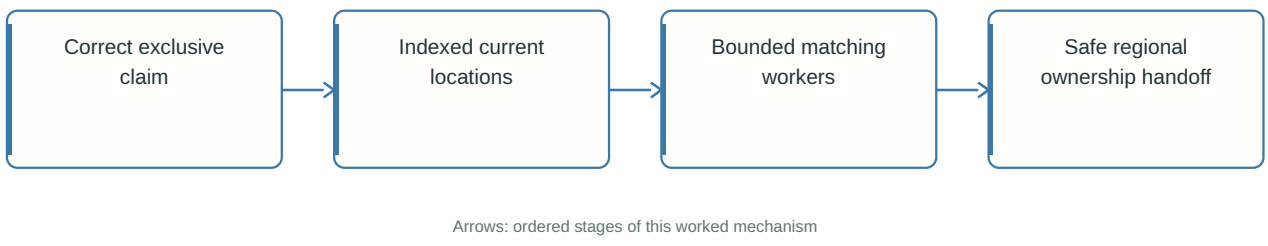


FIG. 22.7.4 **How the design can grow**

Partition location work without creating conflicting assignment owners.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.7.21 · Design review identifying weaknesses and improvements

Review what nearby, recently observed, available, offered, and assigned each mean. A spatial index answers proximity and age questions, but cannot alone establish current ownership.

Test competing riders, out-of-order positions, acceptance after expiry, paused workers, and handoff. One claim should win, and losers should receive meaningful state. Confirm that search projections never become independent writers of authoritative availability. A fast matching demonstration can otherwise conceal the decisive race.

22.7.22 · A short implementation exercise

Build an optimistic driver claim with an expected version and offer expiry. Start two requests from the same version. Verify that one local claim wins and the other conflicts.

Deliver an older position and a delayed acceptance after the winner commits. Neither may overwrite the active assignment. Explain the boundary demonstrated: a local transaction establishes one local owner, while external trip or payment effects still require their own contracts and recovery evidence.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
def accept_offer(offer, driver, now):
    with transaction():
        current = lock_offer(offer.id)
        require(current.pending and now < current.expires_at)
        won = claim_driver_if_available(driver, current.expected_version)
        if not won: raise AssignmentConflict
        bind_trip_and_offer(offer.trip, driver, offer.id)
    return current_trip_state(offer.trip)
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. PostGIS: ST_DWithin ↗

   Official implementation documentation · accessed 21 Oct 2026. Coordinate-system-aware proximity predicate, not exclusive dispatch assignment.

   Access reviewed · 21 October 2026

2. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

3. Saga distributed transactions pattern — Microsoft ↗

   Official engineering guidance. Local transactions and compensation; compensation is not always a perfect inverse.

   Access reviewed · 21 October 2026

Online Ticket Booking

Prerequisites: API Design and Communication Contracts, Database Internals, Transactions, and Concurrency Control, Message Queues, Events, and Distributed Workflows, Security Architecture and Privacy

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A buyer holds a seat and starts payment. The hold expires, another buyer reserves the seat, and the first payment finally succeeds. A paid flag alone cannot tell us which ticket may safely be issued.

We will trace seat ownership, versioned holds, and provider evidence through that sequence. The same rules must also handle repeated requests and delayed expiry tasks without selling one seat twice.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.8.1 · Functional requirements

A buyer browses an event, holds selected seats, pays, and receives tickets. Holds last a defined period; cancellation and refunds follow explicit policies.

The displayed seat map is advisory. The authoritative reservation decides who owns the opportunity to buy. Retries, expiry tasks, and payment callbacks can overlap, but must never confirm one seat for two buyers. Establish that invariant before improving the map or adding a waiting room.

22.8.2 · Nonfunctional requirements

Prevent duplicate confirmed ownership and retain accepted payment and ticket records under the selected failure contract. Pending holds must resolve through supported confirmation, expiry, or recovery.

Set launch admission and waiting objectives, including fairness. Fixed inventory cannot satisfy every simultaneous request. Queue or reject excess demand clearly. Distinguish an admitted transaction's delay from the buyer's total time waiting for admission and payment.

22.8.3 · Explicit assumptions and workload numbers

Assume 50,000 seats and a short peak of 100,000 reservation attempts/s. At 200 raw bytes each, seat records occupy about 10 MB before indexes.

The small inventory size hides heavy contention: many callers want the same ownership changes. Assume two-minute holds and a bounded seat count per user. Estimate retries, conflicts, and expiry work separately. This launch scenario is a teaching workload rather than a database benchmark.

22.8.4 · Naive initial architecture

Start with a transactional seat table. Change available to reserved conditionally and insert the reservation in the same transaction. Check whether the claim actually affected the seat.

With a naive read and unconditional write, two buyers can both read available and both report success. The conditional transition makes one winner visible to the other. Once this boundary is correct, address flash demand, payment uncertainty, and expiry races. A faster interface alone would only accelerate the incorrect history.

22.8.5 · Core API contracts

POST /reservations includes event, seat identities, and idempotency key. The returned resource gives status and expiry. A payment operation belongs to that reservation; authorised ticket reads expose committed issuance.

Bind repeat keys to trusted actor and content so a retry returns the earlier result. Distinguish provider authorisation, capture, decline, and uncertainty under its contract. An expired hold and an uncertain payment require different next actions and should remain distinct states.

22.8.6 · Data model and schema

A seat stores event and seat identity, state, reservation owner, and version. A reservation stores owner, state, expiry, version, and its seat relationships. Payments and tickets have separate identities.

Enforce one relevant confirmed active ticket per seat. Expiry and confirmation check current reservation identity and state/version. A delayed expiry task for an old hold must not release a seat acquired by a newer reservation. Use actual constraints or atomic checks to preserve these relationships.

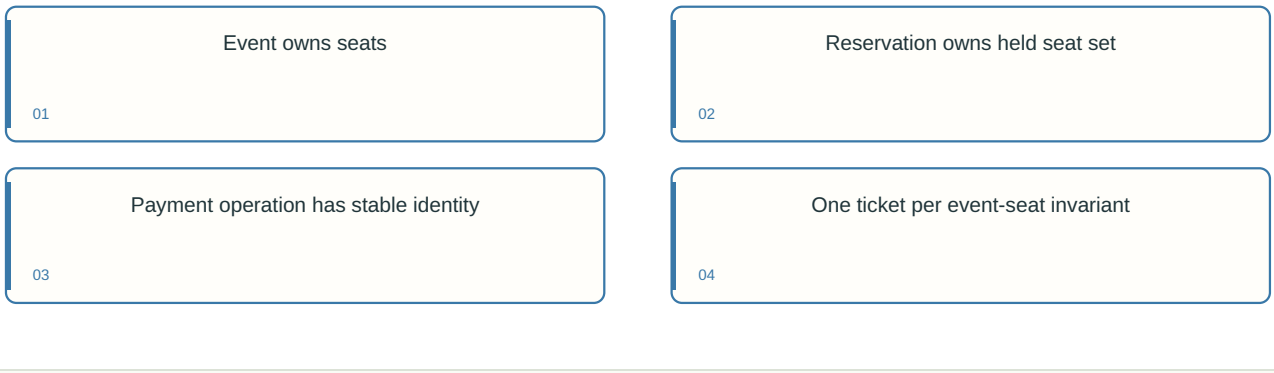


FIG. 22.8.1 **Data ownership and relationships**

Seat, hold, payment, and ticket records must preserve their defined ownership relationships.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.8.7 · Low-level component organization

SeatRepository owns conditional claims or locks. ReservationService applies selection and hold rules. PaymentWorkflow coordinates external effects, TicketIssuer commits valid issuance, and ExpiryWorker conditionally releases old holds.

Represent prices and refunds with explicit amounts and units. Put ownership checks at the shared authoritative boundary, where callbacks and workers also pass them. Return enough state to distinguish conflict, expiry, confirmed success, and provider uncertainty.

22.8.8 · High-level system architecture

Put bounded admission or a waiting room before the ownership store. Keep durable reservations, a payment adapter, workflow or outbox progress, and ticket reads. Cache event descriptions and advisory seat maps separately.

Partitioning by event can keep its transactions local, but a popular event can remain hot. Admission and contention controls still matter. The cache suggests what may be available; the transaction establishes what this buyer reserved. They must not become competing allocation authorities.

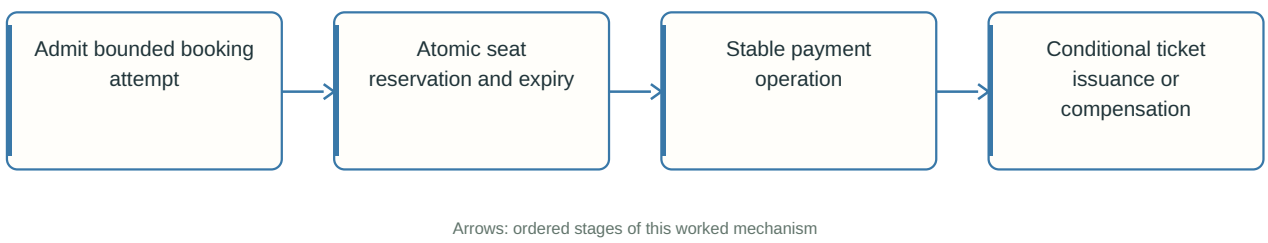


FIG. 22.8.2 **Follow the main operation**

A hold and payment workflow lead to a version-checked confirmation or supported recovery.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.8.9 · Data and control flows

Atomically claim seats and create the hold, then return its expiry after commit. Submit payment with a stable provider identity. Confirmation verifies a still-valid transition and commits the required reservation, seat, and ticket changes.

If payment succeeds after expiry, apply the defined recovery: safely reacquire if current ownership permits, or refund or follow another supported resolution. Never issue against another buyer's seat. Expiry checks current identity and version before release so delayed work cannot undo a later valid state.

22.8.10 · Capacity estimates

Suppose launch admission permits 2,000 attempts/s from an offered 100,000/s. The rest wait or are rejected under policy. The 2,000 limit is a configured input to validate, not demonstrated capacity.

Seat inventory bounds concurrent holds, but does not bound attempts or retries during a two-minute hold. Estimate writes per attempt, duplicate-key checks, conflicts, and expiry scans. Web concurrency may grow rapidly while the same small seat set remains contested.

22.8.11 · End-to-end latency estimates

Suppose the admitted path takes 20 ms at the edge, 30 ms acquiring transaction resources, 40 ms for conditional seat and reservation work, and 20 ms returning the reply. The budget totals 110 ms.

Contention may exceed it, so bound waiting and stop work that is no longer useful. Measure waiting-room time separately. Payment has another expiry-sensitive timeline. Quick admitted transactions do not establish a quick booking journey for buyers still queued outside allocation.

22.8.12 · Failure scenarios

Only one buyer may win after both read available. If reservation commit succeeds but its reply disappears, the repeat key recovers the hold. Repeated payment evidence must not issue another ticket.

Test late payment and stale expiry after confirmation or reassignment. Conditional transitions and reconciliation handle their overlapping histories. If ticket email fails after issuance commits, the ticket still exists and needs another delivery route. A communication failure must not undo accepted ownership.

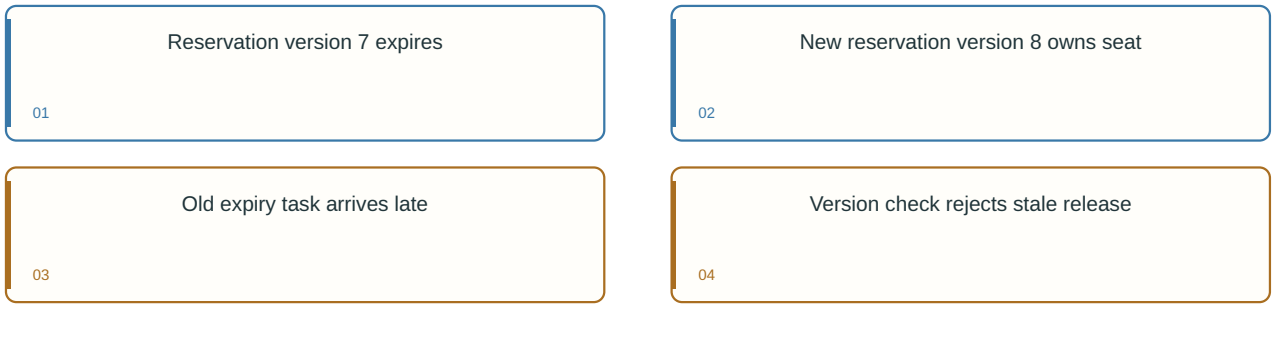


FIG. 22.8.3 **Follow the failure and its correction**

An old expiry or late payment must not overwrite a newer seat owner.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.8.13 · Scalability improvements

Cache advisory data, bound admission, and keep the correct seat transaction short. Partition independent events when measured work justifies it.

Multi-seat claims need an explicit all-or-nothing rule and consistent lock order. Splitting one event across shards complicates group booking, and extra shards do not divide contention for one seat. Scale

surrounding work while preserving the atomic owner. Adopt a more complex allocation protocol only for a demonstrated benefit.

22.8.14 · Consistency requirements

Seat ownership and ticket issuance need an atomic authoritative rule. Displays may lag if buyers understand their advisory status. Payment and email remain external effects with stable identities and tracked progress.

A serializable local transaction can protect local concurrency; it does not include a provider automatically. Retry serialization failures through bounded whole-transaction retries with safe external-effect placement. State which records change together and which outcomes are reconciled later.

22.8.15 · Security model

Authorise holds, payments, tickets, transfers, and refunds. Calculate prices server-side, verify provider callbacks, protect credentials, and limit automated abuse under a defined identity policy.

Scope idempotency keys so one actor cannot recover another's result. Give ticket capabilities appropriate scope and lifetime, and audit important changes. A waiting room controls capacity; identity, permissions, and fairness require their own enforcement and tests.

22.8.16 · Deployment approach

Keep states and expiry meanings compatible during rollouts. Durable expiry survives worker replacement, and database promotion must respect the acknowledged-data rule.

After recovery, reconcile provider evidence before declaring bookings resolved. If write authority is uncertain, pausing allocation may preserve exclusivity while descriptions remain readable. Test with live holds and callbacks. An old restored snapshot cannot establish permission to sell seats confirmed after that snapshot.

22.8.17 · Observability design

Track admitted attempts, claim conflicts, transaction waits, hold age, expiry backlog, payment uncertainty, late successes, and ticket uniqueness. Follow each logical booking through retries and callbacks.

Check invariants across seats, holds, and tickets and compare them with payment evidence. Empty error logs cannot prove that the relationships agree. Measure user-visible waiting and completion separately from raw API latency.

22.8.18 · Infrastructure cost considerations

Include burst admission, transactions, abandoned holds, payment calls, ticket delivery, retention, telemetry, and recovery capacity. Use the launch's duration rather than applying its peak for the entire month.

More web workers cannot remove contention on one seat. Compare designs under equal no-double-ticket and recovery requirements using explicit fictional rates or verified prices. Include retry and expiry work even for buyers who abandon purchase.

22.8.19 · Alternative architectures and trade-offs

Pessimistic locking serialises early and may cause waiting or deadlocks. Optimistic conditional claims produce conflicts that callers must handle clearly. One event owner simplifies order while imposing a capacity boundary.

Distributed allocation can reduce some central work and complicate group-seat atomicity or fairness. First explain why one buyer wins under the chosen mechanism, then integrate that winner with payment and expiry. Acquiring a lock alone does not finish the booking workflow.

22.8.20 · Architecture evolution from small scale to large scale

First implement the seat transaction and repeat-key result. Add durable expiry and payment reconciliation, then launch admission and advisory caching.

Partition independent events after measurement. Change one-event allocation only when demonstrated limits justify the added protocol. Recheck no-double-ticket behaviour at every stage, including old expiry and retry paths. Additional throughput must preserve the earlier safe ownership decisions.

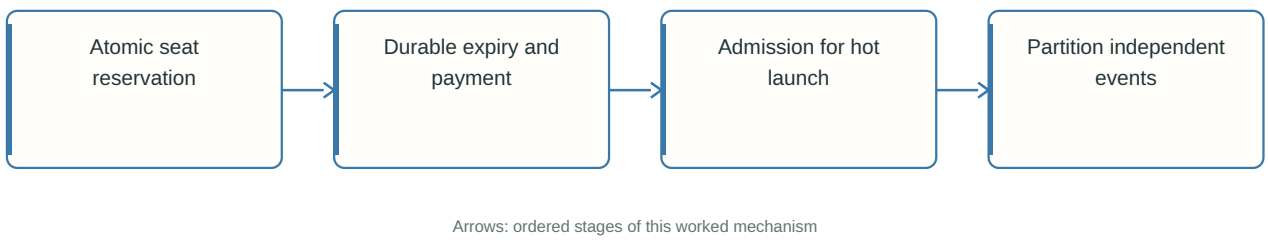


FIG. 22.8.4 **How the design can grow**

Add admission and partition independent events while protecting the same seat invariant.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.8.21 · Design review identifying weaknesses and improvements

Check that expiry and confirmation refer to the same current reservation identity and state. Review successful payment after the hold is gone, and verify that displayed availability never grants ownership.

Test competing buyers, repeated callbacks, commit-then-crash, stale expiry after reassignment, and late payment. A correct happy-path seat update covers only one boundary. The review must explain money and tickets still unresolved after a request ends.

22.8.22 · A short implementation exercise

Reproduce the last-seat race using two connections or an explicit interleaving. Let both buyers read available, then demonstrate the incorrect unconditional writes. Replace them with a conditional claim and reservation transaction, checking affected rows.

Add versioned expiry. Confirm or reassign the seat, then deliver the old expiry task. It must leave the current owner's seat unchanged. Identify the checked identity and version that make this safe, independently of usual task timing.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
with transaction():
    insert_reservation(reservation_id, owner, expires_at)
    updated = reserve_if_available(event, seat, reservation_id)
    if updated != 1: raise SeatUnavailable
# Expiry later releases only the same current reservation version.
# Provider payment is coordinated outside this local transaction.
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

2. Saga distributed transactions pattern — Microsoft ↗

   Official engineering guidance. Local transactions and compensation; compensation is not always a perfect inverse.

   Access reviewed · 21 October 2026

3. RFC 9110: HTTP Semantics, June 2022 ↗

   Original standard. HTTP semantics; application commit policy is separate.

   Access reviewed · 21 October 2026

Payment System

Prerequisites: API Design and Communication Contracts, Database Internals, Transactions, and Concurrency Control, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery, Security Architecture and Privacy

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A provider charges successfully, but the payment service stops before recording the reply. The client sees a timeout and tries again. How can the service recover the existing charge instead of requesting another logical payment?

We will give the operation a durable identity, preserve uncertain outcomes, and reconcile provider evidence. We will also connect that evidence to balanced local postings, because a provider result, an order status, and an accounting record describe different parts of the journey.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.9.1 · Functional requirements

A client creates a payment intent. The service requests authorisation or capture, reports supported outcomes, records balanced accounting entries, and reconciles provider evidence. Refunds have separate operation identities and progress.

Represent money with currency, integer minor units, and an explicit rounding policy. An order's status, a provider operation, and the ledger are related records, but cannot be replaced by one paid flag. The design must explain their sequence, uncertainty, and accounting consequences.

22.9.2 · Nonfunctional requirements

Preserve financial invariants and auditable state. Prevent duplicate relevant provider effects where its contract permits that guarantee, and resolve uncertainty from evidence rather than inventing success or decline.

Measure intent acceptance separately from completion. Under the chosen accounting model, each settled amount needs unique balanced postings and authorised corrections. During a provider outage, a durable pending state can be the correct result. Fast unsupported finality would leave an outcome the system cannot explain later.

22.9.3 · Explicit assumptions and workload numbers

Assume one million intents/day, about 11.6/s, and an illustrative 200/s peak. Suppose each initially produces six operational or audit records of 1 kB. That is six million kB, or 6 GB/day raw.

Ledger details, callbacks, indexes, copies, and retention add storage. Provider concurrency and reconciliation traffic need separate estimates, especially during long waits or recovery. These synthetic assumptions make no claim about actual provider fees or institutional architecture.

22.9.4 · Naive initial architecture

A simple service calls a provider, sets an order flag, and replies. If charging succeeds and the process dies before saving the result, a retry may charge again.

Add a durable operation identity and explicit uncertainty before scaling. Local transactions protect related local changes. Provider keys, status queries, and reconciliation address the gap outside that transaction. Each protects a different part of the history. More instances cannot recover an ambiguous effect for which no stable identity was kept.

22.9.5 · Core API contracts

POST /payment-intents accepts amount_minor, currency, business reference, and idempotency key. Status exposes supported states such as pending, authorised, captured, declined, or uncertain. A refund endpoint creates a separately keyed refund.

Bind keys to actor and financial payload; changed content under an existing key conflicts. Verify provider webhooks instead of trusting client approval fields. Define how clients wait or query uncertainty so a missing reply does not automatically start another financial operation.

22.9.6 · Data model and schema

payment_intent and operation records retain actor, key, fingerprint, progress, and result. Provider attempts keep stable keys, references, and outcomes. A verified-event inbox deduplicates evidence.

Ledger transactions own entries with account, currency, and signed amount. Within each required currency and accounting scope, the signed entries sum to zero: the amount debited is matched by the amount credited. Enforce operation linkage and correction identity. An authorised correction adds a new accounting event rather than rewriting past evidence, keeping the financial history reviewable.

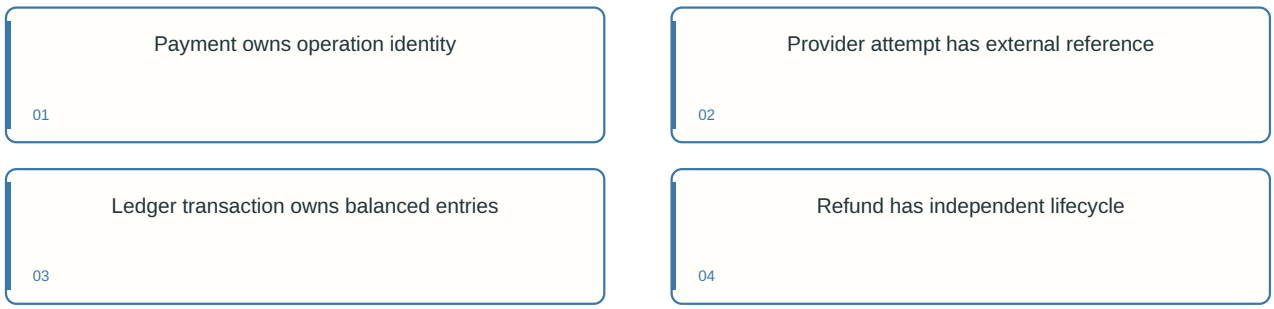


FIG. 22.9.1 **Data ownership and relationships**

Operation identity links provider evidence to payment state and balanced accounting entries.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.9.7 · Low-level component organization

PaymentService controls transitions; Money and policy values validate amounts. ProviderAdapter maps explicit outcomes, OperationRepository recognises duplicates, and LedgerService validates and commits postings. Reconciler compares provider evidence; RefundWorkflow tracks refunds.

These roles can begin inside one application. Map a timeout to uncertainty, not definite decline. Hide protocol details at the adapter while retaining evidence needed for recovery. A useful boundary clarifies meaning and ownership without requiring a network service for every responsibility.

22.9.8 · High-level system architecture

Use an authenticated API, durable intent and operation storage, bounded provider workers, verified callbacks, a ledger boundary, and scheduled reconciliation. Commit any required downstream publication intent with the relevant local change.

Reserve reconciliation capacity so it can repair uncertainty during incidents. Regional serving must preserve provider-operation and ledger authority. Draw provider calls outside the local transaction boundary: this makes the external gap and its recovery path visible.

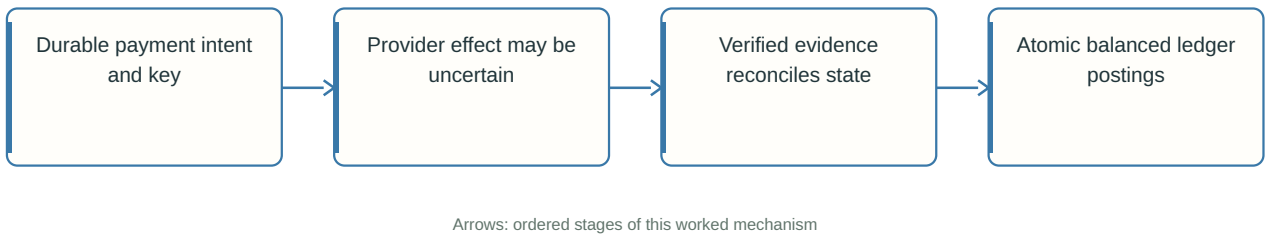


FIG. 22.9.2 **Follow the main operation**

Persist intent, obtain or reconcile provider evidence, and commit the required local result.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.9.9 · Data and control flows

Commit intent and logical operation identity, then submit its stable provider key. Record supported evidence or uncertainty. Capture or settlement evidence drives an idempotent transition that creates the required balanced postings once.

Callbacks can precede the synchronous reply, so valid transitions must handle either order without regressing. Reconciliation queries the stable reference and repairs discrepancies. A local pending row is not permission to resend blindly. Trace each evidence and commit boundary to identify what a crash can leave unresolved.

22.9.10 · Capacity estimates

At a stable 200 provider calls/s and an assumed average 500 ms wait, Little's Law gives 200 × 0.5 = 100 in-flight calls within that boundary.

This estimates average occupancy, not a safe pool size. Account for variance, deadlines, provider limits, and failures. Initial records add 6 GB/day raw. Callbacks and reconciliation continue after request traffic subsides, so they need their own capacity instead of only the daily-average allowance.

22.9.11 · End-to-end latency estimates

Suppose ingress takes 20 ms, intent commit 20 ms, provider work 500 ms, and final local update 20 ms. The successful synchronous budget totals 560 ms.

Returning durable pending after about 40 ms is a different contract with later completion. Neither sum establishes p99. A timeout calls for status recovery or reconciliation; it does not establish that no payment occurred. Measure acceptance and final resolution separately.

22.9.12 · Failure scenarios

Test lost replies after local commit, provider execution followed by process loss, reordered or repeated callbacks, uncertain refunds, and posting-commit failure.

Keep unknown outcomes and repeat only through a supported safe identity or established evidence. If the ledger is unavailable, setting paid bypasses a required boundary rather than repairing it. Recovery must account for provider state and required postings before declaring the operation resolved. A restarted healthy API does not establish agreement between those histories.

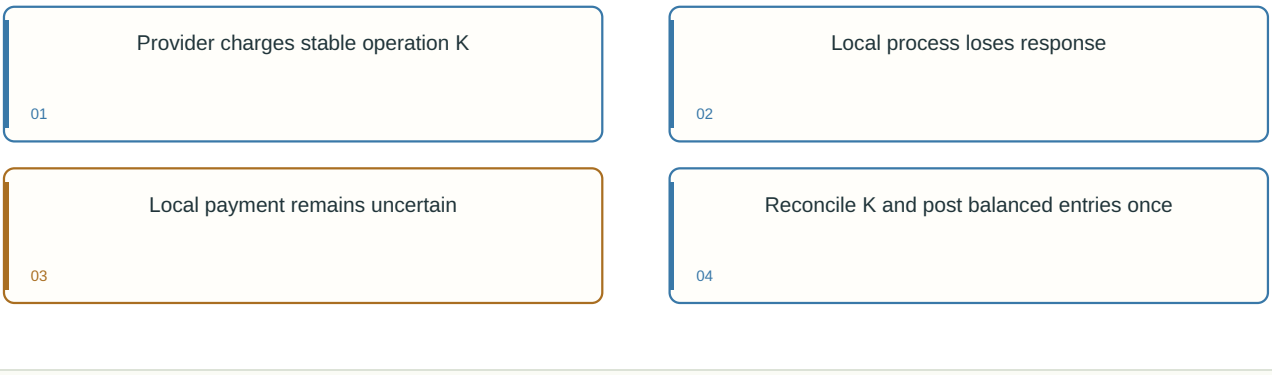


FIG. 22.9.3 **Follow the failure and its correction**

A lost provider reply leaves uncertainty rather than proof that no charge occurred.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.9.13 · Scalability improvements

Scale independent acceptance and workers while bounding provider concurrency and preserving per-payment and relevant per-account invariants. Isolate enough reconciliation capacity to repair uncertain operations during trouble.

Read projections can expose status under a stated freshness rule. Ledger partitioning requires an explicit balancing and transaction scope; a hot account may remain concentrated. Partition only for a demonstrated bottleneck, since additional ownership boundaries also add recovery work.

22.9.14 · Consistency requirements

Local deduplication, payment transitions, and required postings need suitable atomic boundaries. The provider remains outside them and requires its own identity and recovery contract.

Uniqueness and transactions can establish one local posting set without proving one provider execution. Refunds have independent state and may not reverse timing, fees, or downstream consequences perfectly. Define the evidence for each accepted result and retain uncertainty until a valid final transition is supported.

22.9.15 · Security model

Authorise payments and refunds, validate amounts server-side, verify callbacks, and protect credentials. Avoid retaining unnecessary sensitive data through an appropriately reviewed provider integration or tokenisation arrangement.

Restrict and audit privileged corrections. Operators need explicit authority and must not silently rewrite accounting history. Scope effects and queries to tenant and account. Preserve useful identity and outcomes in logs without secrets. These controls do not by themselves assert any financial or legal compliance status.

22.9.16 · Deployment approach

Evolve states and event formats compatibly, retain operation identities during releases and recovery, and control provider configuration and key changes.

An old restore may forget keys for effects already executed externally. Establish replay protection and reconcile before resubmission. Test delayed callbacks from earlier releases and recovery after outages. Restored pending rows must be compared with provider evidence before being treated as untouched work.

22.9.17 · Observability design

Separate acceptance and completion delay. Track age of pending and uncertain work, attempts, duplicate events, callback verification, imbalances, unmatched settlements, refunds, and reconciliation outcomes.

Alert on invariant violations and old unresolved money movement even when HTTP success is high. Evidence must distinguish attempted, provider-accepted, and locally posted states. Stable identities let an

investigator reconstruct those stages without treating one response or log line as proof of the entire journey.

22.9.18 · Infrastructure cost considerations

Include provider attempts, secure storage, ledger work, retained audit evidence, telemetry, reconciliation, and operations. Use symbolic fees or labelled hypothetical inputs unless prices are verified.

Compare cost per reconciled successful business payment as well as per HTTP attempt. Retries can increase attempts while useful outcomes decline. Designs must preserve the same financial invariants; an inexpensive system that cannot explain uncertain charges fails a required part of the service.

22.9.19 · Alternative architectures and trade-offs

Synchronous completion may suit clients expecting immediate results and exposes provider waits. Asynchronous intents make pending progress explicit while adding workflow state.

A local transactional application simplifies related invariants. Separating provider and ledger services may improve ownership or isolation and adds coordination. Append-only accounting history can coexist with mutable status projections. Compare boundaries by the work and recovery they clarify instead of forcing one paid record to serve every purpose.

22.9.20 · Architecture evolution from small scale to large scale

Start with explicit states, currency-aware integer amounts, local transactions, and stable provider identity. Add durable callbacks and reconciliation before multiplying traffic paths.

Then introduce projections and bounded workers for measured needs. Partition only where a demonstrated limit justifies a preserved invariant boundary. Earlier payments must remain explainable through evidence and postings after topology changes. Increased capacity must retain that historical meaning.

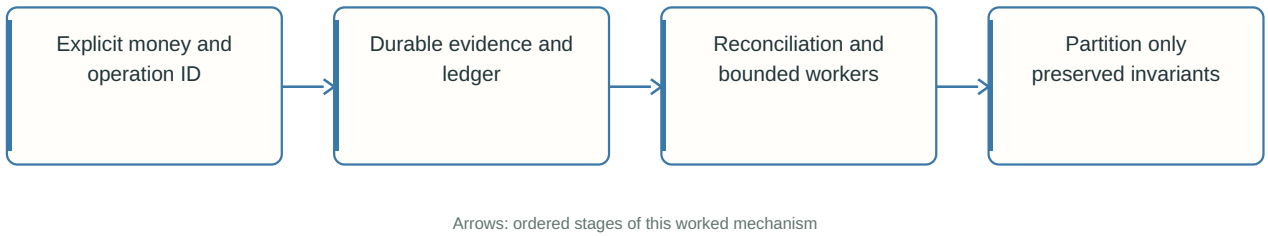


FIG. 22.9.4 **How the design can grow**

Scale useful independent work while keeping each prior operation and posting explainable.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.9.21 · Design review identifying weaknesses and improvements

Ask what evidence establishes financial success, who commits the postings, and how recovery handles an older deduplication snapshot beside newer external effects.

Test charge-then-crash, callback-before-reply, duplicate evidence, partial refunds, and earlier-state restore. Check for one valid local posting set and explicit unresolved state where evidence is missing. A timeout cannot be turned into decline merely to make the request path appear complete.

22.9.22 · A short implementation exercise

Build a synthetic provider that remembers stable keys and a local ledger accepting only balanced entries. Stop after provider execution and before recording the local result.

Reconcile by key and verify that one logical operation produces one balanced posting set. Replay its event and reject changed financial content under the old key. Explain that the test demonstrates this synthetic provider contract and local boundary, rather than a guarantee about arbitrary real providers.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
def apply_settlement(event):
    with transaction():
        if inbox_contains(event.id): return prior_outcome(event.id)
        payment = lock_payment(event.payment_id)
        validate_transition_and_amount(payment, event)
        entries = [debit_entry(event), credit_entry(event)]
        require_balanced_by_currency(entries)
        insert_unique_posting_set(event.operation_id, entries)
        update_payment_and_insert_inbox(payment, event)
# Provider execution is reconciled separately from this transaction.
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. PostgreSQL 18: Transaction Isolation ↗

   Official versioned documentation. Specific PostgreSQL isolation behavior; no all-database equivalence claim.

   Access reviewed · 21 October 2026

2. Transactional Outbox — Chris Richardson ↗

   Pattern author's explanation. Publication intent and duplicate delivery.

   Access reviewed · 21 October 2026

3. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

Distributed Job Scheduler

Prerequisites: Runtime, Processes, Threads, and Concurrency, Replication, Leader Election, and Consensus, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery

4 original figures

Blue: normal flow Green: healthy Amber: caution Red: failure Purple: control / shared infrastructure

Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A worker pauses while running a scheduled job. Its lease expires, so a replacement begins. When the first worker resumes, both may be capable of doing work even though only one currently owns the attempt.

We will give the schedule, occurrence, attempt, and effect separate identities. This lets recovery recognise what it is resuming and shows why rejecting an old local completion is only one part of protecting an external effect.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.10.1 · Functional requirements

A user creates a schedule, the service identifies due occurrences, and workers execute jobs. Users inspect history, configure retries, and cancel supported future work.

Define fixed times versus intervals, timezone handling, missed runs, and overlap. One schedule can produce many occurrences; retrying an occurrence must not invent another logical run for the same due time. Establish this timing meaning before choosing a timer or queue.

22.10.2 · Nonfunctional requirements

Keep accepted schedules and required runs recoverable under supported failures. Bound scheduling delay and prevent superseded attempts from overwriting newer results. Execution may start repeatedly around failure, so effects need suitable identity or enforced fencing.

Define cancellation's boundary. Stopping future dispatch, cancelling pending work, and attempting to interrupt a running effect are different promises. A local cancellation flag does not establish that a remote effect was undone. Run status must preserve this distinction.

22.10.3 · Explicit assumptions and workload numbers

Assume one million schedules produce 100,000 occurrences/hour. Dividing by 3,600 gives about 27.8 runs/s. At an initial 2 kB of state per run, raw growth is 200 MB/hour.

With ten-second average execution at a stable rate, average active work is 27.8 × 10, or about 278 runs. Top-of-hour bursts, long jobs, retries, and retention add requirements. Saved-schedule count alone does not describe the concentration of due work.

22.10.4 · Naive initial architecture

One process scans due schedules and launches local functions. If it saves a run and advances the schedule in separate commits, a crash between them can repeat or miss an occurrence.

Create a stable occurrence record and advance due state in one transaction. A leader can simplify scanning, but leadership alone does not make worker effects occur once. First establish durable identity and atomic progress. Adding another process to the unsafe loop merely creates more opportunities for the same failure.

22.10.5 · Core API contracts

POST /schedules accepts timing, timezone, payload reference, retry rules, and overlap policy. Cancellation changes documented future behaviour. GET /runs/{id} reports attempts and outcome. A run-once operation has its own idempotency identity.

Bound timing-expression and payload complexity. Return durable acceptance separately from execution completion. If an external effect is unresolved, report uncertainty. A worker's missing reply does not prove that its effect never happened.

22.10.6 · Data model and schema

A schedule stores id, version, next_due, timing policy, and state. A run binds unique (schedule_id,occurrence) to input version and progress. Attempts contain epoch, lease deadline, worker, and result. Retain original due time separately from dispatch time.

An epoch numbers successive owners so completion can reject an old attempt. Another attempt still belongs to the same logical run. Outbox and effect records retain durable dispatch and external identities where needed.

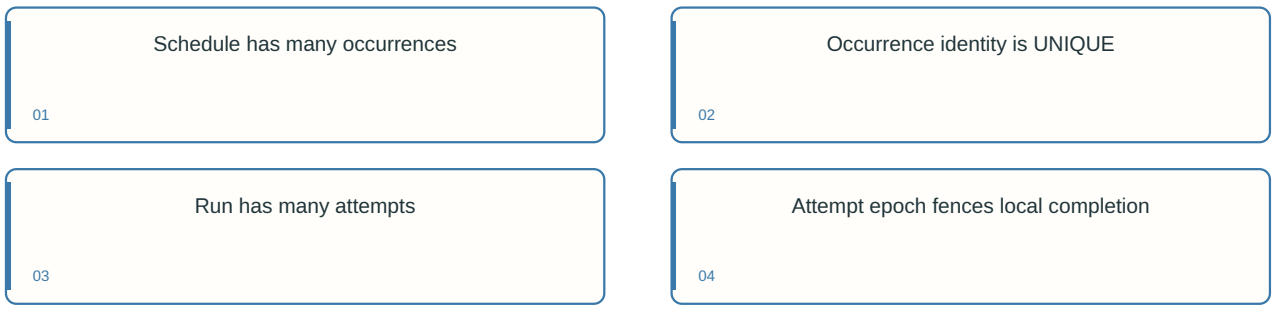


FIG. 22.10.1 **Data ownership and relationships**

One scheduled occurrence can have several attempts, while retaining one logical run identity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.10.7 · Low-level component organization

ScheduleParser and TimePolicy calculate occurrences. DueScanner locates candidates. RunRepository saves the run and next_due atomically. Dispatcher routes work, LeaseManager grants attempts, and WorkerAdapter executes them. Reconciler finds stuck or uncertain outcomes.

The scheduler decides when work is due; a worker acts under a current bounded claim. Each effect adapter needs explicit repetition or fencing semantics. This separation lets tests examine time calculations without accidentally performing an irreversible external operation.

22.10.8 · High-level system architecture

Keep durable schedules, coordinated scanners or partition owners, dispatch records, bounded workers, attempt state, and reconciliation. Store large versioned payloads separately rather than copying them into each retry message.

Timing control and execution have different capacity and failure paths. A control-plane outage may delay new dispatch while existing jobs continue. Worker loss must leave a recoverable run. A queue attached to a cron loop does not supply these behaviours without durable state and ownership rules.

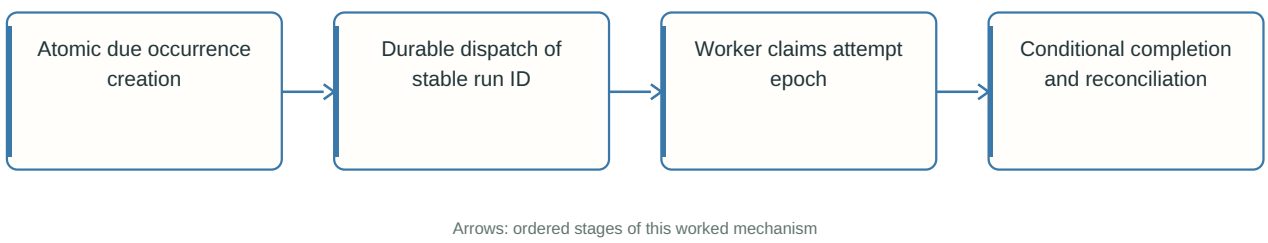


FIG. 22.10.2 **Follow the main operation**

Create due work durably before a current worker attempts its effect.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.10.9 · Data and control flows

A scanner claims due state, creates the unique occurrence and dispatch intent, advances next_due, and commits. A worker then claims an attempt epoch and executes under the stable run identity. It renews its lease where supported and completes only while current.

After expiry, a replacement may take over. The old worker may still resume, so effect-side identity or fencing must limit harmful stale actions. Reconcile runs without an acceptable terminal state. Starting a callback is an intermediate event, rather than the end of the durable history.

22.10.10 · Capacity estimates

At about 27.8 runs/s with ten-second average execution, average concurrency is roughly 278. If each worker sustains four independent I/O jobs, 278 / 4 is about 69.5: roughly seventy workers before headroom and other limits.

CPU-heavy jobs require measured compute capacity instead. Estimate burst concentration, retries, and outage catch-up. A steady-state average cannot establish that the same worker count meets start-time objectives during a large due-time burst.

22.10.11 · End-to-end latency estimates

Scheduling delay runs from due time to dispatch or start. Runtime begins after execution starts. As an illustrative target, require 95% of eligible runs to start within five seconds under supported load.

Scan intervals, claims, queues, cold starts, and concurrency limits consume that budget. A ten-minute task can meet a five-second start target. Give completion deadlines a separate rule and measure both when required, rather than combining all durations into an ambiguous job-latency figure.

22.10.12 · Failure scenarios

Test interruption between occurrence creation and schedule advancement; the transaction must leave both committed or neither. Then pause a worker past its lease so it overlaps a replacement. The current-epoch check must reject the resumed worker's local completion.

This does not undo external effects already accepted. Those require effect-side protection and reconciliation. Also define clock changes, timezone transitions, malformed input, and poison-job handling. Correct due-time arithmetic alone does not establish safe repeated execution.

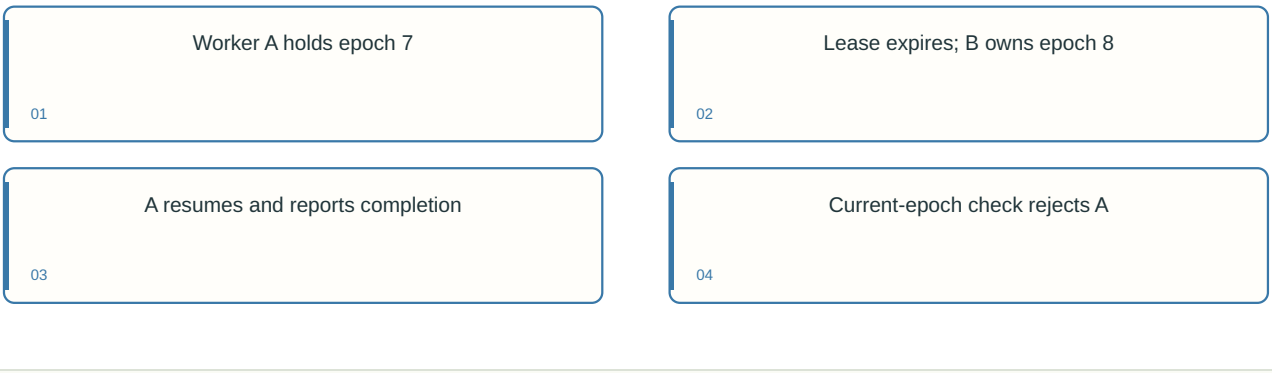


FIG. 22.10.3 **Follow the failure and its correction**

A replaced worker can resume, so its stale completion must be rejected at the protected boundary.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.10.13 · Scalability improvements

Index due time, claim bounded batches, and partition schedules with stable ownership when scanning is limiting. Scale workers according to measured CPU, memory, and I/O demands.

Separate long, priority, and CPU-heavy work where necessary. Bound catch-up so overdue runs do not starve current critical work. Removing a global lock must preserve atomic occurrence creation through

another valid mechanism. Lower contention is useful only if it retains the logical-run invariant.

22.10.14 · Consistency requirements

Creating an occurrence and advancing the schedule require uniqueness and atomicity. Worker delivery can repeat. Current-epoch completion protects local progress, while effect identity or resource-enforced fencing protects the relevant external effect.

A lease is permission valid for an interval; it does not stop a paused process from resuming. It therefore cannot alone guarantee one execution. With fencing, the protected resource recognises current authority and rejects requests carrying stale epochs; the receiver must actually enforce that rule. Define missed-run and overlap policies so incidental timing does not decide which logical runs exist. Schedule, occurrence, attempt, and effect are distinct scopes.

22.10.15 · Security model

Authorise schedule ownership, payload references, run views, and cancellation. Support validated work types rather than arbitrary code or unrestricted URLs supplied through the API.

Bind each run to tenant and immutable input. Give workers only required capabilities and audit administration. A broad administrator credential in every worker makes one compromised job unnecessarily powerful. Follow permission from schedule through task to the actual resource used by execution.

22.10.16 · Deployment approach

Keep schedule and run formats compatible during rollout, including epoch and lease meanings. Drain new claims and save progress before shutdown. During recovery, compare queues with authoritative runs.

An older restored schedule table may lag effects already executed externally. Recover stable occurrence identity and evidence before replaying. Test control-plane loss separately from worker loss. Restarting scans must restore the meaning of due work and prior outcomes, rather than merely resume timer activity.

22.10.17 · Observability design

Measure due-to-dispatch and due-to-start delay, oldest eligible work, repeated attempts, lease expiry, stale-completion rejection, retries, duration, and tenant fairness. Verify that one occurrence maps to one logical run.

CPU may remain low when a broken scanner creates no work. Monitor generation and progress directly, alerting on accepted jobs that stop advancing. Detailed histories must distinguish runs from attempts so operational repetition is not confused with new scheduled demand.

22.10.18 · Infrastructure cost considerations

Include scanning, indexes, history, dispatch, execution, spare capacity, external calls, and recovery. At 2 kB/run and 100,000 runs/hour, raw daily history is 200 MB/hour × 24 = 4.8 GB before attempts and indexes.

Compare scan frequency and batching against the start-delay requirement. Use explicit rates and include catch-up and idle headroom. A lower cost obtained by silently missing occurrences does not satisfy the same service contract.

22.10.19 · Alternative architectures and trade-offs

Polling an indexed due-time field is simple and recoverable. Timer wheels and delayed-message facilities may improve some timing workloads, provided they can be durably reconstructed. Central scanning simplifies authority; partitioned scanning adds coordination while dividing work.

Long-lived workers reduce startup cost. Managed invocation may fit bursts under different limits. Compare timing precision, duration, recovery, and effects. A fast timer data structure solves due lookup, leaving occurrence and attempt semantics to the rest of the design.

22.10.20 · Architecture evolution from small scale to large scale

Begin with deterministic time policy and atomic occurrence creation. Add durable dispatch and bounded execution. Introduce leases and epochs together with safe effect identity before claiming supported worker takeover.

Partition whichever scan or execution path measurements identify. Regional ownership needs transfer and partition rules for due work. Preserve identities throughout, so replacements do not reinterpret retries as new occurrences or lose the evidence needed to resolve older runs.

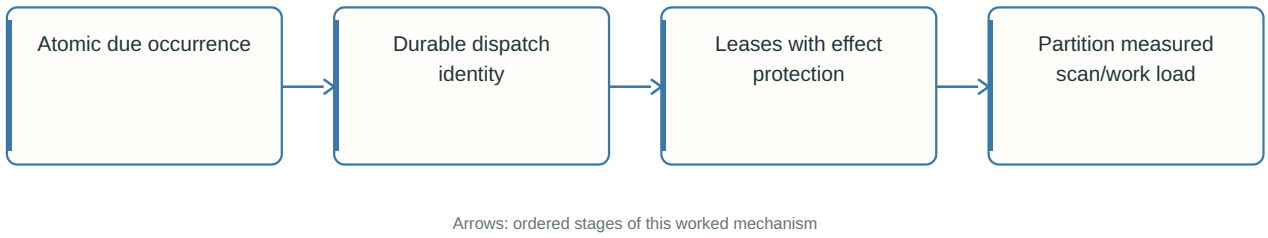


FIG. 22.10.4 **How the design can grow**

Separate scan capacity from execution while preserving occurrence and effect identities.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.10.21 · Design review identifying weaknesses and improvements

Review occurrence identity, next_due atomicity, stale workers, cancellation, and recovery from older state. Each can leave a different gap.

Run competing scanners at commit boundaries, pause a worker past expiry, cancel during execution, cross a timezone transition, and recover after a long outage. Check which runs should exist and which effects remain uncertain. One on-time callback demonstrates timing for one run, rather than the scheduler's full recovery contract.

22.10.22 · A short implementation exercise

Use an injected clock and transactional run table. Start two scanners for the same due occurrence and verify that they establish one logical run.

Pause an epoch-7 attempt, replace it with epoch 8, and submit the old completion. The local store must reject it. Identify the stable key or receiver-enforced fencing still needed for an external effect. The exercise demonstrates local progress protection without claiming that the job physically executes only once.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
with transaction():
    schedule = claim_due_schedule(now)
    run = insert_unique_run(schedule.id, schedule.next_due)
    insert_dispatch_outbox(run.id)
    advance_next_due(schedule)
# Worker completion compares the current attempt epoch atomically.
# A stale worker's external effects need resource-side protection.
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. Distributed Periodic Scheduling — Google SRE ↗

   Engineering book. Durable scheduling concerns; atlas schedule architecture is an original worked design.

   Access reviewed · 21 October 2026

2. In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

   Original research. Raft reference for Unit 15.

   Access reviewed · 21 October 2026

3. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

Collaborative Document Editor

Prerequisites: Networking Foundations for System Design, Distributed Systems Fundamentals, Replication, Leader Election, and Consensus, Message Queues, Events, and Distributed Workflows

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

Two collaborators insert text at the same place. One replica receives X and then Y; the other receives Y and then X. If each simply applies arrival order, their documents may differ.

We will define what an edit means and how its identity and context survive repeated delivery and reconnects. We will then examine the separate questions of durable acceptance, safe history cleanup, and permission to keep editing.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.11.1 · Functional requirements

Collaborators create a document, edit together, reconnect after disconnection, recover required history, and export or delete under access rules.

First choose the document model. Plain text, structured rich text, and embedded objects have different valid operations and conflicts. Define those operations before selecting their transport. Cursor positions and presence are temporary hints, separate from durable content and its history.

22.11.2 · Nonfunctional requirements

Preserve acknowledged operations and enforce a defined access-revocation policy. Online editing should respond locally and meet a supported synchronization-delay objective.

The protocol must converge under its assumptions, but equal final strings alone do not establish intended edits, Unicode correctness, or structural validity. A returning offline user may have lost access. Decide how unaccepted edits are rejected or presented, and distinguish them from accepted history. Optimistic local display is not a durable acknowledgment.

22.11.3 · Explicit assumptions and workload numbers

Assume 100,000 active documents, with 10,000 edited simultaneously by three users each. At two operations/s per user, ingestion is 10,000 × 3 × 2 = 60,000 operations/s.

At 200 retained bytes each, this is 12 MB/s, or 1.0368 TB/day if sustained throughout the day. Actual activity duration, batching, snapshots, metadata, fan-out, and retention change the estimate. Peak concurrency must not silently become a full-day assumption in a real storage plan.

22.11.4 · Naive initial architecture

Start with whole-document updates carrying an expected version. If two writers replace the same version, one succeeds and the other conflicts. This is a clear simple contract, but awkward for continuous collaboration.

An operation-based protocol handles smaller edits and their concurrency. Use a reviewed operational transformation (OT) or conflict-free replicated data type (CRDT) model suited to the document structure. Arbitrary patches applied by arrival order are insufficient: operation meaning, context, identity, and reconnect rules must work together.

22.11.5 · Core API contracts

Creation returns document identity. Sync messages carry document and replica identities, stable operation IDs, known progress, and protocol version. Snapshots return a versioned base and specify the history they already cover.

Authorise exports and membership changes separately. Prevent clients from impersonating another replica. Define acknowledgments and reconnect so each edit is recognisably accepted, pending, or rejected. A successful connection handshake does not authorise every later edit or export.

22.11.6 · Data model and schema

Store owner, protocol version, membership, accepted operation IDs, ordering or causal context, and snapshots with covered positions. Compaction metadata records what can safely be retired.

Sequence CRDTs need stable element IDs and ordering rules. OT needs transformation context for its supported offline interval. A snapshot must identify already included operations; otherwise, a receiver may omit or repeat work even when snapshot and log files are each intact.

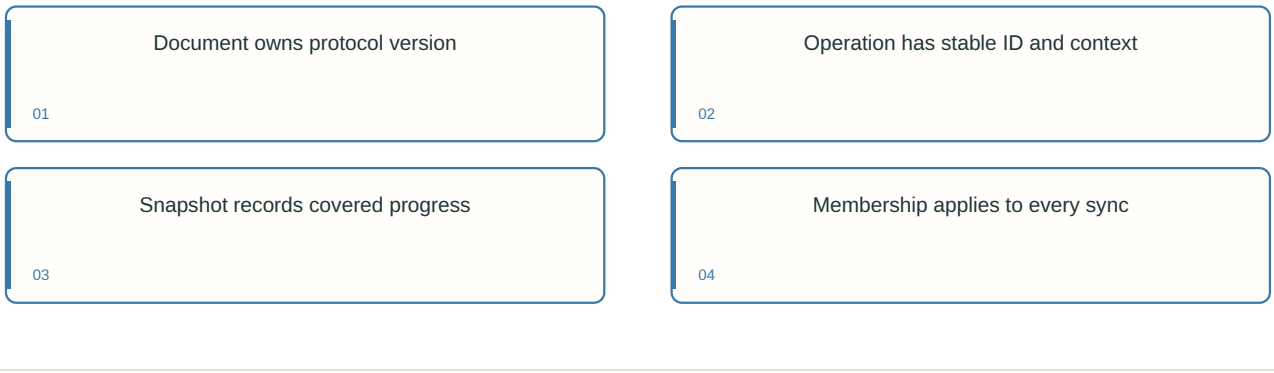


FIG. 22.11.1 **Data ownership and relationships**

Operations, snapshots, and retained context describe the same document history under one protocol.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.11.7 · Low-level component organization

DocumentService owns identity and membership. SyncCoordinator validates protocol use, EditModel applies its rules, and OperationLog persists acceptance. SnapshotBuilder produces recovery bases; Compactor retires only permitted history. Presence uses a temporary path.

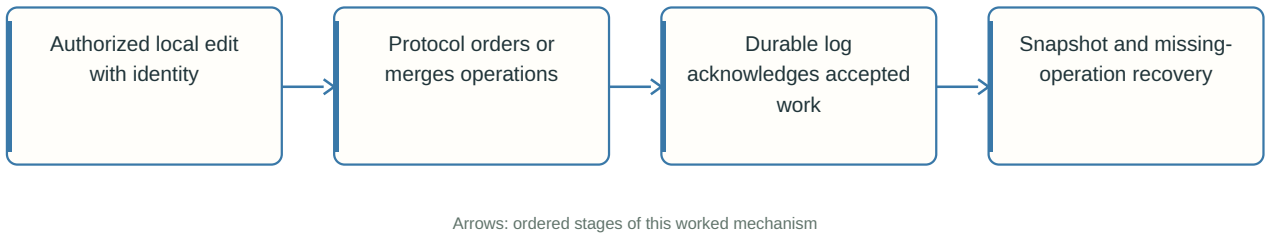
Keep the deterministic edit core apart from rendering and transport. Tests can then deliver the same operations in controlled orders. Expose durable progress explicitly, so a gateway reply is not mistaken for a committed edit.

22.11.8 · High-level system architecture

Use authenticated gateways, client replicas, durable operation storage, snapshots, and the ordering or merge arrangement required by the selected algorithm.

OT commonly uses ordered server context and transformation. CRDT approaches rely on their defined identities and merge rules. These families cannot be substituted as labels on an otherwise undefined box.

Select a supported protocol per document version and plan transitions. Gateway replacement must preserve its required history and context.



Arrows: ordered stages of this worked mechanism

FIG. 22.11.2 **Follow the main operation**

Local edits become accepted operations and reconnectable history through explicit sync rules.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.11.9 · Data and control flows

A client applies an allowed edit locally and assigns a stable identity. The server checks access and protocol context, saves accepted work, then distributes and acknowledges it according to the contract.

Reconnect exchanges missing operations or a compatible snapshot and continuation. Recognise repeats by identity and surface rejected offline changes explicitly. This sequence preserves local responsiveness while giving durable acceptance and access decisions their own boundaries. It also tells users which visible work still needs synchronization.

22.11.10 · Capacity estimates

The assumed 60,000 operations/s at 200 bytes produces 12 MB/s raw history. Fan-out and extra copies multiply the work. One popular document may concentrate transformation, merge metadata, and gateway traffic despite many cold documents elsewhere.

Snapshots reduce replay work in exchange for storage and background compute. Measure batching, document size, offline duration, and tombstone growth. Tombstones retain information about deleted elements where the protocol still needs their identities. Size the hot editor separately from the average.

22.11.11 · End-to-end latency estimates

Local optimistic editing can update the screen before a round trip while indicating unsynchronised work. Suppose ingress takes 30 ms, durable acceptance 20 ms, and peer delivery 50 ms. The proposed sync path totals 100 ms before additional waiting.

Measure p99 separately. Distance, disconnection, and large documents alter the path. Track pending duration as well as frame responsiveness: a smooth local screen does not prove an edit survives loss of that client.

22.11.12 · Failure scenarios

Concurrent inserts need a deterministic interpretation, and repeated delivery must not add a character twice. Missing context, unsafe compaction, and incompatible snapshots can break reconnect even after ordinary online editing works.

Test stale membership and expensive malformed operations too. Keep enough privacy-safe failing-history context to reproduce interleavings. A brief convergence demonstration does not cover long partitions or retired metadata. Recovery assumptions must match the offline behaviour advertised to users.



FIG. 22.11.3 **Follow the failure and its correction**

Concurrent or repeated operations need deterministic interpretation and stable identity.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.11.13 · Scalability improvements

Distribute independent documents, batch suitable operations, bound gateway buffers, and use snapshots to shorten replay. A single hot document may need its own coordination and fan-out design rather than another ordinary hash partition.

Compaction must retain context for the supported offline and merge horizon. An apparently unused old identifier can still anchor a returning operation. Scale cursor updates separately from durable edits, since their retention and correctness requirements differ even over one connection.

22.11.14 · Consistency requirements

OT transforms an operation against relevant concurrent operations. For example, an insertion before a later edit can shift that edit's position; the transform must use the correct ordered context. A sequence CRDT can instead name stable elements and define a shared order for concurrent insertions after the same element.

If X and Y are inserted after one element, replicas need compatible identities and tie-breaking rules to produce the same sequence despite opposite arrival orders. Neither family automatically specifies rich text, undo, or user intent. Access and durable acknowledgment are additional properties. Select and test an actual algorithm with explicit assumptions.

22.11.15 · Security model

Authorise sync sessions and individual operations under the revocation policy. Long-lived connections and returning offline clients require checks at later boundaries. Protect exports and snapshots too.

Validate protocol inputs and avoid content leaks through presence or logs. End-to-end encryption changes server search, validation, retention, and revocation possibilities; design those consequences when required. TLS protects transport, but does not itself provide that document-level guarantee.

22.11.16 · Deployment approach

Negotiate protocol versions and keep accepted history interpretable. Never reuse an operation identity with silently changed meaning. Gateway replacement must rely on durable recovery, rather than one client's memory.

Validate snapshots before retiring source history and test restoration from saved artifacts. Protocol migration and rollback affect stored documents and offline context as well as code. An older executable that misinterprets accepted operations can damage content that remains durably stored.

22.11.17 · Observability design

Measure pending and accepted edits, sync delay, duplicate recognition, reconnect gaps, snapshot age, replay time, compaction growth, protocol rejection, and access denial.

Keep reproducible histories for failing interleavings and test convergence alongside permission and structural validity. User pending duration can reveal harm hidden by gateway latency. Use limited diagnostic artifacts and privacy-safe identities instead of copying private documents into routine traces.

22.11.18 · Infrastructure cost considerations

Include history, snapshots, connections, fan-out, protocol compute, compaction, and telemetry. Replace sustained-rate arithmetic with measured activity durations when estimating expenditure.

Longer offline support can require more context and metadata. Compare designs at an equal recovery horizon. Deleting information still needed by returning clients changes the contract. Storage savings therefore need a valid compaction rule, rather than an arbitrary age threshold.

22.11.19 · Alternative architectures and trade-offs

Whole-document versions suit limited collaboration with explicit conflicts. OT supports particular transformation and ordering models. CRDTs provide specified merges with identity, metadata, and cleanup costs.

A tested editing library may be preferable to a custom algorithm. Evaluate Unicode graphemes, selections, undo, rich-text structure, and offline context as well as final convergence. Choose a complete model the team can verify and maintain; no family supplies every editing behaviour automatically.

22.11.20 · Architecture evolution from small scale to large scale

Begin with versioned documents and clear conflicts. Add a tested operation model, durable log, and reconnect. Introduce snapshots and justified compaction, then distribute independent documents.

Isolate hot workloads and add regional or offline merging only under supported protocol assumptions. Preserve identities, access rules, and deterministic regression histories. Each stage must retain the meaning of accepted edits and the context older supported clients need for recovery.

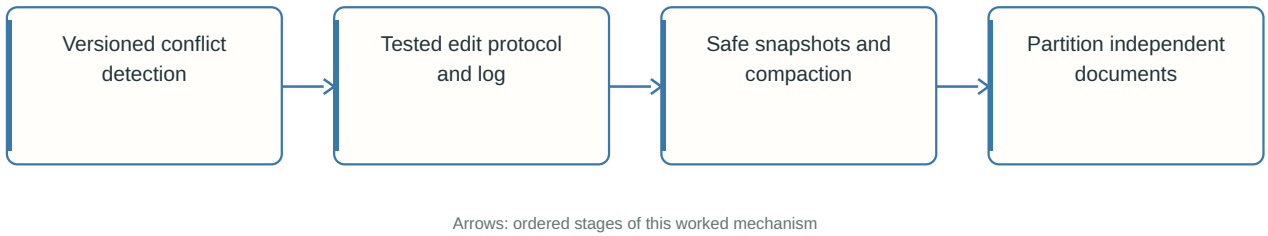


FIG. 22.11.4 **How the design can grow**

Distribute documents and compact history only within the supported offline and merge contract.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.11.21 · Design review identifying weaknesses and improvements

Review convergence, supported offline duration, history retirement, and revocation. Deliver concurrent inserts in opposite orders, repeat a delete, lose an acknowledgment, and reconnect using an old snapshot plus later operations.

Reconnect a revoked user and check document structure separately. Equal strings may conceal forbidden edits or invalid structure. Name the protocol and assumptions supporting each claim, rather than treating the final string comparison as complete validation.

22.11.22 · A short implementation exercise

Implement a miniature merge of two insertions with stable IDs and an explicit tie-break rule. Deliver both orders and repeat each operation. Verify equal final sequences with no duplicate insertion.

State the exercise's limit: it is not a complete rich-text CRDT. Add snapshot-context and revoked-client checks to expose distinct recovery and access boundaries. The example demonstrates identity and deterministic ordering without claiming production-editor correctness from two operations.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
def receive(operation, document):
    with transaction():
        current = lock_document(document.id)
        require_authorized(operation.actor, current)
        require_protocol_version(operation, current)
        if log.contains(operation.id): return prior_ack(operation.id)
        next_state = edit_model.apply(current.state, operation)
        append_unique_operation(operation)
        commit_state_and_progress(current, next_state)
    return durable_ack(operation.id)
# The edit model must implement a specified, tested algorithm.
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. A Conflict-Free Replicated JSON Datatype — Kleppmann and Beresford ↗

   Original research · 2017 revision. One formal CRDT model; the miniature tie-break exercise is not this complete implementation.

   Access reviewed · 21 October 2026

2. RFC 6455: The WebSocket Protocol, 2011 ↗

   Original standard. Bidirectional transport framing; not durable chat delivery.

   Access reviewed · 21 October 2026

3. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

Large-Scale Document Processing Platform

Prerequisites: File Systems, Search Infrastructure, and Data Pipelines, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery, Security Architecture and Privacy, Observability, Monitoring, and Incident Engineering, Deployment, Cloud Infrastructure, and Platform Architecture

4 original figures

Blue: normal flow · Green: healthy · Amber: caution · Red: failure · Purple: control / shared infrastructure · Gray: external / inactive

Solid arrow: labeled action · dashed: dependency or boundary · dotted: acknowledgment where labeled. Snapshot cards compare stated observations; they do not imply network calls.

A document has finished extraction, but some fields are uncertain and need review. Meanwhile, its owner requests deletion. A worker's finished flag cannot explain what may now be published.

We will trace immutable input through extraction, validation, human decisions, and publication. The workflow must keep those stages recoverable, preserve the source of every result, and reject stale work after its authority or document has been withdrawn.

The workload, timing, and prices in this design are teaching assumptions. They are not measured benchmarks or a description of a particular company.

22.12.1 · Functional requirements

A tenant uploads a document. The platform classifies it, extracts fields, sends uncertain results to review, and publishes validated output for authorised search or export. Users inspect progress, correct results, reprocess versions, and request deletion.

Upload success establishes accepted bytes under the upload contract. Extraction, review, and publication remain separate work. Give each stage a visible state so users can tell which output is usable and what remains unresolved.

22.12.2 · Nonfunctional requirements

Preserve accepted inputs and required workflow progress under selected failures. Results identify exact source and pipeline versions. Protect tenant data and set processing-age objectives for supported inputs.

Uncertain extraction must not silently become verified fact. Human decisions need durable actor and rationale information. Search has its own publication delay and permissions. Complete therefore means usable, traceable, permitted output, rather than merely a worker finishing without an exception.

22.12.3 · Explicit assumptions and workload numbers

Assume one million files/day averaging 2 MB, giving 2 TB/day raw input. Average arrival is about 11.6 files/s; a tenfold peak is about 116/s.

At an illustrative five CPU-seconds/file, extraction consumes five million CPU-seconds/day. Dividing by 3,600 gives about 1,389 CPU-hours. Classification, I/O, retries, and review add work or delay. Measure actual files and model behaviour; these are accounting assumptions rather than extraction throughput or accuracy results.

22.12.4 · Naive initial architecture

A prototype saves a file and extracts it within the upload request. Difficult files lengthen the request, process loss can erase progress, and human review has no natural durable waiting point.

Separate resumable byte ingestion from durable stages. Keep immutable source versions and identified outputs so retries and corrections retain evidence about earlier results. This fixes a concrete lifecycle and recovery problem before more workers are introduced for capacity.

22.12.5 · Core API contracts

POST /documents creates an upload intent; completion verifies stored bytes. Status exposes stages, while field reads return versioned values and provenance. Review decisions, reprocessing, and tracked deletion have separate operations.

Search and export expose only authorised published output. Distinguish deletion acceptance from full cleanup. Define which version reprocessing creates: a new pipeline must not silently reinterpret old output under the same identity.

22.12.6 · Data model and schema

A document binds tenant, source version, digest, and state. Upload sessions track bytes. Workflows name input and pipeline versions; attempts carry epochs. Artifacts carry kind, version, and object reference. Fields retain value, confidence, source region, and extractor identity.

Review records identify decision, actor, and version. Track publication and deletion progress, preserving tenant and version scope in keys. Retain sufficient provenance under an explicit lifecycle rather than keeping every raw intermediate forever.

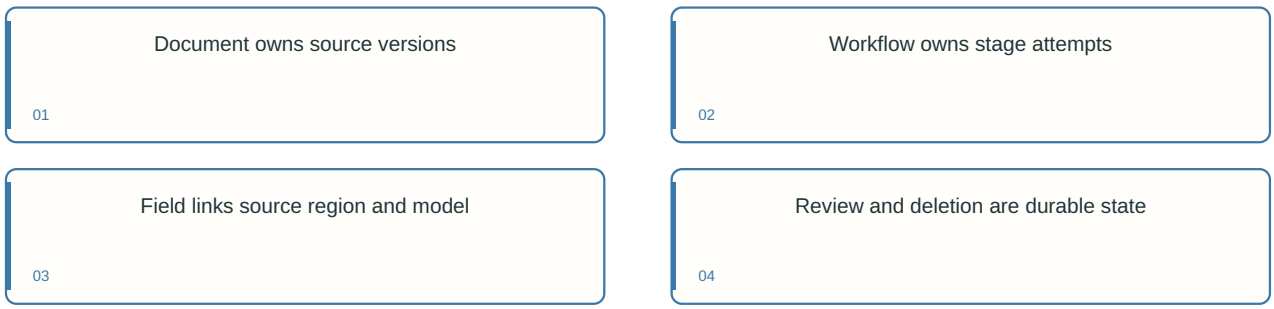


FIG. 22.12.1 **Data ownership and relationships**

Fields and decisions retain tenant, input, transform, and evidence relationships.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.12.7 · Low-level component organization

UploadService and ContentValidator check ingestion. WorkflowCoordinator owns stages. Versioned Classifier and Extractor adapters generate results; ProvenanceRecorder links evidence. ReviewPolicy selects review, ReviewService records decisions, and Publisher creates permitted projections. Reconciler repairs stuck or orphaned work.

Workers receive limited capabilities and cannot change tenant because a message supplies another tenant field. Keep generation of model output separate from deciding whether it is valid, reviewed, and ready to publish.

22.12.8 · High-level system architecture

Use immutable object storage, durable relational workflow metadata, task delivery, bounded isolated compute or model workers, review interfaces, and a search projection. Scoped storage carries bulk bytes;

metadata controls progress.

A document waiting hours for review must release its compute slot and database transaction. Durable workflow state survives without an attached process. Link sources, artifacts, decisions, and publication generations so a failure can be investigated and repaired across those components.

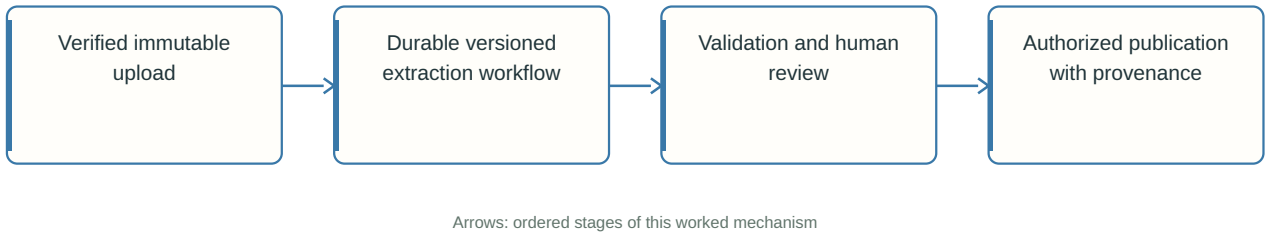


FIG. 22.12.2 **Follow the main operation**

Ingestion, extraction, review, and publication have distinct durable progress.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.12.9 · Data and control flows

Verify upload identity and digest, create a stable workflow, classify and extract, then validate structure and meaning. Save evidence and either publish or request review. A review decision creates a new validated version and publication intent.

Each stage completes conditionally under its current epoch. Reprocessing creates a new run over identified input while retaining earlier provenance. Deletion blocks inappropriate later publication. These persisted conditions protect local progress even when attempts repeat or an old worker resumes.

22.12.10 · Capacity estimates

Thirty days at 2 TB/day requires 60 TB of input before copies. Five million CPU-seconds/day divided by 86,400 gives about 57.9 continuously busy cores. At 60% target utilisation, 57.9 / 0.6 gives about 96.5 cores before other work and failure margin.

A tenfold arrival peak exceeds the average arrival rate. Its duration, queue age, scaling delay, and admission determine whether objectives hold. Include large-file and model-memory distributions rather than treating every document as average-sized.

22.12.11 · End-to-end latency estimates

Upload transfer and durable acceptance have one timeline. Automated work adds queueing, classification, extraction, validation, and publication; human review creates a separate wait.

Suppose queueing takes two seconds, classification one, extraction five, and validation/publication one. The automated path totals nine seconds in this model. These are assumptions to test. Report current stage and age so users can distinguish computation from a pending human decision.

22.12.12 · Failure scenarios

Incomplete uploads, hostile or corrupt files, and crashes after artifact write need different repairs. Repeated tasks, stale attempts, invalid output, concurrent reviews, and missing provenance can each damage another boundary.

Use stable artifact identity and conditional transitions. Quarantine unsupported output with a reason, and test deletion during processing. A finished label with missing, forbidden, or unexplained output still fails the business requirement even when workers report no exception.

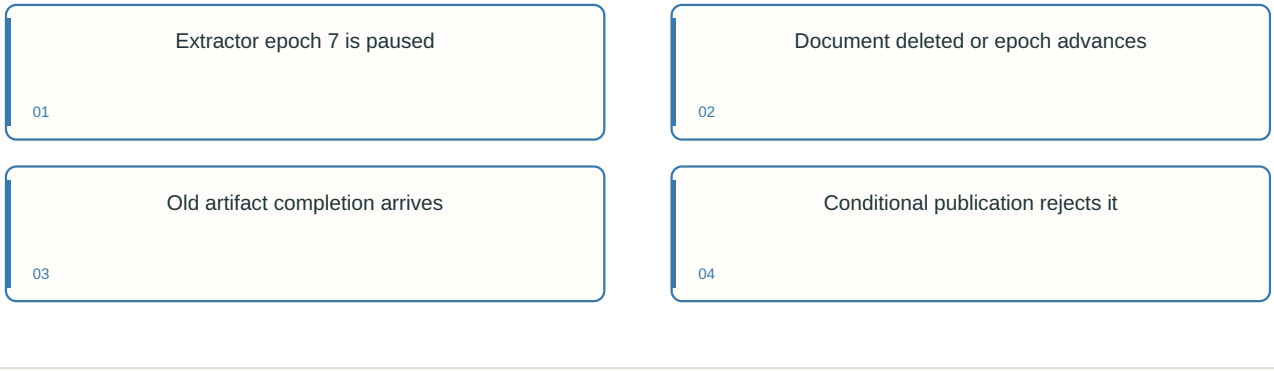


FIG. 22.12.3 **Follow the failure and its correction**

A stale attempt cannot publish after its authority or document has been withdrawn.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.12.13 · Scalability improvements

Transfer bulk bytes directly through storage and scale independent extraction with bounded workers. Partition metadata by a justified tenant or workflow key, accounting for hot tenants and large files.

Test batching and model-memory limits explicitly. More workers can saturate storage or an external model. Reserve separate replay and backfill capacity so upgrades do not starve live inputs or deletion. Expand the

stage limiting useful output rather than multiplying every worker equally.

22.12.14 · Consistency requirements

Source versions are immutable. Related workflow and review changes need suitable atomic transitions. Artifacts identify their input and transform; repeats use stable output identity and current-epoch completion.

Search may lag publication while preserving access rules. External model calls can be uncertain or nondeterministic. Retain relevant requests and version information, validate results, and state the actual reproducibility boundary. A recorded model name alone does not establish that a later run produces the same answer.

22.12.15 · Security model

Enforce tenant scope on objects, tasks, review, search, export, and audit. Bound worker resources and egress, validate supported files, and keep private data and credentials out of routine telemetry.

Treat extracted text as untrusted content rather than infrastructure instructions. Reviewers need scoped access and traceable decisions. Deletion covers sensitive derived output as well as input bytes. Follow permission through every stage that reads, transforms, or republishes the document.

22.12.16 · Deployment approach

Version pipelines and schemas compatibly. Evaluate models on authorised samples or controlled cohorts before replacing output, and retain a usable earlier generation until the transition is verified. Drain workers through durable attempts.

Restore exercises must reconnect objects, workflow state, decisions, and deletion history. Metadata-only restoration can leave missing artifacts or revive obsolete publication. Test the recovered journey before treating the platform as usable again.

22.12.17 · Observability design

Measure accepted bytes, stage waits, compute, retries, stale-completion rejection, invalid output, review age, provenance completeness, publication lag, tenant denial, and deletion progress.

Confidence is not measured accuracy. Use an appropriate labelled evaluation for quality claims. Link source, run, attempt, model, artifact, and reviewer through privacy-safe evidence. This reveals whether a particular result is usable and where it stopped, beyond showing that workers are busy.

22.12.18 · Infrastructure cost considerations

Include source and derived storage, compute or model calls, retries, review labour, search, transfer, telemetry, and recovery. Five million CPU-seconds/day describes compute; review may dominate operating cost.

Use explicit unit rates and a stated review fraction. Avoid invented accuracy or throughput figures. Compare equal provenance, access, quality, and recovery requirements, since removing review or history may lower cost while changing the output users require.

22.12.19 · Alternative architectures and trade-offs

Synchronous extraction fits a tiny demonstration while coupling upload to model delay and failure. Durable stages add metadata and operations but support recovery and human waits. Batching may improve resource use while adding delay and memory demand.

Managed models or workflows delegate selected duties without automatically providing validation, provenance, tenant isolation, or review policy. Compare the complete output and recovery contract. Choose the smallest arrangement that supports those needs for the intended workload.

22.12.20 · Architecture evolution from small scale to large scale

Start with immutable upload and explicit status. Add durable stages and stable artifacts before increasing concurrency. Give provenance and human decisions identities from the beginning.

Then grow storage and workers independently and introduce authorised search and versioned reprocessing. Regional placement needs a named locality or recovery requirement and compatible lifecycle rules. Earlier results must stay explainable, and old work must continue to respect current access and deletion decisions.

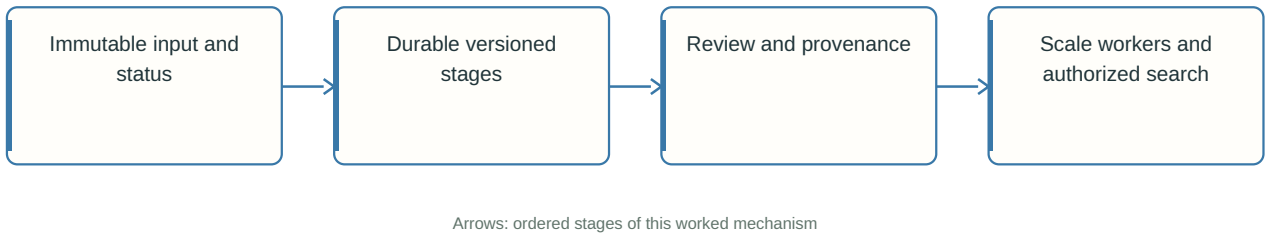


FIG. 22.12.4 **How the design can grow**

Scale stages independently while retaining provenance, access, and deletion behaviour.

Author-created illustration. Numbers are illustrative unless stated otherwise.

22.12.21 · Design review identifying weaknesses and improvements

Check whether a stage can silently change its input version, whether a superseded attempt can publish after deletion, and how reprocessing preserves human decisions.

Test task repeats, artifact-write interruption, invalid output, concurrent reviews, stale completion, missing objects, index lag, and full restoration. A few correct extracted fields cannot establish recoverable progress and provenance. Review the connected lifecycle, including rejection, review, and deletion outcomes.

22.12.22 · A short implementation exercise

Implement three stages with immutable input identity, result version, and attempt epoch. Stop after writing an artifact but before recording completion. Replay it without duplicating the established output.

Add a review state that releases compute while waiting. Pause an old worker, delete the document, then submit its completion. Publication must stay blocked by the authoritative current-state condition. Explain how that persisted check prevents stale work from reversing deletion, independently of whether cancellation reached the worker.

A small implementation of the key rule

PSEUDOCODE · ONE PART OF THE DESIGN

```
def complete_stage(run, epoch, artifact):
    require(artifact.input_version == run.input_version)
    validate_output_schema_and_provenance(artifact)
    with transaction():
        current = lock_workflow(run.id)
        require(current.epoch == epoch and not current.deleted)
        insert_artifact_once(run.id, artifact.version, artifact)
        advance_or_request_review(current, artifact)
        insert_next_stage_outbox_if_needed(current)
# Retry reuses identities; old attempts cannot publish new state.
```

Use the implementation exercise to test the rule and the failure described above. You can also run local examples of seat claims, duplicate handling, ledger balance, and workflow epochs in the mechanism collection.

Unit 22 exercises → · Unit 22 explained solutions →

Sources & further reading

These sources explain the mechanisms used in this chapter. The worked designs and numerical assumptions are our own teaching examples. Links were reviewed on 21 October 2026.

1. OpenLineage: core model and documentation ↗

   Official specification documentation · 1.54.0 at access. Dataset, job, and run lineage concepts.

   Access reviewed · 21 October 2026

2. Apache Kafka 4.1: Design ↗

   Official versioned documentation. Partitioning and scoped transactional/processing behavior; external effects are separate.

   Access reviewed · 21 October 2026

3. OWASP: Authorization ↗

   Official project guidance · accessed 21 Oct 2026. Least privilege, deny by default, and per-resource enforcement.

   Access reviewed · 21 October 2026

4. Transactional Outbox — Chris Richardson ↗

   Pattern author's explanation. Publication intent and duplicate delivery.

   Access reviewed · 21 October 2026

THE COMPLETE EDITION

Contents, from objects to systems

All 22 units and 99 chapters are available. Follow prerequisites or use a focused reading path.

Part 1 · System design foundations

01 · Introduction to System Design — READ NOW

- 1.1 · From a program to a system
- 1.2 · Organising code into modules
- 1.3 · From requirements to design choices
- 1.4 · Drawing the system at different levels

02 · Requirements, Estimation, and Engineering Trade-offs — READ NOW

- 2.1 · Turn user journeys into workloads
- 2.2 · Measure the experience
- 2.3 · Estimate the resources
- 2.4 · Queues, bottlenecks & defensible decisions

03 · Networking Foundations for System Design — READ NOW

- 3.1 · Addresses, sockets & transports
- 3.2 · A secure request, one round trip at a time
- 3.3 · Intermediaries & persistent connections
- 3.4 · Discover, route & survive the network

04 · Runtime, Processes, Threads, and Concurrency

READ NOW

- 4.1 · The runtime owns your resources
- 4.2 · Tasks, threads & event loops
- 4.3 · Shared memory & coordination
- 4.4 · Visibility, cancellation & overload

Part 2 · Low-level design & software architecture

05 · Object-Oriented and Modular Software Design

READ NOW

- 5.1 · Model behavior with objects and values
- 5.2 · Principles under pressure
- 5.3 · Side effects, domain models & collaboration
- 5.4 · Design for change, testing & functions

06 · Design Patterns, Dependency Injection, and Object Lifecycles

READ NOW

- 6.1 · Dependencies, inversion & composition
- 6.2 · Lifetimes, ownership & cleanup
- 6.3 · Injection, locators & factories
- 6.4 · Creational patterns
- 6.5 · Structural patterns
- 6.6 · Behavioral patterns
- 6.7 · Enterprise patterns & boundaries

07 · Application Architecture and Domain-Driven Design

READ NOW

- 7.1 · Layers, monoliths & ports
- 7.2 · Model a bounded context
- 7.3 · Keep business policy inside
- 7.4 · Evolution without a distributed monolith

08 · API Design and Communication Contracts READ NOW

8.1 · Model resources and messages

8.2 · Compatibility & idempotency

8.3 · Control access, load & long operations

8.4 · Evolve a public contract

Part 3 · Data, storage & scalability

09 · Data Models and Storage Technologies READ NOW

9.1 · Storage promises & database families

9.2 · Files, objects & analytical layouts

9.3 · Model relationships and ownership

9.4 · Access patterns, retention & operational cost

10 · Database Internals, Transactions, and Concurrency Control READ NOW

10.1 · Pages, indexes & query plans

10.2 · Commit, logging & versions

10.3 · Races, anomalies & isolation

10.4 · Pools, recovery & ticket correctness

11 · Caching and Content Delivery READ NOW

11.1 · Locality and cache placement

11.2 · Copies, freshness & eviction

11.3 · Hot keys, stampedes & negative results

11.4 · Edges, partitions & the price of a cache

12 · Scalability, Load Balancing, and Data Distribution READ NOW

- 12.1 · Scale compute & route requests
- 12.2 · Replicate and partition data
- 12.3 · Lag, proxies & saturated pools
- 12.4 · Regions, write paths & scale economics

13 · File Systems, Search Infrastructure, and Data Pipelines READ NOW

- 13.1 · Upload bytes without losing identity
- 13.2 · Build and rebuild a search index
- 13.3 · Move data through batch and streams
- 13.4 · Quality, provenance & replay

Part 4 · Distributed systems & reliability

14 · Distributed Systems Fundamentals READ NOW

- 14.1 · Partial failure & the meaning of time
- 14.2 · Consistency as a reader-visible contract
- 14.3 · Partitions, CAP, PACELC & quorums
- 14.4 · Coordination, safety & liveness

15 · Replication, Leader Election, and Consensus READ NOW

- 15.1 · Choose a replication model
- 15.2 · Raft and replicated state machines
- 15.3 · Fences, leases & failure detection
- 15.4 · Conflicts, CRDTs & regional trade-offs

16 · Message Queues, Events, and Distributed Workflows

READ NOW

- 16.1 · Queues, logs & delivery boundaries
- 16.2 · Retries, consumers & event history
- 16.3 · Read models, the outbox & sagas
- 16.4 · Coordinate durable workflows

17 · Reliability, Fault Tolerance, and Disaster Recovery

READ NOW

- 17.1 · Failure domains & redundancy
- 17.2 · Retries, circuit breakers & isolation
- 17.3 · Shed load and degrade deliberately
- 17.4 · Recover, test & learn from failure

Part 5 · Security & production engineering

18 · Security Architecture and Privacy

READ NOW

- 18.1 · Threats, identity & authorization
- 18.2 · Secrets, encryption & trust boundaries
- 18.3 · Tenant isolation, audit & deletion
- 18.4 · Supply chains and security reviews

19 · Observability, Monitoring, and Incident Engineering

READ NOW

- 19.1 · Observe behavior with three signals
- 19.2 · Choose indicators, budgets & alerts
- 19.3 · Profile and investigate production
- 19.4 · Learn from incidents and trace async work

20 · Deployment, Cloud Infrastructure, and Platform Architecture READ NOW

20.1 · Environments, VMs & containers

20.2 · Control planes, scheduling & configuration

20.3 · Releases, flags & schema migrations

20.4 · Platforms, serverless & ownership

21 · Performance, Cost Optimization, and System Validation READ NOW

21.1 · Budgets, bottlenecks & honest benchmarks

21.2 · Capacity, architecture & cost per outcome

21.3 · Efficiency and a useful test portfolio

21.4 · Fitness functions and design validation

Part 6 · Complete system design case studies

22 · End-to-End Design, Architecture Reviews, and Evolution READ NOW

22.1 · URL Shortener

22.2 · Rate Limiter

22.3 · Chat Application

22.4 · Notification Platform

22.5 · Social Media Feed

22.6 · Video Streaming Platform

22.7 · Ride-Hailing System

22.8 · Online Ticket Booking

22.9 · Payment System

22.10 · Distributed Job Scheduler

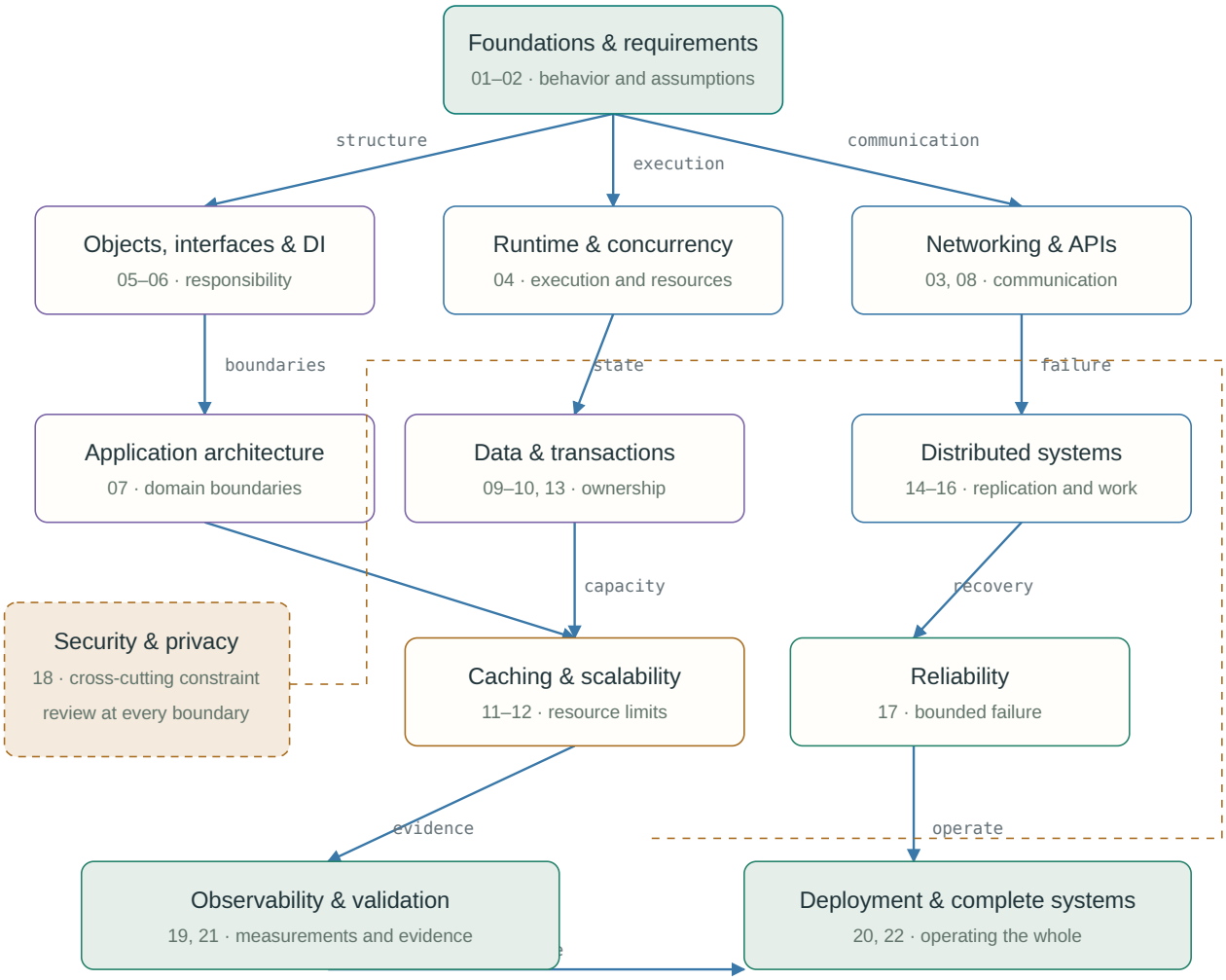
22.11 · Collaborative Document Editor

22.12 · Large-Scale Document Processing Platform

HOW THE IDEAS CONNECT

A map of the territory

The simplified concept spine connects modules, networks, data, distribution, and production engineering.



Simplified concept spine; exact unit and chapter prerequisites appear in Reading paths.

FIG. M.1 **The ideas form a dependency network.**

A simplified prerequisite spine connects low-level structure to storage, networking, distributed behavior, and production engineering. Security constrains the entire design.

Why this view: it shows why a replication or scaling mechanism needs earlier reasoning about state and execution. Author-created illustration.

Solid arrow = conceptual prerequisite · dashed line = cross-cutting constraint. Exact prerequisites are in Reading paths. Author-created illustration.

DEPENDENCY-AWARE ROUTES

Reading paths

Every route below includes the prerequisite closure of its selected units; open any unit for its direct chapter dependencies.

First complete pass

01 → 02 → 03 → 04 → 05 → 06 → 07 → 08 → 09 → 10

Data and scale

01 → 02 → 03 → 04 → 08 → 09 → 10 → 11 → 12 → 13

Distributed correctness

01 → 02 → 03 → 04 → 08 → 09 → 10 → 11 → 12 → 14 → 15 → 16 → 17

Production engineering

01 → 02 → 03 → 04 → 05 → 06 → 07 → 08 → 09 → 10 → 11 → 12 → 14 → 15 → 16 → 17 → 18 → 19 → 20 → 21

Direct prerequisites

Unit	Before this unit
1 · Introduction to System Design	Basic programming
2 · Requirements, Estimation, and Engineering Trade-offs	1
3 · Networking Foundations for System Design	1
4 · Runtime, Processes, Threads, and Concurrency	1
5 · Object-Oriented and Modular Software Design	1
6 · Design Patterns, Dependency Injection, and Object Lifecycles	5
7 · Application Architecture and Domain-Driven Design	5, 6
8 · API Design and Communication Contracts	1, 3
9 · Data Models and Storage Technologies	1
10 · Database Internals, Transactions, and Concurrency Control	4, 9
11 · Caching and Content Delivery	3, 9
12 · Scalability, Load Balancing, and Data Distribution	2, 3, 9, 10, 11
13 · File Systems, Search Infrastructure, and Data Pipelines	8, 9, 10
14 · Distributed Systems Fundamentals	3, 4, 10
15 · Replication, Leader Election, and Consensus	14
16 · Message Queues, Events, and Distributed Workflows	8, 10, 14, 15
17 · Reliability, Fault Tolerance, and Disaster Recovery	2, 12, 15, 16
18 · Security Architecture and Privacy	3, 7, 8, 9
19 · Observability, Monitoring, and Incident Engineering	2, 3, 4, 10

Unit	Before this unit
20 · Deployment, Cloud Infrastructure, and Platform Architecture	3, 7, 17, 18, 19
21 · Performance, Cost Optimization, and System Validation	2, 4, 10, 12, 17, 19, 20
22 · End-to-End Design, Architecture Reviews, and Evolution	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21

The case-study progression

The URL shortener connects estimation, cache freshness, and scaling. Payments connect object boundaries, API identity, transactions, outbox, provider failures, and ledger reconciliation. Tickets connect local concurrency to expiry and external payment. Documents connect immutable input, lineage, durable stages, human review, and publication.

22.1 · URL Shortener — API Design and Communication Contracts, Data Models and Storage Technologies, Caching and Content Delivery, Scalability, Load Balancing, and Data Distribution

22.2 · Rate Limiter — Requirements, Estimation, and Engineering Trade-offs, Runtime, Processes, Threads, and Concurrency, Scalability, Load Balancing, and Data Distribution, Reliability, Fault Tolerance, and Disaster Recovery

22.3 · Chat Application — Networking Foundations for System Design, Data Models and Storage Technologies, Distributed Systems Fundamentals, Message Queues, Events, and Distributed Workflows

22.4 · Notification Platform — Design Patterns, Dependency Injection, and Object Lifecycles, API Design and Communication Contracts, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery

22.5 · Social Media Feed — Data Models and Storage Technologies, Caching and Content Delivery, Scalability, Load Balancing, and Data Distribution, Message Queues, Events, and Distributed Workflows

22.6 · Video Streaming Platform — Networking Foundations for System Design, Caching and Content Delivery, File Systems, Search Infrastructure, and Data Pipelines, Deployment, Cloud Infrastructure, and Platform Architecture

22.7 · Ride-Hailing System — Data Models and Storage Technologies, Scalability, Load Balancing, and Data Distribution, Distributed Systems Fundamentals, Message Queues, Events, and Distributed Workflows

22.8 · Online Ticket Booking — API Design and Communication Contracts, Database Internals, Transactions, and Concurrency Control, Message Queues, Events, and Distributed Workflows, Security Architecture and Privacy

22.9 · Payment System — API Design and Communication Contracts, Database Internals, Transactions, and Concurrency Control, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery, Security Architecture and Privacy

22.10 · Distributed Job Scheduler — Runtime, Processes, Threads, and Concurrency, Replication, Leader Election, and Consensus, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery

22.11 · Collaborative Document Editor — Networking Foundations for System Design, Distributed Systems Fundamentals, Replication, Leader Election, and Consensus, Message Queues, Events, and Distributed Workflows

22.12 · Large-Scale Document Processing Platform — File Systems, Search Infrastructure, and Data Pipelines, Message Queues, Events, and Distributed Workflows, Reliability, Fault Tolerance, and Disaster Recovery, Security Architecture and Privacy, Observability, Monitoring, and Incident Engineering, Deployment, Cloud Infrastructure, and Platform Architecture

ALL 22 UNITS / 242 QUESTIONS

Exercises

Five conceptual questions, three scenarios, one design challenge, one implementation exercise, and one advanced thought experiment per unit.

Unit 1 · Introduction to System Design

CONCEPTUAL / C1

A program adds a task to a dictionary and displays “Saved.” What can it do while running, and what happens when it starts with a new dictionary?

Read the explanation →

CONCEPTUAL / C2

What does the signature create(title) tell a caller? What must the contract explain?

Read the explanation →

CONCEPTUAL / C3

The task service depends on a repository. Does this dependency necessarily make the design difficult to change?

Read the explanation →

CONCEPTUAL / C4

How do a high-level view, a low-level view, and a deployment view help explain the task application?

Read the explanation →

CONCEPTUAL / C5

A service stops accepting requests during recovery but keeps every committed task. Which observation concerns availability, and which concerns durability?

Read the explanation →

SCENARIO / S1

Task 7 is committed, but its success reply is lost. The client waits, then retries. Can it conclude that the first attempt failed?

Read the explanation →

SCENARIO / S2

The web adapter and command-line adapter each enforce an 80-character title limit. Only the web adapter is changed to allow 200 characters. How would you prevent this disagreement?

Read the explanation →

SCENARIO / S3

The team runs two application machines, each with its own unrelated local task database. A user sees a task on one request and cannot find it on the next. What could explain this?

Read the explanation →

DESIGN / D1

A handler adds a task to memory and replies “Saved.” A background writer saves it later. Trace a crash that loses the task, then propose a change that meets the restart requirement.

Read the explanation →

IMPLEMENTATION / I1

Run examples/todo.py. Make a test repository raise a storage error. Then use the real repository to save a task, close it, and read the task through a new connection. What should each check show?

Read the explanation →

THOUGHT EXPERIMENT / T1

The team now requires saved tasks to survive destruction of the only disk. Why is ADR 001 insufficient, and what must the next design address?

Read the explanation →

Unit 2 · Requirements, Estimation, and Engineering Trade-offs

CONCEPTUAL / C1

Opening a feed appears to be one action. Why might it create many requests and writes?

Read the explanation →

CONCEPTUAL / C2

Requests keep arriving faster, but successful completions stay at the same rate. What happens next?

Read the explanation →

CONCEPTUAL / C3

How can a 19.9 ms mean hide a request that took one second?

Read the explanation →

CONCEPTUAL / C4

Which requests belong in an event-based error budget?

Read the explanation →

CONCEPTUAL / C5

When is L = λW useful for estimating work in progress?

Read the explanation →

SCENARIO / S1

A traffic spike ends before new instances can start. What protects the service?

Read the explanation →

SCENARIO / S2

Only 80% of a job can run in parallel. What ideal speedup can four workers give?

Read the explanation →

SCENARIO / S3

Why can requiring fifty parallel calls increase tail latency?

Read the explanation →

DESIGN / D1

A team sizes servers from daily average RPS alone. Identify the missing evidence and propose a safer estimation process.

Read the explanation →

IMPLEMENTATION / I1

Implement a capacity worksheet with explicit user activity, peak, and per-instance inputs.

Read the explanation →

THOUGHT EXPERIMENT / T1

How would simultaneous retries change an estimate based on average user sessions?

Read the explanation →

Unit 3 · Networking Foundations for System Design

CONCEPTUAL / C1

A web service listens on port 443. Why might a browser on another machine still fail to reach it?

Read the explanation →

CONCEPTUAL / C2

A sender writes HELLO and then WORLD through TCP. Why might the receiver not get two separate reads?

Read the explanation →

CONCEPTUAL / C3

The DNS address changes. Why do some clients still use the old server?

Read the explanation →

CONCEPTUAL / C4

The gateway recognises the signed-in user. Why must the task application still check ownership of task 42?

Read the explanation →

CONCEPTUAL / C5

What work can SSE or WebSockets save, and what work do they add?

Read the explanation →

SCENARIO / S1

The server saves task 7, but the reply is lost. What does the client learn from its timeout?

Read the explanation →

SCENARIO / S2

Our cold request takes 155 ms. Why can the next request take 55 ms with the same server work?

Read the explanation →

SCENARIO / S3

All twenty pool connections are borrowed, but CPU has free time. Why can the next request still wait?

Read the explanation →

DESIGN / D1

“Read” appears twice after a create request times out and is sent again. How would you change the request flow?

Read the explanation →

IMPLEMENTATION / I1

Write a cold/warm request calculator. How will you check it?

Read the explanation →

THOUGHT EXPERIMENT / T1

Our application used to call a local function. Now it calls a service in another region. What new events can affect the result?

Read the explanation →

Unit 4 · Runtime, Processes, Threads, and Concurrency

CONCEPTUAL / C1

How are a program, a process, a thread, and a source-code module different?

Read the explanation →

CONCEPTUAL / C2

An upload handler is marked async. Why might other work on its event loop still wait?

Read the explanation →

CONCEPTUAL / C3

Can one core make progress on two uploads without running them at the same instant?

Read the explanation →

CONCEPTUAL / C4

A transfer changes balances from 100,100 through 90,100 to 90,110. Why are separate atomic writes insufficient?

Read the explanation →

CONCEPTUAL / C5

A log says A set the title before B read it. Why do we still need safe publication?

Read the explanation →

SCENARIO / S1

A and B read count zero, then both write one. Which steps must the shared lock protect?

Read the explanation →

SCENARIO / S2

An upload read completes while a callback uses the only event-loop thread for two seconds. When can the upload task continue?

Read the explanation →

SCENARIO / S3

150 jobs arrive and 100 finish each second. If all are accepted, how much extra work remains after ten seconds?

Read the explanation →

DESIGN / D1

Every upload starts another background task. How would you limit the work and keep accepted jobs through a restart?

Read the explanation →

IMPLEMENTATION / I1

Build a lost-update demonstration and a bounded upload queue. What should each show?

Read the explanation →

THOUGHT EXPERIMENT / T1

The client disconnects after an upload job is accepted. Should processing stop?

Read the explanation →

Unit 5 · Object-Oriented and Modular Software Design

CONCEPTUAL / C1

A service can use two fraud policies and two providers. How does composition help?

Read the explanation →

CONCEPTUAL / C2

What is missing from the operation charge(100)?

Read the explanation →

CONCEPTUAL / C3

Does SOLID require an interface around every class? Explain with the payment gateway.

Read the explanation →

CONCEPTUAL / C4

Why is Payment 42 an entity while Money(USD,100) is a value object?

Read the explanation →

CONCEPTUAL / C5

Which part of a function-based payment design makes the real charge?

Read the explanation →

SCENARIO / S1

Two callers each try to reserve 600 from an account with 1,000 available. Why is reserve(amount) alone insufficient?

Read the explanation →

SCENARIO / S2

A test confirms every private method call, but the order is paid after a decline. What should the test check?

Read the explanation →

SCENARIO / S3

A refactor changes timeout into decline. Why is that a behaviour change?

Read the explanation →

DESIGN / D1

How would you split a class that parses requests, calculates fees, calls a provider, and stores results?

Read the explanation →

IMPLEMENTATION / I1

Supply a recording gateway that returns uncertain. What should the small test assert?

Read the explanation →

THOUGHT EXPERIMENT / T1

We replace payment objects with pure functions. Which requirements remain?

Read the explanation →

Unit 6 · Design Patterns, Dependency Injection, and Object Lifecycles

CONCEPTUAL / C1

A notification service receives a provider SDK through its constructor. Has it necessarily achieved dependency inversion?

Read the explanation →

CONCEPTUAL / C2

Two processes each have a singleton sender. Must it be the same object, and is it automatically safe for concurrent calls?

Read the explanation →

CONCEPTUAL / C3

A factory returns a connection. What must its contract tell the caller besides how to query it?

Read the explanation →

CONCEPTUAL / C4

A constructor receives a container, and the service calls resolve() inside its methods. Why are its dependencies hard to see?

Read the explanation →

CONCEPTUAL / C5

Each notification chooses email or SMS when it arrives. How can DI and a factory work together?

Read the explanation →

SCENARIO / S1

An application-wide service retains request A's database session. Request A ends. What can go wrong during request B?

Read the explanation →

SCENARIO / S2

Two first requests arrive together for a lazily created report generator. What needs to be decided?

Read the explanation →

SCENARIO / S3

A pool has ten connections. Every failed request keeps one. Why can a later healthy request become stuck?

Read the explanation →

DESIGN / D1

An application-wide notification service looks up a request connection in a global container and stores it. Propose a clearer arrangement.

Read the explanation →

IMPLEMENTATION / I1

Demonstrate transient, scoped, and singleton identity across two scopes. Then show why retaining a scoped resource does not keep it valid.

Read the explanation →

THOUGHT EXPERIMENT / T1

A saved notification job runs after its creating request and user session have ended. How should it obtain resources and permission to act?

Read the explanation →

Unit 7 · Application Architecture and Domain-Driven Design

CONCEPTUAL / C1

What makes a monolith modular if it has one deployment?

Read the explanation →

CONCEPTUAL / C2

Why can dependency arrows point differently from runtime calls?

Read the explanation →

CONCEPTUAL / C3

Why can catalog Product and fulfilment Item be different valid models?

Read the explanation →

CONCEPTUAL / C4

How should an aggregate boundary follow the rule it protects?

Read the explanation →

CONCEPTUAL / C5

What should an architecture fitness check preserve?

Read the explanation →

SCENARIO / S1

An order commits, then the process dies before publishing its event. What is missing?

Read the explanation →

SCENARIO / S2

Five services share writable tables and always release together. What independence is missing?

Read the explanation →

SCENARIO / S3

When would extracting a module into a service be justified?

Read the explanation →

DESIGN / D1

Several services share tables and require simultaneous releases. Review whether the deployment split is useful.

Read the explanation →

IMPLEMENTATION / I1

Implement shopping modules and one meaningful dependency-boundary check.

Read the explanation →

THOUGHT EXPERIMENT / T1

What evidence would justify extracting payments into a separate service?

Read the explanation →

Unit 8 · API Design and Communication Contracts

CONCEPTUAL / C1

Why give a refund its own identity and status?

Read the explanation →

CONCEPTUAL / C2

What does a keyset cursor improve, and what does it leave unresolved?

Read the explanation →

CONCEPTUAL / C3

What makes an idempotency key more than a string in a request?

Read the explanation →

CONCEPTUAL / C4

Why does signing in not grant access to GET /payments/42?

Read the explanation →

CONCEPTUAL / C5

How can removing an old event decoder break recovery after a successful release?

Read the explanation →

SCENARIO / S1

Does 201 Created for a pending payment mean money was captured?

Read the explanation →

SCENARIO / S2

Four layers each allow three attempts. How much downstream work can one request cause?

Read the explanation →

SCENARIO / S3

A webhook effect commits but its reply is lost. What should redelivery do?

Read the explanation →

DESIGN / D1

A payment API returns a Boolean and provides no operation status. Redesign it for an uncertain provider reply.

Read the explanation →

IMPLEMENTATION / I1

Implement keyset continuation and a durable operation-key model for synthetic payments.

Read the explanation →

THOUGHT EXPERIMENT / T1

Can finite storage support a permanent promise to recognise every old idempotency key?

Read the explanation →

Unit 9 · Data Models and Storage Technologies

CONCEPTUAL / C1

What must be decided before choosing a storage family?

Read the explanation →

CONCEPTUAL / C2

Why is a similar embedding result not proof of truth or permission?

Read the explanation →

CONCEPTUAL / C3

When is storing an address twice correct?

Read the explanation →

CONCEPTUAL / C4

Why is data ownership broader than creating a table?

Read the explanation →

CONCEPTUAL / C5

What work appears when a second storage engine is added?

Read the explanation →

SCENARIO / S1

Why do more cold partitions not solve one very busy conversation partition?

Read the explanation →

SCENARIO / S2

Search still holds a deleted private document. What must the system do?

Read the explanation →

SCENARIO / S3

What prevents an older backup from restoring deleted content into live service?

Read the explanation →

DESIGN / D1

A search engine holds the only copy of private document text. Review ownership, recovery, and deletion.

Read the explanation →

IMPLEMENTATION / I1

Model profiles, financial entries, chat history, events, and PDFs from their required operations.

Read the explanation →

THOUGHT EXPERIMENT / T1

How should restoration from a historical backup respect later deletion decisions?

Read the explanation →

Unit 10 · Database Internals, Transactions, and Concurrency Control

CONCEPTUAL / C1

Why might range and equality queries need different indexes?

Read the explanation →

CONCEPTUAL / C2

Why does an ACID transaction not automatically preserve every business rule?

Read the explanation →

CONCEPTUAL / C3

Why is MVCC alone insufficient to claim serializability?

Read the explanation →

CONCEPTUAL / C4

Why state the engine and statement behaviour when discussing isolation?

Read the explanation →

CONCEPTUAL / C5

How do serializability and linearizability differ?

Read the explanation →

SCENARIO / S1

Two doctors each see the other on call and both go off duty. What rule was missed?

Read the explanation →

SCENARIO / S2

A conditional seat update changes zero rows. What does that mean?

Read the explanation →

SCENARIO / S3

A backup job succeeded. What recovery evidence is still needed?

Read the explanation →

DESIGN / D1

Two doctors can each go off call after seeing the other available. Propose a transaction design preserving at least one on call.

Read the explanation →

IMPLEMENTATION / I1

Run the conditional last-seat claim with two connections to one store.

Read the explanation →

THOUGHT EXPERIMENT / T1

What changes when one local transaction is divided across independent databases?

Read the explanation →

Unit 11 · Caching and Content Delivery

CONCEPTUAL / C1

Why does a working set matter more than the full history size for caching?

Read the explanation →

CONCEPTUAL / C2

How can cache-aside reinsert an old value after invalidation?

Read the explanation →

CONCEPTUAL / C3

Why can a one-minute TTL serve data older than one minute?

Read the explanation →

CONCEPTUAL / C4

For A,B,C,A,D at capacity three, what do ideal LRU and FIFO evict?

Read the explanation →

CONCEPTUAL / C5

When might removing a cache improve the design?

Read the explanation →

SCENARIO / S1

A cache restart sends misses to the origin. How can that load be controlled?

Read the explanation →

SCENARIO / S2

A product is created just after “not found” is cached. What should the policy explain?

Read the explanation →

SCENARIO / S3

Refresh keeps failing while stale data is served. What limit is needed?

Read the explanation →

DESIGN / D1

A cache-aside reader repopulates old data after the writer invalidates it. Define and repair the actual freshness requirement.

Read the explanation →

IMPLEMENTATION / I1

Run ideal LRU, then reproduce stale cache-aside population after invalidation.

Read the explanation →

THOUGHT EXPERIMENT / T1

When should a product stop serving rather than return a stale cached result?

Read the explanation →

Unit 12 · Scalability, Load Balancing, and Data Distribution

CONCEPTUAL / C1

The application becomes slow, so the team adds web servers. Under which conditions would you expect more useful completions, and when might the extra servers mainly create more waiting?

Read the explanation →

CONCEPTUAL / C2

A dataset is divided among several storage owners. Explain what changes for routing, queries, and rules that previously fitted inside one database.

Read the explanation →

CONCEPTUAL / C3

In the rendezvous example, owner D is added to A, B, and C. Explain which assignments can change, and why this does not establish balanced request traffic.

Read the explanation →

CONCEPTUAL / C4

A dashboard says that a replica has received an update. Why is that insufficient evidence that a read can observe it, or that it will survive the required failure?

Read the explanation →

CONCEPTUAL / C5

Why should the cost comparison between one larger node and several smaller nodes use the same failure and recovery requirements?

Read the explanation →

SCENARIO / S1

One partition receives 70% of writes while the other partitions are mostly idle. Which evidence would help you choose between splitting it, changing placement, and redesigning the hot operation?

Read the explanation →

SCENARIO / S2

During a database slowdown, an autoscaler adds application replicas. Explain how this can increase timeouts even though the application now has more machines.

Read the explanation →

SCENARIO / S3

Two application replicas run on one host. What protection do they provide, and which additional evidence would you need before claiming that they survive a host failure?

Read the explanation →

DESIGN / D1

After the team doubles application replicas, database timeouts increase. Describe an investigation and a correction that would preserve service while testing the cause.

Read the explanation →

IMPLEMENTATION / I1

Run the provided rendezvous example with owners A, B, C, and then D. Count assignments before and after, and send most simulated requests to one key. What should the two measurements reveal?

Read the explanation →

THOUGHT EXPERIMENT / T1

Every caller competes for one globally hot resource that can have only one exclusive owner. Can adding many partitions make these ownership decisions independent? Explain what a valid improvement would have to preserve.

Read the explanation →

Unit 13 · File Systems, Search Infrastructure, and Data Pipelines

CONCEPTUAL / C1

Why do acknowledged upload parts not prove the completed file is correct?

Read the explanation →

CONCEPTUAL / C2

Why does a content hash not authorise a download?

Read the explanation →

CONCEPTUAL / C3

What does an inverted index record for “blue car” and “blue bike”?

Read the explanation →

CONCEPTUAL / C4

What can go wrong between a snapshot and its CDC stream?

Read the explanation →

CONCEPTUAL / C5

What provenance makes an extracted field explainable?

Read the explanation →

SCENARIO / S1

Updates and deletes continue during reindexing. What must happen before serving switches?

Read the explanation →

SCENARIO / S2

An event arrives three minutes late. Which time policy matters?

Read the explanation →

SCENARIO / S3

Why is deleting only the PDF insufficient?

Read the explanation →

DESIGN / D1

A reindex copies records while updates and deletions continue. Design the handoff to a new serving generation.

Read the explanation →

IMPLEMENTATION / I1

Implement source-version and attempt checks, then plan a safe index-generation switch.

Read the explanation →

THOUGHT EXPERIMENT / T1

How would you investigate an extracted field after both source and model have changed?

Read the explanation →

Unit 14 · Distributed Systems Fundamentals

CONCEPTUAL / C1

A payment provider commits a charge, but its reply never reaches the order service. Why is this a partial failure, and how should the order service proceed?

Read the explanation →

CONCEPTUAL / C2

A Lamport event has value 5 and another has value 9. What can you conclude from those numbers alone? What extra information can a vector comparison provide?

Read the explanation →

CONCEPTUAL / C3

A register starts at 1. A write of 2 completes before another client's read begins, and no other write occurs. What must a linearizable read return? How does the answer change if the read overlaps the write?

Read the explanation →

CONCEPTUAL / C4

Two live replicas cannot communicate during a partition. One accepts and acknowledges a write. Explain why CAP prevents promising both atomic consistency and completion of every request reaching either side.

Read the explanation →

CONCEPTUAL / C5

Can a booking service prevent double sales while failing to provide useful service? Explain why the progress requirement must be stated carefully.

Read the explanation →

SCENARIO / S1

A session reads version 10 of a profile. Its next request reaches another replica and returns version 9. Which guarantee has failed, and what could the serving replica do instead?

Read the explanation →

SCENARIO / S2

A design uses three fixed replicas, requires two write acknowledgments, and reads two replicas. Explain what R=2, W=2, N=3 proves and what remains to be shown before claiming linearizability.

Read the explanation →

SCENARIO / S3

Leader A pauses with epoch 7. Leader B obtains epoch 8 and begins taking over. A later resumes. Where and how should writes from the earlier owner be prevented?

Read the explanation →

DESIGN / D1

A partition separates three replicas into A and the pair B,C. Both sides receive requests to allocate the same exclusive seat. Propose a policy that preserves one winner, including the steps needed when communication returns.

Read the explanation →

IMPLEMENTATION / I1

Use the mechanism collection to evaluate lamport_receive(5,9), vector_relation([1,0],[0,1]), and vector_relation([1,0],[1,2]). Then enumerate the two-member read and write sets of A,B,C and explain their intersection.

Read the explanation →

THOUGHT EXPERIMENT / T1

During a partition, exclusive seat allocation cannot obtain valid authority. Which other operations could remain useful, and how should their promises and recovery differ?

Read the explanation →

Unit 15 · Replication, Leader Election, and Consensus

CONCEPTUAL / C1

A primary waits for a replica before reporting success. What must “synchronous replication” specify about that acknowledgment?

Read the explanation →

CONCEPTUAL / C2

Replicas agree to apply “create task” in the same log position. Why can choosing a random identifier independently on each replica still break the design?

Read the explanation →

CONCEPTUAL / C3

A Raft leader sees an entry on two of three replicas. What else must be established before treating that entry as committed?

Read the explanation →

CONCEPTUAL / C4

Why can replacing a voting group abruptly with a disjoint group allow conflicting decisions?

Read the explanation →

CONCEPTUAL / C5

Two disconnected regions each sell the last item they see. Why can a convergent counter still fail the stock rule?

Read the explanation →

SCENARIO / S1

A worker carrying epoch 7 resumes after epoch 8 has become current for the attempt. Where and how must the old effect be rejected?

Read the explanation →

SCENARIO / S2

A paused leader misses heartbeats and is falsely suspected of failure. What must remain safe while the system attempts recovery?

Read the explanation →

SCENARIO / S3

A repaired old primary is reachable again. What must happen before it accepts authoritative writes?

Read the explanation →

DESIGN / D1

A worker pauses beyond lease expiry. A new owner begins work, and the old worker returns. Design protection against stale effects and stale release requests.

Read the explanation →

IMPLEMENTATION / I1

Implement a receiver that rejects stale attempt epochs. Which update must be atomic, and how would you check its behaviour?

Read the explanation →

THOUGHT EXPERIMENT / T1

A failed region has been repaired and can receive traffic. What evidence would you require before allowing authoritative writes there again?

Read the explanation →

Unit 16 · Message Queues, Events, and Distributed Workflows

CONCEPTUAL / C1

An ephemeral subscriber is disconnected when an event is published. Why can it not automatically replay that event on return?

Read the explanation →

CONCEPTUAL / C2

A consumer can acknowledge before or after its required effect. How does each order behave if it crashes between the two events?

Read the explanation →

CONCEPTUAL / C3

A stream processor claims exactly-once behaviour. Which work can its transaction cover, and what must be checked before extending that claim to an email?

Read the explanation →

CONCEPTUAL / C4

Why can an outbox relay publish the same event twice even though the order and outbox row committed together?

Read the explanation →

CONCEPTUAL / C5

A saga refunds a payment after checkout fails. Why is that compensation not necessarily an undo?

Read the explanation →

SCENARIO / S1

A local effect commits, but the message is delivered again. How can the consumer preserve one local update?

Read the explanation →

SCENARIO / S2

A worker continues running after its lease expires and a replacement starts. What protects later effects?

Read the explanation →

SCENARIO / S3

A payment remains locally pending after its provider reply is lost. Why should recovery avoid resending it as a new operation?

Read the explanation →

DESIGN / D1

An order commits but its event is never published. Design durable publication and repeated-delivery handling.

Read the explanation →

IMPLEMENTATION / I1

Crash the outbox relay after publication, then process the duplicate with a local inbox transaction.

Read the explanation →

THOUGHT EXPERIMENT / T1

How could one broker transaction be exactly-once while two emails are sent?

Read the explanation →

Unit 17 · Reliability, Fault Tolerance, and Disaster Recovery

CONCEPTUAL / C1

Why are two replicas not necessarily enough surviving capacity?

Read the explanation →

CONCEPTUAL / C2

Why include pool acquisition in the overall deadline?

Read the explanation →

CONCEPTUAL / C3

What does a circuit breaker help, and what remains unresolved?

Read the explanation →

CONCEPTUAL / C4

How can separate worker pools still share a failure bottleneck?

Read the explanation →

CONCEPTUAL / C5

How do RTO and RPO differ?

Read the explanation →

SCENARIO / S1

How should an end-to-end design limit a retry storm?

Read the explanation →

SCENARIO / S2

Can a default cached balance replace an unavailable ledger?

Read the explanation →

SCENARIO / S3

Why not send traffic straight back to a repaired primary?

Read the explanation →

DESIGN / D1

A provider slowdown holds every connection and triggers layered retries. Break the cascade without inventing business success.

Read the explanation →

IMPLEMENTATION / I1

Calculate nested retry amplification and design a bounded provider-outage policy.

Read the explanation →

THOUGHT EXPERIMENT / T1

How can a disaster drill restore infrastructure yet fail the business recovery requirement?

Read the explanation →

Unit 18 · Security Architecture and Privacy

CONCEPTUAL / C1

Why is an internal network address not proof of a caller's identity?

Read the explanation →

CONCEPTUAL / C2

How do OAuth 2.0 and OpenID Connect differ in purpose?

Read the explanation →

CONCEPTUAL / C3

Why use purpose-built password hashing rather than a fast hash?

Read the explanation →

CONCEPTUAL / C4

Which paths need tenant isolation beyond the database query?

Read the explanation →

CONCEPTUAL / C5

Does an artifact's valid signature prove its code is safe?

Read the explanation →

SCENARIO / S1

A tenant A administrator guesses a tenant B document ID. What denies access?

Read the explanation →

SCENARIO / S2

A backup survives but its only decryption key is destroyed. What went wrong?

Read the explanation →

SCENARIO / S3

Membership is revoked after an export is queued. Where must that policy be enforced?

Read the explanation →

DESIGN / D1

A private export is authorised at submission but its worker trusts the old tenant field forever. Review the complete journey.

Read the explanation →

IMPLEMENTATION / I1

Test the synthetic tenant predicate, then trace all data paths using that decision.

Read the explanation →

THOUGHT EXPERIMENT / T1

What should revocation mean for an issued object capability and a queued export?

Read the explanation →

Unit 19 · Observability, Monitoring, and Incident Engineering

CONCEPTUAL / C1

How do logs, metrics, and traces complement one another?

Read the explanation →

CONCEPTUAL / C2

Why is a trace ID not an access credential?

Read the explanation →

CONCEPTUAL / C3

Why can low CPU coexist with severe request delay?

Read the explanation →

CONCEPTUAL / C4

What is tenfold budget burn for a 99.9% success target?

Read the explanation →

CONCEPTUAL / C5

Why avoid unique request IDs as ordinary metric labels?

Read the explanation →

SCENARIO / S1

A five-second request waits 4.6 seconds for a connection. Where should investigation focus?

Read the explanation →

SCENARIO / S2

Serving recovers but charges remain uncertain. Why is recovery incomplete?

Read the explanation →

SCENARIO / S3

HTTP 202 is fast while jobs wait for hours. Which indicators are missing?

Read the explanation →

DESIGN / D1

HTTP acceptance stays fast while background jobs stop finishing. Redesign the service indicators and alert.

Read the explanation →

IMPLEMENTATION / I1

Build the five-second trace model and an asynchronous completion indicator.

Read the explanation →

THOUGHT EXPERIMENT / T1

What evidence would support a latency explanation beyond correlation with a release?

Read the explanation →

Unit 20 · Deployment, Cloud Infrastructure, and Platform Architecture

CONCEPTUAL / C1

What important boundary differs between a VM and a typical container?

Read the explanation →

CONCEPTUAL / C2

How can a control-plane outage differ from a serving outage?

Read the explanation →

CONCEPTUAL / C3

Why must a canary receive representative traffic?

Read the explanation →

CONCEPTUAL / C4

Why can an old executable fail as a rollback?

Read the explanation →

CONCEPTUAL / C5

Which duties remain with a team using a managed database?

Read the explanation →

SCENARIO / S1

Scale-down kills a worker holding an unrecorded job. What was missing?

Read the explanation →

SCENARIO / S2

A feature is disabled after writing a new data format. What risk remains?

Read the explanation →

SCENARIO / S3

Why do two successfully scheduled replicas not necessarily form a correct shared service?

Read the explanation →

DESIGN / D1

A blue-green release shares one database and removes a column needed by the old version. Review its rollback plan.

Read the explanation →

IMPLEMENTATION / I1

Plan and test an expand-and-contract release with old and new readers.

Read the explanation →

THOUGHT EXPERIMENT / T1

How can rollback restore old code while leaving incompatible data or effects?

Read the explanation →

Unit 21 · Performance, Cost Optimization, and System Validation

CONCEPTUAL / C1

What must accompany a reusable benchmark result?

Read the explanation →

CONCEPTUAL / C2

How can coordinated omission hide waiting?

Read the explanation →

CONCEPTUAL / C3

Why is a short measured throughput maximum not a safe production target?

Read the explanation →

CONCEPTUAL / C4

Why can cost per attempt hide increasing cost per useful outcome?

Read the explanation →

CONCEPTUAL / C5

What should an architecture fitness function protect?

Read the explanation →

SCENARIO / S1

An index fixes the slow query. Why might sharding have been a poorer first step?

Read the explanation →

SCENARIO / S2

What does killing a process only before work begins fail to test?

Read the explanation →

SCENARIO / S3

Source and destination counts match but tenant owners differ. Why did validation miss it?

Read the explanation →

DESIGN / D1

A faster and cheaper benchmark configuration omits failures and disables authorisation. Can it replace the original design?

Read the explanation →

IMPLEMENTATION / I1

Compare the three fictional architectures against equal requirements and costs.

Read the explanation →

THOUGHT EXPERIMENT / T1

When could a more expensive architecture be simpler to operate overall?

Read the explanation →

Unit 22 · End-to-End Design, Architecture Reviews, and Evolution

CONCEPTUAL / C1

A short-link click count is temporarily behind, while two buyers are trying to claim one seat. Why can these operations use different consistency rules?

Read the explanation →

CONCEPTUAL / C2

A chat service returns a persisted receipt, then the recipient disconnects before receiving a live frame. What has the receipt established, and how should delivery recover?

Read the explanation →

CONCEPTUAL / C3

Worker A pauses at epoch 7. Its lease expires, and worker B receives epoch 8. Why does the lease alone not guarantee one external execution?

Read the explanation →

CONCEPTUAL / C4

Two document replicas finish with equal text. What else must be checked before calling the collaborative editor correct?

Read the explanation →

CONCEPTUAL / C5

An extraction worker finishes without an exception. What further conditions may be needed before a document-processing workflow is complete?

Read the explanation →

SCENARIO / S1

A payment succeeds after its seat hold has expired. Another buyer may now own the seat. What may the booking service safely do?

Read the explanation →

SCENARIO / S2

A provider charges successfully, the process stops before recording its reply, and the client retries. How can the service avoid another logical charge?

Read the explanation →

SCENARIO / S3

An epoch-7 extraction worker finishes after the document is deleted or its attempt is superseded. What must stop the old artifact from being published?

Read the explanation →

DESIGN / D1

Review a short-link service at 100,000 redirects/s when its cache becomes unavailable. What assumption has changed, and what must the design demonstrate?

Read the explanation →

IMPLEMENTATION / I1

Choose one case study and implement its failure exercise. Which invariant will you check, where will you interrupt execution, and what recovered state should result?

Read the explanation →

THOUGHT EXPERIMENT / T1

You can fund another region, tested restores, or stronger reconciliation first. How would you choose which work reduces the most important risk?

Read the explanation →

ALL 22 UNITS / EXPLAINED ANSWERS

Solutions & reasoning

Each explanation connects the outcome to assumptions, mechanism, and failure. A later or linked chapter provides the full surrounding context.

Unit 1 · Introduction to System Design

UNIT 01 / C1

A program adds a task to a dictionary and displays “Saved.” What can it do while running, and what happens when it starts with a new dictionary?

While the program runs, it can list and update the task in that dictionary. After restart, a new empty dictionary contains none of the previous tasks. To keep the task across a restart, the program must save it in persistent storage and wait for the save to finish before reporting success.

← Return to question

UNIT 01 / C2

What does the signature create(title) tell a caller? What must the contract explain?

The signature names the operation and its input; a typed signature can also describe the result type. The contract explains the behaviour: which titles are valid, what changes when the call succeeds, and which errors may occur. In our persistent application, it also says that creation returns successfully only after commit. Callers need these rules to use the operation correctly.

← Return to question

UNIT 01 / C3

The task service depends on a repository. Does this dependency necessarily make the design difficult to change?

No. The service needs a way to save tasks. A repository contract lets it request that work without knowing the SQL or file layout. A column rename can then be handled in the repository. The useful question is how much one module must know about another and whether a change to those details spreads through the application.

← Return to question

UNIT 01 / C4

How do a high-level view, a low-level view, and a deployment view help explain the task application?

The high-level view shows the main parts, such as the application and database. The low-level view explains the service methods, task records, and repository interface inside the application. The deployment view places the running application and database file on machines. These views describe the same system, so their decisions must agree. A separate repository box in a module diagram does not imply a separate server.

← Return to question

UNIT 01 / C5

A service stops accepting requests during recovery but keeps every committed task. Which observation concerns availability, and which concerns durability?

The period when users cannot perform task operations concerns availability. The fact that committed tasks remain concerns durability under that recovery scenario. A system can preserve data while being temporarily unavailable. Conversely, it can keep replying while losing tasks that were stored only in memory.

← Return to question

UNIT 01 / S1

Task 7 is committed, but its success reply is lost. The client waits, then retries. Can it conclude that the first attempt failed?

No. In this sequence, task 7 exists even though the client has not received the reply. A fresh create request may add task 8. To recognise the retry, the client needs to reuse an operation identifier, and the server must keep that identifier with the committed result. The design must also specify how it checks that the repeated request matches and how long it remembers the result. Chapter 8.2 develops these rules.

← Return to question

UNIT 01 / S2

The web adapter and command-line adapter each enforce an 80-character title limit. Only the web adapter is changed to allow 200 characters. How would you prevent this disagreement?

Move the shared title rule into the task service and make both adapters call it. Each adapter still checks its own input format, such as whether a web request contains valid JSON. Check that the database can store the longer title and update any matching storage constraint. Editing both copied rules fixes today’s mismatch but leaves the same risk for the next change.

← Return to question

UNIT 01 / S3

The team runs two application machines, each with its own unrelated local task database. A user sees a task on one request and cannot find it on the next. What could explain this?

The two requests may have reached different machines. A task saved in one local database is absent from the other. The design now needs a way for the applications to use shared or coordinated data. Adding a load balancer distributes requests; it does not copy or merge the task databases.

← Return to question

UNIT 01 / D1

A handler adds a task to memory and replies “Saved.” A background writer saves it later. Trace a crash that loses the task, then propose a change that meets the restart requirement.

The failing sequence is: add to memory, reply “Saved,” crash before the background write commits. After restart, memory is empty and there is no saved task to reload. Change the order so that success follows a durable commit, and report a write error if the save fails. A local database is enough for our stated single-machine example. A background queue would need its own durable-save and acknowledgment rules before it could support the same promise.

← Return to question

UNIT 01 / I1

Run examples/todo.py. Make a test repository raise a storage error. Then use the real repository to save a task, close it, and read the task through a new connection. What should each check show?

When insert raises a storage error, create must pass that error to its caller rather than return a successful Task. With the real SQLite repository, the task should still be present after closing and reopening the same database. Also submit an invalid title and check that no row is added. The first check covers error handling, the second persistence across reopen, and the third validation. An in-memory substitute cannot establish the persistence result.

← Return to question

UNIT 01 / T1

The team now requires saved tasks to survive destruction of the only disk. Why is ADR 001 insufficient, and what must the next design address?

ADR 001 assumes that committed storage remains intact. Destroying its only disk removes the copy it relies on. The new design needs recoverable data on independent storage and a rule for when it may report success. If copying happens only later, a task reported as saved can be lost before the copy is made. Specify which failures the copies must survive, how much loss is acceptable, and how recovery will work. Then test those cases. Simply counting replicas does not answer these questions.

← Return to question

Unit 2 · Requirements, Estimation, and Engineering Trade-offs

UNIT 02 / C1

Opening a feed appears to be one action. Why might it create many requests and writes?

The browser may fetch a list of posts, request each image, and send analytics. A publish action may save the post once and then write references into many follower feeds. These are separate operations even though the person performed one action.

Count user intentions separately from requests, stored bytes, and physical writes. Otherwise an estimate based only on feed openings or published posts will miss much of the work at the storage and delivery boundaries.

← Return to question

UNIT 02 / C2

Requests keep arriving faster, but successful completions stay at the same rate. What happens next?

Once a limiting resource cannot complete work faster, excess arrivals must wait, be rejected, or leave before completion. At 200 arrivals/s and 180 completions/s, twenty unfinished requests accumulate each second if none are rejected or abandoned.

Compare offered traffic, useful completions, errors, and queue growth. A steady completion rate can coexist with a rapidly worsening user wait; it does not establish that increasing demand is being handled.

← Return to question

UNIT 02 / C3

How can a 19.9 ms mean hide a request that took one second?

The 99 fast requests contribute 99 × 10 = 990 ms, and the slow request contributes 1,000 ms. Their total is 1,990 ms, so the mean is 1,990/100 = 19.9 ms. The arithmetic is correct, but the mean compresses very different experiences into one number.

Inspect the slow observations and the distribution. In this particular 100-request set, nearest-rank p99 is the 99th sorted value, which is still 10 ms. Reporting a percentile alone can also miss an extreme observation.

← Return to question

UNIT 02 / C4

Which requests belong in an event-based error budget?

The requests specified by the objective's eligible population belong in its denominator, and each must be classified using the stated good-outcome rule. Define both before measuring.

At 99.9% success over one million eligible requests, the allowed bad fraction is 0.1% = 0.001. The budget is therefore 1,000,000 × 0.001 = 1,000 failed requests. Excluding the failures the objective was meant to track would make the result misleading. A time-based objective uses time, rather than request count, as its denominator.

← Return to question

UNIT 02 / C5

When is L = λW useful for estimating work in progress?

Use stable long-run averages measured within the same boundary and in matching units. Here L is the average number present, λ is the arrival rate, and W is mean residence time, including relevant waiting. At 500 operations/s and 40 ms = 0.04 seconds, L = 500 × 0.04 = twenty operations.

The result describes average work in progress, not an exact count at every instant or p99 latency. It does not show that a queue with sustained excess arrivals will become stable.

← Return to question

UNIT 02 / S1

A traffic spike ends before new instances can start. What protects the service?

Existing spare capacity can serve some excess work before new instances are ready. A bounded queue can hold a temporary excess if the wait is acceptable, while admission limits reject or defer work before downstream resources become overwhelmed.

Measure the spike's height and duration and include retries. An excess of 1,000 requests/s for 30 seconds adds 30,000 queued requests if none are rejected or completed by additional capacity. Starting instances later cannot undo the waits already experienced; the remaining backlog still needs to drain.

← Return to question

UNIT 02 / S2

Only 80% of a job can run in parallel. What ideal speedup can four workers give?

Imagine a 100-second job: twenty seconds must run sequentially, while eighty can be divided perfectly. Four workers reduce the shareable part to 80/4 = 20 seconds, giving 20 + 20 = 40 seconds overall. The ideal speedup is 100/40 = 2.5 times.

Equivalently, the new duration is 0.2 + 0.8/4 = 0.4 of the original. Coordination can reduce the actual gain further. The unchanged sequential part also limits ideal speedup to five times even with arbitrarily many workers.

← Return to question

UNIT 02 / S3

Why can requiring fifty parallel calls increase tail latency?

The request waits for its slowest required result. If each call independently has a 1% chance of exceeding a threshold, its chance of staying within the threshold is 0.99. All fifty are fast with probability $0.99^{50} \approx 0.605$. Thus at least one is slow with probability $1 - 0.605 \approx 0.395$, or about 39.5%.

Independence is an assumption; shared dependencies can correlate delays. Parallel initiation removes the need to start calls one after another, but it does not remove the wait for the last required result.

← Return to question

UNIT 02 / D1

A team sizes servers from daily average RPS alone. Identify the missing evidence and propose a safer estimation process.

Begin with the user journeys, required outcomes, and operation mix. Estimate the requests, physical writes, stored bytes, and transfer each journey causes. Calculate an average with explicit units, then add separate peak and growth assumptions, including peak duration and the delay before scaling takes effect.

Measure per-instance capacity using a representative workload and the required latency and error targets. Choose operating headroom and check whether the remaining instances can handle traffic during the failures the design must tolerate. Size the database and image path separately.

Treat the worksheet as a workload to test. Verify user latency, rejection, queue growth, and recovery under normal, peak, and degraded conditions. A calculated instance count alone cannot establish that the complete design meets its requirements.

← Return to question

UNIT 02 / I1

Implement a capacity worksheet with explicit user activity, peak, and per-instance inputs.

Make daily users, requests per user per day, peak multiplier, tested RPS per instance, and target utilisation explicit inputs. Validate positive workload and capacity values and require a utilisation fraction strictly between zero and one. Calculate daily requests, divide by 86,400 seconds/day, and multiply by the peak factor.

The defaults give about 5,787.04 average requests/s and 28,935.19 peak requests/s. At an assumed 600 requests/s per instance and a 60% operating target, each contributes 600 × 0.6 = 360 requests/s of planned capacity. Dividing peak demand by 360 gives about 80.38; rounding upward gives 81 active instances. Providing 82 leaves 81 after one instance is lost.

Keep the inputs and intermediate results in the output so changes can be checked. The arithmetic covers this request-serving resource and one-instance loss; database capacity, image delivery, routing, and failures that remove several instances still need their own evidence.

← Return to question

UNIT 02 / T1

How would simultaneous retries change an estimate based on average user sessions?

Retries add attempts without adding new user intentions. A daily session estimate therefore undercounts offered work when failed requests are repeated. If many callers retry at once, those attempts can arrive while the same dependency is still overloaded, creating another concentrated burst.

Model the timing and maximum number of attempts separately from normal sessions. Bound retries and admission, and test useful completions, queue growth, and recovery under that burst. A peak factor based only on ordinary user arrivals may not cover the retry wave.

← Return to question

Unit 3 · Networking Foundations for System Design

UNIT 03 / C1

A web service listens on port 443. Why might a browser on another machine still fail to reach it?

The listener may be attached only to the local loopback address. Or the request or reply may have no usable route, or a firewall may block it. Check the server address, listening address, and both directions of the path. Reaching the program still leaves the application's permission checks to perform.

← Return to question

UNIT 03 / C2

A sender writes HELLO and then WORLD through TCP. Why might the receiver not get two separate reads?

TCP carries an ordered byte stream, not the sender's write boundaries. One read could contain HELLOWO and the next RLD. The application needs a framing rule to recover its messages, such as a length before each one. UDP instead preserves datagram boundaries; QUIC provides its own ordered streams over UDP.

← Return to question

UNIT 03 / C3

The DNS address changes. Why do some clients still use the old server?

A resolver or client can still have a usable cached answer. An already open connection may also continue without another lookup. Updating the published record does not replace every saved answer or close existing connections. Their lifetimes affect when traffic moves.

← Return to question

UNIT 03 / C4

The gateway recognises the signed-in user. Why must the task application still check ownership of task 42?

Knowing who the caller is does not establish which tasks the caller may read. The task application can compare the recognised user with task 42's owner and enforce the permission rule. The gateway can perform common sign-in and allowance checks, while the application uses its resource information.

← Return to question

UNIT 03 / C5

What work can SSE or WebSockets save, and what work do they add?

They can avoid repeated empty polls by keeping a path open for messages. The system then maintains connections and buffers and must handle slow or disconnected clients. If updates occur during disconnection, it needs retained events or a fresh snapshot. A new connection alone cannot recover the missing history.

← Return to question

UNIT 03 / S1

The server saves task 7, but the reply is lost. What does the client learn from its timeout?

It learns only that a reply did not arrive in time. The task can already exist. If the request carried op-9 and the server saved that identity with its result, the client can look up or resend op-9 to recover task 7. Sending a new operation would risk a duplicate.

← Return to question

UNIT 03 / S2

Our cold request takes 155 ms. Why can the next request take 55 ms with the same server work?

The assumed cold stages are DNS 20 ms, TCP setup 40 ms, TLS setup 40 ms, and request plus work 55 ms. They add to 155 ms. A usable warm connection and lookup remove the first three stages in this model. The 55 ms stage already includes its request and reply travel. These durations are assumed, not measured.

← Return to question

UNIT 03 / S3

All twenty pool connections are borrowed, but CPU has free time. Why can the next request still wait?

The request needs an available connection, not just CPU time. Existing borrowers may be waiting on storage, locks, or another operation. Find why they still hold their connections and whether every exit returns them. Adding connections helps only if the database can use the extra work effectively.

← Return to question

UNIT 03 / D1

“Read” appears twice after a create request times out and is sent again. How would you change the request flow?

Give the original request a stable operation identifier. Save the new task and that operation’s result together. An identical retry uses the same identifier and receives the saved result; a changed request must not reuse that identity as if it were identical. This repairs task creation within the stored-operation rules. A remote provider would need its own repetition and recovery rules.

← Return to question

UNIT 03 / I1

Write a cold/warm request calculator. How will you check it?

Store each stage's time in milliseconds and add the stages actually needed. Check that 20, 40, 40, and 55 total 155, and that reusing setup leaves 55. Reject negative and non-finite times rather than producing a misleading total. Also check zero-duration stages and a request that reuses only some setup.

← Return to question

UNIT 03 / T1

Our application used to call a local function. Now it calls a service in another region. What new events can affect the result?

The request must travel to another machine and the reply must travel back. Setup and travel add time. The server can finish while the reply is lost, leaving the caller uncertain. The interface now needs a waiting limit and a way to recognise or find the original operation when repeating it could cause another effect.

← Return to question

Unit 4 · Runtime, Processes, Threads, and Concurrency

UNIT 04 / C1

How are a program, a process, a thread, and a source-code module different?

A program is saved instructions; a process is one running instance. Threads execute within a process and can share its data. A module organises source code. Several modules can run in the same process and stop together when it crashes, so separate modules do not automatically create separate failure boundaries.

← Return to question

UNIT 04 / C2

An upload handler is marked async. Why might other work on its event loop still wait?

The handler may call a blocking library function or keep calculating without yielding. That occupies the loop's thread. Other tasks can become ready but cannot run there until the current work gives up the thread. Async describes how the operation presents completion; it does not change every library call underneath.

← Return to question

UNIT 04 / C3

Can one core make progress on two uploads without running them at the same instant?

Yes. A can start, wait for storage, and resume after the core has run B. Their progress overlaps, so they are concurrent. Running instructions for both at the same instant would require parallel execution resources. Taking turns can still cause a race if both use a shared old value.

← Return to question

UNIT 04 / C4

A transfer changes balances from 100,100 through 90,100 to 90,110. Why are separate atomic writes insufficient?

Each write can be indivisible while the pair still passes through an incomplete state. A reader could see 90,100. Protect both changes as one operation and make readers of the pair follow the same rule, such as taking the same lock. A database transaction can supply a different appropriate boundary for stored balances.

← Return to question

UNIT 04 / C5

A log says A set the title before B read it. Why do we still need safe publication?

Clock order alone does not supply the language's memory-visibility guarantee. Use a defined synchronisation relationship. For example, A sets and shares the record while holding a mutex, releases it, and B later acquires the same mutex before reading. Under the required memory-model rules, A's writes happen before B's read.

← Return to question

UNIT 04 / S1

A and B read count zero, then both write one. Which steps must the shared lock protect?

Protect the read, addition, and write together. If A completes those steps first, B reads one and writes two. Protecting only the assignments still allows both to calculate from zero. Separate private locks also fail because neither makes one caller wait for the other.

← Return to question

UNIT 04 / S2

An upload read completes while a callback uses the only event-loop thread for two seconds. When can the upload task continue?

Its result is ready, but its continuation still needs a turn on the thread. It waits until the callback yields or finishes. The distinction is between a completed I/O operation and execution of the code that uses its result. More queued tasks do not let that one thread run both pieces at once.

← Return to question

UNIT 04 / S3

150 jobs arrive and 100 finish each second. If all are accepted, how much extra work remains after ten seconds?

Each second adds 150 − 100 = 50 unfinished jobs. Ten seconds add 500, assuming none are rejected or abandoned. More queue space stores the excess without changing its growth. A bounded queue can reject or wait when full, and the producer must react to that feedback for arrivals to slow.

← Return to question

UNIT 04 / D1

Every upload starts another background task. How would you limit the work and keep accepted jobs through a restart?

Use a worker limit for active conversions and a queue limit for waiting jobs. Store large file bytes separately and queue small references. Give each accepted job a saved identity, input reference, and progress record. Replacement workers can then find unfinished jobs after restart. Return acceptance only after the required record is saved; define a clear busy or waiting result when capacity is unavailable.

← Return to question

UNIT 04 / I1

Build a lost-update demonstration and a bounded upload queue. What should each show?

Arrange for A and B to read zero before either writes, producing one. Then protect the complete update with a shared lock and show two. For uploads, set limits on both workers and waiting identifiers. Fill the queue and show the busy response. Also make conversion fail and check that its files and other resources are released. An in-memory demonstration needs saved job records before it can promise restart recovery.

← Return to question

UNIT 04 / T1

The client disconnects after an upload job is accepted. Should processing stop?

First decide what acceptance promised. If the job must continue, its saved record and a worker outside the request scope own the remaining work. Request-owned children can stop and release their resources. If cancellation of the job is allowed, record that decision so later workers can obey it. Closing the connection alone neither deletes the saved job nor reverses work already completed.

← Return to question

Unit 5 · Object-Oriented and Modular Software Design

UNIT 05 / C1

A service can use two fraud policies and two providers. How does composition help?

The service receives one fraud-policy object and one gateway object. It asks the policy whether to allow the attempt, then asks the gateway to charge. Either part can be replaced without making a service subclass for every combination.

Each replacement must still preserve its operation's meaning. Composition separates the choices; it does not remove the need for clear behaviour between them.

← Return to question

UNIT 05 / C2

What is missing from the operation charge(100)?

We need the currency and amount units. We also need an operation identifier, valid inputs, possible results, and failure behaviour. In our example, USD 100 minor units means one dollar. Approved confirms a successful charge; declined confirms rejection; uncertain leaves the result unresolved.

The contract must explain supported retries and status checks. A timeout after submission may hide a successful charge, and adding an operation identifier does not by itself make the provider recognise duplicates.

← Return to question

UNIT 05 / C3

Does SOLID require an interface around every class? Explain with the payment gateway.

No. A gateway interface is useful because it lets payment rules use a common operation while each adapter handles provider details. A second provider can then be added without spreading its message fields through the service.

An interface that only adds another forwarding layer may solve no real problem. SOLID helps examine responsibilities, replacements, and dependencies; interface count is not a measure of design quality.

← Return to question

UNIT 05 / C4

Why is Payment 42 an entity while Money(USD,100) is a value object?

Payment 42 keeps its identity as its status changes. Another payment for the same amount has a different identifier and history. Money(USD,100) is recognised by its amount and currency: another equal value can replace it.

Immutability is a separate choice. Money is often immutable, and a payment can also have immutable snapshots linked by identifier 42. Adding money in different currencies still requires an explicit conversion rule.

← Return to question

UNIT 05 / C5

Which part of a function-based payment design makes the real charge?

The outer code performs the provider call. A pure decision function uses explicit inputs and returns a decision or requested action; it makes no charge itself. The outer code records progress and feeds the observed provider result into the next decision.

Calling the pure function again can recompute its decision. It must not automatically repeat a charge that has already run. The code that performs effects still needs operation identity and a way to resolve uncertain results.

← Return to question

UNIT 05 / S1

Two callers each try to reserve 600 from an account with 1,000 available. Why is reserve(amount) alone insufficient?

The method gives the account one place to own the rule. However, if both callers read 1,000 before either writes, both separate checks pass. Merely moving the code into a method does not stop that sequence.

The shared database must make the check and update atomic. After one reservation leaves 400 available, the second check must see insufficient funds and reject 600. A conditional update or suitable transaction can enforce this rule.

← Return to question

UNIT 05 / S2

A test confirms every private method call, but the order is paid after a decline. What should the test check?

It should check the required outcome: the payment is declined, the order remains unpaid, and shipment does not begin. A matching sequence of internal calls can still produce the wrong business state.

Also test approval and uncertainty. Keep separate tests using the real adapter and database for message encoding, commits, and provider behaviour. A replacement gateway makes replies controllable but cannot prove how the real provider behaves.

← Return to question

UNIT 05 / S3

A refactor changes timeout into decline. Why is that a behaviour change?

Decline means the provider definitely rejected the charge. Timeout means the caller stopped waiting; the charge may already have succeeded before the reply was lost. Replacing uncertainty with decline tells the caller something we do not know.

The caller might then start another payment and charge twice. Preserve the uncertain result and operation identifier until a supported status check or recovery procedure resolves it. Include this case in the refactor's checks.

← Return to question

UNIT 05 / D1

How would you split a class that parses requests, calculates fees, calls a provider, and stores results?

First record the results and state changes for approval, decline, timeout, and storage failure. Extract fee calculation and compare the same inputs and outputs. Then separate request parsing, provider translation, and storage in small steps, checking the same behaviours after each move.

Pass the gateway into the payment service. Keep provider field names in the adapter and amount units in the gateway contract. Preserve uncertainty: a failed local save after a successful remote charge does not undo that charge. The design should make each responsibility easier to locate without changing the caller's promised results.

← Return to question

UNIT 05 / I1

Supply a recording gateway that returns uncertain. What should the small test assert?

Give the gateway a list of received commands. Its charge method appends a copy of the command and returns uncertain. Pass this gateway into pay instead of constructing a real provider inside pay.

Assert that pay returns uncertain and that exactly one command was recorded with operation P7, USD currency, and amount_minor=100. This checks the collaborator boundary. It does not establish persistence, duplicate handling, or the order's state after the result.

← Return to question

UNIT 05 / T1

We replace payment objects with pure functions. Which requirements remain?

The amount units, currency rules, permitted states, operation identity, and outcome meanings stay the same. Given the same state and provider result, the function should make the same business decision.

The outer code must still perform and record effects safely. A timeout can still hide a completed charge, and a database write can still fail after that charge. Changing methods into functions does not combine those separate events or make replay safe automatically.

← Return to question

Unit 6 · Design Patterns, Dependency Injection, and Object Lifecycles

UNIT 06 / C1

A notification service receives a provider SDK through its constructor. Has it necessarily achieved dependency inversion?

It uses DI because another part of the program supplied the SDK. It may still create the provider's request types and interpret its special status codes. Those are source dependencies on provider details.

Define Sender using the application's message and outcome meanings. An adapter translates between that contract and the SDK. Passing this adapter into the service combines injection with a dependency direction that keeps provider details outside the notification rule.

← Return to question

UNIT 06 / C2

Two processes each have a singleton sender. Must it be the same object, and is it automatically safe for concurrent calls?

No. The reuse boundary must be stated. Each process can have its own singleton instance, and separate containers may also have separate instances under their rules.

Sharing an instance does not protect its mutable data. If several threads may call it, its implementation must preserve the contract during those calls. Its owner must also keep it valid until its final use.

← Return to question

UNIT 06 / C3

A factory returns a connection. What must its contract tell the caller besides how to query it?

It must state who owns the connection and when it stops being valid. A fresh connection may be transferred to the caller, which must close it. A borrowed connection remains under another owner and must be used within that owner's boundary.

Closing a borrowed connection too early can interrupt another operation. Failing to release an owned connection can leave fewer connections for later callers.

← Return to question

UNIT 06 / C4

A constructor receives a container, and the service calls resolve() inside its methods. Why are its dependencies hard to see?

The constructor names the container, but not the sender or repository actually used. We must inspect the lookups in the methods to discover those needs. A missing registration may appear only when a particular branch runs.

This is service-locator behaviour. Obtain the needed objects at the assembly or scope boundary, then pass the service its specific dependencies.

← Return to question

UNIT 06 / C5

Each notification chooses email or SMS when it arrives. How can DI and a factory work together?

Startup injects SenderFactory into the notification service. When a notification arrives, the service reads its channel and asks the factory for the corresponding sender. DI supplied the factory; the factory makes the later choice.

The return contract also states whether that sender is application-owned, borrowed from a scope, or newly owned by the caller. The service follows that rule when retaining or closing it.

← Return to question

UNIT 06 / S1

An application-wide service retains request A's database session. Request A ends. What can go wrong during request B?

Request A may close the session while the service still holds its reference. Request B then reaches a closed resource. Retained request data may also belong to the wrong user. This is a captive dependency.

Pass a valid session for each operation, or inject a focused factory that opens an operation scope. Finish the operation before closing its resources. Keeping all sessions open indefinitely does not preserve the intended request boundaries.

← Return to question

UNIT 06 / S2

Two first requests arrive together for a lazily created report generator. What needs to be decided?

Both can observe that no generator exists and try to create one. Use supported coordination so the intended number is created and other callers receive only a fully initialised object.

Define construction failure as well. Will later callers retry, or receive the saved error? Who releases resources from the failed attempt? The first request also bears the construction delay; essential setup may be better completed before reporting readiness.

← Return to question

UNIT 06 / S3

A pool has ten connections. Every failed request keeps one. Why can a later healthy request become stuck?

After one leak, nine connections remain available. Repeated failures can leave all ten held. The later request cannot start its database work even if its own query and the database are healthy.

Use an owning resource block or finally path to return the connection when the operation ends, including supported failure and cancellation paths. Roll back unfinished work as the connection’s reuse contract requires.

← Return to question

UNIT 06 / D1

An application-wide notification service looks up a request connection in a global container and stores it. Propose a clearer arrangement.

There are two problems. The global lookup hides the connection dependency. Storing the connection retains a request-owned resource in an object that outlives the request.

Keep the service's lasting dependencies valid for its lifetime. Pass a valid unit of work into each database operation, or inject a narrow factory that creates an operation scope. The request or job owns that scope, supplies the correct user context, and closes resources after the work ends.

← Return to question

UNIT 06 / I1

Demonstrate transient, scoped, and singleton identity across two scopes. Then show why retaining a scoped resource does not keep it valid.

Resolve a scoped object twice inside scope A and twice inside scope B. The identities should be A1,A1 and B1,B1. Transient lookups should create new instances, while singleton lookups within the stated container boundary should return S. Compare identity rather than equal values.

Give the scoped resource a closed flag and record cleanup on scope exit. Exercise both normal completion and an exception. Retain a reference after exit and show that its use is rejected. Then obtain a fresh resource for a new operation and close it at that operation's end.

← Return to question

UNIT 06 / T1

A saved notification job runs after its creating request and user session have ended. How should it obtain resources and permission to act?

Save the necessary job data and identifiers rather than an open connection or session object. The worker opens its own scope, obtains current resources, loads the data, performs the work, and releases those resources.

The application also needs an explicit permission rule. Some jobs carry out work validly accepted earlier; others must check current permission before acting. Choose that rule for this operation. An old session object cannot silently stand for permission that remains current forever.

← Return to question

Unit 7 · Application Architecture and Domain-Driven Design

UNIT 07 / C1

What makes a monolith modular if it has one deployment?

Its modules have clear responsibilities, controlled interfaces, and data ownership. Orders should use a payment contract rather than alter private payment tables. One deployment keeps calls local but does not prevent useful boundaries. Enforce important dependencies through review or appropriate checks.

← Return to question

UNIT 07 / C2

Why can dependency arrows point differently from runtime calls?

A service can depend on an application-owned repository interface, while a database adapter implements it. At runtime, calls execute the supplied adapter. The source arrow describes knowledge or implementation relationships, not necessarily the physical direction of a message or a separate network service.

← Return to question

UNIT 07 / C3

Why can catalog Product and fulfilment Item be different valid models?

They describe different business work. Catalog may care about descriptions and categories, while fulfilment tracks units and locations. A bounded context identifies where each meaning applies. Translate between them deliberately rather than forcing both into one schema that obscures their rules.

← Return to question

UNIT 07 / C4

How should an aggregate boundary follow the rule it protects?

Group state that must change atomically under a root controlling those changes. An order's line values and total may need one consistency boundary. Making the entire shop one aggregate adds contention. Cross-aggregate work needs explicit coordination rather than an assumption that object references create a shared transaction.

← Return to question

UNIT 07 / C5

What should an architecture fitness check preserve?

A meaningful required property, such as allowed dependencies, migration compatibility, or tenant isolation. It should detect erosion of that reasoned boundary without freezing incidental class names. Update the check when the architecture decision legitimately changes, and keep evidence within the check's actual scope.

← Return to question

UNIT 07 / S1

An order commits, then the process dies before publishing its event. What is missing?

Publication intent existed only in memory or a separate unsafe step. An outbox can commit that intent with the order so a relay can resume later. The relay may publish again after losing its own progress record, so consumers still need stable event identity and duplicate handling.

← Return to question

UNIT 07 / S2

Five services share writable tables and always release together. What independence is missing?

They cannot change their contracts or stored representation independently and can remain tightly coupled operationally. The network split adds failure and diagnosis work without delivering much independent deployment. Clarify ownership and boundaries, or consider a modular consolidation where appropriate.

← Return to question

UNIT 07 / S3

When would extracting a module into a service be justified?

Look for a stable capability with a concrete scaling, fault-isolation, or team-ownership need, plus the ability to operate it. Compare the benefit with network failures, versioning, data coordination, security, and on-call costs. A rapidly changing model alone is not evidence that more process boundaries will help.

← Return to question

UNIT 07 / D1

Several services share tables and require simultaneous releases. Review whether the deployment split is useful.

Map who owns decisions and data, what changes together, and what fails together. A modular consolidation may be simpler, or revised contracts and ownership may justify separation. Compare independent operation with the actual network, migration, and incident costs.

← Return to question

UNIT 07 / I1

Implement shopping modules and one meaningful dependency-boundary check.

Let orders use a payment contract without importing private payment storage or writing its tables. Check that boundary, then replace the provider adapter while preserving the order rule. Avoid a test tied to every internal class name; protect the dependency whose violation would spread provider details or bypass ownership.

← Return to question

UNIT 07 / T1

What evidence would justify extracting payments into a separate service?

Look for stable capability ownership, different scaling or isolation needs, and an operating team prepared for network and version failures. Compare those benefits with added data coordination and recovery. Preserve existing payment invariants and histories through migration. A separate deployment is useful only when the required independence can actually be exercised.

← Return to question

Unit 8 · API Design and Communication Contracts

UNIT 08 / C1

Why give a refund its own identity and status?

A refund can be requested later, authorised separately, and complete or fail independently of the original payment. Its own identity supports amount checks, retries, and investigation. Treating it as an untracked toggle on paid loses the history and uncertainty needed for recovery.

← Return to question

UNIT 08 / C2

What does a keyset cursor improve, and what does it leave unresolved?

It continues after a stable ordered key rather than skipping a shifting row count, avoiding some duplicates and omissions from insertions before an offset. It also needs validation and query/tenant scope. Changing records can still affect results; the cursor does not automatically provide one consistent snapshot.

← Return to question

UNIT 08 / C3

What makes an idempotency key more than a string in a request?

Durable state binds owner, key, payload fingerprint, progress, and result with suitable uniqueness and atomicity. Identical retries recover the same operation; different content conflicts. Retention and provider recovery rules matter too. A separate unprotected lookup and insert can race or forget an external effect.

← Return to question

UNIT 08 / C4

Why does signing in not grant access to GET /payments/42?

Authentication identifies the caller. Authorisation must still check this caller's permitted action on payment 42 using trusted membership and resource scope. Guess-resistant IDs and gateway authentication do not establish ownership. Apply the rule on every relevant query and effect path.

← Return to question

UNIT 08 / C5

How can removing an old event decoder break recovery after a successful release?

Historical logs and backups can still contain that version. A restoring or delayed consumer needs to interpret it even when current traffic uses only the newest shape. Keep supported historical samples and transformations, including meanings, defaults, and unknown values, for the required replay horizon.

← Return to question

UNIT 08 / S1

Does 201 Created for a pending payment mean money was captured?

No. It can mean the payment resource was created successfully while its financial operation remains pending. The body and operation contract describe that later state. HTTP response semantics do not make a provider effect part of the local resource transaction.

← Return to question

UNIT 08 / S2

Four layers each allow three attempts. How much downstream work can one request cause?

Up to $3^4 = 81$ attempts in the nested worst case. Early success can reduce actual work, but failure can trigger amplification precisely when capacity is scarce. Use an overall retry budget and deadline, and safe operation identity for effects.

← Return to question

UNIT 08 / S3

A webhook effect commits but its reply is lost. What should redelivery do?

Recognise the stable event identity and avoid repeating the established local effect. Persist recognition and effect together when that is the protected boundary. An unrelated external call needs its own protocol. The sender may legitimately retry because it did not observe acknowledgment.

← Return to question

UNIT 08 / D1

A payment API returns a Boolean and provides no operation status. Redesign it for an uncertain provider reply.

Create a stable intent identity, explicit pending/approved/declined/uncertain semantics, validated money units, and scoped repeat-key handling. Offer status recovery. Define compatibility and permission as well as field shape so clients know what repeating the request means.

← Return to question

UNIT 08 / I1

Implement keyset continuation and a durable operation-key model for synthetic payments.

Use a stable ordered pair for continuation and test an insertion before a page boundary. Bind actor, key, fingerprint, progress, and result under appropriate atomicity. Identical retries return the old operation; changed payloads conflict. A dictionary lookup followed by an unrelated insert does not protect concurrent or restarted callers.

← Return to question

UNIT 08 / T1

Can finite storage support a permanent promise to recognise every old idempotency key?

Not for an unbounded set of arbitrary distinct operations unless another constraint supplies a bound or persistent evidence elsewhere. Define key retention, expiry, reuse, and client expectations. A permanent business identity may require a durable record rather than an expiring cache. Make the promise fit the information the system can retain.

← Return to question

Unit 9 · Data Models and Storage Technologies

UNIT 09 / C1

What must be decided before choosing a storage family?

State the important reads and changes, accepted-result boundary, invariants, retention, and supported failures. Identify the authoritative copy and recovery procedure. A product category does not establish configured isolation, durable acknowledgment, or usable backups, so verify the selected implementation against those needs.

← Return to question

UNIT 09 / C2

Why is a similar embedding result not proof of truth or permission?

An embedding is a derived numerical representation used to find candidates. Approximate similarity can miss items or retrieve imperfect matches. Preserve source versions, verify required evidence, and enforce access. A vector score describes its retrieval relationship, not factual correctness or authorisation.

← Return to question

UNIT 09 / C3

When is storing an address twice correct?

A profile can hold the current address while a completed order retains the address actually used at purchase. Those records represent different facts. A maintained cache of the current address has another meaning and needs an update policy. State whether repetition preserves history or tracks a changing source.

← Return to question

UNIT 09 / C4

Why is data ownership broader than creating a table?

The owner defines allowed changes and access contracts and is responsible for schema evolution, retention, incidents, and recovery. If every service writes the table directly, they share release and transaction assumptions despite separate repositories. Ownership needs an enforcement mechanism and operating responsibility.

← Return to question

UNIT 09 / C5

What work appears when a second storage engine is added?

Backup and restoration, access policy, schema changes, monitoring, expertise, and reconciliation now cross another boundary. If it is a derived search store, also manage lag and rebuilding. Compare the new capability with that ownership cost rather than assuming a specialised engine is automatically a net improvement.

← Return to question

UNIT 09 / S1

Why do more cold partitions not solve one very busy conversation partition?

The hot conversation's requests still reach its owner. Extra unrelated capacity does not divide that key's work. Measure whether the pressure comes from size, reads, writes, or ordering, then choose supported splitting, read distribution, or a changed data model that preserves the required semantics.

← Return to question

UNIT 09 / S2

Search still holds a deleted private document. What must the system do?

Enforce the required access and deletion policy when serving, even while asynchronous cleanup catches up. Carry source identity and version so old updates cannot recreate an invalid visible record. Track physical cleanup and derived-copy lifecycle rather than treating a successful delete in the source as completion everywhere.

← Return to question

UNIT 09 / S3

What prevents an older backup from restoring deleted content into live service?

Recovery must apply the relevant deletion history and lifecycle rules before exposing restored copies. Track raw and derived data, including indexes and replay paths. A narrowly retained marker can help prevent resurrection, but has its own minimisation and retention needs.

← Return to question

UNIT 09 / D1

A search engine holds the only copy of private document text. Review ownership, recovery, and deletion.

Keep a recoverable source and exact identity/version relationships for derived entries. Enforce access while indexing lags, and track deletion across copies and restores. If search truly owns the authoritative data, it needs the corresponding complete durability and recovery contract rather than assumptions borrowed from a projection.

← Return to question

UNIT 09 / I1

Model profiles, financial entries, chat history, events, and PDFs from their required operations.

List access patterns, rules, identity, retention, and recoverability before choosing stores. Several needs may share an engine. Keep immutable bytes distinguishable from metadata and derived search results. Explain how each copy is rebuilt or recovered, rather than assigning one product family per data type without a reason.

← Return to question

UNIT 09 / T1

How should restoration from a historical backup respect later deletion decisions?

Restore into a controlled path, apply relevant deletion decisions, and reconcile raw and derived copies before serving. Rebuild indexes from the permitted current state rather than blindly expose an older snapshot. Keep deletion evidence narrow and purpose-limited. The backup and live-data lifecycle need a combined recovery procedure.

← Return to question

Unit 10 · Database Internals, Transactions, and Concurrency Control

UNIT 10 / C1

Why might range and equality queries need different indexes?

An ordered tree can locate the first key in a range and continue through neighbouring entries. A hash index locates equality candidates using a hash and does not provide that ordered walk. An ordered tree can also serve equality queries, so equality alone does not require a hash index.

Compare the query conditions, ordering, output size, storage locality, update cost, and indexes supported by the engine. The useful choice is the structure that reduces the actual work for the required operations.

← Return to question

UNIT 10 / C2

Why does an ACID transaction not automatically preserve every business rule?

The application and constraints must express the rule correctly. In our transfer, both account changes can commit atomically even if the application calculates the wrong amount. Atomicity groups the changes; it does not validate the business calculation.

Check the invariant, the code and constraints that enforce it, the selected isolation level, and the durability configuration. Each answers a different question about whether the transaction provides the promised result.

← Return to question

UNIT 10 / C3

Why is MVCC alone insufficient to claim serializability?

MVCC retains versions and chooses which version a reader can see. That mechanism can give both doctors a stable view in which the other is still on call. Each can then update a different row, leaving nobody on call if both changes commit.

No correct serial execution of those checks permits that result: the second doctor would see the first off and stay. Check the isolation and conflict rules that prevent this history, rather than inferring serializability from the existence of row versions.

← Return to question

UNIT 10 / C4

Why state the engine and statement behaviour when discussing isolation?

The same isolation name can have different implementations. A plain read and a locking or updating statement can also have different conflict behaviour. In PostgreSQL 18, ordinary Read Committed SELECTs take statement snapshots; Repeatable Read holds a transaction snapshot and prevents phantoms, but can still allow serialization anomalies.

Use the documented engine, version, level, and statement form when analysing a history. When a transaction fails with a retryable serialization conflict, restart its reads and decisions as part of the whole transaction, using fresh state and bounded attempts.

← Return to question

UNIT 10 / C5

How do serializability and linearizability differ?

Serializability requires committed transactions to have an effect equivalent to some serial transaction order. It permits their instructions to overlap and does not by itself add a real-time ordering condition.

Linearizability treats each operation as taking effect between its invocation and reply and respects real-time precedence between operations. Strict serializability combines transaction serialization with real-time order. None of these properties automatically brings a provider call outside the database into its local transaction boundary.

← Return to question

UNIT 10 / S1

Two doctors each see the other on call and both go off duty. What rule was missed?

At least one doctor must remain on call. That invariant covers both rows. Each transaction relied on an earlier read of the other doctor, so the updates can be individually plausible while their combination is invalid. This is write skew, rather than one transaction overwriting the other's row.

Protect the full decision with suitable serializable transactions, coordinated locking, or a shared conflict point. If serialization rejects an attempt, repeat the whole transaction with fresh reads; the remaining doctor must then stay on call.

← Return to question

UNIT 10 / S2

A conditional seat update changes zero rows. What does that mean?

No row satisfied the condition when the database applied the update, so the seat claim did not occur. In the competing-buyer example, the earlier winner changed the availability or version. More generally, the seat might also be missing; the guarded update alone does not distinguish those causes.

Re-read the relevant state and decide whether to report unavailability, a missing seat, or another supported outcome. Do not report a reservation merely because the SQL statement completed without an error. Success requires the guarded transition and its commit to succeed.

← Return to question

UNIT 10 / S3

A backup job succeeded. What recovery evidence is still needed?

Restore the backup in a usable environment with the required decryption keys, schema, credentials, and recovery logs. Open it through a compatible application and check meaningful relationships, such as a reservation agreeing with its seat owner.

Measure how long full recovery takes and identify how much acknowledged history can be recovered. File creation and checksums are useful checks, but do not establish that the service can restore the required state and serve correctly.

← Return to question

UNIT 10 / D1

Two doctors can each go off call after seeing the other available. Propose a transaction design preserving at least one on call.

One design runs every operation that changes this on-call rule in a serializable transaction. Read the relevant doctors, check that another doctor will remain on call, and only then mark the requesting doctor off. Commit before reporting success. If a serialization failure occurs, retry the complete transaction with fresh reads and bounded attempts. Once one doctor has left, the other's new attempt must refuse to leave.

An alternative uses a shared coordination row for the rota with Read Committed isolation. Every participating transaction locks that row before issuing the reads that check the doctors and making its change. A waiting transaction performs those reads after acquiring the lock, so its new statement snapshot includes the preceding transaction's committed decision. All writers must follow the same protocol. Locking only the doctor being changed leaves the original write-skew history possible. Keep external notifications safe under retries and release them only under a deliberate completion policy.

← Return to question

UNIT 10 / I1

Run the conditional last-seat claim with two connections to one store.

Open two connections to the same database file and create one available seat. Call claim_seat for A through one connection, then for B through the other. Assert that A returns true, B returns false, and a fresh read shows A as the owner. Two separate in-memory databases would not test competing claims on one seat.

Inspect the guarded UPDATE and its transaction: owner changes only where the seat exists and owner IS NULL, and success is based on one affected row. Test rollback of an equivalent guarded update before commit and confirm that another connection can still claim the seat afterward. These checks establish the local ownership rule. Payment, expiry, and a genuinely overlapping concurrency stress test require their own checks.

← Return to question

What changes when one local transaction is divided across independent databases?

The two changes no longer automatically share one commit decision. Suppose the seat update commits in one database but inserting the reservation fails in another. Each store can behave correctly locally while the combined booking is incomplete.

First identify which invariant needs agreement. A distributed transaction may supply a suitable shared decision if the stores and operating requirements support it. Alternatively, a workflow can represent intermediate states explicitly and use retries, reconciliation, or compensation to reach an acceptable outcome. That choice must define what the caller sees while work is incomplete. Separate local commits alone do not preserve the former all-or-nothing business promise.

← Return to question

Unit 11 · Caching and Content Delivery

UNIT 11 / C1

Why does a working set matter more than the full history size for caching?

Reuse concerns the data active in the chosen interval. A small cache can serve a much larger history if most requests reuse a small portion. If the active set exceeds it, eviction and reload can churn. Include bytes, object size, scan traffic, and admission behaviour as well as entry count.

← Return to question

UNIT 11 / C2

How can cache-aside reinsert an old value after invalidation?

A reader fetches version one, then pauses. A writer commits version two and invalidates the key. The reader resumes and inserts its old value. Ordering a source write before cache deletion alone does not prevent this history; stronger freshness needs suitable version or coordination rules.

← Return to question

UNIT 11 / C3

Why can a one-minute TTL serve data older than one minute?

TTL starts when the cache inserts the value. If that value already lagged the source by five minutes, another minute of residence can make it six minutes old. Expiry and source freshness are different measurements. Choose the policy according to the operation's tolerated staleness.

← Return to question

UNIT 11 / C4

For A,B,C,A,D at capacity three, what do ideal LRU and FIFO evict?

Ideal LRU refreshes A's recency, leaving B oldest when D arrives, so it evicts B. FIFO preserves insertion order and evicts A. This fixture explains their difference; real engines can approximate policies and use byte limits or separate expiration rules.

← Return to question

UNIT 11 / C5

When might removing a cache improve the design?

When reuse saves less than lookup, encoding, memory, and maintenance cost, or when a simpler source improvement meets the requirement with fewer freshness risks. Test the empty and unavailable paths. A warm cache can hide an origin-capacity problem, while a wrongly scoped cache can create a data leak.

← Return to question

UNIT 11 / S1

A cache restart sends misses to the origin. How can that load be controlled?

Combine same-key loading where useful, bound refresh workers, apply origin admission, and rebuild gradually. Negative caching and expiry jitter address particular patterns. Do not let every new instance independently scan the whole source. Protect recovery capacity as well as normal cache-hit traffic.

← Return to question

UNIT 11 / S2

A product is created just after “not found” is cached. What should the policy explain?

How long the negative entry may hide the new product and whether creation invalidates it. Distinguish genuine absence from temporary source failure and authorisation denial. Keys and lifetimes must prevent one tenant's response from misleading another.

← Return to question

UNIT 11 / S3

Refresh keeps failing while stale data is served. What limit is needed?

Define fresh, stale-permitted, and unusable ages or another explicit degradation bound. Serving must stop or change once the allowed stale condition ends. Some descriptions tolerate old values, but correctness-critical operations need their authoritative checks. Do not silently extend stale use forever.

← Return to question

UNIT 11 / D1

A cache-aside reader repopulates old data after the writer invalidates it. Define and repair the actual freshness requirement.

Trace old read, new commit, invalidation, delayed old insertion. If that stale result is forbidden, use a supported version or coordination protocol. If bounded staleness is allowed, state its limit and validate authoritative state for critical actions. TTL is not a universal repair.

← Return to question

UNIT 11 / I1

Run ideal LRU, then reproduce stale cache-aside population after invalidation.

A,B,C,A,D at capacity three evicts B. Next pause a reader with version one, commit version two and invalidate, then resume old population. Show the stale result. Apply a justified version or coordination rule if the freshness requirement forbids it; shortening TTL alone does not remove the race.

← Return to question

UNIT 11 / T1

When should a product stop serving rather than return a stale cached result?

Stop when the operation requires current authority or the permitted stale interval has ended. An old description may be allowed, while allocating scarce stock or deciding current access may not be. State which actions degrade and which wait or reject, with visible status and a recovery path.

← Return to question

Unit 12 · Scalability, Load Balancing, and Data Distribution

UNIT 12 / C1

The application becomes slow, so the team adds web servers. Under which conditions would you expect more useful completions, and when might the extra servers mainly create more waiting?

More web servers help when they can share the limiting work. If requests spend their time performing independent CPU calculations, additional compute may increase completions while other required resources still have capacity.

If every request instead waits on one saturated database, disk, lock, or provider, extra web servers can send more work to that same constraint. Trace where requests execute and wait, then compare the useful completion rate before and after a change. The intervention should address the stage that limits the required outcome.

← Return to question

UNIT 12 / C2

A dataset is divided among several storage owners. Explain what changes for routing, queries, and rules that previously fitted inside one database.

The application must now find the owner of each key and account for ownership changes. A query confined to one shard can remain local, while a query spanning shards must contact several owners and combine results. Uniqueness and transactions that cross those owners need more than one shard's local check.

The partition key changes which operations stay together. A tenant key can keep a tenant's records local, but one large tenant can still overload its owner. Choose placement and recovery from the important operations and invariants, rather than assuming that division by itself removes the limit.

← Return to question

UNIT 12 / C3

In the rendezvous example, owner D is added to A, B, and C. Explain which assignments can change, and why this does not establish balanced request traffic.

For each key, A, B, and C keep the scores they had before. Their previous winner therefore remains the best of those three. Adding D either leaves that winner in place or gives the key to D if D has the winning score. In this model, a changed assignment cannot jump from A to B merely because D was added.

This property concerns key placement. It neither transfers records nor makes clients agree about membership. It also says nothing about popularity: many keys can be evenly placed while one frequently requested key sends most traffic to a single owner. Measure key distribution and request traffic separately.

← Return to question

UNIT 12 / C4

A dashboard says that a replica has received an update. Why is that insufficient evidence that a read can observe it, or that it will survive the required failure?

Receiving a change, preserving it durably, and applying it to served state are different milestones. A replica may have received newer log records while its read state still reflects an earlier applied position. A read there can therefore miss the update.

Durability depends on the protocol's required persistence milestone and failure assumptions. Identify the source, receipt, persistence, and apply progress relevant to the decision. A freshness decision needs evidence about visibility; a promotion decision must also honour the acknowledgment and possible-loss contract. The phrase “has the update” leaves these distinctions unresolved.

← Return to question

UNIT 12 / C5

Why should the cost comparison between one larger node and several smaller nodes use the same failure and recovery requirements?

A design that is cheap while every node works may have too little surviving capacity after a required failure. Another may recover quickly but lose more acknowledged data than the requirement permits. Those designs are providing different service, so their healthy-state prices alone are not comparable.

First require each candidate to meet the same workload, response-time, data-loss, and interruption objectives. Then include surviving capacity, copies, coordination, transfer, monitoring, and operating work. Compare cost against successful outcomes, including the retries and recovery work needed to obtain them.

← Return to question

UNIT 12 / S1

One partition receives 70% of writes while the other partitions are mostly idle. Which evidence would help you choose between splitting it, changing placement, and redesigning the hot operation?

Inspect which keys produce the writes, how large the records are, and where the owner spends time. Many independent hot keys may be divisible between owners. One huge record may create a different cost from many small writes. One entity whose rule requires exclusive updates can remain serial even after unrelated data moves away.

A range split helps when incoming work actually divides across the resulting ranges. More cold partitions do not automatically divide one hot key. Choose a change that reduces the observed pressure while preserving the required query results and ownership rule, and measure the hot owner's queue again.

← Return to question

UNIT 12 / S2

During a database slowdown, an autoscaler adds application replicas. Explain how this can increase timeouts even though the application now has more machines.

Each application replica may create another connection pool. More instances can therefore produce more simultaneous database borrowers and more competing queries without adding database capacity. If slow queries or lock waits keep connections busy, new callers accumulate both at pools and inside the database. Retries may add further attempts.

Bound aggregate admission across the instances and include acquisition waiting in request deadlines. Inspect connection hold time, query work, and contention before raising limits. Extra application capacity helps only if it increases useful completions rather than increasing the pressure on the same constrained owner.

← Return to question

UNIT 12 / S3

Two application replicas run on one host. What protection do they provide, and which additional evidence would you need before claiming that they survive a host failure?

One process may continue when the other process alone fails, but losing the host can remove both. The host is a shared failure domain. Process count therefore does not establish machine-failure tolerance.

To survive the required host failure, place suitable serving capacity elsewhere and check its state, routing, and takeover procedure. Inspect other shared dependencies, such as storage, power, configuration, and credentials. Also establish that the survivor has enough capacity. Separated processes provide useful protection only for the failures that do not defeat their remaining common dependencies.

← Return to question

UNIT 12 / D1

After the team doubles application replicas, database timeouts increase. Describe an investigation and a correction that would preserve service while testing the cause.

Start with the request path. Measure offered work, useful completions, connection acquisition time, connection hold time, query execution, and lock waiting. Count the pools across all instances and compare their aggregate demand with the database's connection budget. This can distinguish an application compute limit from pressure created by the new callers.

Bound admission and retain capacity needed for recovery while investigating. If a particular query or contention pattern is responsible, change that access path or resource use where the evidence supports it. Then compare completions, queues, timeouts, and slower responses again. Increasing every pool can merely relocate waiting into the database, so the correction must address the demonstrated cause.

← Return to question

UNIT 12 / I1

Run the provided rendezvous example with owners A, B, C, and then D. Count assignments before and after, and send most simulated requests to one key. What should the two measurements reveal?

For every key whose assignment changes, check that its new owner is D. The existing scores for A, B, and C do not change, so adding D cannot make one of those owners overtake their previous winner. Count keys per owner to inspect placement in this fixture.

Next count requests per owner while most requests use one selected key. That key still selects one owner, so its popularity can produce very uneven traffic despite a reasonable distribution of key counts. Explain what would reduce its actual pressure: suitable read copies, reduced repeated work, or a safe change to write ownership. Moving an assignment alone still requires data transfer and coordinated routing.

← Return to question

UNIT 12 / T1

Every caller competes for one globally hot resource that can have only one exclusive owner. Can adding many partitions make these ownership decisions independent? Explain what a valid improvement would have to preserve.

Extra partitions can distribute unrelated records, but they do not divide the decision about this one exclusive resource. Two partitions independently granting ownership could accept incompatible winners and break the rule. The hot decision still needs a protocol that establishes the permitted owner.

Read copies or batching may reduce suitable surrounding work, and a different allocation model may be possible if the requirements permit it. For any proposal, explain which work it removes or distributes and how it prevents conflicting grants. Measure the hot owner's actual work instead of inferring progress from the number of partitions.

← Return to question

Unit 13 · File Systems, Search Infrastructure, and Data Pipelines

UNIT 13 / C1

Why do acknowledged upload parts not prove the completed file is correct?

Suppose storage has accepted parts 1, 2, 3, and 4. That establishes progress for the identified session, but it does not establish that the parts came from the same source revision or were assembled in the intended order. Completion must name the correct parts and verify the final object's size and integrity under the upload protocol. For example, a source file changed between retries can produce a mixture of versions even though no part is missing. A provider-specific entity tag should be treated as a full-content checksum only if the provider defines it that way for this operation.

← Return to question

UNIT 13 / C2

Why does a content hash not authorise a download?

A digest is computed from bytes. It can help identify content under the algorithm's assumptions, but its computation contains no decision about who may read that content. A tenant who knows another tenant's file hash must still pass the ownership and access checks for the requested reference. Content integrity also does not establish that the file is safe to process. If deduplication exposes whether a supplied hash already exists, it may disclose information across tenants, so its visibility and scope need deliberate rules.

← Return to question

UNIT 13 / C3

What does an inverted index record for “blue car” and “blue bike”?

Let D1 contain “blue car” and D2 contain “blue bike.” The posting list for blue contains D1 and D2. The list for car contains D1, and the list for bike contains D2. A query requiring blue and car can intersect their lists and find D1. This assumes the analyser records those words as the terms shown; different tokenisation or normalisation rules can change the terms. The index also needs source versions and appropriate access handling so that finding a candidate does not expose an obsolete or unauthorised representation.

← Return to question

UNIT 13 / C4

What can go wrong between a snapshot and its CDC stream?

Imagine copying a row's old title, then starting the change stream after the title was edited. The copy contains the old value, and the stream never supplies the intervening edit. A defined snapshot and change position must prevent this gap. The destination also needs stable identities, ordering or version information, supported schema handling, and a way to recognise repeated changes. Even with a complete stream, applying rows separately can expose only part of a source transaction. The handoff and destination serving contract must address the consistency readers require.

← Return to question

UNIT 13 / C5

What provenance makes an extracted field explainable?

Identify the particular output and connect it to the exact file version, page or region, extractor or model version, relevant parameters, and any human review that changed it. The references should let an investigator locate the evidence that was actually used. A confidence score does not supply these origins and does not prove the value correct. Keep the evidence needed for the investigation under an explicit privacy and retention policy; an unbounded copy of personal source text in an audit record creates its own lifecycle problem.

← Return to question

UNIT 13 / S1

Updates and deletes continue during reindexing. What must happen before serving switches?

Build the new generation from a defined snapshot or source position, and capture the changes that follow that boundary. Apply updates and deletions through a validated catch-up point, preserving the intended versions. Check document identities, contents, permissions, and deletion handling, then define how changes arriving during the switch reach the live destination. Select the new generation through the serving pointer only once that handoff is established. Keep a usable, compatible rollback path. Matching counts cannot detect a stale document replacing a missing current one.

← Return to question

UNIT 13 / S2

An event arrives three minutes late. Which time policy matters?

First ask whether the result groups events by when they occurred or by when the processor handled them. An event-time window may need to accept the late event into an earlier interval and revise a published count. Define how long such revisions are allowed, how corrections are identified, and what happens after that allowance ends. Consumers must know whether the first publication is provisional. A watermark estimates progress under lateness assumptions; it cannot prove that an event from an earlier time will never arrive.

← Return to question

UNIT 13 / S3

Why is deleting only the PDF insufficient?

The PDF may have produced extracted text, a thumbnail, search postings, logs, and recoverable copies. Those artifacts can retain its information after the original disappears. Record the deletion intent, locate the relevant copies, and track required removals so failed steps can be retried. A worker already holding the input must also be prevented from publishing after deletion, for example by an atomic completion check against deleted state. Recovery from older data must apply the relevant deletion history before restored content becomes available.

← Return to question

UNIT 13 / D1

A reindex copies records while updates and deletions continue. Design the handoff to a new serving generation.

Keep generation A serving while B is built from a defined source snapshot and change position. Capture changes after that position, including deletes, and apply them to B without letting an older version overwrite a newer one. Track progress to a catch-up boundary. Validate source identities, versions, representative query results, access scope, and deletion outcomes; row counts alone are insufficient. Establish how live ingestion continues across the switch, then select B through an explicit serving pointer or routing version. Retain A only as long as the rollback and retention plan requires, and keep it current enough for any promised rollback. Preserve the source and transformation evidence needed to rebuild again.

← Return to question

UNIT 13 / I1

Implement source-version and attempt checks, then plan a safe index-generation switch.

In Workflow.complete, reject completion if the document is deleted, the supplied epoch differs from the current epoch, or the supplied input version differs from the required input. For epoch 8 and file-version-2, the worked example rejects epoch 7 and file-version-1, then accepts the matching pair. A real store must perform the check and output publication atomically and validate the artifact. Keep transform identity and source evidence as well. For the index switch, build from a defined boundary, catch up updates and deletes, validate versions and access, and maintain a continuous ingestion handoff before selecting the new generation. The local predicate does not implement that wider protocol.

← Return to question

UNIT 13 / T1

How would you investigate an extracted field after both source and model have changed?

Locate the disputed output's own identity first. Follow its provenance to the exact source version and region, the model or transform version, relevant parameters, and any review decision. Compare the field with that retained source evidence. Running the current model on the current document examines different inputs and cannot establish what produced the old value. Even rerunning the recorded version may not reproduce a nondeterministic execution exactly. State that limitation separately from missing provenance, and retain the evidence the investigation requires within the applicable lifecycle policy.

← Return to question

Unit 14 · Distributed Systems Fundamentals

UNIT 14 / C1

A payment provider commits a charge, but its reply never reaches the order service. Why is this a partial failure, and how should the order service proceed?

The provider completed its part while the order service failed to learn the result. The local order may remain pending even though the charge exists. Keep that uncertainty in a durable operation record and query the provider using the stable reference where supported. Then reconcile the local order with the observed outcome. Rolling back local data cannot erase the charge, and blindly submitting another charge can repeat the effect.

← Return to question

UNIT 14 / C2

A Lamport event has value 5 and another has value 9. What can you conclude from those numbers alone? What extra information can a vector comparison provide?

The numbers alone do not prove that event 5 happened before event 9. Lamport's guarantee works in the other direction: if one event happens before another, the later event receives a larger value under the algorithm. Independent events can also receive different values. With correctly maintained vector clocks for the same participant set, componentwise comparison can distinguish a recorded dependency from concurrency. Neither clock decides how conflicting business updates should be merged.

← Return to question

UNIT 14 / C3

A register starts at 1. A write of 2 completes before another client's read begins, and no other write occurs. What must a linearizable read return? How does the answer change if the read overlaps the write?

The later read must return 2 because the completed write must precede it in the allowed order. If the operations overlap, the read may return 1 or 2, provided its result fits a legal common history. We can place the read before or after the write within their invocation-to-response intervals. A write whose reply times out is different from a confirmed failed write: its effect may still need to be included.

← Return to question

UNIT 14 / C4

Two live replicas cannot communicate during a partition. One accepts and acknowledges a write. Explain why CAP prevents promising both atomic consistency and completion of every request reaching either side.

A later read on the other side cannot learn the new value while communication is cut off. Returning its old value can violate atomic consistency; waiting indefinitely or refusing the operation sacrifices the formal availability requirement. CAP concerns these defined properties for the shared object under the partition model. It does not classify an entire product permanently or determine normal-operation latency. Specify which operations wait, reject, or return an explicitly weaker result.

← Return to question

UNIT 14 / C5

Can a booking service prevent double sales while failing to provide useful service? Explain why the progress requirement must be stated carefully.

Yes. It can leave accepted bookings pending forever without ever assigning one seat twice. This preserves the safety rule but violates a promise that accepted bookings eventually resolve. Rejecting every request also avoids double sales, and leaves no accepted booking to violate that particular promise. To rule out this second behaviour, require useful progress for valid requests under stated healthy and reachable conditions. Safety, completion of accepted work, and admission of useful work are distinct requirements.

← Return to question

UNIT 14 / S1

A session reads version 10 of a profile. Its next request reaches another replica and returns version 9. Which guarantee has failed, and what could the serving replica do instead?

If monotonic reads were promised, the second response violates them because the session lost progress it had already observed. Carry a required version or replication position with the session. A replica below that position can wait to catch up, route to a sufficiently current owner, or report unavailability for that promised read. The progress information also needs a defined path through failover and device changes; ordinary sticky routing alone does not cover those transitions.

← Return to question

UNIT 14 / S2

A design uses three fixed replicas, requires two write acknowledgments, and reads two replicas. Explain what R=2, W=2, N=3 proves and what remains to be shown before claiming linearizability.

Because R+W=4 is greater than N=3, every read set intersects every write set. A read therefore contacts a participant from a completed write's quorum, assuming both use the same fixed membership. The protocol must still explain what acknowledgments persist, how versions and concurrent writers are ordered, how reads treat unfinished writes, and how recovery and membership changes preserve that evidence. For example, a read must not expose a pending write and then allow a later read to forget it without an intervening write. Set overlap does not establish those rules.

← Return to question

UNIT 14 / S3

Leader A pauses with epoch 7. Leader B obtains epoch 8 and begins taking over. A later resumes. Where and how should writes from the earlier owner be prevented?

The resource accepting the protected effect must enforce authority. For example, it can atomically check the fencing epoch with each write and reject epochs below the highest one it has accepted. Once epoch 8 is established there, A's epoch-7 request is rejected. The takeover must establish the needed fence before relying on exclusive new-owner effects, and the authority record must survive the failures covered by the contract. A timer in A's process cannot provide this enforcement. Test delayed messages and resumed processes as well as clean crashes.

← Return to question

UNIT 14 / D1

A partition separates three replicas into A and the pair B,C. Both sides receive requests to allocate the same exclusive seat. Propose a policy that preserves one winner, including the steps needed when communication returns.

Use a suitable authority protocol, such as one requiring a majority decision in the fixed three-member configuration, together with enforcement at the seat store. B and C can potentially establish the required decision; A alone cannot issue a new exclusive allocation. Before the new owner's effects are relied on, establish the authority fence so a delayed or resumed older owner cannot also succeed. Requests without valid authority wait or receive an explicit failure. Operations explicitly permitted to use stale information may continue separately. On recovery, discover and retain the authoritative allocation, reconcile pending requests, and bring the old participant up to date before allowing it to act. Two conflicting successful sales cannot be repaired merely by calling them a merge.

← Return to question

UNIT 14 / I1

Use the mechanism collection to evaluate lamport_receive(5,9), vector_relation([1,0],[0,1]), and vector_relation([1,0],[1,2]). Then enumerate the two-member read and write sets of A,B,C and explain their intersection.

lamport_receive(5,9) returns max(5,9)+1=10. The first vector comparison returns concurrent: the first entry is larger in [1,0], while the second is larger in [0,1]. The next returns before because every entry of [1,0] is no larger than the corresponding entry of [1,2], and at least one is smaller. The two-member sets are {A,B}, {A,C}, and {B,C}; any read/write pair shares at least one member. These calculations agree with the supplied code. To claim a storage guarantee, still specify the version order, acknowledgment durability, read handling of unfinished writes, and membership rules.

← Return to question

UNIT 14 / T1

During a partition, exclusive seat allocation cannot obtain valid authority. Which other operations could remain useful, and how should their promises and recovery differ?

A public event description can be served from a cached copy if its permitted age and access rules allow it. A feed read may also continue under an explicitly weaker freshness contract. These responses do not grant a seat, so they can have different serving rules from allocation. The allocation request must wait or fail explicitly where authority cannot be established. Make any stale view clear enough that it is not mistaken for a confirmed booking. When communication returns, reconcile pending allocations against the authoritative decision and resume eligible work without issuing a second winner.

← Return to question

Unit 15 · Replication, Leader Election, and Consensus

UNIT 15 / C1

A primary waits for a replica before reporting success. What must “synchronous replication” specify about that acknowledgment?

Name the required replicas and the event each reply establishes: receipt in memory, durable storage, or application to readable state. These events provide different protection. A record held only in memory can disappear when the replica process restarts, while a durably stored record may survive that restart without yet appearing in local reads.

Also state the failure assumptions and which histories are eligible for promotion. Waiting for a durable copy is useful only if failover preserves the promised history. The extra wait adds latency and can prevent success while a required replica is unreachable.

← Return to question

UNIT 15 / C2

Replicas agree to apply “create task” in the same log position. Why can choosing a random identifier independently on each replica still break the design?

The same ordered command must produce equivalent state from equivalent starting state. If replicas choose different identifiers, their task collections differ despite agreement about command order. Include the chosen identifier in the agreed command, or use a supported mechanism that makes the decision consistent. Reading current time independently creates the same kind of problem.

This is deterministic command application. It also has a boundary: applying one command on several replicas must not produce several unprotected external charges. External effect ownership, duplicate handling, and recovery require their own mechanism.

← Return to question

UNIT 15 / C3

A Raft leader sees an entry on two of three replicas. What else must be established before treating that entry as committed?

The participants must belong to the valid voter configuration, follow the durable-state rules, and have elected an eligible leader under Raft's term and log-comparison rules. For the direct majority-replication rule, the entry being used to advance commitment must belong to the leader's current term. Its commitment also commits the preceding prefix.

An arbitrary earlier-term entry on a majority does not satisfy that rule: a later eligible leader could still replace such an uncommitted entry in some histories. This is why copy counts must be interpreted within the full protocol. Membership changes and reads also need their specified rules.

← Return to question

UNIT 15 / C4

Why can replacing a voting group abruptly with a disjoint group allow conflicting decisions?

The old group can still form one majority while the new group forms another. If those decision sets share no participant carrying the earlier evidence, each group may accept a different history. Having a safe group before the change and another safe group afterward does not make the change safe.

Use the implementation's supported membership protocol. Raft's joint-consensus transition temporarily requires the relevant majority of each configuration before moving to the new configuration alone. Catch new members up appropriately, preserve persistent identities, and test a leader failure during the transition.

← Return to question

UNIT 15 / C5

Two disconnected regions each sell the last item they see. Why can a convergent counter still fail the stock rule?

Both regions start from a view containing one available item and accept a sale without observing the other sale. A merge that retains both decrements can make every replica agree on a negative balance. Agreement has preserved the updates, but the rule “never allocate more items than exist” has already been violated.

Convergence concerns the result of the data type's update and merge rules; it does not enforce arbitrary business invariants. Exclusive allocation needs coordination, separately assigned spending rights, or another design whose allowed operations preserve the stock rule.

← Return to question

UNIT 15 / S1

A worker carrying epoch 7 resumes after epoch 8 has become current for the attempt. Where and how must the old effect be rejected?

The receiver accepting the protected effect must know that epoch 8 is current and reject the request carrying 7. It must compare authority and apply the permitted effect atomically, so authority cannot change between a successful check and the update. The required authority state must survive the failures covered by the contract.

A token recorded only in the coordinator is insufficient if the external store never checks it. Likewise, a local workflow completion check protects that local update; it does not automatically protect a remote file write or payment. Establish how every protected receiver learns and enforces the current generation.

← Return to question

UNIT 15 / S2

A paused leader misses heartbeats and is falsely suspected of failure. What must remain safe while the system attempts recovery?

The suspicion must not allow incompatible accepted decisions or effects. The old leader may still run and send delayed requests. A replacement must obtain authority through the protocol's voting, log-eligibility, and persistence rules; a timeout alone cannot grant that authority. Protected external receivers may also need fencing against the old owner.

Timeout tuning changes recovery speed and how often false suspicions occur. Safety must tolerate those suspicions being wrong. A received heartbeat, meanwhile, does not guarantee spare capacity or that an ordinary request can finish within its deadline.

← Return to question

UNIT 15 / S3

A repaired old primary is reachable again. What must happen before it accepts authoritative writes?

It must rejoin under the current ownership and membership rules and establish a history compatible with the committed history. Depending on the protocol and its stored state, it may need to catch up, repair an uncommitted suffix, reconcile permitted conflicts, or be rebuilt. Its former leader role does not authorise it now.

Keep obsolete writes fenced and apply the application's acknowledged-data policy when considering promotion. Verify routing and readiness before sending write traffic. Test this return path, called failback, as well as the initial failover; resuming an old primary prematurely can create a second inconsistency after the visible outage.

← Return to question

UNIT 15 / D1

A worker pauses beyond lease expiry. A new owner begins work, and the old worker returns. Design protection against stale effects and stale release requests.

Give each ownership grant an ordered generation and a distinct owner identity through an authoritative coordinator. Carry the generation with protected work. At every protected receiver, establish the applicable current generation and atomically compare it with the incoming request before applying the effect. Persist this state as required by the failure model. Once epoch 8 is established there, an epoch 7 action must fail.

Require release and renewal to match the owner and generation, so an old holder cannot remove or extend the new holder's permission. Specify how a new generation becomes known at each receiver; merely issuing a token does not fence an unaware store. Finally, separate stale actions from effects already accepted before the change. Fencing does not undo those earlier effects or make an unrelated external API enforce authority.

← Return to question

UNIT 15 / I1

Implement a receiver that rejects stale attempt epochs. Which update must be atomic, and how would you check its behaviour?

Keep the attempt's current epoch and workflow state in persistent storage. When epoch 8 is established as owner, a completion request for epoch 7 must not alter the state. Perform the epoch comparison and the permitted state transition in one transaction or atomic conditional update. Checking the epoch first and writing in a later unprotected step leaves a race with an ownership change.

Check that completion at 7 fails after takeover at 8, that the valid completion at 8 can succeed, and that restart retains the authority decision. Define repeated completion behaviour according to the workflow's contract. Then identify external effects separately: this receiver protects its own stored transition. Any remote effect that could be caused by the resumed old worker also needs suitable receiver-side authority enforcement.

← Return to question

UNIT 15 / T1

A failed region has been repaired and can receive traffic. What evidence would you require before allowing authoritative writes there again?

Reachability shows that communication is possible; it does not establish authority or a safe history. Require the current ownership and voter configuration, a history compatible with committed decisions, and the durable copies required by the acknowledgment policy. An old leader may need rebuilding and must remain fenced while obsolete authority could still act.

Also verify that routing directs writes to the authorised participants and that the region has enough capacity to resume its assigned work. Apply the supported rejoin procedure and reconcile uncertain external effects. The evidence should cover safe failback under current state, rather than assuming that restoring the old machines restores the old permission to lead.

← Return to question

Unit 16 · Message Queues, Events, and Distributed Workflows

UNIT 16 / C1

An ephemeral subscriber is disconnected when an event is published. Why can it not automatically replay that event on return?

The publication policy may retain no record for an absent subscriber, so there is nothing to replay. Recovery requires retained records and a stored way to locate the next work, such as a log position. Check the broker's actual retention and subscription policy; pub/sub alone does not promise durable replay.

← Return to question

UNIT 16 / C2

A consumer can acknowledge before or after its required effect. How does each order behave if it crashes between the two events?

Acknowledging first can leave the broker finished while the effect is missing. Applying the effect first can leave the effect completed while broker progress is missing, causing another attempt. An atomic inbox and local effect transaction can make that repetition safe. Neither acknowledgment position alone covers an unrelated external provider.

← Return to question

UNIT 16 / C3

A stream processor claims exactly-once behaviour. Which work can its transaction cover, and what must be checked before extending that claim to an email?

Its supported protocol can cover the records and consumed progress it commits atomically, under its fault assumptions. An arbitrary email server lies outside that transaction unless a further protocol covers the send. Name the protected effect and identity scope. Idempotency may give repeated executions one relevant result; it does not mean execution started only once.

← Return to question

UNIT 16 / C4

Why can an outbox relay publish the same event twice even though the order and outbox row committed together?

The relay can publish successfully and crash before recording that it sent the event. Recovery still sees a pending row and publishes again. A stable event identity and consumer duplicate handling address this repetition. The original database transaction saved the obligation to publish; it did not include the entire network delivery.

← Return to question

UNIT 16 / C5

A saga refunds a payment after checkout fails. Why is that compensation not necessarily an undo?

The refund is a further action. It can fail, remain uncertain, happen later, and have different fees or user-visible consequences from the original payment. Persist its progress and reconcile uncertainty. Because some effects cannot be perfectly reversed, the business must define an acceptable recovery outcome.

← Return to question

UNIT 16 / S1

A local effect commits, but the message is delivered again. How can the consumer preserve one local update?

Store the unique event identifier and the local update in the same transaction. If it commits, a duplicate finds the established identity and skips another update; if it rolls back, neither fact is committed. Retain recognition for the supported replay horizon. An external call substituted for the local update needs its own protocol.

← Return to question

UNIT 16 / S2

A worker continues running after its lease expires and a replacement starts. What protects later effects?

Check the current epoch when accepting local completion so an obsolete attempt cannot overwrite the replacement's result. The actual effect receiver also needs suitable fencing or a stable duplicate identity. Lease expiry cannot stop the old process. Distinguish effects already accepted before replacement from later stale attempts, and reconcile outcomes that remain uncertain.

← Return to question

UNIT 16 / S3

A payment remains locally pending after its provider reply is lost. Why should recovery avoid resending it as a new operation?

The provider may already have charged the card. A new identity can create a second financial effect. Query by the original stable operation identity and repair local progress, or use the provider's supported safe retry protocol. Reconciliation establishes what happened to the existing intention; pending local state is not evidence that it never ran.

← Return to question

UNIT 16 / D1

An order commits but its event is never published. Design durable publication and repeated-delivery handling.

Commit the order change and outbox intent in one local transaction. Have a relay retry pending publication with a stable event ID, allowing for a crash after publication but before its sent marker. A subscriber should commit recognition of that event ID with its local effect. Other external effects need their own identity and reconciliation contract.

← Return to question

UNIT 16 / I1

Crash the outbox relay after publication, then process the duplicate with a local inbox transaction.

The fixture should show two deliveries of the same event and one counter change. The unique event identity and local increment must commit together, so a duplicate skips the established effect. If the counter is replaced conceptually by email, identify the missing provider protocol: the local inbox transaction cannot include an arbitrary external send across its failure gap.

← Return to question

UNIT 16 / T1

How could one broker transaction be exactly-once while two emails are sent?

The broker can atomically commit consumed progress and produced records while email remains outside its transaction. A consumer can send an email, lose the completion reply, and send again during recovery. Use the provider's contract, stable identity where supported, and reconciliation. The broker's scoped guarantee does not automatically cover the email effect.

← Return to question

Unit 17 · Reliability, Fault Tolerance, and Disaster Recovery

UNIT 17 / C1

Why are two replicas not necessarily enough surviving capacity?

Assume each replica can meet its target at 100 requests per second. At 70% each, the pair serves 140 requests per second. Losing one leaves capacity for only 100, so the service needs more headroom or a permitted way to reduce or reject work. Continuing to queue the excess will not solve a sustained gap.

Also check independence, usable state, and traffic movement. A second replica cannot take over if the same storage or configuration failure disables it too.

← Return to question

UNIT 17 / C2

Why include pool acquisition in the overall deadline?

The caller spends its permitted lifetime waiting for a connection as well as executing the query. If every stage starts a fresh full timeout, the combined operation may continue after its result is no longer useful. Calculate the remaining time before each stage and release owned resources when work ends.

A timeout still leaves a remote effect uncertain if the request may have reached the provider. Stopping the wait does not establish that the write never happened.

← Return to question

UNIT 17 / C3

What does a circuit breaker help, and what remains unresolved?

An open breaker rejects selected calls promptly, reducing repeated waits and resource use while a dependency is failing. Half-open probes admit limited work to test recovery before ordinary calls resume. The policy must define which outcomes open or close the breaker and bound the probes.

The dependency still needs to recover, and the application still needs an acceptable response. Rejecting or retaining a pending payment can preserve its meaning; fabricating approval cannot.

← Return to question

UNIT 17 / C4

How can separate worker pools still share a failure bottleneck?

Email and payment workers may use separate pools but share one database or host. If email fills the database connection limit or exhausts memory through an unbounded queue, payment can still stop. Trace the resource through which harm spreads, bound its use, and measure saturation and rejection for each allocation.

The combined limits must fit shared capacity. Partitioning also has a cost: one pool can be idle while another rejects work.

← Return to question

UNIT 17 / C5

How do RTO and RPO differ?

RTO is the target time to restore the defined service. RPO specifies the acceptable gap in recoverable data history relative to the incident. A noon incident followed by service return at 12:20 gives twenty minutes of interruption. Recovering data only through 11:55 gives five minutes of data-loss exposure.

Compare these separate measurements with the agreed targets. Count detection, restoration, validation, and traffic return in the recovery journey; measure actual lost records separately from the time-based exposure.

← Return to question

UNIT 17 / S1

How should an end-to-end design limit a retry storm?

First identify which layer owns retries and enforce an overall attempt budget and deadline across the path. Four independently retrying layers can multiply work even if each has a modest local limit. Backoff with jitter spreads timing but does not reduce the configured total by itself.

Bound admission and resource use, keep a stable identity where the receiver supports safe repetition, and track useful intents and completions separately from attempts. Reserve capacity for recovery and for resolving effects whose outcomes remain unknown.

← Return to question

UNIT 17 / S2

Can a default cached balance replace an unavailable ledger?

A stale observed balance may be displayed only under a contract that permits that observation, with the necessary freshness and access rules. A made-up default does not establish the account's state. Neither can supply authority for a financial action that requires a current ledger decision.

A cached help page has a different meaning: an older explanation may remain useful. Decide what the particular substitute permits, which actions must pause, and what the user should be told.

← Return to question

UNIT 17 / S3

Why not send traffic straight back to a repaired primary?

The repaired primary may hold history from before failover and no longer have write authority. It must catch up or be rebuilt from the current history before a controlled handover. Fence any owner that should no longer write, then verify capacity and routing.

Existing connections and cached addresses can still direct calls along old paths. Test failback through the user-visible journey, including an old owner's attempted write, rather than declaring success when its process starts.

← Return to question

UNIT 17 / D1

A provider slowdown holds every connection and triggers layered retries. Break the cascade without inventing business success.

Start with the resource history: slow calls retain connections, queues grow, and retries add work to the reduced capacity. Apply an overall deadline and release owned resources; bound admission, queues, and retries; isolate optional workloads; and use a breaker with limited recovery probes.

Return failure or a valid pending or degraded outcome according to the operation's contract. Preserve identities and track uncertain external effects so they can be reconciled. Reserve recovery capacity and test the fallback under the combined failed traffic, since it can become another bottleneck.

← Return to question

UNIT 17 / I1

Calculate nested retry amplification and design a bounded provider-outage policy.

With three attempts at each of four nested layers, the worst-case downstream allowance is 3×3×3×3 = 81 attempts for one logical request. This includes the initial attempt at each layer; deadlines or earlier success may stop execution sooner.

Give the operation an overall retry budget and deadline, bound admission, and isolate the resources at risk. Use backoff and jitter for retry timing and limited probes for recovery. Return a permitted pending or failure result rather than fabricated payment approval. After the provider returns, reconcile uncertain operations using their retained identities.

← Return to question

How can a disaster drill restore infrastructure yet fail the business recovery requirement?

Machines can run while restored data is too old for the RPO, backups lack usable keys, accepted jobs are missing, or competing owners can still write. Validation and traffic return may also push useful service beyond the RTO even when machine startup is fast.

Test the full business journey with its required state, identities, authority, credentials, and recovery objectives. Include unresolved effects and catch-up work. A successful drill provides evidence for the scenario it exercised, rather than every possible disaster.

← Return to question

Unit 18 · Security Architecture and Privacy

UNIT 18 / C1

Why is an internal network address not proof of a caller's identity?

An internal address tells us something about the route, but does not by itself identify the requesting workload or grant resource permission. A compromised internal worker can use that same route. Validate the caller's credential at the receiving boundary and check the requested action against the actual resource. If identity arrives in forwarded headers, accept those headers only from the configured trusted source.

← Return to question

UNIT 18 / C2

How do OAuth 2.0 and OpenID Connect differ in purpose?

OAuth 2.0 supports delegated permission to access resources. OpenID Connect adds a defined identity layer, including an ID token for the client. In a login journey, the client validates the intended identity result; the API separately validates its access credential. Issuer, audience, expiry, and required flow context matter. An arbitrary access token cannot be substituted for a login assertion.

← Return to question

UNIT 18 / C3

Why use purpose-built password hashing rather than a fast hash?

An attacker with the database can test guesses offline. A fast hash permits many cheap guesses; password hashing deliberately adds work, with salts preventing reuse of the same precomputed result across users. Use a maintained implementation and current deployment-appropriate settings. Store the settings for later upgrades and protect the separate online login and recovery paths.

← Return to question

UNIT 18 / C4

Which paths need tenant isolation beyond the database query?

Follow a document through cache keys, job messages, worker inputs and outputs, search, export objects, download capabilities, telemetry, and administration. Each path needs trustworthy tenant and resource context. If A and B both have document 7, a shared cache key of 7 merges distinct records even when the database query was correct. Include the tenant in the identity and enforce permission on delivery.

← Return to question

UNIT 18 / C5

Does an artifact's valid signature prove its code is safe?

A checked signature can establish the expected signing relationship and that the signed content has not changed, under the scheme's assumptions. It does not establish that the signer, dependency, or build was safe. Connect reviewed inputs to the artifact actually deployed, control build and release privileges, and retain a way to identify and replace affected releases.

← Return to question

UNIT 18 / S1

A tenant A administrator guesses a tenant B document ID. What denies access?

Validate the actor, resolve A's permitted tenant context, and load or select the document under that scope. B's document must fail the resource and action policy: the administrator role applies to A. Apply the same rule to direct download, cache, export, and worker paths so another route cannot bypass the denial. Knowing the identifier and possessing a valid session do not grant the missing permission.

← Return to question

UNIT 18 / S2

A backup survives but its only decryption key is destroyed. What went wrong?

The recovery plan preserved ciphertext but lost a required dependency. The backup needs the key version that encrypted its data, any protected data key, and the credentials or authority needed to use those keys. Test an actual restore with those dependencies before retiring an old version. Keeping the backup bytes alone does not make the data recoverable.

← Return to question

UNIT 18 / S3

Membership is revoked after an export is queued. Where must that policy be enforced?

State the revocation contract first. If revoked members must receive no newly published exports, check current membership at execution and publication as required, coordinate those checks with publication, and durably record denial or cancellation. Protect download according to its own contract. A capability already issued may remain usable until expiry unless the delivery mechanism can revoke it; do not assume the queue-time decision covers these later stages.

← Return to question

UNIT 18 / D1

A private export is authorised at submission but its worker trusts the old tenant field forever. Review the complete journey.

Resolve the requester and permitted tenant from trusted state, select scoped resources, and durably record the export operation. Queue a reference to that operation; let the worker resolve its scope and enforce the chosen current permission policy. Limit its input and output authority, keep the result private, and control publication and download. Track cancellation and deletion so retries cannot publish obsolete output. Record meaningful audit outcomes. Test membership removal between submission, execution, publication, and download, including the behaviour of any capability already issued.

← Return to question

UNIT 18 / I1

Test the synthetic tenant predicate, then trace all data paths using that decision.

The shown assertions permit active actor A to read an A resource and reject reading a B resource. Also test an inactive actor and an action absent from its permissions: both should be denied. These checks test the predicate with supplied attributes, not credential validation or completeness of the attributes. Then trace trusted tenant scope through cache keys, job resolution, object access, export, and search. A correct predicate cannot protect a path that bypasses it.

← Return to question

UNIT 18 / T1

What should revocation mean for an issued object capability and a queued export?

Define when each form of authority stops working. A queued export can check current membership before the later actions required by policy. An already issued object capability may remain usable until expiry unless the delivery mechanism supports earlier revocation. If immediate download denial is required, choose a delivery design that can enforce it rather than relying on expiry alone. Test revocation before execution, before publication, and after link issuance, and describe the resulting lifetime accurately to users.

← Return to question

Unit 19 · Observability, Monitoring, and Incident Engineering

UNIT 19 / C1

How do logs, metrics, and traces complement one another?

Metrics summarise a population, so a latency distribution can establish how widespread a slowdown is. A trace connects one execution's work and waits, helping locate its delay. Logs record particular events and context, such as an attempt's outcome class. Stable operation identities help relate the appropriate records. Compare individual histories with the population, allowing for sampling, dropped records, and collection limits; one trace or missing log does not establish every user's outcome.

← Return to question

UNIT 19 / C2

Why is a trace ID not an access credential?

A trace ID labels diagnostic context. A caller may supply it without proving who they are or whether they may access a resource. Validate or replace incoming context according to policy, then use the intended authentication and authorisation evidence for the action. Propagating tracing context connects evidence about the operation; it grants no resource permission.

← Return to question

UNIT 19 / C3

Why can low CPU coexist with severe request delay?

A request waiting for a connection, lock, storage response, or remote call may use little processor time. RED can show rising request duration while USE exposes pressure on the resource holding up progress. For a full pool, inspect waiting requests and the work occupying its connections. Extra CPU will not by itself release those connections, so locate the delay before choosing capacity to add.

← Return to question

UNIT 19 / C4

What is tenfold budget burn for a 99.9% success target?

The allowed bad fraction is 100% − 99.9% = 0.1%, or 0.001. An observed 1% bad fraction is 0.01, so burn rate is 0.01 / 0.001 = 10. The observed fraction is ten times the allowed fraction for the defined eligible events. State the windows and denominator. Remaining budget and event volume are also needed to judge how long the allowance will last; the ratio alone does not supply that duration.

← Return to question

UNIT 19 / C5

Why avoid unique request IDs as ordinary metric labels?

Each combination of label values identifies a time series. A new request ID can therefore create another series, making the number of series and its storage and processing cost grow with requests. Use bounded metric dimensions for population measurements. Keep detailed operation identities in appropriate logs or traces, with retention and sampling chosen for the investigation they support.

← Return to question

UNIT 19 / S1

A five-second request waits 4.6 seconds for a connection. Where should investigation focus?

Acquisition takes 4.6 / 5 = 0.92, or 92%, of this request's elapsed time. Examine pool occupancy, queueing, connection hold times, and the work holding connections, including relevant transactions or lock waits. Compare normal and slow paths and population measurements before extending the explanation. The trace locates a wait; it does not establish why the pool is full. Faster execution of this path's 100 ms query would remove little of its five-second duration.

← Return to question

UNIT 19 / S2

Serving recovers but charges remain uncertain. Why is recovery incomplete?

A responsive API establishes that new serving work can proceed, but it does not resolve previously accepted payment outcomes. Some operations may still need provider-status reconciliation or required local postings. Follow their durable identities through the supported recovery path to safe final states. Verify those outcomes and assign any remaining recovery work; an infrastructure alert clearing supplies no proof that every accepted charge is resolved.

← Return to question

UNIT 19 / S3

HTTP 202 is fast while jobs wait for hours. Which indicators are missing?

Measure acceptance-to-start delay, stage waits and processing durations, useful completions, and acceptance-to-required-outcome duration. Also track the age of unfinished work, especially the oldest unfinished operation: completed-job durations exclude jobs that never finish. Follow stable operation identity across attempts and stage commits to locate stuck work. HTTP acceptance measures the beginning of this journey and can remain fast while later progress stops.

← Return to question

UNIT 19 / D1

HTTP acceptance stays fast while background jobs stop finishing. Redesign the service indicators and alert.

Define which accepted jobs are eligible, which terminal outcomes satisfy the service, and the relevant age limits. Preserve operation identity across delivery, attempts, stage commits, retries, and final outcomes. Track queue delay separately from processing duration, useful completions, and current age of unfinished work. Break down the stage or owner where progress stops. Alert on actionable completion harm or oldest unfinished age, with an owner and a procedure for locating and recovering the work. Keep detailed operation IDs out of ordinary metric labels; use drill-down records to investigate individual jobs.

← Return to question

UNIT 19 / I1

Build the five-second trace model and an asynchronous completion indicator.

Use the fixture's sequential stages: 300 ms elsewhere + 4,600 ms pool acquisition + 100 ms query = 5,000 ms. Check that pool acquisition accounts for 4,600 / 5,000 = 92%. These assertions check the constructed accounting, not a measured cause of saturation. For jobs, record accepted_at and the required outcome under a durable operation identity. For an unfinished job compute now − accepted_at; for a completed job compute outcome_at − accepted_at. Track useful completions and oldest unfinished age alongside stage wait and processing times. Store detailed identities in suitable records and aggregate with bounded metric dimensions.

← Return to question

What evidence would support a latency explanation beyond correlation with a release?

Locate the extra wait or execution on affected paths and compare them with normal paths and population measurements. If the hypothesis is that a release holds connections longer, inspect changed request behaviour, connection hold times, occupancy, and acquisition waits in the timeline. Consider competing explanations and test a focused intervention under comparable conditions. Support is stronger when the mechanism predicts the symptom and changing that mechanism changes the observed behaviour; a release marker alone establishes timing.

← Return to question

Unit 20 · Deployment, Cloud Infrastructure, and Platform Architecture

UNIT 20 / C1

What important boundary differs between a VM and a typical container?

A VM supplies a guest operating system on virtualised hardware; containers commonly share a host kernel while isolating processes. Both need resource, identity, network, storage, and image management. Check actual isolation and shared failure domains rather than treating packaging as a complete security or reliability guarantee.

← Return to question

UNIT 20 / C2

How can a control-plane outage differ from a serving outage?

Existing workloads may keep serving while new placement or configuration changes cannot proceed. Replacement may need a dependency that current requests do not use. Draw control and data paths separately and define cached-configuration validity. Avoid unnecessary fresh control decisions in each request.

← Return to question

UNIT 20 / C3

Why must a canary receive representative traffic?

A candidate exposed only to cheap reads can appear healthy while failing the expensive checkout path. Select cohorts and indicators that exercise the release risks, with stop conditions and sufficient observations. A small exposure is useful only when it tests behaviour relevant to the broader rollout.

← Return to question

UNIT 20 / C4

Why can an old executable fail as a rollback?

New code may have written data or a schema that the old reader cannot understand. Expand-and-contract preserves an overlap period before obsolete state is removed. Identify when contraction ends safe rollback and what forward repair remains. Reversing traffic does not reverse every stored or external effect.

← Return to question

UNIT 20 / C5

Which duties remain with a team using a managed database?

Schema, access, retention, application transaction use, acknowledgment choices, and validation of recovery still require owners. Provider tasks vary by contract. Test maintenance, throttling, failover, and credential changes instead of assuming “managed” covers every application correctness need.

← Return to question

UNIT 20 / S1

Scale-down kills a worker holding an unrecorded job. What was missing?

Durable accepted-work identity and a safe drain or transfer protocol. Stop new claims, persist progress, and let another valid owner recover as required. Killing a process cannot preserve work that existed only in its memory, regardless of how well the platform schedules replacements.

← Return to question

UNIT 20 / S2

A feature is disabled after writing a new data format. What risk remains?

Old code and clients may still read the new state. A flag changes exposure, not stored compatibility or past effects. Keep migration and reader support explicit, including what rollback or forward repair is supported after the new data exists.

← Return to question

UNIT 20 / S3

Why do two successfully scheduled replicas not necessarily form a correct shared service?

Each may use an unrelated local database or incompatible configuration. Placement establishes running instances, not common state ownership. Define the shared or coordinated data model, identity, and failure behaviour before distributing requests across them.

← Return to question

UNIT 20 / D1

A blue-green release shares one database and removes a column needed by the old version. Review its rollback plan.

Parallel app environments do not make shared state reversible. Use compatible expansion and overlap-safe readers, backfill and verify, then retire old readers before contraction. Mark the end of safe rollback and the forward repair path. Test representative mixed-version traffic.

← Return to question

UNIT 20 / I1

Plan and test an expand-and-contract release with old and new readers.

Add compatible state, deploy overlap-safe readers, backfill with bounded progress, and verify. Retire old readers before removing obsolete schema. Test rollback before that point and identify what requires forward repair afterward. Expose representative traffic to a canary, including important expensive paths.

← Return to question

UNIT 20 / T1

How can rollback restore old code while leaving incompatible data or effects?

Old code can return while new schemas, values, provider calls, and published messages remain. Maintain compatibility during overlap and identify the point where contraction ends safe rollback. Some outcomes need forward repair or compensation. Restoring an artifact cannot reverse every state transition it previously performed.

← Return to question

Unit 21 · Performance, Cost Optimization, and System Validation

UNIT 21 / C1

What must accompany a reusable benchmark result?

Workload, data distribution, hardware or capacity, versions, warmup, concurrency, duration, success definition, variability, and relevant failures. A function microbenchmark cannot establish full application throughput. Preserve enough setup to reproduce and interpret the claim rather than presenting one number without its conditions.

← Return to question

UNIT 21 / C2

How can coordinated omission hide waiting?

A generator waits for a slow reply before issuing later work that independent real arrivals would already have submitted. Some queueing and delay disappear from the observed population. Choose a generator model matching the workload and record offered, completed, failed, and abandoned work.

← Return to question

UNIT 21 / C3

Why is a short measured throughput maximum not a safe production target?

Real operation includes variance, maintenance, background work, and required failure capacity. Near saturation, extra arrivals sharply worsen waiting or rejection. Locate the useful knee and reserve margin against the actual objective rather than running indefinitely at one short benchmark's ceiling.

← Return to question

UNIT 21 / C4

Why can cost per attempt hide increasing cost per useful outcome?

Retries and failures can enlarge the attempt denominator without producing additional completed orders or documents. Define successful business outcomes and the accounting period, while retaining technical breakdowns. Compare candidates with equivalent correctness and service objectives.

← Return to question

UNIT 21 / C5

What should an architecture fitness function protect?

A meaningful required property such as module dependencies, tenant isolation, compatibility, or tested recovery. It should permit harmless implementation changes and be updated when a valid architecture decision changes the requirement. Its pass establishes only its covered evidence, not universal correctness.

← Return to question

UNIT 21 / S1

An index fixes the slow query. Why might sharding have been a poorer first step?

Sharding changes ownership, routing, transactions, and migration while the avoidable scan may remain. A targeted implementation fix can address the measured mechanism with less risk. Architectural change is justified when the underlying limit actually requires it, not because it sounds more scalable.

← Return to question

UNIT 21 / S2

What does killing a process only before work begins fail to test?

It misses gaps such as commit before reply, output write before completion record, or resumed work after lease expiry. Inject faults at the boundary in question and check both serving and required state. Successful restart alone does not establish that accepted work survived.

← Return to question

UNIT 21 / S3

Source and destination counts match but tenant owners differ. Why did validation miss it?

A count compares quantity, not identity or meaning. Compare keys, versions, relationships, ownership, and business invariants as appropriate. State whether the check examines all records or samples. Migration, replay, and recovery need evidence that the right data reached the right scope.

← Return to question

UNIT 21 / D1

A faster and cheaper benchmark configuration omits failures and disables authorisation. Can it replace the original design?

It tests a different workload and service contract. Restore mandatory correctness and access checks, include failed and waiting work, disclose setup, and compare useful outcomes under the original requirements. A favourable number cannot compensate for removing the properties the service must provide.

← Return to question

UNIT 21 / I1

Compare the three fictional architectures against equal requirements and costs.

cost_options totals 371, 551, and 1,402 monthly units. Keep these inputs separate from measured capacity and reliability. Use the constructed latency and independent-component examples only within their assumptions, then state which real tests would establish each candidate meets the original workload and service goals.

← Return to question

UNIT 21 / T1

When could a more expensive architecture be simpler to operate overall?

Compare total work for people as well as the invoice. A managed service may reduce patching, backup, or recovery burden while adding charges and leaving application responsibilities. Evaluate the same correctness and service goals with explicit staffing assumptions. Choose the design the team can understand, change, and recover reliably.

← Return to question

Unit 22 · End-to-End Design, Architecture Reviews, and Evolution

UNIT 22 / C1

A short-link click count is temporarily behind, while two buyers are trying to claim one seat. Why can these operations use different consistency rules?

The delayed count can often be corrected by processing identified click events later. During the delay, the displayed total is less fresh, but no exclusive resource has necessarily been allocated twice.

A seat claim must decide who owns the seat before confirmation. If two callers both win and the records converge later, the service has already broken its promise. State each operation's observable rule and failure behaviour: delayed aggregation may suit the count, while the claim needs one coordinated ownership decision.

← Return to question

UNIT 22 / C2

A chat service returns a persisted receipt, then the recipient disconnects before receiving a live frame. What has the receipt established, and how should delivery recover?

The receipt establishes that the accepted message is retained under the stated storage contract. It does not establish that a recipient device received the frame or that a person read the message.

The returning device fetches history after its known sequence, detects gaps, and recognises repeated messages by identity. Device-delivery and human-read evidence are tracked separately. This makes a missed live frame recoverable without changing the meaning of the persistence receipt.

← Return to question

UNIT 22 / C3

Worker A pauses at epoch 7. Its lease expires, and worker B receives epoch 8. Why does the lease alone not guarantee one external execution?

Lease expiry changes who currently has permission to work; it does not stop A's paused process. A can resume and perform an action alongside B. An atomic current-epoch check rejects A's local completion, preserving local progress.

The external receiver needs supported stable effect identity or enforced fencing to protect its own boundary. Fencing requires the receiver to reject stale authority; merely attaching an epoch is insufficient. An effect accepted before authority changed may still require reconciliation. Neither the lease nor the local completion check by itself establishes one external execution.

← Return to question

UNIT 22 / C4

Two document replicas finish with equal text. What else must be checked before calling the collaborative editor correct?

Equal text establishes only that final comparison for that history. The replicas may both contain an unauthorised edit, invalid rich-text structure, incorrect Unicode treatment, or broken undo behaviour.

First check convergence under the chosen protocol and its supported operation context. Then test access, structural validity, identity-based repetition, and reconnect from snapshots and retained history. Include offline clients whose access has changed. These properties need separate evidence; convergence does not supply them automatically.

← Return to question

UNIT 22 / C5

An extraction worker finishes without an exception. What further conditions may be needed before a document-processing workflow is complete?

The output must be usable, permitted, and linked to its exact source and pipeline versions. It may still need structural validation, human review, or publication. A worker finishing only establishes progress through that attempt.

Track extraction, review decisions, and publication separately. Preserve the evidence, actor, and version needed to explain the result. If deletion or authority changes before publication, current-state checks must still prevent stale output from becoming available. Define complete in terms of the required output journey.

← Return to question

UNIT 22 / S1

A payment succeeds after its seat hold has expired. Another buyer may now own the seat. What may the booking service safely do?

Read the current authoritative ownership and apply the documented recovery policy. The service may reacquire the seat only through a valid claim if it is still available. Otherwise, it must follow a refund or another supported resolution; the successful payment cannot justify issuing against another buyer's confirmed seat.

Keep the provider operation identity and reconcile the financial result. Expiry does not erase a charge that happened externally. Seat ownership and payment recovery must both be resolved without allowing one to overwrite the other's evidence.

← Return to question

UNIT 22 / S2

A provider charges successfully, the process stops before recording its reply, and the client retries. How can the service avoid another logical charge?

Persist intent and stable provider operation identity before submission. On retry, recover that operation rather than inventing a new key. Use the provider's supported safe-repeat or status protocol and reconcile its existing effect before considering another submission.

Apply verified evidence through an idempotent local transition that creates one valid balanced posting set. The local key and transaction protect local records; they cannot impose unsupported deduplication on an arbitrary provider. If available evidence cannot resolve the effect yet, retain uncertainty instead of declaring failure and charging afresh.

← Return to question

UNIT 22 / S3

An epoch-7 extraction worker finishes after the document is deleted or its attempt is superseded. What must stop the old artifact from being published?

At the authoritative completion boundary, check current epoch, input version, and live deletion state together with the state change. Reject the artifact's completion if those conditions no longer hold. A check performed only before computation can become stale while the worker runs.

Publication and derived serving paths must respect the same current lifecycle state. Sending a cancellation message is insufficient because the worker may already be running, paused, or unreachable. The persisted conditional transition supplies the local protection independently of worker timing.

← Return to question

UNIT 22 / D1

Review a short-link service at 100,000 redirects/s when its cache becomes unavailable. What assumption has changed, and what must the design demonstrate?

With a warm 90% hit ratio, only 10% of 100,000 redirects/s reaches the source: 10,000 reads/s before retries or coalescing. Cache loss removes that assumption, so fallback demand can approach the full offered rate.

Define bounded origin admission and the permitted response when capacity is exhausted. Test cold-cache capacity, hot links, and recovery while preserving expiry and revocation rules. Extra cache replicas can reduce some failures, but do not establish safe fallback or immediate invalidation. Record the supported degraded outcome and the evidence for it.

← Return to question

UNIT 22 / I1

Choose one case study and implement its failure exercise. Which invariant will you check, where will you interrupt execution, and what recovered state should result?

Name the exact protected boundary and an observable invariant before writing the test. For example, a short-link creation may commit its mapping and operation result, lose the reply, then return that same mapping on a repeated key without a second creation. A seat exercise can instead check that an old expiry never releases a newer reservation.

Inject the interruption at the relevant boundary and inspect recovered records, including repeats or stale work. Explain why the mechanism enforces the result. Use the provided fixtures as examples, while keeping external guarantees within the actual mock, provider, or editing-protocol contract. A passing local exercise does not establish unsupported external behaviour.

← Return to question

You can fund another region, tested restores, or stronger reconciliation first. How would you choose which work reduces the most important risk?

Start with the most harmful unmet requirement and a failure history showing the gap. If the sole copy can be destroyed, reconciliation cannot recreate its missing bytes. If provider effects have no recoverable identity, another region repeats the same uncertainty. A restore test addresses a different question: whether saved data can become a usable recovered system.

Compare the risk reduced, operating work added, and evidence each option can provide. Record why the selected change addresses the present gap and what observation would justify reconsidering the choice. Additional infrastructure is useful when it repairs the required failure behaviour.

← Return to question

THE REFERENCE DESK

A searchable glossary

Definitions and concept summaries link directly to their complete treatment, including individual patterns.

441 of 441 terms

Abstract Factory	One selected factory supplies a compatible family of related objects. Read the lesson →
Abstraction	Presenting the operations a caller needs while hiding implementation details; the contract still states the behaviour the caller relies on. Read the lesson →
Abstraction, encapsulation, polymorphism, inheritance	Abstraction hides details behind an operation. Encapsulation controls state changes. Polymorphism lets compatible implementations share a calling form. Inheritance derives a class from another class. Read the lesson →
ACID properties	ACID separates atomicity, invariant preservation, isolation, and conditional durability; application rules still need correct expression. Read the lesson →
Active-active vs active-passive	Active-active and active-passive assign serving roles for a particular function, each needing tested transitions. Read the lesson →
Adapter	Adapter translates an existing interface and its outcomes into the contract a caller needs. Read the lesson →

Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy	Structural patterns arrange objects to translate, extend, group, simplify, share, or control access. Read the lesson →
ADR	Architecture Decision Record: a short record of a design choice, why it was made, its consequences, and when to reconsider it. Read the lesson →
Aggregates, entities, value objects, domain services	An aggregate groups changes requiring one consistency boundary, with a root controlling its members' transitions. Read the lesson →
Alert design and alert fatigue	Useful alerts identify actionable harm or impending objective loss with an owner and response. Read the lesson →
Amdahl's Law and bottlenecks	Amdahl's Law bounds fixed-work speedup when part of the job remains sequential. Read the lesson →
Anti-corruption layers	An anti-corruption layer translates external meanings so they do not silently redefine the local model. Read the lesson →
API documentation and discoverability	Useful API documentation explains complete success and failure histories beyond machine-readable message shape. Read the lesson →
API fundamentals and interface contracts	An API is a behavioural communication contract, including identity, effects, failures, timing, and supported change. Read the lesson →

API gateways and BFF	Gateways provide shared boundary policy; BFFs adapt interactions to particular client needs. Read the lesson →
API security and common vulnerabilities	API security follows how accepted input is used, including ownership, injection, resource, and destination risks. Read the lesson →
API versioning and compatibility	Compatibility preserves supported meanings and client behaviour across a planned transition. Read the lesson →
Application architecture	The organization and interaction of application responsibilities and their dependency rules. Read the lesson →
Architecture fitness functions	Architecture fitness checks protect required properties without freezing irrelevant implementation details. Read the lesson →
At-most-once and at-least-once processing	At-most-once can lose required effects; at-least-once can repeat them and therefore needs safe effect handling. Read the lesson →
Audit trails	Audit trails record scoped actors, actions, and actual outcomes while protecting integrity and sensitive context. Read the lesson →
Authentication vs authorization	Authentication establishes identity; authorisation grants a particular action on a resource under policy. Read the lesson →

Authentication vs authorization at API boundaries	Authentication identifies a caller; authorisation checks the permitted action on the particular resource. Read the lesson →
Autoscaling	Autoscaling must include useful signals, startup, draining, durable work, downstream limits, and spending bounds. Read the lesson →
Autoscaling metrics	Autoscaling reacts to workload signals with delay, bounds, warmup, and downstream-capacity constraints. Read the lesson →
Availability	Whether a system can provide the required service when needed. A measurement must define which requests and results count. Read the lesson →
Availability vs durability	Availability concerns useful serving; durability concerns committed data surviving the specified failure model. Read the lesson →
Availability, reliability, durability, consistency	Availability, durability, reliability, and consistency concern different serving, persistence, correctness, and observation requirements. Read the lesson →
Average vs median vs p95/p99 latency	Means and percentiles describe different features of a timing distribution; combined percentiles require appropriate underlying data. Read the lesson →
B-tree/B+tree and hash indexing	Ordered trees support ranges and equality; hash indexes target equality under their implementation's rules. Read the lesson →

Backpressure	A mechanism that communicates or enforces downstream capacity limits on upstream work. Read the lesson →
Backpressure and replay	Backpressure and replay need capacity budgets, retained identity, and useful completion indicators. Read the lesson →
Backup, recovery, and durability	Recovery needs tested backups, logs, keys, compatible applications, and an explicit acknowledged-data policy. Read the lesson →
Backups and restore testing	Usable backups require independent storage, keys, compatible state, and tested application restoration. Read the lesson →
Batch vs streaming pipelines	Batch works on bounded input; streaming maintains continuing progress, with different freshness and recovery costs. Read the lesson →
Benchmark methodology and statistical pitfalls	Benchmark validity depends on realistic input, sufficient observations, honest failures, and a generator that represents arrivals correctly. Read the lesson →
Binary/object storage	Large binary objects and mutable workflow metadata need stable identities and an explicit lifecycle handoff. Read the lesson →
Blameless postmortems	Blameless postmortems reconstruct impact and conditions, with specific owned improvements and verification evidence. Read the lesson →

Blocking vs nonblocking I/O	A blocking read holds the calling thread waiting. Nonblocking I/O lets that thread do other work while data is unavailable. Read the lesson →
Bottleneck identification	Bottleneck diagnosis connects useful outcomes and critical-path waits to constrained resources. Read the lesson →
Boundary	A line that separates the parts being considered from their surroundings. A diagram should explain whether it marks a system, process, machine, or trust boundary. Read the lesson →
Bounded contexts and ubiquitous language	A bounded context defines a model's valid scope; ubiquitous language makes its concepts consistent within that scope. Read the lesson →
Bridge	Bridge keeps two kinds of variation separate by having one object use another through a contract. Read the lesson →
Browser, CDN, application, and database caches	Browser, edge, application, and database caches hold different copies with independent freshness and trust requirements. Read the lesson →
Builder	A builder collects choices in stages and validates them before returning a finished object. Read the lesson →
Bulkheads	Bulkheads partition relevant resources so one workload cannot consume another's entire budget. Read the lesson →

Business logic vs orchestration vs infrastructure	Business logic decides outcomes, orchestration sequences work, and infrastructure performs technical effects. Read the lesson →
Cache consistency and invalidation	Invalidation coordinates copies with source changes; stronger freshness can require versions or additional coordination. Read the lesson →
Cache locality and the working set	Locality and the active working set determine which data can usefully fit and remain in a cache. Read the lesson →
Cache partitioning and replication	Cache partitions and replicas need failure, remapping, fallback, and rebuild-capacity policies. Read the lesson →
Cache stampedes, penetration, and avalanche	Stampedes, absent-key traffic, and broad expiry need controlled loading and origin protection. Read the lesson →
Cache warming and prefetching	Warming and prefetching spend speculative work that should be justified by actual future reuse. Read the lesson →
Cache-aside, read-through, write-through, write-behind	Cache population and write patterns specify ordering and failure gaps; their names alone do not remove races. Read the lesson →
Cancellation and structured concurrency	A parent scope accounts for its children. Cancellation lets them stop and release resources; saved continuing jobs need a longer-lived owner. Read the lesson →

CAP	Under a network partition, the theorem rules out simultaneously guaranteeing its formal consistency and availability properties. It is not a general “pick any two” menu. Read the lesson →
CAP theorem, with its actual assumptions	CAP concerns the incompatibility of defined atomic consistency and availability guarantees under its partition assumptions. Read the lesson →
Capacity and growth planning	Capacity planning connects growth scenarios and resource limits to the time and evidence needed for expansion. Read the lesson →
Capacity monitoring	Capacity monitoring tracks meaningful approaching limits, growth uncertainty, and the time available to respond. Read the lesson →
Capacity reservation vs elastic capacity	Reserved and elastic capacity trade commitment, idle risk, startup delay, and commercial terms against measured demand. Read the lesson →
Cascading failures	Cascading failures spread through resource pressure and retries, including potentially overloaded fallback paths. Read the lesson →
CDN and edge networking	A CDN can reuse a permitted nearby copy. Its cache key, freshness rule, and access rules determine whether reuse is valid. Read the lesson →
CDN architecture and origin shielding	CDN edges and shields distribute suitable representations under explicit key, capability, freshness, and purge rules. Read the lesson →

Chain of Responsibility	Chain of Responsibility passes work through handlers that explicitly stop or continue the request. Read the lesson →
Change Data Capture (CDC)	CDC transports committed source changes through a supported handoff and ordering mechanism. Read the lesson →
Chaos engineering	Chaos exercises test explicit hypotheses with bounded scope, observations, stop conditions, and recovery ownership. Read the lesson →
Checksums and content addressing	Checksums detect corruption under algorithm assumptions; content addressing identifies bytes without establishing permission or safety. Read the lesson →
Choosing storage based on access patterns	Storage selection follows actual queries and invariants, including the complete cost of another engine. Read the lesson →
Choreography vs orchestration	Choreography distributes event reactions; orchestration gives workflow direction and progress an explicit owner. Read the lesson →
CI/CD	CI, delivery, and deployment preserve artifact identity and run checks appropriate to actual release risks. Read the lesson →
Circuit breakers	Circuit breakers limit futile calls and probe recovery without repairing dependencies or inventing valid outcomes. Read the lesson →

Clean Architecture	Clean Architecture directs source dependencies toward policy through application-owned abstractions. Read the lesson →
Cloud vs on-premises trade-offs	Cloud/on-premises comparisons need equal workload, ownership, procurement, recovery, and verified cost assumptions. Read the lesson →
Cohesion	How closely the work inside a module belongs together. Read the lesson →
Command	Command represents a requested action as data so its execution can be handled separately. Read the lesson →
Command-query separation	Queries report information; commands request changes. Clear operation meanings do not require separate read and write infrastructure. Read the lesson →
Component	A part of a system with a defined responsibility. Several components may run inside one process. Read the lesson →
Composite	Composite lets callers use single objects and groups through the same operation. Read the lesson →
Composition root	The startup code that creates the objects an application needs and passes them to the parts that use them. Read the lesson →
Composition roots and DI containers	The composition root connects concrete objects; a DI container can automate their construction and reuse. Read the lesson →

Composition vs inheritance	Composition connects objects with separate jobs. Inheritance is suitable only when the subclass preserves the behaviour callers expect from its parent. Read the lesson →
Compute, storage, networking, and telemetry costs	Total cost follows compute, requests, data, copies, transfer, telemetry, and operating work under explicit rates. Read the lesson →
Concurrency	Multiple activities in progress with potentially interleaved execution; parallel execution is not required. Read the lesson →
Concurrency vs parallelism	Concurrent work overlaps in progress. Parallel work executes simultaneously. Taking turns can still interfere with shared updates. Read the lesson →
Configuration and secret delivery	Configuration and secret delivery need validation, versions, rollout/reload semantics, and a recovery path. Read the lesson →
Conflict resolution and CRDT introduction	CRDTs converge through specified update/merge rules but do not enforce arbitrary business invariants. Read the lesson →
Connection establishment and connection reuse	A handshake prepares a new connection. Reusing a usable connection avoids repeating that setup for every request. Read the lesson →
Connection pool saturation	Pool saturation includes waiting and held connections; aggregate demand across application replicas must remain bounded. Read the lesson →

Connection pooling and keepalive	A connection pool lends reusable connections. Borrowers wait when it is full and must return connections when their use ends. Read the lesson →
Consensus and replicated state machines	Replicated state machines apply agreed commands deterministically; external effects remain a separate boundary. Read the lesson →
Consistency model	A contract specifying which observations of shared or replicated state are allowed. Read the lesson →
Consistency models	A consistency model defines permitted observations across operations, sessions, and state copies. Read the lesson →
Consistent hashing and rendezvous hashing	Consistent and rendezvous hashing limit some assignment changes without guaranteeing even traffic or moving the data itself. Read the lesson →
Constructor, property/setter, and method injection	Constructor, setter, and method injection supply dependencies at different times. Read the lesson →
Consumer acknowledgements	Consumer acknowledgment advances broker progress, with loss or repetition gaps around separately committed effects. Read the lesson →
Consumer-driven contracts	Consumer-driven contracts protect meaningful assumptions of actual clients without freezing incidental formatting. Read the lesson →

Container (C4)	An application or data store in a C4 diagram. It need not be a Docker or other operating-system container. Read the lesson →
Container fundamentals	Containers package execution while runtime controls, persistent-state placement, identity, and shutdown still need design. Read the lesson →
Contract	The rules for how an operation behaves, including valid inputs, results, state changes, and errors. Read the lesson →
Control plane vs data plane	Control planes decide configuration and placement; data planes serve traffic with different startup and failure dependencies. Read the lesson →
Controllers, services, repositories, adapters	Controllers, services, repositories, and adapters are responsibilities, not mandatory class-name conventions. Read the lesson →
Cost per request or transaction	Cost per useful outcome avoids denominators inflated by failed or repeated attempts. Read the lesson →
Cost, security, operational complexity, and maintainability as constraints	Feasible designs must satisfy mandatory security, cost, and operating constraints before preferences are ranked. Read the lesson →
Cost-performance optimization	Cost-performance optimisation compares feasible equivalent service outcomes and checks downstream effects. Read the lesson →

Coupling	The dependencies between parts of a system: what one part must know about or rely on in another. Read the lesson →
CPU, memory, disk, network, and database bottlenecks	Bottlenecks can be computation or waiting at CPU, memory, storage, network, locks, or data services. Read the lesson →
CPU-bound vs I/O-bound workloads	I/O-bound work mainly waits for input or output. CPU-bound work mainly needs instructions to be executed. Read the lesson →
CQRS	CQRS separates models for changing and reading data when their different needs justify the extra work. Read the lesson →
Cursor vs offset pagination	Keyset cursors continue after ordered keys; offsets skip positions that can shift as data changes. Read the lesson →
Dashboards	Dashboards organise evidence for decisions with clear units, denominators, windows, gaps, and ownership. Read the lesson →
Data consistency validation	Data consistency checks compare identities, versions, relationships, and invariants beyond equal counts. Read the lesson →
Data lakes and warehouses	Lakes and warehouses organise raw and curated analytics, requiring ownership, catalog, quality, and lifecycle rules. Read the lesson →

Data minimization	Data minimisation retains information justified by a stated purpose and lifecycle. Read the lesson →
Data ownership	Data ownership includes who changes a fact and who maintains its access, schema, retention, and recovery. Read the lesson →
Data provenance and lineage	Provenance identifies origins; lineage links transformations so outputs can be investigated and corrected. Read the lesson →
Data quality, schema validation, and replay	Quality checks validate meaning as well as shape, while replay needs stable versions and safe effect handling. Read the lesson →
Data retention and lifecycle	Retention follows raw and derived copies through expiry, archival, deletion, replay, and recovery. Read the lesson →
Database connection pooling	Database pools reuse bounded connections, with acquisition, hold time, and downstream contention included in deadlines. Read the lesson →
Database proxying	Database proxies add routing or pooling facilities whose session-state and retry contracts need verification. Read the lesson →
Deadlocks, starvation, livelocks	Deadlock waits in a cycle; starvation denies one caller progress; livelock repeats activity without completing the work. Read the lesson →

Decision matrices and trade-off analysis	A decision matrix makes weighted judgments explicit; it does not replace correctness evidence or measurements. Read the lesson →
Decorator	Decorator adds behaviour around an operation while preserving its interface; wrapper order can change the meaning. Read the lesson →
Defense in depth	Defense in depth combines complementary controls rather than repeating one unverified assumption at several layers. Read the lesson →
Delivery guarantees	Delivery guarantees need a stated record or effect boundary and do not automatically compose across external systems. Read the lesson →
Dependency cycles and captive dependencies	A cycle blocks construction; a captive dependency can survive construction but be retained beyond its valid lifetime. Read the lesson →
Dependency injection	Supplying collaborators to an object or function rather than having it locate or construct them internally. Read the lesson →
Dependency Injection (DI)	DI supplies a dependency from outside the object or operation that uses it. Read the lesson →
Dependency inversion principle vs DI	DI concerns how a dependency is supplied; DIP concerns which contracts the code depends on. Read the lesson →

Dependency rules and module boundaries	Module dependency rules limit knowledge and bypass paths, using review and useful enforcement where appropriate. Read the lesson →
Dependency, dependency direction, and inversion	A dependency is something code needs; a contract states how that dependency may be used. Read the lesson →
Deployment topologies and networking	Deployment topology places execution, state, routes, trust, and failure domains on real infrastructure. Read the lesson →
Designing data-intensive applications	Data-intensive design connects versioned transformations with explicit authority, validation, progress, and recovery. Read the lesson →
Development vs staging vs production	Environments have different purposes while relevant protocols, state transitions, and configuration meanings need deliberate compatibility. Read the lesson →
DI vs factory pattern	DI can supply a factory that selects or constructs a dependency when later input is available. Read the lesson →
DI vs service locator	A service locator obtains dependencies through lookup; explicit injection makes the receiver's needs visible. Read the lesson →
Dirty reads, nonrepeatable reads, phantom reads	Dirty, nonrepeatable, and phantom reads describe different concurrent histories whose occurrence depends on isolation implementation. Read the lesson →

Disposal, cleanup, and resource lifecycles	Structured cleanup returns owned resources when work ends, including supported error and cancellation paths. Read the lesson →
Distributed coordination	Coordination establishes shared decisions under an explicit failure and progress protocol. Read the lesson →
Distributed locks and leases	Locks and leases grant permission with identity and expiry rules, while resumed stale effects may still need fencing. Read the lesson →
Distributed monolith antipattern	A distributed monolith has multiple processes without useful independent change or operation. Read the lesson →
Distributed tracing	Tracing relates spans while sampled executions still need comparison with wider timing and error evidence. Read the lesson →
DNS resolution and DNS caching	DNS finds records for a name. Cached answers can remain in use until their permitted lifetime ends. Read the lesson →
Document databases	Document storage groups nested data by identity, with explicit size, schema, index, and cross-document rules. Read the lesson →
Domain events	A domain event records a meaningful accepted fact; durable publication needs a separate explicit mechanism. Read the lesson →

Domain models, entities, value objects	An entity is recognised by identity; a value object by its value. A domain model gives business concepts and their rules a clear representation. Read the lesson →
Domain-Driven Design (DDD)	Domain-Driven Design develops a shared business model with domain experts where rules and meanings need clarification. Read the lesson →
DRY, KISS, YAGNI	DRY avoids duplicate knowledge. KISS favours an understandable solution. YAGNI defers speculative features while keeping present requirements complete. Read the lesson →
Durability	Whether committed data survives the specified failures, such as a process restart or loss of a disk. Read the lesson →
Durable job execution	Durable jobs retain identity, input, state, and attempt authority beyond the initiating process. Read the lesson →
Encapsulation	Controlling access to internal data so callers use the operations that protect its rules. Read the lesson →
Encryption in transit and at rest	Transit and storage encryption protect different boundaries under key/access assumptions and do not replace authorisation. Read the lesson →
ER diagrams and relational modeling	ER modelling identifies keys, relationships, cardinalities, constraints, and record lifecycles. Read the lesson →
Error budget	The allowed unreliability implied by a chosen SLO over its measurement window. Read the lesson →

ETL vs ELT	ETL transforms before target loading; ELT transforms after loading, changing raw-data placement and governance work. Read the lesson →
Evaluating a design against its original requirements	Design validation traces original requirements to tests, measurements, reviews, and clearly unresolved assumptions. Read the lesson →
Event loops, tasks, futures, promises	A task can pause on an unfinished future. Completion makes it ready; the event loop must still give it a turn. Read the lesson →
Event sourcing	Event sourcing makes interpretable accepted events authoritative, with snapshot, correction, privacy, and replay obligations. Read the lesson →
Event-driven architecture	Event-driven components react independently, with delivery, ordering, replay, and progress obligations. Read the lesson →
Eventual consistency	A convergence property under the model's assumptions; it alone gives neither an immediate freshness bound nor a complete conflict policy. Read the lesson →
Eviction: LRU, LFU, FIFO, and alternatives	LRU, LFU, and FIFO rank eviction differently; admission, bytes, expiry, and approximations also affect behaviour. Read the lesson →
Evolutionary architecture	Evolutionary architecture changes through evidence while checks protect important required properties. Read the lesson →

Exactly-once processing	A scoped guarantee about effects or records within a defined system boundary and assumptions. It does not automatically include arbitrary external side effects. Read the lesson →
Exactly-once processing within defined boundaries	Exactly-once guarantees cover only the supported atomic boundary and its identity, retention, and failure assumptions. Read the lesson →
Facade	Facade provides a focused entry point to several cooperating operations with clear result meanings. Read the lesson →
Factory Method	A subclass overrides a creation operation while the surrounding processing steps stay shared. Read the lesson →
Factory Method, Abstract Factory, Builder, Prototype, Singleton	Creational patterns organise how objects are selected, assembled, copied, or shared. Read the lesson →
Failover and failback	Failover and failback transfer traffic or authority while preserving current history, fencing, and capacity. Read the lesson →
Failure detection	Failure detection produces suspicions under timing assumptions; authority safety must tolerate false suspicion. Read the lesson →
Failure domain	Elements exposed to a common cause of failure, such as a process, host, disk, or region. Read the lesson →

Failure domains and fault isolation	Failure domains identify shared causes; isolation limits their spread through resources and dependencies. Read the lesson →
Fallbacks	Fallbacks substitute defined business outcomes, with their own access, duration, capacity, and recovery risks. Read the lesson →
Fault models	A fault model specifies failures and persistence assumptions a protocol claims to tolerate. Read the lesson →
Fault-injection tests	Fault injection targets meaningful interruptions and validates both useful recovery and required state. Read the lesson →
Feature flags	Feature flags control exposure with ownership and safe defaults, but do not reverse stored effects or replace permission. Read the lesson →
Flyweight	Flyweight shares suitable unchanged data while each use supplies its own changing context. Read the lesson →
Functional programming alternatives to object-centric design	Pure functions expose decisions as inputs and outputs. Separate code performs and records effects, including recovery from uncertain results. Read the lesson →
Gathering and prioritizing requirements	Requirements state necessary outcomes and constraints; prioritisation identifies what matters most and what still needs evidence. Read the lesson →

Golden signals	Golden signals connect latency, traffic, errors, and saturation using meaningful eligible outcomes. Read the lesson →
Graceful degradation	Graceful degradation preserves a permitted subset of behaviour with visible freshness and action limits. Read the lesson →
Graph databases	Graph storage supports relationship traversal whose depth, degree, cost, and access scope need limits. Read the lesson →
Health checking	Health probes support bounded local decisions rather than establishing the complete business service guarantee. Read the lesson →
Health checks and failover	Readiness, liveness, and failover make different decisions and require state and capacity beyond a probe response. Read the lesson →
Hexagonal/Ports and Adapters	Hexagonal architecture uses application ports and adapters to separate core operations from external mechanisms. Read the lesson →
High cohesion and low coupling	Cohesion concerns related work within a module. Coupling concerns what one module must know about another. Read the lesson →
HLD	High-Level Design: a view of a system's main parts, where data is kept, and how the parts work together. Read the lesson →

Horizontal scaling economics	Horizontal scaling adds baseline and coordination costs whose usefulness depends on divisible work and shared limits. Read the lesson →
Hot keys	A hot key concentrates work on an owner and needs a remedy suited to its actual read or write pressure. Read the lesson →
Hot partitions and skew	Hot partitions require diagnosis of popularity, size, and ownership pressure rather than more unrelated capacity. Read the lesson →
How databases store and retrieve records	A record lookup performs physical and visibility work through indexes, pages, memory, and storage. Read the lesson →
How the internet works from the application perspective	A browser sends an HTTP request for a resource, and the server returns a response that the browser can display. Read the lesson →
HTTP/1.1, HTTP/2, HTTP/3	HTTP/2 shares a TCP connection between streams; HTTP/3 uses QUIC streams. HTTP request and response meanings remain. Read the lesson →
Idempotency	Repeated application of an operation has the same relevant effect as one application, under its defined identity and scope. Read the lesson →
Idempotency and idempotency keys	Idempotency keys bind repeated attempts to one logical operation through scoped durable identity and result rules. Read the lesson →

Idempotent consumers	Idempotent consumers commit recognition and the protected local effect together, retaining identity for supported replay. Read the lesson →
Immutability and side-effect management	Immutable values cannot be edited after creation. Pure functions calculate from inputs without external effects. Provider calls and writes still need explicit execution and recovery. Read the lesson →
Immutable infrastructure	Immutable infrastructure replaces defined instances, while runtime state, migrations, and emergency drift remain managed concerns. Read the lesson →
In-process vs distributed caches	In-process caches duplicate local copies; distributed caches add a shared network boundary and its failure behaviour. Read the lesson →
Incident analysis and prevention	Incident analysis links contributing mechanisms to verified corrective changes without substituting blame for explanation. Read the lesson →
Incident response	Incident response coordinates mitigation, decisions, communication, and remaining uncertain business outcomes. Read the lesson →
Index selection and query optimization	Indexes improve selected reads while adding space, maintenance, and write costs. Read the lesson →
Indexing pipelines	Indexing stages need identified inputs, repeat-safe output, validated completion, and an owner for failed progress. Read the lesson →

Infrastructure as Code	Infrastructure as Code makes desired resources reviewable without eliminating state, privileges, drift, or destructive-change risks. Read the lesson →
Infrastructure ownership and managed services	Managed services transfer selected duties while clients retain responsibilities defined by their actual contract. Read the lesson →
Interface	The operations a component offers to its callers. Its contract explains how those operations behave. Read the lesson →
Interfaces and contracts	An interface names operations. A contract explains valid inputs, result meanings, effects, and failure or retry behaviour. Read the lesson →
Invariant	A rule that must hold whenever the system is in a valid state, such as unique task identifiers. Read the lesson →
Inversion of Control (IoC)	IoC lets a framework or coordinator control when some application code runs. Read the lesson →
IP addresses, ports, sockets, routing	An IP address leads toward a machine. Its operating system uses ports and sockets to deliver data to the right program. Read the lesson →
Isolation levels and implementation differences	Isolation behaviour depends on engine, version, and statement form; retries require fresh whole-transaction reasoning. Read the lesson →

Iterator	Iterator supplies items in sequence while hiding the traversal details and respecting resource lifetimes. Read the lesson →
Key management and rotation	Key management tracks data dependencies through creation, rotation, historical recovery, and destruction. Read the lesson →
Key-value databases	Key-value storage serves keyed operations; other queries and expiry need their own supported structures and policy. Read the lesson →
Kubernetes fundamentals	Kubernetes reconciles workload resources; application transactions, access, backups, and service objectives remain explicit responsibilities. Read the lesson →
L4 vs L7 load balancing	L4 routing uses transport addresses and ports. L7 routing can use HTTP content such as the requested path. Read the lesson →
Lamport clocks and vector clocks	Lamport clocks preserve causal order in one direction; vector comparisons can identify concurrency under their model. Read the lesson →
Large-file upload architecture	Direct large-file upload reduces application byte handling while retaining permission, quota, and completion responsibilities. Read the lesson →
Latency	Elapsed time for an operation under a specified measurement boundary. Read the lesson →

Latency caused by network round trips	The four assumed stages take 20 + 40 + 40 + 55 = 155 ms. Reusing setup leaves the same 55 ms request stage. Read the lesson →
Latency, throughput, QPS, RPS, TPS	Latency is elapsed time per operation; throughput is completed work per interval. Define which queries, requests, or transactions are counted. Read the lesson →
Law of Demeter	The Law of Demeter limits knowledge of distant objects. A caller should use a useful operation instead of depending on an internal lookup path. Read the lesson →
Layered/N-tier architecture	Layered architecture separates kinds of responsibility; physical tiers are a separate deployment choice. Read the lesson →
Lazy initialization	Lazy initialisation moves construction to first use, together with its delay and possible failure. Read the lesson →
Leader/follower replication	Leader/follower replication orders updates, while receipt, persistence, commitment, and application can mark different progress. Read the lesson →
Leaderless replication	Leaderless protocols reconcile several replicas with implementation-specific version, repair, and membership rules. Read the lesson →
Least privilege	Least privilege limits effective identity capabilities to the work and resources actually needed. Read the lesson →

Lifecycle policies and retention	Lifecycle rules follow originals, intermediates, indexes, logs, and recovery copies through required deletion. Read the lesson →
Linearizability	A correctness condition in which operations appear atomic and respect real-time precedence. Read the lesson →
Little's Law and queueing intuition	Little's Law relates stable long-run averages: work in progress equals rate times residence time within one boundary. Read the lesson →
LLD	Low-Level Design: the internal detail of a component, including its functions, classes, interfaces, and algorithms. Read the lesson →
Load balancers vs API gateways	A load balancer chooses an application instance. An API gateway can perform common API checks before application work. Read the lesson →
Load distribution algorithms	Balancing algorithms distribute selections using different load clues whose usefulness depends on request and connection behaviour. Read the lesson →
Load shedding	Load shedding rejects selected work early to preserve useful higher-priority outcomes under overload. Read the lesson →
Load, stress, spike, and soak testing	Load, stress, spike, and soak tests answer different expected-load, limit, abrupt-change, and long-duration questions. Read the lesson →

Locking and optimistic concurrency control	Locks coordinate conflicting access; optimistic checks validate expected state before applying changes. Read the lesson →
Log ordering and durability	A log entry's presence, durability, commitment, and application are distinct claims during replication and recovery. Read the lesson →
Logs, metrics, and traces	Logs record events, metrics aggregate populations, and traces connect execution work and waits. Read the lesson →
Lost updates, write skew, serialization anomalies	Lost updates overwrite work; write skew can violate a multi-row rule even when separate rows are protected. Read the lesson →
LSM trees and write amplification	LSM organisation batches sorted writes and compaction, trading ingestion behaviour against read work and amplification. Read the lesson →
Mediator	Mediator places a focused coordination rule between collaborators instead of linking each directly to the others. Read the lesson →
Membership changes	Membership changes need a supported transition preserving decision safety between old and new configurations. Read the lesson →
Memento	Memento captures supported local state so its owner can restore it later. Read the lesson →

Memory visibility and happens-before relationships	Safe publication uses the language's synchronisation rules so the reader can observe the completed value. Read the lesson →
Memory, stack, heap, and object lifetimes	The call stack records unfinished calls. Objects can outlive a call when later work still needs them. Read the lesson →
Metadata vs binary data	Immutable source versions keep changing annotations and processing attempts traceable to the bytes they used. Read the lesson →
Modular monoliths	A modular monolith has one release unit with enforced internal contracts and data ownership. Read the lesson →
Modularity	Organising a system into parts with clear responsibilities and interfaces. Read the lesson →
Monitoring vs observability	Monitoring follows known signals; observability helps explain behaviour through correlated outputs and instrumentation. Read the lesson →
Monolith vs microservices	Microservices seek useful independent deployment and operation while adding network, version, and coordination work. Read the lesson →
Multi-environment configuration	Environment values should be validated and explicit without silently changing business meanings. Read the lesson →

Multi-leader replication	Multiple write leaders need valid conflict semantics, not just deterministic convergence that discards business meaning. Read the lesson →
Multi-region replication trade-offs	Regional replication needs authority, acknowledgment geography, conflict, failover, and safe return-to-service rules. Read the lesson →
Multi-region strategies	Multi-region operation needs write authority, read freshness, loss, latency, and rejoin policies. Read the lesson →
Multi-tenant isolation	Tenant isolation must hold across queries, caches, jobs, objects, search, exports, telemetry, and administration. Read the lesson →
Multi-zone and multi-region deployments	Zone and region placement address different correlations while adding propagation, cost, and recovery dependencies. Read the lesson →
Multipart and resumable uploads	Resumable or multipart upload tracks confirmed pieces and separately verifies the completed object's identity and integrity. Read the lesson →
Mutexes, semaphores, atomics, condition variables	A mutex gives exclusive entry; a semaphore counts permits; an atomic operation protects one action; a condition variable lets a waiter recheck shared state. Read the lesson →
MVCC	Multiversion concurrency control: retaining versions to help coordinate concurrent database access. Exact visibility and isolation depend on implementation. Read the lesson →

NAT, firewalls, ingress, egress	NAT translates addresses and remembers return mappings. A firewall allows or blocks traffic. Ingress enters a boundary; egress leaves it. Read the lesson →
Negative caching	Negative caching stores a scoped absence or failure meaning for a deliberate lifetime. Read the lesson →
Network partitions, timeouts, packet loss	A timeout leaves the remote outcome uncertain. A recorded operation identity can let the client find or retry the original operation. Read the lesson →
Network segmentation	Segmentation restricts network reachability while application identity and resource permissions remain necessary. Read the lesson →
Network uncertainty	Network delay and failure can appear alike; timeouts bound waiting but cannot prove a remote process stopped. Read the lesson →
Nonfunctional requirement	A requirement about the quality or conditions of a system's behaviour, such as response time or persistence. Read the lesson →
Normalization vs denormalization	Normalisation reduces repeated facts; deliberate copies require a clear historical or maintained-current meaning. Read the lesson →
OAuth 2.0 and OpenID Connect	OAuth delegates access; OpenID Connect adds identity assertions with flow-specific validation. Read the lesson →

Object collaboration	Object collaboration follows requests, results, and ownership. A coordinator tracks progress; an error does not undo earlier effects. Read the lesson →
Object creation vs object ownership	Ownership determines who releases a resource; a reference alone does not transfer that responsibility. Read the lesson →
Object storage vs block vs file storage	Object, block, and file interfaces expose different byte-access and lifecycle contracts. Read the lesson →
Observability across asynchronous workflows	Asynchronous observability follows accepted work through later attempts, stage waits, and required final outcomes. Read the lesson →
Observer	Observer calls subscribed listeners when an event occurs under explicit timing and failure rules. Read the lesson →
OLTP vs OLAP	OLTP serves short operational transactions; OLAP serves analytical scans and aggregation, often with different interference costs. Read the lesson →
On-call and runbooks	On-call needs useful alerts, access, escalation, rehearsed runbooks, and service ownership that reduces recurring toil. Read the lesson →
Onion Architecture	Onion Architecture organises inward policy dependencies; concrete contracts matter more than diagram shape. Read the lesson →

Ordering guarantees	Publication, delivery, execution, and effect order can differ when work proceeds concurrently. Read the lesson →
PACELC as a useful extension	PACELC highlights partition choices and ordinary latency/consistency costs as a design lens, not a complete protocol. Read the lesson →
Pages, records, indexes, and buffer pools	Pages organise records; buffer pools cache pages, while query locality determines which work can be avoided. Read the lesson →
Pagination, filtering, sorting	Pagination bounds query and response work using defined order, filtering, size, and access rules. Read the lesson →
Partial failures	Partial failure permits one participant's effect to succeed while another fails to observe or record it. Read the lesson →
Partitioning and sharding	Sharding distributes partitions among owners, adding routing, query, transaction, and movement requirements. Read the lesson →
Partitions, consumer groups, offsets	Partitions provide ordered lanes; consumer groups and offsets track ownership and progress within those lanes. Read the lesson →
Password hashing	Password hashing uses a purpose-built salted work mechanism with maintained implementations and deployment-appropriate parameters. Read the lesson →

Paxos: conceptual foundations	Paxos preserves chosen decisions through numbered proposals and required prior evidence across intersecting majorities. Read the lesson →
Peak vs average load	Peak load and its duration can differ substantially from an evenly distributed daily average. Read the lesson →
Performance budgets	Performance budgets allocate acceptable end-to-end costs under a workload and remain targets until measured. Read the lesson →
Performance tuning vs architecture changes	Tuning changes implementation costs; architecture changes alter major ownership or placement and need proportionate evidence. Read the lesson →
Physical vs logical clocks	Physical clocks estimate time; logical clocks represent ordering relationships without necessarily measuring seconds. Read the lesson →
Platform engineering basics	Platform engineering offers reusable operating capabilities whose limits and ownership remain visible to developer users. Read the lesson →
Poison messages	Poison messages need diagnosis and controlled quarantine rather than unlimited identical retries. Read the lesson →
Polyglot persistence and its operational costs	Polyglot persistence combines engines for useful capabilities while adding operating and cross-store coordination work. Read the lesson →

Primary/replica architectures	Primary/replica topology needs explicit write authority, acknowledgment, promotion, and stale-owner protection. Read the lesson →
Privacy, retention, and deletion	Privacy lifecycle spans collection, access, correction, retention, deletion, and historical recovery of copies. Read the lesson →
Producers, consumers, brokers	Producers, brokers, and consumers acknowledge different stages that must be named to reason about ownership. Read the lesson →
Profiling	Profiling locates runtime work or waits under measured conditions, which must be related to user outcomes. Read the lesson →
Profiling and benchmarking	Profiles locate runtime work; benchmarks compare controlled conditions with disclosed workloads, versions, variation, and outcomes. Read the lesson →
Program vs process vs thread	A program is saved instructions. A process runs them. Threads within a process execute work and can share data. Read the lesson →
Prototype	Prototype creates an object by copying configured state with explicit rules for copied and shared values. Read the lesson →
Proxy	Proxy represents a target through its interface while controlling access, creation, or location. Read the lesson →

Public vs internal APIs	Internal APIs still cross trust, version, and failure boundaries; public APIs add explicit audience and support commitments. Read the lesson →
Query and application performance diagnosis	Diagnosis separates waiting from execution and tests proposed bottleneck mechanisms against evidence. Read the lesson →
Query planning and EXPLAIN	A query planner chooses physical execution using estimates; compare plans with actual work where safely supported. Read the lesson →
Queue saturation and throughput ceilings	Saturation limits useful completion; added arrivals become waiting or rejection, requiring operating margin. Read the lesson →
Queueing and overload	Queues absorb temporary mismatch but sustained overload requires admission, capacity, or workload changes. Read the lesson →
Queues vs logs vs pub/sub	Queues distribute work, logs retain ordered history, and pub/sub follows its stated subscription and retention contract. Read the lesson →
Quorum	A set of participants whose responses meet a protocol's decision rule. Intersection alone does not establish all read or write correctness properties. Read the lesson →
Quorum-based systems	A quorum is a permitted decision set within a protocol whose members preserve the required evidence. Read the lesson →

Quorums and read/write trade-offs	Quorum intersection is a fixed-set property; version, durability, concurrency, and membership rules determine its use. Read the lesson →
Race conditions and critical sections	Both callers read zero before either writes. Protecting the whole read–modify–write sequence lets two increments produce two. Read the lesson →
Raft: terms, elections, log replication, commit	Raft combines term-based elections, log freshness, replication, commitment, and membership rules to preserve history. Read the lesson →
Range partitioning vs hash partitioning	Range placement supports ordered locality; hashing spreads keys, with different scan and hot-key trade-offs. Read the lesson →
Rate limiting	Rate limiting applies a scoped pace and burst rule with an explicit state-failure policy. Read the lesson →
Rate limits and quotas	Rate limits control pace and bursts; quotas cap accumulated use under defined scope and failure rules. Read the lesson →
Read replicas and read/write splitting	Read/write splitting must respect observation and transaction needs, not merely classify by SQL verb. Read the lesson →
Read-heavy vs write-heavy systems	A read/write ratio counts work at a specified boundary; one user action can cause many physical operations. Read the lesson →

Read-your-writes and monotonic reads	Read-your-writes and monotonic reads preserve particular session observations across supported routing and recovery paths. Read the lesson →
Rebalancing	Rebalancing transfers current data and authority while preserving concurrent writes, routing, validation, and recovery. Read the lesson →
Reconciliation after partial failure	Reconciliation compares intended and observed outcomes and repairs partial failure through safe identified operations. Read the lesson →
RED and USE methods	RED focuses on serving rate, errors, and duration; USE examines resource utilisation, saturation, and errors. Read the lesson →
Redis-like data structures	Redis-like data structures offer specific operations; local command atomicity does not automatically cover a larger workflow. Read the lesson →
Redundancy	Redundancy requires independent failure exposure, valid state, usable takeover, and enough surviving capacity. Read the lesson →
Refactoring and recognizing code smells	Refactoring changes structure while preserving observable behaviour. Code smells guide investigation; the repair should address a concrete difficulty. Read the lesson →
Referential integrity	Referential integrity protects valid relationships using mechanisms appropriate to the ownership and transaction boundary. Read the lesson →

Regression testing	Regression tests preserve important behaviour while allowing deliberate contract changes and tolerating measured noise appropriately. Read the lesson →
Relational databases and SQL	Relational storage models facts with keys and constraints; SQL expresses operations whose physical cost needs investigation. Read the lesson →
Replication	Replication maintains copies under explicit acknowledgment, authority, lag, and recovery rules. Read the lesson →
Replication lag	Replica receipt, persistence, and apply progress can differ and support different freshness or recovery claims. Read the lesson →
Repository	Repository supplies application-level operations for finding and saving data under an explicit storage contract. Read the lesson →
Repository, Unit of Work, Specification, CQRS, and dependency boundaries	Data access, rule evaluation, grouped changes, and read models solve different problems. Read the lesson →
Request scopes, session scopes, and custom scopes	A scope names a boundary for reuse and cleanup; a later job needs its own valid resources. Read the lesson →
Request/response schemas	Schemas describe structure; validation also checks business meaning, size limits, and safe error behaviour. Read the lesson →

Resource exhaustion and backpressure	If arrivals exceed completions, waiting work grows. Backpressure makes the producer respond to the receiving side's limited capacity. Read the lesson →
Resource modeling	Resource modelling gives business objects and operations stable identities and meaningful lifecycles. Read the lesson →
REST, GraphQL, gRPC, RPC	REST, GraphQL, and RPC styles organise interaction differently; none alone supplies safe retries or compatibility. Read the lesson →
Retries and dead-letter queues	Retry and dead-letter policies need bounds, useful error context, ownership, and a safe repair/replay path. Read the lesson →
Retries and timeouts	Retries spend bounded time and capacity and need safe repetition, particularly for uncertain effects. Read the lesson →
Retries, exponential backoff, jitter	Retries spend bounded capacity and time, with backoff, jitter, classification, and safe effect identity. Read the lesson →
Retry storms	Retry storms amplify failure traffic; timing randomisation alone does not bound total attempts. Read the lesson →
Reverse proxies vs forward proxies	A forward proxy acts for clients. A reverse proxy receives requests in front of application servers. Read the lesson →

Role-based and attribute-based access control	RBAC assigns scoped role permissions; ABAC evaluates trusted attributes, with enforcement and testing required for either. Read the lesson →
Rollbacks and database migrations	Rollback depends on compatible state; expand-and-contract maintains a supported overlap before removing old structures. Read the lesson →
Rolling, blue-green, and canary deployments	Rolling, blue-green, and canary exposure need representative verification, compatible state, and sufficient transition capacity. Read the lesson →
Root-cause investigation	Cause investigation distinguishes triggers from contributing mechanisms and tests explanations beyond correlation. Read the lesson →
Row-oriented vs column-oriented storage	Row and column layouts group physical values differently to support different read and update patterns. Read the lesson →
RPO	Recovery Point Objective: an objective for the amount of data loss measured in time. Read the lesson →
RTO	Recovery Time Objective: an objective for restoring service after disruption. Read the lesson →
RTO and RPO	RTO targets recovery time; RPO targets acceptable data-loss exposure in time. Read the lesson →
Safety vs liveness	Safety excludes bad outcomes; liveness requires progress under specified environmental conditions. Read the lesson →

Saga	A coordinated sequence of local transactions with compensating actions; compensation need not be a perfect inverse. Read the lesson →
Sagas and compensating transactions	Sagas coordinate local steps and supported compensations with durable progress and explicit uncertainty. Read the lesson →
Sampling and telemetry cost	Sampling and retention balance diagnostic questions, privacy, cardinality, and telemetry capacity. Read the lesson →
Scale-out economics	Scale economics includes surviving capacity, coordination, transfer, baseline resources, and operating effort per useful outcome. Read the lesson →
Scaling writes vs scaling reads	Read scaling can reuse copies; write scaling must divide or reduce ownership work while preserving invariants. Read the lesson →
Scheduling and context switching	The scheduler chooses a ready thread. A context switch saves one thread's execution state and restores another's. Read the lesson →
Scheduling and service discovery	Scheduling selects eligible resources; discovery advertises endpoints whose readiness and shared-state behaviour still need handling. Read the lesson →
Scheduling, leases, and worker coordination	Scheduling separates stable due occurrences from attempts, with atomic creation, time, overlap, and stale-worker rules. Read the lesson →

Schema evolution	Schema evolution must preserve supported interpretation of stored events, historical data, and delayed readers. Read the lesson →
Search engines and inverted indexes	An inverted index maps analysed terms to document postings, with source and access metadata for safe serving. Read the lesson →
Search indexes	Search indexes are retrieval representations with source-version, freshness, access, and rebuild obligations. Read the lesson →
Search ingestion and reindexing	Reindexing builds and validates a new generation while catching up edits and deletes before serving switches. Read the lesson →
Secrets management	Secrets need controlled distribution, ownership, rotation, revocation, and protection from accidental exposure. Read the lesson →
Secure-by-design reviews	Secure reviews follow sensitive operations and changed conditions through every required enforcement boundary. Read the lesson →
Security vs usability trade-offs	Security and usability need a workable safe journey without trading away mandatory isolation or correctness. Read the lesson →
Sequential and causal consistency	Sequential consistency preserves participant program order; causal consistency preserves dependencies while allowing unrelated orders to differ. Read the lesson →

Serializability	A transactional correctness condition requiring equivalence to some serial execution; real-time order is a separate issue. Read the lesson →
Serializability vs linearizability	Serializability concerns transaction equivalence; strict serializability additionally respects real-time constraints. Read the lesson →
Serializable execution and serializability	Serializability concerns serial transaction equivalence; linearizability also concerns operation real-time precedence. Read the lesson →
Serverless and function-oriented architectures	Serverless execution delegates platform work while retaining limits, state, identity, and retry requirements. Read the lesson →
Serverless compute	Serverless delegates selected platform duties while retaining limits, durable state, identity, and safe retries. Read the lesson →
Service discovery	Service discovery supplies possible destinations. A destination can stop after the list is obtained, so callers must handle that change. Read the lesson →
Sessions, cookies, tokens	Sessions, tokens, and cookies have different state and transport roles with explicit validation and revocation rules. Read the lesson →
Shared memory vs message passing	Shared memory lets callers change the same value. Message passing can give one owner control of that value. Read the lesson →

Single points of failure	A single point of failure can be any shared dependency defeating the required service or recovery. Read the lesson →
Singleton	Singleton restricts access to one instance within a stated program boundary; it supplies no cross-process uniqueness. Read the lesson →
SLI / SLO / SLA	Indicator: measured behavior. Objective: target for that behavior. Agreement: an explicit service agreement, potentially with consequences. Read the lesson →
SLI, SLO, SLA, and error budgets	An SLI measures behaviour, an SLO targets it over a window, and an SLA states agreement obligations. Read the lesson →
SLIs, SLOs, error budgets	SLIs measure behaviour; SLOs target it; error-budget burn compares observed shortfall with its allowance. Read the lesson →
SLO-based reliability decisions	SLO evidence guides reliability investment without hiding critical low-volume harm inside a broad denominator. Read the lesson →
SOLID principles, including their limitations	SOLID examines responsibilities, extension, valid replacement, focused interfaces, and dependency direction. Apply each principle to an actual change, rather than adding layers by habit. Read the lesson →
Specification	Specification names a reusable condition; changing data later still requires checking the current rule. Read the lesson →

Split-brain prevention and fencing tokens	Fencing epochs reject stale authority only when receivers enforce them atomically with protected effects. Read the lesson →
Split-brain situations	Split brain creates conflicting authorities; stale actors need prevention or enforcement at the protected effect boundary. Read the lesson →
Stale-while-revalidate	Stale-while-revalidate serves an allowed older value while refresh proceeds, with a limit on unusable age. Read the lesson →
State	State changes an object's permitted behaviour as its lifecycle advances through allowed transitions. Read the lesson →
Stateless vs stateful services	Stateless instances keep no required per-client conversation only locally, while using appropriate external state owners. Read the lesson →
Status codes and error models	Protocol status and business state can differ: a created resource may still represent pending work. Read the lesson →
Sticky sessions	Sticky sessions preserve affinity while adding load-skew and recovery consequences for locally held state. Read the lesson →
Storage, memory, bandwidth, and CPU estimates	Estimate requests, bytes, memory, CPU, and transfer separately using explicit workload assumptions. Read the lesson →

Strategy	Strategy supplies a replaceable algorithm through a shared operation. Read the lesson →
Strategy, Observer, Command, State, Chain of Responsibility, Template Method, Iterator, Mediator, Memento, Visitor	Behavioural patterns organise decisions, actions, reactions, and coordination between objects. Read the lesson →
Stream processing fundamentals	Stream processing needs keyed state, event/processing time, lateness, corrections, and recoverable progress. Read the lesson →
Streaming APIs and long-running operations	Long-running and streaming APIs need explicit ownership, progress, cancellation, buffering, and recovery contracts. Read the lesson →
Strong consistency and eventual consistency	Eventual convergence needs stated repair assumptions and conflict semantics; “strong” needs a named precise model. Read the lesson →
Strongly consistent metadata services	Consistent metadata services own small control decisions, with explicit consequences when new decisions cannot proceed. Read the lesson →
Structured logging	Structured logs give fields stable meanings, bounded size, safe content, and explicit event boundaries. Read the lesson →
Supply-chain security	Supply-chain controls protect inputs, builds, artifacts, credentials, deployment, and compromised-release response. Read the lesson →

Sustainability and resource efficiency	Resource efficiency removes unnecessary work and transfer while broader sustainability claims require actual supporting evidence. Read the lesson →
Synchronous vs asynchronous communication	Asynchronous submission separates acceptance from completion and needs an owner for the intervening work. Read the lesson →
Synchronous vs asynchronous execution	A synchronous interface returns completion before the caller proceeds; an asynchronous interface separates starting from receiving the result. Read the lesson →
Synchronous vs asynchronous replication	Synchronous replication waits for named milestones; asynchronous acknowledgment can precede another durable copy. Read the lesson →
Tail latency analysis	Tail analysis connects timing distributions and representative paths without incorrectly averaging percentile summaries. Read the lesson →
Tail latency and latency budgets	A request needing many results can be delayed by its slowest dependency; budget the complete path. Read the lesson →
TCP vs UDP vs QUIC	TCP carries an ordered byte stream; UDP carries datagrams; QUIC supplies secure streams over UDP. The application still interprets the data. Read the lesson →
Technical vs business boundaries	Business boundaries group capabilities; technical layers organise mechanisms within or across them. Read the lesson →

Tell, Don't Ask	Tell, Don’t Ask keeps a state-changing rule with its owner. Concurrent writers still need an atomic check and update at the shared storage boundary. Read the lesson →
Template Method	Template Method fixes a sequence of steps while letting subclasses replace selected steps. Read the lesson →
Testability and design for change	A test double makes outcomes controllable. Dependency injection supplies the collaborator from outside. Tests must also check the real provider and storage boundaries. Read the lesson →
The fallacies of distributed computing	Distributed-computing fallacies expose hidden assumptions about networks, cost, topology, trust, and administration. Read the lesson →
Thread pools and worker pools	A pool limits active workers. A separate queue limit controls waiting jobs. Restart recovery also needs saved job records. Read the lesson →
Thread safety and immutability	Immutable records stay unchanged. Their mutable collection still needs rules for concurrent access. Read the lesson →
Threat modeling	Threat modelling connects assets, actors, abuse paths, and impact to controls and concrete tests. Read the lesson →
Throughput	Completed work per unit time under a specified workload and success definition. Read the lesson →

Time, clocks, and event ordering	Physical timestamps, elapsed durations, and causal order answer different questions and need suitable clocks or protocols. Read the lesson →
Time-series databases	Time-series data combines measurements, times, and dimensions with explicit cardinality, lateness, and retention handling. Read the lesson →
Timeouts	Timeouts bound waiting; overall deadlines include acquisition and stage waits without proving remote nonexecution. Read the lesson →
TLS and HTTPS	TLS checks peer identity and protects messages in transit. The application separately checks permission to use a resource. Read the lesson →
Trace IDs, span IDs, context propagation	Trace context links observations across calls or messages and must not be treated as authorisation. Read the lesson →
Trade-off	A choice that accepts one cost or limitation to gain another benefit, judged against the requirements. Read the lesson →
Transaction commit and rollback	Commit accepts a transaction under the engine protocol; rollback abandons uncommitted work and cannot reverse unrelated remote effects. Read the lesson →
Transactional outbox	Recording a state change and publication intent in one local transaction, then relaying the intent; duplicate delivery still requires handling. Read the lesson →

Transactions across multiple operations	Related statements need an explicit common transaction when their invariant requires one all-or-nothing change. Read the lesson →
Transient, scoped, singleton, and framework-specific lifetimes	Transient creates per lookup, scoped reuses within a scope, and singleton reuses within its stated container boundary. Read the lesson →
Trust boundaries	Trust boundaries mark changes in identity, authority, and confidence that require appropriate validation. Read the lesson →
Trust boundary	A point across which identity, privilege, or confidence in input changes. Read the lesson →
TTL and cache expiration	TTL limits intended residence after insertion, not necessarily the source age of the inserted fact. Read the lesson →
Twelve-Factor App principles	Twelve-Factor practices guide repeatable service operation while workload-specific correctness and recovery remain required. Read the lesson →
Unit of Work	Unit of Work tracks related data changes and coordinates writing them within a defined transaction boundary. Read the lesson →
Unit, integration, contract, end-to-end, and performance tests	Test scopes provide different evidence; choose checks by failures they can detect rather than a mandatory count ratio. Read the lesson →

User journeys and workload characteristics	A workload describes the operations users cause, their frequency, payloads, and dependent work. Read the lesson →
Vector indexes: introductory architecture only	Vector indexes retrieve similarity candidates from embeddings; scores establish neither truth nor access permission. Read the lesson →
Vertical vs horizontal scaling	Vertical scaling enlarges a node; horizontal scaling adds nodes, with gains limited by the work that can divide. Read the lesson →
Virtual machines vs containers	VMs virtualise hardware with guest systems; typical containers share a host kernel, with actual isolation and failure scope verified. Read the lesson →
Visitor	Visitor groups an operation across known element kinds; adding a kind can require changes to many visitors. Read the lesson →
Webhooks and delivery acknowledgments	Webhooks are asynchronous deliveries requiring authenticity, stable event identity, and defined acknowledgment behaviour. Read the lesson →
WebSockets, Server-Sent Events, and polling	Polling repeatedly asks for updates. SSE sends server events; WebSockets allows both ends to send. Missed updates still need recovery. Read the lesson →
What data storage must guarantee	Storage guarantees state accepted outcomes, observation rules, ownership, retention, and recovery under specified failures. Read the lesson →

What makes a distributed system difficult	Distributed participants observe only parts of an operation, so legal outcomes need explicit message and recovery protocols. Read the lesson →
When caching is harmful	Caching can be harmful when it saves little, weakens scope, hides origin limits, or adds excessive lifecycle work. Read the lesson →
When not to use microservices	Microservice extraction needs concrete benefits and operating capability sufficient to justify its added coordination cost. Read the lesson →
Why caching works	Caching saves repeated work when reuse outweighs access, storage, and maintenance costs under the required semantics. Read the lesson →
Wide-column stores	Wide-column designs commonly combine partition keys with ordered dimensions, requiring bounded partitions and query-driven modelling. Read the lesson →
Workflow engines	Workflow engines persist coordination under a programming model while external activities still need safe effect contracts. Read the lesson →
Write-ahead logging	Write-ahead logging preserves recovery information before corresponding data pages, under configured persistence rules. Read the lesson →
Zero-trust principles	Zero-trust principles avoid implicit permission from location alone and make trust evidence and limits explicit. Read the lesson →

No terms match that search.

THE VISUAL INDEX

Every diagram has a job

575 original figures: chapter mechanisms, full-design views, and the required unit plates. The separate master map is Figure M.1.

FIG. 1.1.1

The application and storage work together

Include the parts needed to explain the behaviour you are studying.

FIG. 1.1.2

Creating a task changes the stored state

Read the state before and after the operation to see exactly what changed.

FIG. 1.1.3

Save before reporting success

Compare the two versions by restarting the process.

FIG. 1.1.4

Two levels of detail in one design

Check that both views agree about saving a task before reporting success.

FIG. 1.2.1

Shared storage details make changes spread

Follow the arrows to find which parts know about the shared data.

FIG. 1.2.2

Separate responsibilities inside one process

Each kind of work has a clear place in the code.

FIG. 1.2.3

The service uses a repository contract

The dashed arrows show code dependencies. These objects can run in the same process.

FIG. 1.2.4

Each layer works at a different level of detail

Follow one request through the layers to connect the user's action to the work below it.

FIG. 1.3.1

Turn “safe” into a checkable requirement

A specific requirement tells us which test to run.

FIG. 1.3.2

Choose, record, and review a design

Keep the reasons for a decision alongside the decision itself.

FIG. 1.3.3

Check requirements before scoring preferences

An easy-to-run choice is useful only if it meets the requirements.

FIG. 1.3.4

Availability and durability answer different questions

Check both whether requests succeed and whether saved data survives the stated failure.

FIG. 1.4.1

C4: zoom from the system into its code

Select the level that helps your reader answer a particular question.

FIG. 1.4.2

The application and database share one machine

Show physical placement when asking what can fail together.

FIG. 1.4.3

The task is saved, but the reply is lost

Trace the events in order to see why the client can be unsure even when a task exists.

FIG. 1.4.4

Each design change answers a new need

State the requirement that makes each change necessary.

FIG. 2.1.1

Gathering and prioritizing requirements · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.1.2

User journeys and workload characteristics · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.1.3

Read-heavy vs write-heavy systems · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.1.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.2.1

Latency, throughput, QPS, RPS, TPS · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.2.2

A small mean can hide one slow request

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 2.2.3

Availability, reliability, durability, consistency · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.2.4

SLI, SLO, SLA, and error budgets · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.3.1

Capacity and growth planning · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.3.2

Different workload dimensions have different magnitudes

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 2.3.3

Peak vs average load · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.3.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.4.1

Little's Law and queueing intuition · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.4.2

Amdahl's Law and bottlenecks · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.4.3

Tail latency and latency budgets · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.4.4

Cost, security, operational complexity, and maintainability as constraints · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 2.4.5

Decision matrices and trade-off analysis · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.1.1

Follow a request from the browser · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.1.2

Find the machine and the program · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.1.3

What does each transport deliver?

Make the internal structure or state relation visible before claiming its behavior.

FIG. 3.1.4

Follow the task-list request

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 3.2.1

Find an address from a name · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.2.2

Protect the conversation with TLS · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.2.3

Carry several HTTP exchanges · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.2.4

Three messages prepare the TCP connection

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 3.2.5

A cold request spends time before server work

Separate stage waits and execution rather than attach one unexplained number to the entire request.

FIG. 3.3.1

Forward a request through a proxy · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.3.2

Share traffic and check common rules · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.3.3

Route by port or by path · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.3.4

Send updates to the browser · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.4.1

Follow traffic out and back · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.4.2

Find a service whose address can change · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.4.3

Reuse a nearby copy · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.4.4

What does a timeout tell us · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 3.4.5

Borrow and return connections · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.1.1

Start a program and follow its threads · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.1.2

Each thread has a call stack; objects can be shared

Make the internal structure or state relation visible before claiming its behavior.

FIG. 4.1.3

Let threads take turns · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.1.4

Find out whether work calculates or waits · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.1.5

Start now and receive a result later · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.2.1

Wait without holding the thread · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.2.2

Distinguish overlap from simultaneous work · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.2.3

Follow a task through an event loop · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.2.4

Give workers a bounded waiting queue · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.3.1

Share a value or send a message · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.3.2

Follow a lost counter update · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.3.3

Choose the right coordination tool · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.3.4

Each worker waits for a lock held by the other

Make the internal structure or state relation visible before claiming its behavior.

FIG. 4.4.1

Make a completed value visible to a reader · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.4.2

Keep shared data safe to use · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.4.3

Slow arrivals when workers cannot keep up · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 4.4.4

End related tasks and release their resources · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.1.1

Four ways to organise an object · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.1.2

Build with parts or derive a class · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.1.3

State exactly what a charge means · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.1.4

Keep related work together · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.1.5

Use SOLID to examine a real change · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.2.1

Keep one copy of a shared rule · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.2.2

Avoid knowing the whole object chain · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.2.3

Ask the owner to perform the change · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.2.4

Make reads and changes easy to distinguish · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.3.1

Calculate new values, then perform effects · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.3.2

Distinguish identity from value · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.3.3

Follow the conversation between objects · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.3.4

Test decisions with a replaceable gateway · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.4.1

Improve the structure one step at a time · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.4.2

Express decisions as functions · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 5.4.3

Test the payment rule without making a payment

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 5.4.4

Keep calculation separate from effects

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.1.1

See what the service needs · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.1.2

Let another part of the program make the call · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.1.3

Pass the sender into the service · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.1.4

Supply a dependency when it becomes necessary · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.1.5

Separate injection from dependency inversion · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.1.6

Assemble the application in one clear place · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.2.1

Which requests receive the same object?

Make the internal structure or state relation visible before claiming its behavior.

FIG. 6.2.2

Give each request or job its own boundary · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.2.3

Decide who must close the resource · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.2.4

Release resources on every exit path · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.2.5

Create an expensive object only when needed · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.2.6

Find cycles and dependencies kept too long · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.3.1

Show dependencies instead of looking them up inside · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.3.2

Inject a factory for a choice made later · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.3.3

Supply the sender from outside

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.3.4

Choose a sender when the message arrives

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.4.1

Choose how an object is created · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.4.2

Factory Method · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.4.3

Abstract Factory · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.4.4

Builder · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.4.5

Prototype · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.4.6

Singleton · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.1

Arrange objects around an existing operation · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.5.2

Adapter · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.3

Bridge · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.4

Composite · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.5

Decorator · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.6

Facade · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.7

Flyweight · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.5.8

Proxy · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.1

Give each behaviour a clear home · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.6.2

Strategy · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.3

Observer · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.4

Command · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.5

State · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.6

Chain of Responsibility · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.7

Template Method · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.8

Iterator · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.9

Mediator · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.10

Memento · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.6.11

Visitor · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.7.1

Separate data access, rules, and commit decisions · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.7.2

Repository · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.7.3

Unit of Work · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.7.4

Specification · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.7.5

CQRS · the example

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 7.1.1

Layered/N-tier architecture · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.1.2

Modular monoliths · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.1.3

Monolith vs microservices · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.1.4

Clean Architecture · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.1.5

Hexagonal/Ports and Adapters · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.1.6

Onion Architecture · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.2.1

Domain-Driven Design (DDD) · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.2.2

Context boundaries protect different business meanings

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 7.2.3

Aggregates, entities, value objects, domain services · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.2.4

Domain events · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.3.1

Dependency rules and module boundaries · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.3.2

Business logic vs orchestration vs infrastructure · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.3.3

Controllers, services, repositories, adapters · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.3.4

Anti-corruption layers · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.3.5

Event-driven architecture · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.4.1

Serverless and function-oriented architectures · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.4.2

Evolutionary architecture · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.4.3

Technical vs business boundaries · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.4.4

Distributed monolith antipattern · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 7.4.5

When not to use microservices · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.1.1

API fundamentals and interface contracts · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.1.2

REST, GraphQL, gRPC, RPC · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.1.3

Resource modeling · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.1.4

Request/response schemas · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.1.5

Status codes and error models · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.1.6

Pagination, filtering, sorting · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.2.1

Cursor vs offset pagination · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.2.2

API versioning and compatibility · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.2.3

A payment retry follows durable operation identity

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 8.2.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.3.1

Authentication vs authorization at API boundaries · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.3.2

Rate limits and quotas · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.3.3

Retries and timeouts · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.3.4

Streaming APIs and long-running operations · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.3.5

Webhooks and delivery acknowledgments · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.4.1

Schema evolution · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.4.2

Consumer-driven contracts · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.4.3

API gateways and BFF · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.4.4

Public vs internal APIs · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 8.4.5

API documentation and discoverability · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.1

What data storage must guarantee · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.2

Relational databases and SQL · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.3

Document databases · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.4

Key-value databases · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.5

Wide-column stores · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.6

Graph databases · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.7

Time-series databases · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.8

Search indexes · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.1.9

Vector indexes: introductory architecture only · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.2.1

Object storage vs block vs file storage · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.2.2

OLTP vs OLAP · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.2.3

Row-oriented vs column-oriented storage · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.2.4

Normalization vs denormalization · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.3.1

An order schema preserves relationships and history

Make the internal structure or state relation visible before claiming its behavior.

FIG. 9.3.2

Referential integrity · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.3.3

Data ownership · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.3.4

Choosing storage based on access patterns · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.4.1

Data retention and lifecycle · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.4.2

Polyglot persistence and its operational costs · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 9.4.3

Follow the related complete design

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 9.4.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.1.1

How databases store and retrieve records · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.1.2

Pages, records, indexes, and buffer pools · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.1.3

B+tree lookup narrows an ordered key range

Make the internal structure or state relation visible before claiming its behavior.

FIG. 10.1.4

LSM trees and write amplification · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.1.5

Query planning and EXPLAIN · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.1.6

Index selection and query optimization · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.2.1

ACID properties · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.2.2

Transaction commit and rollback · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.2.3

Write-ahead logging · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.2.4

Two snapshots can legitimately see different versions

Make the internal structure or state relation visible before claiming its behavior.

FIG. 10.3.1

Locking and optimistic concurrency control · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.3.2

Dirty reads, nonrepeatable reads, phantom reads · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.3.3

Lost updates, write skew, serialization anomalies · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.3.4

Isolation levels and implementation differences · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.3.5

Serializable execution and serializability · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.4.1

Database connection pooling · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.4.2

Transactions across multiple operations · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.4.3

Backup, recovery, and durability · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 10.4.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.1.1

Why caching works · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.1.2

Cache locality and the working set · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.1.3

In-process vs distributed caches · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.1.4

Browser, CDN, application, and database caches · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.2.1

Cache-aside, read-through, write-through, write-behind · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.2.2

TTL and cache expiration · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.2.3

A stale cache is a separate state copy

Make the internal structure or state relation visible before claiming its behavior.

FIG. 11.2.4

LRU and FIFO disagree after the same accesses

Make the internal structure or state relation visible before claiming its behavior.

FIG. 11.3.1

Hot keys · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.3.2

Cache stampedes, penetration, and avalanche · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.3.3

Negative caching · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.3.4

Stale-while-revalidate · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.3.5

Cache warming and prefetching · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.4.1

Redis-like data structures · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.4.2

CDN architecture and origin shielding · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.4.3

Cache partitioning and replication · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 11.4.4

When caching is harmful · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.1.1

Vertical vs horizontal scaling · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.1.2

Stateless vs stateful services · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.1.3

Load distribution algorithms · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.1.4

Health checks and failover · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.1.5

Sticky sessions · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.1.6

Autoscaling metrics · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.2.1

Replication · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.2.2

Partitioning and sharding · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.2.3

Range partitioning vs hash partitioning · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.2.4

A new owner takes only part of the hash space

Make the internal structure or state relation visible before claiming its behavior.

FIG. 12.2.5

Rebalancing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.2.6

Averages hide a hot partition

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 12.3.1

Read replicas and read/write splitting · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.3.2

Replication lag · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.3.3

Database proxying · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.3.4

Connection pool saturation · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.3.5

Bottleneck identification · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.4.1

Queueing and overload · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.4.2

Single points of failure · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.4.3

Multi-region strategies · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.4.4

Scaling writes vs scaling reads · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 12.4.5

Scale-out economics · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.1.1

Binary/object storage · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.1.2

The upload pipeline separates bytes from control

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 13.1.3

Multipart and resumable uploads · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.1.4

Checksums and content addressing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.1.5

Metadata vs binary data · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.2.1

An inverted index is a term-to-document map

Make the internal structure or state relation visible before claiming its behavior.

FIG. 13.2.2

Indexing pipelines · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.2.3

Search ingestion and reindexing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.2.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.3.1

Batch vs streaming pipelines · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.3.2

ETL vs ELT · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.3.3

Change Data Capture (CDC) · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.3.4

Data lakes and warehouses · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.3.5

Stream processing fundamentals · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.4.1

Data provenance and lineage · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.4.2

Data quality, schema validation, and replay · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.4.3

Lifecycle policies and retention · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 13.4.4

Designing data-intensive applications · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.1

What makes a distributed system difficult · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.2

Partial failures · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.3

Network uncertainty · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.4

Fault models · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.5

Time, clocks, and event ordering · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.6

Physical vs logical clocks · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.1.7

Vector comparison exposes concurrent events

Make the internal structure or state relation visible before claiming its behavior.

FIG. 14.2.1

Consistency models · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.2.2

Strong consistency and eventual consistency · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.2.3

Linearizability · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.2.4

Serializability vs linearizability · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.2.5

Sequential and causal consistency · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.2.6

Read-your-writes and monotonic reads · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.3.1

CAP theorem, with its actual assumptions · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.3.2

PACELC as a useful extension · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.3.3

Intersecting quorums still need a protocol

Make the internal structure or state relation visible before claiming its behavior.

FIG. 14.3.4

A partition can leave two live but disconnected sides

Make the internal structure or state relation visible before claiming its behavior.

FIG. 14.4.1

Distributed coordination · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.4.2

The fallacies of distributed computing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.4.3

Safety vs liveness · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 14.4.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.1.1

Primary/replica architectures · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.1.2

Leader/follower replication · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.1.3

Multi-leader replication · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.1.4

Leaderless replication · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.1.5

Synchronous vs asynchronous replication · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.1.6

Quorum-based systems · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.2.1

Consensus and replicated state machines · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.2.2

Raft election changes authority under protocol rules

Make the internal structure or state relation visible before claiming its behavior.

FIG. 15.2.3

Paxos: conceptual foundations · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.2.4

Presence in a log is not the same as commitment

Make the internal structure or state relation visible before claiming its behavior.

FIG. 15.2.5

Membership changes · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.3.1

Split-brain prevention and fencing tokens · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.3.2

Failure detection · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.3.3

Strongly consistent metadata services · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.3.4

Distributed locks and leases · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.4.1

Conflict resolution and CRDT introduction · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.4.2

Multi-region replication trade-offs · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 15.4.3

Follow the related complete design

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 15.4.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.1

Synchronous vs asynchronous communication · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.2

Queues vs logs vs pub/sub · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.3

Producers, consumers, brokers · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.4

Partitions, consumer groups, offsets · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.5

Delivery guarantees · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.6

At-most-once and at-least-once processing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.7

Exactly-once processing within defined boundaries · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.8

Ordering guarantees · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.1.9

Consumer acknowledgements · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.2.1

Retries and dead-letter queues · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.2.2

Poison messages · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.2.3

Idempotent consumers · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.2.4

Event sourcing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.3.1

CQRS · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.3.2

The outbox closes a local publication gap

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 16.3.3

A failed shipment starts tracked compensation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 16.3.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.4.1

Choreography vs orchestration · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.4.2

Workflow engines · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.4.3

Durable job execution · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.4.4

Scheduling, leases, and worker coordination · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.4.5

Backpressure and replay · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 16.4.6

Reconciliation after partial failure · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.1.1

Failure domains and fault isolation · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.1.2

Availability vs durability · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.1.3

Redundancy · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.1.4

Active-active vs active-passive · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.2.1

Timeouts · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.2.2

Retries, exponential backoff, jitter · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.2.3

Retry storms · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.2.4

A circuit breaker bounds futile calls

Make the internal structure or state relation visible before claiming its behavior.

FIG. 17.2.5

Bulkheads · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.2.6

Rate limiting · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.3.1

Load shedding · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.3.2

Graceful degradation · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.3.3

Fallbacks · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.3.4

Health checking · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.1

Recovery time and recoverable data are different

Make the internal structure or state relation visible before claiming its behavior.

FIG. 17.4.2

Backups and restore testing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.3

Failover and failback · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.4

Multi-zone and multi-region deployments · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.5

Chaos engineering · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.6

SLO-based reliability decisions · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.7

Cascading failures · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 17.4.8

Incident analysis and prevention · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.1

Threat modeling · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.2

Trust boundaries · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.3

Authentication vs authorization · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.4

Sessions, cookies, tokens · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.5

OAuth 2.0 and OpenID Connect · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.6

Role-based and attribute-based access control · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.1.7

Least privilege · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.1

Defense in depth · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.2

Secrets management · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.3

Encryption in transit and at rest · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.4

Password hashing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.5

Key management and rotation · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.6

Network segmentation · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.2.7

Zero-trust principles · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.3.1

API security and common vulnerabilities · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.3.2

Tenant identity must survive every trust boundary

Make the internal structure or state relation visible before claiming its behavior.

FIG. 18.3.3

Data minimization · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.3.4

Audit trails · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.3.5

Privacy, retention, and deletion · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.4.1

Supply-chain security · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.4.2

Security vs usability trade-offs · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.4.3

Secure-by-design reviews · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 18.4.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.1.1

Monitoring vs observability · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.1.2

Logs, metrics, and traces · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.1.3

Structured logging · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.1.4

This slow request is dominated by waiting

Separate stage waits and execution rather than attach one unexplained number to the entire request.

FIG. 19.1.5

Trace IDs, span IDs, context propagation · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.1.6

Golden signals · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.2.1

RED and USE methods · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.2.2

SLIs, SLOs, error budgets · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.2.3

Alert design and alert fatigue · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.2.4

Sampling and telemetry cost · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.2.5

Dashboards · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.3.1

Profiling · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.3.2

Query and application performance diagnosis · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.3.3

Incident response · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.3.4

On-call and runbooks · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.3.5

Root-cause investigation · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.4.1

Blameless postmortems · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.4.2

Capacity monitoring · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.4.3

Observability across asynchronous workflows · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 19.4.4

State and ownership in the related design

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.1.1

Development vs staging vs production · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.1.2

VMs and containers isolate at different layers

Make the internal structure or state relation visible before claiming its behavior.

FIG. 20.1.3

Container fundamentals · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.1.4

Kubernetes fundamentals · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.2.1

Control plane vs data plane · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.2.2

Scheduling and service discovery · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.2.3

Autoscaling · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.2.4

Configuration and secret delivery · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.2.5

Immutable infrastructure · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.3.1

Infrastructure as Code · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.3.2

CI/CD · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.3.3

Canary exposure expands only after verification

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 20.3.4

Feature flags · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.3.5

Rollbacks and database migrations · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.3.6

Multi-environment configuration · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.4.1

Twelve-Factor App principles · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.4.2

Serverless compute · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.4.3

Cloud vs on-premises trade-offs · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.4.4

Infrastructure ownership and managed services · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.4.5

Deployment topologies and networking · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 20.4.6

Platform engineering basics · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.1.1

Performance budgets · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.1.2

CPU, memory, disk, network, and database bottlenecks · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.1.3

Profiling and benchmarking · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.1.4

Load, stress, spike, and soak testing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.1.5

Benchmark methodology and statistical pitfalls · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.1.6

Tail latency analysis · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.2.1

Queue saturation and throughput ceilings · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.2.2

Performance tuning vs architecture changes · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.2.3

Architecture costs are not capacity guarantees

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 21.2.4

Compute, storage, networking, and telemetry costs · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.2.5

Cost per request or transaction · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.2.6

Capacity reservation vs elastic capacity · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.3.1

Cost-performance optimization · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.3.2

Sustainability and resource efficiency · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.3.3

Unit, integration, contract, end-to-end, and performance tests · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.3.4

Fault-injection tests · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.4.1

Data consistency validation · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.4.2

Architecture fitness functions · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.4.3

Regression testing · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 21.4.4

Evaluating a design against its original requirements · the example

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.1.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.1.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.1.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.1.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.2.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.2.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.2.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.2.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.3.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.3.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.3.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.3.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.4.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.4.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.4.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.4.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.5.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.5.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.5.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.5.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.6.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.6.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.6.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.6.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.7.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.7.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.7.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.7.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.8.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.8.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.8.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.8.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.9.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.9.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.9.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.9.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.10.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.10.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.10.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.10.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.11.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.11.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.11.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.11.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.12.1

Data ownership and relationships

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.12.2

Follow the main operation

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 22.12.3

Follow the failure and its correction

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 22.12.4

How the design can grow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 2.0.1

Request-volume timeline

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 2.0.2

throughput-latency curve

Queueing can dominate near a modeled capacity ceiling; this is calculated theory, not a benchmark.

FIG. 2.0.3

latency waterfall

Separate stage waits and execution rather than attach one unexplained number to the entire request.

FIG. 2.0.4

capacity planning worksheet

Compare the stated dimensions under one explicit contract.

FIG. 3.0.1

Complete HTTP request timeline

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 3.0.2

TCP/TLS interaction

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 3.0.3

load-balancing topology

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 3.0.4

WebSocket vs polling communication

Compare the stated dimensions under one explicit contract.

FIG. 4.0.1

Each thread has a call stack; objects can be shared

Make the internal structure or state relation visible before claiming its behavior.

FIG. 4.0.2

scheduling timeline

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 4.0.3

race-condition interleaving

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 4.0.4

Each worker waits for a lock held by the other

Make the internal structure or state relation visible before claiming its behavior.

FIG. 4.0.5

event-loop flow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 5.0.1

Before/after class diagrams

Compare the stated dimensions under one explicit contract.

FIG. 5.0.2

dependency graph

Read these as source relationships, not network calls.

FIG. 5.0.3

inheritance vs composition

Compare the stated dimensions under one explicit contract.

FIG. 5.0.4

cohesion/coupling illustrations

Inspect the concrete state, values, and assumptions rather than infer a guarantee from a component name.

FIG. 6.0.1

Object creation timelines

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 6.0.2

Which requests receive the same object?

Make the internal structure or state relation visible before claiming its behavior.

FIG. 6.0.3

UML class relationships

Read these as source relationships, not network calls.

FIG. 6.0.4

dependency inversion before/after

Compare the stated dimensions under one explicit contract.

FIG. 6.0.5

pattern comparisons

Compare the stated dimensions under one explicit contract.

FIG. 7.0.1

Layered vs Hexagonal

Compare the stated dimensions under one explicit contract.

FIG. 7.0.2

modular monolith vs microservices

Compare the stated dimensions under one explicit contract.

FIG. 7.0.3

dependency arrows

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 7.0.4

bounded context map

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 8.0.1

REST request flow

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 8.0.2

API gateway topology

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 8.0.3

synchronous vs asynchronous API

Compare the stated dimensions under one explicit contract.

FIG. 8.0.4

payment state-machine diagram

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 9.0.1

Database taxonomy

Compare the stated dimensions under one explicit contract.

FIG. 9.0.2

physical storage representations

Compare the stated dimensions under one explicit contract.

FIG. 9.0.3

normalized vs denormalized schema

Compare the stated dimensions under one explicit contract.

FIG. 9.0.4

An order schema preserves relationships and history

Make the internal structure or state relation visible before claiming its behavior.

FIG. 10.0.1

B+tree lookup narrows an ordered key range

Make the internal structure or state relation visible before claiming its behavior.

FIG. 10.0.2

Two snapshots can legitimately see different versions

Make the internal structure or state relation visible before claiming its behavior.

FIG. 10.0.3

concurrent transaction interleavings

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 10.0.4

lock wait graph

Compare the stated dimensions under one explicit contract.

FIG. 10.0.5

WAL and recovery

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 11.0.1

Cache hit/miss sequence

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 11.0.2

cache hierarchy

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 11.0.3

LRU and FIFO disagree after the same accesses

Make the internal structure or state relation visible before claiming its behavior.

FIG. 11.0.4

stampede timeline

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 11.0.5

geographic CDN delivery

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 12.0.1

Architecture evolution across four load levels

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 12.0.2

A new owner takes only part of the hash space

Make the internal structure or state relation visible before claiming its behavior.

FIG. 12.0.3

Averages hide a hot partition

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 12.0.4

load-balancer failover

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 13.0.1

Upload pipeline

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 13.0.2

An inverted index is a term-to-document map

Make the internal structure or state relation visible before claiming its behavior.

FIG. 13.0.3

ingestion DAG

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 13.0.4

batch vs stream timeline

Compare the stated dimensions under one explicit contract.

FIG. 13.0.5

lineage graph

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 14.0.1

A partition can leave two live but disconnected sides

Make the internal structure or state relation visible before claiming its behavior.

FIG. 14.0.2

causality timeline

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 14.0.3

consistency model comparisons

Compare the stated dimensions under one explicit contract.

FIG. 14.0.4

Intersecting quorums still need a protocol

Make the internal structure or state relation visible before claiming its behavior.

FIG. 15.0.1

Raft election changes authority under protocol rules

Make the internal structure or state relation visible before claiming its behavior.

FIG. 15.0.2

election timeline

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 15.0.3

Presence in a log is not the same as commitment

Make the internal structure or state relation visible before claiming its behavior.

FIG. 15.0.4

quorum intersection

Compare the stated dimensions under one explicit contract.

FIG. 15.0.5

network partition recovery

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 16.0.1

Queue lifecycle

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 16.0.2

consumer partition assignment

Compare the stated dimensions under one explicit contract.

FIG. 16.0.3

Saga happy/failure paths

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 16.0.4

The outbox closes a local publication gap

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 16.0.5

event replay

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 17.0.1

Retry amplification timeline

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 17.0.2

A circuit breaker bounds futile calls

Make the internal structure or state relation visible before claiming its behavior.

FIG. 17.0.3

bulkhead isolation

Compare the stated dimensions under one explicit contract.

FIG. 17.0.4

failure-domain map

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 17.0.5

Recovery time and recoverable data are different

Make the internal structure or state relation visible before claiming its behavior.

FIG. 18.0.1

Tenant identity must survive every trust boundary

Make the internal structure or state relation visible before claiming its behavior.

FIG. 18.0.2

authentication sequence

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 18.0.3

authorization decision flow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 18.0.4

Tenant identity must survive every trust boundary

Make the internal structure or state relation visible before claiming its behavior.

FIG. 19.0.1

This slow request is dominated by waiting

Separate stage waits and execution rather than attach one unexplained number to the entire request.

FIG. 19.0.2

request ID propagation

Follow explicit message ordering and locate the acknowledgment or effect boundary.

FIG. 19.0.3

metrics-to-root-cause workflow

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 19.0.4

error-budget burn chart

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 20.0.1

VMs and containers isolate at different layers

Make the internal structure or state relation visible before claiming its behavior.

FIG. 20.0.2

Kubernetes cluster

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 20.0.3

CI/CD pipeline

Follow the stated ordering and identify which step owns the relevant outcome.

FIG. 20.0.4

blue-green deployment

Compare the stated dimensions under one explicit contract.

FIG. 20.0.5

Canary exposure expands only after verification

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 21.0.1

Performance bottleneck tree

Each branch is a hypothesis requiring profiling, traces, and workload evidence.

FIG. 21.0.2

throughput-latency curve

Queueing can dominate near a modeled capacity ceiling; this is calculated theory, not a benchmark.

FIG. 21.0.3

Architecture costs are not capacity guarantees

Compare calculated magnitudes under the same explicit model; bar length is not a benchmark.

FIG. 21.0.4

test pyramid

Test scope and feedback cost differ; choose by the failure evidence needed.

FIG. 21.0.5

design decision matrix

Compare the stated dimensions under one explicit contract.

THE FORMULA REFERENCE

Symbols with assumptions

Every indexed model is introduced in the lessons. Inputs and guarantee boundaries accompany the notation.

State transition

```
T(S, c) → (S′, result)
```

Identify valid state changes and preserved invariants. This abstracts away implementation and failures.

Derivation and worked example →

Weighted preference

```
score(a) = Σ wᵢ sᵢ(a) / Σ wᵢ
```

Use nonnegative weights with positive total. Compare only feasible designs. Scores are illustrative judgments, not benchmark results.

Derivation and worked example →

Request rate

```
λ = daily users × requests per user per day / 86,400
```

An average workload estimate. Peak factors must be assumed or measured separately.

Derivation and worked example →

Little's Law

```
L = λ W
```

Long-run stable averages with consistent boundaries and units. It is not a universal p99 prediction.

Derivation and worked example →

Amdahl's Law

```
speedup(N) = 1 / [(1 − p) + p/N]
```

Idealized fixed workload and parallel fraction p; excludes extra coordination overhead unless modeled.

Derivation and worked example →

Storage growth

```
bytes/day = records/day × bytes/record × retained copies
```

Account separately for indexes, compression, retention, metadata, and headroom.

Derivation and worked example →

Error allowance

```
allowed bad events = (1 − target fraction) × eligible events
```

Valid for an event-based SLO; time-based objectives need their own denominator.

Derivation and worked example →

Token refill

```
tokens′ = min(capacity, tokens + refill_rate × elapsed)
```

Admission consumes tokens atomically. Distributed clocks and state ownership need additional design.

Derivation and worked example →

Quorum intersection

```
R + W > N
```

An intersection condition for reads and writes on a fixed replica set. Not by itself a proof of linearizability.

Derivation and worked example →

Cost per outcome

```
cost / successful business outcomes
```

Define accounting period and include capacity, storage, networking, telemetry, and operational work where relevant.

Derivation and worked example →

Ideal M/M/1 mean residence

```
W = S / (1 − rho)
```

S is mean service time and rho is utilization, below one. This single-server model assumes Poisson arrivals and independent exponential service times. W includes waiting and service; it is a mean, so it cannot predict p99.

Derivation and worked example →

Fan-out slow probability

```
P(any slow) = 1 − (1 − p)^N
```

Requires independent equal-probability call outcomes in this model. Shared failures can correlate tails.

Derivation and worked example →

Retry amplification

```
maximum attempts = a^L
```

a allowed attempts at each of L nested independently retrying layers; actual execution can terminate earlier.

Derivation and worked example →

Busy-capacity worksheet

```
instances = ceil(peak RPS / (assumed capacity × utilization target))
```

Capacity is an explicit hypothetical or tested input; other bottlenecks and surviving failure capacity remain separate.

Derivation and worked example →

Error-budget burn

```
burn = observed bad fraction / allowed bad fraction
```

Use the same eligible-event denominator and a positive allowance.

Derivation and worked example →

Cache service-time average

```
E[T] = h × T_hit + (1 − h) × T_miss
```

A simple service-time mixture, excluding queue effects, dependence, and tail inference.

Derivation and worked example →

Backlog growth

```
growth/s = arrival/s − completion/s
```

Sustained positive growth requires rejection, reduction, or more useful capacity; a bigger queue only changes waiting.

Derivation and worked example →

Raw daily storage

```
bytes/day = records/s × bytes/record × 86400
```

Add indexes, history, metadata, replication, retention, compression and migration separately.

Derivation and worked example →

THE PATTERN INDEX

Start with the problem, then the pattern

Individual LLD patterns include their intent, collaborators, miniature implementation, costs, and alternatives.

6.1 · Dependencies, inversion & composition

Dependency Injection | Composition Root | Inversion of Control

6.4 · Creational patterns

Factory Method | Abstract Factory | Builder | Prototype | Singleton

6.5 · Structural patterns

Adapter | Bridge | Composite | Decorator | Facade | Flyweight | Proxy

6.6 · Behavioral patterns

Strategy | Observer | Command | State | Chain of Responsibility | Template Method | Iterator | Mediator | Memento | Visitor

6.7 · Enterprise patterns & boundaries

Repository | Unit of Work | Specification | CQRS

Application Architecture and Domain-Driven Design

Modular Monolith | Layered Architecture | Hexagonal Architecture | Clean Architecture | Anti-corruption Layer

Caching and Content Delivery

Cache-aside | Read-through | Write-through | Write-behind | Stale-while-revalidate

Message Queues, Events, and Distributed Workflows

Transactional Outbox | Saga | Event Sourcing | Orchestration | Choreography

Reliability, Fault Tolerance, and Disaster Recovery

Circuit Breaker | Bulkhead | Retry with Jitter | Load Shedding

Deployment, Cloud Infrastructure, and Platform Architecture

Rolling Deployment | Blue-green Deployment | Canary Release

THE READING ROOM

Source bibliography

Original research and specifications are distinguished from university teaching, implementation documentation, and engineering guidance.

Access and the stated use were reviewed on 21 October 2026. Versioned documentation is pinned where practical. A checked link does not prove a production system correct; worked workloads, costs, and architectures are original illustrative models. Recheck security guidance and implementation versions when applying them.

61 of 61 sources

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

Atomic Commit in SQLite ↗

Commit and operating-system assumptions

Access reviewed · 21 October 2026

ORIGINAL STANDARD

RFC 9110: HTTP Semantics, June 2022 ↗

HTTP semantics; application commit policy is separate

Access reviewed · 21 October 2026

ORIGINAL RESEARCH · ACM DOI

On the criteria to be used in decomposing systems into modules — D. L. Parnas, 1972 ↗

Module decomposition criteria; metadata and abstract verified

Access reviewed · 21 October 2026

ORIGINAL ENGINEERING ARTICLE

Inversion of Control Containers and the Dependency Injection pattern — Martin Fowler, 2004 ↗

Dependency injection and wiring

Access reviewed · 21 October 2026

ORIGINAL PRACTICE ARTICLE

Documenting Architecture Decisions — Michael Nygard, 2011 ↗

ADR structure and decision history

Access reviewed · 21 October 2026

OFFICIAL GUIDANCE · ACCESSED 21 OCT 2026

AWS Well-Architected: the six pillars ↗

Review lenses, not correctness guarantees

Access reviewed · 21 October 2026

ORIGINAL MODEL DOCUMENTATION

C4: diagrams ↗

Formal diagram levels and supporting views

Access reviewed · 21 October 2026

ORIGINAL MODEL DOCUMENTATION

C4: container abstraction ↗

Meaning of container

Access reviewed · 21 October 2026

ENGINEERING BOOK

Implementing SLOs — The SRE Workbook ↗

Service indicators, objectives, and error budgets; not a source of the atlas workload numbers

Access reviewed · 21 October 2026

ORIGINAL RESEARCH · PUBLISHER RECORD

A Proof for the Queuing Formula: L = lambda W — John D. C. Little, 1961 ↗

Long-run queue accounting; publisher record inspected, full proof access may depend on access rights

Access reviewed · 21 October 2026

ORIGINAL RESEARCH

The Tail at Scale — Dean and Barroso, 2013 ↗

Tail-latency considerations; atlas probabilities and latency examples are author-calculated

Access reviewed · 21 October 2026

ORIGINAL SPECIFICATION

RFC 9000: QUIC Transport, 2021 ↗

QUIC streams and transport behavior; application walkthrough is author-created

Access reviewed · 21 October 2026

ORIGINAL SPECIFICATION

RFC 8446: TLS 1.3, 2018 ↗

Secure transport handshake and protocol scope

Access reviewed · 21 October 2026

UNIVERSITY TEXTBOOK

Operating Systems: Three Easy Pieces ↗

Runtime, concurrency, and persistence

Access reviewed · 21 October 2026

LANGUAGE SPECIFICATION · ACCESSED 21 OCT 2026

The Go Memory Model ↗

One concrete memory-model contract; other languages differ

Access reviewed · 21 October 2026

ENGINEERING TEACHING REFERENCE

The Catalog of Design Patterns — Refactoring Guru ↗

Pattern taxonomy; miniature examples and judgments are original to this atlas

Access reviewed · 21 October 2026

OFFICIAL GUIDANCE · ACCESSED 21 OCT 2026

Architecture styles — Microsoft Architecture Center ↗

Style comparisons; provider-specific recommendations are not mandatory in the atlas

Access reviewed · 21 October 2026

ORIGINAL ENGINEERING EXPLANATION

Bounded Context — Martin Fowler, 2014 ↗

Model boundaries and language

Access reviewed · 21 October 2026

ORIGINAL ENGINEERING ARTICLE

Microservices — James Lewis and Martin Fowler, 2014 ↗

Architecture characteristics and costs; no exact company architecture is asserted here

Access reviewed · 21 October 2026

PROJECT DOCUMENTATION

OWASP API Security Top 10, 2023 edition ↗

Threat taxonomy for Unit 18

Access reviewed · 21 October 2026

ORIGINAL PROTOCOL SECURITY GUIDANCE

RFC 9700: OAuth 2.0 Security Best Current Practice, 2025 ↗

OAuth threat and flow guidance; implementation should use maintained protocol libraries

Access reviewed · 21 October 2026

UNIVERSITY COURSES

Carnegie Mellon Database Group: courses ↗

Database internals reading

Access reviewed · 21 October 2026

OFFICIAL VERSIONED DOCUMENTATION

PostgreSQL 18: Index Types ↗

Supported access structures and their purposes

Access reviewed · 21 October 2026

OFFICIAL VERSIONED DOCUMENTATION

PostgreSQL 18: Transaction Isolation ↗

Specific PostgreSQL isolation behavior; no all-database equivalence claim

Access reviewed · 21 October 2026

OFFICIAL VERSIONED DOCUMENTATION

PostgreSQL 18: Write-Ahead Logging ↗

Logging and recovery ordering; durability remains configuration-dependent

Access reviewed · 21 October 2026

OFFICIAL IMPLEMENTATION DOCUMENTATION · ACCESSED 21 OCT 2026

RocksDB Overview ↗

An LSM-based engine; exact tuning is implementation-specific

Access reviewed · 21 October 2026

ORIGINAL STANDARD

RFC 9111: HTTP Caching, 2022 ↗

HTTP cache semantics; application cache races are separate examples

Access reviewed · 21 October 2026

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

Redis: Key eviction ↗

Implementation policies can approximate ideal LRU/LFU

Access reviewed · 21 October 2026

OFFICIAL ENGINEERING EXPLANATION

Content delivery networks — Cloudflare Learning Center ↗

CDN role; atlas delivery and traffic numbers are illustrative

Access reviewed · 21 October 2026

OFFICIAL GUIDANCE · ACCESSED 21 OCT 2026

Application best practices — Microsoft Architecture Center ↗

Scaling and application boundary review

Access reviewed · 21 October 2026

ORIGINAL RESEARCH · AUTHOR ORGANIZATION

Dynamo: Amazon’s highly available key-value store, 2007 ↗

Replication, partitioning, and reconciliation ideas; capstones are original designs, not Dynamo deployments

Access reviewed · 21 October 2026

OFFICIAL SPECIFICATION DOCUMENTATION · 1.54.0 AT ACCESS

OpenLineage: core model and documentation ↗

Dataset, job, and run lineage concepts

Access reviewed · 21 October 2026

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

Elasticsearch: Reindex documents ↗

Implementation reindex controls; full generation-switch protocol in the atlas is an original design

Access reviewed · 21 October 2026

OFFICIAL VERSIONED DOCUMENTATION

Apache Kafka 4.1: Design ↗

Partitioning and scoped transactional/processing behavior; external effects are separate

Access reviewed · 21 October 2026

PRIMARY PRACTITIONER RESEARCH REFERENCE

Jepsen: Consistency models ↗

Formal-history distinctions and references to underlying literature

Access reviewed · 21 October 2026

ORIGINAL RESEARCH

Time, Clocks, and the Ordering of Events in a Distributed System — Lamport, 1978 ↗

Ordering reference for Unit 14

Access reviewed · 21 October 2026

ORIGINAL RESEARCH · UNIVERSITY-HOSTED PAPER

Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services — Gilbert and Lynch, 2002 ↗

CAP assumptions and impossibility; PDF text extraction is imperfect, original page inspected

Access reviewed · 21 October 2026

UNIVERSITY COURSE

MIT 6.5840: Distributed Systems, Spring 2026 ↗

Advanced distributed-systems reading

Access reviewed · 21 October 2026

ORIGINAL RESEARCH

In Search of an Understandable Consensus Algorithm — Ongaro & Ousterhout ↗

Raft reference for Unit 15

Access reviewed · 21 October 2026

ORIGINAL RESEARCH

Paxos Made Simple — Leslie Lamport, 2001 ↗

Conceptual consensus safety; atlas does not present toy code as production Paxos

Access reviewed · 21 October 2026

ORIGINAL RESEARCH · 2017 REVISION

A Conflict-Free Replicated JSON Datatype — Kleppmann and Beresford ↗

One formal CRDT model; the miniature tie-break exercise is not this complete implementation

Access reviewed · 21 October 2026

OFFICIAL ENGINEERING GUIDANCE

Saga distributed transactions pattern — Microsoft ↗

Local transactions and compensation; compensation is not always a perfect inverse

Access reviewed · 21 October 2026

PATTERN AUTHOR'S EXPLANATION

Transactional Outbox — Chris Richardson ↗

Publication intent and duplicate delivery

Access reviewed · 21 October 2026

ENGINEERING BOOK

Handling Overload — Google SRE ↗

Admission, capacity, and overload response

Access reviewed · 21 October 2026

OFFICIAL GUIDANCE · ACCESSED 21 OCT 2026

Disaster recovery planning guide — Google Cloud ↗

Recovery planning, distinct from a measured achievement of an objective

Access reviewed · 21 October 2026

OFFICIAL GUIDANCE · 2024 REVISION AT ACCESS

Reliability Pillar — AWS Well-Architected ↗

Reliability review lenses and operations; provider-neutral teaching uses the concepts

Access reviewed · 21 October 2026

OFFICIAL PROJECT GUIDANCE · ACCESSED 21 OCT 2026

OWASP: Authorization ↗

Least privilege, deny by default, and per-resource enforcement

Access reviewed · 21 October 2026

OFFICIAL PROJECT GUIDANCE · ACCESSED 21 OCT 2026

OWASP: Password Storage ↗

Password-specific hashing and current deployment parameter guidance

Access reviewed · 21 October 2026

ORIGINAL SPECIFICATION · DECEMBER 2023

OpenID Connect Core 1.0, errata set 2 ↗

Identity layer and validation requirements; distinct from API authorization

Access reviewed · 21 October 2026

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

OpenTelemetry: Signals ↗

Complementary observability signals

Access reviewed · 21 October 2026

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

OpenTelemetry: Context propagation ↗

Trace context relationships; trace identity does not authorize access

Access reviewed · 21 October 2026

ENGINEERING BOOK

Alerting on SLOs — The SRE Workbook ↗

Budget burn and actionable alerts

Access reviewed · 21 October 2026

ORIGINAL METHODOLOGY

The Twelve-Factor App ↗

Application deployment conventions

Access reviewed · 21 October 2026

OFFICIAL DOCUMENTATION · ACCESSED 21 OCT 2026

Kubernetes components ↗

Platform reference for Unit 20

Access reviewed · 21 October 2026

ENGINEERING BOOK

Canarying Releases — The SRE Workbook ↗

Representative rollout evaluation and stop conditions

Access reviewed · 21 October 2026

ORIGINAL ENGINEERING ARTICLE

The Practical Test Pyramid — Ham Vocke ↗

Test scope and feedback trade-offs, not a mandatory test-count ratio

Access reviewed · 21 October 2026

ORIGINAL PROTOCOL DESCRIPTION · INFORMATIONAL RFC

RFC 2697: A Single Rate Three Color Marker, 1999 ↗

Token-based traffic metering example; atlas admission bucket is a simpler model

Access reviewed · 21 October 2026

ORIGINAL STANDARD

RFC 6455: The WebSocket Protocol, 2011 ↗

Bidirectional transport framing; not durable chat delivery

Access reviewed · 21 October 2026

ORIGINAL PROTOCOL DESCRIPTION · INFORMATIONAL RFC

RFC 8216: HTTP Live Streaming, 2017 ↗

One adaptive streaming protocol; do not confuse Informational RFC with Standards Track

Access reviewed · 21 October 2026

OFFICIAL IMPLEMENTATION DOCUMENTATION · ACCESSED 21 OCT 2026

PostGIS: ST_DWithin ↗

Coordinate-system-aware proximity predicate, not exclusive dispatch assignment

Access reviewed · 21 October 2026

ENGINEERING BOOK

Distributed Periodic Scheduling — Google SRE ↗

Durable scheduling concerns; atlas schedule architecture is an original worked design

Access reviewed · 21 October 2026

No sources match that search.

PRODUCTION AND EVIDENCE

Completion manifest

All units are produced. Coverage, technical review, executable tests, and browser verification are separate kinds of evidence.

COMPLETE FIRST EDITION

22 units · 99 chapters · 390 exact syllabus items · 12 full case studies · 242 questions with separate explained solutions.

Unit	Chapters produced	Concept / case coverage	Questions / answers
1 · Introduction to System Design	4 / 4	13	11 / 11
2 · Requirements, Estimation, and Engineering Trade-offs	4 / 4	15	11 / 11
3 · Networking Foundations for System Design	4 / 4	17	11 / 11
4 · Runtime, Processes, Threads, and Concurrency	4 / 4	17	11 / 11
5 · Object-Oriented and Modular Software Design	4 / 4	15	11 / 11
6 · Design Patterns, Dependency Injection, and Object Lifecycles	7 / 7	18	11 / 11
7 · Application Architecture and Domain-Driven Design	4 / 4	20	11 / 11
8 · API Design and Communication Contracts	4 / 4	19	11 / 11

Unit	Chapters produced	Concept / case coverage	Questions / answers
9 · Data Models and Storage Technologies	4 / 4	19	11 / 11
10 · Database Internals, Transactions, and Concurrency Control	4 / 4	18	11 / 11
11 · Caching and Content Delivery	4 / 4	17	11 / 11
12 · Scalability, Load Balancing, and Data Distribution	4 / 4	22	11 / 11
13 · File Systems, Search Infrastructure, and Data Pipelines	4 / 4	17	11 / 11
14 · Distributed Systems Fundamentals	4 / 4	20	11 / 11
15 · Replication, Leader Election, and Consensus	4 / 4	17	11 / 11
16 · Message Queues, Events, and Distributed Workflows	4 / 4	22	11 / 11
17 · Reliability, Fault Tolerance, and Disaster Recovery	4 / 4	22	11 / 11
18 · Security Architecture and Privacy	4 / 4	22	11 / 11
19 · Observability, Monitoring, and Incident Engineering	4 / 4	19	11 / 11
20 · Deployment, Cloud Infrastructure, and Platform Architecture	4 / 4	21	11 / 11
21 · Performance, Cost Optimization, and System Validation	4 / 4	20	11 / 11

Unit	Chapters produced	Concept / case coverage	Questions / answers
22 · End-to-End Design, Architecture Reviews, and Evolution	12 / 12	12 × 22 sections	11 / 11

What the evidence means

Content checks establish presence, exact syllabus allocation, figures, references, and assessment. Executable examples test bounded arithmetic and local invariants. Browser checks test reading, interactions, layouts, offline behavior, and print assembly. The book does not report illustrative objectives as measured production achievements or toy protocols as reviewed production consensus, security, or financial integrations.

The original request is retained in BRIEF.txt. Additional optional case subjects are suggestions beyond the twelve mandatory worked designs, not omitted required chapters.